EECS 4415 · YORK UNIVERSITY

Big Data Systems Master Study Guide

Lectures 1 – 5, from first principles to exam-ready.
Every slide explained. Every quiz solved step by step.

L1 L2 L3 L4 L5

Lecture 1	Introduction to Big Data, data science, Bonferroni's principle
Lecture 2	Distributed file systems, Map-Reduce, combiners, joins, cost model
Lecture 3	Spark: RDDs, transformations & actions, DataFrames, ecosystem
Lecture 4	Frequent itemsets, association rules, A-Priori, PCY, Multistage, Multihash, Sampling, SON
Lecture 5	Clustering: hierarchical, k-means, CURE, BFR (theory), k-means on Map-Reduce

Study loop: slides of one lecture → matching chapter of this guide → back to the slides → cover-and-solve the quizzes → theory Q&A bank.
Every section carries a slide badge such as **Lecture 4 · Slides 8–9** so you can jump between this guide and the slides.

Table of contents

Lecture 1 — Introduction to Big Data, Data Science & Bonferroni's Principle 7

Slide map 7
The big-data era: why everybody cares 8
What is data analytics? 9
Meaningfulness of analytics: Bonferroni's principle 12
What matters when dealing with data? Challenges, modalities, operators 19
Data science and its cultures 20
What will we learn? Data types, computation models, problems, tools 21
About the course: logistics, grading, prerequisites 24
Theory Q&A bank 25
One-page cheat sheet 28
Common mistakes & traps 29

Lecture 2 — Distributed Computing: Distributed File Systems & Map-Reduce 30

Slide map 31
Why one computer is not enough 32
Motivating example: reading the web 33
Cluster architecture 38
Cluster computing challenges 39
The three ideas of Map-Reduce 47
The distributed file system 47
Closing the storage story: issue, idea, solution 53
The warm-up task: counting words 54
Map-Reduce overview: four stages 54
The data model: Map step and Reduce step 55
Word count, step by step 56
The word-count pseudo-code 58
The programming model, more specifically 66
Example: host size 67
Example: language model (5-word sequences) 75
Other uses of Map-Reduce 82
The Map-Reduce picture, and how it runs in parallel 82
What the environment takes care of 83
Slide 54: the programmer's view of the workflow 84
Where the data lives: data flow 85
Coordination: the master 85
Dealing with failures 86
Fine-granularity tasks and pipelining 87
Refinement: combiners 89
Refinement: the partition function 100
Refinement: backup tasks (stragglers) 102
How many map and reduce tasks? 103
Problems suited for Map-Reduce 103
Example: join by Map-Reduce 104
Cost measures for Map-Reduce algorithms 105
Implementations and cloud computing 113
Readings, resources and actions 114
Theory Q&A bank 115
One-page cheat sheet 119
Common mistakes & traps 120

Lecture 3 — Spark: Beyond Map-Reduce (RDDs, DataFrames, the Ecosystem) 122

Slide map 122
Where this lecture starts: Spark extends Map-Reduce 123
Map-Reduce recap: why we scale out 124
The MapReduce execution overview 125
Problems with MapReduce 126
Data-flow systems 128
The Spark software stack 129
Data lakes: Spark connects to everything 129
Spark: the most popular data-flow system 130
The key construct: the RDD 131
Higher-level APIs: DataFrames and Datasets 133
Part 1 in one table: problem and fix 134
The RDD: Spark's basic data container 134
RDD operations: transformations, actions, lazy evaluation 137
The task scheduler: general DAGs, stages and shuffles 139
Higher-level APIs: DataFrame and Dataset 141
Spark's libraries 142
Spark vs Hadoop MapReduce 142
Which problems suit MapReduce? 143
Decision checklist: MapReduce or Spark? 144
Tutorial and further reading 144
Theory Q&A bank 145
One-page cheat sheet 147
Common mistakes & traps 148

Lecture 4 — Frequent Itemsets & Association Rules (A-Priori, PCY, SON) 150

Slide map 150
Where this lecture starts: finding what goes together 152
The market-basket model 152
Where the model is used 154
Roadmap of the lecture 154
Support and frequent itemsets 155
Association rules and confidence 163
Interesting rules and the interest measure 164
The mining problem and the two-step recipe 170
Worked example: generating the rules 172
How frequent itemsets are found at scale: the computation model 180
Why frequent pairs are the hard part 182
Counting pairs in memory: the naive algorithm 183
The A-Priori algorithm 192
The PCY (Park-Chen-Yu) algorithm 210
Refinements of PCY: Multistage and Multihash 221
Fewer passes: the limited-pass idea 235
Random sampling 236
The SON algorithm 238
SON on a cluster: two MapReduce rounds 240
Sampling versus SON 252
Putting it all together: the cat-and-dog baskets 253
More practice slides: A-Priori, PCY bitmap, Multistage 260
Chapter summary and what to read next 262
Theory Q&A bank 264
One-page cheat sheet 268
Common mistakes & traps 269

Lecture 5 — Clustering (Hierarchical, k-means, CURE, BFR, k-means on Map-Reduce) 270

Slide map....270
Where clustering fits, and what the lecture covers....272
The problem of clustering....272
Pictures of clusters, outliers and difficulty....273
Why clustering is hard: high dimensions....275
Application: clustering documents to find topics....275
Overview: two families of clustering methods....276
Hierarchical (agglomerative) clustering....277
The non-Euclidean case: clustroid....279
Why hierarchical clustering does not scale....288
Worked example: hierarchical clustering of 12 points (k = 6)....288
Next: from a tree of merges to k-means....297
k-means: point-assignment clustering....297
Choosing k....307
The CURE algorithm: clusters of any shape, large data....310
BFR: k-means for data that does not fit in memory....321
The three sets of points: DS, CS and RS....323
What is stored: N, SUM and SUMSQ....325
Processing one memory load....326
Q1: How close is "close enough"? The Mahalanobis distance....328
Q2: Should two compressed sets be combined?....330
BFR in one page....332
Three scalable ways to cluster: k-means, CURE and BFR....345
k-means on MapReduce....346
Chapter summary and what to read next....366
Theory Q&A bank....367
One-page cheat sheet....372
Common mistakes & traps....373

READ ME FIRST

How to use this guide

The study loop (do this for each lecture)



Suggested loop for each of Lectures 1–5.

- **Every section in this guide starts with a slide badge** (for example Lecture 4 · Slides 8–9). It tells you exactly which slides that section explains. When you finish a section, flip to those slides and see how they read now.
- Each chapter opens with a **Roadmap** (what you must be able to do) and a **Slide map** (slide → page of this guide). It ends with a **theory Q&A bank**, a **one-page cheat sheet** and a list of **common mistakes**.
- The slides repeatedly say "*Review all the examples (treat them as quiz questions)*". So **every quiz and every worked example in Lectures 1–5 is solved in full** in this guide, as a teal box labelled **Quiz L*n*-Q*k***. A complete list is in the Quiz index (p. 374).
- Coloured boxes: **Definition** (blue), **Key idea** (amber), **Beginner intuition** (teal), **Exam tip** (green), **Common mistake** (red), **Formula** (purple), **Not examined** (grey, dashed).

Slide numbers vs PDF page numbers

All references in this guide use the **printed slide number** (bottom-right corner of each slide). For Lectures 1–3 this equals the page number in your PDF viewer. For Lectures 4 and 5 a few slides are hidden in the PDF, so the viewer page is slightly lower than the printed number:

Lecture	Printed slide → PDF page
1, 2, 3	identical (slide 12 = page 12)
4	slides 1–47: identical. Slides 49–86: PDF page = printed number − 1 (printed slide 48 is not in the PDF).
5	offset grows from 0 to 13 (e.g., printed 17 = PDF p. 11, printed 64 = PDF p. 52, printed 84 = PDF p. 72). The slide map at the start of Lecture 5 lists every pair.

Scope of this guide, as you specified

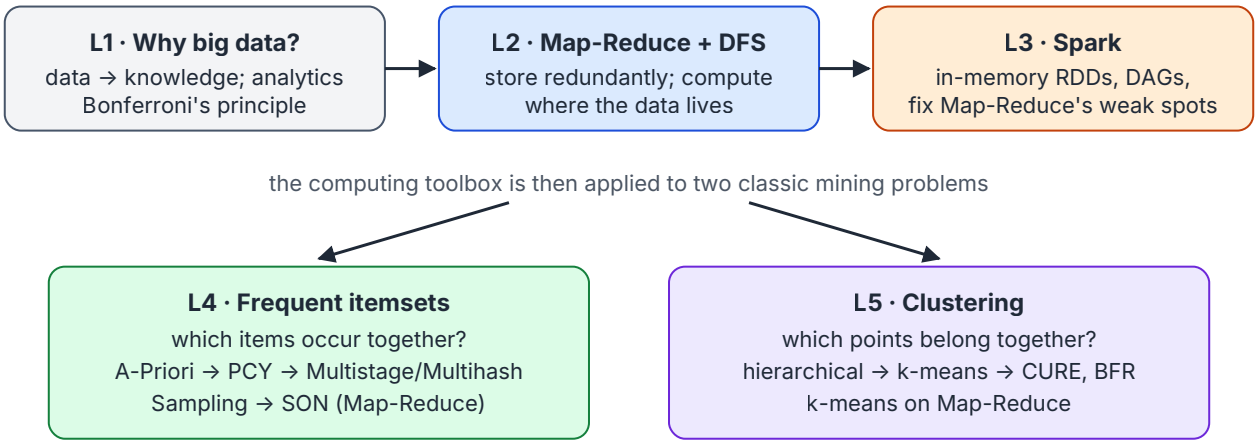
Rule	How this guide handles it
No code will be asked (Lectures 1–5)	Slide pseudo-code (word count, Spark one-liners) is explained *in words*. Where code is shown at all, it sits in a grey dashed box "Not examined — code". Focus on what each step *does* and why.
BFR (Lecture 5): theory only, no numericals	BFR is taught conceptually and in depth: purpose, assumptions, DS / CS / RS, what is stored and why, the processing loop, the Mahalanobis test, merging compressed sets, the last round. The single BFR numeric quiz on the slides is solved anyway (so no quiz is missing) but flagged as **Not examined**.

Partial marks	Every solved quiz lists numbered steps (formula → substitution → result), a boxed final answer and a "Where the marks come from" line. Imitate this layout in the exam.
Everything, beginner to advanced	Each concept: plain-English intuition → precise definition → step-by-step mechanism → tiny numeric example → exam angle → common mistakes.
Accuracy	Every number in the quiz solutions was recomputed (Python) and checked by an independent second solver. Where a slide has a typo or omission, a red **Slide erratum** box says so.

How to write answers that earn partial marks

1. **Write the formula or rule first** (e.g. confidence = support(I ∪ {j}) ÷ support(I)). Even if the arithmetic goes wrong, you keep the method mark.
2. **Substitute the numbers visibly** (e.g. 2 ÷ 4) — do not jump to the answer.
3. **Show intermediate tables** (counts per item, counts per pair, mapper output, reducer output, distances to centroids). Markers award marks per correct intermediate result.
4. **State the final answer clearly**, with units and a one-line interpretation ("not interesting because interest < 0.5").
5. **For "describe / explain" questions**: use short numbered steps that mirror the algorithm's steps on the slides, then add one sentence on *why* (e.g. "monotonicity: if a set is frequent, all its subsets are frequent"). Keywords from the slides earn marks.
6. **If you get stuck**, state the assumption you will make and continue — follow-through marks are common.

The big picture of Lectures 1–5



How the five lectures fit together. Recurring themes: **scale** (data too big for one machine), **memory** (the real bottleneck), and **number of passes** over the data.

THREE QUESTIONS TO ASK ABOUT EVERY ALGORITHM IN THIS COURSE

1. **What is the bottleneck?** (network, disk passes, main memory, CPU)
2. **What is the trick that removes it?** (move computation to data, combiner, monotonicity, hash-bitmap, summary statistics, representatives ...)
3. **What does the trick cost?** (extra pass, false positives, assumption about the data)

LECTURE 1

Introduction to Big Data, Data Science & Bonferroni's Principle

ROADMAP — WHAT THIS LECTURE IS ABOUT

- Why data is exploding and why it is valuable (slides 1–9), and where this course sits: **store → manage → analyse**; this class is about **analysis**.
- What **data analytics** means: finding patterns that are **Valid, Useful, Unexpected, Understandable**; descriptive versus predictive tasks (slides 10–11).
- The most calculation-heavy examinable idea of the lecture: **Bonferroni's principle** and the **hotel example** (about 250,000 innocent "suspicious" pairs) (slides 12–14).
- The framework of challenges, data modalities and data operators (slide 15); data science and its cultures (slides 16–17).
- The course map: data types, models of computation, problems, tools (slides 18–21); logistics and grading (slides 22–29, not examined in depth).

Exam checklist — by the end you can:

- define data analytics and name and explain its four properties (V-U-U-U);
- contrast descriptive and predictive methods with the slide examples (clustering, recommender systems);
- state Bonferroni's principle and **derive the hotel answer step by step** (10^{-9} per day, 10^{-18} for two days, 5×10^{17} pairs of people, 5×10^{5} pairs of days, product ≈ 250,000) and explain what it means;
- list the three challenges (quality, streaming, scalability), five modalities and five operators of slide 15;
- say what data science overlaps with and what this class stresses (scalability, algorithms, computing architectures, automation);
- list the types of data and models of computation the course covers and place the later lectures on the course map.

Slide map

Lecture 1 · Slides 1–29

For Lecture 1 the printed slide number equals the PDF page number. Slides 1, 5 and 22 carry no printed number but are pages 1, 5 and 22.

Slide(s)	PDF page(s)	Topic	In this guide
1	1	Cover: course title, instructor, textbook (MMD)	(p. 8)
2–3	2–3	The era of big data: data explosion and demand	(p. 8)
4	4	"Data contains value"	(p. 9)
5	5	Section divider: what is data analytics?	(p. 9)
6–7	6–7	Facebook "country" and users-by-country map	(p. 9)
8	8	Store, manage, analyse; big data analytics ≈ data science	(p. 10)
9	9	Good news: demand for big data skills	(p. 11)
10	10	What is data analytics: Valid, Useful, Unexpected, Understandable	(p. 11)
11	11	Tasks: descriptive versus predictive	(p. 12)

12	12	Meaningfulness of analytics: Bonferroni's principle	(p. 12)
13–14	13–14	Hotel example and answer (250,000 pairs); Quiz L1-Q1	(p. 13)
15	15	What matters with data: challenges, modalities, operators	(p. 19)
16	16	Data science: cultures (databases, ML, CS theory)	(p. 20)
17	17	This class: what it stresses	(p. 21)
18	18	What will we learn: types of data, models of computation	(p. 21)
19	19	What will we learn (cont.): problems and tools	(p. 22)
20	20	How it all fits together (course map)	(p. 23)
21	21	"How do you want that data?" cartoon	(p. 24)
22–25	22–25	About the course: staff, logistics, communication	(p. 24)
26	26	Work for the course: grading split	(p. 24)
27–29	27–29	Equipment, prerequisites, actions (read MMD Chapter 1)	(p. 24)
1–29	1–29	Theory Q&A bank, cheat sheet, common mistakes	(p. 25) · (p. 28) · (p. 29)

The big-data era: why everybody cares

Lecture 1 · Slides 1–4

Slide 1 is the cover page. It names the course (EECS 4415 Big Data Systems: Course Introduction), the instructor (Jarek Szlichta) and one important acknowledgement: the course is built on the textbook **Mining of Massive Datasets** (MMD) by Jure Leskovec, Anand Rajaraman and Jeff Ullman. When a topic in these notes feels thin, MMD is the place to read more. The lab web address on the slide is a link, not exam material.

Data is exploding

Lecture 1 · Slides 2–3

> **BEGINNER INTUITION**
> Every tap, scroll, like, purchase and GPS ping leaves a small record behind. One record is tiny. Billions of people leaving records all day, every day, adds up to more data than any single computer, or any single person, could ever look at. That pile of records is what people call **big data**.

Slide 2 (a picture of a data-centre aisle). Rows of server racks, blue network cables and a technician inspecting a rack: this is where big data physically lives. The caption gives one scale statistic: **15 out of 17 sectors in the United States have more data stored per company than the US Library of Congress**. The Library of Congress is used as a yardstick for "a very large amount of data". The slide gives no year or source.

Beyond the slides: the statistic comes from a 2011 McKinsey Global Institute report on big data. Treat that as background only; the exam point is the message, "almost every industry holds huge amounts of data".

Slide 3 (Big Data-Driven Era) makes two separate points:

1. **Supply: a massive data explosion.** Social networks produce statuses, shared content and **click streams** (the continuous record of every click a user makes). The slide's example is "Platform X", which receives about **500 million posts per day**. Check: 500,000,000 ÷ 86,400 seconds ≈ 5,787 posts every second, around the clock.
2. **Demand: everyone is interested.** Business organizations, governments and hospitals all want to use data, and the trade press and popular press shout "Big Data!".

Lots of data plus lots of interest is the reason this course exists: we need *methods* and *systems* that can cope.

SLIDE ERRATUM

Slide 3 says "500 millions of posts daily". Correct English is "500 million posts daily". Also, "Platform X" is the social network formerly called Twitter (beyond the slides); the slide gives no source or year for the figure.

"Data contains value"

Lecture 1 · Slide 4

Slide 4 shows the cover of a *Popular Science* special issue titled **"Data is power: how information is driving the future"**, with the caption **"Data contains value"**. The cover headlines list what data is used for: feeding the world, solving cold cases, battling malware, predicting our fate, and even asking whether a crime can be prevented before it begins ("Officer Algorithm"). The headlines are only a visual list; the slide does not discuss them.

KEY IDEA

The **value** is not in the raw bytes. It is in what you can learn from the data and decide using it. Raw data that nobody analyses is just a cost (storage, electricity). That thought leads straight to the next section.

What is data analytics?

Lecture 1 · Slides 5–11

One-sentence definition

Lecture 1 · Slide 5

Slide 5 is a section divider (it carries no printed number). Its subtitle is the shortest definition you can memorise:

DEFINITION — DATA ANALYTICS

Discovering knowledge and actionable insights from data. Two ingredients: *knowledge* (you understand something new) and *actionable* (you can make a decision or take an action because of it).

How big is "big"? The Facebook "country" and the user map

Lecture 1 · Slides 6–7

Slide 6 ("Facebook Country..") ranks "countries" by population, treating Facebook's user base as one of them:

Rank	"Country"	People (billions)	Ratio to Facebook
1	Facebook (users)	3	1.00
2	India	1.47	Facebook is 3 ÷ 1.47 ≈ 2.04 times larger
3	China	1.41	Facebook is 3 ÷ 1.41 ≈ 2.13 times larger

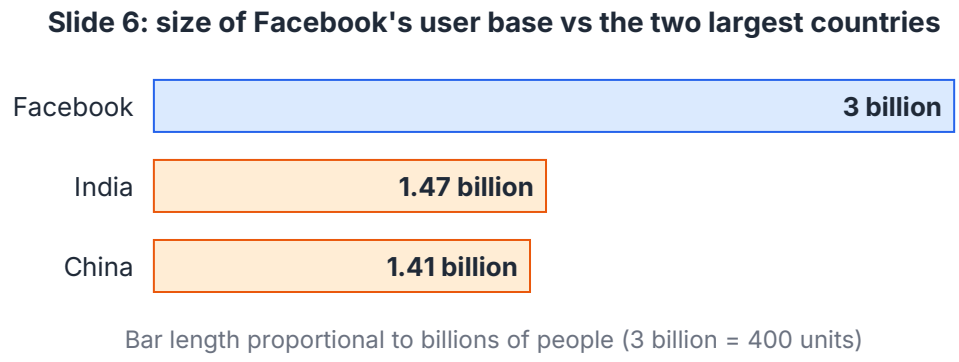


Figure 1. Slide 6 redrawn: a single company's user base is about twice the population of either of the largest real countries.

The point: a single online platform counts users in numbers larger than any country's population (users are accounts, not necessarily distinct people). The slide gives no year; the numbers are rounded.

Slide 7 ("Users by Country") is a world map with a number printed on every country and a colour legend running from **4.8K** (orange) to **448.1M** (blue), plus a grey swatch "Unavailable". The slide does not name the platform, date or unit; it follows slide 6, so these are presumably Facebook users (an inference). Read the map this way:

- **Colour = order of magnitude.** Orange means thousands to tens of millions; blue means hundreds of millions.
- **Very uneven.** India is solid blue at the top of the scale (448.1M). The USA (258.4M) is lilac, Indonesia (188M) and Brazil (165M) are mauve, Mexico is 102.5M, and most other countries are orange.
- **Gaps in the data.** China shows only 3.9M although its population is 1.41 billion (slide 6), and a few countries are grey "Unavailable". Real data is incomplete and uneven. This is the *quality* challenge you meet again on slide 15. Beyond the slides: the slide does not explain China's low count; Facebook is blocked in mainland China, so the map reflects platform reach rather than population.

Country (as read from the map)	Users printed on slide 7
India	448.1M (maximum of the scale)
USA	258.4M
Indonesia	188M
Brazil	165M
Mexico	102.5M
China	3.9M (very low versus its population)

SLIDE ERRATUM

The footer of slide 7 reads "EECS 6415 Big Data Systems"; it should be 4415. The country names above are identified from the map positions; the slide itself prints only numbers.

NOT EXAMINED — SLIDES 2–7 STATISTICS

The figures on slides 2, 3, 6 and 7 (15 of 17 sectors, 500 million posts, 3 billion users, the country counts) are motivation. You will not be asked to recall them, but you should be able to say what they illustrate: huge scale, global reach, uneven and incomplete data.

Store, manage, analyse: where this class sits

Lecture 1 · Slide 8

Slide 8 says that to extract knowledge, data needs to be **stored**, **managed** and **analyzed**, and points at the last one: **"ANALYZED ← this class"**. The centred statement is **Big Data Analytics ≈ Data Science**.



Figure 2. The three stages on slide 8. Database courses focus on storing and managing; EECS 4415 focuses on analysing at big-data scale.

- **Stored:** the data must be kept somewhere (disks, a distributed file system).
- **Managed:** organised so it can be found, kept consistent and retrieved (databases).
- **Analyzed:** turned into patterns and models. This is the focus of the course.
- **"≈"** means "roughly equivalent": in this course the terms *big data analytics* and *data science* are used as near-synonyms. The slide does not define "data science" further.

Good news: demand for big data skills

Lecture 1 · Slide 9

This is the "why take this course" slide. It makes three claims:

- The **data science market** is expected to grow from **$220 billion in 2026** to **$975 billion in 2034**. Check: 975 ÷ 220 ≈ 4.43 times in 8 years, which is a compound growth of about 20.5% per year.
- **80%** of firms across the globe are investing in building a skilled data analytics division.
- Salaries in data analytics and AI are surging; a data scientist earns about **$190K**.

None of these figures has a source or a definition of units on the slide. They are motivation, not exam content.

What is data analytics? The four properties (V-U-U-U)

Lecture 1 · Slide 10

BEGINNER INTUITION

Imagine you ask a very fast but very naive assistant to "find something interesting" in a spreadsheet of a million sales. It will hand you hundreds of "findings". Most are junk: coincidences, things everybody already knows, or tangled formulas no one can read. The four properties below are the filter that separates a real finding from junk.

DEFINITION — DATA ANALYTICS (SLIDE 10)

Given lots of data, **discover patterns and models** that are **Valid, Useful, Unexpected and Understandable**.

Property	Slide wording	What it means	Example
Valid	hold on new data with some certainty	The pattern generalises to *new* data, not only the data you looked at. "Some certainty" means with high probability, not 100%.	"Customers who buy A also buy B" still holds next month.
Useful	should be possible to act on the item	You can do something with it (the "item" is the discovered pattern).	Put A and B on the same shelf.
Unexpected	non-obvious to the system	The system did not already know it; it tells you something new.	A surprising link between two products.
Understandable	humans should be able to interpret the pattern	A person can read it and see why it makes sense.	A short rule, not 10,000 coefficients.

EXAM TIP

"Name and explain the properties of a good discovered pattern" is a standard short answer. Give the four words and one sentence each (the table above). A pattern failing **Valid** is the Bonferroni problem of the next sections: it was just noise that will not reappear.

COMMON MISTAKE

Writing "novel" or "new" for **Unexpected** is acceptable as a paraphrase, but do not confuse **Valid** (true on *new* data) with "true on the data we have". Almost any pattern is true on the data used to find it.

Data analytics tasks: descriptive vs predictive

Lecture 1 · Slide 11

	Descriptive methods	Predictive methods
Slide definition	Find human-interpretable patterns that describe the data	Use some variables to predict unknown or future values of other variables
Question answered	What is in the data I have?	What will happen / what is not known yet?
Slide example	**Clustering**	**Recommender systems**
Everyday picture	Sorting a pile of photos into piles of similar ones, without being told the piles.	Guessing which film a friend will like from the films they liked before.

- **Clustering** groups similar items together without being told the groups in advance. Lecture 5 covers scalable clustering (grouping similar points; k-means, CURE and BFR are the specific algorithms, each explained there).
- A **recommender system** predicts how much a user will like an item they have not seen yet. Slide 19 lists it as a real-world problem and slide 20 puts it in the "Apps" column.

EXAM TIP

Distinguish by the question: descriptive describes what *is* in the data; predictive estimates something *not yet known*. Always add the slide's example for each.

Meaningfulness of analytics: Bonferroni's principle

Lecture 1 · Slides 12–14

The risk and the principle

Lecture 1 · Slide 12

BEGINNER INTUITION

Look at enough clouds and you will see a face in one of them. The more places you look, the more "faces" you find, and none of them means anything. A data analyst searching a mountain of data for "interesting" patterns is in the same position: if the search is wide enough, pure randomness will produce patterns that look real.

Slide 12 says: "A risk with Data Analytics is that an analyst can 'discover' patterns that are meaningless. Statisticians call it **Bonferroni's principle**:" and then stops. The statement itself is missing.

SLIDE ERRATUM

Slide 12 ends with a colon and an empty slide body: the actual statement of Bonferroni's principle never appears in the slides. Supply it yourself (next box) and quote it in the exam.

DEFINITION — BONFERRONI'S PRINCIPLE (BEYOND THE SLIDES: STANDARD STATEMENT, MMD §1.2.3)

Calculate the **expected number of occurrences** of the events you are looking for, **on the assumption that the data is random**. If this number is **significantly larger** than the number of real instances you hope to find, then you must expect that almost anything you find is **bogus**, that is, a statistical artifact rather than evidence of what you are looking for.

Informal version: if you look in more places for interesting patterns than your amount of data can support, you are bound to find noise.

THE RECIPE IN THREE STEPS

1. Pretend the data is completely random (nobody is doing anything special).
2. Compute how many "interesting" matches that random world would still give.
3. If that number is much bigger than the number of true cases you expect, the matches are not trustworthy evidence.

The hotel example

Lecture 1 · Slides 13–14

The task (slide 13). Find (unrelated) people who **at least twice have stayed at the same hotel on the same day**. Suppose such meetings are a sign of wrongdoing. The data:

DEFINITION — C(N, 2) AND EXPECTED NUMBER

C(n, 2) = n(n−1)/2 is the number of ways to choose 2 things from n, order ignored. Example: 4 people give 4 × 3 ÷ 2 = 6 pairs; for huge n this is about $n^2/2$. **Expected number** = the average count you would see over many random repetitions; for N chances, each with probability p, the expected number of successes is N × p (this holds even when the chances are not independent). Example: 600 rolls of a die are expected to give 600 × 1/6 = 100 sixes.

The same numbers in four lines (full derivation in Quiz L1-Q1):

1. One day: P(both people are in some hotel) = 0.01 × 0.01 = 10^{-4}; P(same hotel | both in hotels) = $1/10^5$; so P(same hotel on one given day) = 10^{-9}.
2. Two given days: $10^{-9} \times 10^{-9} = 10^{-18}$.
3. Pairs of people ≈ $(10^9)^2/2 = 5 \times 10^{17}$; pairs of days ≈ $(1000)^2/2 = 5 \times 10^5$.
4. Expected = $5 \times 10^{17} \times 5 \times 10^5 \times 10^{-18}$ = 250,000 innocent pairs.

Quantity	Slide value	Consequence
People tracked	10^9	about 5×10^{17} pairs of people
Days	1,000	about 5×10^5 pairs of days
Chance a person is in a hotel on a given day	1% (1 day in 100)	$10^9 \times 1\% = 10^7$ people in hotels each day
Hotel capacity	100 people	$10^7 \div 100 = 10^5$ hotels

The slide asks: **if everyone behaves randomly (no terrorists at all), will the analytics detect anything suspicious?** Common sense says "no: nothing is going on". The surprising answer, on slide 14, is yes.

The result (slide 14). The **expected number of "suspicious" pairs of people is 250,000**. The slide continues: that is too many combinations to check, so we need additional evidence to find suspicious pairs more efficiently.

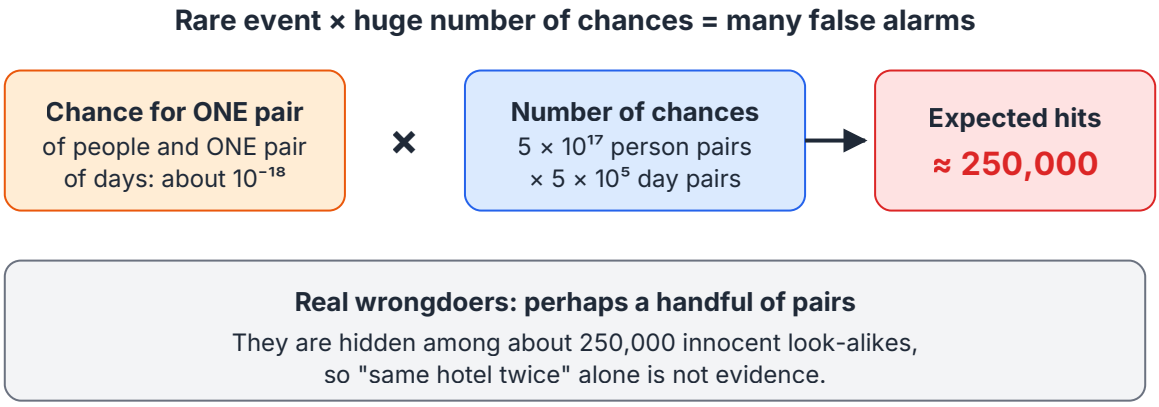


Figure 3. The shape of the hotel argument (derived step by step in Quiz L1-Q1). Numbers are rounded as in the slides.

Why does it happen? Each individual coincidence is fantastically unlikely, but there are a staggering number of pairs of people and pairs of days that could produce one. Multiply a tiny probability by an enormous number of

chances and you get a large expected count. It is the same reason someone wins a lottery most weeks even though your own ticket almost never wins.

Quick check of the setup: $10^9 \times 0.01 \div 100 = 10^5$ hotels (matches slide 13). Exact arithmetic gives about 249,750 expected pairs, which rounds to the slide 14 answer of 250,000. The slide prints only the final answer; the full derivation is walked through in the quiz below.

KEY IDEA — WHAT THE LESSON IS

250,000 innocent pairs would be flagged by pure chance. The suspicious-pair rule is therefore **meaningless on its own** (it fails *Valid* and *Useful* from slide 10). To catch real wrongdoers we need **additional evidence** (more conditions that random behaviour would almost never satisfy), and a more efficient way than examining 250,000 pairs by hand.

EXAM TIP

For a hotel-style question write: (1) probability for one pair of people on one day, (2) on two days, (3) number of pairs of people, (4) number of pairs of days, (5) multiply, (6) interpret ("far more than the handful of real cases, so most are bogus"). Each step carries marks. Remember that "pairs" means $C(n, 2) \approx n^2/2$, not n^2.

COMMON MISTAKE

Do not conclude "the data is fine, the algorithm found suspects". The point is the opposite: when the *random* expectation is already large compared to the true cases, the finding says nothing. Another slip: using n^2 instead of $n^2/2$ for pairs, which doubles the count.

Try it yourself first: cover the solution of Quiz L1-Q1, state the principle, derive 250,000 and say what it means, then check your steps against the solution.

Quiz L1-Q1 Bonferroni's principle: the hotel example Lecture 1 · Slides 12–14

QUESTION (AS ON THE SLIDES)

Slide 12. "A risk with 'Data Analytics' is that an analyst can 'discover' patterns that are meaningless. Statisticians call it **Bonferroni's principle**:" (the slide text stops at this colon; nothing follows it in the PDF).

Slide 13 — Example. We want to find (unrelated) people who **at least twice have stayed at the same hotel on the same day**. Data:

- 10^9 people being tracked;
- 1,000 days;
- each person stays in a hotel 1% of the time (1 day out of 100);
- hotels hold 100 people (so 10^5 hotels).

If everyone behaves randomly (i.e., no terrorists) will the data analytics detect anything suspicious?

Slide 14. "Expected number of 'suspicious' pairs of people: 250,000 ... too many combinations to check – we need to have some additional evidence to find 'suspicious' pairs of people in some more efficient way."

The slides print only the final number 250,000, so the derivation is yours to show. Parts (a)–(c) split the slide question into pieces that can each earn marks:

(a) State Bonferroni's principle.

(b) If everyone behaves randomly, will the analytics detect anything suspicious? Derive the expected number of "suspicious" pairs of people: the probability that two given people are in the same hotel on one given day and on two given days, the number of pairs of people, the number of pairs of days, and the product.

(c) Explain what this number means: why it is meaningless as evidence of wrongdoing, and what lesson we draw.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Think of a lottery. Your own ticket almost never wins, yet somebody wins most weeks because so many tickets are sold. Here a "ticket" is a *pair of people together with a pair of days*. Each ticket has an absurdly small chance (10^{-18}) of showing the coincidence, but there are about 2.5×10^{23} tickets, so a great many "win" by luck alone. Those lucky pairs look suspicious, yet nobody did anything wrong.

Beyond the slides — the birthday paradox has the same shape: among 23 people there are C(23, 2) = 253 pairs, each with only a 1-in-365 chance of sharing a birthday, but the chance that *some* pair matches is about 51%. Bonferroni's principle says: **before you believe a pattern, ask how many such patterns pure randomness would give you.**

PART (A) — DEFINITION: BONFERRONI'S PRINCIPLE

What the slide says (Lecture 1 · Slide 12): "A risk with 'Data Analytics' is that an analyst can 'discover' patterns that are meaningless. Statisticians call it Bonferroni's principle."

Beyond the slides (the standard statement, close to the wording of the textbook *Mining of Massive Datasets*, §1.2.3): *calculate the expected number of occurrences of the events you are looking for, on the assumption that the data is random. If this number is significantly larger than the number of real instances you hope to find, then you must expect almost anything you find to be bogus, i.e. a statistical artifact rather than evidence of what you are looking for.*

In beginner language: if you look in more places for interesting patterns than your amount of data can support, you are bound to "find" patterns that are only noise. (The underlying statistical theorem is called the Bonferroni correction; the informal rule above is Bonferroni's principle.)

SLIDE ERRATUM

Slide 12 ends with "Statisticians call it Bonferroni's principle:" followed by an empty slide body. The statement the colon promises is missing from the PDF (the page text is empty below the colon and the only images are the title banner). The slides therefore define the principle only as "the risk that an analyst 'discovers' meaningless patterns"; use the textbook statement above for a complete definition in an exam. Also, **slide 14 prints the answer 250,000 with no derivation**; steps 1–8 below are the standard textbook derivation and they reproduce the slide's number.

1. **(b) Formula / idea — what are we counting?** Assume everyone behaves randomly. A "suspicious event" is a *combination of four things*: two different people X and Y and two different days d_1 and d_2, such that X and Y were in the same hotel on d_1 *and* in the same hotel on d_2 (the hotel on d_1 may differ from the hotel on d_2). If we cannot predict the exact count we can still predict its **average (expected) value**:

   FORMULA

   E = (number of pairs of people) × (number of pairs of days) × P(one given pair of people is together on both given days)

   Beyond the slides: this is *linearity of expectation* — the expected number of "yes" events is the sum of their probabilities; every event here has the same probability, so we multiply (how many events) × (probability of each).

2. **(b) Random model, and a check of "10^5 hotels".** "Behaves randomly" means: independently on each day, a person is in *some* hotel with probability 1/100 (= 1%), and if so is equally likely to be in any of the hotels. Check of slide 13's "so 10^5 hotels": people in hotels on one day = $10^9 \times \frac{1}{100} = 10^7$; each hotel holds 100 people, so the number of hotels is $10^7 \div 100 = \mathbf{10^5}$ ✓ (not $10^9 \div 100$, which would ignore that only 1% are away on a given day).

3. **(b) P(both people are in *some* hotel on a given day).** Each person independently has probability 1/100, and for independent events we *multiply* probabilities: $\frac{1}{100} \times \frac{1}{100} = 10^{-2} \times 10^{-2} = \mathbf{10^{-4}}$.
4. **(b) P(they are in the *same* hotel on a given day).** Given that both are in hotels, fix X's hotel; Y picks one of the 10^5 equally likely hotels, so the chance Y picks X's hotel is $1/10^5 = 10^{-5}$. Rule: P(A and B) = P(A) × P(B given A), so P(same hotel, one given day) = $10^{-4} \times 10^{-5} = \mathbf{10^{-9}}$ (one chance in a billion).
5. **(b) P(same hotel on *two* given days).** Different days are independent, so multiply the two one-day probabilities: $(10^{-9})^2 = \mathbf{10^{-18}}$. (We need X and Y together on each of the two days, not the *same* hotel on both days.)
6. **(b) Number of pairs of people.** Choose 2 people out of 10^9, order not mattering (the pair {X, Y} is the same as {Y, X}): $C(10^9, 2) = \frac{10^9 \times (10^9 - 1)}{2} \approx \frac{(10^9)^2}{2} = \frac{10^{18}}{2} = \mathbf{5 \times 10^{17}}$. (Check the formula on a tiny case: 4 people give 4 × 3 ÷ 2 = 6 pairs. For huge n, n − 1 ≈ n, so C(n, 2) ≈ n²/2.)
7. **(b) Number of pairs of days.** $C(1000, 2) = \frac{1000 \times 999}{2} = 499{,}500 \approx \frac{1000^2}{2} = \mathbf{5 \times 10^5}$.
8. **(b) Multiply everything.** $E = (5 \times 10^{17}) \times (5 \times 10^5) \times 10^{-18}$. Multiply the numbers: 5 × 5 = 25. Add the exponents: 17 + 5 − 18 = 4. So $E = 25 \times 10^4 = \mathbf{250{,}000}$, exactly the slide's answer. (Python check with the exact C(n, 2) values: $4.99999999 \times 10^{17} \times 499{,}500 \times 10^{-18} = 249{,}749.99975 \approx 249{,}750$; the slide's 250,000 is the rounded n²/2 version, a difference of 0.1%.)
9. **(b) Sanity check from the other direction.** One given pair of people looks suspicious with probability ≈ $499{,}500 \times 10^{-18} \approx 5 \times 10^{-13}$ (about 1 in 2 trillion). Multiplying by the 5×10^{17} pairs of people gives 2.5×10^5 again. (Beyond the slides: an exact binomial calculation that counts each pair of people once, however often they coincide, gives 249,749.83 — the same to within 0.2 out of 250,000.)
10. **(c) Answer to the slide's question.** "Will the data analytics detect anything suspicious if everyone behaves randomly?" **Yes**: about 250,000 pairs of people will look suspicious (involving at most 500,000 different people, since a pair has two members), although nobody in the data is a terrorist. By assumption **every one of these hits is a pure coincidence, a false alarm**.
11. **(c) Why the number is meaningless as evidence.** Apply Bonferroni's principle: compare the expected number of *chance* matches (250,000) with the number of *real* cases we hope to find. Beyond the slides — suppose, for illustration, there were about 10 real pairs of evil-doers. They would be buried among about 250,010 flagged pairs, so only 10 ÷ 250,010 ≈ 0.004% (about 1 flagged pair in 25,000) would be real. Because chance matches vastly outnumber real ones, being flagged tells us almost nothing, and nobody can investigate 250,000 pairs — slide 14's "too many combinations to check".
12. **(c) The lesson (Slide 14).** Do **not** trust a pattern merely because the data shows it. Either (i) show that the expected number of chance occurrences is much smaller than the number found, or (ii) bring **additional evidence** that shrinks the set of candidate pairs before you look ("some more efficient way"), for example examine only pairs of people already linked by other clues, or demand a stronger pattern. The "stronger pattern" table further below shows how this removes the chance matches.

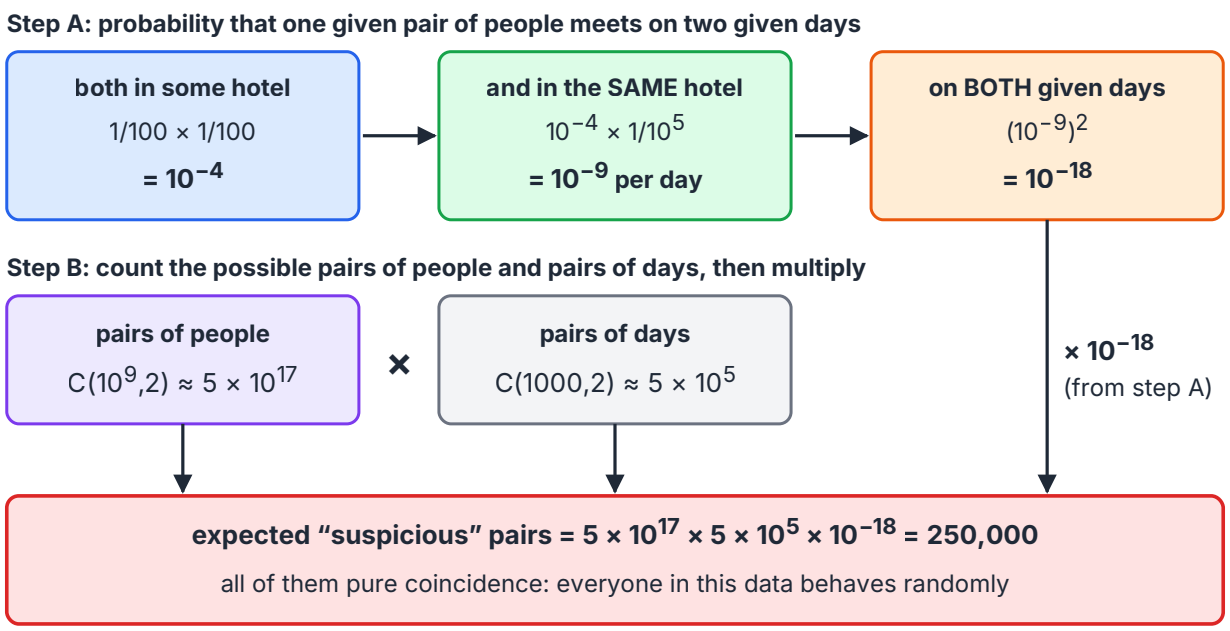


Figure L1-Q1. How the slide's 250,000 is built: the chance of one coincidence is tiny (10^{-18}), but the number of pairs of people and pairs of days to try is astronomically large.

Quantity	Slide-style rounding	Exact
Pairs of people C(10^9, 2)	5×10^{17}	499,999,999,500,000,000
Pairs of days C(1000, 2)	5×10^5	499,500
Expected suspicious pairs	**250,000**	249,749.99975 ≈ 249,750

KEY IDEA

The same formula in symbols: with N people, D days, a fraction p of days spent in a hotel and H hotels, $E = C(N, 2) \times C(D, 2) \times (p^2/H)^2$. Here $p^2/H = 10^{-4}/10^5 = 10^{-9}$, which is why one day gives 10^{-9} and two days give 10^{-18}. The expected count is huge because the **number of candidate combinations** ($\approx 2.5 \times 10^{23}$) is far bigger than 1 ÷ (chance probability) = 10^{18}. Beyond the slides: searching more data finds more noise — double the number of people while keeping the 10^5 hotels and the number of pairs quadruples, so the expected number of chance matches is about 4 times larger (≈ 1,000,000).

Beyond the slides — how a stronger pattern changes the answer. Same data, but now we demand that the two people were together on k given days: $E_k = C(10^9, 2) \times C(1000, k) \times (10^{-9})^k$ (all computed exactly in Python).

Coincidences demanded	Expected chance-only pairs	Reading
k = 1 day	$\approx 5 \times 10^{11}$	hopeless: half a trillion pairs
k = 2 days (the slide)	$\approx 2.5 \times 10^5$ (249,750 exact)	too many to check
k = 3 days	≈ 0.083	**about an 8% chance that even one random pair shows up**
k = 4 days	$\approx 2 \times 10^{-8}$	**essentially never by chance**

This is also why "at least twice" is practically the same as "exactly twice" on the slide: pairs that meet three or more times by luck number only about 0.08 in total, so the 250,000 is not inflated by them.

COMMON MISTAKE

- **Skipping a factor in the one-day probability.** "Both in a hotel" (10^{-4}) is not "same hotel" (10^{-9}). Leaving out the $1/10^5$ makes the answer 10^{10} times too large ($\approx 2.5 \times 10^{15}$); using only 10^{-5} (leaving out the 10^{-4}) gives an absurd 2.5×10^{13}.
- **Using n^2 instead of $n(n-1)/2$** for the pairs (ordered instead of unordered pairs): each of the two counts is doubled, so the result becomes 10^6 instead of 250,000, four times too big. The "÷ 2" appears twice (people and days).
- **Computing the number of hotels as $10^9 \div 100 = 10^7$.** The slide's 10^5 is the number of hotels needed for the 1% of people who are away on a given day.
- **Requiring the same hotel on both days.** The hotels on the two days may differ; only "together on each day" is needed.
- **Stopping at 10^{-18}** and calling the event "practically impossible". You must multiply by the number of opportunities.
- **Reading 250,000 as a probability or as "250,000 guilty pairs found".** It is the *expected number of innocent pairs that look guilty* in data that contains no terrorists.
- **Concluding "data mining is useless".** The lesson is to *calibrate* (compare with the chance level) and to add evidence, not to stop analysing.

EXAM TIP

Write the formula first (pairs of people × pairs of days × probability), then show each probability and each power of ten in order: $10^{-9} \rightarrow 10^{-18} \rightarrow 5 \times 10^{17}$ pairs of people → 5×10^5 pairs of days → 250,000. Use the word "expected", say that all of them are false positives because the behaviour was random, and finish with one sentence: "The expected number of chance matches, 250,000, is far larger than the number of real cases we hope to find, so by Bonferroni's principle almost every match is bogus; we need additional evidence or a smaller candidate set." Even if you slip on one power of ten, the earlier lines keep their marks.

PRACTICE (SELF-MADE, NOT FROM THE COURSE)

- Same setup but only **100 days** are recorded. Pairs of days $\approx 100^2/2 = 5 \times 10^3$, so $E \approx 5 \times 10^{17} \times 5 \times 10^3 \times 10^{-18}$ = **2,500** (exact 2,475).
- Original data but **hotels hold only 10 people** (so $10^7 \div 10 = 10^6$ hotels): P(same hotel, one day) = $10^{-4}/10^6 = 10^{-10}$; two days 10^{-20}; $E \approx 5 \times 10^{17} \times 5 \times 10^5 \times 10^{-20}$ = **2,500** (exact 2,497.5).

Final answer: (a) Bonferroni's principle — an analyst who looks for patterns in a huge amount of data will "discover" meaningless ones; compute the expected number of chance occurrences under random data, and if it is much larger than the number of real cases hoped for, expect nearly everything found to be bogus. (b) P(same hotel, one given day) = $10^{-4} \times 10^{-5} = 10^{-9}$; two given days = 10^{-18}; pairs of people $\approx 5 \times 10^{17}$; pairs of days $\approx 5 \times 10^5$; expected "suspicious" pairs = $5 \times 10^{17} \times 5 \times 10^5 \times 10^{-18} = 250{,}000$ (exact C(n, 2) version 249,750). (c) Even when everyone behaves randomly the analytics still flags about 250,000 pairs; they are chance coincidences, so being flagged is not evidence of wrongdoing and the list is far too long to check — we need additional evidence or a much smaller candidate set.

Where the marks come from: (1) State Bonferroni's principle: expected chance occurrences vs. number of real cases, so most findings are bogus. (2) Identify the event (pair of people + pair of days) and the formula: expected number = count × probability. (3) P(both in a hotel on a day) = 10^{-4}. (4) Divide by 10^5 hotels to get 10^{-9} for one day. (5) Square it for two days: 10^{-18}. (6) Pairs of people $\approx 5 \times 10^{17}$ ($n^2/2$, unordered). (7) Pairs of days $\approx 5 \times 10^5$. (8) Product = 250,000, called an *expected* number. (9) Interpretation: all false alarms because behaviour was random, too many to check, so we need additional evidence or a smaller search space.

Slide check: the final number 250,000 matches the slide (Slide 14; it is the $n^2/2$ approximation, the exact C(n, 2) value is 249,750). Slide erratum: Slide 14 gives no derivation, and Slide 12 is missing the statement of Bonferroni's principle after its colon. Both are supplied above (the definition from the textbook, labelled "Beyond the slides"; the derivation re-computed in Python).

What matters when dealing with data? Challenges, modalities, operators

Lecture 1 · Slide 15

Slide 15 is a conceptual diagram with three arrows meeting at a point, like the x, y and z axes of a 3-D graph. Any big-data problem sits at some point in this 3-D space: you choose one value on each axis.

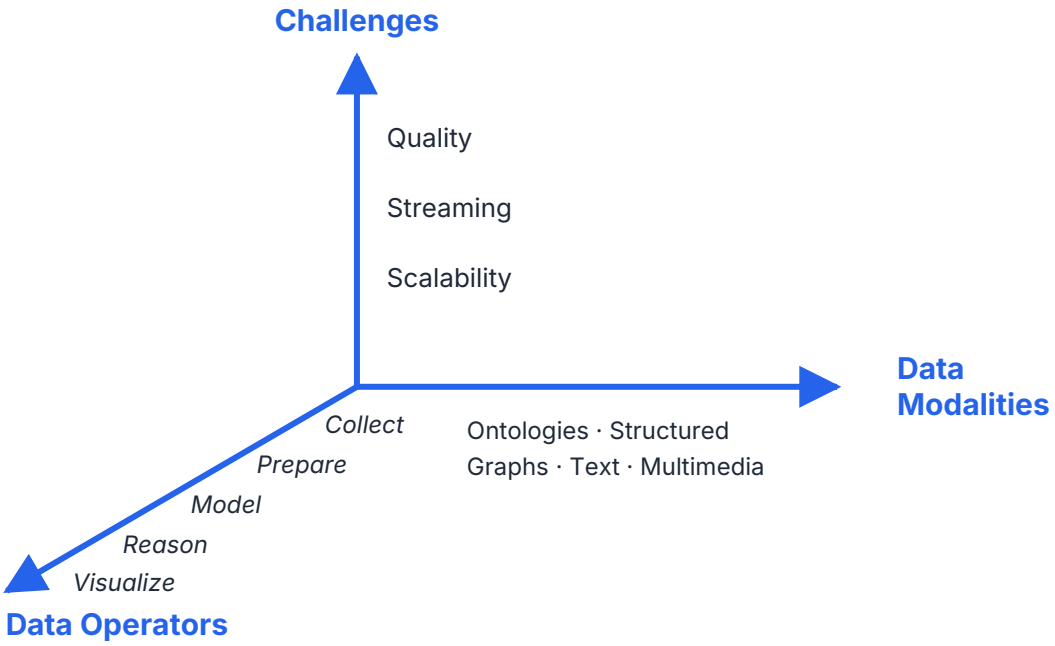


Figure 4. Slide 15 redrawn. Challenges (vertical), data modalities (horizontal), data operators (diagonal, from raw data to insight).

Axis	Values on the slide	Plain meaning
Challenges (what makes it hard)	Quality · Streaming · Scalability	**Quality:** data is dirty, missing or inconsistent (recall China's 3.9M and the "Unavailable" countries on slide 7). **Streaming:** data arrives continuously and must be handled on the fly (see click streams, slide 3). **Scalability:** too big for one machine, so the method must grow with the data.
Data modalities (what form the data takes)	Ontologies · Structured · Graphs · Text · Multimedia	**Ontologies:** formal descriptions of concepts and their relations. **Structured:** tables with fixed columns. **Graphs:** nodes and links (social networks, the web). **Text:** documents. **Multimedia:** images, audio, video.
Data operators (steps from raw data to insight)	Collect · Prepare · Model · Reason · Visualize	Gather the data; clean and reshape it; build a model of it; draw conclusions from the model; show the results to people. (The slide does not explain "Reason"; read it as drawing inferences from the model.)

Beyond the slides: the plain-meaning column is this guide's own explanation; slide 15 prints only the labels.

The slide does not explain the axes or how they interact; it is a framework to remember. The course samples it: the later lectures attack **scalability** (MapReduce and Spark, frameworks for computing on many machines,

Lectures 2–3; SON, a frequent-itemsets algorithm, Lecture 4), **streaming** (infinite data), and several **modalities** (high-dimensional data, graphs).

> **EXAM TIP**
> If asked for "the challenges when dealing with big data", answer with the three on the slide (quality, streaming, scalability) and one sentence each. Mention the other two axes (modalities and operators) to show you understand the framework.

Data science and its cultures

Lecture 1 · Slides 16–17

Where data science comes from

Lecture 1 · Slide 16

Data science is not a single old field. It sits where three older fields overlap, and each contributes one ingredient:

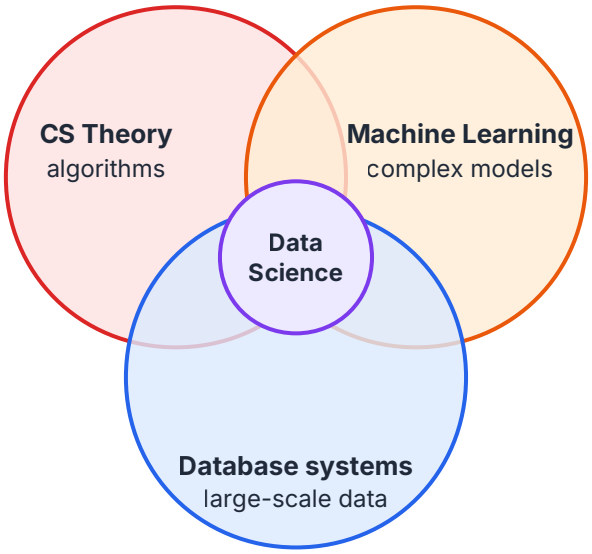


Figure 5. Slide 16: data science at the common intersection of CS theory, machine learning and database systems.

Field	What it contributes (slide 16)
Databases	Large-scale data: storing and querying huge data collections
Machine Learning (ML)	Complex models learned from data
CS Theory	Algorithms: efficient procedures with provable behaviour

Different cultures, same job

Lecture 1 · Slide 16

- **Database person:** data science is an extreme form of **analytic processing**, that is, queries that examine large amounts of data. Beyond the slides: this means scan and aggregate, as opposed to quick look-ups of single records.
- **ML person:** data science is the **inference of models**: fitting a model (rules, parameters, clusters) to the data so that it can predict or describe.
- Slide verdict: **"In this class we will do all!"** You will see the database view (queries and big scans via MapReduce and Spark), the ML view (clustering, models) and the algorithm view (hashing, sampling, streaming algorithms).

SLIDE NOTE

Minor label inconsistency: the bullets say "Databases" but the circle says "Database systems". They mean the same thing.

What this class stresses

Lecture 1 · Slide 17

Slide 17 redraws the Venn diagram with **Statistics** replacing CS Theory (the circles are now Statistics, Machine Learning, Database systems, with Data Science in the middle). The text says the class **overlaps with databases, statistics, machine learning and artificial intelligence (AI)**, but puts **more stress on** four things:

Emphasis	Meaning	Question it asks
Scalability	The method still works when the data grows by a large factor.	Does it still run when data grows 1000 times?
Algorithms	Efficient procedures, not merely correct ones.	How much time and memory does it need?
Computing architectures	The hardware and software setup: clusters, distributed file systems, MapReduce and Spark style frameworks.	Where does the data live and who computes what?
Automation for handling large data	The system, not a person, deals with the size (the slide does not elaborate; think of automatic parallelisation, fault tolerance and data placement).	Who does the heavy lifting?

In short: less proving statistical theory, more making things run on huge data.

SLIDE NOTE

Slide 17's text mentions artificial intelligence but the diagram does not draw it, and the diagram uses Statistics where slide 16 used CS Theory. These are inconsistencies in labelling, not factual errors.

What will we learn? Data types, computation models, problems, tools

Lecture 1 · Slides 18–21

Two axes: types of data and models of computation

Lecture 1 · Slide 18

Types of data (the shape)	What it means
High dimensional	Each item has very many attributes (for example a document described by thousands of word counts). Distances and similarity become tricky. Beyond the slides: a document described by 10,000 word counts is a point in 10,000 dimensions, and in such spaces almost all points look about equally far apart, which is why special tools are needed. LSH and dimensionality reduction appear on the course map but are not part of Lectures 1–5; Lecture 5 covers scalable clustering.
Graph	Items are nodes connected by links (web pages, friends). Ranking and community finding use the link structure.
Infinite / never-ending	A data stream that never stops; you cannot store it all and may see each item only once.
Labeled	Examples come with known answers (for example emails marked spam or not). This is the setting of supervised machine learning.

Models of computation	What it means	Beyond the slides: where it comes in
Single machine, in memory	Everything fits in one computer's RAM; classic algorithms apply.	Classic algorithms; also what each worker runs on its own share of the data
Distributed computing	Data and work are spread over many machines.	Lecture 2 (MapReduce), Lecture 3 (Spark), Lecture 4 (SON), Lecture 5 (clustering with MapReduce)
Streams and online algorithms	Data arrives over time; each item can be seen once or a few times with limited memory.	Infinite data topics (filtering streams, Bloom filters)

KEY IDEA

The right algorithm depends on the pair (type of data, model of computation). Same task, different setting, different method. This slide leaves that implicit.

Real-world problems and tools

Lecture 1 · Slide 19

Slide 19 (also titled "What will we learn?") continues with applications (the "why") and tools (the "how"):

Real-world problem	One-line meaning
Recommender systems	Suggest products or films a user is likely to want.
Link analysis	Use the link structure of the web to rank pages (for example PageRank).
Spam detection	Recognise spam emails or web spam.
Duplicate document detection	Find near-identical documents or pages.

Tool	One-line meaning	Where in this course
MapReduce	A programming model for processing huge data on a cluster by splitting work into map and reduce phases.	Lecture 2
Spark	A faster cluster framework that keeps data in memory between steps.	Lecture 3
Locality Sensitive Hashing (LSH)	A hashing trick so that similar items land in the same bucket, enabling fast similarity search.	High-dimensional data
Bloom filter	A compact, probabilistic set-membership test (may say "yes" wrongly, never "no" wrongly).	Streams
Dynamic programming	Solve a problem by reusing answers to smaller sub-problems.	Named on slide 19 as a tool; also EECS 3101 background. Not a Lectures 1–5 topic in this guide.

Beyond the slides: the one-line meanings in this table and the "Where in this course" column are background from this guide; the slide only lists names. The later lectures give the precise versions. Slide 19 is not exhaustive: clustering, frequent itemsets and BFR (which appear in Lectures 4 and 5) are not in its lists.

The course map: how it all fits together

Lecture 1 · Slide 20

Slide 20 is the table of contents of the course: five columns of three boxes each. Only the columns carry meaning; the rows are not related to each other.

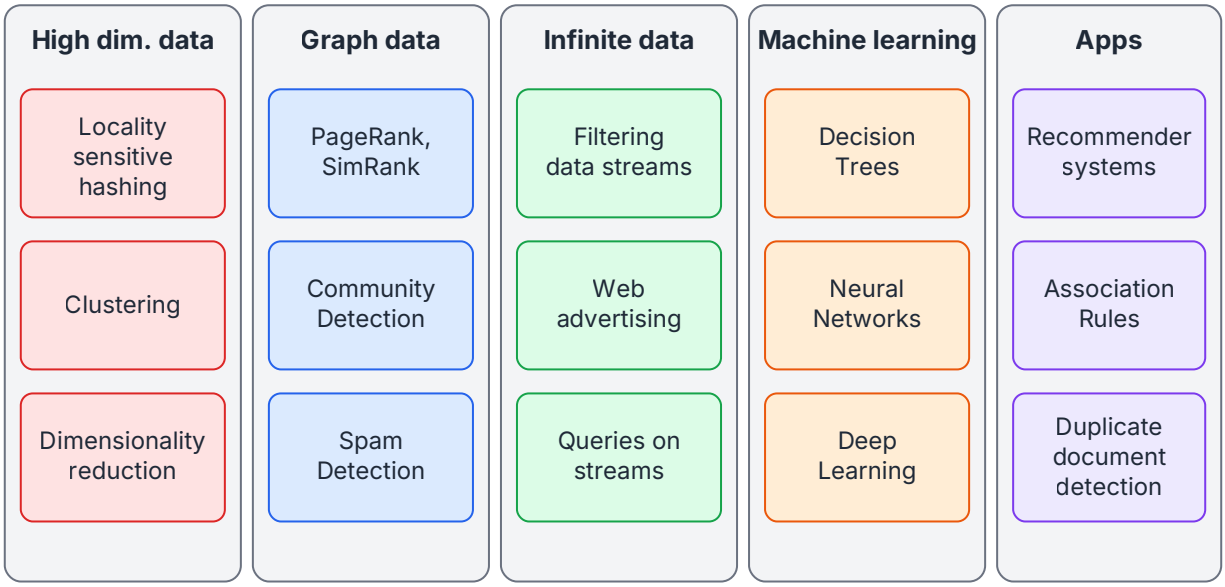


Figure 6. Slide 20 redrawn. Columns 1–3 group topics by the type of data, column 4 is machine learning, column 5 is applications.

Column	Topics	What they do
High dim. data	Locality sensitive hashing; Clustering; Dimensionality reduction	LSH finds similar items fast; clustering groups them (Lecture 5 covers scalable clustering); dimensionality reduction cuts the number of attributes while keeping the structure.
Graph data	PageRank, SimRank; Community detection; Spam detection	PageRank scores page importance from links; SimRank scores node similarity; community detection finds tightly linked groups; link-based spam detection finds suspicious linking.
Infinite data	Filtering data streams; Web advertising; Queries on streams	Process an endless flow with small memory: keep or drop items, choose ads on the fly, answer questions about the recent stream.
Machine learning	Decision trees; Neural networks; Deep learning	Models learned from labeled data (this is the "labeled" type on slide 18).
Apps	Recommender systems; Association rules; Duplicate document detection	End applications. Association rules and frequent itemsets are Lecture 4; **Beyond the slides:** duplicate detection relies on LSH from column 1 (a dependency the slide does not draw).

Beyond the slides: a one-line preview of the lectures covered by this study guide: **Lecture 2** MapReduce (distributed computing); **Lecture 3** Spark; **Lecture 4** large-data frequent pattern analysis (frequent itemsets, association rules, SON); **Lecture 5** scalable clustering (k-means, CURE, BFR).

SLIDE NOTE

Cosmetic word-wrap glitches on slide 20 ("Recommen/der systems", "Dimensional/ity reduction"). The correct names are "Recommender systems" and "Dimensionality reduction". "Spam detection" appears both here (graph column) and in slide 19's problem list.

"How do you want that data?"

Lecture 1 · Slide 21

Slide 21 is a cartoon of a chef in a tall hat at a barbecue grill, wearing an apron that says "I (heart) data", with the caption **"How do you want that data?"** It is a pun on a chef asking how you want your steak cooked. The serious reading: data can be "served" in different shapes (high-dimensional, graph, stream, labeled) and cooked in different ways (single machine, distributed, streaming), the two axes of slides 18–20. There is nothing to memorise here.

About the course: logistics, grading, prerequisites

Lecture 1 · Slides 22–29

Slide 22 ("About the Course") is only a divider; slides 23–29 are administration. A short summary is enough for studying, but know the grading split and the prerequisites.

Slide	Topic	What it says
23	Course staff	The teaching assistant (TA) is Rafael Dolores (rafd47@yorku.ca). All questions about course projects go to the TA.
24	Logistics	Course page is eClass (eclass.yorku.ca); lecture slides are posted at least 30 minutes before the lecture; office hours are online via Zoom (times on eClass). Reading: the book **Mining of Massive Datasets (MMD)** by Leskovec, Rajaraman and Ullman, with a free PDF on eClass.
25	Communication	Projects are posted on eClass; use e-mail (szlichta@yorku.ca) for course matters; announcements go on eClass, so check it regularly. Auditors are welcome to sit in, subject to space.
26	Work for the course	See the grading table below. Big-data graduate positions are advertised.
27	Equipment	A pictures-only slide: a laptop, the Google Colab logo, a pen and lined paper. Read it as: bring a laptop (programming work uses Google Colab, a free cloud notebook environment, beyond the slides), and pen and paper for written work.
28	Prerequisites	**EECS 3421** (Introduction to Database Management Systems) and **EECS 3101** (Design and Analysis of Algorithms). Some background is provided, but the class assumes you know the basics of both.
29	Actions	**Read Chapter 1 of the course book MMD**, which also includes the prerequisite material.

Grading split

Lecture 1 · Slide 26

Component	Weight
Projects (two parts of 10%: midterm report Parts 1–3, final report Parts 4–6)	20%
Midterm I	35%
Presentation and Q&A	10%
Midterm II	35%
Total	**100%**

Check: 20 + 35 + 10 + 35 = 100. The two midterms together are 70%, so exams dominate the grade. The slide's "20% (x2 10%)" is ambiguous; it means two project components of 10% each.

NOT EXAMINED — LOGISTICS

Names, e-mail addresses, web addresses and the equipment picture will not be examined. The grading weights and prerequisites are worth knowing for planning your study time.

QUICK RECAP OF SLIDES 1–29

- Data is exploding (slides 2–3) and contains value (slide 4); data analytics is discovering knowledge and actionable insights from data (slide 5).
- Data must be stored, managed and **analysed**; this class is about analysis; big data analytics ≈ data science (slide 8).
- Good patterns are **Valid, Useful, Unexpected, Understandable** (slide 10). Tasks are **descriptive** (clustering) or **predictive** (recommender systems) (slide 11).
- **Bonferroni's principle:** if the expected number of matches under pure randomness is much larger than the real cases, most findings are bogus. Hotel example: about 250,000 innocent suspicious pairs (slides 12–14, Quiz L1-Q1).
- Three challenges: **quality, streaming, scalability**; with data modalities and data operators they form the framework of slide 15.
- Data science sits among databases, ML and CS theory/statistics; this class stresses **scalability, algorithms, computing architectures, automation** (slides 16–17).
- Data types: high-dimensional, graph, infinite, labeled; computation models: single machine, distributed, streams (slide 18). Tools: MapReduce, Spark, LSH, Bloom filters, dynamic programming (slide 19). The course map is slide 20.
- Logistics: prerequisites EECS 3421 and 3101; grading 20/35/10/35; read MMD Chapter 1 (slides 22–29).

Theory Q&A bank

Lecture 1 · Slides 1–29

Cover the answer, write your own, then compare. Each model answer is short enough to write in an exam; the numbered steps are the marking points.

Q1. What is data analytics? Why does it matter now?

Lecture 1 · Slides 2–5, 8

Data analytics is **discovering knowledge and actionable insights from data**. It matters now because (1) data is being produced at enormous scale (social networks, shared content, click streams), (2) demand is high from business, government and hospitals, and (3) the value is not in storing the data but in analysing it. Data must be stored, managed and analysed; this course concentrates on the last step. Big data analytics ≈ data science.

Q2. Name and explain the four properties of a good discovered pattern.

Lecture 1 · Slide 10

1. **Valid:** it holds on new data with some certainty, not only on the data it was found in.
2. **Useful:** it is possible to act on it.
3. **Unexpected:** it is non-obvious to the system, so it adds knowledge.
4. **Understandable:** humans can interpret it.

A pattern produced by pure chance fails **Valid**; that is the Bonferroni problem.

Q3. Compare descriptive and predictive data analytics methods.

Lecture 1 · Slide 11

	Descriptive	Predictive
Goal	Find human-interpretable patterns that describe the data	Use some variables to predict unknown or future values of other variables
Question	What is in the data I have?	What is not known yet / will happen?
Slide example	Clustering	Recommender systems

Q4. State Bonferroni's principle.

Lecture 1 · Slide 12 (statement supplied beyond the slides, MMD §1.2.3)

Calculate the **expected number of occurrences** of the events you are looking for, **assuming the data is random**. If this number is **significantly larger** than the number of real instances you hope to find, then almost anything you find is **bogus**: a statistical artifact, not evidence. In short: if you search in more places than the data can support, you will find noise. Slide 12 itself only names the risk (an analyst "discovers" meaningless patterns); its statement is missing.

Q5. Explain the hotel example and its conclusion.

Lecture 1 · Slides 13–14

Task: find (unrelated) people who stayed at the same hotel on the same day at least twice. Data: 10^9 people, 1,000 days, each person in a hotel 1% of days, hotels hold 100 people, so $10^9 \times 1\% \div 100 = 10^5$ hotels. Assume everyone behaves randomly.

1. Both people in some hotel on a given day: $0.01 \times 0.01 = 10^{-4}$.
2. Same hotel (Y picks X's hotel among 10^5): $10^{-4} \times 10^{-5} = 10^{-9}$.
3. Same hotel on two given days: $(10^{-9})^2 = 10^{-18}$.
4. Pairs of people $\approx (10^9)^2/2 = 5 \times 10^{17}$; pairs of days $\approx 1000^2/2 = 5 \times 10^5$.
5. Expected pairs = $5 \times 10^{17} \times 5 \times 10^5 \times 10^{-18} = 250{,}000$.

Conclusion: even with nobody doing anything wrong, about 250,000 pairs look suspicious. Far more than the few real cases, so the rule is meaningless alone; we need additional evidence and a more efficient way than checking all pairs (slide 14).

Q6. Why does a very unlikely coincidence still happen many times?

Lecture 1 · Slides 13–14

Each single event is rare (10^{-18} for one pair of people and one pair of days), but the number of opportunities is huge ($5 \times 10^{17} \times 5 \times 10^5 \approx 2.5 \times 10^{23}$). Expected count = probability × number of opportunities = $2.5 \times 10^{23} \times 10^{-18} = 250{,}000$. A tiny probability times an enormous number of trials gives a large expected number of false positives.

Q7. What are the three challenges, the data modalities and the operators in the framework "What matters when dealing with data?"

Lecture 1 · Slide 15

- **Challenges:** quality (dirty or missing data), streaming (data arrives continuously), scalability (too big for one machine).
- **Data modalities:** ontologies, structured data, graphs, text, multimedia.
- **Data operators:** collect, prepare, model, reason, visualize.

A problem is a point in this 3-D space: one choice on each axis.

Q8. What fields does data science overlap with? Contrast the database and the machine-learning view.

Lecture 1 · Slides 16–17

It overlaps databases (large-scale data), machine learning (complex models) and CS theory (algorithms); slide 17 also names statistics and AI. To a **database person** data science is an extreme form of **analytic processing**: queries that examine large amounts of data. To an **ML person** it is the **inference of models** from data. The slide's verdict: "in this class we will do all".

Q9. What does this class stress compared with the neighbouring fields?

Lecture 1 · Slide 17

More stress on: (1) **scalability** (still works when data grows hugely), (2) **algorithms** (efficient, not just correct), (3) **computing architectures** (clusters, distributed file systems, MapReduce and Spark), (4) **automation for handling large data** (the system, not a person, copes with the size).

Q10. What types of data and models of computation does the course cover?

Lecture 1 · Slide 18

- **Data:** high dimensional, graph, infinite/never-ending, labeled.
- **Computation:** single machine in memory, distributed computing, streams and online algorithms.

The best algorithm depends on the pair (data type, computation model).

Q11. List the real-world problems and tools named in the course introduction.

Lecture 1 · Slide 19

Problems: recommender systems, link analysis, spam detection, duplicate document detection. **Tools:** MapReduce and Spark (distributed computing), Locality Sensitive Hashing and Bloom filters, dynamic programming.

Q12. Describe how the course topics are organised ("how it all fits together").

Lecture 1 · Slide 20

Five columns. **High-dimensional data:** LSH, clustering, dimensionality reduction. **Graph data:** PageRank/SimRank, community detection, spam detection. **Infinite data:** filtering data streams, web advertising, queries on streams. **Machine learning:** decision trees, neural networks, deep learning. **Apps:** recommender systems, association rules, duplicate document detection. Columns carry the meaning; rows are unrelated.

Q13. Why is a statement such as "these 250,000 pairs are suspects" not reliable, and what would make findings more trustworthy?

Lecture 1 · Slides 10, 12–14

Under pure randomness we already expect about 250,000 such pairs, so being one of them is not evidence. Findings become more trustworthy when (1) additional evidence is required, i.e. more conditions that random behaviour would almost never satisfy, (2) the pattern is checked on new data (Valid), and (3) the expected number of random matches is computed first and compared with the number of real cases hoped for.

Q14. Where does this class sit in the data pipeline, and how are big data analytics and data science related?

Lecture 1 · Slide 8

The pipeline is store → manage → analyse. This class is the **analyse** step. The slide writes Big Data Analytics ≈ Data Science: roughly equivalent terms in this course.

One-page cheat sheet

Lecture 1 · Slides 1–29

Item	Key facts	Slide
Data analytics	Discovering knowledge and actionable insights from data; store → manage → **analyse**; big data analytics ≈ data science	5, 8
V-U-U-U	**Valid** (holds on new data), **Useful** (can act), **Unexpected** (non-obvious), **Understandable** (human-interpretable)	10
Descriptive	Find human-interpretable patterns describing data; e.g. clustering	11
Predictive	Predict unknown or future values from other variables; e.g. recommender systems	11
Bonferroni's principle	Expected number of matches in *random* data ≫ real cases ⇒ findings are bogus	12
Framework	Challenges: quality, streaming, scalability. Modalities: ontologies, structured, graphs, text, multimedia. Operators: collect, prepare, model, reason, visualize	15
Data science overlaps	Databases (large-scale data), ML (complex models), CS theory (algorithms); slide 17 also statistics, AI	16–17
This class stresses	Scalability, algorithms, computing architectures, automation for large data	17
Data types	High dimensional, graph, infinite, labeled	18
Computation models	Single machine in memory, distributed, streams and online	18
Problems / tools	Recommenders, link analysis, spam, duplicates / MapReduce, Spark, LSH, Bloom filters, dynamic programming	19

HOTEL EXAMPLE — THE WHOLE CALCULATION (SLIDES 13–14)

Hotels = $10^9 \times 0.01 \div 100 = 10^5$. P(same hotel, one day) = $0.01 \times 0.01 \times 10^{-5} = 10^{-9}$. P(two days) = $(10^{-9})^2 = 10^{-18}$.

Pairs of people ≈ $n^2/2 = 5 \times 10^{17}$. Pairs of days ≈ $1000^2/2 = 5 \times 10^5$ (exact 499,500).

Expected suspicious pairs = $5 \times 10^{17} \times 5 \times 10^5 \times 10^{-18} = 250{,}000$ (exact arithmetic gives 249,749.99975).

BONFERRONI RECIPE IN 3 STEPS

1. Assume the data is random (nothing special happens).
2. Compute the expected number of "interesting" matches = (number of possible combinations) × (probability each matches).
3. If this is much larger than the number of real cases you hope for, treat findings as bogus; demand more evidence.

Number	Meaning	Number	Meaning
10^9	people tracked	1,000	days
1%	chance in hotel on a day	100	hotel capacity
10^5	hotels	250,000	expected false "suspicious" pairs
20 / 35 / 10 / 35	grading: projects / midterm I / presentation / midterm II (%)	EECS 3421, 3101	prerequisites

Common mistakes & traps

Lecture 1 · Slides 10–20

- **Writing the formula but not the principle.** Slide 12 gives no statement of Bonferroni's principle; you must supply "expected number under random data ≫ real cases ⇒ bogus".
- **Using n^2 instead of $C(n, 2) \approx n^2/2$** for pairs of people or days. That quadruples the hotel answer to 10^6 instead of 250,000 (twice for people, twice for days).
- **Forgetting that only 1% of people are in a hotel on a given day**: the number of hotels is $10^9 \times 1\% \div 100 = 10^5$, not $10^9 \div 100 = 10^7$.
- **Forgetting a step in the probability:** both in a hotel (10^{-4}) *and* the same hotel (10^{-5}) give 10^{-9} per day; skipping the first factor gives 10^{-5} and a wrong answer.
- **Not squaring for two days.** Two days means $(10^{-9})^2 = 10^{-18}$, not 2×10^{-9}.
- **Concluding that 250,000 pairs are real suspects.** The point is the opposite: they are false alarms in a world with no wrongdoers.
- **Confusing Valid with "true on the current data".** Valid means it holds on *new* data.
- **Mixing up descriptive and predictive.** Clustering is descriptive; recommender systems are predictive (slide 11).
- **Listing the wrong axis values.** Challenges are only quality, streaming, scalability; "structured" and "text" are modalities; "visualize" is an operator.
- **Mixing slide 16 and 17.** Slide 16's Venn has CS Theory; slide 17's has Statistics; the four "more stress" items are on slide 17.
- **Treating the logistics and statistics (3 billion users, $975 billion) as exam content.** They are motivation; do know the prerequisites and grading split.

LECTURE 2

Distributed Computing: Distributed File Systems & Map-Reduce

ROADMAP — WHAT THIS LECTURE IS ABOUT

- **The problem (Slides 4–16):** one computer needs about 46 days just to read 200 TB, so big data is stored and processed on a **cluster of commodity nodes** (racks, switches, a backbone). Clusters bring two hard facts: **machines fail every day** (about 1,000 a day at 1M machines) and **moving data over the network is slow** (10 TB at 1 Gbps takes about a day).
- **The three ideas of Map-Reduce (Slide 16):** store data redundantly, move computation close to the data, and give the programmer a simple model that hides the complexity.
- **Storage (Slides 17–25):** the **distributed file system** (GFS / HDFS): huge files cut into 16–64 MB **chunks**, each replicated 2× or 3× on different chunk servers (different racks if possible), a **master / Name Node** that holds only metadata, and a **client library** that reads straight from the chunk servers.
- **Computation (Slides 26–57):** the four stages **Map → group by key (sort and shuffle) → Reduce → write result**; the programmer writes only Map and Reduce. Worked examples: word count, host size, 5-word sequences. Then how it runs in parallel, where data lives, the master, and **failure handling**.
- **Refinements and extras (Slides 58–80):** many small tasks and pipelining, **combiners** (sum, average, median), the **partition function**, **backup tasks** for stragglers, how many map and reduce tasks, the **join**, **cost measures**, implementations (Hadoop, Hive, Pig) and cloud computing (EC2, S3, EMR).
- No code is examined: the pseudo-code on Slides 41–42 is explained in words only. (Lecture 2 has no BFR content; BFR belongs to Lecture 5.)

Exam checklist — by the end of this lecture you can:

- reproduce the 200 TB calculation (2×10^8 MB ÷ 50 MB/s = 4×10^6 s ≈ 46 days) and its 1,000-disc version (4,000 s), and say which assumption the speed-up needs;
- draw and explain the cluster architecture (16–64 nodes per rack, 1 Gbps in a rack, 2–10 Gbps backbone) and rank same node < same rack < other rack by cost;
- derive the failure rate (1,000 servers ÷ 1,000 days = 1 a day; 1M machines = 1,000 a day) and the 10 TB / 1 Gbps transfer time (80,000 s ≈ 0.93 day);
- state the three ideas of Map-Reduce and match each to the problem it solves;
- describe the DFS: chunks, replication, never two copies on one server, master/Name Node (metadata only), client library, recovery from failure, data locality;
- write the Map and Reduce signatures, say how many calls of each occur, and design key and value for word count, host size, 5-word sequences and the natural join;
- trace a small job by hand through map, partition, shuffle, sort and group, and reduce, and explain what the environment (not the programmer) handles;
- say where input, intermediate and output data are stored, what the master does, and what is redone after a map-worker, reduce-worker or master failure (and why);
- explain fine-granularity tasks, M and R, combiners (when legal, the average trick, why median fails), the partition function, and backup tasks;
- define communication cost, computation cost, total cost and elapsed cost, and name the implementations and cloud services on Slides 76–77.

Slide map

Lecture 2 · Slides 1–80

For Lecture 2 the printed slide number equals the PDF page number. Slide 75 carries no printed number but is PDF page 75 (and Slide 70 is a section divider). Slide 11 is a photograph and Slides 78–80 are readings and actions.

Slide(s)	PDF page(s)	Topic	In this guide
1–3	1–3	Title, agenda (DFS, Map-Reduce model, refinements), motivation: large-scale analytics, hard parallel programming	(p. 32)
4	4	Single-node architecture: CPU, memory, disk; "classical" data mining	(p. 33)
5–6	5–6	Google example: reading 200 TB at 50 MB/s takes 4 million seconds, 46+ days	(p. 33)
7–8	7–8	Commodity cluster and Ethernet; split data into chunks, stripe over 1,000 discs, 4,000 s	(p. 34)
9	9	Cluster architecture: rack of 16–64 nodes, switch, 1 Gbps in a rack	(p. 38)
10–11	10–11	Modern data centres: backbone 2–10 Gbps, ~1M machines; photo of racks	(p. 39)
12–13	12–13	Challenge 1: machine failures (1,000 a day) and the two questions they raise	(p. 39)
14	14	Challenge 2: network bottleneck, 10 TB takes about 1 day at 1 Gbps	(p. 40)
15–16	15–16	Large-scale computing goal; the three ideas of Map-Reduce	(p. 47)
17	17	Storage infrastructure: the distributed file system (GFS, HDFS)	(p. 47)
18	18	Typical usage pattern: huge files, rare in-place updates, reads and appends	(p. 48)
19	19	Chunks (16–64 MB) and chunk servers	(p. 48)
20	20	Replication 2× or 3×, replicas in different racks	(p. 49)
21–22	21–22	File C split into chunks C0–C5; copies never on the same chunk server	(p. 49)
23	23	Seamless recovery from failure; chunk servers are also compute servers (data locality)	(p. 50), (p. 51)
24	24	Master node (Name Node) and client library	(p. 52)
25	25	Idea and solution: bring computation to the data; Map-Reduce as programming model	(p. 53)
26	26	Warm-up task: word count (also term-search statistics, web-server logs)	(p. 54)
27–28	27–28	Overview: Map, group by key, Reduce, write result; "outline stays the same"	(p. 54)
29–35	29–35	Map step and Reduce step on key-value pairs (shapes and colours)	(p. 55)
36–40	36–40	Word count step by step; "only sequential reads"	(p. 56)
41–42	41–42	Word-count pseudo-code (explained in words, not examined)	(p. 58)
43–44	43–44	The programming model more specifically: Map(k,v) and Reduce(k', list of v')	(p. 66)
45–47	45–47	Example: host size (total bytes per host)	(p. 67)
48–49	48–49	Example: language model (counting 5-word sequences)	(p. 75)

50	50	Other uses: link analysis, machine learning, frequent itemsets, clustering	(p. 82)
51–52	51–52	The Map-Reduce diagram and how it runs in parallel (partition, shuffle, sort and group)	(p. 82)
53	53	What the environment takes care of (scheduling, communication, partitioning, group by key, failures)	(p. 83)
54	54	The programmer's view of the workflow (Map, shuffle, Reduce)	(p. 84)
55	55	Data flow: input and output in the DFS, intermediate on local disk	(p. 85)
56	56	Coordination: the master, task states, pings	(p. 85)
57	57	Dealing with failures: map worker, reduce worker, master	(p. 86)
58	58	Task granularity and pipelining (Gantt chart)	(p. 87)
59–65	59–65	Combiners: motivation, definition, example, commutative and associative, average, median	(p. 89)
66–67	66–67	Partition function: hash(key) mod R; hash(hostname(URL)) mod R	(p. 100)
68	68	Backup tasks for stragglers	(p. 102)
69	69	How many map and reduce tasks (M much larger than nodes, R smaller than M)	(p. 103)
70	70	Section divider: problems suited for Map-Reduce; join by Map-Reduce	(p. 103)
71–72	71–72	Natural join R(A,B) ⋈ S(B,C) by Map-Reduce	(p. 104)
73–74	73–74	Cost measures: communication, computation, total and elapsed cost	(p. 105)
75–76	75–76	Implementations: Google MapReduce and GFS, Hadoop and HDFS, Hive, Pig	(p. 113)
77	77	Cloud computing: EC2, S3, EMR	(p. 114)
78–80	78–80	Readings (GFS and MapReduce papers), resources, actions ("treat the examples as quiz questions")	(p. 114)
1–80	1–80	Theory Q&A, one-page cheat sheet, common mistakes	(p. 115), (p. 119), (p. 120)

Why one computer is not enough

Lecture 2 · Slides 1–4

Where this lecture sits

Lecture 2 · Slides 1–3

Slide 1 is only the title card: **Distributed Computing: Map-Reduce** (EECS 4415 Big Data Systems, Jarek Szlichta). Slide 2 is the agenda, which is also the map of the whole lecture:

1. **Distributed File System** (DFS): how to *store* data that is too big for one machine, safely. This part is Slides 4–25 and is the subject of this half of the chapter.
2. **Computational model: Map-Reduce**: how to *compute* on that data with a very simple programming model.
3. **Refinements**: improvements to the basic model.

Slide 3 (titled “MapReduce”) states the whole motivation in four lines. The goal is **large-scale computing for data analytics**, meaning running analyses over data far bigger than one machine can hold. There are two challenges: *how to distribute the computation*, and the fact that *distributed/parallel programming is hard*. Map-Reduce is presented as the answer to both: it is Google’s computational/data-manipulation model and an “elegant way to work with big data”. The slide only makes the claim; the rest of the lecture justifies it.

BEGINNER INTUITION

Imagine you must count every word in every book of a national library. One person can do it, but it will take a lifetime. Hiring 1,000 people helps, but now you must decide who gets which books, hand the books out, collect the partial counts, and cope with people who fall ill halfway. Map-Reduce is a system that does that boring management for you, so you only describe *what* to count.

Naming note: the slides write "MapReduce", "Map-reduce" and "Map-Reduce". They all mean the same thing.

The single-node (classical) architecture

Lecture 2 · Slide 4

Slide 4 shows the picture every programmer has in mind: **one machine (a "node")** made of three parts, stacked top to bottom: the **CPU** (the processor that executes instructions), **memory** (RAM, fast but small and temporary) and the **disk** (large and permanent, but much slower). The slide labels this picture **"Classical" data mining and machine learning**: traditional methods assume the data fits on this one machine, so you simply load it and compute.

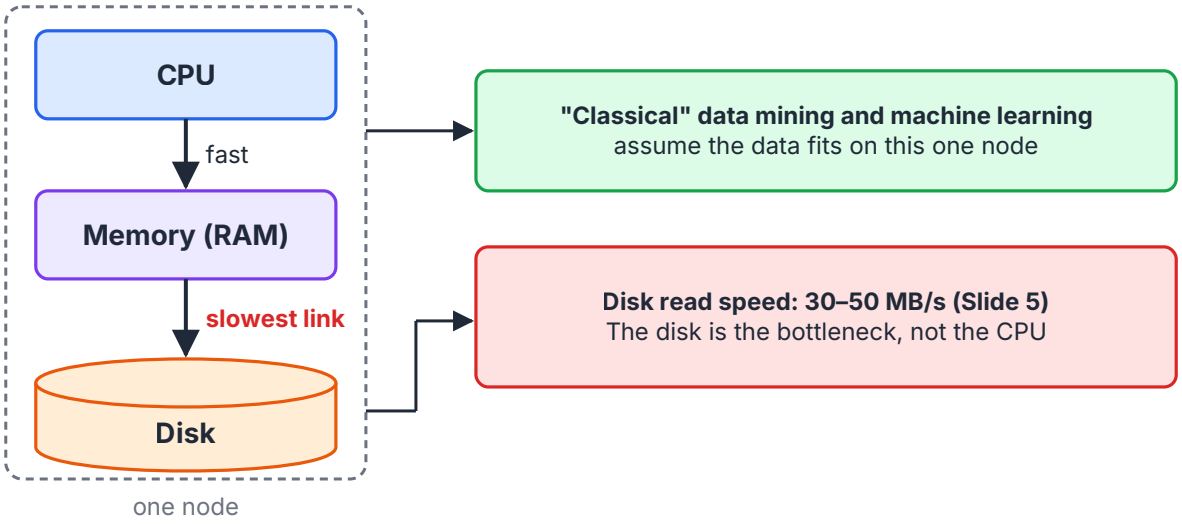


Figure L2p1-1. The single-node picture of Slide 4. The disk is the slowest part, and its speed (Slide 5) will turn out to be the problem.

DEFINITION: NODE

A **node** is one computer in a cluster: a CPU, some memory and a disk. "Single-node" means everything happens on one such computer.

Everything that follows is about what breaks when the data no longer fits this picture. The next four slides give the first and most dramatic reason: the disk is too slow.

Motivating example: reading the web

Lecture 2 · Slides 5–8

The question (Slide 5) and the answer (Slide 6)

Lecture 2 · Slides 5–6

Slide 5 sets the scene with a calculation you should be able to do from memory. There are **10+ billion web pages** and the average page is **20 KB**, so the data is $10 \times 10^9 \times 20$ KB $= 2 \times 10^{11}$ KB = **200 TB** (terabytes). A single computer reads from its disk at **30–50 MB/sec**; the slide says to assume a modern figure of **50 MB/sec**. The question: *how long does it take to read 200 TB at 50 MB a second?*

FORMULA

$\text{time} = \frac{\text{data size}}{\text{read bandwidth}}$ ("time = distance ÷ speed"). Convert the data size into the unit of the rate *first* (here 1 TB = 10^6 MB), so that the units cancel and leave seconds.

Slide 6 gives the answer: **4 million seconds = 46+ days** (exactly 4×10^6 s ÷ 86,400 s/day = 46.3 days). Just *reading* the data once, doing nothing with it, takes over six weeks. The full derivation with every unit conversion is in **Quiz L2-Q1** below, so it is not repeated here.

COMMON MISTAKE

The slide's numbers are best-case. It uses the top of the 30–50 MB/s range and treats "10+ billion" and "200+ TB" as exact. With 30 MB/s the time would be larger still, so 46 days is a *lower bound* for one disk. Do not forget that 1 TB = 10^6 MB (decimal prefixes, as on the slides); using 10^3 or 10^9 gives answers that are off by factors of 1,000.

Commodity cluster and the idea of striping (Slides 7–8)

Lecture 2 · Slides 7–8

Slide 7 makes two points. First, reading is only the beginning: it "takes even more to *do* something useful with the data" (parsing, counting, joining, ranking). Second, the industry settled on a **standard architecture**: a **cluster of commodity nodes** joined by a **commodity network (Ethernet)**.

DEFINITION: COMMODITY HARDWARE

Commodity machines are ordinary, cheap, off-the-shelf computers (not a custom supercomputer). A **cluster** is a large group of such machines wired together and used as one big computer. **Ethernet** is the standard cheap network technology that connects them.

Slide 8 gives the obvious fix: **split the data into chunks**, have **multiple discs and CPUs**, **stripe** (spread) the data across the discs, and **process in parallel**. With 1,000 discs and CPUs the 4 million seconds become 4,000,000 ÷ 1,000 = **4,000 seconds** (about 67 minutes). Each disc now reads only 200 TB ÷ 1,000 = 0.2 TB = 200 GB.

BEGINNER INTUITION

One garden hose empties a swimming pool slowly, and a cleverer person holding the hose does not help. A thousand hoses, each draining its own slice of the pool at the same time, empty it a thousand times faster. The disk is the hose; the CPU speed hardly matters.

COMMON MISTAKE

4,000 s is an **ideal** speed-up. It assumes the data is split perfectly evenly, every disc runs at full speed independently, and nothing fails or waits. It ignores moving results around and combining them. The next slides are exactly about those ignored costs, so do not quote "1,000 discs = 1,000× faster" without stating the assumption.

Slide wording note: Slide 8 says "a 1,000 discs", which should read "1,000 discs", and it spells "discs" while Slide 4 says "Disk". Both mean the same device.

Try it yourself first: cover the solution and compute the read time for one computer, then for 1,000 discs, before you look.

Quiz L2-Q1 How long to read 200 TB from disk? Lecture 2 · Slides 5–8

QUESTION (AS ON THE SLIDES)

Slide 5 — "Motivation: Google Example". 10+ billion web pages × 20 KB = 200+ TB. The average size of a web page is 20 KB, so 10 billion × 20 KB = 200 TB. One computer reads 30–50 MB/sec from disk; assume a modern read bandwidth of **50 MB/sec**. *How long does it take to read 200 TB at 50 MB a second?*

Slides 6–8 (the slides' own answers, which the parts below turn into questions). Slide 6: "Time to read = 4 million seconds = 46+ days!" Slide 7: "Takes even more to *do* something useful with the data! Standard architecture for such problems is emerging: cluster of commodity nodes; commodity network (ethernet) to connect them." Slide 8: "The obvious thing is to split the data into chunks: have multiple discs and CPUs, stripe the data across multiple discs, process in parallel. For example, if you had 1,000 discs and CPUs: 4 million seconds divided by 1,000, which is 4,000 seconds."

(a) How long does **one computer** take to read the 200 TB at 50 MB/s? Show every unit conversion (1 TB = 10^6 MB) and give the answer in seconds, minutes, hours and days. *(This is the question printed on Slide 5.)*

(b) How long if you have **1,000 discs and CPUs** reading in parallel? State the assumption this needs. *(Slide 8 as a question.)*

(c) Explain why this calculation motivates a **cluster of commodity nodes** joined by a commodity network. *(Slides 7–8 as a short explain-question, so you can earn the marks for it.)*

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

> **BEGINNER INTUITION**
>
> Picture emptying a huge swimming pool through **one garden hose**. It does not matter how clever you are (the CPU): the hose (the disk's read speed) limits how fast the water (data) comes out. To empty the pool quickly you do not look for a magic hose, you connect **1,000 hoses** and let each one drain its own slice of the pool at the same time. That is what the slides propose: cut the data into pieces, give each piece its own disk and CPU, and read all the pieces in parallel.

Unit ladder (decimal prefixes, as the slide's numbers imply)	Conversion factor
1 MB, 1 GB, 1 TB	1 MB = 10^3 KB; 1 GB = 10^3 MB; 1 TB = 10^3 GB, so **1 TB = 10^6 MB** = 10^9 KB (the one to remember)
Time	1 min = 60 s; 1 h = 3,600 s; 1 day = 24 × 3,600 = **86,400 s**

Part (a) — one computer reads 200 TB

1. **Formula / idea:** reading is a "rate × time = amount" problem, so $t = \frac{\text{data size } D}{\text{read bandwidth } R}$ (the same as "time = distance ÷ speed"). Here D = 200 TB and R = 50 MB/s. Only the disk's read speed matters, because it is the bottleneck. The units must match: if D is in MB and R in MB/s, the MB cancel and t comes out in seconds.
2. **Check the data size (Slide 5's "200 TB"):** 10 billion = 10 × 10^9 = 10^{10} pages. Total = 10^{10} × 20 KB = 2 × 10^{11} KB. With 1 TB = 10^9 KB: 2 × 10^{11} KB ÷ 10^9 KB/TB = **200 TB**, exactly as on the slide.
3. **Convert D into the unit of the rate (MB):** the rate is in MB/s, so convert TB to MB first, otherwise the units will not cancel. 200 TB × 10^6 MB/TB = 2 × 10^8 MB = **200,000,000 MB**. (Same result from the other side: 2 × 10^{11} KB ÷ 10^3 KB/MB = 2 × 10^8 MB.)
4. **Substitute and divide:** t = 2 × 10^8 MB ÷ 50 MB/s = 4,000,000 s, because (MB) ÷ (MB/s) leaves seconds. In powers of ten: 2 × 10^8 ÷ (5 × 10^1) = 0.4 × 10^7 = **4 × 10^6 s = 4 million seconds** (the number on Slide 6).
5. **Convert seconds to human units:** minutes: 4,000,000 ÷ 60 = 66,666.7 min; hours: 66,666.7 ÷ 60 = 1,111.1 h (= 4,000,000 ÷ 3,600); days: 1,111.1 ÷ 24 = **46.296 days** (= 4,000,000 ÷ 86,400). Exact breakdown: 46 days = 46 × 86,400 = 3,974,400 s; the remainder 25,600 s = 7 h (25,200 s) + 400 s = 6

min 40 s, so 4,000,000 s = **46 d 7 h 6 min 40 s** ≈ 6.6 weeks ≈ 1½ months. This is the slide's "46+ days". **Answer to (a): 4×10^6 s ≈ 46.3 days for one computer, just to read the data once.**

Part (b) — 1,000 discs and CPUs in parallel

1. **Idea (Slide 8):** split the data into chunks, stripe (spread) it evenly over n = 1,000 discs, give every disc its own CPU, and let all of them read their own chunk *at the same time*, each at 50 MB/s.
2. **Assumptions (write them down for marks):** (i) the data is spread *evenly*, so every disc has the same amount to read; (ii) each disc still delivers its full 50 MB/s; (iii) the discs work independently: no waiting for the network, no failures, no time to combine results; (iv) only reading is timed. This is the **ideal case** (perfect linear speed-up).
3. **Method 1 — divide the data:** each disc holds D/n = 200 TB ÷ 1,000 = 0.2 TB = 200 GB = 2×10^5 MB. All discs run at once, so the whole job takes the time of one chunk: $t = 2 \times 10^5$ MB ÷ 50 MB/s = **4,000 s**.
4. **Method 2 — divide the time (the slide's shortcut):** 4,000,000 s ÷ 1,000 = **4,000 s**. Both methods agree. Cross-check with the combined bandwidth: 1,000 × 50 MB/s = 50,000 MB/s = 50 GB/s, and 2×10^8 MB ÷ 5×10^4 MB/s = 4,000 s. In words: 1,000 discs behave like one imaginary disc that is 1,000 times faster.
5. **Convert:** 4,000 s ÷ 60 = 66.7 min = 1 h 6 min 40 s ≈ 1.11 h (4,000 ÷ 3,600). The speed-up is 4,000,000 ÷ 4,000 = 1,000 (the number of discs). **Answer to (b): 4,000 s ≈ 66.7 min ≈ 1.1 h, instead of 46.3 days.**

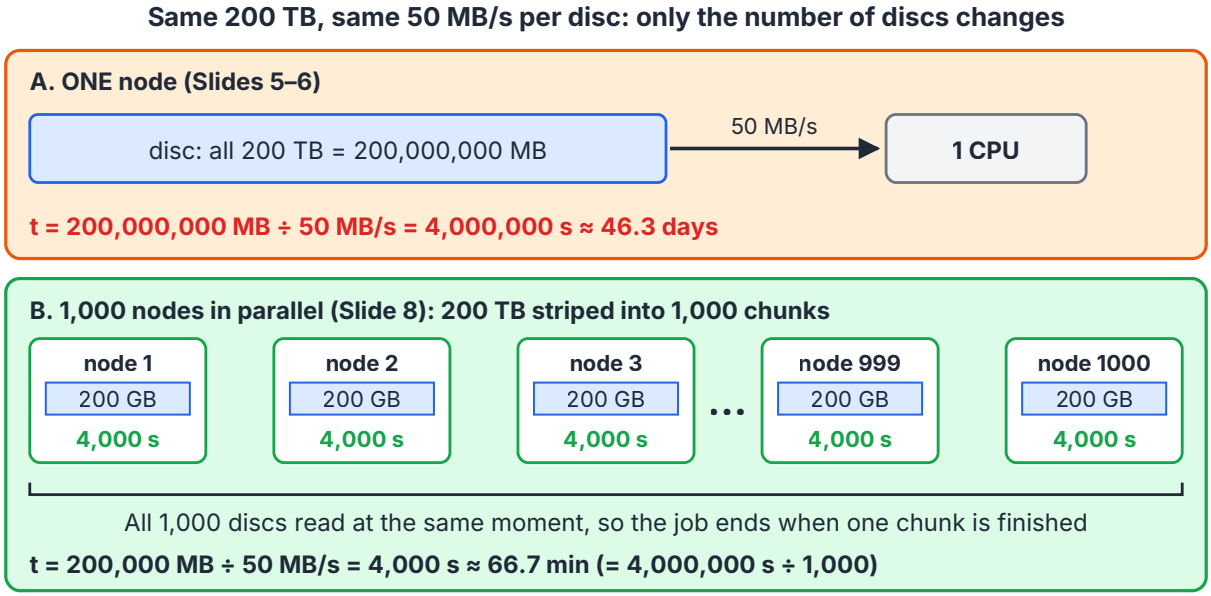
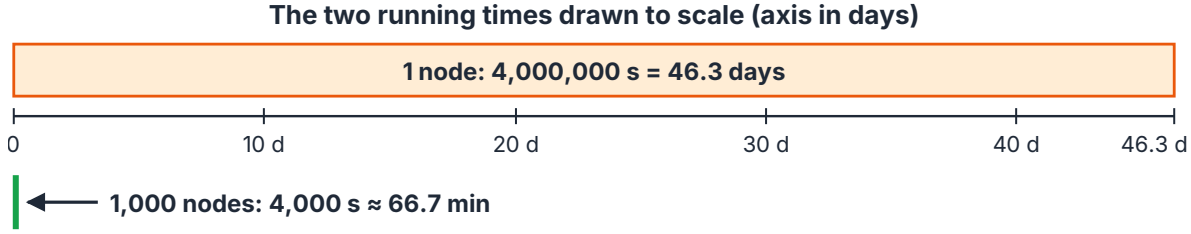


Figure L2-Q1-1. The mechanism: striping the 200 TB over 1,000 discs leaves each disc only 200 GB (2×10^5 MB) to read, so the whole job takes the time of one small chunk. Boxes are not to scale.



The green sliver is 1/1,000 of the top bar (drawn a little wider than true scale so it stays visible).

Figure L2-Q1-2. On one time axis the parallel run is a sliver: one thousandth of the single-node bar.

Part (c) — why this motivates clusters of commodity nodes

1. **The bottleneck is the read speed of one disc, not the arithmetic.** One computer reads only 30–50 MB/s (Slide 5), so its time grows in direct proportion to the data: 200 TB takes 46 days. "Classical" data mining and machine learning run on one node of CPU, memory and disc (Slide 4). We cannot make one disc 1,000 times faster, but we *can* use 1,000 discs.

2. **Reading is only the minimum.** Slide 7: "Takes even more to *do* something useful with the data!" (parsing, counting, ranking ...). So we need the CPUs, memory and discs of many machines together, not only many discs.
3. **The fix is parallelism (Slide 8):** split the data into chunks, use multiple discs and CPUs, stripe the data across the discs, and process in parallel. With 1,000 nodes the read drops from 46 days to about an hour.
4. **Hence the standard architecture (Slide 7):** a **cluster of commodity nodes** (ordinary, inexpensive, off-the-shelf machines, each with CPU, memory and disc) connected by a **commodity network** (ethernet). Slides 9–10 show it: racks of 16–64 nodes, 1 Gbps between any pair of nodes in a rack through a switch, a 2–10 Gbps backbone between racks, and Google was guesstimated to have about 1 million machines. Beyond the slides: thousands of cheap machines give far more total disc bandwidth, CPU power and storage per dollar than one very expensive machine ("scale out" instead of "scale up"), and capacity grows simply by adding nodes.
5. **The price (what the rest of Lecture 2 solves):** distributed / parallel programming is hard (Slide 3); machines fail (one server stays up about 1,000 days, so 1,000 servers lose about 1 per day and 1 million machines about 1,000 per day, Slide 12); data must stay available and long computations must survive failures (Slide 13); and the network is a bottleneck (1 Gbps: moving 10 TB takes about 1 day, Slide 14). So we need a framework that does not move data around much: the distributed file system and Map-Reduce.

IDEAL SPEED-UP: WHAT THE SLIDE ASSUMES

The "4,000 seconds" is an **ideal** number: perfectly even chunks, every disc reading at full speed, no time spent moving data over the network, no failures, and only *reading* counted. Beyond the slides: in practice the data must first be spread over the discs, chunks may be unequal (the job waits for the slowest node), failed work must be redone, and partial results must be combined, so the real time is longer and the 1,000-fold speed-up is an upper limit. The slide itself only gives the ideal division by 1,000.

COMMON MISTAKE

- **Wrong power of ten:** using 1 TB = 10^3 MB gives 4,000 s instead of 4,000,000 s; using 10^9 MB gives 4×10^9 s. TB to MB needs two steps of 1,000 (TB → GB → MB), so 1 TB = 10^6 MB. Always check against the slide: 4 million seconds.
- **Bits versus bytes:** disc speed is in MB/s (bytes); the network figures on Slides 9–10 are in Gbps (bits). Beyond the slides: 1 byte = 8 bits, so 50 MB/s = 400 Mbps and 1 Gbps = 125 MB/s at best.
- **Stopping at "4 million seconds":** 4×10^6 is seconds, not days. Divide by 86,400 to get 46.3 days.
- **Multiplying instead of dividing by n:** more discs means *less* time. The speed of each disc stays 50 MB/s; only the amount each disc must read shrinks.
- **Saying "buy a faster CPU":** the bottleneck here is disc read bandwidth, so the answer is more discs working at the same time.

EXAM TIP

Write the formula (time = size ÷ rate), convert the size into the unit of the rate (MB), show the units cancelling (MB ÷ MB/s = s), then convert seconds to days with ÷ 86,400. For the parallel case state the assumption (even striping, independent reads) and the shortcut $t \div n$. If the exam changes the numbers, the method is identical: t = (number of items × size per item) ÷ (rate per disc × number of discs). Part (c) is normally 3–4 sentences: disc speed is the bottleneck, parallelism on many cheap nodes gives the speed-up, and the cost is failures and the network, which the distributed file system and Map-Reduce handle.

Practice variants (same method; not from the slides)

Discs / CPUs n	Time = 4,000,000 s ÷ n	In friendlier units
1	4,000,000 s	46.3 days
10	400,000 s	111.1 h = 4.63 days
100	40,000 s	11.1 h
1,000 (the slide)	**4,000 s**	**66.7 min ≈ 1.11 h**
10,000	400 s	6.7 min

- Slower end of the quoted range (30 MB/s): $t = 2 \times 10^8 \div 30 = 6{,}666{,}667$ s ≈ 77.2 days on one computer. Only the rate changes.
- Backwards: discs needed to read 200 TB in 1 hour = 4,000,000 s ÷ 3,600 s = 1,111.1, so at least 1,112 discs (round *up*; 1,111 discs would need 3,600.4 s).
- Binary prefixes (1 TB = 2^{40} B, 1 MB = 2^{20} B) would give 4,194,304 s ≈ 48.5 days. The slides use powers of ten, so use 4×10^6 s ≈ 46.3 days.

Final answer:

(a) One computer: 200 TB = 2×10^8 MB, so $t = 2 \times 10^8$ MB ÷ 50 MB/s = 4×10^6 s (4 million seconds) ≈ 66,667 min ≈ 1,111 h ≈ 46.3 days (46 d 7 h 6 min 40 s; the slide's "46+ days").

(b) 1,000 discs and CPUs in parallel: each reads 200 GB = 2×10^5 MB, so t = 4,000,000 s ÷ 1,000 = 4,000 s ≈ 66.7 min ≈ 1.1 h, a 1,000-fold speed-up (ideal case: even striping, independent reads, only reading counted).

(c) One disc's read speed is the bottleneck (46 days just to read, more to process), so the data is striped over a cluster of cheap commodity nodes joined by commodity ethernet that read and process the chunks in parallel; the price is node failures, a network bottleneck and hard programming, which the distributed file system and Map-Reduce are designed to handle.

Where the marks come from: (1) formula time = data size ÷ read rate, with D = 200 TB and R = 50 MB/s; (2) data size and unit conversion: 10 billion × 20 KB = 200 TB and 200 TB = 200×10^6 MB = 2×10^8 MB; (3) correct division, units cancelling: $2 \times 10^8 \div 50 = 4 \times 10^6$ s; (4) conversion to days with 86,400 s/day: 46.3 days (and minutes/hours if asked); (5) parallel case: divide the data (200 GB per disc) or the time by 1,000, giving 4,000 s, with the even-striping / no-overhead assumption stated; (6) conversion 4,000 s ≈ 66.7 min; (7) explanation: disc bandwidth is the bottleneck, so use many discs and CPUs in parallel, built from cheap commodity nodes and commodity ethernet, and mention the cost (failures, network, programming). Even if the final number is wrong, steps (1), (2), (5) and (7) can still earn marks.

Slide check: matches the slide's answers. Slide 5 says "200+ TB", Slide 6 says "4 million seconds = 46+ days" (exact: 4,000,000 s = 46.296 days = 46 d 7 h 6 min 40 s), and Slide 8 says 4 million ÷ 1,000 = 4,000 seconds. The "+" in "10+ billion", "200+ TB" and "46+ days" means "at least"; with exactly 10 billion pages the figures are exactly 200 TB, 4,000,000 s and 46.3 days. Notes (not errors): Slide 5 quotes a 30–50 MB/s range but then assumes the optimistic 50 MB/s (30 MB/s would give 77.2 days); the slides do not convert 4,000 s into minutes (66.7 min), which is added above. No slide erratum.

Cluster architecture

Lecture 2 · Slides 9–11

One rack (Slide 9)

Lecture 2 · Slide 9

Slide 9 draws the basic building block. A **rack** is a physical frame that holds **16–64 nodes**; each node is the CPU + memory + disk of Slide 4. All nodes in the rack plug into one **switch** (a network device that forwards data between the machines plugged into it). The slide labels the link speed: **1 Gbps between any pair of nodes in a rack**.

DEFINITION: RACK, SWITCH, GBPS

A **rack** = 16–64 nodes behind one **rack switch**. **Gbps** = gigabits per second (10^9 *bits* per second). Network speeds are quoted in bits, disk speeds in bytes; 1 byte = 8 bits, so **1 Gbps = 125 MB/s**.

COMMON MISTAKE

Gbps is not GB/s. Lower-case b is a bit, capital B is a byte. Forgetting the factor of 8 makes every network calculation 8 times too fast.

Many racks: the data centre (Slides 10–11)

Lecture 2 · Slides 10–11

Slide 10 (“Modern Data Centers”) repeats the rack and joins the racks through higher-level switches. Traffic between two racks must climb through a **backbone** of **2–10 Gbps**. The slide adds that “it was guestimated that Google had **~1M machines**”. Slide 11 is a photograph of rows of server racks packed with cables: it shows what “a cluster of commodity nodes” physically looks like and contains nothing to memorise.

The result is a **tree**: nodes at the leaves, rack switches above them, a backbone above those. The numbers needed to read this picture (how many racks 1M machines need, what 1 Gbps and 2–10 Gbps mean in MB/s) are worked out in **Quiz L2-Q2** below, together with a labelled diagram of the hierarchy.

Where is the data?	Link used	Speed (slides)	Network cost
On the same node’s disk	no network at all	disk read ≈ 50 MB/s (Slide 5)	**cheapest: no network**
On another node in the same rack	rack switch	1 Gbps (125 MB/s) per pair (Slide 9)	cheap
On a node in another rack	rack switches + backbone	2–10 Gbps, shared (Slide 10)	most expensive

This ranking (same node < same rack < other rack) is the reason the rest of the lecture keeps talking about *locality*. The “network cost” column is the standard reading of the slides’ picture. The ranking is by network use and sharing, not raw bandwidth: a local disk (50 MB/s) is slower per reader than a 1 Gbps link (125 MB/s), but it uses no shared network and scales with the number of nodes.

BEYOND THE SLIDES

The slides do not say whether “2–10 Gbps” is per rack or the total capacity. In *Mining of Massive Datasets* (Section 2.1.1) the point is that many nodes may want to use the inter-rack links simultaneously, so each gets only a share. Quiz L2-Q2 works out the fair-share numbers.

Cluster computing challenges

Lecture 2 · Slides 12–14

Challenge 1: machines fail (Slides 12–13)

Lecture 2 · Slides 12–13

Cheap machines break. Slide 12: **one server may stay up 3 years (1,000 days)**. If you have **1,000 servers**, expect to lose **1 a day**; if Google had **~1M machines**, then **1,000 machines fail every day**.

FORMULA: EXPECTED FAILURES PER DAY

failures/day ≈ (number of machines) ÷ (average lifetime in days). Check: 1,000 ÷ 1,000 = 1 per day; 1,000,000 ÷ 1,000 = 1,000 per day. This assumes failures are independent and happen at a steady average rate.

SLIDE ERRATUM (MINOR)

Slide 12 has the typo “loose” for “lose”. Also “3 years (1,000 days)” is a rounding: 3 × 365 = 1,095 days. The conclusion is unchanged. The quiz below derives rates per hour and the average gap between failures.

BEGINNER INTUITION

A building with 1,000 light bulbs that each last about 1,000 days sees one bulb die on an average day. Nobody panics; spare bulbs are kept. A data centre with a million bulbs sees a thousand die every day, so replacing them is routine, not an emergency. The software, not the hardware, must absorb failure.

Slide 13 turns this into two design questions, and they are the two halves of the lecture:

- **Storage:** how do we store data *persistently* (not lost) and keep it *available* (readable) if nodes can fail? Answer: the distributed file system (Slides 17–24).
- **Computation:** how do we deal with node failures *during a long-running computation*? Answer: the Map-Reduce framework re-runs lost work instead of starting over (second half of the lecture).

(Slides 12 and 13 both carry the title "Challenges (1)": the numbering groups challenges, not slides.)

Challenge 2: the network bottleneck (Slide 14)

Lecture 2 · Slide 14

Slide 14 ("Challenges (2)") asks *how do you distribute computation?* and names the obstacle: the **network bottleneck**. Network bandwidth is **1 Gbps** and **moving 10 TB takes approximately 1 day**. (Exactly: 10 TB = 8×10^{13} bits ÷ 10^{9} bit/s = 80,000 s = 0.93 days; the quiz derives it.) The conclusion on the slide: **we need a framework that does not move data around so much during computation**.

KEY IDEA

Data is heavy and the network is slow, but a program is small. So send the *program* to the data, not the data to the program. This single idea shapes the whole design of the DFS and Map-Reduce.

BEGINNER INTUITION

You can carry an entire library through one narrow door to a reader (slow), or you can send the reader, with a page of instructions, into the library. The data is the library; the program is the reader.

Try it yourself first: before opening the solution, write down the expected failures per day for 1,000 and for ~1M machines, and the time to move 10 TB at 1 Gbps.

Quiz L2-Q2 Cluster failures and the network bottleneck (the numbers) Lecture 2 · Slides 9–14

QUESTION (BUILT FROM THE NUMBERS ON THE SLIDES)

Slides 9–14 state facts about how a cluster is built and what goes wrong with it; they do not pose a numbered question. The four parts below are practice-style questions built directly from those numbers. You must be able to *derive* every number and *explain* what it implies. **Data given on the slides:**

Slide	What it says
9 (Cluster Architecture)	Each node has a CPU, memory (Mem) and a disk. **Each rack contains 16–64 nodes**; a switch gives **1 Gbps between any pair of nodes in a rack**.
10 (Modern Data Centers)	Rack switches are joined through a higher switch by a **2–10 Gbps backbone between racks**. "It was guestimated that Google had **~1M machines**."
11	A photograph of racks of servers only. No text, no numbers.
12 (Challenges 1)	Machines fail (node failures). One server may stay up **3 years (1,000 days)**. If you have **1,000 servers**, expect to lose 1/day, so **1 failure/day**. People estimated Google had ~1M machines, so **1,000 machines fail every day**.
13	How to store data persistently and keep it available if nodes can fail? How to deal with node failures during a long-running computation?
14 (Challenges 2)	Network bottleneck: network bandwidth = **1 Gbps**; moving **10 TB takes approximately 1 day**. Need a framework that does not move data around so much during computation.

(a) About how many racks does a cluster of ~1M machines need, and what are the in-rack and between-rack speeds expressed in MB/s?

(b) A server lasts about 1,000 days. Derive the expected number of failures per day for 1,000 servers and for ~1M machines (also per hour, and the average time between two failures).

(c) Derive exactly how long it takes to move 10 TB over a 1 Gbps link, showing every unit conversion, and compare with the slide's "approximately 1 day". How would the answer change if the 2–10 Gbps backbone speed were available?

(d) Explain why the numbers in (b) and (c) motivate (i) storing data redundantly and (ii) moving the computation to the data instead of the data to the computation.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Failures. A building has 1,000 light bulbs and each bulb lasts about 1,000 days on average. Then, on an average day, one bulb burns out. Nobody is alarmed; you keep spare bulbs and replace it. A data centre with a million cheap machines is a building with a million bulbs: about a thousand burn out *every day*, so failure is part of normal operation, not an emergency.

Network. To get a huge pile of books read, you can carry the whole library through one narrow door to the reader (slow), or you can send the reader (a few pages of instructions) to the library. The data is the library, the program is the reader. A 10 TB "library" through a 1 Gbps "door" takes about a day.

Unit facts used in this quiz (decimal prefixes, as on the slides)	Value
Network speed is quoted in **bits** per second	1 Gbps = 10^9 bit/s. 1 byte = 8 bits, so 1 Gbps = 1.25×10^8 byte/s = **125 MB/s**
Data sizes are quoted in **bytes**	1 TB = 10^{12} bytes = 10^6 MB, so 10 TB = 10^{13} bytes = 8×10^{13} bits
Time	1 h = 3,600 s; 1 day = 24 h = 86,400 s

COMMON MISTAKE

Gbps is not GB/s. The lower-case "b" means *bit*, the capital "B" means *byte*. Forgetting the factor 8 gives $10^{13} \div 10^{9} = 10^{4}$ s ≈ 2.8 h, which is 8 times too small. Also: "1 failure per 1,000 days" is the rate of *one* server, not of the whole cluster, and "1 failure per day" is an *average*, not a timetable.

Part (a): the architecture numbers (Slides 9–10)

1. **Formula / idea:** number of racks = (number of machines) ÷ (nodes per rack). Machines ≈ 1M = 10^{6} (Slide 10; a guess, hence "guestimated"). Nodes per rack = 16 to 64 (Slides 9–10). More nodes per rack means fewer racks.
2. **Compute both ends:** $10^{6} \div 64$ = **15,625 racks** (fullest racks) and $10^{6} \div 16$ = **62,500 racks** (smallest racks). So roughly 1.6×10^{4} to 6.3×10^{4} racks; a middle value of 32 nodes per rack gives 31,250 racks. The picture on Slide 10 (nodes, rack switches, a higher switch) is repeated tens of thousands of times: the cluster is a hierarchy.
3. **Convert the speeds into one unit (bits to bytes, ÷ 8):** inside a rack, any pair of nodes talks at 1 Gbps = $10^{9} \div 8 = 1.25 \times 10^{8}$ B/s = **125 MB/s**. The backbone between racks is 2–10 Gbps = **250–1,250 MB/s**. For scale, one local disk reads about 50 MB/s = 400 Mbps (Slide 5).
4. **Beyond the slides:** the backbone is shared. The slides do not say whether "2–10 Gbps" is per rack or in total; take it as the capacity available for traffic that leaves a rack (this is the picture in *Mining of Massive Datasets*, Section 2.1.1: inter-rack bandwidth is only somewhat larger than the in-rack Ethernet, yet many node pairs may need it). All nodes of a rack must then share it, so if every node of a rack sends to other racks at the same time, each gets only its fair share:

Nodes per rack	Fair share at 2 Gbps	Fair share at 10 Gbps
16	2,000 Mbps ÷ 16 = 125 Mbps	10,000 Mbps ÷ 16 = 625 Mbps
64	2,000 Mbps ÷ 64 = 31.25 Mbps	10,000 Mbps ÷ 64 = 156.25 Mbps

The demand-to-capacity ratio is (nodes × 1 Gbps) ÷ backbone = 16 ÷ 10 = 1.6 up to 64 ÷ 2 = 32. Conclusion: moving data between racks is the most expensive kind of network use, which is one more reason to keep computation near its data.

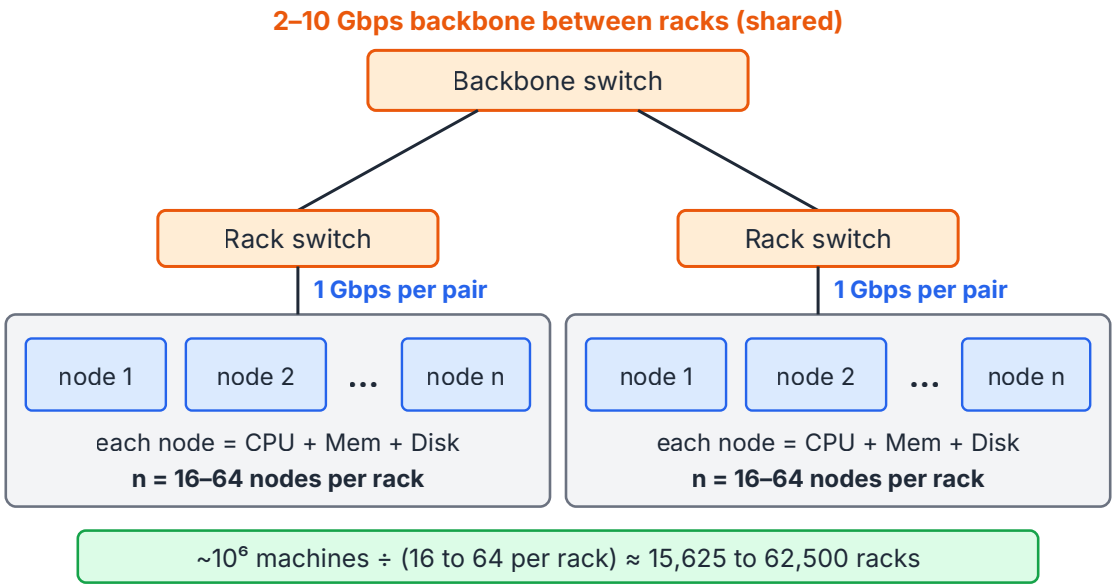


Figure L2-Q2-1. The architecture of Slides 9–10 with the numbers of part (a). Traffic inside a rack crosses one rack switch at 1 Gbps per pair; traffic between racks must also cross the shared 2–10 Gbps backbone.

Part (b): failures (Slide 12)

1. **Model and formula:** if one server lasts on average L = 1,000 days, then on average it fails at a rate of $1/\mathrm{L}$ = 1/1,000 = 10^{-3} failures per server per day (a 0.1 % chance on any given day). Why multiply? Expected

values simply add up across servers, so N identical servers give N times the single-server rate. We assume, as the slide does, that the rate is constant over time.

> **FORMULA**
> expected failures per day $F = N \times (1 \div L) = \frac{N}{L}$ (here L = 1,000 days). Average time between failures = 1 day ÷ F = $L \div N$ days.

2. **1,000 servers:** $F = 1{,}000 \times 10^{-3} = 1{,}000 \div 1{,}000 =$ **1 failure per day**. This is exactly the slide's "expect to lose 1/day". Unit check: servers × (failures ÷ (server · day)) = failures per day. Per hour: 1 ÷ 24 ≈ 0.042. Average gap between failures = 86,400 s ÷ 1 = 86,400 s = **24 h**.
3. **~1M machines:** $F = 10^6 \times 10^{-3} = 10^6 \div 10^3 =$ **1,000 failures per day** (slide: "1,000 machines fail every day!"). Shortcut: the cluster is 1,000 times larger, so there are 1,000 times as many failures. Per hour: 1,000 ÷ 24 ≈ **41.7**; per minute: 1,000 ÷ 1,440 ≈ 0.69. Average gap = 86,400 s ÷ 1,000 = **86.4 s**, i.e. one machine dies every minute and a half, around the clock (1,000 × 365 = 365,000 per year).
4. **Scaling table (same formula, more or fewer servers):**

Servers N	Failures per day = $N \div 1{,}000$	Failures per hour	Average time between failures
16 (a small rack)	0.016	0.00067	62.5 days
64 (a full rack)	0.064	0.0027	15.6 days
1,000	**1**	0.042	**24 h**
10,000	10	0.42	2.4 h (8,640 s)
1,000,000	**1,000**	41.7	**86.4 s**

   The more machines, the more often something fails: **the failure rate grows in direct proportion to the cluster size**.
5. **Precision check on "3 years (1,000 days)":** 3 × 365 = 1,095 days, so the slide rounds 1,095 down to 1,000 (about 9 % lower) to keep the arithmetic easy. With 1,095 days: 1,000 servers give 1,000 ÷ 1,095 ≈ 0.91 failures/day and 10^6 machines give 10^6 ÷ 1,095 ≈ 913 failures/day. Same order of magnitude, so the conclusion is unchanged. In an exam, quote the slide's 1,000 days.
6. **Beyond the slides:** what failures mean for a long computation (Slide 13, second question). Expected failures during a job = N × (job length in days) ÷ 1,000. Assuming independent failures at a constant rate, the chance of at least one failure is $1 - e^{-\lambda}$, where λ is that expected number:

Job	Expected failures λ	P(at least one failure) = $1 - e^{-\lambda}$
1,000 servers, 4,000 s (the parallel read of Slide 8)	1,000 × (4,000 ÷ 86,400) ÷ 1,000 = 0.046	≈ 4.5 %
1,000 servers, 1 day	1	$1 - e^{-1} \approx 63.2$ %
1,000 servers, 10 days	10	≈ 99.9955 %
10^6 machines, 1 hour	$10^6 \div 24 \div 1{,}000 = 41.7$	≈ 100 %

   The longer the job and the bigger the cluster, the more certain a failure becomes. A method that restarts the whole job after every failure would almost never finish.

> **SLIDE ERRATUM**
> Slide 12 writes "expect to *loose* 1/day": this is a typo for "*lose*". Also "3 years (1,000 days)" is a rounding (3 years = 1,095 days), and "1,000 servers: 1 failure/day" is an *average* (expected value), not a guarantee. Slide 14 does not say which link the 1 Gbps refers to (presumably the in-rack, per-node figure of Slides 9–10; the slide does not say) or how "approximately 1 day" was obtained; part (c) derives it. None of this changes the argument.

Part (c): the network bottleneck (Slide 14)

1. **Formula / idea:** time = amount of data ÷ rate. Both must be in the *same units*: bits and bit/s, or bytes and byte/s. Network speeds are quoted in bits, data sizes in bytes, so convert first: $t = \frac{\text{data size } D \text{ (bits)}}{\text{bandwidth } B \text{ (bits/s)}}$.
2. **Data size in bits:** 10 TB = 10×10^{12} bytes = 10^{13} bytes. Multiply by 8 bits per byte: $10^{13} \times 8$ = **8×10^{13} bits**.
3. **Bandwidth in bits per second:** 1 Gbps = 1 giga-bit per second = 10^9 bit/s (Slide 14).
4. **Divide (units cancel: bits ÷ (bits/s) = s):** $t = 8 \times 10^{13} \div 10^9 = 8 \times 10^4$ s = **80,000 s**.
5. **Convert to human units:** 80,000 ÷ 60 = 1,333.3 min; 80,000 ÷ 3,600 = **22.2 h** (exactly 22 h 13 min 20 s); 80,000 ÷ 86,400 = **0.926 day**. So the slide's "approximately 1 day" is correct as an order-of-magnitude statement: it is about 22 hours. **Cross-check in bytes:** 1 Gbps = 125 MB/s and 10 TB = 10^7 MB, so $t = 10^7 \div 125$ = 80,000 s, the same answer. **Sanity check:** in one day a 1 Gbps link at full speed carries 125 MB/s × 86,400 s = 1.08×10^7 MB = 10.8 TB, so 10 TB fills almost exactly one day.
6. **This is a best case:** it assumes the link runs at full line rate with no other traffic and no protocol overhead. **Beyond the slides:** at a realistic 70–80 % efficiency the time is 80,000 ÷ 0.8 to 80,000 ÷ 0.7 s = 27.8–31.7 h, i.e. more than a day. If "TB" meant binary tebibytes (2^{40} bytes), then 10 TiB = 8.80×10^{13} bits would take 87,961 s ≈ 24.4 h ≈ 1.02 days, almost exactly the slide's "1 day". Either way: "about a day".
7. **What-if table (the faster backbone):** same data, bigger bandwidth, $t = 8 \times 10^{13} \div B$.

Link speed B	Time for 10 TB	In hours	In days
1 Gbps (Slide 14)	**80,000 s**	**22.2 h**	**0.93**
2 Gbps (Slide 10, low end)	40,000 s	11.1 h	0.46
10 Gbps (Slide 10, high end)	8,000 s	2.2 h	0.093

Even the best case is hours. **Beyond the slides:** these rows are optimistic lower bounds, because the backbone is shared by many nodes (part a) and one node's own 1 Gbps connection caps what it can send or receive. Scale up: the 200 TB of Slide 5 over one 1 Gbps link = $200 \times 10^{12} \times 8 \div 10^9 = 1.6 \times 10^6$ s ≈ 18.5 days just to copy it.

Part (d): why these numbers lead to the ideas of Map-Reduce (Slides 13, 14, 16)

1. **(i) Failures lead to redundancy (answers Slide 13).**
   - If every chunk of data lived on *one* machine, then because ~1,000 machines die per day, the data on them would be lost or unavailable every day, and a petabyte-scale data set would always have holes. (Beyond the slides: one year of 1 failure/day on 1,000 servers is 365 failures, about 31 % of the servers once a dead server cannot die twice, since $1 - 0.999^{365} \approx 0.306$.) The data must therefore be **stored redundantly on multiple nodes** (Slide 16). A distributed file system (Slide 17: GFS, HDFS) keeps each chunk (16–64 MB, Slide 19) on 2 or 3 different machines (Slide 20), never two copies on the same chunk server (Slide 22), and tries to keep replicas in different racks (Slide 20). When a node dies, the copies on other nodes are still readable (Slide 23: "seamless recovery from disk or machine failure").
   - **Beyond the slides:** a rough calculation. With a failure chance of p = 1/1,000 per node per day and independent failures, the chance that *all* r copies of a chunk are lost in the same day is p^r: 10^{-3} for 1 copy, 10^{-6} for 2 copies, 10^{-9} for 3 copies. Each extra copy makes loss 1,000 times less likely, at the price of using r times more disk. Different racks matter because a rack switch or power failure takes out a whole rack at once, which breaks the "independent" assumption.
   - **Long-running computation:** by the table in part (b), a failure during a long job on a large cluster is almost certain, so the framework must detect it and re-run only the lost pieces of work on other machines instead of starting over. This is Map-Reduce's "handling machine failures" (Slide 53): the

master pings workers (Slide 56) and resets the tasks of a failed worker to idle so they are rescheduled elsewhere (Slide 57).

2. **(ii) A slow network leads to moving the computation to the data (answers Slide 14).** Compare the two ways of processing the same 10 TB. **Beyond the slides (illustration with assumed numbers):** a program of 1 MB, the 10 TB spread evenly over 1,000 nodes, each node reading its own disk at 50 MB/s (Slide 5), processing time ignored.

Strategy	What travels, and the arithmetic	Time
Move the **data** to the computation	10 TB = 8×10^{13} bits through a 1 Gbps link (part c)	80,000 s ≈ 22.2 h
Move the **program** to the data	~1 MB = 8×10^{6} bits per node: $8 \times 10^{6} \div 10^{9} = 0.008$ s per node, so 1,000 × 0.008 = 8 s from one source	**≈ 8 s**
…then each node reads its own share from its **local disk**, all in parallel	10 TB ÷ 1,000 nodes = 10 GB per node; 10 GB ÷ 50 MB/s = $10^{10} \div (5 \times 10^{7})$ = 200 s	**200 s ≈ 3.3 min**

The data is 10 TB ÷ 1 MB = 10^{7} times larger than the program. Total with code shipping ≈ 8 + 200 = 208 s versus 80,000 s: 80,000 ÷ 208 ≈ **385 times faster** (≈ 400 times if the 8 s is ignored, since 80,000 ÷ 200 = 400). Equivalent view: 1,000 disks give 1,000 × 50 MB/s = 50 GB/s = 400 Gbps of combined local read bandwidth, 400 times one 1 Gbps link.

> **COMMON MISTAKE**
> The gain does *not* come from a disk being faster than a link: one disk (50 MB/s = 400 Mbps) is actually slower than a 1 Gbps link, and reading all 10 TB from a single disk would take $10^{13} \div (5 \times 10^{7})$ = 200,000 s ≈ 55.6 h. The gain comes from **parallelism**: every node reads *its own* data locally, so nothing has to squeeze through the one scarce, shared resource, the network.

Hence Slide 14: “Need a framework that does not move data around so much during computation”, and Slides 16, 23 and 25: “Move computation close to data to minimize data movement”, “Bring computation directly to the data! Chunk servers also serve as compute servers”, and “Issue: copying data over a network takes time. Idea: bring computation close to the data”. In Map-Reduce the scheduler even tries to schedule map tasks close to the physical location of their input (Slide 55).

3. **Putting it together:** the two numbers give two design rules, and Map-Reduce (Slide 16) adds a third:

Number (slide)	Problem it causes	Design response
1,000 failures/day for 1M machines; 1 per day for 1,000 (Slide 12)	Data lost or unavailable; long jobs interrupted (Slide 13)	Store data redundantly (2–3 replicas, in different racks); re-run only failed tasks
10 TB over 1 Gbps ≈ 22 h ≈ 1 day (Slide 14); 1 Gbps in rack, 2–10 Gbps between racks (Slides 9–10)	Copying data across the network dominates the run time	Move computation close to data; minimise data movement
Both of the above make programming hard (Slide 15)	Programmers should not handle failures and data placement by hand	A simple programming model (Map-Reduce) that hides the complexity

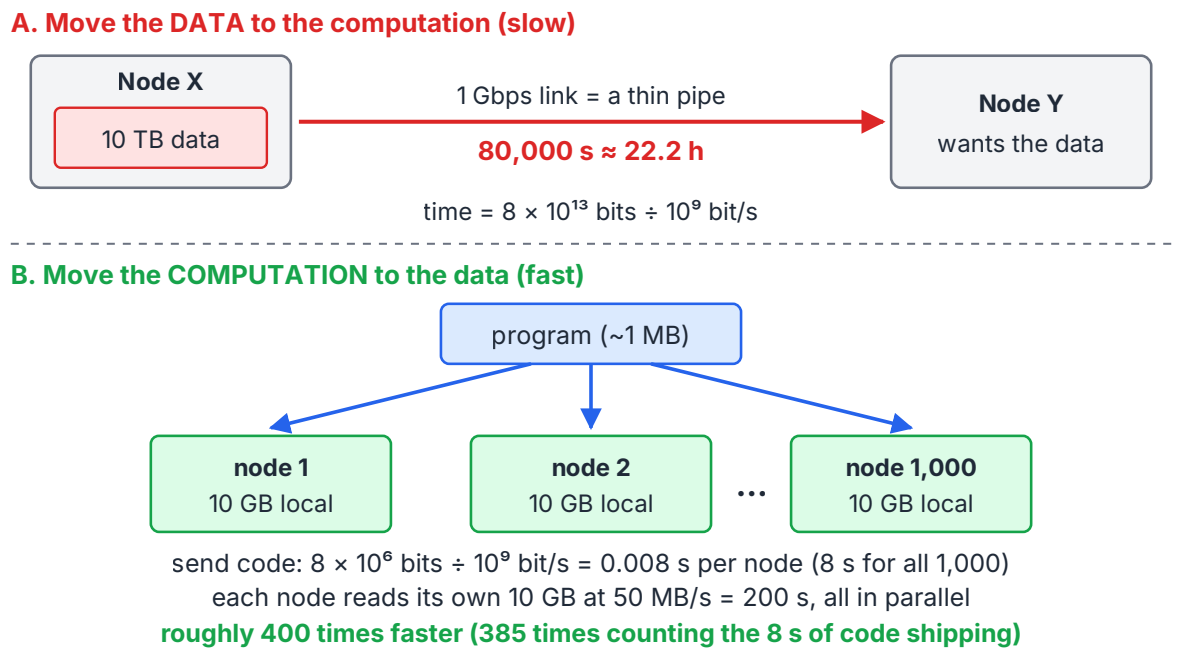


Figure L2-Q2-2. Why Slide 14 asks for a framework that does not move data around: the huge object (10 TB of data) stays where it is and the tiny object (the program, 1 MB assumed) travels. Illustrative numbers computed in parts (c) and (d).

EXAM TIP

For any "how long to move or read X" question, write the three lines: (1) formula $t = D \div B$; (2) convert *both* numbers to the same unit (write "× 8" for bytes to bits explicitly); (3) divide with units and convert seconds to hours or days. Each line earns marks even if the arithmetic slips. For failure questions, state the assumption (average lifetime 1,000 days, constant rate) and say the answer is an expected value. For "why does it motivate X", always link **one number to one consequence to one design rule**: "1,000 failures/day → data and tasks are lost all the time → replicate the data and re-run only failed tasks", and "10 TB takes ~1 day over 1 Gbps → copying data is the bottleneck → move the program to the data".

Final answer:

(a) ~10^6 machines ÷ (16 to 64 nodes per rack) = 15,625 to 62,500 racks. In-rack 1 Gbps = 125 MB/s per pair; backbone 2–10 Gbps = 250–1,250 MB/s, shared between racks.
(b) Expected failures per day $F = N \div 1{,}000$: 1,000 servers → 1 failure/day (one every 24 h); ~1M machines → 1,000 failures/day ≈ 41.7 per hour ≈ one every 86.4 s.
(c) 10 TB = 8 × 10^{13} bits ÷ 10^9 bit/s = 8 × 10^4 s = 80,000 s ≈ 22.2 h ≈ 0.93 day ≈ "about 1 day" (best case); 2 Gbps → 11.1 h, 10 Gbps → 2.2 h.
(d) Frequent failures ⇒ store each chunk redundantly (2–3 replicas on different machines and racks) and re-run only the lost tasks. A slow, shared network ⇒ move the small computation to where the big data already is, instead of moving the data (about 200 s versus 80,000 s in the illustration).

Where the marks come from: (1) Rack count: formula machines ÷ nodes per rack and both values 15,625 and 62,500; speeds converted to MB/s (125 and 250–1,250). (2) Failure rate per server = 1/1,000 per day and $F = N \div 1{,}000$. (3) 1 failure/day for 1,000 servers and 1,000/day for 1M machines (with the per-hour or 86.4 s interpretation as bonus detail). (4) Unit conversion for the network: TB → bytes → bits (× 8) and Gbps = 10^9 bit/s. (5) Division 8 × 10^{13} ÷ 10^9 = 8 × 10^4 s and conversion to about 22 h ≈ 1 day; the backbone comparison (11.1 h, 2.2 h). (6) Explanation (i): failures are certain at scale, so store data redundantly (replication, different racks) and recover/re-run only lost tasks. (7) Explanation (ii): the network is the bottleneck, so move the small program to the large data and minimise data movement.

Slide check: matches the slides' answers (1 failure/day for 1,000 servers; 1,000 failures/day for ~1M machines; 10 TB over 1 Gbps = 22.2 h = 0.93 day, which is "approximately 1 day"). Slide erratum (minor, no answer changes): "loose" should read "lose" (Slide 12); "3 years (1,000 days)" is a rounding of 1,095 days; Slide 14 gives no derivation and no unit convention (decimal TB assumed; binary TB gives 24.4 h). See the Slide erratum box above.

The three ideas of Map-Reduce

Lecture 2 · Slides 15–16

Slide 15 ("Large-scale Computing (3)"; the "(3)" is a leftover from an older deck) restates the goal: **large-scale computing** for **data analytics** problems on **commodity hardware**. The question it poses is the programmer's problem: **how can we make it easy to write distributed programs?** Distributed programming is hard (you must split data, schedule work, move data, survive failures, combine results), so we need a **simple model that hides most of the complexity**.

Slide 16 answers with the three bullets that summarise the lecture. Memorise them; "What challenges does Map-Reduce address and how?" is a classic short-answer question.

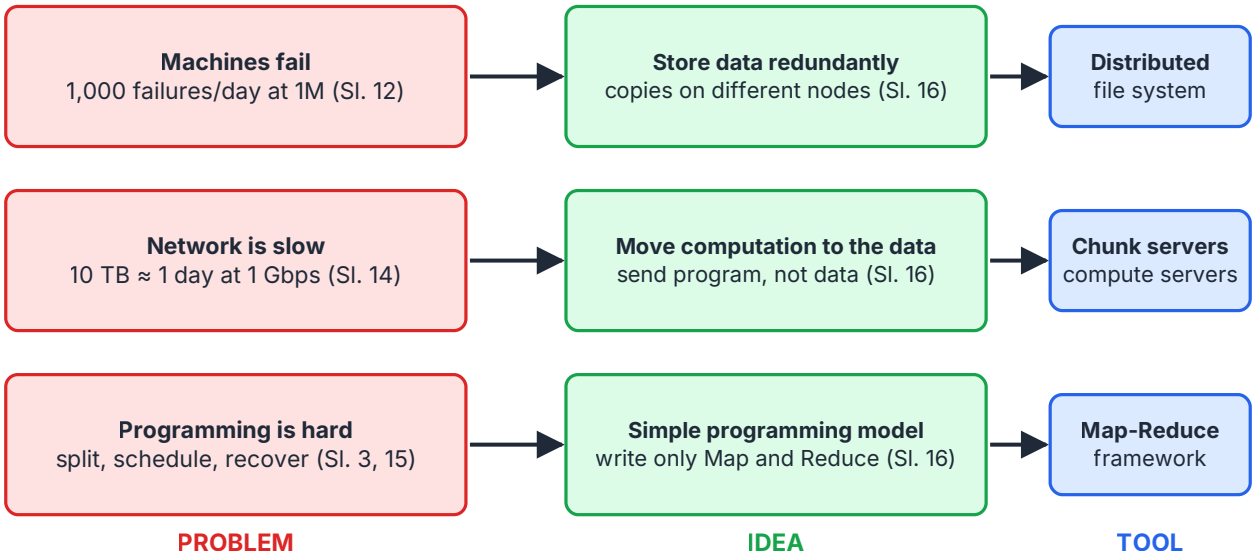


Figure L2p1-2. Slides 12–16 in one picture: each cluster problem is answered by one of the three ideas of Map-Reduce.

Idea (Slide 16)	Problem it answers	Where it is realised
Store data redundantly on multiple nodes for persistence and availability	Machines fail (Slide 12)	Distributed file system: chunks and replicas (Slides 19–23)
Move computation close to data to minimise data movement	Network bottleneck (Slide 14)	Chunk servers double as compute servers (Slide 23)
Simple programming model to hide the complexity of all this "magic"	Distributed programming is hard (Slides 3, 15)	Map and Reduce functions; the framework handles the rest (second half of the lecture)

DEFINITIONS

Persistence: data is not lost when a machine or disk dies. **Availability:** the data can still be read while some machines are down. **Programming model:** the style of program you must write for a system; here, you supply two functions (Map and Reduce) and the system runs them on the cluster.

The distributed file system

Lecture 2 · Slides 17–24

What a DFS is (Slide 17)

Lecture 2 · Slide 17

Slide 17 ("Storage Infrastructure") states the **problem**: if nodes fail, how do we store data persistently? And the **answer**: a **Distributed File System (DFS)**.

DEFINITION: DISTRIBUTED FILE SYSTEM

A file system that stores your data across a cluster but stores **each piece of data multiple times**. It provides **redundancy** (extra copies) and **availability** (data stays readable when some nodes are down). Two implementations are named on the slide: **GFS** (Google File System) and **HDFS** (Hadoop Distributed File System).

BEGINNER INTUITION

A library has one book on a shelf. If the shelf collapses, the book is gone. Now photocopy every chapter of every book, put the copies in different rooms and different buildings, and keep a catalogue saying where each chapter is. Losing any one room loses nothing. A DFS is that library, with disks instead of shelves and the cluster instead of buildings.

Beyond the slides: GFS was described in a 2003 Google paper; HDFS is the open-source clone and follows the same design, so the vocabularies map one to one (see the table in the master-node section).

Typical usage pattern (Slide 18)

Lecture 2 · Slide 18

Slide 18 lists the assumptions the DFS is designed for. They explain its design:

- **Huge files** (hundreds of GB to TB): a file cannot fit on one machine, so it must be cut into pieces spread over machines.
- **Data is rarely updated in place** ("in place" = overwriting bytes in the middle of an existing file).
- **Reads and appends are common**: files are mostly scanned and extended at the end, like logs, web crawls and click streams.

WHY THIS MATTERS

If copies are never edited in the middle, keeping several copies identical is easy and cheap: you only ever add new data. A DFS is therefore **not** a general-purpose file system for many small, frequently edited files. Beyond the slides: that is why HDFS is a poor fit for lots of tiny files or for low-latency random updates.

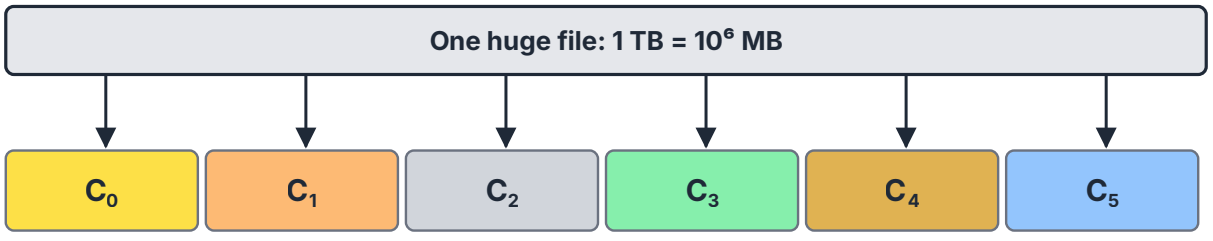
Chunks and chunk servers (Slide 19)

Lecture 2 · Slide 19

Slide 19 introduces the building blocks, with a picture of empty dashed boxes named "Chunk server 1, 2, 3, … N". Data is kept in **chunks** spread across machines, and **each chunk is typically 16–64 MB**.

DEFINITIONS

A **chunk** is one fixed-size piece of a big file (16–64 MB). A **chunk server** is a machine (node) whose disk stores chunks. **N** is just "the large number of chunk servers". In HDFS a chunk is called a **block**.



cut into fixed-size chunks (16–64 MB each; Slide 21 draws six of them for File 1)
at 64 MB per chunk, 1 TB gives $10^6 \div 64 = 15{,}625$ chunks (62,500 chunks at 16 MB)

Figure L2p1-3. A file is just a sequence of equal-size chunks (the last one may be smaller).

Tiny example: a 1 TB file (10^6 MB) in 64 MB chunks is $10^6 \div 64$ = **15,625 chunks**; in 16 MB chunks it is $10^6 \div 16$ = 62,500 chunks. (With binary units, 1 TB = 1,048,576 MB, you get 16,384 and 65,536; the order of magnitude is

the same.) Different chunks of the same file normally land on different servers, so many servers can read the file at once, which is exactly the striping of Slide 8.

Beyond the slides: chunks are large (not 4 KB) so that the master's list of chunks stays small, and so a program can read a chunk with one long sequential read instead of many slow disk seeks.

Replication (Slide 20)

Lecture 2 · Slide 20

Slide 20 adds the rule that makes the system reliable: **each chunk is replicated on different machines, usually 2× or 3×**, which **ensures persistence and availability**. The slide also says: **try to keep replicas in different racks**.

> **DEFINITION: REPLICATION FACTOR**
> The **replication factor** is the number of copies of each chunk (2× or 3×). A **replica** is one of the copies.

Why different racks? All 16–64 nodes of a rack share one switch (and often one power feed). If that switch fails, the whole rack disappears at once. If every copy of a chunk lived in that rack, the chunk would be unreachable. Replicas in different racks survive a rack failure.

- **Cost:** 3× replication stores 3× the raw data. A 1 TB file occupies 3 TB (46,875 chunk copies at 64 MB).
- **Best effort:** "try to" means the rack rule is a goal, not a guarantee. "2× or 3×" is a typical, configurable range.

> **BEYOND THE SLIDES**
> Why replication works. Suppose, simplifying, that a chunk's copies sit on random machines and 1,000 of 1,000,000 machines die in a day (Slide 12). One machine dies with probability $1{,}000 \div 1{,}000{,}000 = 10^{-3}$. A chunk with 2 copies loses both with probability about $(10^{-3})^2 = 10^{-6}$; with 3 copies about 10^{-9}. Over the 15,625 chunks of the 1 TB file that is about 1.6 % per day at 2× ($15{,}625 \times 10^{-6}$) but only 0.0016 % at 3× ($15{,}625 \times 10^{-9}$), *if no lost copy were ever rebuilt*. Re-creating lost copies quickly (next sections) is what keeps the real risk tiny. This is a simplified independent-failure model, not from the slides.

A file placed on the cluster (Slides 21–22)

Lecture 2 · Slides 21–22

Slide 21: **File 1 (named C) is divided into six chunks C0–C5**. (The letter C is the file's name and the subscript is the chunk index.) The picture scatters the chunks over chunk servers, one copy each: server 1 holds C0, C1, C5, C2; server 2 holds C3; server 3 holds C4; server N is empty. They need not be in order or evenly spread.

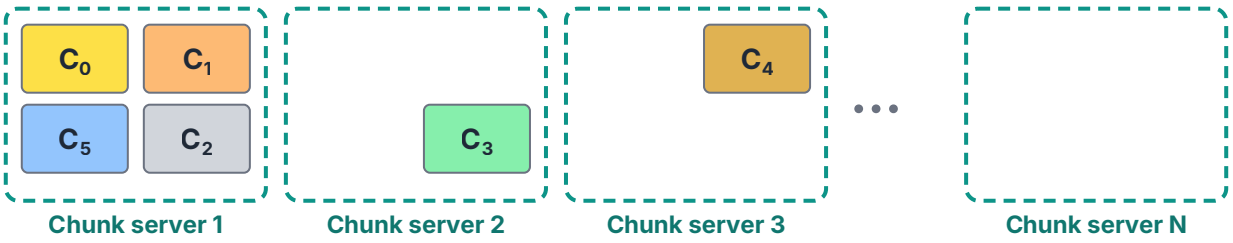


Figure L2p1-4. Slide 21 redrawn: File 1 (C) spread over the chunk servers, ONE copy of each chunk. Fragile: losing chunk server 1 loses C0, C1, C2 and C5.

This is **not safe**: lose server 1 and C0, C1, C2 and C5 are gone. Slide 22 repeats the picture and adds the fix: **"You need copies of these chunks (e.g., C1). Copies are never on the same chunk server."** The replicated layout (2×) is:

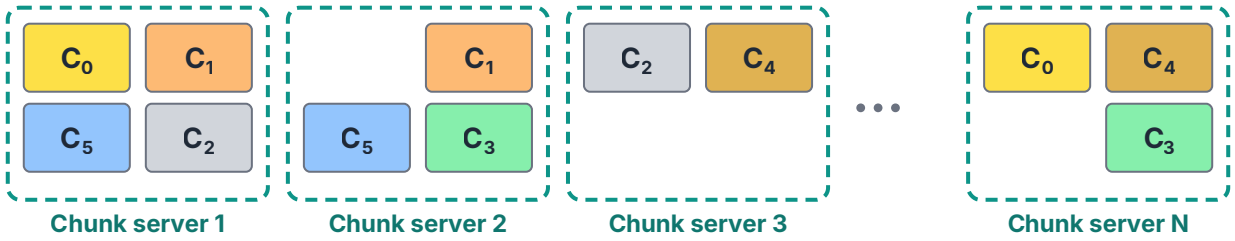


Figure L2p1-5. Slide 22 redrawn: every chunk now has exactly two copies (12 boxes = 6 chunks × 2), and no server holds two copies of the same chunk. Servers between 3 and N are hidden behind the "…".

Chunk	C0	C1	C2	C3	C4	C5
Copies on servers	1, N	1, 2	1, 3	2, N	3, N	1, 2

Check: every chunk has exactly two copies, 6 × 2 = 12 boxes (4 on server 1 + 3 on server 2 + 2 on server 3 + 3 on server N = 12), and no server holds two copies of the same chunk.

KEY IDEA: NEVER THE SAME SERVER

Two copies on the same machine would die together and give no protection. The stronger wish of Slide 20 (different *racks*) is a further layer: servers 1 and 2 above both hold C1 *and* C5, so if they shared a rack, a rack failure would destroy both chunks. The figure shows only the machine-level rule; it does not show racks.

SLIDE ERRATUM (TYPOS ONLY)

Slide 22 reads "copies of this chunks (e.g, C1)"; it should be "these chunks (e.g., C1)". The labels C3 and C4 are faint and identified only by colour; the placement above was read from the figure and cross-checked by counting boxes.

Seamless recovery from failure (Slide 23)

Lecture 2 · Slide 23

Slide 23 calls this a **reliable distributed file system** with **seamless recovery from disk or machine failure**. "Seamless" means the application using the file does not notice and does not need special code. Here is the mechanism, using Figure L2p1-5:

1. **Failure.** Chunk server 1 crashes. Its four chunks C0, C1, C2, C5 each lose one copy.
2. **Reads continue.** Each lost chunk still has a second copy elsewhere (C0 on N, C1 and C5 on server 2, C2 on server 3), so a reader is simply directed to a surviving copy. Nothing is unavailable.
3. **Re-replication.** The system notices that these chunks are down to one copy and makes a new copy of each on a server that does not already hold it, restoring 2×.
4. **Back to normal.** Every chunk again has two copies on two different servers.

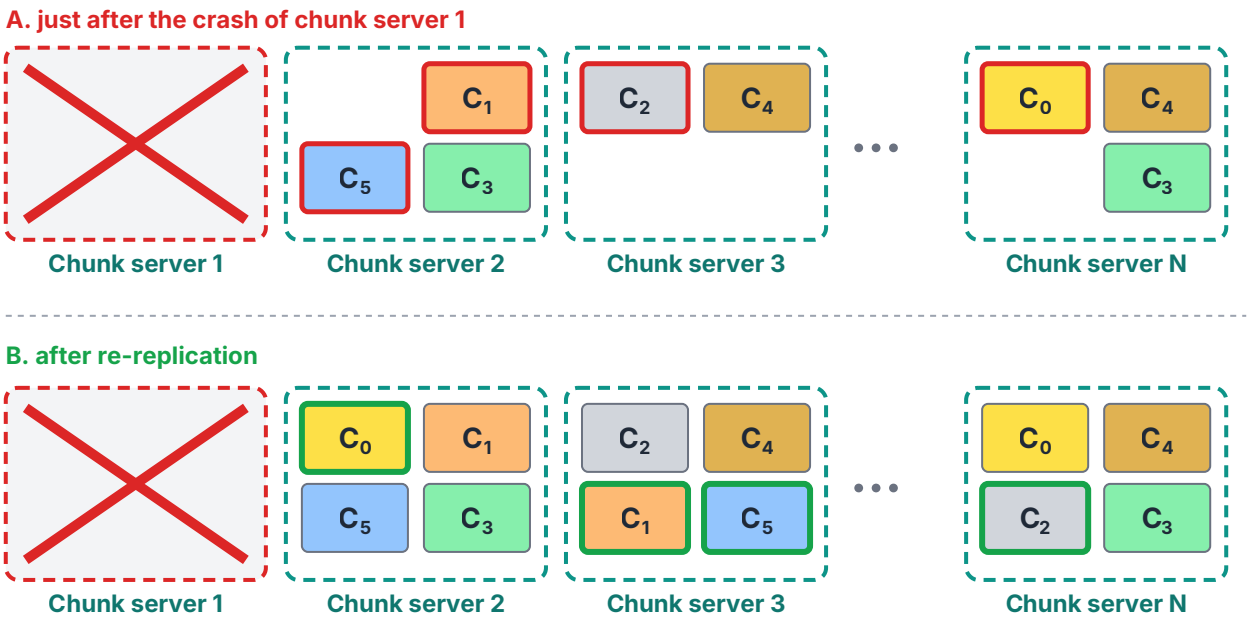


Figure L2p1-6. "Seamless recovery" (Slide 23) on the layout of Slide 22. No chunk is lost when server 1 dies; the missing copies are re-created elsewhere.

(Beyond the slides: the master node of the next section is the component that notices the failure, because it knows where every chunk is. The slides do not show this step; they only promise the outcome.)

> **EXAM TIP**
> If asked *how the DFS tolerates a machine failure*, write the chain: chunks are replicated on different machines (2×/3×, different racks if possible) → data stays readable from another replica → the system re-creates the lost copies → no data loss and no interruption. Without the re-creation step, repeated failures would eventually destroy all copies.

Computation moves to the data (Slide 23)

Lecture 2 · Slide 23

The lower half of Slide 23 carries the second Map-Reduce idea in two banners: **"Bring computation directly to the data!"** and **"Chunk servers also serve as compute servers."** The same machines that store the chunks also run the computation, so a task can run on a server that already holds its chunk and read from its local disk with no network transfer. This is called **data locality**.

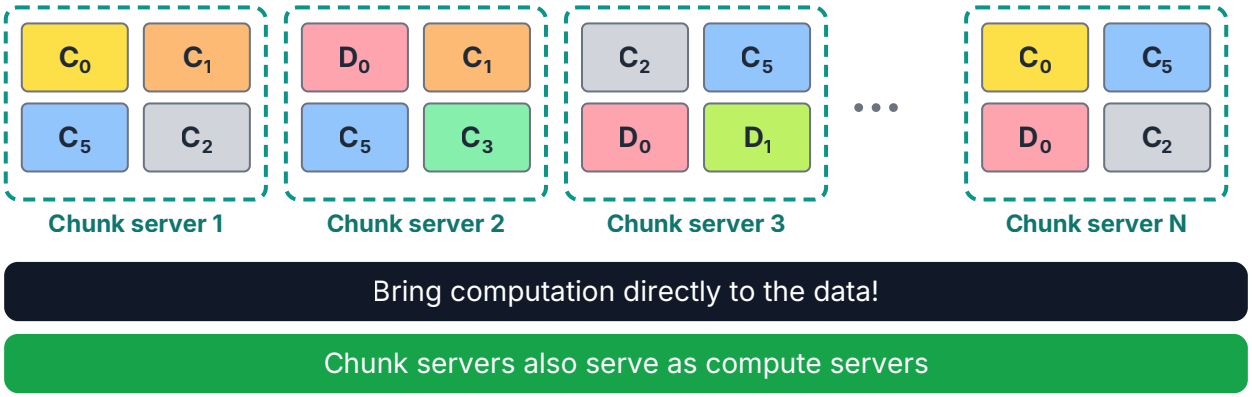


Figure L2p1-8. Slide 23 redrawn: two files (C and D) share the same cluster, and every chunk server can run tasks on the chunks it stores. Counts on the slide are an illustration (e.g. C3 and D1 appear once because their other copies sit on the hidden servers).

The figure also shows that **one cluster holds many files**: the slide adds a second file **D** divided into two chunks **D0 and D1** (pink and light green) next to file C.

COMMON MISTAKE

Do not read replica counts off Slide 23. It is an illustration, not a consistent 2×/3× layout: C3 and D1 appear once, C5 four times, C4 not at all. The servers between 3 and N are hidden behind the "…" and may hold the missing copies. Colours also differ from Slides 21–22 (C3 is bright green here, dark green there).

If a computation cannot run on a node holding the chunk, the next best choice is a node in the same rack (link at 1 Gbps), and the worst choice is a node in another rack (shared 2–10 Gbps backbone); see the distance table in the cluster section. Beyond the slides: the Map-Reduce scheduler uses exactly this preference order when it assigns tasks.

Master node and client library (Slide 24)

Lecture 2 · Slide 24

Someone must remember where every chunk is. Slide 24 introduces two components:

- **Master node** (in the Google File System; a.k.a. **Name Node** in Hadoop's HDFS). It **stores metadata about where files are stored** and **might be replicated**.
- **Client library for file access.** It **talks to the master to find chunk servers**, then **connects directly to the chunk servers to access data** without going through the master.

DEFINITION: METADATA

Metadata is "data about data": which chunks make up file C and which chunk servers hold each chunk's copies. The master stores this catalogue; it does **not** store the file's bytes.

How a read works, step by step

Lecture 2 · Slide 24

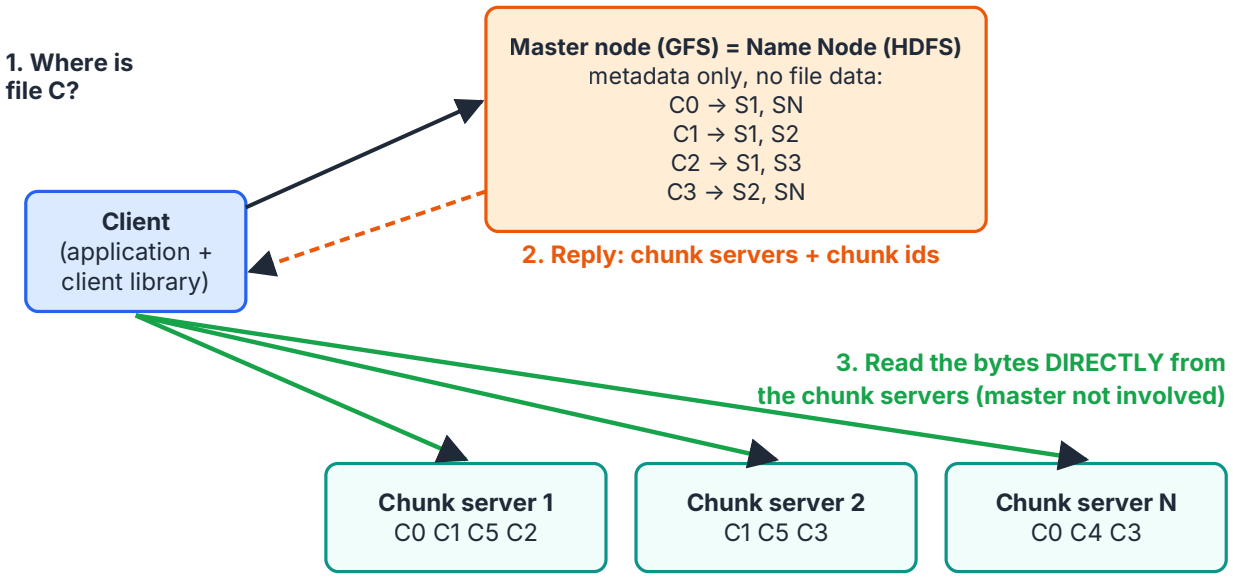


Figure L2p1-7. How a read works (Slide 24). The master answers a tiny metadata question once; the large data flows straight between client and chunk servers.

1. The application asks the client library to open file C.
2. The library asks the **master**: "where are the chunks of C?"
3. The master replies with, for each chunk, the list of chunk servers holding a copy (a small message).
4. The library connects **directly** to a chunk server for each chunk and reads the bytes. The master is no longer involved, so it never becomes a bottleneck for the terabytes of data traffic.

BEGINNER INTUITION

The master is the library catalogue; the chunk servers are the shelves. You ask the catalogue once ("which shelf holds chapter 3?"), then walk to the shelf and read yourself. The librarian never carries the books.

COMMON MISTAKE

"Without master node" on the slide means the *data transfer* bypasses the master, not that the master is never contacted. Also, "might be replicated" is vague: a single, un-replicated master would be a single point of failure, which is why it may have standby copies. Beyond the slides: chunk servers report the chunks they hold to the master, which is how it learns locations.

Role	GFS (Google) term	HDFS (Hadoop) term	What it does
Catalogue of locations	Master node	Name Node	Stores metadata only; may be replicated (Slide 24)
Machine storing data	Chunk server	Data Node (beyond the slides)	Stores chunks on its disk; also runs computation
Piece of a file	Chunk (16–64 MB)	Block	Replicated 2×–3× on different servers (different racks if possible)
Application interface	Client library		Asks the master, then reads/writes chunk servers directly

Closing the storage story: issue, idea, solution

Lecture 2 · Slide 25

Slide 25 ("Idea and Solution") ties it together in three beats:

- **Issue:** copying data over a network takes time (the 10 TB ≈ 1 day of Slide 14).
- **Idea:** bring computation close to the data (Slide 23: chunk servers are compute servers).
- **Map-Reduce addresses these problems.** It is Google's computational/data-manipulation model, an elegant way to work with big data, and a **programming model**: you supply the two functions Map and Reduce and the system runs them on the cluster.

The slide does not yet define Map or Reduce; the next slides do, starting with the word-count warm-up. Beyond the slides: Map-Reduce was introduced by Google (Dean and Ghemawat, 2004) and Hadoop MapReduce is its open-source implementation. Note the slight circularity of the slide's wording ("Programming model → Map-Reduce"): Map-Reduce *is* the programming model.

QUICK RECAP

- One node reading 200 TB at 50 MB/s needs 4×10^6 s ≈ 46 days; 1,000 discs in parallel (ideal case) need 4,000 s.
- Standard architecture: a cluster of commodity nodes (CPU + memory + disk), 16–64 per rack, 1 Gbps in a rack, 2–10 Gbps backbone between racks; ~1M machines at Google.
- Challenge 1: failures are daily routine (1,000 machines a day at 1M). Challenge 2: network bottleneck (10 TB ≈ 1 day at 1 Gbps). Challenge 3: distributed programming is hard.
- Three ideas: store data redundantly, move computation close to data, simple programming model.
- DFS (GFS/HDFS): huge files, rare in-place updates, reads and appends; chunks of 16–64 MB; replicated 2×–3× on different servers and, if possible, racks.
- Chunk servers also compute (data locality). The master/Name Node holds only metadata; clients ask it once, then talk directly to chunk servers.

PRACTICE: CHECK YOURSELF (SELF-MADE, NOT FROM THE COURSE)

1. Why is a replica never placed on the same chunk server as another copy of the same chunk? *(Both would die together; Slide 22.)*
2. A 640 GB file uses 64 MB chunks and 3× replication. How many chunks and how many chunk copies? *(640,000 MB ÷ 64 = 10,000 chunks; × 3 = 30,000 copies; 1.92 TB of raw storage.)*
3. Does the master node ever see the file's bytes during a read? *(No: metadata only; data flows between client and chunk servers.)*

The warm-up task: counting words

Lecture 2 · Slide 26

Slides 1–25 explained *why* big data needs many machines and *where* the data lives (a distributed file system, DFS, with chunks replicated on chunk servers). Slide 26 now asks *how* to compute on it. The lecture uses one problem to teach the whole programming model:

WARM-UP TASK (SLIDE 26)

We have a huge text document. Count the number of times each **distinct word** appears in the file. This is **word count**, the "hello world" of Map-Reduce.

Slide 26 lists two sample applications that have exactly the same shape:

- **Term search statistics:** count how often each search term was queried (the "word" is a query term).
- **Web-server logs:** count the hits per URL to find the popular URLs (the "word" is a URL).

BEGINNER INTUITION

On one machine this is easy: read the words one by one and keep a tally table (`the → 1204, big → 87, …`). The difficulty is purely one of scale: the file is stored as chunks on hundreds of machines, no single machine can read it all in reasonable time (Slides 5–8), and machines fail (Slides 12–13). Map-Reduce is a way to describe this tally so that the system can run it on all the chunks at the same time.

Slide 26 only *poses* the task. The solution is built step by step on Slides 27–42; try to predict it as you read. The complete practice problem is Quiz L2-Q3, placed after Slide 42.

Map-Reduce overview: four stages

Lecture 2 · Slides 27–28

Every Map-Reduce job, whatever the problem, follows the same four stages (Slide 27):

1. **Map.** Scan the input file *record-at-a-time*. For each record, extract (output) "something you care about" as **(key, value) pairs**. A **record** is one unit of input, for example one line of a text file or one row of a log.
2. **Group by key (sort and shuffle).** Collect all pairs that have the same key and bring them together.
3. **Reduce.** For each key, *aggregate, summarise, filter or transform* the values collected for it.
4. **Write the result** (to the distributed file system, as Slide 55 later states).

DEFINITIONS

- **Key-value pair (k, v):** a two-part item. The **key** says what the item is about (a word, a host name); the **value** is the data attached to it (a count, a size). Everything that flows through Map-Reduce is a key-value pair.
- **Shuffle:** moving the pairs across the network so that equal keys end up on the same machine. **Sort:** ordering the pairs by key so equal keys sit next to each other.

Slide 28 is a one-sentence take-away in a black box: **"Outline stays the same, Map and Reduce change to fit the problem."** The four-stage skeleton is fixed and supplied by the framework; the only things you change from problem to problem are the two functions you plug in.

	Programmer provides	Framework (the "environment") provides
Map	**the Map function: what to extract from each record**	reading the chunks, running many Map calls in parallel
Group by key	nothing	**partitioning, shuffling over the network, sorting and grouping**
Reduce	**the Reduce function: how to combine the values of one key**	running many Reduce calls in parallel
Write the result	nothing	**storing the output in the DFS (Slide 55)**
Everything else	nothing	**scheduling, communication, failure handling (Slide 53, below)**

KEY IDEA

Think of Map-Reduce as an assembly line (the fixed skeleton) with two interchangeable tools (Map and Reduce) that you clip in. The programmer writes **only** Map and Reduce; the grouping step is **not** written by the programmer.

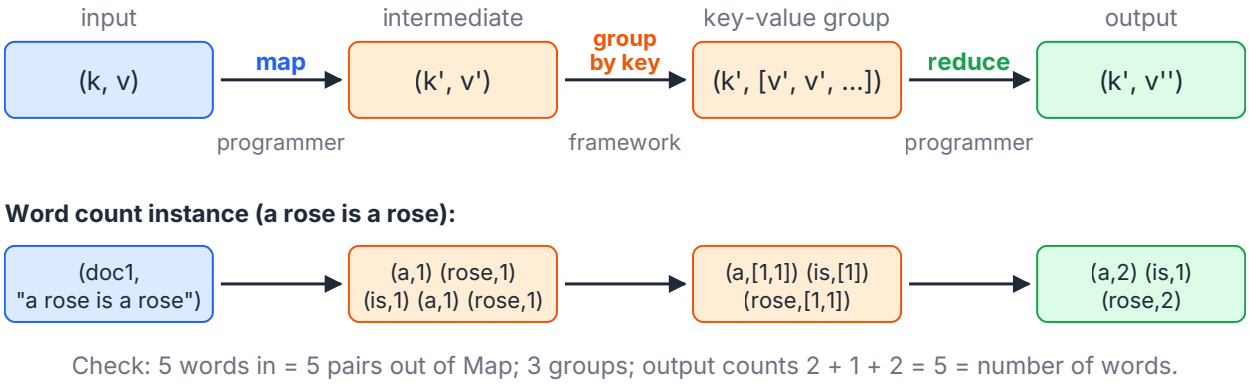
EXAM TIP

"Who writes what?" is a classic short question. Answer: the programmer supplies Map and Reduce (and names the input files); the framework does the group-by-key (sort and shuffle) and everything about machines. The slides state this explicitly through the "Provided by the programmer" captions on Slides 36–38, which sit over Map and Reduce but not over Group by key.

The data model: Map step and Reduce step

Lecture 2 · Slides 29–35

Slides 29–35 draw the same pipeline abstractly, with shapes instead of words. Shape and colour are only a drawing convention: **colour = key identity** (a purple key stays purple from column to column) and **shape = type** (triangle/rectangle for input keys/values, diamond/parallelogram for intermediate, diamond/octagon for output). Figure L2-7 redraws the whole sequence with the types written out and a word-count instance underneath.



Check: 5 words in = 5 pairs out of Map; 3 groups; output counts 2 + 1 + 2 = 5 = number of words.

Figure L2-7 — Types in Map-Reduce (Slides 29–35). The practice sentence is invented for this guide; it is not on the slides.

The Map step

Lecture 2 · Slides 29–31

- **Slide 29: input key-value pairs.** The input is a (typically huge) set of pairs. What `k` and `v` mean depends on the application, for example (document name, document text) or (position in file, one line of text).
- **Slide 30: the first map application.** The function **map** is applied to *one* input pair and produces **intermediate key-value pairs**. On the slide, one input pair becomes two intermediate pairs. Map may emit zero, one or many pairs; it is **not** "one in, one out". The new keys are chosen by the programmer and need not resemble

the input key (for word count: input key = document name, intermediate key = a word). "Intermediate" simply means "not the final answer yet".

- **Slide 31: map is applied to every input pair.** The intermediate collection is the *union* of what all the map calls emitted. It is still unorganised: the same key may appear many times, from different inputs.

> **KEY IDEA: INDEPENDENCE GIVES PARALLELISM**
> Each map call looks at its own input pair only. So the calls can run on different machines at the same time, and if one fails it can simply be run again without affecting the others. This independence is the source of Map-Reduce's parallelism and of its easy failure recovery (Slide 57, below).

The Reduce step: group by key, then reduce

Lecture 2 · Slides 32–35

- **Slide 32: group by key.** Starting from the intermediate pairs, the next operation brings together all pairs that share a key. It is performed by the system. (The slide title says "Reduce Step" because the lecture treats grouping as the front part of the reduce phase.)
- **Slide 33: key-value groups.** The flat list becomes a list of `(key, list of all its values)`. Each distinct key appears **exactly once**, with a variable-length list: three values for the purple key, two for ochre, one for yellow. The order of values inside a group carries no meaning. All values of one key must arrive at *one* machine, which is why the shuffle is expensive.
- **Slide 34: first reduce. Reduce** is applied once per key group. It receives the key and all its values and emits an **output key-value pair**, typically (key, one combined value) such as a sum, count or maximum. The shape changes from parallelogram to octagon to signal that the output value is a different kind of value.
- **Slide 35: the complete picture.** Intermediate pairs → (group by key) → key-value groups → (reduce, once per key) → output pairs. The sizes shrink to the right: 3 values → 1, 2 values → 1, 1 value → 1.

> **SLIDE ERRATUM (MINOR, SCHEMATIC FIGURES)**
> Slides 33 and 35 are drawings, not literal traces: the purple group has three values although only one purple pair is visible in the left column (the rest hide in the "..."), and the light-teal key of the intermediate column has no group or output. The yellow group also has an output pair but no drawn "reduce" arrow. Do not try to match the figure pair for pair; read it as a picture of the types.

> **QUICK RECAP (SLIDES 29–35)**
> - Input (k, v) → **map** (once per input pair, zero or more outputs) → intermediate (k', v').
> - Intermediate pairs → **group by key** (by the system) → one group (k', list of v') per distinct key.
> - Each group → **reduce** (once per key) → output pair (k', v'').

Word count, step by step

Lecture 2 · Slides 36–40

Slides 36–40 build one picture in five steps, using a "Big document" that begins "The crew of the space shuttle Endeavor recently returned to Earth as ambassadors, harbingers of a new era of space exploration ...". Three coloured lists appear from left to right: teal (Map output), amber (after Group by key), green (after Reduce).

Slide	What is added	What it shows
36	**Map** box, caption "Provided by the programmer"; teal list `(The,1) (crew, 1) (of,1) (the,1) (space,1) (shuttle,1) (Endeavor,1) (recently,1) …`	For *every word occurrence* Map emits (word, 1): "I saw this word once." Map does not count anything; the counting happens later. The eight pairs are exactly the first eight words of the document.

37	**Group by key** box ("Collect all pairs with same key"), amber list `(crew,1) (crew,1) (space,1) (the,1) (the,1) (the,1) (shuttle,1) (recently,1) …`	Equal keys become adjacent. **No** "Provided by the programmer" caption: the system does this.
38	**Reduce** box, second "Provided by the programmer" caption; green list `(crew,2) (space,1) (the,3) (shuttle,1) (recently,1) …`	Reduce **sums** the 1s of each key: two 1s for crew give 2, three 1s for the give 3. Output = (word, number of occurrences).
39	Three pairs of red lines cutting the document *and* the Map output into four pieces	The big document is cut into pieces (chunks); each piece produces its own part of the Map output. Separate machines (map tasks) read their pieces at the same time. This is where parallelism enters. (The slide gives no words for the red lines; this is the natural reading.)
40	Red lines around the three `(the,1)` pairs and the one `(the,3)`; big arrow "Only sequential reads"	One key group (`the`) is handled as a unit and can be reduced independently, possibly on another machine. The whole pipeline needs only **sequential reads**.

Why "only sequential reads" matters

Lecture 2 · Slide 40

A **sequential read** streams through a file from start to end; **random access** jumps around it. Disks are fast at the first and slow at the second (every jump costs a seek). Map scans its chunk from start to end, and the sort-and-group works on whole files of pairs, so nothing needs to leap around. **Beyond the slides:** that is the standard reasoning; the slide only states the take-away. It fits the lecture's earlier assumption of a disk read speed of 30–50 MB/s (Slides 5–8): the job should always read at that streaming speed.

SLIDE ERRATUM (SLIDES 36–38: THE LISTS ARE SCHEMATIC)

- The Map list contains both `(The,1)` and `(the,1)` (capital T), but the grouped and reduced lists show only `the` with three pairs. Either the figure is schematic or case is ignored. The pseudo-code on Slides 41–42 does **no lower-casing**, so under that code `The` and `the` are two different keys. Remember: a key is compared exactly, so normalising case is the Map function's job.
- The grouped list omits `(of,1)` and `(Endeavor,1)` from the Map list, its keys are in no particular order, and counts such as `(crew,2)` and `(the,3)` cannot be derived from the visible excerpt alone (it has `crew` once). The "…" hides the rest of the document. Treat the three lists as an illustrative selection, not an exact trace.
- Slide 38 says Reduce "collects all values belonging to the key and outputs". Strictly, the *collecting* was done by Group by key; Reduce *combines* the collected values (here by addition).
- Slide 36 says "Read input and produces" (should be "produce"); the document text contains the typo "man/mache" (probably "man/machine"). Harmless.

The word-count pseudo-code

Lecture 2 · Slides 41–42

NOT EXAMINED — CODE (UNDERSTAND, DON'T MEMORISE)

Slides 41–42 give pseudo-code for word count. Explained in words, no code to learn:

- **Map** receives one input pair: the document *name* (key) and the document *text* (value). It walks over every word `w` of the text and, for each occurrence, **emits** the pair `(w, 1)`. The key (document name) is not used. This produces exactly the teal list of Slides 36–40.
- **Reduce** receives one word (the key) and an iterator over all the counts collected for it. It starts a running total at 0, adds each count to it, and finally emits `(word, total)` once, after the loop. For key `the` with counts `[1,1,1]` the total is 3, so it emits `(the, 3)`, as on Slide 38.
- The whole program is these two short functions. Grouping and everything else is done by the framework.

Not specified by the pseudo-code: how text is split into words, whether punctuation is removed and whether case is normalised. On Slide 42 the comment says "value: an iterator over counts" while the parameter is named `values`; a typo.

Try it yourself first: cover the solution, decide the key and value for word count, then trace the pipeline on the practice input with two mappers and two reducers.

QUICK MATHS: MOD, HASH, STRAGGLER, PARTITION (USED BEFORE THEY ARE EXPLAINED)

$x \bmod R$ is the remainder after dividing x by R (7 mod 3 = 1). A **hash function** turns a key into a big number, so $\text{hash}(\text{key}) \bmod R$ is always one of 0, ..., R−1, i.e. a reduce-task number. A **partition function** is exactly this key-to-reduce-task rule (details in the partition-function section of Part 3). A **straggler** is a task that runs unusually slowly (defined in the backup-tasks section of Part 3).

Quiz L2-Q3 Word count with Map-Reduce (the warm-up task) Lecture 2 · Slides 26–42

QUESTION (AS ON THE SLIDES)

Warm-up task (Slide 26). "We have a huge text document. Count the number of times each distinct word appears in the file." Sample applications on the slide: term search statistics; analysing web server logs to find popular URLs.

The slides' answer (Slides 27–40). The Map-Reduce outline **Map** (scan the input file record-at-a-time; output something you care about from each record, as keys and values) → **Group by key** (Sort and Shuffle) → **Reduce** (aggregate, summarise, filter or transform) → **write the result**, run on a text excerpt ("*The crew of the space shuttle Endeavor recently returned to Earth …*") with these three lists:

After Map (Slide 36)	After Group by key (Slide 37)	After Reduce (Slide 38)
`(The,1) (crew,1) (of,1) (the,1) (space,1) (shuttle,1) (Endeavor,1) (recently,1) …`	`(crew,1) (crew,1) (space,1) (the,1) (the,1) (the,1) (shuttle,1) (recently,1) …`	`(crew,2) (space,1) (the,3) (shuttle,1) (recently,1) …`

Slides 41–42 then give the Map and Reduce pseudo-code, and Slide 40 adds red lines and a red arrow "Only sequential reads".

The slides pose no numbered question, so this block turns Slides 26–42 into five exam-style parts. **Practice:** the 3-line input in (b) is invented for this guide (it is not on the slides), and so are the task numbers and the toy partition rule stated in (b).

(a) State precisely what the **programmer** provides and what the **framework** does. Which **key** and which **value** do you choose for word count, and why?

(b) Trace the whole pipeline Map → Group by key (sort & shuffle) → Reduce on this *practice* input, with **2 mappers** and **2 reducers**:

Line	Text (the value of the input pair)	Goes to
1	`The crew of the space shuttle`	Map task 0
2	`crew recently returned to Earth`	Map task 1
3	`the Earth is blue`	Map task 1

Set-up for this practice: Map writes every word in lower case; there are $R = 2$ reduce tasks; if no partition rule is given, use the toy rule "reduce task = (number of letters in the key) mod 2".

(c) Reproduce the slide's example counts `(crew,2) (space,1) (the,3) (shuttle,1) (recently,1)` from the grouped pairs of Slides 37–38 and show where each one comes from.

(d) Explain the slide pseudo-code (Slides 41–42) in words.

(e) What does the red arrow "Only sequential reads" on Slide 40 mean, and why is it good?

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Counting votes in a huge election. A mountain of ballot papers must be tallied by candidate. Many volunteers each take a small stack. For *every* ballot they read, they write one slip "(candidate, 1)" — they do not add anything up yet. That is **Map**. Then the post office sorts all slips of all volunteers into piles, one pile per candidate, and delivers every pile to a counting desk; the same candidate's slips always land at the same desk. That is **Group by key** (nobody on the staff has to write this part: it is the framework's job). Finally each desk adds up the 1s in each pile and writes "(candidate, total)". That is **Reduce**. You only describe how to make a slip (Map) and how to add up a pile (Reduce).

KEY IDEA

The four-stage outline (Map, Group by key, Reduce, Write the result) *never changes*; only Map and Reduce change to fit the problem (Slide 28). For word count: the **key** is the word, the **value** is 1, and Reduce **adds**. Every item is a (key, value) pair. "Same key ⇒ same place": that single rule is what makes the final counts correct. The same outline solves the sample applications of Slide 26: to find popular URLs in web server logs, Map outputs `(URL,1)` for every log line and Reduce is unchanged; for term search statistics, Map outputs `(search term,1)`.

Part (a) — who provides what (Slides 27, 28, 36–38, 44, 53–55, 66–67)

1. **Formula / idea — the four stages (Slide 27):** Map (scan the input record-at-a-time; output something you care about from each record, as keys and values) → Group by key (Sort and Shuffle) → Reduce (aggregate, summarise, filter or transform) → write the result. *Why two functions are enough:* Map says "what do I extract from one record?" and Reduce says "what do I do with everything that shares a key?". Everything else is routine plumbing that is identical for every job.
2. **Choose the key and the value (the one design decision of this job).** To know how often a word occurs, *all* its occurrences must be brought to one place and added up. Group by key brings together everything that shares a key, so the key must be the **word**. Each time a word is seen its tally goes up by exactly one, so the value is **1**; adding the 1s later gives the total. This is exactly what Slide 36 shows: `(crew,1)`, `(space,1)`, …
3. **The shape of the data at each stage (Slides 29–35).** Everything is a (key, value) pair. Input pairs go through *map* and become intermediate pairs (one input pair can produce any number of them, Slides 30–31); *group by key* gathers all intermediate values that share a key into one group (Slides 32–33); *reduce* turns each group into one output pair (Slides 34–35).

Stage	Slides	What it holds	Word count
Input key-value pairs	29	(k, v), one per record	(line number, text of the line)
Intermediate key-value pairs	30–31	(k′, v′), many per input pair	`(the,1)` — one for each word occurrence
Key-value groups	32–33	(k′, v′ v′ v′ …): all values of one key	`(the,[1,1,1])`
Output key-value pairs	34–35	(k′, v′′), one per group	`(the,3)`

4. **The two programmer-written functions, in the slides' notation (Slide 44):**

FORMULA

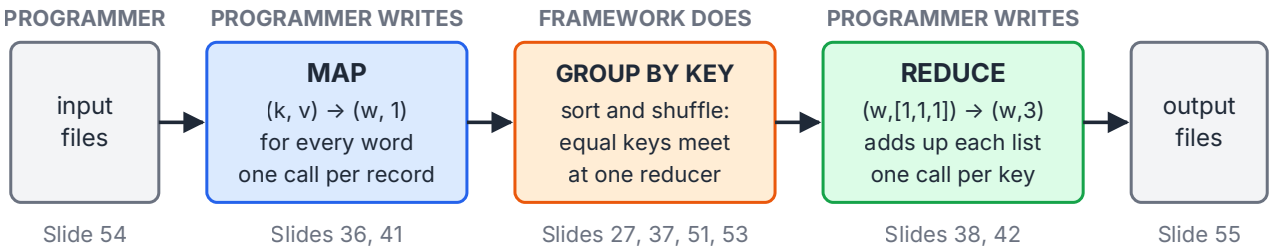
Map$(k, v) \rightarrow <k', v'>*$ (one input pair in; a set of zero or more key–value pairs out; one Map call for every input pair)

Reduce$(k', <v'>*) \rightarrow k', v''$ (a key and the list of all its values in; the key and one result out; one Reduce call per unique key k')

   For word count: Map(line id, line text) → a pair `(word, 1)` *for every word occurrence*; Reduce(word, list of 1s) → `(word, sum of the list)`. (Slide 44's example: "value is a single line in the file", so one Map call per line. Slide 44 also says all values with the same key are "reduced together and processed in v' order"; for addition the order is irrelevant, Slide 62.)
5. **Division of labour — table:**

Piece	Who	What it does (slide wording) and the word-count meaning	Slides
Map	**programmer**	"Read input and produces a set of key-value pairs." Word count: for each word w in the record, output `(w, 1)`.	27, 36, 41
Group by key	framework	"Collect all pairs with same key" — Sort and Shuffle (also named "Hash merge, Shuffle, Sort, Partition"). Not labelled "Provided by the programmer" on the slides.	27, 37, 51
Reduce	**programmer**	"Collect all values belonging to the key and output." Word count: add up the 1s of one word and output `(word, total)`.	27, 38, 42
Input files	**programmer**	"Programmer specifies: Map and Reduce and input files."	54
Everything else	framework	"Map-Reduce environment takes care of": scheduling the program's execution across a set of machines; managing required inter-machine communication; partitioning the data; performing the group by key step; handling machine failures.	53
Optional extras	programmer	A *combiner* that pre-adds on the mapper (Slides 59–62); a custom partition function — the default is `hash(key) mod R` (Slides 66–67).	59–67

6. **Where the data lives (Slide 55):** the input and the final output are on the distributed file system (DFS); the scheduler tries to place map tasks "close" to the physical location of their input data (the Slide 25 idea: bring the computation to the data); intermediate pairs are kept on the *local* file system of the mapper's machine until the reducers fetch them.



The framework also: splits the input into chunks, schedules the tasks across machines, moves data between machines, and handles failures (Slide 53). Every stage works on (key, value) pairs.

Figure L2-Q3-1. Division of labour for word count: the programmer names the input files and writes only **Map** (blue) and **Reduce** (green); the framework supplies **Group by key** (orange) and everything else.

Part (b) — trace on the practice input (Slides 36–40, 51–54, 66)

THREE SET-UP DECISIONS (STATE THEM IN YOUR ANSWER)

1. Which words are "the same"? We let Map write each word in *lower case* (so "The" and "the" are one word) and our input has no punctuation (a real Map would also strip it). The slide's final `(the,3)` only works if "The" and "the" are merged (see the erratum below). **2. How many tasks?** The file is cut into 2 chunks (one DFS chunk per Map task is common, Slide 69), so there are 2 map tasks, and we use $R = 2$ reduce tasks. Real jobs use many more (Slide 69: far more map tasks than nodes, and usually R smaller than the number of map tasks); two of each just keeps the picture small. We number tasks from 0 as Slide 54 does. **3. Which reducer gets which key?** The real default is `hash(key) mod R` (Slide 66), which cannot be computed by hand, so we use a simple *toy rule* instead (Step 3).

1. **Step 1 — split the input (framework).** The framework partitions the data into chunks and hands one chunk to each Map task (Slide 53: "partitioning the data"; Slide 39 shows the document cut into pieces). Chunk 0 = line 1 (goes to Map task 0). Chunk 1 = lines 2 and 3 (goes to Map task 1). Chunk boundaries fall between records, so no line is cut in half. *Why:* each Map call needs only its own record, so chunks can be processed on different machines at the same time.
2. **Step 2 — Map (programmer's function; one call per line).** Rule: for every word occurrence w output `(lower-case w, 1)`. Map does **not** count: if "the" occurs twice in a line it simply outputs two `(the,1)` pairs.

Map task	Map call (input record)	Pairs output (in reading order)	#
0	line 1: "The crew of the space shuttle"	`(the,1) (crew,1) (of,1) (the,1) (space,1) (shuttle,1)`	6
1	line 2: "crew recently returned to Earth"	`(crew,1) (recently,1) (returned,1) (to,1) (earth,1)`	5
	line 3: "the Earth is blue"	`(the,1) (earth,1) (is,1) (blue,1)`	4
Total intermediate pairs = 6 + 5 + 4 (one per word occurrence; the input has 15 words)			**15**

There were 3 Map calls (one per input record, Slide 44) — 1 on Map task 0 and 2 on Map task 1.

3. **Step 3 — Group by key, part A: partition (framework).** Every pair must be sent to the reduce task that is responsible for its key, and the rule must give the *same* task for the same key, whichever mapper produced the pair (Slide 66: "records with the same intermediate key end up at the same worker"; Slide 52 draws a "Partitioning Function" at the end of every Map task). In this example we use the simple **toy rule** below.

> **FORMULA**
> Reduce task number = (number of letters in the key) mod R, with R = 2. (Any rule that looks only at the key would do; what matters is that the same key always gets the same number.)

Key	the	space	shuttle	earth	crew	of	recently	returned	to	is	blue
letters	3	5	7	5	4	2	8	8	2	2	4
mod 2 → task	1	1	1	1	0	0	0	0	0	0	0

Pairs on each route: Map 0 → Reduce 0: `crew, of` (2); Map 0 → Reduce 1: `the, the, space, shuttle` (4); Map 1 → Reduce 0: `crew, recently, returned, to, is, blue` (6); Map 1 → Reduce 1: `earth, the, earth` (3). Check: 2 + 4 + 6 + 3 = 15. Notice `crew` and `the` are each made by *both* mappers, yet each ends up on a single reducer. *Why must the rule look at the key only?* If the three `(the,1)` pairs could be split between two reducers (say 2 at one and 1 at the other), the output would contain `(the,2)` and `(the,1)` — two partial counts instead of `(the,3)`. With any key-only rule the final counts are the same; only the split of the words between the two output files changes.

4. **Step 4 — Group by key, part B: shuffle, then sort and group (framework).** *Shuffle* = copy the pairs across the network from the mappers' local disks to the reducers: every Map task sends to every Reduce task (the all-to-all arrows of Slide 54), and the master tells the reducers where each mapper's files are (Slide 56). Each reducer then *sorts* its pairs by key, which puts equal keys next to each other, and *groups* them into one list per key (Slide 52: "Sort and Group"). The result is the "key-value groups" of Slides 33–35:

Reduce task	Pairs received	After sort & group: (key, list of values)	#keys
0	8	`(blue,[1]) (crew,[1,1]) (is,[1]) (of,[1]) (recently,[1]) (returned,[1]) (to,[1])`	7
1	7	`(earth,[1,1]) (shuttle,[1]) (space,[1]) (the,[1,1,1])`	4

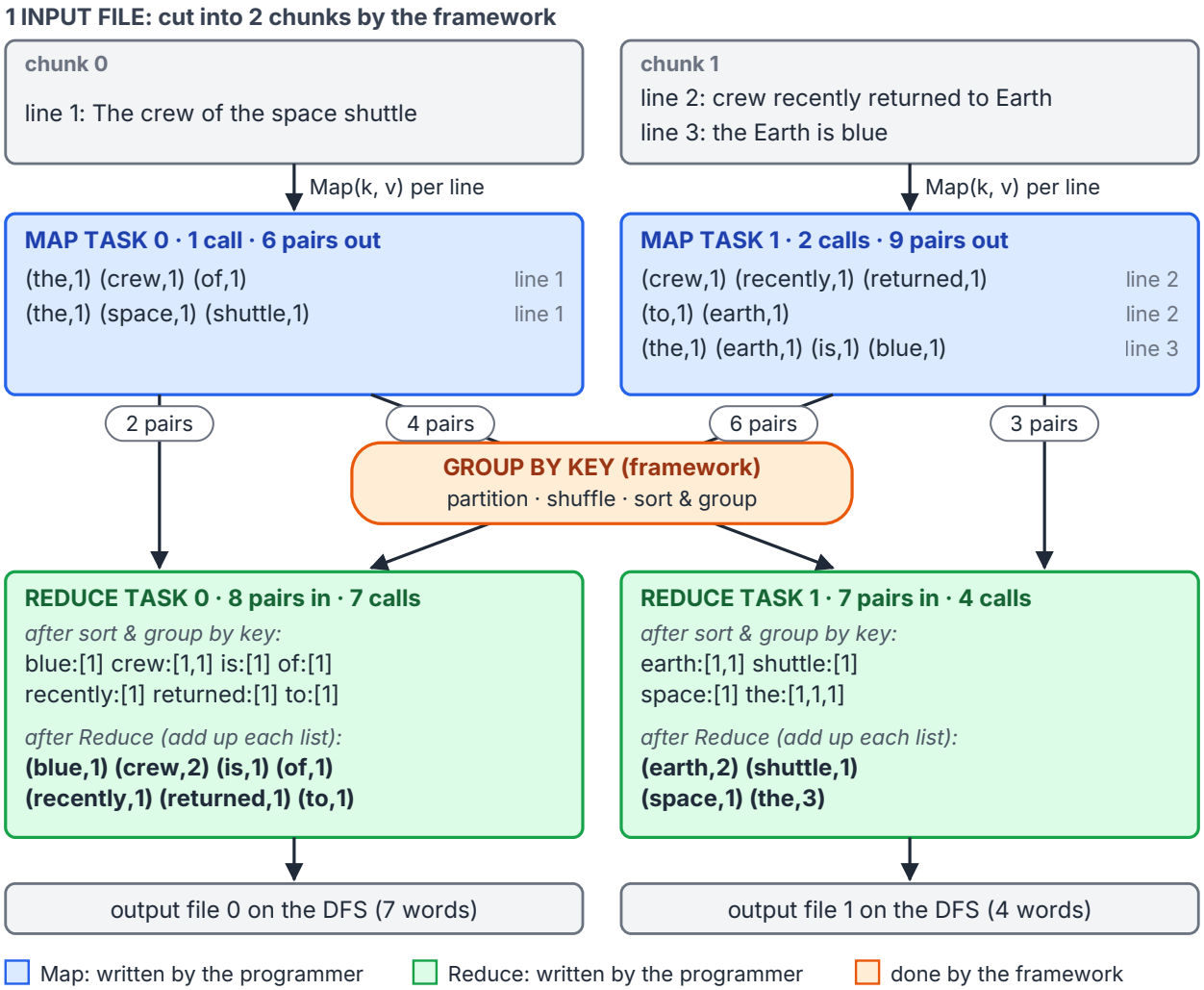
Within one reducer the keys come out in alphabetical order; there is *no* global order across the two reducers, because they hold different sets of words. Pairs received: 8 + 7 = 15, none lost.

5. **Step 5 — Reduce (programmer's function; one call per key).** Rule: result = sum of the values in the list; output `(key, result)`.

Reduce task	Reduce calls (key: sum)	Output pairs
0	`blue: 1 · crew: 1+1 · is: 1 · of: 1 · recently: 1 · returned: 1 · to: 1`	`(blue,1) (crew,2) (is,1) (of,1) (recently,1) (returned,1) (to,1)`
1	`earth: 1+1 · shuttle: 1 · space: 1 · the: 1+1+1`	`(earth,2) (shuttle,1) (space,1) (the,3)`

There are 7 + 4 = **11 Reduce calls**, one per distinct word (Slide 44: "one Reduce function call per unique key"), not one per pair (there are 15 pairs).

6. **Step 6 — write the result (Slides 27, 54, 55).** Each reduce task writes its own output file to the DFS ("Out 0" and "Out 1" on Slide 54): output file 0 holds 7 pairs, output file 1 holds 4 pairs. Together they are the answer. (Output is often the input of another Map-Reduce job, Slide 55.)
7. **Step 7 — sanity checks (quick marks-savers).** (i) pairs output by Map = words in the input: 15 = 15. (ii) the final counts add up to the same number: (1+2+1+1+1+1+1) + (2+1+1+3) = 8 + 7 = 15. (iii) Reduce calls = distinct words: 11. (iv) Map calls = input records: 3. (v) count by hand from the text: "the" occurs 2 + 0 + 1 = 3 times (lines 1, 2, 3), "crew" 2 (lines 1 and 2), "earth" 2 (lines 2 and 3), and the other eight words once each, which matches the Reduce output. If any check fails, a pair was lost or double-counted.



Toy partition rule: Reduce task = (letters in key) mod 2. Real default: hash(key) mod R (Slide 66).

Figure L2-Q3-2. The whole trace of part (b) in one picture (compare Slides 51–52 and 54). Blue = programmer's Map, orange = framework's group by key, green = programmer's Reduce. The numbers on the four arrows are the pairs that travel from each Map task to each Reduce task; they add up to 15. The two crossing arrows are the shuffle.

COMMON MISTAKE

- **Adding inside Map.** Writing `(the,2)` for a line that contains "the" twice is wrong for this Map: Map outputs one `(w,1)` per *occurrence* (Slide 41); adding is Reduce's job. (Pre-adding on the map side is a separate, optional optimisation, the combiner of Slides 59–62.)
- **Reduce is called once per distinct key**, not once per pair (Slide 44): 11 calls here, not 15.
- **One key, one reducer.** The final list is *per key*, so a word that appears in two lines or on two mappers must still appear only **once** in the output.
- **Mappers never talk to each other.** Intermediate data travels only from the map side to the reduce side, through the shuffle.
- **Case and punctuation.** "The" and "the" are two different keys unless Map lower-cases the word; decide, and say so in your answer.

Part (c) — reproducing the slide's counts (Slides 36–38)

1. **Formula / idea:** final count of a word = sum of the values in its group = number of times Map output that word (because every value is 1). $\mathrm{count}(w)$ = ∑ of the list for key w = number of `(w,1)` pairs.
2. **Read the Map box of Slide 36.** It is Map applied to the first words of the document, *"The crew of the space shuttle Endeavor recently"*: 8 words give 8 pairs, `(The,1) (crew,1) (of,1) (the,1) (space,1) (shuttle,1) (Endeavor,1) (recently,1)`; the dots "…." stand for the pairs of the rest of the document.
3. **Take the Group-by-key box of Slides 37–38 (8 grouped pairs) and apply Reduce to each group (Slide 38):**

Key	Grouped pairs (orange box)	Value list	Reduce computation	Output pair (green box)	Where in our trace (b)
`crew`	`(crew,1) (crew,1)`	[1, 1]	1 + 1 = 2	`(crew,2)` ✓	Reduce 0 (both mappers contribute)
`space`	`(space,1)`	[1]	1	`(space,1)` ✓	Reduce 1
`the`	`(the,1) (the,1) (the,1)`	[1, 1, 1]	1 + 1 + 1 = 3	`(the,3)` ✓	Reduce 1 (both mappers contribute)
`shuttle`	`(shuttle,1)`	[1]	1	`(shuttle,1)` ✓	Reduce 1
`recently`	`(recently,1)`	[1]	1	`(recently,1)` ✓	Reduce 0

   Output: `(crew,2) (space,1) (the,3) (shuttle,1) (recently,1) …` — exactly the slide. Check: 8 grouped pairs in, and 2 + 1 + 3 + 1 + 1 = 8 counted occurrences out.
4. **Why the practice input of (b) was chosen this way:** it contains "the" three times (twice in line 1, once in line 3) and "crew" twice (lines 1 and 2), and "space", "shuttle", "recently" once each, so the same five results appear in our trace; the extra words (`of, returned, to, earth, is, blue`) make the group-by-key step more interesting. The slide's "…" means more pairs exist in the real big document; we only see a sample of them.

SLIDE ERRATUM (MINOR — THE SLIDE FIGURES ARE ILLUSTRATIVE)

- **"The" vs "the".** Slide 36 lists `(The,1)` and `(the,1)` as two separate pairs, but Slides 37–38 then show only lower-case `(the,1)` pairs and the result `(the,3)`. With the pseudo-code of Slides 41–42 taken literally (each word *w* is used exactly as written, no case conversion) "The" and "the" are *different keys* and would be counted separately. **Correct version:** the slide's result is right only if Map first converts every word to lower case; that is what we did. If you want case-insensitive counts, lower-case the word inside Map and say so.
- **The numbers are schematic.** The text excerpt visible on Slide 36 already contains lower-case "the" five times, "The" twice (seven "the" in all, ignoring case), "crew" once and "space" at least twice ("space shuttle", "era of space exploration"), so `(the,3)` and `(crew,2)` cannot be counts of the visible text; also `(of,1)` and `(Endeavor,1)` disappear between the Map box and the Group box. Treat the slide's lists as a picture of the *pattern* (many `(w,1)` → groups → sums), not as numbers to re-derive from the photo of the document.

- **Order.** The groups on Slides 37–38 are in order of appearance (crew, space, the, shuttle, recently), not alphabetical. This does not matter: group by key only guarantees that *equal keys are together*.

Part (d) — the pseudo-code explained in words (Slides 41–42)

NOT EXAMINED — CODE (UNDERSTAND, DON'T MEMORISE)

Slide 41 shows the first function, Slide 42 adds the second (the code is the slide's; no code will be asked in the exam):

```
map(key, value):
// key: document name; value: text of the document
  for each word w in value:
     emit(w, 1)

reduce(key, values):
// key: a word; value: an iterator over counts
      result = 0
      for each count v in values:
            result += v
      emit(key, result)
```

In words — map: it is handed one input record (a document name, or in our trace a line number, as the key, and the text as the value). It looks at every word in the text, one after the other, and for each word it writes down the pair (word, 1) with `emit` ("emit" = hand one output pair to the framework). It keeps no tally. The input key (the document name) is not used. **reduce:** it is handed one word and an iterator (a one-by-one walk through the list) over all the counts the Map calls produced for that word. It starts a running total at 0, adds each count to it, and finally emits (word, total). Because Map only produces 1s, total = how many times the word occurred. The two `emit` lines are the only way these functions hand their results back to the framework; grouping is not in the code at all because the framework does it. (Slide 41 says the value is "text of the document", Slide 44 says "a single line in the file": both are fine, because Map is called once per input record, whatever a record is.)

Part (e) — Slides 39–40: the red lines and "Only sequential reads"

1. **What the slides show.** On Slide 39 red lines cut the document, and the Map output next to it, into bands; on Slide 40 more red lines fence off the three `(the,1)` pairs in the group box and the matching result `(the,3)`, and a big red arrow pointing down carries the words "Only sequential reads". The slides do not label the red lines; *our reading* is that each band is a piece of work that a different task can handle (document pieces by different Map tasks, Slides 52 and 69; all the pairs of one key — here "the" — as one independent unit of Reduce work).
2. **Meaning of "Only sequential reads".** Every phase works through its data **from top to bottom in one pass**, never jumping to random positions: Map scans its input record-at-a-time (Slide 27); the group step streams the pairs and puts equal keys together; Reduce walks through each group's values once (the "iterator over counts" of Slide 42).
3. **Why that is good (Slides 18–19).** The data are huge files (hundreds of GB to TB) that are mostly read and appended to and rarely updated in place (Slide 18), stored as large chunks of 16–64 MB (Slide 19). Reading big blocks in order is the access pattern that disks and distributed file systems handle best, so no phase needs an expensive random lookup into the big file. *Beyond the slides:* a disk seek to a random position is far slower than streaming the same amount of data in order, which is why large-scale systems are built around scans and sorting rather than random access.

EXAM TIP

For a "trace Map-Reduce on this input" question, draw the three columns of Slides 36–38: **(1)** Map output, one `(w,1)` per word occurrence; **(2)** after group by key: each key with the *list* of its values; **(3)** Reduce output, one pair per key. Say which key and value you chose, and label which part is written by the programmer (Map, Reduce) and which by the framework (group by key). Finish with the sanity checks of Step 7 (sum of counts = number of words; Reduce calls = distinct words); partial marks are given for each column even if a later one is wrong. Slogan: *programmer = Map + Reduce; framework = everything in between, plus scheduling, failures and I/O.*

PREVIEW (SLIDES 59–62) — WHAT A COMBINER WOULD CHANGE HERE

If each mapper pre-added its own pairs before sending (a *combiner*; for word count it is the same function as Reduce), Map task 0 would send `(the,2)` instead of two `(the,1)`, and Map task 1 would send `(earth,2)` instead of two `(earth,1)`: 13 pairs (5 + 8) would travel instead of 15, but the final answer is identical. That is why Reduce must **add** its values and never just count the list length once a combiner is used.

Final answer: key = word, value = 1. Programmer = Map (one call per input pair: output `(w,1)` for every word occurrence) + Reduce (one call per distinct key: `(w, [1,…,1])` → `(w, ∑ list)`) + the input files; framework = splitting, scheduling, group by key (partition, shuffle, sort and group), communication, failure handling, writing the output. Practice input: Map outputs 15 pairs (6 + 9) in 3 calls; Reduce 0 gets 8 pairs and outputs `(blue,1) (crew,2) (is,1) (of,1) (recently,1) (returned,1) (to,1)`; Reduce 1 gets 7 pairs and outputs `(earth,2) (shuttle,1) (space,1) (the,3)` (11 distinct words = 11 Reduce calls; counts sum to 15). Slide counts: `(crew,2) (space,1) (the,3) (shuttle,1) (recently,1)` = sums of the groups [1,1], [1], [1,1,1], [1], [1]. "Only sequential reads" = every phase streams its data in one ordered pass, with no random access into the huge file.

Where the marks come from: (1) Choosing key = word and value = 1, with the reason (equal words must meet to be added). (2) Correct roles: programmer writes Map and Reduce (and names the input files), framework does splitting, scheduling, group by key, data movement, failures; Map(k,v) → <k′,v′>* and Reduce(k′,<v′>*) → k′,v′′ with one Map call per input pair and one Reduce call per key. (3) Map output is (word, 1) per occurrence with the right 15 pairs. (4) Group by key: same key to the same reducer, shuffle across mappers, sort and group into lists (the list for "the" is [1,1,1]). (5) Reduce sums each list, one call per distinct key (11). (6) Final counts correct, including the five slide counts. (7) Diagram or table in the order Map → Group → Reduce with the stage owners labelled. (8) Explaining the pseudo-code in words (map outputs (w,1); reduce adds up). (9) "Only sequential reads" explained. (A typical split only — marking schemes vary.)

Slide check: matches the slide's answer — the five counts (crew,2) (space,1) (the,3) (shuttle,1) (recently,1) are reproduced, and the Map → Group by key → Reduce structure is exactly that of Slides 36–38. Slide erratum (minor): the slide's "The"/"the" handling and its schematic counts, explained in the box above.

The programming model, more specifically

Lecture 2 · Slides 43–44

Slides 43–44 restate the model in general notation. The **input** is a set of key-value pairs, and the programmer specifies two methods:

SIGNATURES (SLIDES 43–44)

Map(k, v) → <k', v'>*: takes one key-value pair and outputs a set (list) of key-value pairs.
Reduce(k', <v'>*) → k', v'': takes one key k' and the list of *all* values v' emitted for it, and outputs (k', v'') where v'' is the aggregated value.

In one line: Map: (k, v) → list of (k', v'). Reduce: (k', list of v') → (k', v'').

Reading the notation: the star `*` means "zero or more" (a list); `<k', v'>` is one pair; primes mark that the type changed (k to k', v to v' to v''). For example, *value is a single line in the file* is one possible record choice (Slide 43).

	Map	Reduce
Input	one pair (k, v)	one key k' with the list of all its v'
Output	any number of pairs (k', v'), including none	the aggregated pair (k', v'')
How many calls	**one Map call for every (k, v) pair**	**one Reduce call per *unique* key k'**
Depends on other calls?	no: every call is independent	no: every key is independent

Worked example — counting calls Lecture 2 · Slides 43–44, 51

Take the document "a rose is a rose" as **one** input pair. That is **1 Map call**. It emits 5 intermediate pairs. They contain 3 distinct keys (`a`, `rose`, `is`), so there are **3 Reduce calls**, one per key: `a` gets [1, 1], `rose` gets [1, 1], `is` gets [1]. Output: `(a,2) (rose,2) (is,1)`. If the input were instead split into the two lines "a rose" and "is a rose" as separate input pairs, there would be 2 Map calls, but still 3 Reduce calls: the number of Reduce calls depends on distinct keys, not on how the input is cut.

COMMON MISTAKE: CALL VERSUS TASK

A **call** is one invocation of the Map or Reduce function. A **task** is a unit of work run by a worker machine, and it makes *many* calls. "One Reduce call per unique key" does not mean one Reduce task per key: on Slide 52 Reduce task 1 handles keys k2, k4 and k5, i.e. three Reduce calls.

SLIDE ERRATUM (SLIDE 44)

- "All values v' with same key k' are reduced together and processed in *v'* order" is ambiguous. **Beyond the slides:** Map-Reduce sorts the *keys* (Slide 52, "Sort and Group") but does **not** guarantee any order for the *values* of one key. A Reduce function must therefore not depend on value order (summing, counting and maximum do not).
- Map's output is written as a list `<k', v'>*` but Reduce's output as `k', v''`, with no star. A Reduce call may in general emit several pairs; the word-count code on Slide 42 emits exactly one.

Example: host size

Lecture 2 · Slides 45–47

Problem (Slide 45). A large web corpus has a metadata file with one record per page: `(URL, size, date, …)`. For each **host**, find the **total number of bytes**. There may be many URLs for the same host name.

The **host** is the machine-name part of a URL. In the sample URL `http://www.youtube.com/watch?v=QhcwLyyEjOA` the protocol is `http://`, the host is `www.youtube.com`, the path is `/watch` and `v=QhcwLyyEjOA` is the video id. (The odd spaces in the slide's rendering, "http:// www. youtube.com /watch", are a typesetting artefact; real URLs contain no spaces.) The task is "group by host, SUM(size)" over a corpus too big for one machine.

Answer (Slides 46–47). The hint on the slide, "many URLs for the same host", says: records with the same host must be brought together and added. That is group-by-key plus reduce.

1. **Map:** for each record, output `(hostname(URL), size)`. `hostname(URL)` is a helper that cuts the host out of the URL. The key is the host; the value is the page size; date and other fields are dropped. The choice of key is what makes the grouping collect "all pages of one host".
2. **Group by key:** done by the framework; for each host it builds the list of all its sizes.
3. **Reduce:** sum the sizes for each host and output `(host, total bytes)`.

Worked example — three records Lecture 2 · Slides 46–47

Three invented records: `(http://www.youtube.com/a, 3000)`, `(http://www.yorku.ca/b, 500)`, `(http://www.youtube.com/c, 2000)`. Map: `(www.youtube.com,3000)`, `(www.yorku.ca,500)`, `(www.youtube.com, 2000)`. Group: `(www.youtube.com,[3000,2000])`, `(www.yorku.ca,[500])`. Reduce: `(www.youtube.com, 5000)`, `(www.yorku.ca, 500)`. Check: 3000 + 2000 + 500 = 5500 bytes in, 5000 + 500 = 5500 out.

EXAM TIP

For any "compute X per Y" question, ask: "what must meet in one place to compute one answer?" That thing is the **key**. Here one answer = one host, so key = host. Then Map = emit (key, the number you need), Reduce = combine the numbers. Slide 47 lists the three steps; write all three, and say that Group by key is done by the framework.

Try it yourself first: design the job, then trace it on the 8-record practice file with two mappers and two reducers.

Quiz L2-Q4 **Host size: total bytes per host** Lecture 2 · Slides 45–47

QUESTION (AS ON THE SLIDES)

Slide 45, "Example: Host size". "Suppose we have a large web corpus with a metadata file formatted as follows: each record of the form `(URL, size, date, …)`. Sample URL: `http://www.youtube.com/watch?v=QhcwLyyEjOA`. For each host, find the total number of bytes. There might be many URLs for the same host name." Solve it with Map-Reduce. (Slides 46–47, "Answer: Host size", show the outline of the answer: cover them and try first.)

The slides pose no numbered parts and give no numbers, so this block turns them into four exam-style parts. The 8-record input in (c) is **invented for practice** (all sizes, dates and video IDs are made up; only record 1 uses the slide's sample URL).

(a) Design the Map-Reduce job: what are the key and the value, and what do **Map**, **Group by key** and **Reduce** each do?

(b) Explain how `hostname(URL)` is extracted from a URL.

(c) Trace the job on this practice metadata file, using **2 mappers** and **2 reducers**: show the Map output, the shuffle groups and the Reduce output (with the numbers).

#	URL	size (bytes)	date	Read by
1	`http://www.youtube.com/watch?v=QhcwLyyEjOA`	52400	2025-01-12	Mapper M1
2	`http://www.youtube.com/watch?v=x7Kp2LmQ9aB`	61700	2025-01-12	Mapper M1
3	`http://www.yorku.ca/index.html`	18300	2025-01-13	Mapper M1
4	`http://www.youtube.com/watch?v=T3nR8vWc5Yd`	47100	2025-01-13	Mapper M1
5	`http://en.wikipedia.org/wiki/MapReduce`	33900	2025-01-14	Mapper M2
6	`http://www.youtube.com/watch?v=Z9fHq4LsE2u`	58800	2025-01-14	Mapper M2
7	`http://www.yorku.ca/eecs/4415`	12600	2025-01-15	Mapper M2
8	`http://en.wikipedia.org/wiki/Big_data`	29700	2025-01-15	Mapper M2

Assume the framework's partition function sends the host `www.youtube.com` to Reducer R0 and the hosts `en.wikipedia.org` and `www.yorku.ca` to Reducer R1. Four YouTube URLs (records 1, 2, 4, 6) share one host name, exactly the situation the slide warns about.

(d) Can a **combiner** be used in this job? Why is it legal here? Show its effect on the practice input.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION — RECEIPTS SORTED BY SHOP

Imagine a giant heap of receipts and a team of helpers. Each helper takes a few receipts and, for *each* receipt, writes one slip: "shop name – amount". Everything else on the receipt (date, items) is ignored. The slips are then sorted into trays, one tray per shop, so every slip of one shop ends up in the same tray whichever helper wrote it. Finally one accountant per tray adds up the amounts. Here the "shop" is the **host** (`www.youtube.com`), the "amount" is the page **size in bytes**; helpers = **Map**, sorting into trays = **Group by key** (done by the framework), accountants = **Reduce**.

Steps are tagged (a)–(d) to match the four parts of the question.

1. **(a) Idea: decide the key and the value.** The question asks for *one number per host*: the sum of the sizes of *all* pages of that host. As a formula, for one host h:

   FORMULA — WHAT WE COMPUTE

   $\text{total}(h) = \sum \text{size}(r)$ over all records r whose $\text{hostname}(\text{URL}_r) = h$

   The sizes of one host's pages sit at different places of the file (possibly on different machines), so they must first be brought together and only then added. "Bring equal things together, then combine them" is exactly the outline of Slide 27 (Map, Group by key, Reduce), and what decides which items are brought together is the **key**. So: **key = host name** (what we group by), **value = size** (what we add up; we assume the size field is in bytes, as the question asks for bytes). The date and all other fields are not needed, so Map drops them. This is the word-count template of Slides 41–42 with one change: word count emits `(word, 1)` and adds the 1s; here we emit `(host, size)` and add the sizes.

   COMMON MISTAKE

   Using the whole **URL** as the key. Then every URL forms its own group, sizes of different URLs of the same host are never brought together, and you get a size per *page*, not per *host*. That is why the slide stresses "there might be many URLs for the same host name".

2. **(a) Map (Slide 46).** In the slide's notation $\text{Map}(k, v) \rightarrow <k', v'>*$ (Slide 44). The input pair is (k, v) with v = one record `(URL, size, date, …)` (Slide 44: "value is a single line in the file"); k is just a position or record number that this job ignores. Map computes `hostname(URL)` and outputs **exactly one pair per record**:

   Map: (URL, size, date, ...) → (hostname(URL), size)

   There is **one Map call for every input pair** (Slide 44), i.e. one per record: 8 calls for our 8 records. Each page belongs to exactly one host and adds its bytes to that host only, hence one pair. (Beyond the slides: a Map call may also emit zero pairs, for example for a malformed record with no URL, because the output is "a set of key-value pairs".) Notice that Map does *no adding*: it sees one record at a time and knows nothing about the other records of the host.

3. **(a) Group by key (Slide 47).** Done by the framework, **not by the programmer** (Slides 27, 51, 53). It collects all pairs with the same key using "Hash merge, Shuffle, Sort, Partition" (Slide 51): a partition function decides which reducer owns each key (default $\text{hash}(\text{key}) \bmod R$, Slide 66, where R = number of reducers), the pairs are moved across the network (the *shuffle*, Slide 54) and sorted so that equal keys sit together. Result: for every host one list `host → [size, size, …]`. Because the destination depends only on the key, **all pairs of one host reach the same reducer**, which is what makes a correct total possible.

4. **(a) Reduce (Slide 47: "Sum the sizes for each host").** $\text{Reduce}(k', <v'>*) \rightarrow k', v''$ (Slide 44). It receives one host and the list of *all* sizes for that host, and outputs **(host, ∑ sizes)**. There is **one Reduce call per unique key** (Slide 44): one per distinct host, so 3 calls in our example, not 8. Here k' = host, v' = size of one page, v'' = total bytes of the host. (Slide 44 says the values of a key are "processed in v' order"; for a sum the order does not matter.)

5. **(b) Where the host sits in a URL.** A URL is built from parts: `scheme://host[:port]/path?query`. For the slide's sample the parts are `http://` (scheme, i.e. protocol), `www.youtube.com` (host name) and `/`

`watch?v=QhcwLyyEjOA` (path and query, which pick one page on that host). `hostname(URL)` must return only the middle part. This is why many URLs share one host: the host part stays the same while the path and the video ID change.



Figure L2-Q4-1. The three parts of the sample URL of Slide 45. Only the host name becomes the key. (The gaps in "http:// www. youtube.com /watch..." on the slide are only colouring; a real URL has no spaces.)

6. **(b) Recipe for hostname(URL)** (the slides use the function without defining it, so this is the standard definition):
   1. Remove the scheme: everything up to and including `://`.
   2. Keep the characters up to the first `/` (or `?` or `#`); if there is none, keep everything to the end.
   3. Beyond the slides: if present, also drop a port number (`:80`) and any `user@` prefix, and write the result in lower case (host names are not case-sensitive).

   Map only cuts a string: no web access is needed. Examples:

URL	hostname(URL) = key	Comment
`http://www.youtube.com/watch?v=QhcwLyyEjOA`	**`www.youtube.com`**	the slide's sample
`http://www.youtube.com/results?search_query=mapreduce`	**`www.youtube.com`**	different page, **same host**, so the same group
`http://www.yorku.ca/index.html`	**`www.yorku.ca`**	path dropped
`http://en.wikipedia.org:80/wiki/Big_data`	**`en.wikipedia.org`**	port `:80` dropped
`HTTP://WWW.YouTube.com/watch?v=abc`	**`www.youtube.com`**	lower-cased, so the same key as the sample
`https://www.youtube.com/watch?v=QhcwLyyEjOA`	**`www.youtube.com`**	the scheme is not part of the host, so http and https pages share a key
`http://youtube.com/watch?v=abc`	`youtube.com`	a different string, hence a *different* key from `www.youtube.com`

   Convention to state in your answer: two host spellings are the same host only if the strings are equal after this clean-up, so `www.youtube.com`, `youtube.com` and `m.youtube.com` are three keys. If the question wanted them merged you would have to say so (for example "strip a leading www."); the Map-Reduce structure does not change, only `hostname()` does. In practice a standard URL-parsing routine does this job; no code is examined.
7. **(c) Split the input; Map task versus Map call.** The real file is cut into blocks and each block is read by one Map task (Slide 69: "one DFS chunk per map task is common"; DFS = distributed file system). Here records 1–4 go to Mapper M1 and records 5–8 to Mapper M2. A Map *task* is a process; a Map *call* is one invocation of the Map function (Slide 44: one per record). So M1 makes 4 calls, M2 makes 4 calls, 8 in total.
8. **(c) Map phase (Slide 46).** For each record compute `hostname(URL)` and keep the size. Repeated hosts are *not* merged here.

Record	Mapper	hostname(URL)	size	Map output pair
1	M1	`www.youtube.com`	52400	`(www.youtube.com, 52400)`
2	M1	`www.youtube.com`	61700	`(www.youtube.com, 61700)`
3	M1	`www.yorku.ca`	18300	`(www.yorku.ca, 18300)`
4	M1	`www.youtube.com`	47100	`(www.youtube.com, 47100)`
5	M2	`en.wikipedia.org`	33900	`(en.wikipedia.org, 33900)`
6	M2	`www.youtube.com`	58800	`(www.youtube.com, 58800)`
7	M2	`www.yorku.ca`	12600	`(www.yorku.ca, 12600)`
8	M2	`en.wikipedia.org`	29700	`(en.wikipedia.org, 29700)`

Total Map output = **8 pairs**, one per record (the dates were dropped). Notice that M1 alone produced three `www.youtube.com` pairs and M2 one more.

9. **(c) Group by key, part 1: partition (Slides 47, 51, 66).** Every mapper sorts its pairs into one bucket per reducer using a partition function of the *key only*. The default is $\text{reducer} = \text{hash}(\text{key}) \bmod \text{R}$ with $\text{R} = 2$ (Slide 66); here we simply use the assignment given in the question (R0: `www.youtube.com`; R1: `en.wikipedia.org` and `www.yorku.ca`). A real hash could split the three hosts differently; the only guarantee is that **one host always goes to one reducer**. **Why only the key?** If the YouTube pairs were split between two reducers, each would output a partial total and the answer for the host would be wrong.

From	Bucket for R0 (`www.youtube.com`)	Bucket for R1 (`en.wikipedia.org`, `www.yorku.ca`)
M1	`(www.youtube.com, 52400)` `(www.youtube.com, 61700)` `(www.youtube.com, 47100)` — 3 pairs	`(www.yorku.ca, 18300)` — 1 pair
M2	`(www.youtube.com, 58800)` — 1 pair	`(en.wikipedia.org, 33900)` `(www.yorku.ca, 12600)` `(en.wikipedia.org, 29700)` — 3 pairs

Check: 3 + 1 + 1 + 3 = 8 pairs, none lost.

10. **(c) Group by key, part 2: shuffle and sort (Slides 27, 51, 54).** Each reducer fetches its bucket from every mapper (the all-to-all "Shuffle" of Slide 54, where data crosses between machines) and sorts the pairs so that equal keys sit together. The pairs of one key become one group `host → [list of sizes]`.

Reducer	Pairs received	After sort and group (key → list of values)
R0	3 (from M1) + 1 (from M2) = **4**	`www.youtube.com → [52400, 61700, 47100, 58800]`
R1	1 (from M1) + 3 (from M2) = **4**	`en.wikipedia.org → [33900, 29700]`; `www.yorku.ca → [18300, 12600]`

Check: 4 + 4 = 8. The YouTube list mixes sizes written by *two different mappers*; only the shuffle brings them together. (The order of the numbers inside a list does not matter for a sum.)

11. **(c) Reduce phase (Slide 47; one call per unique key, Slide 44).** Add up each list, $\text{v}'' = \sum \text{v}'$. Three distinct hosts give **3 Reduce calls**.

Reducer	Key (host)	Value list	Sum	Output pair
R0	`www.youtube.com`	[52400, 61700, 47100, 58800]	52400 + 61700 = 114100; 114100 + 47100 = 161200; 161200 + 58800 = 220000	**`(www.youtube.com, 220000)`**
R1	`en.wikipedia.org`	[33900, 29700]	33900 + 29700 = 63600	**`(en.wikipedia.org, 63600)`**
R1	`www.yorku.ca`	[18300, 12600]	18300 + 12600 = 30900	**`(www.yorku.ca, 30900)`**

R0 writes `(www.youtube.com, 220000)` to its output file; R1 writes `(en.wikipedia.org, 63600)` and `(www.yorku.ca, 30900)` (two output files, like "Out 0" and "Out 1" on Slide 54).

12. **(c) Sanity check (always do this).** Go back to the raw table and add by hand. YouTube = records 1, 2, 4, 6: 52400 + 61700 + 47100 + 58800 = 220000. York U = records 3, 7: 18300 + 12600 = 30900. Wikipedia = records 5, 8: 33900 + 29700 = 63600. All three match the Reduce output. The grand total of all 8 sizes is 52400 + 18300 + 61700 + 33900 + 47100 + 12600 + 58800 + 29700 = **314500** = 220000 + 30900 + 63600, so no byte was lost or counted twice. Also: 8 pairs entered the groups, and 3 distinct hosts gave 3 Reduce calls and 3 output pairs. Figure L2-Q4-2 shows the whole run.

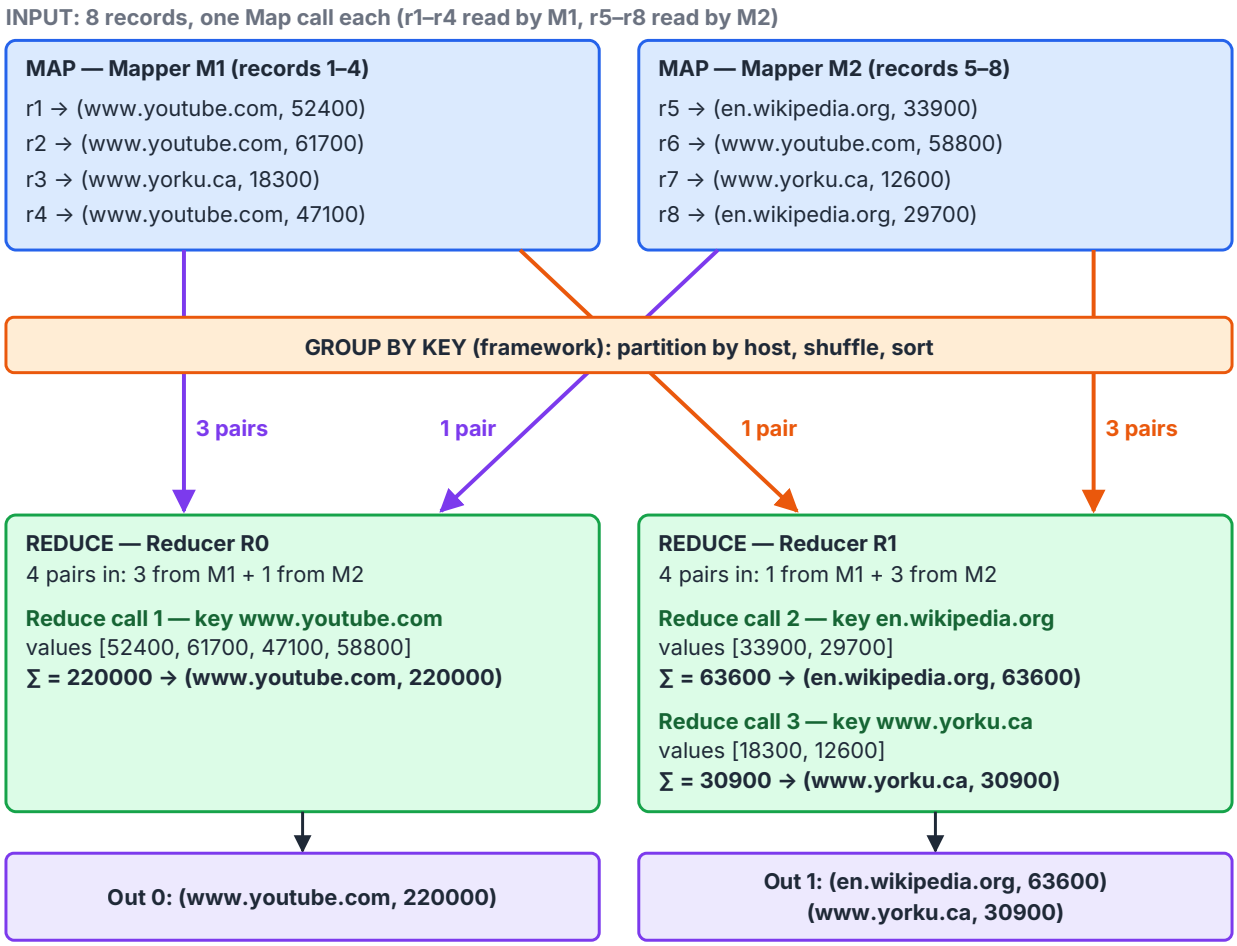
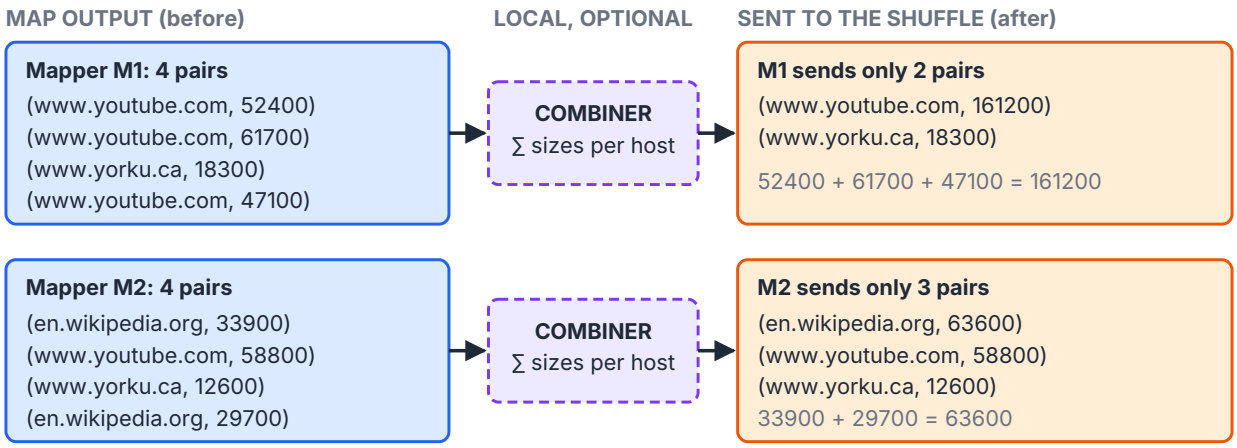


Figure L2-Q4-2. The host-size job on the practice input (no combiner). Purple arrows carry pairs to R0, orange arrows to R1; the labels give the number of pairs per mapper-to-reducer bucket (3 + 1 = 4 for R0, 1 + 3 = 4 for R1). The crossing of the two arrow families is the shuffle (compare Slides 51 and 54).

13. **(d) The problem a combiner solves (Slide 59).** "Often a Map task will produce many pairs of the form (k, v_1), (k, v_2), … for the same key k." Here mapper M1 emits three separate pairs for `www.youtube.com`, and with a real web corpus a mapper reads very many pages of a popular host. Sending every one of these pairs over the network is wasteful, because the reducer is only going to add them anyway.
14. **(d) What a combiner does (Slides 60–61).** It "pre-aggregates values in the mapper": $\text{combine}(k, \text{list}(v_1)) \rightarrow v_2$. It runs on the **same machine as one mapper**, takes *all pairs of that single mapper* and merges the pairs of each key into one pair *before* anything is shuffled. "Much less data needs to be copied and shuffled!" For this job the combiner simply adds the sizes per host, the same code as Reduce (Slide 62: "Combiner is usually same as the reduce function").

Mapper	Map output (before)	After the combiner (sent to the shuffle)	Pairs
M1	`(www.youtube.com, 52400)` `(www.youtube.com, 61700)` `(www.yorku.ca, 18300)` `(www.youtube.com, 47100)`	`(www.youtube.com, 161200)` `(www.yorku.ca, 18300)` since 52400 + 61700 + 47100 = 161200	4 → 2
M2	`(en.wikipedia.org, 33900)` `(www.youtube.com, 58800)` `(www.yorku.ca, 12600)` `(en.wikipedia.org, 29700)`	`(en.wikipedia.org, 63600)` `(www.youtube.com, 58800)` `(www.yorku.ca, 12600)` since 33900 + 29700 = 63600	4 → 3

The shuffle now moves **5 pairs instead of 8** ((8 − 5) ÷ 8 = 37.5% fewer): R0 receives `(www.youtube.com, 161200)` from M1 and `(www.youtube.com, 58800)` from M2 (2 pairs); R1 receives 1 pair from M1 and 2 pairs from M2 (3 pairs). Reduce then finishes the job: `www.youtube.com` 161200 + 58800 = **220000**; `www.yorku.ca` 18300 + 12600 = **30900**; `en.wikipedia.org` = **63600**. The final answer is **identical** to the one without a combiner.



Pairs shuffled: 8 without a combiner → 5 with a combiner (37.5% fewer)

Reducers then add: YouTube 161200 + 58800 = 220000; York U 18300 + 12600 = 30900; Wikipedia 63600. These are the same totals as without the combiner.

Figure L2-Q4-3. A combiner works inside one mapper (compare Slide 61, where the green boxes enclose one block + its mapper + its combiner). It shrinks the data before the shuffle but does not change the final totals.

15. **(d) Why a combiner is legal here (Slide 62).** A combiner may be used only when the reduce function is **commutative and associative**, because the framework does not control how the values of a key are split among mappers or in which order they are added. **Sum** has both properties: $a + b = b + a$ and $(a + b) + c = a + (b + c)$. Check on the YouTube numbers: adding per mapper first gives $(52400 + 61700 + 47100) + 58800 = 161200 + 58800 = 220000$, which equals $52400 + (61700 + 47100 + 58800) = 52400 + 167600 = 220000$. A partial sum is just a regrouping of the same addends. A second requirement is that the combiner's output pairs have the **same form as the map output**, a pair `(host, number of bytes)`, so Reduce cannot tell whether a number is one page or the sum of several pages and simply adds. Beyond the slides: the framework may run a combiner zero, one or several times, so the job must give the right answer with or without it; for a sum it does. The combiner is only an optimisation for the network; the reducer is still required.

> **COMMON MISTAKE — A COMBINER IS NOT ALLOWED FOR EVERY REDUCER (SLIDES 63–65)**
>
> Suppose the question asked for the **average** page size per host. Average is not commutative and associative (Slide 63), so reusing the reducer as combiner would be wrong. On YouTube: M1's local average = (52400 + 61700 + 47100) ÷ 3 = 53733.33, M2's = 58800; the average of the two averages is (53733.33 + 58800) ÷ 2 = 56266.67, but the true average is 220000 ÷ 4 = **55000**. Fix (Slide 64): ship **(sum, count)** pairs, M1: (161200, 3) and M2: (58800, 1); add them to (220000, 4) and divide only at the very end: 220000 ÷ 4 = 55000. The median cannot be repaired this way at all (Slide 65). Our job only *sums*, so the plain combiner is safe.

16. **Link to Slides 66–67 (partitioning).** The default partition function $\text{hash}(\text{key}) \bmod R$ (Slide 66) already sends all pairs of one host to the same reducer here, because the *key is the host*. Slide 67 shows the override $\text{hash}(\text{hostname}(\text{URL})) \bmod R$, which ensures "URLs from a host end up in the same output file" even when the key is something finer, such as the full URL (Beyond the slides: that reading of the slide is our interpretation). Beyond the slides: a very popular host such as `www.youtube.com` makes one reducer receive many more pairs than the others (skew); the combiner eases this too, since each mapper sends at most one pair per host.

COMMON MISTAKES

- **Key = URL instead of host** (step 1): you must group by the *host* to add up pages of the same host.
- **Adding inside Map.** Map emits one pair per record and does no summing; totals appear after grouping. (Pre-adding inside the mapper is a separate optimisation, the combiner, which comes on top and does not replace Reduce.)
- **Reduce is called once per host** (3 calls here), not once per record (8).
- **Keeping the date or other fields in the value.** Only the size is needed; extra fields would also have to be shuffled.
- **Inconsistent hostname().** `WWW.YouTube.com` and `www.youtube.com` would become two keys unless `hostname()` lower-cases; and `youtube.com` is not `www.youtube.com` unless you say so.
- **Using a combiner for a function that is not commutative and associative** (average, median) without the (sum, count) fix.
- **Confusing combiner and reducer.** The combiner is local (one mapper), optional, and only cuts network traffic; the reducer receives the data of all mappers and produces the final answer.

EXAM TIP

For any "find the total per X" question write: (1) **key = X, value = the number to add**, with one reason; (2) Map in one line, e.g. `(URL, size, date) → (hostname(URL), size)`; (3) "Group by key" is done by the framework; (4) Reduce: **sum**; (5) a small table Map output → groups → Reduce output with a sanity check (here 314500 = 220000 + 30900 + 63600); (6) one line on the **combiner**: "same as Reduce, legal because sum is commutative and associative, shuffles 5 instead of 8 pairs". Each item is separate partial credit. If the question asked for the *number of URLs per host* instead, only the value changes: emit `(hostname(URL), 1)` and sum, exactly as in word count.

Final answer: key = hostname(URL), value = size. Map: for each record output `(hostname(URL), size)` (one call per record). Group by key (framework): collect all sizes of each host at one reducer. Reduce: sum the sizes of each host (one call per host). `hostname(URL)` = the text between `://` and the next `/` (port dropped, lower-cased), e.g. `www.youtube.com` for `http://www.youtube.com/watch?v=QhcwLyyEjOA`. Practice input: Map emits 8 pairs; R0 gets 4 and outputs `(www.youtube.com, 220000)`; R1 gets 4 and outputs `(en.wikipedia.org, 63600)` and `(www.yorku.ca, 30900)`; grand total 314500 bytes. A combiner (same code as Reduce) is legal because sum is commutative and associative; it cuts the shuffle from 8 to 5 pairs and leaves the totals unchanged.

Where the marks come from: (1) key = host, value = size, with the reason (pages of one host must be brought together; URL as key would not work); (2) Map: one call per record outputs `(hostname(URL), size)`, other fields dropped; (3) Group by key is a separate step done by the framework, all pairs of a host reach the same reducer; (4) Reduce: sum per host, one call per unique key; (5) explanation of hostname(URL): drop the scheme, keep up to the next "/", ignore path and query; (6) Map output table (8 pairs); (7) groups after shuffle and sort; (8) correct sums 220000, 63600, 30900 and the check 314500; (9) combiner named as pre-aggregation in the mapper, same as Reduce, with the reason (sum is commutative and associative) and its effect (8 → 5 pairs); (10) optionally, why average or median would break it. (Typical split only; marking schemes vary.)

Slide check: matches the slide's answer. Map outputs `(hostname(URL), size)`, then Group by key, then Reduce sums the sizes for each host (Slides 46–47). No slide erratum on Slides 45–47. The slides stop at this outline: they give no data, do not define `hostname(URL)`, do not name the key and value explicitly and do not apply a combiner

to this example (combiners appear on Slides 59–62 with word count); those parts were solved here as an extension of the outline. The gaps in the sample URL “http:// www. youtube.com /watch?v=…” on Slide 45 are only typesetting.

Example: language model (5-word sequences)

Lecture 2 · Slides 48–49

Problem (Slide 48). Count the number of times each **5-word sequence** occurs in a large corpus of documents. A 5-word sequence (**Beyond the slides:** a "5-gram") is any run of 5 *consecutive* words; the windows overlap, one starting at every word position. Such counts are the raw material of a **language model** (used for autocomplete, speech recognition and machine translation), which is why the slide has that title.

Answer (Slide 49). It is word count where the thing counted is a 5-word window instead of a single word.

1. **Map:** slide a 5-word window along each document and, for each position, emit `(5-word sequence, count)`, where the count is 1 per occurrence (the slide leaves the 1 implicit).
2. **Group by key:** the framework collects the counts of each 5-word sequence, for example `(the cat sat on the, [1, 1, 1])`.
3. **Reduce:** combine the counts by adding them up: `(the cat sat on the, 3)`.

The key is the 5-word sequence, and that choice alone decides what is counted. Compare the three examples so far:

Example	Slides	Key	Value emitted by Map	Reduce does
Word count	26, 36–42	a word	1	sum
Host size	45–47	host name	page size in bytes	sum
Language model	48–49	5-word sequence	1 (count)	sum

This is Slide 28 in action: the outline stays the same; Map and Reduce change to fit the problem.

Try it yourself first: write down the key and value, then work the two-document trace and the "n − 4 windows" counting argument.

Quiz L2-Q5 Counting 5-word sequences in a large corpus (language model) Lecture 2 · Slides 48–49

QUESTION (AS ON THE SLIDES)

Slide 48 — "Example: Language Model". "Count number of times each 5-word sequence occurs in a large corpus of documents." Slide 49 ("Answer: Language Model") gives the outline of the answer: cover it and try first.

The slides give one question and a three-line answer, with no data and no numbered parts, so this block turns them into four exam-style parts. Part (a) is what the slides ask. Parts (b)–(d), the two documents in (b) and the sentence in (d) are **Practice** written for this guide (invented, not on the slides). Assumptions: words are separated by spaces, the text is already lower-case with punctuation removed, and one Map call reads one document, so a 5-word sequence never runs across the end of a document.

(a) Design the Map-Reduce job: what are the key and the value, what does **Map** emit, what does **Group by key** do and what does **Reduce** compute?

(b) Practice: trace the job on two short documents with **2 mappers** (M1 reads D1, M2 reads D2) and **2 reducers**. Show the sliding windows, the Map output, the group by key (shuffle) step and the Reduce output.

Doc	Text	Read by
D1	`the quick brown fox jumps over the dog`	Mapper M1
D2	`yesterday the quick brown fox jumps high`	Mapper M2

(c) Practice: explain why a document of n words yields exactly $n - 4$ five-word sequences. What happens if $n < 5$?

(d) Practice: what is a combiner? Show on the sentence `to be or not to be or not to be` how it helps in this job, and explain why it is allowed here.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION — A KEYHOLE THAT IS 5 WORDS WIDE

Word count (Slides 36–42) counted single words. Here the thing we count is a **phrase of exactly five consecutive words**. Picture reading the text through a keyhole that shows five words at a time. You note down the five visible words on an index card with a tally mark "1", slide the keyhole **one word** to the right, note down again, and so on until the keyhole runs off the end. That is **Map**. Then you put all cards with the same phrase into one pigeonhole: that is **Group by key**. Finally you add up the tally marks in each pigeonhole: that is **Reduce**. Nobody ever needs the whole book at once, which is why the job can be split over thousands of machines.

Beyond the slides: the slide is titled "Language Model" because a table of 5-word-sequence counts lets a program estimate how likely a word is to follow the four words before it, roughly $P(w_5 \mid w_1\, w_2\, w_3\, w_4) \approx \text{count}(w_1 \ldots w_5) \div \text{count}(w_1 \ldots w_4)$. Such tables (built from billions of web pages) are used in machine translation, speech recognition and auto-complete. That is why the corpus is huge and the counting must be distributed.

Steps are tagged (a)–(d) to match the four parts of the question.

1. **(a) Idea: what must be brought together (Slides 27–28, 44).** To know how often one 5-word sequence occurs, *all* its occurrences, which may sit in documents on many different machines, must meet in one place and be added. "Bring equal things together, then combine them" is exactly the Map-Reduce outline of Slides 27–28 ("Outline stays the same, Map and Reduce change to fit the problem"). It is the same pattern as word count (Slides 36–42) and host size (Slides 45–47), with a 5-word sequence in place of a single word. The programmer supplies only `Map(k, v) → <k′, v′>*` and `Reduce(k′, <v′>*) → k′, v″` (Slide 44); the system does Group by key. So three decisions remain:
   - **Key** = the **whole 5-word sequence** (all five words, in order, treated as *one* string).
   - **Value** = **1** (one sighting; Slide 49 calls it "count").
   - **Reduce** = add the values of each key.

Why the whole sequence is the key: Group by key gathers pairs whose keys are *equal*. Only if the key is the complete sequence do two sightings of the same sequence land in the same group; sequences that differ in even one word are different keys and are counted separately.

2. **(a) Map (Slide 49: "Extract (5-word sequence, count) from document").** Let a document have words $w_1, w_2, \ldots, w_n$. One Map call reads one document (Slide 44: one Map call per (k, v) pair; in word count the pair is (document name, text), Slide 41). It slides a window of 5 words along the text, **one word at a time**, and for each start position $s = 1, 2, \ldots, n - 4$ emits **("$w_s\ w_{s+1}\ w_{s+2}\ w_{s+3}\ w_{s+4}$", 1)**.

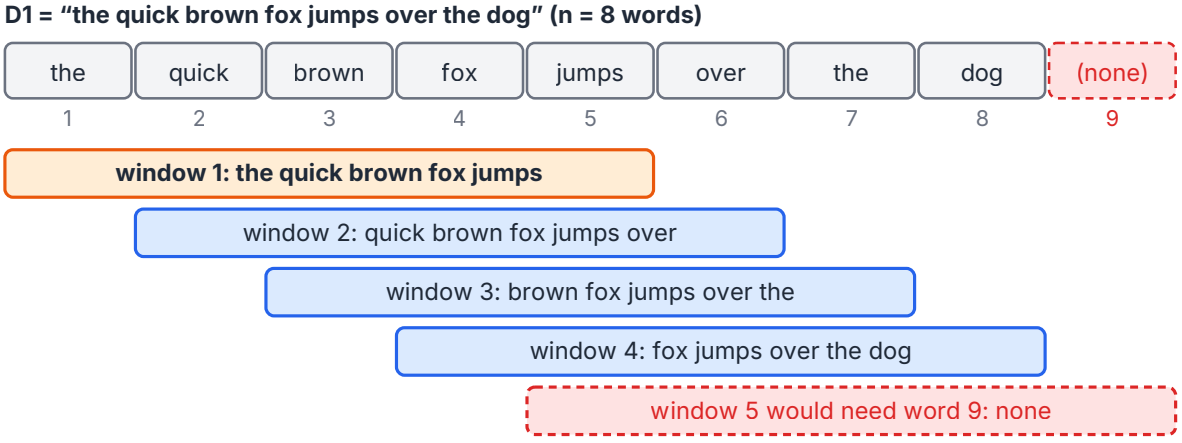
   Why one word at a time, not five: a sequence may start at *any* word, so stepping by 5 would miss 4 out of every 5 sequences. *Why Map does no adding:* a Map call sees only its own document and cannot know the global total; totals are Reduce's job. (The "count" on Slide 49 is 1 per window; it becomes a partial sum only if a combiner has already run, see (d).)

3. **(a) Group by key (Slide 49; done by the framework, the programmer writes nothing).** The system collects the pairs from *all* mappers and sends every pair with the same key to the same Reduce worker ("hash merge, shuffle, sort, partition", Slide 51; the default rule is `hash(key) mod R`, Slide 66; the environment does this for you, Slide 53). Result: one group `(5-gram, [1, 1, …, 1])` per *distinct* 5-word sequence, with as many 1s as the sequence has occurrences. (Beyond the slides: a 5-word sequence is also called a *5-gram*; the guide uses both names.)

4. **(a) Reduce (Slide 49: "Combines the counts (adds them up)").** There is one Reduce call per unique key (Slide 44). For key g with value list $[v_1, \ldots, v_m]$ it outputs **$(g, v_1 + \ldots + v_m)$**, i.e. count(g) = $\sum v_i$. All Reduce outputs together form the table of counts the question asks for. *Why we write "add the values" and not "count the list items":* with a combiner (step 11) a value can be larger than 1, and adding is still right.

> **KEY IDEA**
> Every Map call looks only at *one document* and every Reduce call looks only at *one 5-gram*, so no machine ever needs the whole corpus or the whole table of counts. Only the Group by key step moves data between machines. That is why the job scales to a "large corpus of documents".

5. **(b) Step 1: the sliding windows.** A document of n words gives windows starting at word 1, 2, …, $n - 4$ (reason in (c)). D1 has $n = 8$ words, so $8 - 4 = 4$ windows. D2 has $n = 7$ words, so $7 - 4 = 3$ windows. Figure 1 shows D1.



Each window starts one word to the right of the previous one; the last legal start is word n − 4 = 4.
Every window becomes one Map pair (window, 1): 4 pairs = 8 − 4.

Figure 1 — Sliding a 5-word window along D1: 8 words give 4 windows, and the 5th would need a 9th word that does not exist (orange = the 5-gram that D2 will also produce).

Doc (n)	Window s	Words used	5-word sequence
D1 (8)	1	1–5	**`the quick brown fox jumps`**
	2	2–6	`quick brown fox jumps over`
	3	3–7	`brown fox jumps over the`
	4	4–8	`fox jumps over the dog`
D2 (7)	1	1–5	`yesterday the quick brown fox`
	2	2–6	**`the quick brown fox jumps`**
	3	3–7	`quick brown fox jumps high`

The shaded 5-gram `the quick brown fox jumps` is window 1 of D1 but window 2 of D2: a repeated sequence does not need to sit at the same position in both documents.

6. **(b) Step 2: Map output.** Every window becomes a pair (window, 1):

Mapper	Pairs emitted	Count
M1 (D1)	`(the quick brown fox jumps, 1)` · `(quick brown fox jumps over, 1)` · `(brown fox jumps over the, 1)` · `(fox jumps over the dog, 1)`	4 = 8 − 4
M2 (D2)	`(yesterday the quick brown fox, 1)` · `(the quick brown fox jumps, 1)` · `(quick brown fox jumps high, 1)`	3 = 7 − 4

Total Map output = 4 + 3 = **7 pairs**. Note that `(the quick brown fox jumps, 1)` was produced **twice, on two different machines**; neither mapper can know that the other one saw it too.

7. **(b) Step 3: Group by key (sort and shuffle).** The framework sorts the 7 pairs by key and puts equal keys into one group. The groups are shared out among the 2 reducers; a real system uses *hash(key) mod R* (Slide 66), so the R0/R1 split below is only an illustration. Any split that keeps a whole key on one reducer is correct.

Key (5-word sequence)	Grouped values	Came from	Reducer
`brown fox jumps over the`	[1]	M1 (D1 window 3)	R0
`fox jumps over the dog`	[1]	M1 (D1 window 4)	R0
`quick brown fox jumps high`	[1]	M2 (D2 window 3)	R0
`quick brown fox jumps over`	[1]	M1 (D1 window 2)	R1
`the quick brown fox jumps`	**[1, 1]**	**M1 (D1 window 1) and M2 (D2 window 2)**	**R1**
`yesterday the quick brown fox`	[1]	M2 (D2 window 1)	R1

7 pairs in → **6 groups** out (6 distinct 5-word sequences). Only `the quick brown fox jumps` has a list of length 2. **Careful:** `quick brown fox jumps over` and `quick brown fox jumps high` share four words but are *different keys*, because a key matches only if all five words match, in the same order. Figure 2 follows the repeated key through the whole job.

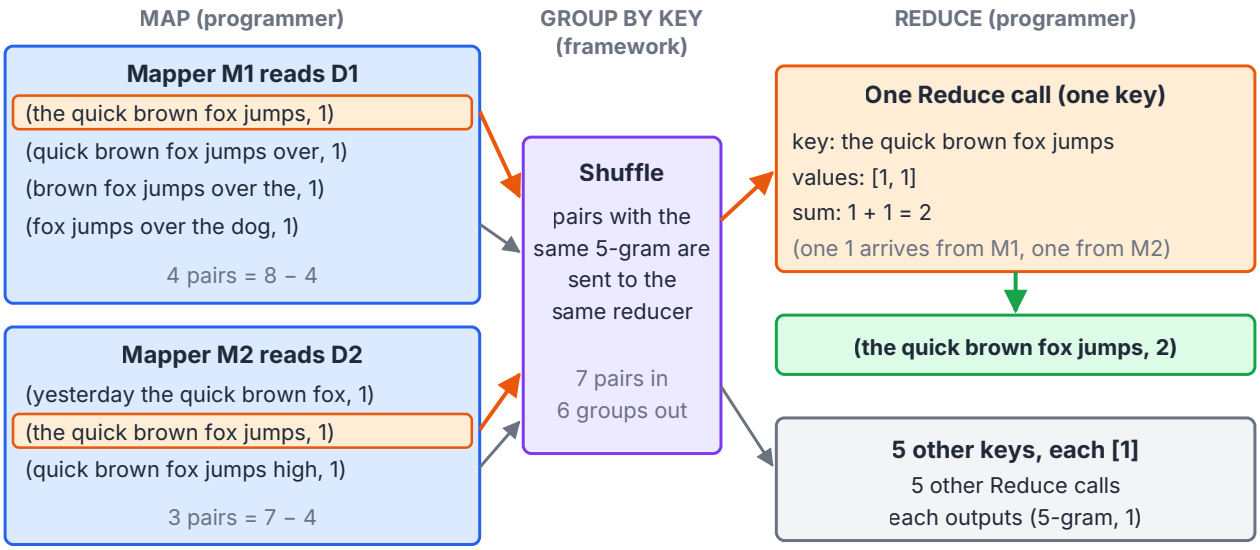


Figure 2 — Why Group by key matters: the same 5-gram produced on two different machines (orange) is brought to one Reduce call and added; the other five 5-grams occur once.

8. **(b) Step 4: Reduce output.** 6 groups → 6 Reduce calls (one per unique key, Slide 44), each adding up its list:

Key	Reduce computes	Output pair
`brown fox jumps over the`	1	`(brown fox jumps over the, 1)`
`fox jumps over the dog`	1	`(fox jumps over the dog, 1)`
`quick brown fox jumps high`	1	`(quick brown fox jumps high, 1)`
`quick brown fox jumps over`	1	`(quick brown fox jumps over, 1)`
`the quick brown fox jumps`	**1 + 1 = 2**	**`(the quick brown fox jumps, 2)`**
`yesterday the quick brown fox`	1	`(yesterday the quick brown fox, 1)`

Sanity checks. (i) Pairs emitted = (8 − 4) + (7 − 4) = 4 + 3 = **7**. (ii) The output counts add up to 1 + 1 + 1 + 1 + 2 + 1 = **7** = number of Map pairs, so nothing was lost or invented (Reduce only adds). (iii) Distinct 5-grams = 7 − 1 = **6** = number of groups = number of Reduce calls = number of output lines (one 5-gram was seen twice, so one pair "disappears" in the grouping).

9. **(c) Why n − 4: argument 1, count the start positions.** Number the words 1 to n. A window is fixed by the position s of its *first* word. It then occupies words s, $s+1$, $s+2$, $s+3$, $s+4$ (five words), so its *last* word is word $s + 4$, which must exist: $s + 4 \le n$, i.e. $s \le n - 4$. Also $s \ge 1$. The legal starts are 1, 2, …, $n - 4$, and a list of the integers 1 up to m has m members, so there are $\mathbf{n - 4}$ **windows**. In words: the *last 4 words* of a text can never start a window, because there are not enough words after them.
10. **(c) Argument 2, count the new words; the case n < 5.** The first window uses the first 5 words. Each later window is the previous one moved one word to the right, so it adds exactly *one* new word at the end. The words left after the first five are $n - 5$, giving $1 + (n - 5) = n - 4$ windows. Both arguments agree:

n (words)	Possible start positions s (need s + 4 ≤ n)	Windows
≤ 4	none (no room for even one window)	0
5	s = 1	1
6	s = 1, 2	2
7 (D2)	s = 1, 2, 3	**3**
8 (D1)	s = 1, 2, 3, 4	**4**
9	s = 1, …, 5	5
10 (sentence in (d))	s = 1, …, 6	6
n	s = 1, 2, …, n − 4	**n − 4**

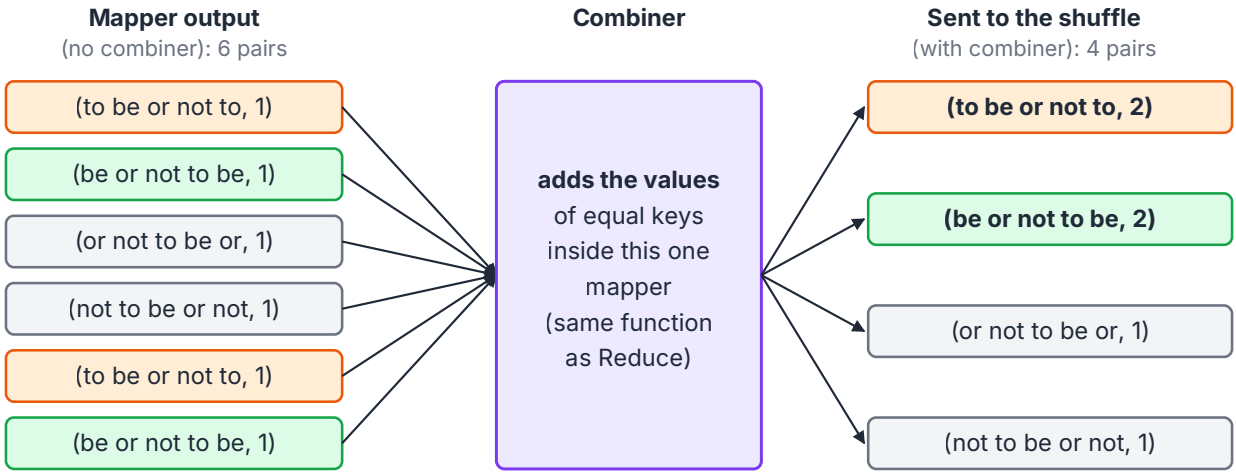
Checks against part (b): D2 ($n = 7$) → 3 ✓, D1 ($n = 8$) → 4 ✓. If $n < 5$ the document emits **0** pairs, so the count is max(0, $n - 4$), never negative. **Beyond the slides:** for windows of k words the count is $n - k + 1$; for $k = 5$ that is $n - 4$.
Consequence for the whole corpus: Map emits ∑ over documents of max(0, $n_d - 4$) pairs, i.e. almost **one pair per word** of the corpus. Each pair carries a 5-word key, so (Beyond the slides) the Map output holds about five times as much text as the input. Shuffling it is expensive, which is why a combiner matters (d).

> **COMMON MISTAKE**
> Cutting the text into *non-overlapping* blocks of five (about $n \div 5$, rounded down: D1 would give only 1) misses almost all 5-grams. The window must slide by **one** word. Another frequent slip is the off-by-one answer $n - 5$, which gives 0 for a 5-word document that clearly contains one 5-gram.

11. **(d) What a combiner is (Slides 59–62).** A Map task often produces many pairs with the *same key* (Slide 59). A **combiner** is a mini-Reduce that runs **on the mapper's own machine, before the shuffle**: `combine(k, list(v1)) → v2` (Slide 60). It "combines the values of all keys of a single mapper (single machine)", so "much less data needs to be copied and shuffled" (Slide 61). For this job the combiner is **the same function as Reduce: add the counts** (Slide 62: "Combiner is usually same as the reduce function"). Pairs leaving a mapper then carry partial counts instead of 1s.
12. **(d) Worked example.** `to be or not to be or not to be` has $n = 10$ words, so 10 − 4 = 6 windows (shaded rows = 5-grams that occur twice):

s	Window (words s to s+4)	Map emits
1	**`to be or not to`**	`(to be or not to, 1)`
2	**`be or not to be`**	`(be or not to be, 1)`
3	`or not to be or`	`(or not to be or, 1)`
4	`not to be or not`	`(not to be or not, 1)`
5	**`to be or not to` (repeat of s = 1)**	`(to be or not to, 1)`
6	**`be or not to be` (repeat of s = 2)**	`(be or not to be, 1)`

Without a combiner this mapper ships **6 pairs** over the network. **With** the combiner it adds the 1s per key first and ships only **4 pairs**: `(to be or not to, 2)`, `(be or not to be, 2)`, `(or not to be or, 1)`, `(not to be or not, 1)`. The counts still add up (2 + 2 + 1 + 1 = 6) and the traffic drops by (6 − 4) ÷ 6 = 33 %. Figure 3 shows the data flow.



The counts still add up: 2 + 2 + 1 + 1 = 6. Traffic falls from 6 to 4 pairs (33 % less).

Figure 3 — A combiner inside one mapper: the two pairs for each repeated 5-gram are merged before anything leaves the machine, so 6 pairs become 4.

13. **(d) Effect on the trace in (b).** M1 emits 4 *different* keys and M2 emits 3 *different* keys, so a combiner has nothing to merge: still 7 pairs. The repeated key sits on two *different* mappers, and a combiner only

merges within one machine (Slide 61); the Reducer is still needed to add the two partial counts. If one mapper had read *both* documents, the combiner would turn the two pairs into `(the quick brown fox jumps, 2)` and ship 6 pairs instead of 7. On a real corpus, frequent phrases occur thousands of times per mapper, so the saving is large.

14. **(d) Why it is allowed (Slide 62).** A combiner may be used only if the reduce function is **commutative and associative**: $a + b = b + a$ and $(a + b) + c = a + (b + c)$. Summation is both, so adding the 1s in groups first (per mapper) and the partial sums later gives the *same total* as adding all the 1s at once, in any order. Example: if a second mapper also emitted `(to be or not to, 1)`, the Reducer would receive [2, 1] and add 2 + 1 = 3, the same as 1 + 1 + 1. (Counting the list *length* would wrongly give 2, which is why Reduce adds the values.) The combiner is only an **optimisation**: the final output is identical with or without it. **Beyond the slides:** the framework may run a combiner zero, one or several times, so the program must still be correct without it. Contrast: the average is *not* commutative and associative (Slides 63–64) and the median cannot be split into such functions (Slide 65), so they cannot use the combiner trick directly. That is not an issue here because we only add.

EXAM TIP

Write the answer under the three slide headings **Map / Group by key / Reduce**, each with its exact key–value format: (1) Map → (5-word sequence, 1) for every sliding window, one word at a time, giving n − 4 pairs per document; (2) Group by key → (5-word sequence, [1, 1, ...]), all pairs with the same 5-gram at the same reducer; (3) Reduce → (5-word sequence, sum). Then add a short trace (2 documents, one repeated 5-gram) and one line on the combiner: same function as Reduce, runs on the mapper, valid because + is commutative and associative, saves network traffic. Each of these is a separate partial mark.

COMMON MISTAKES

- Using the *word* as the key (that is word count, not 5-word-sequence count), or the document name as the key.
- Treating `quick brown fox jumps over` and `quick brown fox jumps high` as "the same" because four words agree. The whole sequence is the key.
- Stepping the window by 5 words instead of 1, or the off-by-one slips n, n − 5 and n − 3 instead of n − 4.
- Adding up counts inside Map: one Map call sees one document only; the global total exists only after Group by key.
- Letting Reduce count the list length instead of adding the values (breaks as soon as a combiner is used).
- Thinking the combiner replaces the Reducer: equal keys on *different* mappers can only be added together after the shuffle.

Final answer: (a) Map: for every document slide a window of 5 consecutive words one word at a time and emit `(5-word sequence, 1)`, giving $n - 4$ pairs for a document of $n \geq 5$ words (0 if $n < 5$). Group by key (framework): all pairs with the same 5-word sequence go to the same reducer as `(5-gram, [1, 1, …])`. Reduce: add the list and output `(5-gram, total count)`, one call per distinct 5-gram. (b) D1 gives 4 windows and D2 gives 3, so 7 Map pairs → 6 groups → 6 Reduce calls with output `(brown fox jumps over the, 1)`, `(fox jumps over the dog, 1)`, `(quick brown fox jumps high, 1)`, `(quick brown fox jumps over, 1)`, `(the quick brown fox jumps, 2)`, `(yesterday the quick brown fox, 1)`; counts add up to 7. (c) Start positions are s = 1, …, n − 4 (because s + 4 ≤ n), so n − 4 windows, and 0 if n < 5. (d) A combiner is a mapper-side pre-sum (same function as Reduce, allowed because addition is commutative and associative) that cuts the data shuffled without changing the result, e.g. 6 pairs → 4 for `to be or not to be or not to be`.

> **Where the marks come from:** (1) Map: key = the 5-word sequence (sliding window of 5 consecutive words, step of one word), value = 1 (“count”); (2) Group by key: all pairs with the same 5-gram are brought to the same reducer, a framework step; (3) Reduce: add the values per key → (5-gram, count), one call per distinct key; (4) trace: correct overlapping windows (4 for D1, 3 for D2), 7 Map pairs, 6 groups, the repeated 5-gram `the quick brown fox jumps` with list [1, 1] and final count 2, every other count 1, and the check that the counts add up to 7; (5) n − 4 with a reason (the last window starts at word n − 4, or the first window uses 5 words and each further word adds one) and 0 when n < 5; (6) combiner: pre-aggregation on the mapper, same as Reduce, valid because the sum is commutative and associative, less data shuffled, plus a small example (6 → 4 pairs). The exact mark split is set by the instructor; the list shows where each step can earn credit.

Slide check: matches the slide’s answer. Slide 49 gives only the outline (Map: extract `(5-word sequence, count)` from document; Group by key; Reduce: combine the counts, i.e. add them up) and this solution agrees with it. No slide erratum. The outline leaves implicit (omissions, not errors) that the “count” emitted by Map is 1 per window, that the extraction is a sliding window with a one-word step giving n − 4 pairs per n-word text, and that a combiner can be used (combiners are on Slides 59–62); these are filled in above. Slide 48 gives no data and does not say why 5-grams are counted; the language-model background is marked “Beyond the slides”.

Other uses of Map-Reduce

Lecture 2 · Slide 50

Slide 50 is a one-line list of further problems that fit the model: **link analysis and graph processing**, **machine-learning algorithms**, **mining frequent itemsets** and **clustering**. The slide gives no detail; you only need to quote these four areas. They preview the course: frequent itemsets are the subject of Lecture 4 and clustering of Lecture 5 (**Beyond the slides:** PageRank is the classic link-analysis computation, and it is iterative, which is why "output is input to another job" on Slide 55 matters).

The Map-Reduce picture, and how it runs in parallel

Lecture 2 · Slides 51–52

Slide 51: the single-flow diagram

Lecture 2 · Slide 51

Slide 51 shows the whole job top to bottom. A "Big document" is cut into **7 input pieces**. Each piece goes through one **M** (a Map call). The "Intermediate" row holds the emitted pairs, written `key:value`; one Map call emitted *nothing* (the empty second cell). One box **Group by Key** regroups them into the "Grouped" row, and one **R** per key writes the "Output" row.

Piece	1	2	3	4	5	6	7
Intermediate	k1:v k1:v k2:v	(empty)	k1:v	k3:v k4:v	k4:v k5:v	k4:v	k1:v k3:v

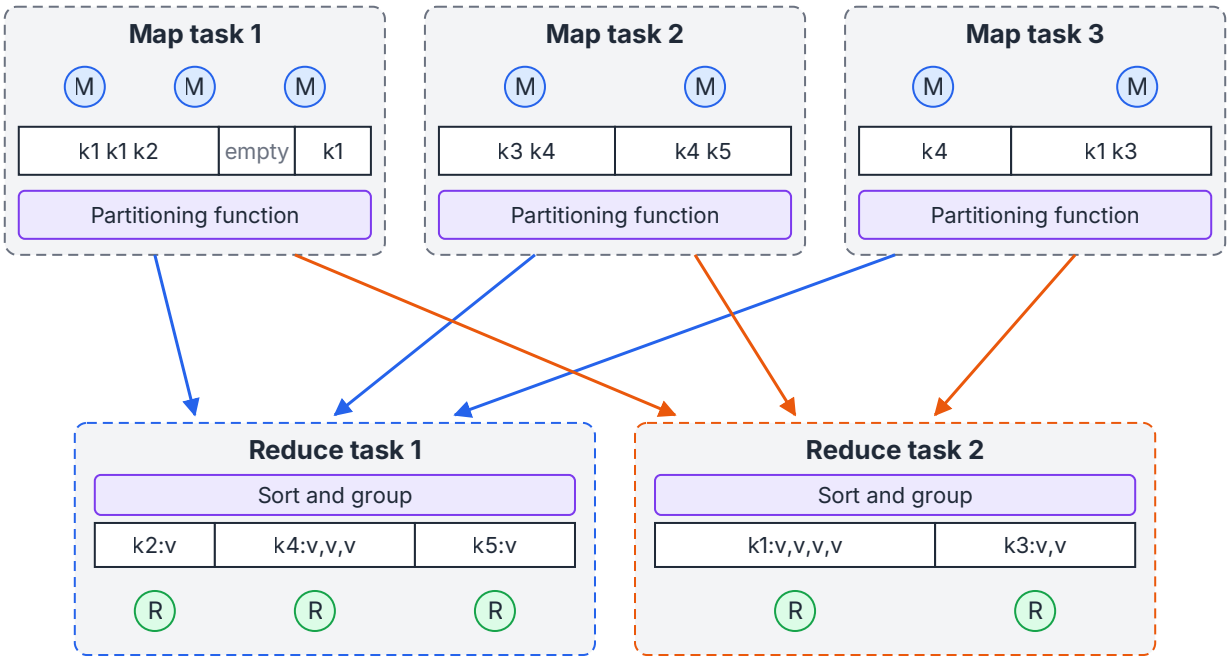
Count it: 11 pairs in total; k1 four times, k2 once, k3 twice, k4 three times, k5 once (4 + 1 + 2 + 3 + 1 = 11). The Grouped row is therefore `k1:v,v,v,v | k2:v | k3:v,v | k4:v,v,v | k5:v`: **5 distinct keys = 5 Reduce calls**, while there are 7 Map calls (one per input piece). The slide's three coloured call-outs repeat what you know: MAP reads input and produces key-value pairs; Group by key collects all pairs with the same key (and names the machinery "Hash merge, Shuffle, Sort, Partition"); Reduce collects all values belonging to the key and outputs.

> **TERMS ON THE GROUP-BY-KEY CALL-OUT (SLIDE 51)**
>
> The slide lists the words without defining them. **Beyond the slides: partition** = decide which reduce task receives a given key, typically $\mathrm{hash}(\mathrm{key}) \bmod \mathrm{R}$ where R is the number of reduce tasks; **shuffle** = move the pairs over the network from map workers to reduce workers; **sort / hash merge** = order or merge the arrived pairs so that equal keys are adjacent.

Slide 52: in parallel

Lecture 2 · Slide 52

Slide 51 draws Group by Key as one box in the middle, which hides that this step is itself distributed. Slide 52 draws the same job the way it runs on a cluster, with the caption **"All phases are distributed with many tasks doing the work."** The 7 pieces are shared among **3 Map tasks** (3 + 2 + 2 Map calls). At the end of each Map task a **Partitioning Function** looks only at the key of each pair and decides which of the **2 Reduce tasks** must receive it. Each Reduce task receives parts from *every* Map task (3 × 2 = 6 arrows: the shuffle), **sorts and groups** them by key and calls R once per key.



Shuffle = the 6 arrows (3 map tasks × 2 reduce tasks).

Blue: keys k2, k4, k5 to Reduce task 1. Orange: keys k1, k3 to Reduce task 2.

Figure L2-8 — Map-Reduce in parallel (Slides 51–52). The partitioning function sends each key to exactly one Reduce task; Map task 2 sends k4, k5 to Reduce task 1 and k3 to Reduce task 2.

Worked example — follow key k4 **Lecture 2 · Slide 52**

k4 occurs three times: twice in Map task 2 (cells "k3 k4" and "k4 k5") and once in Map task 3. Each Map task's partitioning function sends its k4 pairs to Reduce task 1, so Reduce task 1 receives `k4:v,v,v` and makes **one** R call that sees all three values. Whichever Map task made them, they cannot be split between the two Reduce tasks; otherwise the total would be wrong. Likewise all four k1 pairs (Map task 1 twice in cell 1 and once in cell 3, Map task 3 once) land in Reduce task 2: `k1:v,v,v,v`.

So group-by-key is done by the combination **partition at the map side + shuffle across the network + sort and group at the reduce side**. The key-to-task assignment in the figure (k2, k4, k5 → Reduce 1; k1, k3 → Reduce 2) is illustrative; the slide does not give the partitioning rule (typically $\text{hash}(\text{key}) \bmod R$, **Beyond the slides**).

What the environment takes care of

Lecture 2 · Slide 53

Slide 53 lists, in five items, what you do **not** have to write. The **Map-Reduce environment** (the runtime, such as Google's system or Hadoop) takes care of:

#	The environment takes care of	In plain words
1	**Scheduling** the program's execution across a set of machines	decides which task runs on which machine and when

2	Managing required **inter-machine communication**	moves data between machines
3	**Partitioning the data**	splits the input into pieces so that many Maps can run in parallel (and, per Slide 52, divides the intermediate keys among reduce tasks)
4	Performing the **group by key** step	the sort and shuffle between Map and Reduce
5	Handling machine **failures**	detects dead machines and re-runs their tasks (Slide 57)

This is the answer to the challenge of Slides 15–16: distributed programming is hard, so give the programmer a simple model that hides the complexity. The slide's "partitioning the data" is ambiguous (input splitting versus key partitioning); both are done by the system.

EXAM TIP

"What does the Map-Reduce environment take care of?" has a five-item answer: scheduling, communication, partitioning the data, group by key, failures. Learn the list as **S-C-P-G-F**.

Slide 54: the programmer's view of the workflow

Lecture 2 · Slide 54

Slide 54 condenses the same idea to a small picture. **Programmer specifies:** Map, Reduce and the input files. **Workflow:** Map → sort and shuffle → Reduce. **All phases are distributed with many tasks doing the work.** The picture has three inputs (Input 0, 1, 2) feeding Map 0, 1, 2; a wide band labelled **Shuffle**; two reduce tasks Reduce 0 and Reduce 1, each receiving one arrow from *every* Map (6 shuffle arrows); and one output each, Out 0 and Out 1. The shuffle is an all-to-all exchange, so it is the most network-heavy step (**Beyond the slides**).

- The number of Map tasks follows the number of input pieces (3 here); the number of Reduce tasks (2 here) is chosen by the user (**Beyond the slides**).
- The "Input 0/1/2" circles are input *pieces* (splits), not necessarily separate files.
- One Reduce task produces one output piece, so a job with R reduce tasks writes R output pieces.

SLIDE ERRATUM (MINOR, SLIDES 51–54)

- Slide 51 draws Group by Key as one central box; Slide 52 shows it is distributed. Use Slide 52's version when you explain the system.
- Slide 52 numbers tasks from 1 (Map task 1, Reduce task 1); Slide 54 numbers from 0 (Map 0, Reduce 0). Same idea, different labels; some diagonal arrows on Slide 52 also start slightly outside their boxes (drawing glitch).
- Slide 54 writes "Sorts & Shuffles" (grammar); Slide 27 calls the same stage "Group by key (sort and shuffle)". It also omits that the programmer may choose the number of reducers.

Term	Meaning	Example on the slides
Map call	one invocation of Map on one input pair	the 7 M circles of Slide 51
Map task	a worker's job over one input piece; makes many Map calls	Map task 1 on Slide 52 (3 M calls)
Reduce call	one invocation of Reduce on one key group	the 5 R circles of Slide 51
Reduce task	a worker's job for a set of keys; makes many Reduce calls	Reduce task 1 on Slide 52 (3 R calls)

Where the data lives: data flow

Lecture 2 · Slide 55

Three rules, one per bullet:

1. **Input and final output** are stored on the **distributed file system (DFS)**, the replicated, fault-tolerant GFS/HDFS of Slides 17–24. The scheduler tries to schedule map tasks **"close"** to the physical storage location of their input chunk: "bring the computation to the data" (Slides 16, 23, 25). (**Beyond the slides:** "close" means the same machine first, then the same rack.)
2. **Intermediate results** (the Map output) are stored on the **local file system** of the map worker, *not* in the DFS. **Beyond the slides:** they are temporary and can be recomputed, so replicating them would be wasted work. The consequence, that they vanish with the machine, drives the failure rules below.
3. **Output is often input to another Map-Reduce task**, so large problems are solved by chains or iterations of jobs.

Data	Stored on	Replicated?	Survives a worker crash?
Input chunks	**DFS**	yes (2x or 3x)	**yes**
Intermediate (Map output)	local disk of the map worker	no	no
Final output	**DFS**	yes	**yes**

Coordination: the master

Lecture 2 · Slide 56

A job has one **master node** (the coordinator) and many **worker nodes** that run map and reduce tasks. The master takes care of coordination:

- **Task status:** every task is **idle** (waiting to be assigned), **in-progress** or **completed**.
- **Idle tasks** get scheduled as workers become available.
- When a **map task completes**, it sends the master the **location and sizes of its intermediate files** (Beyond the slides: one file region per reduce task). The data itself stays on the map worker's local disk.
- The master **pushes this information to the reducers**. (**Beyond the slides:** only the *locations* are pushed; the reducers then read the data from the map workers' disks.)
- The master **pings workers periodically** to detect failures. A worker that does not answer is declared failed.

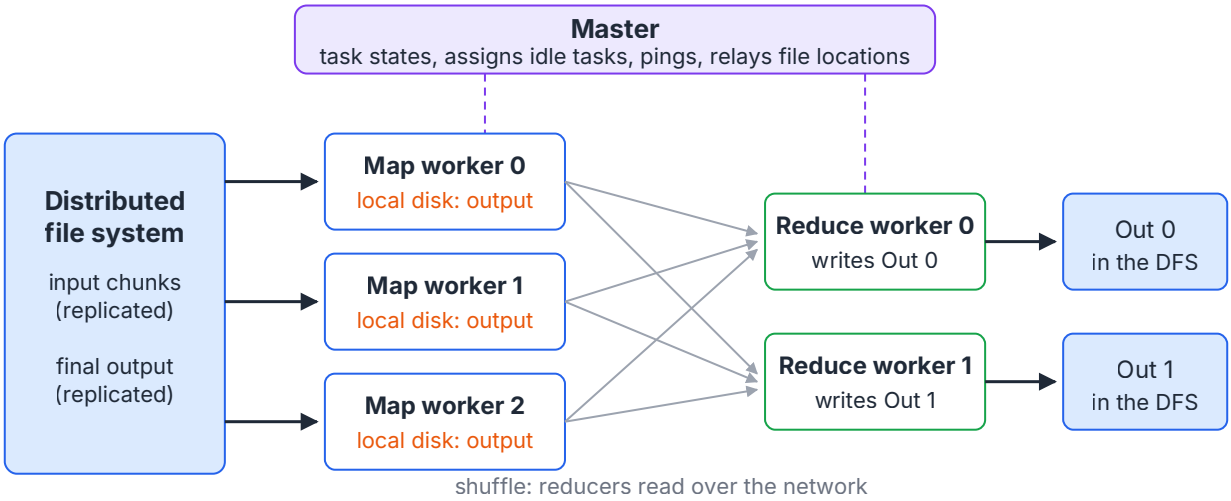


Figure L2-9 — Data flow and master (Slides 54–56). Input and output live in the DFS (blue); intermediate output sits on each map worker's local disk (orange). Dashed purple lines: the master's coordination.

Practice (beyond the slides) — one map task finishes Lecture 2 · Slide 56

Map task 0 (state idle) is assigned to a free worker, becomes in-progress, reads its chunk (ideally from that machine's own disk) and writes its intermediate pairs into **R** local files, one per reduce task (R = 2 here, so 2 files). On completion it tells the master "done; my 2 files are here, with these sizes". The master marks it completed and passes the file locations to Reduce 0 and Reduce 1. The reducers fetch their file from every map worker (3 maps × 2 reducers = 6 fetches).

Dealing with failures

Lecture 2 · Slide 57

Jobs run for a long time on thousands of cheap machines, where failures are routine (Slides 12–13), so the framework must recover automatically. Slide 57 gives three cases, from least to most serious. The master finds failed workers by its periodic pings (Slide 56); a worker that does not answer is dead.

Failure	What the slide says	Why (the reason the slide leaves implicit)
Map worker fails	Map tasks **completed or in-progress** at that worker are reset to idle; idle tasks are eventually rescheduled on other workers.	A map task's output is on the **local disk** of that worker and is lost with the machine. A completed map task is therefore *not* done any more: reducers still need its output.
Reduce worker fails	Only **in-progress** tasks are reset to idle; the idle reduce task is restarted on other workers.	The output of a **completed** reduce task was already written to the **distributed file system**, which is replicated, so it survives. Nothing needs redoing.
Master fails	The Map-Reduce job is **aborted** and the client is notified.	The master holds all the task states. There is a single master (a single point of failure). Beyond the slides: a failure of one machine during any single job is unlikely (Dean and Ghemawat). The client must resubmit.

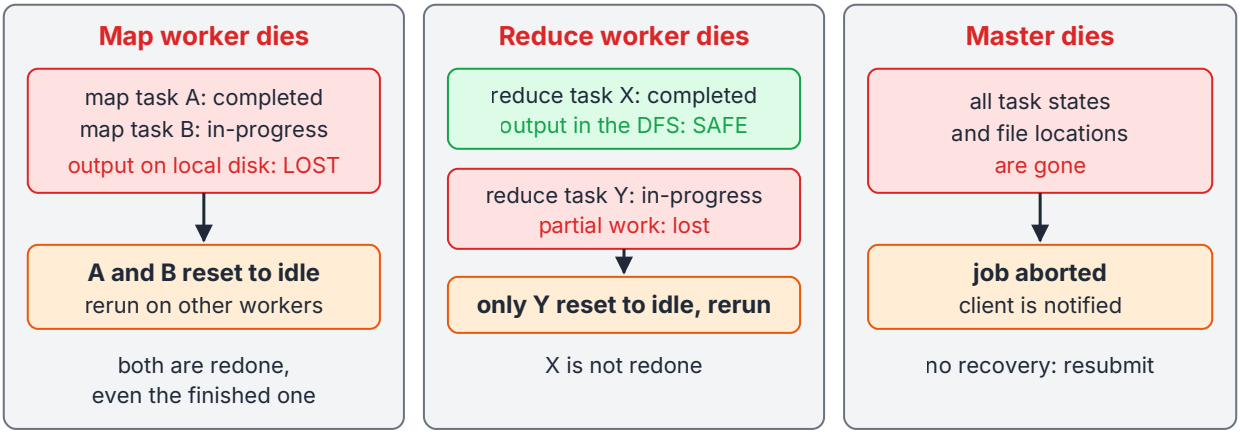


Figure L2-10 — What is redone after each failure (Slide 57).

KEY IDEA: WHY COMPLETED MAP TASKS ARE REDONE BUT COMPLETED REDUCE TASKS ARE NOT

It is a question of **where the output lives**. A map task's output is written to the **local disk** of its worker (Slide 55); when the worker dies, the disk is unreachable, so the work is lost even though the task was "completed", and reducers that have not yet fetched that output need it. A reduce task's output goes to the **distributed file system** (Slide 55), which keeps replicas on other machines, so once completed it is safe and need not be redone. In-progress tasks, of either kind, have produced nothing durable yet and are always redone.

Worked example — a map worker dies Lecture 2 · Slide 57

Worker W ran map task A (completed) and was running map task B (in-progress) when it stopped answering pings. The master resets **both** A and B to idle, and gives them to other workers as they become free. Reduce tasks that still needed A's files are told the new location after A reruns (**Beyond the slides**). If instead W had been a reduce worker running reduce task X (completed) and Y (in-progress), only Y would be reset to idle.

COMMON MISTAKES

- Saying "completed tasks are never redone". For **map** tasks they are, because their output is on the failed worker's local disk.
- Saying "reduce tasks are redone if the reduce worker fails" without the qualifier **only the in-progress ones**.
- Forgetting that master failure is **not recovered**: the job is aborted and the client notified.
- Writing that intermediate data is stored in the DFS. It is on the **local FS**; only input and final output are in the DFS.

EXAM TIP

For a "what happens when worker X fails?" question, answer in the order: **(1)** the master detects it by missing pings, **(2)** which tasks are reset to idle (map: completed and in-progress; reduce: in-progress only), **(3)** the reason (local disk versus DFS), **(4)** idle tasks are rescheduled on other workers as they become available. Slide wording note: "Idle Reduce task is restarted" is loose; the task is started afresh because it was reset to idle.

QUICK RECAP (SLIDES 26–57)

- Word count is the model example: Map emits (word, 1) per word; the framework groups by key; Reduce sums.
- Four stages: Map, group by key (sort and shuffle), Reduce, write result. The programmer writes **only** Map and Reduce; the outline stays the same.
- Signatures: Map(k, v) → list of (k', v'), one call per input pair; Reduce(k', list of v') → (k', v''), one call per unique key.
- Choose the **key** as "what must meet to give one answer": word, host, 5-word sequence.
- Parallel run: Map tasks → partitioning function → shuffle (every map to every reduce) → sort and group → Reduce tasks.
- The environment handles scheduling, communication, partitioning, group by key and failures.
- Input and final output are in the DFS (map tasks scheduled close to the data); intermediate data is on local disks.
- The master tracks task states (idle, in-progress, completed), receives file locations and sizes from finished map tasks, and pings workers.
- Map worker fails: completed and in-progress map tasks redone. Reduce worker fails: only in-progress reduce tasks redone. Master fails: job aborted.

Fine-granularity tasks and pipelining

Lecture 2 · Slide 58

The previous slides showed how the master hands out work and what happens when a worker fails (Slide 57). Slide 58 asks a design question: *how big should one task be?* Its single bullet gives the answer: **fine granularity tasks: map tasks >> machines**. In words: cut the job into **many more map tasks than there are machines**, each task small.

BEGINNER INTUITION

Four removal workers must empty a house. If you give each worker one whole floor, the worker with the cluttered attic finishes last while the others stand around. If you instead cut the work into 40 small jobs ("this room", "that cupboard") on a whiteboard, whoever finishes grabs the next job, and everybody is busy until the very end. Small tasks = good load balancing. And if one worker falls ill, only a small job has to be redone.

Reading the Gantt chart on Slide 58

Lecture 2 · Slide 58

The slide has no legend, so here is how to read it. Time runs left to right. The **User Program** calls `MapReduce()` and then simply waits. The **Master** assigns tasks to four worker machines. Green bars are map work, red bars are "reads", blue bars are reduce work. **Read i.j** means "reduce task *i* reads (fetches) the output of map task *j*" (this is the shuffle, seen from the reducer's side).

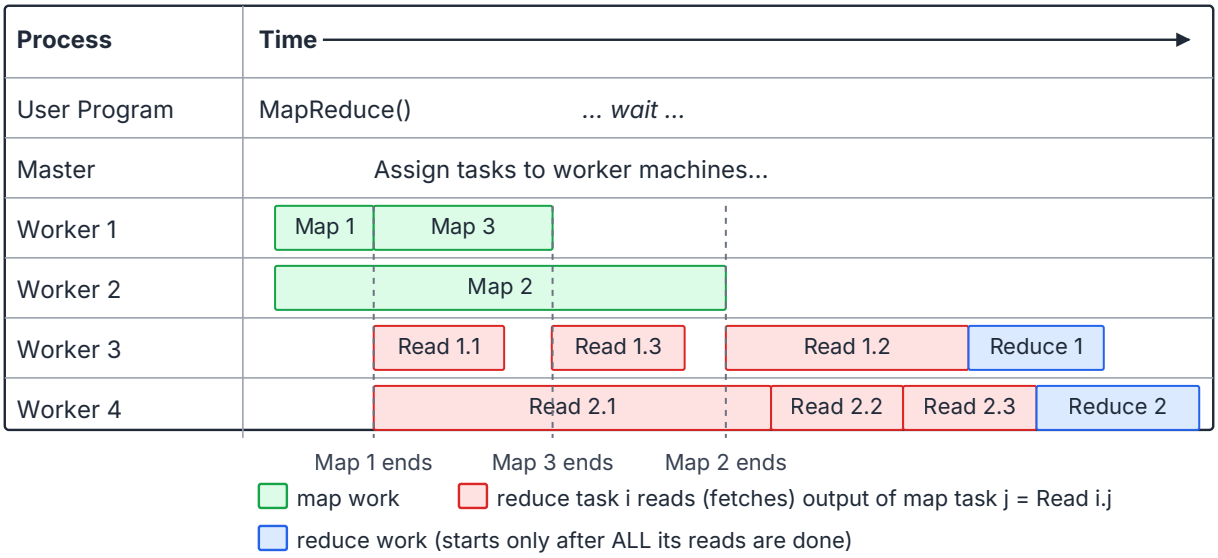


Figure L2-11 (redrawn from Slide 58). Time runs left to right. Three map tasks share two workers; each reduce task starts copying a map output the moment that map task ends, so copying overlaps with the map work still running.

Three things to notice:

1. **More tasks than machines.** Worker 1 runs Map 1 and then Map 3, while Worker 2 runs the long Map 2. Three map tasks share two machines. The machine that finished early took the next task instead of idling.
2. **Pipelining.** Reads begin as soon as the matching map task ends. Read 1.1 and Read 2.1 start when Map 1 ends; Read 1.3 starts when Map 3 ends; Read 1.2 starts when Map 2 ends. So network copying overlaps with map computation still going on elsewhere. The shuffle does *not* wait for all maps to finish.
3. **The reduce barrier.** A Reduce bar starts only after *all* of that reduce task's reads are done (Reduce 1 after Read 1.2, Reduce 2 after Read 2.3), because the reduce function needs every value of a key.

The white gaps in Worker 3's row are idle time waiting for the next map output. Worker 4's Read 2.2 starts later than Map 2 ends because Worker 4 is still busy with the long Read 2.1 (it reads one output at a time).

DEFINITION: FINE GRANULARITY AND PIPELINING

Fine granularity: splitting a job into many small tasks (number of map tasks M much larger than the number of machines). **Pipelining:** overlapping stages, so that the shuffle of finished map outputs runs while other map tasks are still computing. Only the reduce function has to wait for the group-by-key barrier.

EXAM TIP

"Why use many more map tasks than machines?" Give two reasons: (1) **dynamic load balancing** (fast machines simply take more tasks), (2) **fast recovery** (a failed worker loses only small tasks, and they can be re-run in parallel on many other machines). Slide 69 repeats both; see the rule of thumb later in this part.

COMMON MISTAKE

Do not write that reduce tasks start *computing* before all maps finish. Only the *copying* (reads) starts early. The reduce function itself needs all values for each key.

Refinement: combiners

Lecture 2 · Slides 59–65

Slides 59 to 65 form one story: *what* a combiner is (59–61), *when* it is allowed (62), and why it works for sum, needs a trick for average and fails for median (62–65).

The problem: many pairs for the same key

Lecture 2 · Slide 59

Slide 59: "Often a Map task will produce many pairs of the form (k,v_1), (k,v_2), ... for the same key k, e.g. popular words in the word count example." Think of a chunk of English text: the word "the" appears hundreds of times, so the mapper emits hundreds of `("the", 1)` pairs. Each pair is written to local disk, sent across the network (the bottleneck, Lecture 2 Slide 14) and sorted at the reducer, although they could have been added up locally first.

The slide's figure is the generic data flow: three map tasks (each calls the map function **M** several times and has a "Partitioning Function" that decides the target reduce task), arrows crossing to two reduce tasks (each does "Sort and Group", then calls the reduce function **R** once per key). No combiner is drawn. The point is the repeated keys. Counting the pairs in that figure:

Map task	Pairs emitted (keys)	Pairs without combiner	Pairs with a combiner (one per distinct key)
Map Task 1	k1, k1, k2, k1	4	2 (k1, k2)
Map Task 2	k3, k4, k4, k5	4	3 (k3, k4, k5)
Map Task 3	k4, k1, k3	3	3
Total shuffled	k1 ×4, k4 ×3, k3 ×2, k2 ×1, k5 ×1	**11**	**8**

The right-hand column is Practice (assuming the values of one key can be merged, which Slide 59 itself does not claim). The left column is read off the figure; the key counts agree with the reduce side of the figure (Reduce Task 2 receives `k1:v,v,v,v` and `k3:v,v`; Reduce Task 1 receives k2, `k4:v,v,v` and k5). Note also that a map call may emit zero pairs: the empty second cell of Map Task 1.

Definition: the combiner

Lecture 2 · Slide 60

Slide 60: "Can save network time by **pre-aggregating values in the mapper**: `combine(k, list(`v_1`)) →` v_2."

DEFINITION: COMBINER

A **combiner** is a mini-reducer that runs **on the mapper's own machine**, right after the map function and **before anything is sent over the network**. For each key k that this one mapper produced, it takes the mapper's list of values for k and replaces it by a **single** value. It cuts the number of pairs to be shuffled, which saves network time, disk I/O and sorting work.

- **Where it runs:** on the mapper machine, on the output of that machine's map tasks only. It never sees values from other mappers.
- **What it is:** an *optimisation*, not part of the logic. The final answer must be identical with or without it.
- **Beyond the slides:** the framework is free to run the combiner zero, one or several times on any subset of a key's values. Your program must therefore be correct whatever happens. The combiner's output must have the **same type** as the map output value (the reducer receives combiner outputs in place of, or mixed with, raw map outputs).

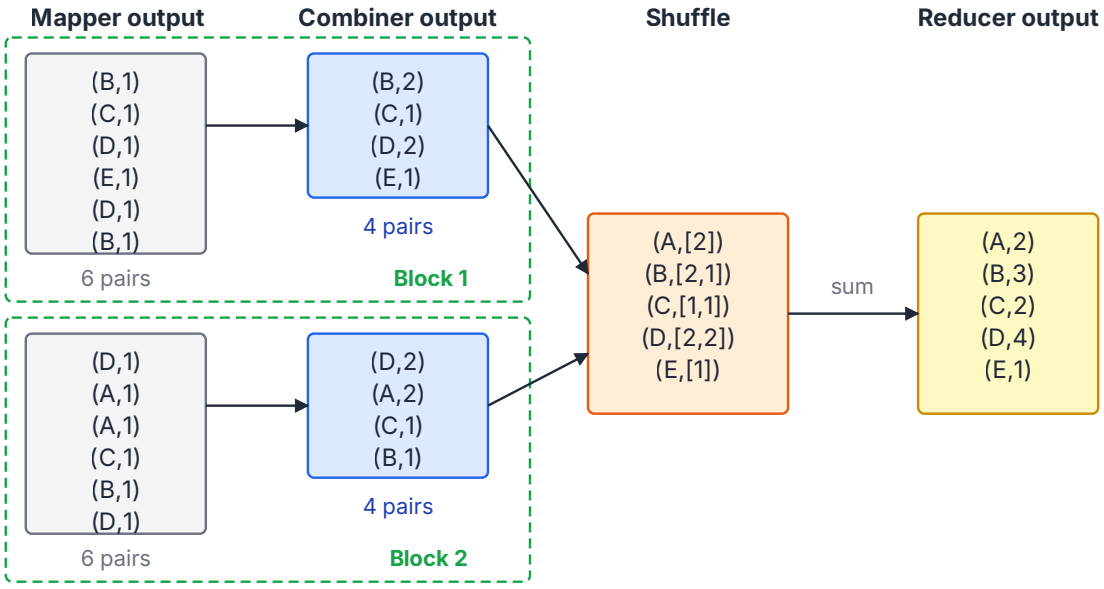
Worked example: word count with a combiner

Lecture 2 · Slide 61

Slide 61 draws two input blocks, each with its own Mapper and Combiner (each green frame is one machine), then Shuffle and Reducer. The mapper turns each input pair into (its *second* item, 1). Follow Block 1:

1. Input: `(A,B) (A,C) (A,D) (B,E) (B,D) (C,B)`.
2. Mapper output (6 pairs): `(B,1) (C,1) (D,1) (E,1) (D,1) (B,1)`.
3. Combiner: the key B occurs twice, D twice, C and E once. Output (4 pairs): `(B,2) (C,1) (D,2) (E,1)`.

Block 2 gives mapper output `(D,1) (A,1) (A,1) (C,1) (B,1) (D,1)` and combiner output `(D,2) (A,2) (C,1) (B,1)`. The Shuffle groups the 8 combined pairs by key: `(A,[2]) (B,[2,1]) (C,[1,1]) (D,[2,2]) (E,[1])`. The Reducer adds each list: `(A,2) (B,3) (C,2) (D,4) (E,1)`.



Each green dashed frame is one machine: the Combiner runs there, before the network.
Pairs crossing the network: 6 + 6 = 12 without a combiner, 4 + 4 = 8 with one.

Figure L2-12 (data of Slide 61). The combiner shrinks each mapper's output before the shuffle; the reducer simply adds the partial counts (B: 2 + 1 = 3).

Slide 61's conclusion: **much less data needs to be copied and shuffled**. Here 12 pairs shrink to 8 (a third fewer); with real text and very skewed word frequencies the saving is far larger. The reducer's job did not change; it just adds a few bigger numbers (B: 2 + 1 instead of 1 + 1 + 1).

> **SLIDE ERRATUM (MINOR)**
> Slide 61 calls this "our word counting example", but the mapper emits only the *second* item of each input pair (`(A,B)` gives `(B,1)`). So it counts how often each letter occurs in the second position, not all words. It is a schematic; every number on the slide is internally consistent (checked above).

When is a combiner legal? Commutative and associative

Lecture 2 · Slide 62

Slide 62: "Combiner is usually the same as the reduce function. Works only if the reduce function is **commutative and associative**." Example: Sum, where `a + b = b + a` and `(a + b) + c = a + (b + c)`.

Why this condition? The framework may combine any subset of a key's values, in any grouping and any order, and then the reducer finishes with the partial results. The final answer is guaranteed to be unchanged only if "combine the pieces" equals "combine everything at once".

- **Commutative** (order does not matter): values arrive from different mappers in arbitrary order.
- **Associative** (grouping does not matter): lets you pre-add one part, e.g. `(1+2+3) + 4 = 1 + (2+3+4)`.

Slide 62's hand drawing shows exactly this: the row of values of one key is cut in two, each half is added separately (a circled "+"), and the two partial sums are added. For word count the combiner and the reducer are the same function: sum.

Beyond the slides: other functions with both properties include count (a sum of 1s), maximum, minimum, product and set union. They all work as their own combiner.

Average: not combinable directly, but fixable

Lecture 2 · Slides 63–64

Slide 63 states "Average is not commutative and associative" and draws the wrong idea: each mapper averages its part (ave_1, ave_2) and the reducer computes `(ave1 + ave2) / 2`, marked with a big X. It is wrong because an average forgets *how many* values stand behind it. Numeric check (Practice): the values 1, 2, 3 on mapper 1 and 4 on mapper 2 give averages 2 and 4, and (2 + 4)/2 = **3**, but the true average of 1, 2, 3, 4 is 10/4 = **2.5**. See Figure L2-13 (middle row).

SLIDE ERRATUM (WORDING)

The slide's phrase is shorthand. Strictly, the two-argument average `(a+b)/2` *is* commutative; what fails is associativity, and more importantly that merging partial averages ignores their sizes. The correct statement: **an average cannot be computed by combining partial averages.**

Slide 64 gives the fix: carry **(Sum, count)** pairs, then `ave = Sum / count`.

1. **Map:** emit `(k, (value, 1))`, not the bare value.
2. **Combiner:** for each key add the pairs *component-wise*: $(s_1, c_1) + (s_2, c_2) = (s_1+s_2, c_1+c_2)$. This operation is commutative and associative, so it is a legal combiner. **No division here.**
3. **Reducer:** add all pairs the same way, then divide once: $Average = (Sum_1 + Sum_2 + Sum_3) / (count_1 + count_2 + count_3)$ (the slide's three-part drawing).

Check: {1,2,3} gives (6, 3); {4} gives (4, 1); combined (10, 4); 10/4 = 2.5, correct.

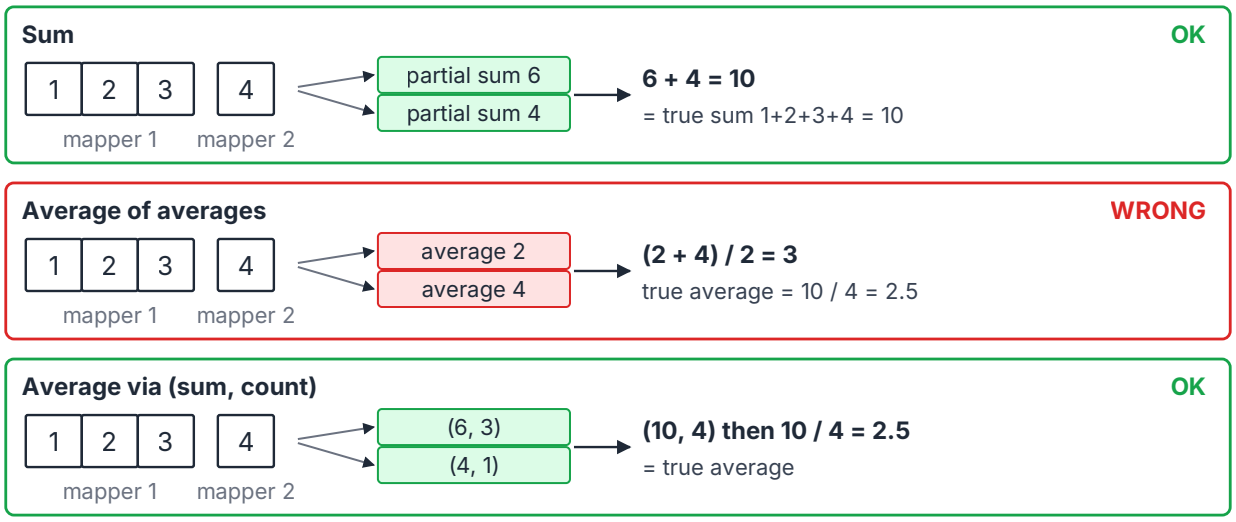


Figure L2-13 (idea of Slides 62-64, with Practice numbers 1, 2, 3, 4). Each mapper summarises its part; the reducer combines the summaries.

KEY IDEA

Any function can use a combiner if it can be rewritten as **a commutative and associative merge of small partial summaries, followed by a cheap final step** (for average: merge by adding (sum, count), finish by dividing). Note that here the combiner is *not* the same as the reducer (the reducer divides, the combiner must not), so "combiner = reduce function" on Slide 62 holds literally only for sum-like functions.

Median: no combiner trick

Lecture 2 · Slide 65

Slide 65: for the median "there is actually a mathematical proof that it is impossible to split it into commutative and associative functions, so we cannot use the combiner trick." (The proof is asserted, not shown; you only need the conclusion.) Intuition: no small, fixed-size summary of a part of the data lets you recover the overall middle value. Practice check: parts {1,2,3} and {10} have medians 2 and 10; the median of those medians is 6, but the true median of {1,2,3,10} is 2.5. So for a median the reducer must receive **all** the values of the key (full shuffle, no combiner).

Reduce function	Combiner allowed?	What the mapper-side combiner keeps
Sum, count, max, min (slide: sum)	**Yes**	the partial sum / count / max / min
Average	**Yes, with the trick**	the pair (sum, count); divide only in the reducer
Median	No	nothing smaller than all the values

EXAM TIP: THE 3-QUESTION TEST

Before using a combiner ask: (1) Is the reduce operation **commutative and associative**? (2) Does the combiner output have the **same type** as the map output value? (3) Is the answer the **same with or without** it (it may run 0, 1 or many times)? If you must justify "average cannot be combined directly", use a two-part counterexample with different part sizes (as above). Never answer "average is not associative" without showing the numbers.

Try it yourself first: cover the solution of the next block, then write down what a combiner is, what it saves on Slide 61's data, and what happens for sum, average and median. Compare with the model solution.

Quiz L2-Q7 **Combiners: sum vs average vs median** Lecture 2 · Slides 59–65

QUESTION (AS ON THE SLIDES)

Refinement: Combiners (Slides 59–65). These seven slides contain no numbered exercise; they build one argument: *what a combiner is, when the reduce function may be reused as one, and why that works for sum, fails for average (but can be repaired) and is impossible for median.* Any link of this chain can be asked in an exam, so this block solves the whole chain. What the slides give:

- **Slide 59:** "Often a Map task will produce many pairs of the form (k,v_1), (k,v_2), … for the same key k; e.g. popular words in the word count example." The figure shows three Map tasks feeding two Reduce tasks; Map Task 1 itself emits `k1:v` three times and Reduce Task 2 receives `k1:v,v,v,v`.
- **Slide 60:** "Can save network time by pre-aggregating values in the mapper: combine(k, list(v_1)) → v_2".
- **Slide 61:** "Back to our word counting example: Combiner combines the values of all keys of a single mapper (single machine) … Much less data needs to be copied and shuffled!" plus the figure whose data are tabulated below.
- **Slide 62:** "Combiner is usually same as the reduce function. Works only if reduce function is commutative and associative." Sum: a + b = b + a and (a + b) + c = a + (b + c). A hand-drawing shows the row of values of one key cut by a vertical line into two parts, each part fed into a circled "+", and the two partial results joined again.
- **Slide 63:** "Average is not commutative and associative." Hand-drawing: the two parts give ave_1 and ave_2, and (ave_1 + ave_2)/2 is circled and marked with a big X (wrong).
- **Slide 64:** Average: (Sum, count); ave = sum / count. Drawing with three parts, each giving a circled (Sum_i, $count_i$): Average = (Sum_1 + Sum_2 + Sum_3) / ($count_1$ + $count_2$ + $count_3$).
- **Slide 65:** Median: "There is actually a mathematical proof that it is impossible to split it into commutative and associative functions. So we cannot use the combiner 'trick'."

Data printed in the Slide 61 figure (two input blocks, each processed by its own Mapper and Combiner; then Shuffle and Reducer):

	Input records	Mapper output	Combiner output
Block 1	`(A,B) (A,C) (A,D) (B,E) (B,D) (C,B)`	`(B,1) (C,1) (D,1) (E,1) (D,1) (B,1)`	`(B,2) (C,1) (D,2) (E,1)`
Block 2	`(C,D) (C,A) (D,A) (E,C) (E,B) (E,D)`	`(D,1) (A,1) (A,1) (C,1) (B,1) (D,1)`	`(D,2) (A,2) (C,1) (B,1)`
Shuffle	`(A,[2]) (B,[2,1]) (C,[1,1]) (D,[2,2]) (E,[1])`		
Reducer	`(A,2) (B,3) (C,2) (D,4) (E,1)`		

Reading the figure: each Mapper turns a record such as (A,B) into (its second letter, 1), so the job simply counts how often each letter occurs in the second position; the five letters A–E play the role of "words". The slides give no numbers for the average and median discussions beyond the hand-drawings, so the numbers used in parts (b)–(e) are Practice values written for this guide: the same five numbers `1, 2, 3, 4, 5` are used for sum, average and median so that the three can be compared.

(a) What is a combiner, **where does it run** and **what does it save**? Apply it to the two blocks of Slide 61: how many key–value pairs have to be shuffled with and without the combiner, and what does the Reducer output?

(b) When can the reduce function be used as a combiner? Show that **sum** satisfies the condition, using word count (a mapper that emits (the,1) three times).

(c) Show that **average** does not satisfy it: give a numerical counterexample (the values 1, 2, 3, 4, 5 split as {1,2} and {3,4,5}) and prove the failure in general.

(d) Show how the average **can** be computed correctly with a combiner (Slide 64).

(e) Why can the **median** not use the combiner trick (Slide 65)?

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Imagine a school election. Each classroom (a "mapper") first tallies its own ballots and sends *one slip per candidate* to the central office (the "reducer"), instead of carrying every single ballot across town (the "network"). That is a combiner. It is safe for **counting** votes: partial tallies simply add up, in any order and any grouping. It is **not** safe for "the average age of the voters": a class of 2 students and a class of 3 students must not be averaged as if they were equal, because an average has forgotten how many people stand behind it; the fix is to send "total age and head-count" instead. And for the **median** age, no small slip from a classroom is ever enough: the central office needs the actual ages. Throughout, the rule is: **the answer must not change, only the traffic may.**

1. **Idea — what a combiner is and where it runs (Slides 59–60).** In the Map-Reduce flow (Slides 36–44, 51) every Map call emits (key, value) pairs, the system moves them over the network to the reducers (the **shuffle** = group by key) and one Reduce call processes all values of one key. A Map task often emits the *same key many times* (Slide 59): in word count it emits (the,1) every single time it reads "the". In the Slide 59 figure Map Task 1 emits `k1:v` three times, so Reduce Task 2 receives `k1:v,v,v` (three values from Task 1 and one from Task 3), and each of those pairs crossed the network separately. A **combiner** is a small "mini-reduce" that runs **on the same machine as the map task, on that map task's own output, before anything is sent**. Slide 60 writes it **combine(k, list(v_1)) → v_2**: for one key k it takes the *list* of values this one mapper produced and returns *one* value v_2 that stands for the whole list ("pre-aggregating values in the mapper").
   Slide 59 check (counting only; the figure's values are just "v", so assume they can be added): the three Map tasks emit 4 + 4 + 3 = **11** pairs. With a combiner Task 1 sends k1, k2 (2 pairs), Task 2 sends k3, k4, k5 (3 pairs) and Task 3 sends k4, k1, k3 (3 pairs) = **8** pairs; Reduce Task 2 would then receive 2 values for k1 instead of 4.
2. **Part (a): apply the combiner to the Slide 61 data and count pairs.** A combiner groups *its own mapper's* pairs by key and adds the values of each key. For the two blocks:

Block	Mapper output (raw)	What the combiner does	Combiner output	pairs: raw → combined
1	`(B,1) (C,1) (D,1) (E,1) (D,1) (B,1)`	B twice → 1+1 = 2; D twice → 1+1 = 2; C, E once	`(B,2) (C,1) (D,2) (E,1)`	6 → 4
2	`(D,1) (A,1) (A,1) (C,1) (B,1) (D,1)`	D twice → 2; A twice → 2; C, B once	`(D,2) (A,2) (C,1) (B,1)`	6 → 4
Total pairs sent to the shuffle				**12 → 8**

   Shuffle (group by key) of the 8 combined pairs: A: [2]; B: [2,1] (2 from Block 1, 1 from Block 2); C: [1,1]; D: [2,2]; E: [1]. **Reducer** (adds the values of each key; one Reduce call per unique key, Slide 44): A = 2; B = 2+1 = 3; C = 1+1 = 2; D = 2+2 = 4; E = 1. Output **(A,2) (B,3) (C,2) (D,4) (E,1)**, exactly the Slide 61 result. Check: 2+3+2+4+1 = 12 = number of Mapper output pairs ✓. *Without* the combiner the Reducer would receive (B,[1,1,1]), (D,[1,1,1,1]), … (12 pairs) and print the same five totals.

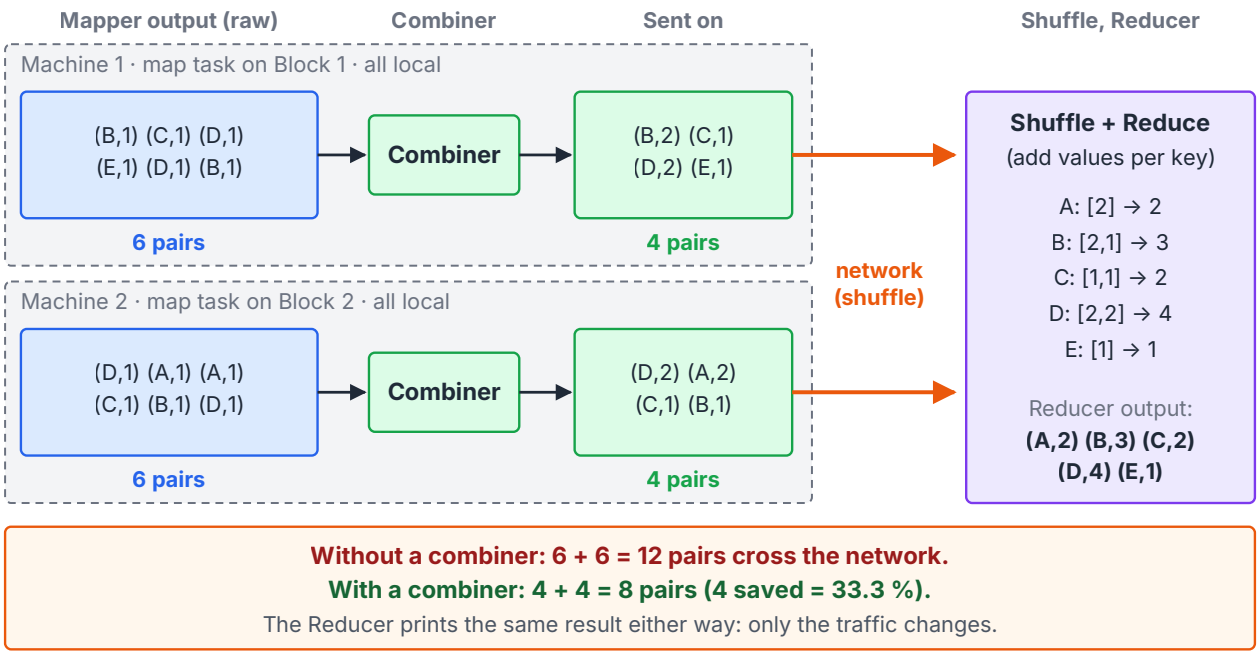


Figure L2-Q7-1. The Slide 61 data. The combiner runs on each map machine (dashed boxes) before the shuffle; 8 instead of 12 pairs travel, and the reducer still adds the partial counts of the two machines (B: 2 + 1 = 3).

- **Where it runs:** on the machine of *each* map task, on that task's output only ("a single mapper (single machine)", Slide 61). It cannot merge Block 1's B with Block 2's B: the two partial values 2 and 1 still travel, so the **reducer must still do the final addition** (B: 2+1 = 3). The combiner does not replace the reducer.
- **What it saves:** network time (Slide 60) and shuffle work (copying, sorting, grouping): "much less data needs to be copied and shuffled" (Slide 61). Here 4 of 12 pairs (one third, 33.3 %) disappear; for a really popular word the saving is enormous (1,000,000 pairs (the,1) from one mapper become the single pair (the,1000000)). At most one pair per distinct key per mapper is ever sent.
- **What it does not change:** the number of Map calls (the map work is still done), the number of Reduce calls (one per unique key) and, when the combiner is legal (next steps), the final output. It is a pure optimisation.

3. **Part (b), formula / idea: the condition "commutative and associative" (Slide 62).** The combiner is "usually the same as the reduce function". The system, not the programmer, decides how the values of a key are split between mappers and in what order the partial results are merged. The final answer must not depend on those accidents. So for *every* way of cutting the values $v_1, \ldots, v_n$ of a key into groups we need

$$\text{reduce}(v_1, \ldots, v_n) = \text{reduce}(\text{combine}(\text{group 1}), \text{combine}(\text{group 2}), \ldots).$$

Writing the two-value operation as $f(a, b)$, this holds when f is

FORMULA

Commutative (the *order* does not matter): $f(a, b) = f(b, a)$. **Associative** (the *grouping / brackets* do not matter): $f(f(a, b), c) = f(a, f(b, c))$. For sum, $f(a, b) = a + b$: $a + b = b + a$ and $(a + b) + c = a + (b + c)$ (Slide 62).

Commutative = values from different mappers may arrive in any order; associative = it does not matter which values were pre-combined together on which machine. Slide 62 draws exactly this: the row of values of key k is cut by a vertical line into two parts (two mappers); each part is "added" (circled +) and then the two partial sums are added again.

SLIDE ERRATUM (OMISSION, NOT AN ERROR)

Slide 62 gives the commutative/associative test but does not mention two practical rules. *Beyond the slides:* in Hadoop the combiner is optional and the system may run it zero, one or several times on a map task's output, and its output must have the same type as the Map output (the reducer cannot tell whether a combiner ran). So correctness must never depend on whether, or how often, it ran; this is the real reason for the condition above, and it is why the average (part d) needs the mapper to emit (value, 1) pairs from the start. Also, "usually the same as the reduce function" is not "always the same" (see part d).

COMMON MISTAKE

Do not write "the combiner must be identical to the reducer" or "the combiner may be anything". The test is: **would the final answer be the same however the values were split and ordered?**

4. **Part (b): show that sum works (word count).** For "+" on numbers, $a + b = b + a$ and $(a + b) + c = a + (b + c)$, so any order and any bracketing of the same numbers give the same total. Hence total = (sum of group 1) + (sum of group 2) + …, and "sum" can serve as combiner *and* reducer.
The required example: a mapper emits (the,1), (the,1), (the,1). The combiner replaces them by **(the, 1+1+1) = (the,3)** (3 pairs → 1). The brackets do not matter: (1+1)+1 = 2+1 = 3 and 1+(1+1) = 1+2 = 3. The same check on Slide 61, key B: it occurs twice at mapper 1 and once at mapper 2, combiner 1 gives 1+1 = 2, combiner 2 leaves 1, the reducer adds 2+1 = 3 = 1+1+1 ✓.
A complete small example (Practice, written for this guide): Map task 1 reads "the cat the dog the" and Map task 2 reads "the sat on the mat".

Map task	Raw Map output	After combiner	pairs
1	`(the,1)` `(cat,1)` `(the,1)` `(dog,1)` `(the,1)`	`(the,3)` `(cat,1)` `(dog,1)`	5 → 3
2	`(the,1)` `(sat,1)` `(on,1)` `(the,1)` `(mat,1)`	`(the,2)` `(sat,1)` `(on,1)` `(mat,1)`	5 → 4
Total pairs over the network			**10 → 7**

The reducer receives the: [3,2] and adds **3 + 2 = 5**; the other words appear once. Without any combiner it would add 1+1+1+1+1 = 5: **the same result**. Other bracketings agree: (1+1)+(1+1+1) = 5 = 1+(1+1+1+1).
Same idea with the numbers 1, 2, 3, 4, 5 (the values of one key, Slide 62 drawing): mapper 1 holds {1,2}, mapper 2 holds {3,4,5}. Partial sums 1+2 = 3 and 3+4+5 = 12; the reducer adds 3 + 12 = **15** = 1+2+3+4+5 ✓. (*Beyond the slides:* count, which is a sum of 1's, and also max, min and product are commutative and associative, so they can use their own reduce function as combiner too.)

5. **Part (c): average fails. First the numerical counterexample (Slide 63).** Suppose we naively use "average" as the combiner (combiner = reduce = average) for the values 1, 2, 3, 4, 5 of one key, split as in the Slide 62/63 drawings: mapper 1 gets {1,2}, mapper 2 gets {3,4,5}.

Quantity	Computation	Result
ave_1 (combiner at mapper 1)	(1 + 2) / 2	1.5
ave_2 (combiner at mapper 2)	(3 + 4 + 5) / 3 = 12 / 3	4
Average of the averages (the crossed-out recipe of Slide 63)	(ave_1 + ave_2) / 2 = (1.5 + 4) / 2 = 5.5 / 2	2.75
True average of all five values	(1 + 2 + 3 + 4 + 5) / 5 = 15 / 5	**3**

Since **2.75 ≠ 3**, using average as the combiner gives a wrong answer. The result also depends on *how* the values happened to be split, which the programmer does not control:

Split of {1,2,3,4,5}	ave_1	ave_2	$(ave_1 + ave_2)/2$	true average	correct?
{1} | {2,3,4,5}	1	3.5	2.25	3	×
{1,2} | {3,4,5} (Slide 63 style)	1.5	4	2.75	3	×
{1,2,3} | {4,5}	2	4.5	3.25	3	×
{1,2,3,4} | {5}	2.5	5	3.75	3	×
{1,5} | {2,3,4}	3	3	3	3	✓ **(by luck)**

Four splits give four different wrong answers; the fifth is right only by luck, because both group averages happen to equal 3. A correct combiner must give 3 for *every* split.

6. **Part (c): proof that "average of averages" is wrong in general.** Let mapper 1 hold n_1 values with average a_1 (so their sum is n_1a_1) and mapper 2 hold n_2 values with average a_2. Then

   true average $T = \dfrac{n_1a_1 + n_2a_2}{n_1 + n_2}$ (a *weighted* average: each group counts in proportion to its size)

   average of averages $W = \dfrac{a_1 + a_2}{2}$ (both groups get equal weight)

   Subtract, using the common denominator $2(n_1 + n_2)$:

   $$T - W = \frac{2(n_1a_1 + n_2a_2) - (n_1 + n_2)(a_1 + a_2)}{2(n_1 + n_2)} = \frac{n_1a_1 + n_2a_2 - n_1a_2 - n_2a_1}{2(n_1 + n_2)} = \frac{(n_1 - n_2)(a_1 - a_2)}{2(n_1 + n_2)}.$$

   So $W = T$ only when $n_1 = n_2$ (equal group sizes) or $a_1 = a_2$ (equal averages). In every other case the combiner is wrong, and since the system chooses the split we can guarantee neither. Numeric check with our data ($n_1 = 2$, $a_1 = 1.5$, $n_2 = 3$, $a_2 = 4$): $(2-3)(1.5-4)/(2\cdot5) = (-1)(-2.5)/10 = 0.25 = 3 - 2.75$ ✓.

   Two more ways to see the same failure.

   - **Not associative (Slide 63's claim).** The two-argument average $avg(a,b) = (a+b)/2$ *is* commutative (avg(1,2) = 1.5 = avg(2,1)) but it is **not associative**: avg(avg(1,2),3) = avg(1.5,3) = 2.25, whereas avg(1,avg(2,3)) = avg(1,2.5) = 1.75: two different answers for the same three numbers (and neither equals their true average 2). Different brackets = different ways of pre-combining on different machines, so the condition of the "Formula" box is violated.
   - **An average forgets how many values it stands for.** The pair of partial averages (1.5, 4) arises from {1,2} | {3,4,5} (true average 15/5 = 3) and equally from {1,2} | {4} (true average 7/3 ≈ 2.33). Same partial results, different correct answers, so *no* formula that uses only the partial averages can be right. The missing information is the count.

> **KEY IDEA**
> A plain average has **forgotten its weight**. The average 1.5 stands for 2 values and the average 4 stands for 3 values, so they must not get equal weight. The fix is to carry the weight (the count) alongside.

> **SLIDE ERRATUM (WORDING)**
> Slide 63 says "Average is not commutative and associative". Read it as "average is **not** (commutative **and** associative)": the failing property is **associativity**, while the two-argument average is commutative. A safe exam sentence: "averaging partial averages is wrong because it ignores how many values each partial average represents; equivalently avg is not associative."

7. **Part (d): the fix, carry (sum, count) and divide at the very end (Slide 64).** Sum and count are each commutative and associative, so carry *both* and compute the average only once, after everything has been merged: **ave = sum / count**.
   - **Map:** turn every value v into the pair $(v, 1)$ = (sum, count) of a one-element group. (*Beyond the slides:* so the Map output values are already (sum, count) pairs; this keeps the combiner's input type equal to its output type.)
   - **Adding pairs** means adding each coordinate separately: $(s_1,c_1) + (s_2,c_2) = (s_1+s_2,\ c_1+c_2)$.

- **Commutative:** $(s_1+s_2, c_1+c_2) = (s_2+s_1, c_2+c_1)$, because ordinary + is commutative in each coordinate.
- **Associative:** $((s_1,c_1) + (s_2,c_2)) + (s_3,c_3) = ((s_1+s_2)+s_3, (c_1+c_2)+c_3) = (s_1+(s_2+s_3), c_1+(c_2+c_3)) = (s_1,c_1) + ((s_2,c_2) + (s_3,c_3))$, again because + is associative in each coordinate.
- **Combiner:** add all pairs of the same key on the mapper's machine and output *one pair* (no division!). **Reducer:** add all pairs it receives, then divide **once**: ave = sum/count.

Run on 1, 2, 3, 4, 5 split {1,2} | {3,4,5}: mapper 1: (1,1) + (2,1) = **(3,2)**; mapper 2: (3,1) + (4,1) + (5,1) = **(12,3)**. Reducer: (3,2) + (12,3) = (3+12, 2+3) = **(15,5)**; then ave = 15/5 = **3** ✓, the true average (Figure L2-Q7-2, right panel).
Other splits give the same pair. {1,2,3} | {4,5}: (6,3) + (9,2) = (15,5). Three mappers {1} | {2,3} | {4,5}: partial pairs (1,1), (5,2), (9,2). Left bracketing: ((1,1) + (5,2)) + (9,2) = (6,3) + (9,2) = (15,5). Right bracketing: (1,1) + ((5,2) + (9,2)) = (1,1) + (14,4) = (15,5). Reversed order: (9,2) + (5,2) + (1,1) = (15,5). All give 15/5 = 3. (Computer check for this guide: all 52 ways of splitting {1,...,5} into groups, in every order and bracketing, always give (15,5).) This is exactly the Slide 64 drawing: Average = $(Sum_1+Sum_2+Sum_3)/(count_1+count_2+count_3)$.
Link with the proof above: a pair (sum, count) is an average together with its weight, since sum = count × average. The formula for T is exactly the weighted average of the group averages: (2 × 1.5 + 3 × 4)/5 = 15/5 = 3.

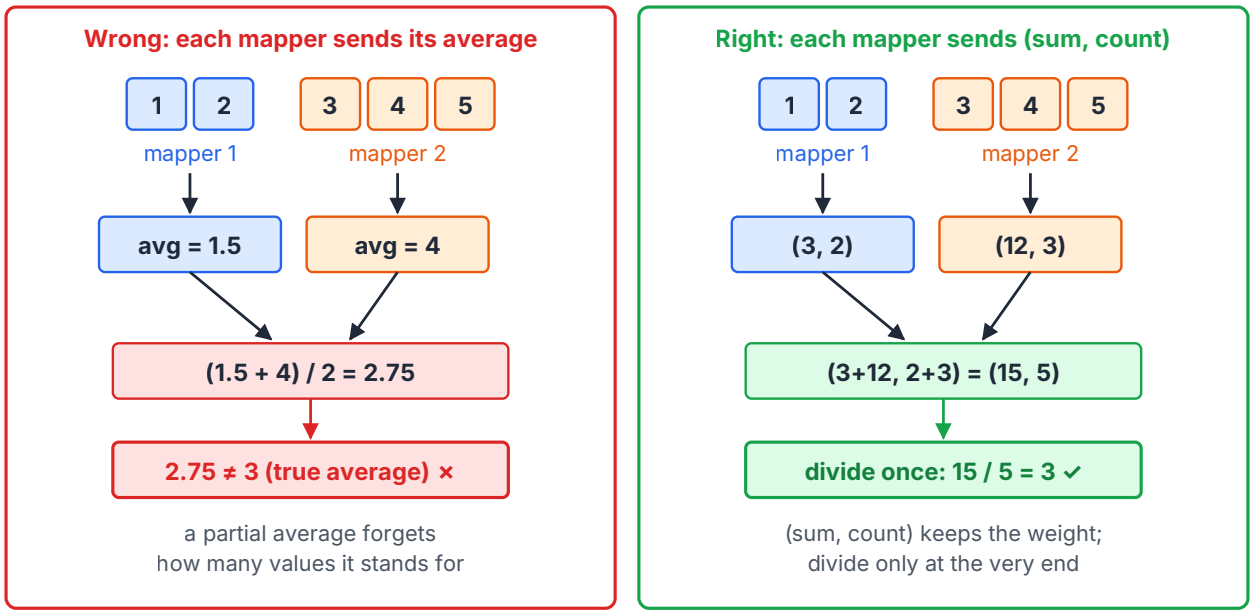


Figure L2-Q7-2. Same data, same split {1,2} | {3,4,5}. Left: combining by averaging gives 2.75 (wrong). Right: combining by adding (sum, count) pairs and dividing once at the end gives 3 (correct).

COMMON MISTAKE
(1) Dividing inside the combiner: that turns the pair back into a plain average and the weight is lost again. **The division is the one step that is not commutative and associative, so it waits for the reducer, once, at the very end.** (2) Concluding "combiner = reducer" for average: here the combiner outputs a (sum, count) *pair* while the reducer outputs a *number*; Slide 62's "usually the same" is not "always the same". The network still gains: each mapper ships one small pair per key instead of one pair per value.

8. **Part (e): the median cannot use the combiner trick (Slide 65).** The median is the middle value of the sorted list (for an even number of values, the mean of the two middle ones). It depends on the *rank* (sorted position) of every value, not on a sum. For 1, 2, 3, 4, 5 the median is **3**.
(i) The naive attempt fails. Split {1,2} | {3,4,5}: partial medians 1.5 and 4; the median of these two is (1.5+4)/2 = 2.75 ≠ 3. Another example: 1, ..., 9 split {1,2,9} | {3,4,5} | {6,7,8}: partial medians 2, 4, 7; the median of the medians is **4**, but the true median of 1, ..., 9 is **5**.

(ii) No rescue by adding a count (the fix that worked for the average). Compare two data sets whose two mappers send *identical* summaries (median, count), namely (2, 3) and (5, 3) (Practice values):

Data set	Mapper 1 values	Mapper 2 values	partial medians	counts	true median of all 6 values
P	{1, 2, 3}	{4, 5, 6}	2 and 5	3 and 3	sorted 1 2 3 4 5 6 → (3+4)/2 = **3.5**
Q	{1, 2, 3}	{0, 5, 100}	2 and 5	3 and 3	sorted 0 1 2 3 5 100 → (2+3)/2 = **2.5**

The reducer would see **exactly the same** information, (2, 3) and (5, 3), in P and in Q, yet it must answer 3.5 for P and 2.5 for Q. **No formula applied to (partial median, count) can be right for both**, however cleverly the partial results are merged: the partial medians threw away the information that decides the answer.

(iii) Conclusion (the slide's claim). There is no way to split the median into commutative-and-associative pieces that each summarise a group by a small number of values; Slide 65 says a mathematical proof exists (the P/Q table shows the idea on a small example). So **we cannot use the combiner "trick"**: for the median every value of a key must be shuffled to the reducer, which sorts them and takes the middle one. The shuffle is not reduced (the combiner is simply left out).

> **BEYOND THE SLIDES: WHY NO SHORT SUMMARY CAN WORK**
>
> Statisticians call aggregates like sum, count, max "distributive", average "algebraic" (a fixed-size summary such as (sum, count) suffices) and median "holistic" (no fixed-size summary exists). Idea of the proof: take two different sorted lists of the same length *n* that first differ at position *i*. Add a second list made of enough very small and very large numbers (as the 0 and 100 in Q above) so that the overall median lands exactly on position *i* of the original list. The two overall medians then differ, so any valid summary must tell the two lists apart; hence it must distinguish all lists, i.e. it must be as large as the data itself. (The "union of the lists" is commutative and associative too, but it saves nothing.) In practice one uses approximate quantile summaries or extra Map-Reduce rounds, giving up the simple exact one-round trick.

9. **Summary: which reduce functions can use a combiner?**

Function	Commutative & associative as is?	Combiner sends	Reducer does	On 1–5 split {1,2} | {3,4,5}
Sum, count (word count)	**yes (Slide 62)**	partial sum, e.g. (the,3)	adds partial sums	3 + 12 = 15 ✓
Average, plain	no (Slide 63)	— (do not use)	—	(1.5 + 4)/2 = 2.75, true 3 ×
Average via (sum, count)	**yes, for the pairs (Slide 64)**	pair (sum, count), no division	adds pairs, then divides once	(3,2) + (12,3) = (15,5), 15/5 = 3 ✓
Median	no, and no small summary exists (Slide 65)	nothing useful: all values must go	sorts all values, takes the middle	medians 1.5 and 4 cannot give 3 ×

> **EXAM TIP**
>
> A full-mark answer has five sentences: (1) a combiner pre-aggregates a mapper's values per key on the mapper's own machine, saving network and shuffle; (2) the reduce function may be reused if it is commutative and associative, because the framework controls the split and order; (3) sum qualifies, e.g. (the,1)×3 → (the,3); (4) average does not: {1,2},{3,4,5} give (1.5+4)/2 = 2.75 ≠ 3; fix: send (sum,count), add, divide once at the end → 15/5 = 3; (5) median cannot be decomposed (proof exists), so no combiner trick. Always write the true answer next to the wrong one (2.75 versus 3): that is where the marks are.

COMMON MISTAKES

(1) Averaging the averages (Slide 63's crossed-out recipe). (2) Dividing sum by count inside the combiner: only the very last step may divide. (3) Writing "the combiner replaces the reducer": it only sees one mapper, so the reducer is still required. (4) Saying "the combiner saves computation in the map": it saves network and shuffle work, not map work. (5) Thinking the median can be fixed with (sum, count): it cannot.

Final answer:

(a) A combiner is a mini-reduce, combine(k, list(v_1)) → v_2, run on each map machine on that mapper's own output before the shuffle; it pre-aggregates per key and saves network and shuffle traffic (not map work). Slide 61: 12 pairs → 8 pairs shuffled (33.3 % fewer); the Reducer still adds the partial counts and outputs (A,2) (B,3) (C,2) (D,4) (E,1), the same as without a combiner.

(b) The reduce function can be reused as the combiner only if it is commutative and associative ($f(a,b) = f(b,a)$ and $f(f(a,b),c) = f(a,f(b,c))$), because the system chooses the split and order. Sum is: (the,1) three times → (the,3); example 10 → 7 pairs, same final counts (the = 3 + 2 = 5).

(c) Average is not: {1,2} | {3,4,5} gives (1.5+4)/2 = 2.75 ≠ true average 3. In general $T - W = (n_1 - n_2)(a_1 - a_2)/(2(n_1 + n_2)) \neq 0$ unless the sizes or the averages are equal; avg is not associative (2.25 ≠ 1.75); a partial average forgets its count.

(d) Send (sum, count) pairs, which add coordinate-wise (commutative and associative): (3,2) + (12,3) = (15,5); divide once at the very end, in the reducer: ave = 15/5 = 3 ✓.

(e) The median cannot be split into commutative and associative functions (Slide 65): identical partial summaries (medians 2 and 5, counts 3 and 3) give true medians 3.5 (set P) and 2.5 (set Q). So there is no combiner trick: all values must be shuffled to the reducer, which sorts them and takes the middle one.

Where the marks come from: (1) definition of a combiner, where it runs (on the mapper's machine, on that mapper's output only, before the shuffle) and what it saves (network/shuffle); (2) Slide 61 numbers: 12 → 8 pairs, shuffle groups, Reducer output (A,2) (B,3) (C,2) (D,4) (E,1); (3) the condition: commutative and associative, with the two equations, and the reason (the system chooses split and order); (4) sum satisfies it, with the (the,1)×3 → (the,3) example and the final total unchanged; (5) average counterexample with numbers: 1.5, 4, 2.75 versus 3; (6) the reason it fails (partial averages lose the counts / avg is not associative), ideally with the weighted-average proof; (7) the fix: (sum, count) pairs, shown commutative and associative, (15,5), divide once at the very end, 15/5 = 3; (8) median: cannot be decomposed (Slide 65), so no combiner and all values go to the reducer, ideally with the (median, count) counterexample (3.5 versus 2.5).

Slide check: the slides give no numerical answers for this topic (they argue qualitatively), and everything above agrees with them: sum works (62), average does not (63) but works via (sum, count) (64), median cannot use the trick (65). The Slide 61 figure is reproduced exactly (12 → 8 pairs; output (A,2) (B,3) (C,2) (D,4) (E,1)); every Practice number was recomputed in Python. No numerical erratum. Slide erratum (wording and omission only): (1) Slide 63's "not commutative and associative" should be read as "fails associativity" (the two-argument average is commutative); (2) Slide 62 omits that the combiner's output type must equal the Map output type and that the system may run it zero, one or several times (*beyond the slides*); (3) Slide 62's "works only if ... commutative and associative" is the test for reusing the reduce function *unchanged* as combiner, and Slide 64 shows a failing function can still be combined after re-expressing it through (sum, count); (4) Slide 65's "impossible" is exact only for small fixed-size partial results (the union of all values is trivially commutative and associative but saves nothing); in the exam quote the slide's version.

Refinement: the partition function

Lecture 2 · Slides 66–67

Between map and reduce the framework must decide **which reduce task gets which key**. Slide 66: "Want to control how keys get partitioned": a **partition** is the set of keys that go to one reduce task, and the one hard requirement is that **records with the same intermediate key end up at the same worker**. If the values of one key were split over two reducers, each would compute a wrong, partial answer.

DEFINITION: PARTITION FUNCTION

A function from an intermediate key to a number in `0, 1, …, R−1`, the index of the reduce task that receives it. The system's default is **hash(key) mod R**, where R is the number of reduce tasks, `hash` turns the key into a (large) number and `mod` takes the remainder after division by R.

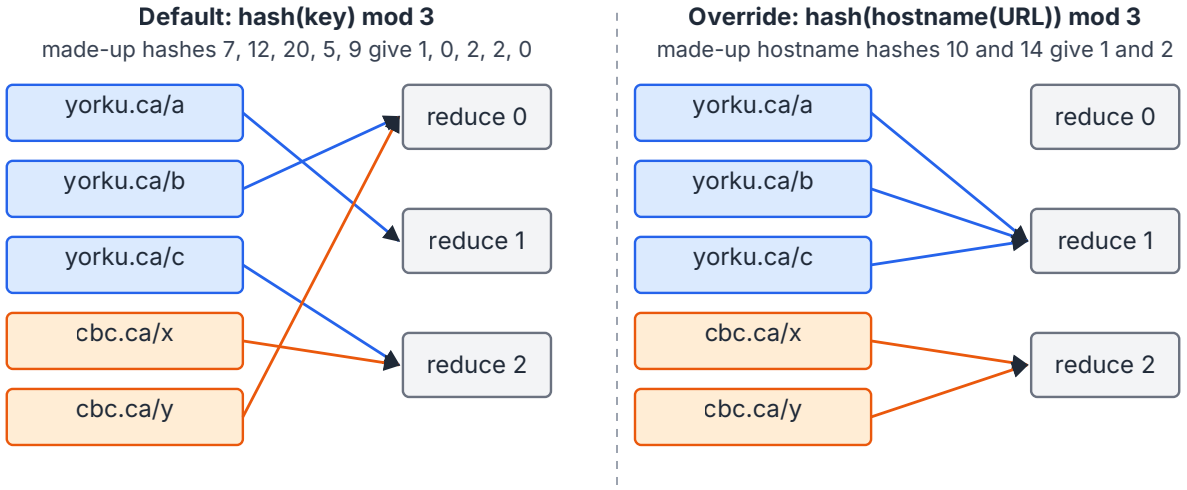
Why it works: same key gives the same hash and the same remainder, hence the same reduce task. Different keys are spread roughly evenly, which balances the load. Each map task applies the function to every pair it emits and writes its output into R buckets (the "Partitioning Function" bars of Slide 59), one per reduce task.

Tiny example (Practice, made-up hash values): with R = 3, a key with hash 7 goes to task 7 mod 3 = 1; a key with hash 9 goes to task 0; hash 20 goes to task 2.

Overriding the hash

Lecture 2 · Slide 67

Slide 67: "Sometimes useful to override the hash function: e.g. **hash(hostname(URL)) mod R** ensures URLs from a host end up in the same output file." The default would send `yorku.ca/a`, `yorku.ca/b` and `yorku.ca/c` to arbitrary reduce tasks. If you hash only the hostname part (`yorku.ca`), all of that site's URLs share one hash and therefore one reduce task and one output file (each reduce task writes its own output file; there are R in total). The key itself is unchanged (still the full URL); only the *routing* uses a function of the key.



Left: the five URLs scatter over all three reduce tasks (and output files). Right: each host stays together.

Figure L2-14 (Slides 66-67; the hash values are made up for the illustration). Colour = web site. Same key, same hash, same reduce task - in both versions.

Beyond the slides: hash partitioning balances load only on average. A very frequent key (such as "the") or one huge host sends all of its pairs to a single reduce task, which becomes a straggler. Each reduce task writes its own output file, so there are R output files (Slide 66's "reduce worker" means a reduce task).

COMMON MISTAKES

- A custom partition function must still map **equal keys to the same reducer**. Any function of the key (such as its hostname) satisfies this, a random choice does not.
- Beyond the slides: a custom partitioner can **unbalance** the load (one huge site means one slow reduce task), and a hash cannot split a single very popular key such as "the" across reducers.
- Slide 66 says "reduce worker" and "set of keys" loosely; precisely, the unit is a reduce *task*, and a worker may run several.

Refinement: backup tasks (stragglers)

Lecture 2 · Slide 68

Slide 68 is organised as Problem, Solution, Effect.

- **Problem:** slow workers significantly lengthen the job completion time. Reasons listed: other jobs on the machine, bad disks, "weird things". A machine that is alive but slow is a **straggler**. A phase (map phase or reduce phase) finishes only when its *last* task finishes, so one straggler delays everything. This is different from a *failure* (Slide 57): the worker still works, just slowly.
- **Solution:** near the end of a phase, the master spawns **backup copies** of the tasks still in progress on other, idle machines. **Whichever one finishes first "wins"**; the other is discarded.
- **Effect:** dramatically shortens job completion time.

Worked example (Practice, numbers invented): one straggler Lecture 2 · Slide 68

Three tasks should each take 10 minutes. Workers 1 and 2 do tasks A and B in 10 minutes. Worker 3 has a bad disk and needs 40 minutes for task C.

1. Without backups: the phase ends when C ends, at **40 min**; Workers 1 and 2 idle for 30 minutes.
2. With a backup: at minute 10 the master starts a copy of C on idle Worker 1. A normal-speed copy needs 10 minutes, so it finishes at minute 10 + 10 = **20 min**; the slow original is cancelled.
3. Saving: 40 → 20 minutes. Cost: one extra copy of one task.

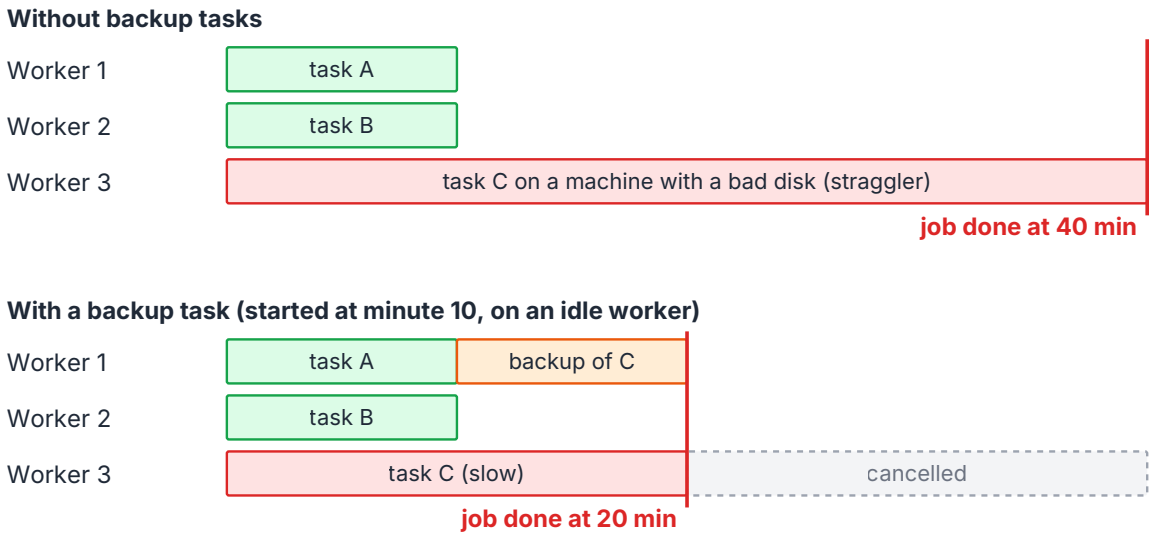


Figure L2-15 (idea of Slide 68, Practice numbers). A phase ends when its last task ends. The duplicate on an idle machine finishes first, so the straggler's copy is thrown away.

Why duplicating is safe: map and reduce functions are deterministic and tasks are independent, so running a task twice gives the same output; the master simply keeps the first. **Beyond the slides:** this is also called *speculative execution*; Dean and Ghemawat report that a sort job took about 44% longer with backup tasks disabled. The slide does not say how a straggler is recognised (the master watches progress near the end of a phase).

EXAM TIP

Typical question: "What is a straggler and how does Map-Reduce handle it?" Answer in three lines: slow-but-alive machine (other load, bad disk) delays the whole phase; near the end of the phase the master launches backup copies of the remaining tasks on other machines; the first copy to finish wins, shortening the job at a small extra cost. Do not confuse with failure recovery.

How many map and reduce tasks?

Lecture 2 · Slide 69

A job has $\mathbf{M}$ **map tasks** and $\mathbf{R}$ **reduce tasks**. Slide 69 gives the **rule of thumb**:

- Make M **much larger than the number of nodes** in the cluster.
- **One DFS chunk per map task is common.**
- This speeds up **recovery from worker failures** and gives **dynamic load balancing**.
- **Usually $\mathbf{R}$ is smaller than $\mathbf{M}$.**

Worked example (Practice) Lecture 2 · Slide 69

Input 1 TB, chunk size 64 MB, cluster of 1,000 nodes. One map task per chunk: M = 1,000,000 MB / 64 MB = 15,625 (about 16,000 if 1 TB is counted as 2^{20} MB: 16,384). That is roughly **16 map tasks per node**, so the cluster stays busy and a dead node loses only about 16 small tasks that are re-run on many machines at once.

Why $\mathbf{R} < \mathbf{M}$? The slide gives no reason. **Beyond the slides:** every reduce task writes one output file, and many tiny output files are unwanted; the output is also usually smaller than the input. R is often a small multiple of the number of machines. Remember that R is the modulus in `hash(key) mod R` (Slide 66).

SLIDE ERRATUM (MINOR)

The title says "jobs" but the content is about **tasks**: one Map-Reduce *job* consists of M map tasks and R reduce tasks. "Rule of a thumb" should read "rule of thumb".

Problems suited for Map-Reduce

Lecture 2 · Slide 70

Slide 70 is a section divider announcing two topics: "Problems Suited for Map-Reduce" and "Join by Map-Reduce". It carries no printed number and no further content; the join comes next. The slide itself lists no criteria, so the guidance below is **Beyond the slides:** a summary of the course background.

Well suited	Poorly suited
One huge input processed in one or a few passes (scan, filter, count, aggregate, join)	Small data that fits on one machine (the cluster overhead dominates)
Work that splits into independent pieces (map) followed by a group-by-key and per-key aggregation (reduce)	Jobs needing **low latency** or random access/updates to single records
Data that is written once and read many times (as for the DFS, Slides 17–18)	**Iterative** algorithms with many rounds: each round re-reads and re-writes the DFS (the motivation for Spark in Lecture 3)
Examples in this lecture: word count, host size, language model, join	Tasks where pieces must constantly exchange state with each other

Example: join by Map-Reduce

Lecture 2 · Slides 71–72

The problem

Lecture 2 · Slide 71

A **relation** is a table; a **tuple** is one row. Slide 71 asks for the **natural join** R(A,B) ⋈ S(B,C), where R and S are each stored in files and tuples are pairs (*a*,*b*) or (*b*,*c*). They are "stored in files" because they are huge and live in the DFS.

> **DEFINITION: NATURAL JOIN**
> R(A,B) ⋈ S(B,C) glues every tuple (*a*,*b*) of R to every tuple (*b*,*c*) of S that has the **same value of the shared attribute B**, producing (*a*,*b*,*c*). A tuple whose *b* has no partner on the other side produces nothing.

Slide 71's data: R = {(a_1,b_1), (a_2,b_1), (a_3,b_2), (a_4,b_3)}, S = {(b_2,c_1), (b_2,c_2), (b_3,c_3)}. Value b_1 occurs only in R (a_1 and a_2 vanish); b_2 occurs once in R and twice in S (two result rows); b_3 once on each side. The join has three tuples: (a_3,b_2,c_1), (a_3,b_2,c_2), (a_4,b_3,c_3).

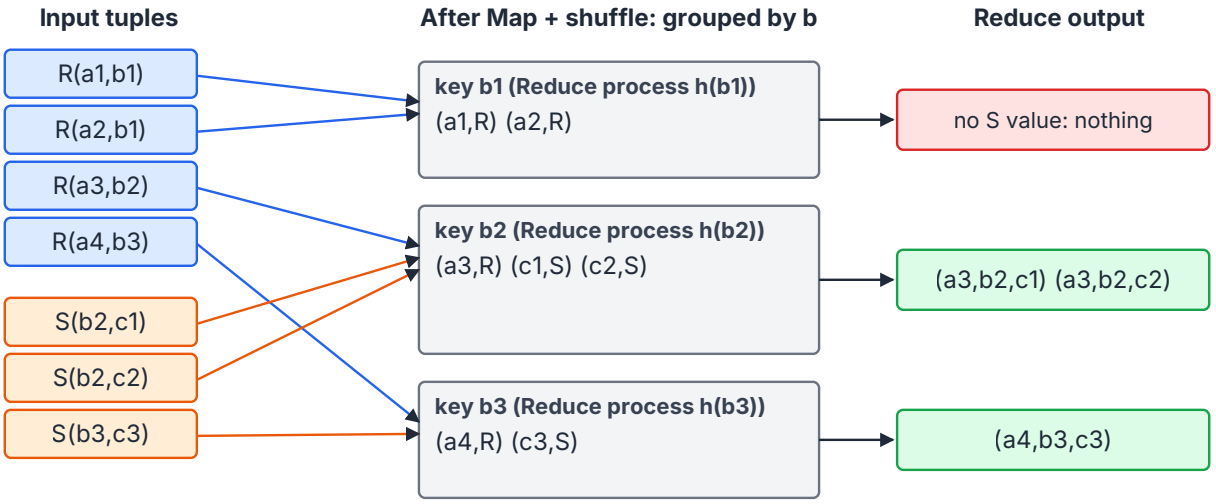
> **SLIDE ERRATUM**
> The result table on Slide 71 has only the columns A and C (and the next slide outputs (*a*,*b*,*c*)). The natural join has schema (A,B,C). The slide's three rows are the (A,C) *projection* of the correct result, so its tuples are right but the column B is missing. Use (A,B,C) in your answer.

The algorithm

Lecture 2 · Slide 72

Idea: tuples that must be glued share the same *b*, and Map-Reduce's group-by-key brings equal keys together. So make *b* the key.

1. **Map:** each R tuple (*a*,*b*) becomes the pair `(b, (a, R))`; each S tuple (*b*,*c*) becomes `(b, (c, S))`. The **tag** R or S records the origin, because a reducer receives a mixed bag and must know which side each value came from.
2. **Shuffle:** every pair with key *b* goes to Reduce process *h*(*b*), where *h* is a hash function ("Hadoop does this automatically"). Beyond the slides: in Hadoop this is the default partition function hash(key) mod R of Slide 66.
3. **Reduce:** for one key *b*, match every (*a*,R) with every (*c*,S) and output (*a*,*b*,*c*), a cross product per key.



Map: R(a,b) becomes (b,(a,R)); S(b,c) becomes (b,(c,S)).
Reduce: pair every R-value with every S-value of the same b.

Figure L2-16 (data of Slide 71, method of Slide 72). 7 input tuples produce 7 key-value pairs; group b1 has no partner, so 3 result tuples come out.

Trace on the slide data: 4 + 3 = 7 pairs are shuffled. b_1 has values [(a_1,R), (a_2,R)] and no S value, so no output; b_2 pairs a_3 with c_1 and c_2; b_3 pairs a_4 with c_3. Result: 3 tuples, as above.

COMMON MISTAKES

- Forgetting the **tag**; without it the reducer cannot tell R values from S values.
- "Match all (b,(a,R)) with all (b,(c,S))" means *for the same key b*: a reducer for b_2 never sees b_3 pairs.
- Outputting something for a key that appears on one side only (dangling tuples give nothing).
- The slide says "process" where the rest of the lecture says "task"; one Reduce process handles many keys.

Cost measures for Map-Reduce algorithms

Lecture 2 · Slides 73–74

How do we compare two Map-Reduce algorithms? Slide 73: "In MapReduce we quantify the cost of an algorithm using":

DEFINITION: THE TWO COST MEASURES (SLIDE 73)

1. **Communication cost** = **total I/O of all processes**: how much data all map and reduce tasks read and write, i.e. how much data is moved around.
2. **(Elapsed) computation cost**: analogous, but counting only the **running time** of the processes.

The note on the slide: "the big-O notation is not the most useful (adding more machines is always an option)". Classic analysis says "this takes *n* log *n* steps"; but with 1,000 machines the work simply divides, so what matters is how *much data moves* and how *long you wait*.

What the measures mean

Lecture 2 · Slide 74

- Either the I/O (communication) cost or the processing (computation) cost **dominates**; **ignore one or the other** (analyse the bigger one).
- **Total cost** tells what you pay in rent from "your friendly neighborhood cloud": the sum of work over *all* machines.
- **Elapsed cost** is **wall-clock time** using parallelism: how long you wait.

	Total cost	Elapsed cost
What it adds up	work of all machines (machine-hours, bytes moved by all)	the longest chain of dependent work (wall-clock time)
Pays for	the cloud bill	your waiting time
Effect of adding machines	stays about the same	goes down

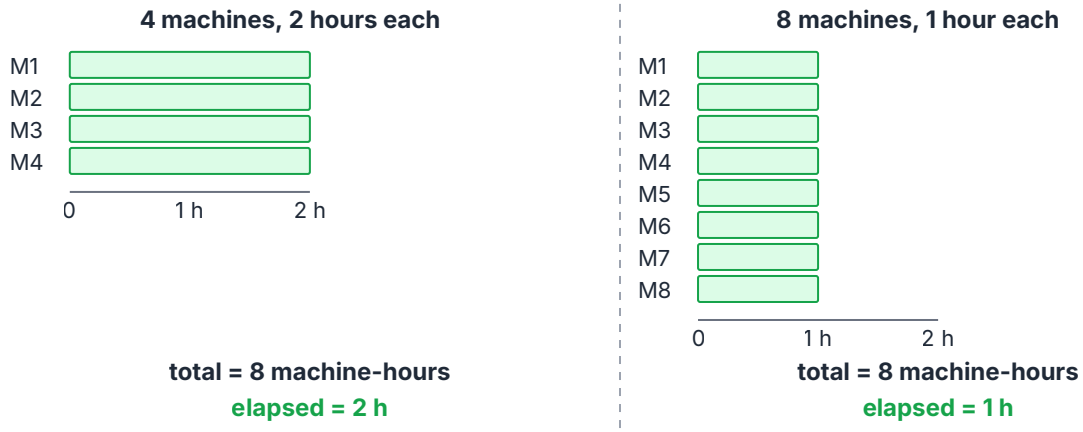


Figure L2-17 (idea of Slide 74, Practice numbers, ideal speed-up assumed). Total cost is what the cloud bills; elapsed cost is how long you wait.

Worked example (Practice, assumed numbers): communication cost of a job Lecture 2 · Slide 73 and Slide 61

1,000 map tasks each read a 64 MB chunk, and (no combiner) each emits about as much as it reads, which the reducers then read.

1. Map input: 1,000 × 64 MB = 64,000 MB.
2. Reduce input (the shuffled data): about 64,000 MB.
3. Communication cost, convention B (textbook: count each task's input only) ≈ 64,000 + 64,000 = **128,000 MB** (about 128 GB). Convention A (literal Slide 73: every byte read or written) would also add the Map output written (about 64,000 MB) and the final output, so about 192,000 MB plus the output. State which convention you use; the cost is linear in the data either way.
4. With a combiner that cuts the shuffled data tenfold: 64,000 + 6,400 = **70,400 MB**.

Beyond the slides (textbook, Mining of Massive Datasets §2.3): convention B, the textbook, counts each task's *input* size and does not count the final output, and defines "elapsed" cost as the largest total along any chain of dependent tasks, since independent tasks run simultaneously. Slide 73's phrase "total I/O" is vague on this point; the same example is applied to the join in the quiz below.

COMMON MISTAKE

Quoting big-O as "the" cost, or saying that more machines reduce *total* cost. They reduce *elapsed* cost; total cost stays (or even grows, because of communication overhead).

Try it yourself first: for the Slide 71 data, write the Map output, the groups and the Reduce output, then say what the communication cost is, before looking at the solution.

Quiz L2-Q6 Natural join R(A,B) ⋈ S(B,C) with Map-Reduce — Lecture 2 · Slides 70–74

QUESTION (AS ON THE SLIDES)

Slide 71, "Example: Join By Map-Reduce" (Slide 70 is only the section divider "Join by Map-Reduce"). **Compute the natural join R(A,B) ⋈ S(B,C).** R and S are each stored in files. Tuples are pairs (a,b) or (b,c). The slide prints the two input tables and the result. (The slide writes a with subscript 1, b with subscript 1, and so on; here they are typed a1, b1, ...)

A	B
a1	b1
a2	b1
a3	b2
a4	b3

R

⋈

B	C
b2	c1
b2	c2
b3	c3

S

=

A	C
a3	c1
a3	c2
a4	c3

result as printed on Slide 71

Slide 72, "Map-Reduce Join", gives the method in words: a Map process turns each input tuple R(a,b) into the key-value pair (b,(a,R)) and each input tuple S(b,c) into (b,(c,S)); each key-value pair with key b is sent to Reduce process h(b) ("Hadoop does this automatically"); each Reduce process matches all the pairs (b,(a,R)) with all (b,(c,S)) and outputs (a,b,c). **Slides 73–74** then define the cost measures of a Map-Reduce algorithm. The slides pose one task and give no numbers beyond the tables, so this block splits it into the parts an exam could ask:

(a) Give the result of R ⋈ S and check every tuple.

(b) Map: list the output pairs (b,(a,R)) and (b,(c,S)), tagged by relation.

(c) Shuffle and Reduce: show which pairs go to Reduce process h(b) and what every Reduce call outputs.

(d) *Practice (written for this guide, not on the slides):* explain why a join key that occurs on only **one** side (for example b1, which occurs only in R) produces **no output**.

(e) Slides 73–74: what is the **communication cost** (= total I/O of all processes), and how do the **elapsed** and **total** cost differ? Discuss briefly using this example.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Think of two lists at a university. List **R** says which *student* is in which *tutorial group* (student a, group b). List **S** says which *room* each group uses (group b, room c). "Join" means: work out, for each student, the room(s) where they must go, by gluing a student line and a room line together *whenever the group is the same*. Doing this on a huge pile of papers is easy if you first **sort everything into one box per group**: then the students of group b2 and the rooms of group b2 are in the same box, and you simply pair every student with every room in that box. Map-Reduce does exactly this. **Map** writes the group on the front of every paper; the **shuffle** posts each paper to the box (Reduce process) h(b) that owns that group; **Reduce** pairs things up inside each group. A coloured sticker (the **tag** R or S) tells the clerk whether a paper is a student line or a room line. A group that has students but no room (b1) gives no pairs at all.

1. **Formula / idea — what a natural join is.** The natural join glues together every pair of tuples that **agree on the attribute they share**, B, and keeps that shared value only once:

   #### FORMULA

   R(A,B) ⋈ S(B,C) = { (a,b,c) : (a,b) ∈ R and (b,c) ∈ S }. The result has the three attributes (A,B,C). If the value b occurs r_b times in R and s_b times in S, it contributes $r_b \times s_b$ result tuples, so $|R \bowtie S| = \sum_b r_b \cdot s_b$.

Why this suits Map-Reduce (Slides 32–35, 44): the framework's **Group by key** step brings together all values that have the same key. Tuples that must be glued share the same b. So choose **key = b**, and the system itself delivers to one place exactly the tuples that have to meet.

2. **Part (a) — compute the expected answer directly (nested comparison).** |R| = 4 and |S| = 3. Compare every R-tuple with every S-tuple (4 × 3 = 12 comparisons) and keep a pair only if the B values are equal. This gives the answer that Map-Reduce must reproduce, and it lets us check the slide:

R-tuple ↓ / S-tuple →	(b2,c1)	(b2,c2)	(b3,c3)
(a1,b1)	× b1 ≠ b2	× b1 ≠ b2	× b1 ≠ b3
(a2,b1)	× b1 ≠ b2	× b1 ≠ b2	× b1 ≠ b3
(a3,b2)	✓ **(a3,b2,c1)**	✓ **(a3,b2,c2)**	× b2 ≠ b3
(a4,b3)	× b3 ≠ b2	× b3 ≠ b2	✓ **(a4,b3,c3)**

   Three of the 12 pairs match, so the expected join has **3 tuples**: (a3,b2,c1), (a3,b2,c2), (a4,b3,c3). The two R-tuples with b1 find no partner in S. Dropping the B column gives (a3,c1), (a3,c2), (a4,c3), which is the table printed on Slide 71. Beyond the slides: comparing all pairs is fine for 12 pairs but hopeless for billions of tuples; the Map-Reduce method below never compares tuples with different b.

3. **Part (b) — Map (Slide 72): one Map call per input tuple (Slide 43), key = the join attribute b, value = the other attribute plus a tag.** Every R-tuple (a,b) becomes the pair **(b,(a,R))**; every S-tuple (b,c) becomes **(b,(c,S))**. Applied to the data:

Input file	Input tuple	Map output pair (key, (value, tag))
R	(a1,b1)	`(b1,(a1,R))`
	(a2,b1)	`(b1,(a2,R))`
	(a3,b2)	`(b2,(a3,R))`
	(a4,b3)	`(b3,(a4,R))`
S	(b2,c1)	`(b2,(c1,S))`
	(b2,c2)	`(b2,(c2,S))`
	(b3,c3)	`(b3,(c3,S))`

   That is **7 Map calls and 7 output pairs** (4 from R plus 3 from S). *Why key = b?* It is the only attribute shared by R and S, and Group by key gathers all pairs with equal b. *Why the tag?* The Reduce side receives a mixed bag of values such as a3, c1, c2 with no column names; without a tag it could not tell whether a value is an A-value (from R) or a C-value (from S), and it could wrongly glue two R-values together. Beyond the slides: in Hadoop a Map task knows which input file it is reading, so it can attach the right tag.

4. **Part (c) — Shuffle / group by key (Slide 72: "Send each key-value pair with key b to Reduce process h(b)").** The hash function h turns a key into the number of a Reduce process. Its defining property is that **the same b always goes to the same Reduce process**, so all R-pairs and S-pairs of one b meet. "Hadoop does this automatically": the programmer writes nothing for it. (Slide 66 gives the default hash(key) mod R; Slide 69 defines R as the number of Reduce tasks; the system's default partition function is hash(key) mod R, where R is the number of Reduce tasks.) The slides fix neither h nor the number of Reduce processes, so for the picture I **assume R = 2 Reduce processes**, numbered 0 and 1, with h(b1) = 0, h(b2) = 0, h(b3) = 1 (illustration only; any other h gives the same final join). Then:

Reduce process	Keys it owns	Pairs it receives	#pairs
0	b1, b2	`(b1,(a1,R))`, `(b1,(a2,R))`, `(b2,(a3,R))`, `(b2,(c1,S))`, `(b2,(c2,S))`	5
1	b3	`(b3,(a4,R))`, `(b3,(c3,S))`	2

   Inside each process the pairs are grouped by key, giving three key groups: b1 → [(a1,R), (a2,R)]; b2 → [(a3,R), (c1,S), (c2,S)]; b3 → [(a4,R), (c3,S)]. The pairs arrive in arbitrary order, mixed R and S, which is why the tag is needed.

COMMON MISTAKE

Slide 72 says each Reduce *process* matches "all the pairs (b,(a,R)) with all (b,(c,S))". Read this as pairs with the **same b**. Reduce process 0 holds two different keys (b1 and b2); it must **not** pair a b1-tuple with a b2-tuple. The unit of work is the **Reduce call**: one call per unique key (Slide 44), here **3 calls**, even though there are only 2 Reduce processes. (Also: on Slides 66 and 69 the number of Reduce tasks is called R; it has nothing to do with relation R.)

5. **Part (c) — Reduce (Slide 72): inside one call, separate the values by tag and output every R–S combination.** For key b, build the **R-list** (values tagged R, the a-values) and the **S-list** (values tagged S, the c-values), and output (a,b,c) for **every** a in the R-list together with **every** c in the S-list. That is $r_b \times s_b$ outputs for the key. The three Reduce calls:

Reduce call	Key	R-list	S-list	combinations	Output of this call
1 (process 0)	b1	a1, a2	(empty)	2 × 0 = 0	nothing
2 (process 0)	b2	a3	c1, c2	1 × 2 = 2	**(a3,b2,c1), (a3,b2,c2)**
3 (process 1)	b3	a4	c3	1 × 1 = 1	**(a4,b3,c3)**
Total				**3**	**3 tuples**

6. **Part (a) again — collect the output and check it.** The join is the union of all Reduce outputs: $\mathbf{R} \bowtie \mathbf{S}$ = **{ (a3,b2,c1), (a3,b2,c2), (a4,b3,c3) }**, as a table:

A	B	C
a3	b2	c1
a3	b2	c2
a4	b3	c3

   Three checks: (i) it equals the direct comparison in step 2; (ii) the count formula gives $\sum r_b \cdot s_b = 2{\cdot}0 + 1{\cdot}2 + 1{\cdot}1 = 3$ ✓; (iii) removing the B column leaves exactly the three (A,C) rows of the table on Slide 71 ✓.
7. **Draw the data flow.** Figure 1 shows the whole job: 7 input tuples → 7 tagged pairs → shuffle (arrow colour = key) → 3 Reduce calls on 2 Reduce processes → 3 result tuples.

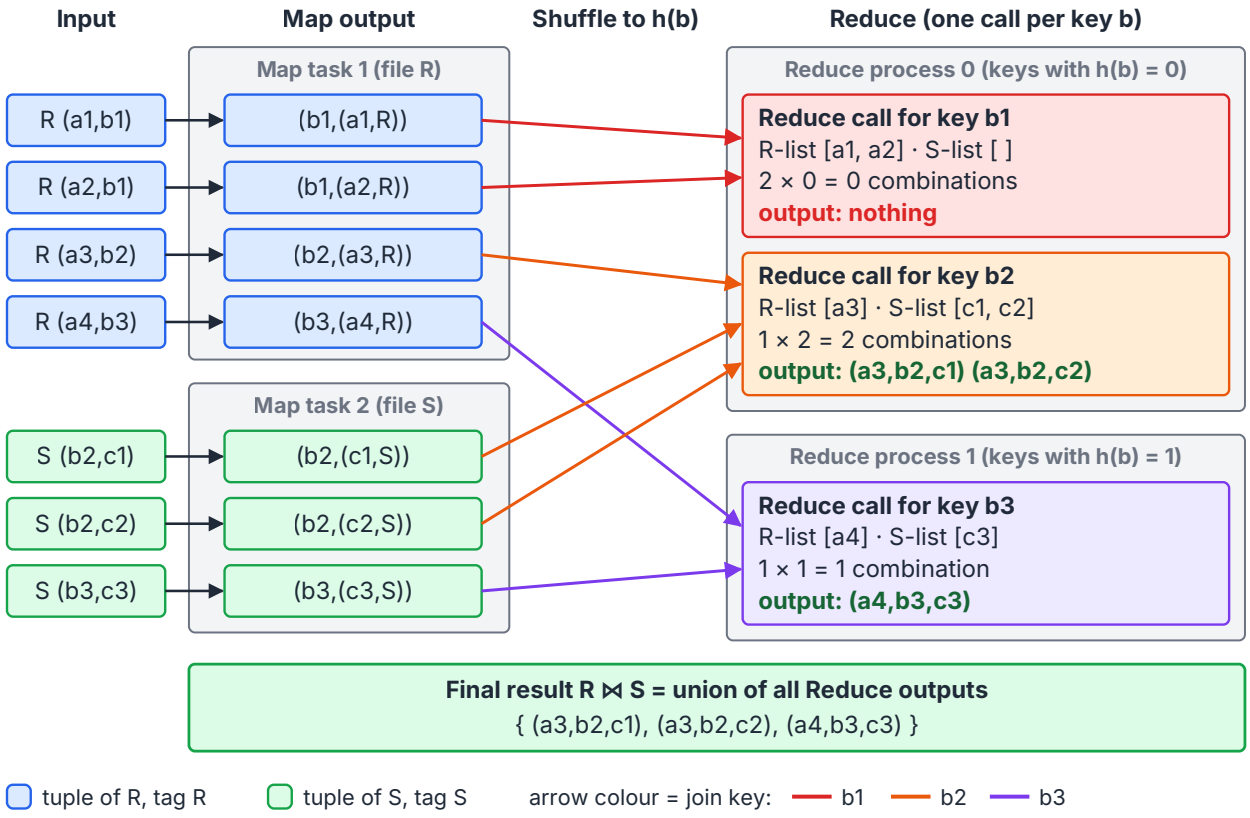


Figure 1 — Map-Reduce join R(A,B) ⋈ S(B,C). Map tags every tuple with its relation and uses B as key; the shuffle sends each pair to Reduce process h(b) (h(b1) = h(b2) = 0, h(b3) = 1 is an assumed example); every Reduce call glues its R-list with its S-list. The red key b1 reaches its Reduce call but finds an empty S-list, so nothing comes out.

8. **Part (d) — why a key found on one side only gives no output.** Take b1. Both pairs with key b1 come from R: the Reduce call receives R-list = [a1, a2] but S-list = [ ] (empty). The number of combinations is $r_{b1} \times s_{b1} = 2 \times 0 = \mathbf{0}$, so there is nothing to glue and the call outputs nothing. *Why this is the right answer:* a natural join keeps (a,b,c) only if **both** (a,b) ∈ R **and** (b,c) ∈ S. There is no tuple (b1,c) in S, so (a1,b1) and (a2,b1) are *dangling tuples* (tuples with no join partner). A natural (inner) join discards them, which is why a1 and a2 are missing from the table on Slide 71. The same happens for a key that occurs **only in S**: its R-list is empty, so $0 \times s_b = 0$ outputs. In general, if either list is empty the product is 0. (Padding the missing side with NULL would be a different operation, an *outer* join, not a natural join.) The price: the two b1 pairs were still read, shuffled and received before being thrown away, because Map cannot know in advance which keys have a partner. This wasted traffic counts in the communication cost below. *Practice (self-made check):* if S also contained (b1,c4), the b1 Reduce call would have R-list [a1, a2] and S-list [c4], so 2 × 1 = 2 outputs, (a1,b1,c4) and (a2,b1,c4), and the join would grow from 3 to 5 tuples.
9. **Part (e) — communication cost (Slide 73): count the total I/O (input/output, i.e. reading and writing data) of all processes.** Slide 73 gives only the definition: **communication cost = total I/O of all processes**. To make it concrete, count every tuple or pair that a process reads or writes (unit: one tuple = one unit; the slide fixes no unit, so this tally is my illustration). With $r = |R| = 4$, $s = |S| = 3$ and $o = |R \bowtie S| = 3$:

Stage	What is read or written	General	Here
Map tasks read their input	every tuple of R and S	$r + s$	7
Map tasks write pairs	exactly one pair per input tuple	$r + s$	7
Reduce tasks read pairs	every pair is shuffled to its reducer	$r + s$	7
Reduce tasks write the result	one (a,b,c) per match	o	3
Total I/O of all processes	7 + 7 + 7 + 3	$\mathbf{3(r + s) + o}$	**24**

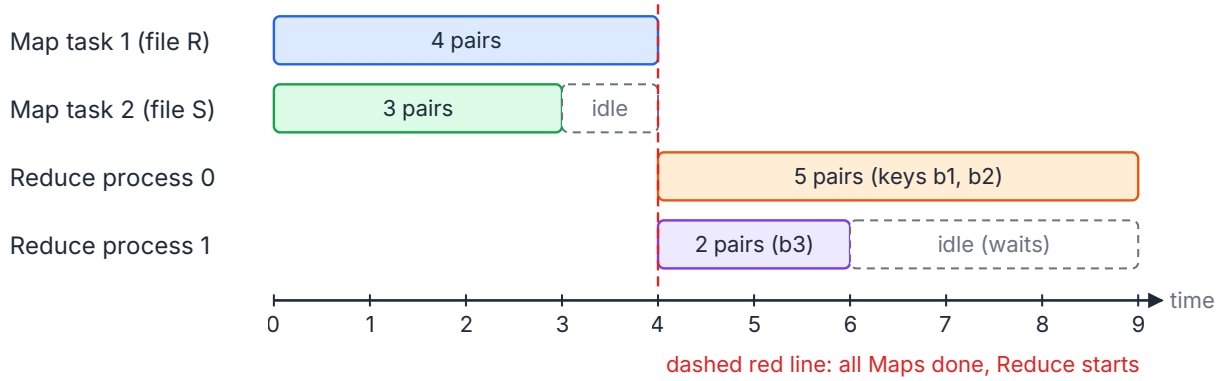
So the cost grows **linearly** with the size of the inputs plus the size of the output: the whole of both relations is read, shipped across the network once and received, including the dangling b1 pairs. Beyond the slides: a combiner (Slides 59–62) cannot shrink this traffic, because a join does not aggregate anything; every tuple must reach its reducer. Beyond the slides: the textbook (*Mining of Massive Datasets*, Ch. 2) counts only the *input* size of each task, because one task's output is the next task's input; then the cost here is 7 (Map input) + 7 (Reduce input) = 14, or 17 if the final output of 3 tuples is also counted, i.e. $O(r + s + o)$. Whichever convention you use, state it; the conclusion (linear in the data) is the same. Real systems count bytes rather than tuples.

10. **Part (e) — elapsed versus total cost (Slides 73–74).** Slide 73 names two measures: (1) **communication cost** = total I/O of all processes, and (2) **(elapsed) computation cost**, which is analogous but counts only the *running time* of the processes. It adds that big-O notation is not the most useful measure here, because "adding more machines is always an option". Slide 74 explains the meaning:

	Total cost	Elapsed cost
What it measures	the work of *all* processes on all machines, added up	wall-clock time from start to end when processes run in parallel; only the slowest chain counts
Slide 74 wording	"tells what you pay in rent from your friendly neighborhood cloud"	"wall-clock time using parallelism"
Effect of adding machines	about the same (the work is only spread out)	goes down, because tasks run at the same time

Slide 74 also says that either the I/O (communication) cost or the processing (computation) cost dominates, so you can ignore one or the other. For this join the per-tuple processing is trivial (build a pair, compare two keys), so **communication dominates** and the communication cost is the number to look at (my reading of the example).
Illustration (assumed cost model, made up for this guide): every pair costs 1 time unit to process in a task, shuffle time is ignored, and Reduce starts only when all Map tasks are done. The 2 Map tasks process 4 and 3 pairs; the 2 Reduce processes (our h) process 5 and 2 pairs (Figure 2).

Illustration (assumed cost model: 1 pair = 1 time unit per task)



Total cost = 4 + 3 + 5 + 2 = 14 units of work (the rent).
Elapsed cost = 4 (slowest Map) + 5 (slowest Reduce) = 9 units (the wall-clock wait).

Figure 2 — Total versus elapsed cost for the join job (illustrative cost model, not from the slides). The tasks overlap in time, so the elapsed cost (9) is smaller than the sum of all work (14). Reduce process 0 is the bottleneck because it owns two keys.

Number of Reduce processes	Pairs per Reduce process	Total cost	Elapsed cost
1	7	4 + 3 + 7 = 14	4 + 7 = 11
2 (our h)	5 and 2	14	4 + 5 = 9
3 (one per key)	2, 3 and 2	14	4 + 3 = 7

Adding Reduce processes **lowers the elapsed cost but leaves the total work unchanged** (14 units); the elapsed cost is limited by the most heavily loaded process. This is the point of Slide 73's note: since more machines can always be added, what you really trade off is money (total) against waiting time

(elapsed). Here there are only 3 keys, so a 4th Reduce process would receive nothing and sit idle (Slide 69: usually R is smaller than M, the number of Map tasks). Beyond the slides: if one value of b is very common, its Reduce call receives huge lists and becomes the slowest task, so the elapsed time is set by that call ("skew").

SLIDE ERRATUM (MINOR)

(1) The result table on Slide 71 has only the columns A and C, but the natural join $R(A,B) \bowtie S(B,C)$ has the schema **(A,B,C)**, and Slide 72 itself says that Reduce outputs **(a,b,c)**. The slide table is the join with column B dropped (a projection onto A and C). The A–C pairs are correct. In an answer, write the full tuples (a,b,c) and, if you also show the A–C table, say that B has been left out. (2) Slide 72's "each Reduce process matches all the pairs (b,(a,R)) with all (b,(c,S))" is only correct per key: matching happens inside one Reduce call (one per unique key, Slide 44), never across different keys that happen to sit on the same process.

EXAM TIP

A full-marks answer has five visible parts: (1) **key = join attribute B**; (2) the two Map rules with **tags**: R(a,b) → (b,(a,R)) and S(b,c) → (b,(c,S)); (3) all 7 Map output pairs written out; (4) the **shuffle** statement "pairs with the same b go to the same Reduce process h(b)", with the groups per key; (5) per key, the **R-list × S-list** combinations and the final tuples. If asked about cost, quote the definition ("communication cost = total I/O of all processes") and the contrast "total = cloud rent, elapsed = wall-clock time with parallelism". Even if the arithmetic goes wrong, each of these lines is a separate mark.

COMMON MISTAKE

- Forgetting the tags, so the Reduce side cannot tell a-values from c-values.
- Choosing A or C as the key. The key must be the **shared** attribute B; otherwise tuples that should join go to different reducers and never meet.
- Pairing an R-value with another R-value, or pairing values that belong to *different* keys just because they landed on the same Reduce process.
- Outputting something for b1, for example (a1,b1,NULL). A natural join outputs nothing for unmatched keys.
- Giving only (a,c). The Reduce output is (a,b,c); the (A,C) table is the same join with B left out.
- Saying "one Reduce process per key". There is one Reduce *call* per key; a process may own several keys.

Final answer:
(a) R ⋈ S = { (a3,b2,c1), (a3,b2,c2), (a4,b3,c3) } (3 tuples; on columns A,C only: (a3,c1), (a3,c2), (a4,c3), as on Slide 71).
(b) Map: R(a,b) → (b,(a,R)); S(b,c) → (b,(c,S)). 7 pairs: (b1,(a1,R)), (b1,(a2,R)), (b2,(a3,R)), (b3,(a4,R)), (b2,(c1,S)), (b2,(c2,S)), (b3,(c3,S)).
(c) Shuffle: every pair with key b goes to Reduce process h(b); groups: b1 → [a1,a2 from R], b2 → [a3 from R; c1,c2 from S], b3 → [a4 from R; c3 from S]. Reduce (one call per key, output = all R–S combinations): b1 → nothing (2 × 0); b2 → (a3,b2,c1), (a3,b2,c2) (1 × 2); b3 → (a4,b3,c3) (1 × 1).
(d) A key with only R-tuples (b1) or only S-tuples has an empty list on one side, so $r_b \times s_b = 0$ and nothing is output; those tuples are dangling and do not appear in the join.
(e) Communication cost = total I/O of all processes. Convention A (literal Slide 73, every read and write counted): 3(r + s) + o = 24 tuple transfers here. Convention B (textbook, task inputs only): 14, or 17 with the output. State which convention you use; it grows linearly with input plus output size. Total cost (sum of all work, the cloud rent) is 14 units while elapsed cost (wall-clock with parallelism) is 9 units in the illustrative model; more Reduce processes lower elapsed but not total.

> **Where the marks come from:** (1) The natural-join definition (a,b,c) with (a,b) ∈ R and (b,c) ∈ S, and the correct 3-tuple result — (a). (2) Key = the join attribute B; Map rule for R-tuples, (b,(a,R)), and for S-tuples, (b,(c,S)), including the tags; all 7 output pairs written correctly — (b). (3) Shuffle: same b → same Reduce process h(b); the grouped lists per key — (c). (4) Reduce rule: match every R-value with every S-value of the same key and output (a,b,c); correct per-key outputs (nothing for b1, two tuples for b2, one for b3) — (c). (5) A key present on one side only gives an empty list, a product of 0 and dangling tuples — (d). (6) Communication cost = total I/O of all processes, ideally with the count 24 or the formula 3(r + s) + o; total (rent) versus elapsed (wall-clock with parallelism) — (e). (This split is typical, not the examiner's actual scheme; every step above is a separate credit opportunity.)

Slide check: matches the slide's answer. The R and S tables were read from the Slide 71 image tuple by tuple and checked by hand and by computer; the result printed on Slide 71, (a3,c1), (a3,c2), (a4,c3), is exactly the A and C columns of the join computed here, and the method is the one on Slide 72. Notes: (i) minor erratum above (the slide result table omits column B although the join and Slide 72 output (a,b,c)); (ii) Slide 72's "all the pairs ... with all ..." must be read as "with the same b"; (iii) the slides give no hash function, number of reducers, or cost numbers for this example, so h, the 2 Reduce processes, the count 24 and the 14 / 9 time units are my illustrative assumptions (Slides 73–74 give only the definitions).

Implementations and cloud computing

Lecture 2 · Slides 75–77

Slide 75 is the divider "Map-Reduce Implementations and Pointers" (unnumbered; "pointers" means references and links). Map-Reduce is a **model**; Slide 76 names the systems that implement it.

Implementations

Lecture 2 · Slide 76

System	What it is (Slide 76)	Storage
Google MapReduce	Google's original; **not available outside Google**	**GFS** (Google File System) for stable storage
Hadoop	An **open-source implementation in Java**	**HDFS** (Hadoop Distributed File System) for stable storage
Hive, Pig	Provide **SQL-like abstractions** on top of the Hadoop Map-Reduce layer	(they run on Hadoop and HDFS)

Stable storage means storage that keeps data safe and available despite machine failures, because it is replicated (Slides 21–22), unlike a worker's local disk where intermediate map output lives and is lost with the machine. **Hive and Pig** let analysts write short queries or scripts instead of Java map and reduce programs; the system compiles them into Map-Reduce jobs. **Beyond the slides:** Hive's language HiveQL is SQL-like; Pig's language Pig Latin is a step-by-step data-flow script, so "SQL-like" fits Hive better than Pig.

> **SLIDE ERRATUM (MINOR)**
> The download link on Slide 76 (`lucene.apache.org/hadoop`) and the wiki/Javadoc links on Slide 79 date from when Hadoop was a Lucene sub-project and are obsolete; Hadoop now has its own Apache site. They are not examinable. HDFS is not expanded on the slide (Hadoop Distributed File System).

> **BEYOND THE SLIDES: GFS AND HDFS TERMINOLOGY**
> The two file systems share one design (Lecture 2 Slides 17–25); the vocabulary differs. Defaults can vary by version, so treat sizes as typical.

Role	GFS term (Google)	HDFS term (Hadoop)	Slide wording
Keeps metadata: which chunks form which file, where the copies are	Master	Name Node	"Master node (Name Node)"
Unit into which a file is cut	Chunk (typically 64 MB)	Block (default 128 MB in recent versions, 64 MB in early ones)	"Chunks" of 16–64 MB
Machine that stores the pieces (and can also compute)	Chunk server	Data Node	"Chunk servers"
Copies of each piece	Replicas, 3 by default	Replicas, 3 by default	"Replicated 2x or 3x"
Map-Reduce on top of it	Google MapReduce	Hadoop MapReduce	Slide 76

Cloud computing

Lecture 2 · Slide 77

Slide 77: **cloud computing** is the "ability to rent computing by the hour", plus additional services such as persistent storage. You do not buy a cluster; you rent machines and release them when the job ends, which is why total cost (Slide 74) is literally your bill. Amazon's offering:

Service	What it is
EC2, Elastic Compute Cloud	rentable virtual machines (the compute)
S3	stable storage for the data (Beyond the slides: Simple Storage Service)
EMR, Elastic Map Reduce	ready-made Hadoop Map-Reduce service

The slide adds: "Hadoop can be run on EC2" (you can install and run it yourself on rented machines). **Beyond the slides:** S3 and EMR are separate Amazon services used together with EC2, not parts of it, although the slide nests them under EC2.

Readings, resources and actions

Lecture 2 · Slides 78–80

- **Readings (Slide 78):** the two original Google papers. (1) Ghemawat, Gobioff, Leung: *The Google File System* (the storage layer, Slide 76). (2) Dean and Ghemawat: *MapReduce: Simplified Data Processing on Large Clusters* (the model and its fault-tolerant implementation). **Beyond the slides:** GFS appeared at SOSP 2003, MapReduce at OSDI 2004.
- **Resources (Slide 79):** the Hadoop Wiki (Getting Started, Map/Reduce Overview with two pages, Eclipse Environment) and the Javadoc (generated API documentation of the Hadoop classes). Practical pointers for people who want to try Hadoop; not examined, and since no code is examined you only need to know they exist.
- **Actions (Slide 80):** (1) **"Review all the examples (treat [them] as quiz questions)!"** (2) Read Chapter 2 of the course book (*Mining of Massive Datasets*, "MapReduce and the New Software Stack"; electronic version on eClass). (3) A Spark / Map-Reduce programming tutorial in the next class (the link is a Colab notebook).

> **EXAM TIP: THE SINGLE MOST USEFUL HINT OF THE LECTURE**
> Slide 80 says every worked example is a potential quiz question. In this lecture that means word count, host size, the language model, the **combiner example (Slide 61)** and the **join (Slides 71–72)**. For each be able to state the Map function, the Reduce function and the key and value types, in words, and trace a small input by hand. (Slide 80 has a typo, "treat is as", which should read "treat them as".)

QUICK RECAP (SLIDES 58–80)

- **Granularity:** many more map tasks than machines ($M \gg$ nodes, one DFS chunk per map task) gives load balancing and fast recovery; R is usually smaller than M. Shuffle reads start as each map ends (pipelining); reduce waits for all its reads.
- **Combiner:** runs on the mapper's machine before the shuffle; legal when reduce is commutative and associative (sum, count, max, min). Average: carry (sum, count), divide at the end. Median: impossible. Slide 61: 12 pairs become 8.
- **Partition function:** default hash(key) mod R; equal keys must reach the same reducer; override such as hash(hostname(URL)) mod R keeps a site's URLs in one output file.
- **Backup tasks:** near the end of a phase duplicate the slow tasks; the first copy to finish wins.
- **Join:** map to (b,(a,R)) and (b,(c,S)); shuffle by *h*(b); reduce pairs R-values with S-values per key and outputs (a,b,c); one-sided keys give nothing.
- **Cost:** communication cost = total I/O of all processes; computation cost = running time; total cost = the cloud bill, elapsed cost = wall-clock time; big-O is less useful because more machines are always an option.
- **Systems:** Google MapReduce + GFS (not public); Hadoop + HDFS (open source, Java); Hive and Pig on top of Hadoop; Amazon EC2 + S3 + EMR; readings are the GFS and MapReduce papers.

Theory Q&A bank

Lecture 2 · Slides 1–80

Cover the answer, write your own, then compare. Each model answer is short enough to write in an exam; numbered steps are the marking points. The slides pose Q1 directly (Slide 5: how long to read 200 TB) and the two design questions behind Q3 (Slides 13 and 15). Slide 80 tells you to treat all the worked examples as quiz questions, which is why the worked-example questions are included. The rest are likely exam questions built from the slides.

Q1. How long does it take one computer to read 200 TB, and how do we speed it up?

Lecture 2 · Slides 5–8

1. Data: 10+ billion pages × 20 KB = 2×10^{11} KB = 200 TB = 2×10^{8} MB (1 TB = 10^{6} MB).
2. Time = data ÷ bandwidth = 2×10^{8} MB ÷ 50 MB/s = 4×10^{6} s ≈ 46 days (4×10^{6} ÷ 86,400 = 46.3).
3. Fix: split the data into chunks, stripe it over many discs and process in parallel. With 1,000 discs and CPUs: 4×10^{6} ÷ 1,000 = 4,000 s ≈ 67 minutes.
4. The 4,000 s is ideal: it assumes an even split, independent discs at full speed, and no failures or result-combining cost. Reading is also only the start; useful work takes longer (Slide 7).

Q2. Describe the architecture of a compute cluster and rank the cost of moving data.

Lecture 2 · Slides 9–11

A **node** = CPU + memory + disk. A **rack** holds 16–64 nodes joined by one switch with 1 Gbps (= 125 MB/s) between any pair. Racks are joined through higher-level switches by a **backbone** of 2–10 Gbps; a data centre can hold ~1M machines (Google estimate). All of it is **commodity** hardware and Ethernet. Cheapest to read: data on the same node (no network); next: another node in the same rack; most expensive: another rack (shared backbone). This ranking is why the lecture stresses *locality*.

Q3. What challenges does cluster computing face, and how does Map-Reduce address them?

Lecture 2 · Slides 12–16

Challenge (slide)	Numbers	Map-Reduce idea (Slide 16)
Machines fail (12–13): how to store data persistently and keep it available, and how to survive failures during a long computation?	1 server up ~1,000 days: 1,000 servers lose 1 a day; 1M machines lose 1,000 a day	Store data **redundantly** on multiple nodes (DFS replicas); the framework re-runs lost tasks
Network bottleneck (14)	1 Gbps: moving 10 TB takes about a day	**Move computation close to the data**: chunk servers also compute
Distributed programming is hard (3, 15): how to make it easy?	—	**Simple programming model**: write Map and Reduce; the framework hides the complexity

Q4. What is a distributed file system? Explain chunks, replication, the master and how a read works.

Lecture 2 · Slides 17–24

1. A DFS (GFS at Google, HDFS in Hadoop) stores files across a cluster and keeps **each piece several times**, for redundancy and availability. It is designed for huge files (100s of GB to TB), rarely updated in place, mostly read and appended.
2. A file is cut into **chunks** of 16–64 MB stored on **chunk servers**. Each chunk is replicated 2× or 3× on **different servers** (never two copies on one server) and, if possible, in **different racks**.
3. The **master node** (Name Node in HDFS) keeps only **metadata**: which chunks make up a file and where their copies are. It may be replicated.
4. **Read:** the client library asks the master where the chunks are, then connects **directly to the chunk servers** to read the data. The data never passes through the master.
5. **Failure:** if a server dies, each affected chunk still has another copy, so reads continue; the lost copies are re-created elsewhere (seamless recovery).

Q5. What are the four stages of a Map-Reduce job, who writes what, and what does the environment handle?

Lecture 2 · Slides 27–28, 53–54

Stages: (1) **Map**: apply the Map function to each input record, emitting (key, value) pairs; (2) **Group by key** (sort and shuffle): bring all pairs with the same key together; (3) **Reduce**: aggregate the values of each key; (4) **write the result**. The programmer supplies only the **Map and Reduce functions** (and the input files); "the outline stays the same, Map and Reduce change to fit the problem". The environment takes care of: **scheduling**, **inter-machine communication**, **partitioning the data**, the **group-by-key** step and **machine failures**.

Q6. State the signatures of Map and Reduce. How many calls of each are made?

Lecture 2 · Slides 29–35, 43–44

Map(k, v) → <k', v'>*: one call per input pair; emits zero or more intermediate pairs. **Reduce(k', <v'>*) → (k', v'')**: one call per **unique intermediate key**, receiving all its values, emitting the aggregated pair. Example: "a rose is a rose" as one input pair gives 1 Map call, 5 pairs, 3 distinct keys and so 3 Reduce calls. A *call* is not a *task*: a task (run by a worker) makes many calls.

Q7. Design a Map-Reduce solution for (a) word count, (b) host size, (c) counting 5-word sequences.

Lecture 2 · Slides 26, 36–49

Method: ask "what must meet in one place to give one answer?"; that is the **key**.

Problem	Map emits	Group by key (framework)	Reduce
(a) Word count	(word, 1) for every word occurrence	(word, [1, 1, ...])	sum: (word, total)
(b) Host size	(hostname(URL), size) per record	(host, [sizes])	sum: (host, total bytes)
(c) 5-word sequences	(5-word window, 1) for each window position	(sequence, [1, 1, ...])	sum: (sequence, count)

Same skeleton each time; only the key, the emitted value and the Reduce operation change.

Q8. Explain how a Map-Reduce job runs in parallel on a cluster.

Lecture 2 · Slides 51–52, 54

1. The input is divided into pieces; each piece is processed by a **Map task** (many Map calls) on some worker.
2. At the end of each Map task a **partitioning function** looks at each pair's key and sends it to one of the **R Reduce tasks** (typically hash(key) mod R), so equal keys always go to the same reducer.
3. **Shuffle:** every Reduce task fetches its part from every Map task (all-to-all, M × R transfers).
4. Each Reduce task **sorts and groups** by key and calls Reduce once per key. Each Reduce task writes its own output piece.

All phases are distributed, with many tasks doing the work.

Q9. Where is data stored during a Map-Reduce job, and what does the master do?

Lecture 2 · Slides 55–56

Input and final output are in the **DFS** (replicated); map tasks are scheduled close to their input chunk. **Intermediate** (map output) is on the **local file system** of the map worker (not replicated). Output is often the input of another job. The **master** tracks each task's state (**idle, in-progress, completed**), schedules idle tasks as workers become free, receives the **location and sizes of intermediate files** from each completed map task and pushes this information to the reducers, and **pings workers** periodically to detect failures.

Q10. What happens when a map worker, a reduce worker or the master fails? Why the difference?

Lecture 2 · Slide 57

Failure	Action	Reason
Map worker	All its map tasks, **completed and in-progress**, reset to idle and rescheduled elsewhere	Output sits on the dead worker's **local disk**, so completed work is lost
Reduce worker	Only **in-progress** tasks reset to idle and restarted	Completed output is already in the replicated **DFS**
Master	Job **aborted**; client notified	Master holds all task state; Beyond the slides: with a single master, a failure during one job is unlikely

Detection is by the master's periodic pings.

Q11. Why should there be many more map tasks than machines? What about R?

Lecture 2 · Slides 58, 69

Rule of thumb: **M much larger than the number of nodes**, often one DFS chunk per map task; **R usually smaller than M**. Reasons: (1) **dynamic load balancing**: fast machines simply take more small tasks, so nobody idles while one machine is stuck with a big task; (2) **faster recovery**: a failed worker loses only small tasks, which are re-run in parallel on many other machines. Also, **pipelining**: the shuffle reads of a finished map task start while other maps still run; only the reduce function must wait for all its reads. Example: 1 TB in 64 MB chunks gives 15,625 map tasks, about 16 per node on 1,000 nodes. (Beyond the slides: R is small because each reduce task writes one output file.)

Q12. What is a combiner? When can it be used? How do you handle average and median?

Lecture 2 · Slides 59–65

1. A **combiner** pre-aggregates the values of each key **inside the mapper**, before the shuffle: combine(k, list(v_1)) → v_2. It reduces the data copied and shuffled (Slide 61: 12 pairs become 8).
2. It is usually the same as the reduce function and works **only if reduce is commutative and associative** (sum, count, max, min).
3. **Average** is not (averaging averages ignores sizes: parts {1,2,3} and {4} give (2+4)/2 = 3, but the true average is 2.5). Fix: carry **(sum, count)** pairs; combiners add them component-wise; the reducer adds and divides once.
4. **Median**: there is a proof that it cannot be split into commutative and associative functions, so no combiner trick; the reducer needs all the values.

The answer must be the same with or without the combiner.

Q13. What is the partition function and why override it?

Lecture 2 · Slides 66–67

It decides which reduce task receives each intermediate key. Requirement: **records with the same key must end up at the same reducer**. Default: **hash(key) mod R** (a number 0 ... R−1). Override when you want related keys together: **hash(hostname(URL)) mod R** sends all URLs of one host to the same reduce task, hence the same output file. The key itself is unchanged; only the routing uses a function of the key. Risk (beyond the slides): load imbalance if one host is huge.

Q14. What is a straggler and how are stragglers handled?

Lecture 2 · Slide 68

Problem: a slow (not failed) worker, due to other jobs, a bad disk or "weird things", delays the whole phase because a phase ends when its last task ends. **Solution:** near the end of a phase the master launches **backup copies** of the remaining in-progress tasks on other machines; **whichever copy finishes first wins** and the other is discarded. **Effect:** job completion time drops dramatically for a small extra cost. Safe because tasks are independent and deterministic. Different from failure recovery: here the worker still works, just slowly.

Q15. Explain how to compute the natural join R(A,B) ⋈ S(B,C) with Map-Reduce.

Lecture 2 · Slides 71–72

1. **Key:** the shared attribute b, because tuples to be glued share it.
2. **Map:** R tuple (a,b) → (b, (a, R)); S tuple (b,c) → (b, (c, S)). The **tag** R or S lets the reducer tell the two sides apart.
3. **Shuffle:** pairs with key b go to Reduce process h(b), h a hash function (Hadoop does this automatically).
4. **Reduce:** for each b, pair every (a,R) with every (c,S) and output (a,b,c). A b present on one side only gives no output.

On the Slide 71 data: 4 + 3 = 7 pairs shuffled; b_1 has no S partner; output (a_3,b_2,c_1), (a_3,b_2,c_2), (a_4,b_3,c_3).

Q16. How do we measure the cost of a Map-Reduce algorithm? Why not big-O?

Lecture 2 · Slides 73–74

Communication cost = total I/O of all processes (how much data is read and written and moved). State your convention in an exam: counting every read and write (literal Slide 73), or counting only task inputs (textbook). **(Elapsed) computation cost** = analogous, but counting running time. Usually one of the two dominates and the other is ignored. **Total cost** (sum over all machines) is the bill from "your friendly neighborhood cloud"; **elapsed cost** is wall-clock time using parallelism. Adding machines lowers elapsed cost but not total cost. Big-O is "not the most useful" because adding more machines is always an option; what matters is how much data moves and how long you wait. (Slide 80 also advises treating all examples in the lecture as quiz questions: word count, host size, language model, combiner example and join.)

One-page cheat sheet

Lecture 2 · Slides 1–80

Numbers to know (decimal prefixes)

Lecture 2 · Slides 5–14

Fact	Value / derivation	Slide
Web data	10+ billion pages × 20 KB = 200 TB = 2×10^8 MB	5
Read time, 1 disc at 50 MB/s	$2 \times 10^8 \div 50 = 4 \times 10^6$ s ≈ 46.3 days (disc range 30–50 MB/s)	6
Read time, 1,000 discs	$4 \times 10^6 \div 1{,}000 = 4{,}000$ s ≈ 67 min (ideal)	8
Rack, in-rack link, backbone	16–64 nodes; 1 Gbps = 125 MB/s (bits ÷ 8); backbone 2–10 Gbps	9–10
Machines at Google	~1M (guesstimate)	10, 12
Failures	machines ÷ lifetime (1,000 days): 1,000 → 1 a day; 1M → 1,000 a day (≈ one every 86 s)	12
Move 10 TB at 1 Gbps	8×10^{13} bits ÷ 10^9 = 80,000 s ≈ 22 h ≈ 0.93 day ("about 1 day")	14
Chunk size, replication	16–64 MB; 2× or 3×; 1 TB at 64 MB = 15,625 chunks	19–20

Definitions

Lecture 2 · Slides 12–69

Term	Meaning
Three ideas (16)	store data redundantly; move computation close to the data; simple programming model
DFS (17)	stores each piece of data multiple times; redundancy + availability; GFS / HDFS
Master / Name Node (24)	metadata only (where chunks are); may be replicated; client reads directly from chunk servers
Map (43)	(k, v) → list of (k', v'); one call per input pair
Reduce (44)	(k', list of v') → (k', v''); one call per unique key
Environment (53)	scheduling, communication, partitioning, group by key, failures (S-C-P-G-F)
Task states (56)	idle, in-progress, completed
Combiner (60)	mapper-side pre-aggregation; needs commutative + associative reduce
Partition function (66)	hash(key) mod R; same key, same reducer
Straggler / backup task (68)	slow worker; duplicate near the end of a phase; first to finish wins
M and R (69)	M ≫ nodes (one chunk per map task); R < M
Costs (73–74)	communication = total I/O; computation = running time; total = bill; elapsed = wall clock

Algorithms in steps

Lecture 2 · Slides 26–72

Algorithm	Steps
Word count	1. Map: emit (w, 1) for every word. 2. Framework groups by key. 3. Reduce: sum the 1s. Output (w, n).
Host size	1. Map: (hostname(URL), size). 2. Group by host. 3. Reduce: sum sizes.
5-word sequences	1. Map: slide a 5-word window, emit (window, 1) (n − 4 pairs for n words). 2. Group. 3. Reduce: sum.
Join R ⋈ S	1. Map: (b,(a,R)) and (b,(c,S)). 2. Shuffle by h(b). 3. Reduce: all (a,R) × all (c,S) → (a,b,c).
Average with combiner	1. Map: (k,(value,1)). 2. Combine: add (sum,count) component-wise. 3. Reduce: add, then sum ÷ count.
Read from the DFS	1. Client library asks master for chunk locations. 2. Master replies with servers. 3. Client reads from chunk servers directly.
Parallel run	1. Map tasks on input pieces. 2. Partition by hash(key) mod R. 3. Shuffle to R reduce tasks. 4. Sort and group. 5. Reduce, write to DFS.

Comparisons

Lecture 2 · Slides 55–77

Data	Stored on	Survives worker crash?
Input chunks, final output	**DFS (replicated)**	**yes**
Intermediate (map output)	map worker's local disk	no

Reduce function	Combiner?	Why
Sum, count, max, min	**yes**	commutative and associative
Average	**yes, with (sum, count)**	divide only at the end
Median	no	proved impossible

	Google	Open source / cloud
Storage	GFS (master, chunk server, chunk)	HDFS (Name Node, Data Node, block)
Map-Reduce	Google MapReduce (not available outside Google)	Hadoop (open source, Java); Hive and Pig give SQL-like abstractions on top
Cloud (77)	rent computing by the hour: Amazon EC2 (compute), S3 (storage), EMR (Elastic Map Reduce)	

Common mistakes & traps

Lecture 2 · Slides 1–80

- **Units:** Gbps is bits per second, GB/s is bytes. 1 Gbps = 125 MB/s. Convert 1 TB = 10^6 MB before dividing by MB/s; otherwise answers are off by 10^3 or 8 (Slides 5, 9, 14).
- **The 4,000 s is ideal.** Never write "1,000 discs = 1,000× faster" without the assumptions (even split, no failures, no combining cost) (Slide 8).
- **The master does not carry data.** It stores metadata; clients read from chunk servers directly. It does not store file bytes (Slide 24).
- **Replicas on one server are useless.** Copies must be on different servers, and racks if possible. Do not read replica counts from Slide 23, which is only an illustration (Slides 20–23).
- **Group by key is not written by the programmer.** Only Map and Reduce are; the framework does sort and shuffle (Slides 28, 36–38, 53).
- **Calls versus tasks.** One Reduce call per unique key, but one Reduce task handles many keys; Map is not "one in, one out" (zero or more pairs) (Slides 30, 44, 52).
- **Intermediate data is on the local disk, not in the DFS** (Slide 55). **Failure rules:** a map worker failure redoes completed *and* in-progress map tasks; a reduce worker failure redoes only in-progress tasks; master failure aborts the job (Slide 57).

- **Combiner is only an optimisation.** The answer must not change; reduce must be commutative and associative. Never average partial averages; never divide in the combiner (Slides 62–64). Median has no combiner (Slide 65).
- **Partition function must send equal keys to the same reducer.** A random choice breaks the result (Slide 66).
- **Reducers do not compute before all maps finish** (only the copying starts early, Slide 58). **Stragglers are not failures**: backups are launched near the end of a phase and the first copy to finish wins (Slide 68).
- **Join:** forgetting the R/S tag, outputting rows for keys present on one side only, or writing the result as (A,C) instead of (A,B,C) (Slides 71–72).
- **Cost:** more machines lower elapsed cost, not total cost; big-O is not the main measure (Slides 73–74).

LECTURE 3

Spark: Beyond Map-Reduce (RDDs, DataFrames, the Ecosystem)

ROADMAP — WHAT THIS LECTURE IS ABOUT

- Map-Reduce (Lecture 2) saved the day for scaling out, but it pays for **disk I/O between every step** and it **does not compose**: real programs need chains of jobs (slides 3–6).
- The fix is a **data-flow system**: any number of stages, any functions, data moving in one direction only (a **DAG**, slide 7).
- **Apache Spark** is the most popular one: a core engine plus Spark SQL, Streaming, MLlib and GraphX, connected to many data stores (slides 8–10).
- Its key construct is the **RDD** (Resilient Distributed Dataset): a partitioned, read-only collection cached in memory, changed only through lazy **transformations** and result-producing **actions** (slides 11–16).
- The **task scheduler** runs general DAGs in stages, pipelining steps and avoiding shuffles (slide 17); **DataFrames and Datasets** add structure and SQL (slides 12, 18–19); the libraries are listed on slide 20.
- Finally Spark is compared with Hadoop MapReduce and we ask which problems suit MapReduce at all (slides 21–23).

Exam checklist — by the end you can:

- state what Map-Reduce gave us and its **two limitations** (performance, programmability), and explain "does not compose well";
- define a **data-flow system** and a **DAG**, and say how they generalise Map-Reduce;
- list what **Spark adds** to MapReduce (fast data sharing, general DAGs, richer functions, Hadoop compatibility) and the weakness each one fixes;
- define an **RDD** (partitioned, read-only, cached with disk fallback, created from stored data or other RDDs) and explain "resilient" and lineage;
- separate **transformations** from **actions**, and explain **lazy evaluation** and why it helps (trace a lineage graph);
- explain what the **task scheduler** does: stages, shuffles, pipelining, cache and partition awareness;
- contrast **RDD, DataFrame and Dataset**; name the Spark libraries and describe GraphX's graph type;
- **compare Spark and Hadoop MapReduce** (with the memory caveat) and decide which problems suit MapReduce.

Scope note: no code is examined in Lectures 1–5. Slide code is explained in words only. This lecture contains no BFR material.

Slide map

Lecture 3 · Slides 1–24

For Lecture 3 the printed slide number equals the PDF page number. Slide 22 is an unnumbered section divider but is still page 22.

Slide(s)	PDF page(s)	Topic	In this guide
1–2	1–2	Title slide and the section divider "Spark: Extends Map Reduce"	(p. 123)
3	3	Map-Reduce recap: search engines, scale up or scale out, what MapReduce gave us	(p. 124)
4	4	The MapReduce execution overview (fork, assign, read, local write, remote read, write)	(p. 125)

5–6	5–6	Problems with MapReduce: disk I/O, two limitations, "does not compose well"	(p. 126)
7	7	Data-flow systems: generalise to any ranks and functions; DAG	(p. 128)
8	8	The Spark software stack: core, Streaming, GraphX, MLlib, SparkSQL	(p. 129)
9	9	Data lakes: Spark connects to many storage systems	(p. 129)
10	10	Spark, the most popular data-flow system: additions to MapReduce	(p. 130)
11	11	Spark overview: the RDD, transformations and actions	(p. 131)
12	12	Spark overview: higher-level APIs, DataFrames and Datasets, SQL	(p. 133)
3–12	3–12	Part 1 summary: problem and fix, side by side	(p. 134)
13–14	13–14	The RDD in detail: definition, properties, data flow figure, partitions in memory and on disk	(p. 134)
15	15	RDD operations: transformations, actions, lazy evaluation	(p. 137)
16	16	Lineage graph with two actions	(p. 139)
17	17	Task scheduler: general DAGs, stages, pipelining, cache and partitioning awareness	(p. 139)
18–19	18–19	DataFrame and Dataset; RDD versus DataFrame code comparison (code not examined)	(p. 141)
20	20	Useful libraries: Spark SQL, Streaming, MLlib, GraphX (directed multigraph)	(p. 142)
21	21	Spark versus Hadoop MapReduce: performance, ease of use, data processing	(p. 142)
22–23	22–23	Problems suited and NOT suited to MapReduce (sequential versus random access)	(p. 143)
21, 23	21, 23	Decision checklist: MapReduce or Spark?	(p. 144)
24	24	Tutorial and further reading	(p. 144)
1–24	1–24	Theory Q&A, cheat sheet and traps for the whole lecture	(p. 145)

Where this lecture starts: Spark extends Map-Reduce

Lecture 3 · Slides 1–2

Slide 1 is the title card ("Spark and Tutorial", presenter Rafael Dolores). The large black block on it is an empty picture area, not content. The word "Tutorial" signals that a hands-on Spark session follows the theory; that part is outside this section. Slide 2 is a section divider with one line: **"Spark: Extends Map Reduce"**.

That one line is the storyline of Slides 3–12. Spark is presented as a *generalisation* of the Map-Reduce model from Lecture 2, not as something unrelated. The slides follow a fixed argument:

1. **Recap** what Map-Reduce gave us (Slides 3–4).
2. **Criticise** it: two limitations (Slides 5–6).
3. **Generalise** it: data-flow systems (Slide 7).
4. **Introduce Spark**: its ecosystem, what it adds, its key data type (Slides 8–12).

> **KEY IDEA**
> Every feature of Spark on Slides 10–12 is an answer to a specific weakness of Map-Reduce on Slides 5–6. When you study, always pair them: *problem → Spark's fix*. A table doing exactly this is at the end of this part.

SPELLING NOTE

The slides write "Map Reduce", "MapReduce" and "map-reduce" for the same thing. This guide uses "Map-Reduce" for the model and "MapReduce" when the slide does.

Map-Reduce recap: why we scale out

Lecture 3 · Slide 3

Search engines, scale up and scale out

Lecture 3 · Slide 3

Slide 3 opens with history. **Search engines** (the slide says "i.e. Google"; it should read "e.g.", because Google is just an example) created one of the earliest big-data problems. To answer a search query quickly, the engine must **crawl** (automatically download) billions of web pages and build an index over them. That volume does not fit on one computer.

The slide then asks: **"Scale up or scale out?"** It never answers in words; the colours do (scale up in red, scale out in green). The answer is **scale out**.

DEFINITION: SCALE UP VS SCALE OUT

Scale up (vertical scaling): make *one* machine bigger, with more CPU, RAM and disk. **Scale out** (horizontal scaling): add *many* ordinary (commodity) machines and let them work together as a cluster.

BEGINNER INTUITION

You must move 10 000 boxes. Scaling up is buying one gigantic truck: very expensive, there is a limit to how big a truck can be, and if it breaks down all work stops. Scaling out is hiring 100 ordinary vans: cheap, you can add a van any time, and one broken van only delays a little. The price is organisation: someone must decide which van takes which boxes.

	Scale up	Scale out
How	One bigger machine	Many commodity machines in a cluster
Cost	Grows steeply; special hardware	Grows roughly in proportion to the machines added
Limit	Hard ceiling (biggest machine you can buy)	Keep adding machines
Failure	Single point of failure	Failures are normal, so software must tolerate them
Hard part	Cost	Storage, coordination and programming across machines
Choice for big data	×	✓

The Cost, Limit and Failure rows are background (Beyond the slides: standard cost and failure arguments from the textbook); the slide itself only gives the question and the colour hint. Scaling out creates three new hard problems, storage, coordination and programming, and the next bullet of the slide says who solves them.

What Map-Reduce gave us

Lecture 3 · Slide 3

"The **MapReduce** programming model saved the day." The slide lists four things it provides:

Slide bullet	Plain meaning	Where it came from (Lecture 2)
Handles persistent storage	Files are stored safely across many machines and survive crashes.	The distributed file system (HDFS/GFS): files split into chunks, each chunk replicated.

Handles distributed worker coordination	The framework decides which worker does which task, notices failed workers, and restarts their tasks. (The slide has a stray "'s": "coordination's".)	The master process.
Brings computation closer to the data	Move the small program to the machine that holds the data, not terabytes of data to the program. This is called **data locality**. (The slide says "Bring"; it should be "Brings".)	The master schedules map tasks on the machines (or racks) that hold the input chunk.
Simpler to work with from the programmer's POV (point of view)	The programmer writes only `map` and `reduce` functions; everything above is hidden.	The Map-Reduce programming model.

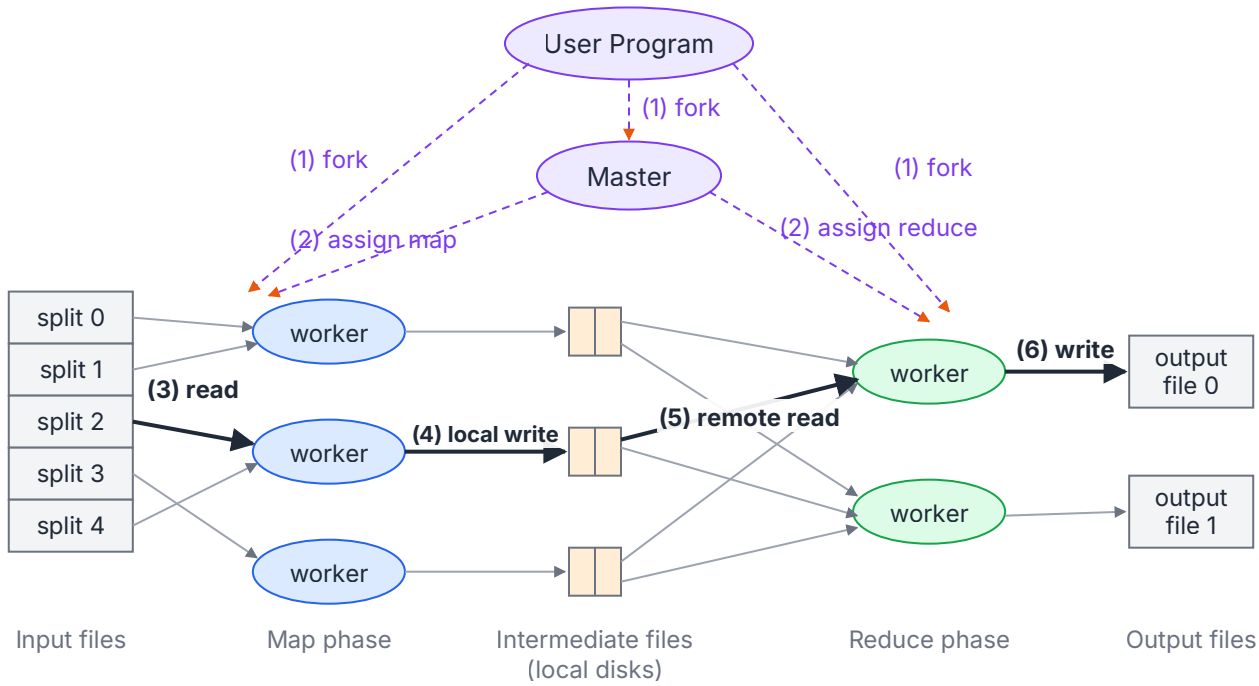
EXAM TIP

A typical short question: "What problems does MapReduce solve for the programmer?" Answer with the four bullets above and add the one-line reason: scaling out creates storage, coordination and programming problems, and MapReduce hides all three.

The MapReduce execution overview

Lecture 3 · Slide 4

Slide 4 contains no text, only a diagram (it is the classic "execution overview" figure from the 2004 Google MapReduce paper by Dean and Ghemawat; the slide gives no citation). Our redrawing below keeps all its parts and its six numbered steps. Read the figure from the top (control) to the bottom (data).



Dashed = control messages. Dark = numbered example path.
Grey = same flows for the other tasks.

Figure 3.1. MapReduce execution overview (redrawn from Lecture 3, Slide 4). Intermediate files sit on the map workers' local disks; every intermediate file feeds both reducers (the shuffle).

The six steps, in words

Lecture 3 · Slide 4

1. **(1) fork.** The **User Program** starts (forks) copies of itself: one **Master** and many **workers**.

2. **(2) assign map / assign reduce.** The master hands tasks to idle workers: some get map tasks, some get reduce tasks.
3. **(3) read.** A map worker reads its **input split** (one chunk of the input file; split 2 in the figure).
4. **(4) local write.** It runs the user's map function and writes the resulting intermediate key-value pairs to its **own local disk** (not to the distributed file system).
5. **(5) remote read.** Each reduce worker fetches ("pulls") the pieces it needs from *every* map worker's local disk. This all-to-all fan-out is the **shuffle**.
6. **(6) write.** The reduce worker runs the reduce function and writes one **output file**.

EXAM TIP

Remember three counting facts: the number of map tasks follows the number of input splits; the number of reduce tasks (and so of output files) is chosen by the user; and intermediate data lives on **local disks**. Only the final output goes to the distributed file system. Only one path is drawn in black on the slide, but steps 3–6 happen for every task.

COMMON MISTAKE

Do not confuse the control flow (dotted arrows 1 and 2, small messages) with the data flow (solid arrows 3–6, the actual data). Another frequent error is to say map output is written to HDFS; it is written to the mapper's local disk. This matters again on the next slide.

Problems with MapReduce

Lecture 3 · Slides 5–6

Limitation 1: disk I/O overhead

Lecture 3 · Slide 5

Slide 5 states the first problem: **MapReduce incurs substantial overheads due to disk I/O** (input/output). The sub-bullet explains where: the **outputs of mappers (M) are saved on the disk and then read again by reducers (R)**.

The slide's figure shows a chain: *Data on Disk* (input from stable storage) → HDFS read → mappers M1–M3 → HDFS write → *Tuples (on Disk)* → HDFS read → reducers R1–R3 → HDFS write → *Tuples (on Disk)* → "..." (the chain goes on). **HDFS** (Hadoop Distributed File System) is the disk-based, replicated file system of Lecture 2. Every arrow into a box is a disk read, every arrow out is a disk write.

BEGINNER INTUITION

Imagine a team of clerks. After finishing each step of a task, every clerk must file the result in a basement archive, and the next clerk must walk down and fetch it before starting. The basement trips take far longer than the actual work. Keeping the paper on the desk (memory) between steps would be much faster. That is the whole difference between MapReduce and Spark.

Disks (and the network used to replicate files) are orders of magnitude slower than RAM. For a single pass over data this does not matter much. It matters hugely for **multi-step** work, for example iterative algorithms such as machine learning or PageRank, where the same data is processed again and again: every pass pays the disk price.

Worked example — counting the disk touches Lecture 3 · Slide 5

A job needs k successive MapReduce jobs, each reading its input from HDFS and writing its output to HDFS. (Practice example with made-up speeds, not from the slides.)

1. **Disk touches in a chain of k jobs:** k reads + k writes = 2k. For k = 5 that is 10; for k = 10 it is 20.
2. **Minimum actually needed:** read the original input once, write the final result once = 2, whatever k is.
3. **Time of one pass over 100 GB:** at an assumed 100 MB/s disk speed, 100 000 MB ÷ 100 MB/s = 1000 s. At an assumed 10 000 MB/s memory speed, 100 000 ÷ 10 000 = 10 s, which is 100 times faster.

Result: 2k − 2 of the disk touches are wasted work; Spark's design removes them by keeping intermediate results in memory.

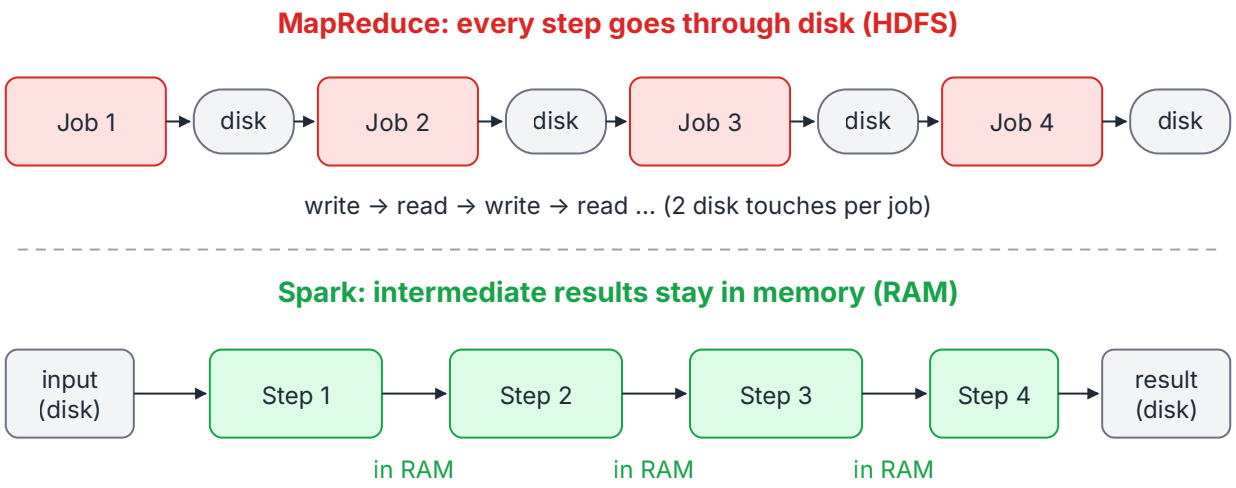


Figure 3.2. The central contrast of Slides 5, 6 and 10: a chain of MapReduce jobs writes to and reads from disk between every step; Spark passes data between steps in memory.

SLIDE ERRATUM

Slide 5's figure (a) labels the map stage "Iteration-1" and the reduce stage "Iteration-2". In reality one MapReduce job already contains a map phase *and* a reduce phase; a second iteration is a second whole job. (b) It draws "HDFS write" after the mappers, while Slide 4 (and real Hadoop) puts map output on the mappers' *local* disks; HDFS is written only for reducer output. The text of the slide ("saved on the disk and then read again by reducers") is correct either way, and the conclusion (heavy disk I/O) stands. In the exam, follow the slide's message: mapper output goes to disk, reducers read it back.

The two major limitations, side by side

Lecture 3 · Slide 6

Slide 6 names **two major limitations of MapReduce** and then sums them up in one sentence:

Limitation	Slide's explanation	What it means in practice
1. Performance bottlenecks	Saving to disk is typically much slower than in-memory work.	The disk-I/O overhead of Slide 5. Each step pays for writes and reads.
2. Potential difficulty of programming directly in MapReduce	Some big-data problems/ algorithms are not easily described as map-reduce.	The rigid shape "one map, then one reduce" does not fit many algorithms (iterative, multi-stage, graph, interactive). The programmer must force them into it.

KEY IDEA: "DOES NOT COMPOSE WELL"

The summary line is: **"MapReduce does not compose well for all large applications"**, because **many times one needs to chain multiple map-reduce steps**. "Compose" means building a big program out of small pieces. With MapReduce each piece is a separate job; the pieces can only be glued together through files on disk, so each link of the chain pays limitation 1, and the programmer carries limitation 2 (writing and managing the chain by hand).

EXAM TIP

"State the two limitations of MapReduce" is a likely question. Answer: (1) performance, because intermediate results go to disk which is much slower than memory; (2) programmability, because many problems do not naturally fit one map plus one reduce and need chains of jobs, so MapReduce does not compose well.

Data-flow systems

Lecture 3 · Slide 7

Slide 7 shows the abstract fix. Start with what Map-Reduce looks like from a distance: **two "ranks" of tasks**, one for Map and a second for Reduce, and data flows from the first rank to the second. ("Rank" is the slide's word; "stage" or "layer" is more common.)

DEFINITION: DATA-FLOW SYSTEM

A system in which a computation is a network of tasks and data flows through it. It **generalises** MapReduce in two ways: (1) it allows **any number of tasks/ranks**, and (2) it allows **functions other than Map and Reduce**. Data flows in **one direction only**, so the network is a **DAG** (directed acyclic graph). The slide leaves out the closing parenthesis after "directed acyclic graph".

DEFINITION: DAG

A graph whose nodes are tasks and whose arrows are the data flowing from one task to the next (**directed**), with no way to start at a node and follow arrows back to it (**acyclic**, no loops).

BEGINNER INTUITION

MapReduce is an assembly line with exactly two stations. A data-flow system is an assembly line with as many stations as you need, and each station may do any job (filter, join, sort, group, ...). The only rule: a product never goes back to an earlier station.

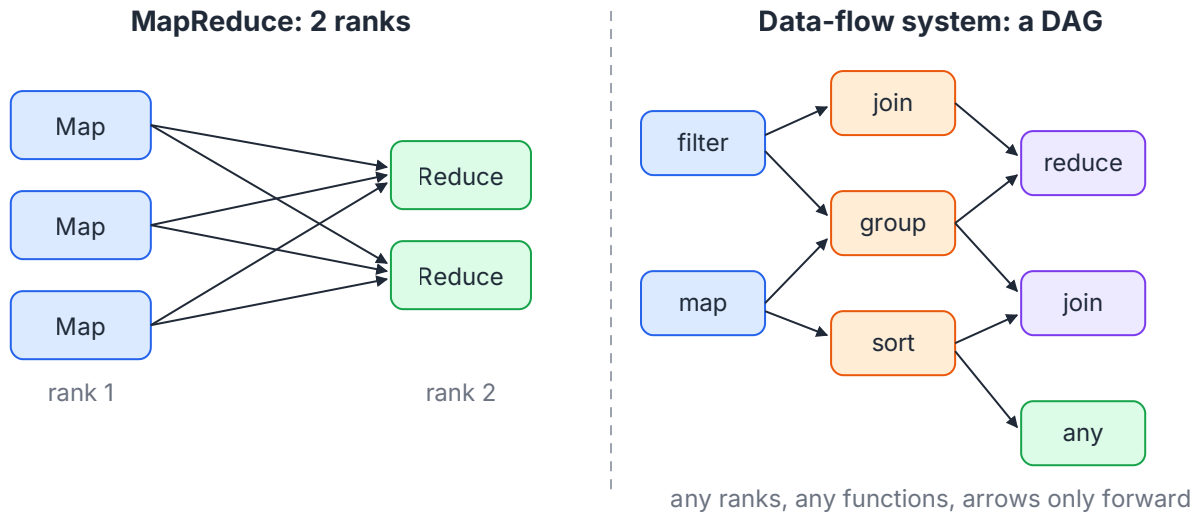


Figure 3.3. Slide 7 drawn out. Left: MapReduce's fixed two ranks. Right: a data-flow system, where any functions can be wired into a forward-only graph (a DAG). The right-hand graph is illustrative; the slide has no figure.

SLIDE ERRATUM

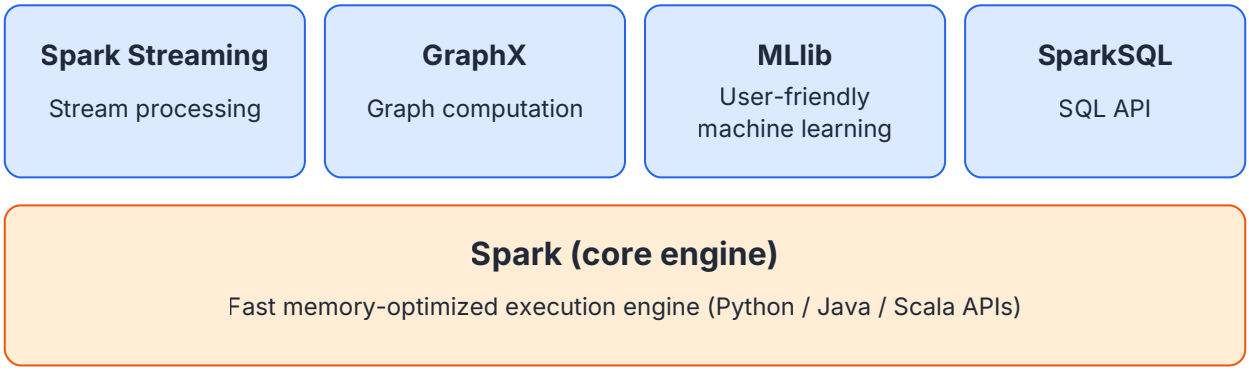
On Slide 7 the bullet "Data flow is in one direction only (DAG ...)" is printed under item 2, but it is a property of the *whole* generalisation (items 1 and 2 both), not of item 2 alone.

Beyond the slides: a DAG is what lets the system plan the whole computation in advance and, after a failure, re-run only the lost part rather than everything. Loops in an algorithm (iterations) are expressed by writing a longer acyclic graph, not by a cycle in the graph.

The Spark software stack

Lecture 3 · Slide 8

Slide 8 ("Data Analytics Software Stack") is one picture: four library boxes sitting on top of a single wide engine box. We redraw it with a plain explanation of each box.



One engine underneath, four libraries on top

Figure 3.4. The Spark stack (Lecture 3, Slide 8). All four libraries run on the same core engine.

Box	Slide's subtitle	What it is for
Spark (core)	Fast memory-optimized execution engine (Python/ Java/Scala APIs)	The engine that runs tasks over a cluster, keeping data in memory. You program it from Python, Java or Scala.
Spark Streaming	Stream processing	Processing live data that keeps arriving (clicks, sensor readings), (Beyond the slides: classic Spark Streaming processes the stream in small batches.)
GraphX	Graph computation	Algorithms on graphs (Beyond the slides: for example PageRank).
MLlib	User-friendly machine learning	Ready-made machine-learning algorithms (classification, clustering, ...).
SparkSQL	SQL API	Query structured data with SQL; this is where DataFrames come in (Slide 12).

> **KEY IDEA**
> **One engine, many workloads.** Streaming, graphs, machine learning and SQL all reuse the same fast core, so one system and one skill set covers jobs that used to need separate specialised systems. (That last comparison with Hadoop is background; the slide only shows the stack.)

Beyond the slides: Spark also has an R API, and newer versions add "Structured Streaming" next to Spark Streaming. The slide does not show the cluster managers or storage layers beneath Spark; the next slide covers the storage side.

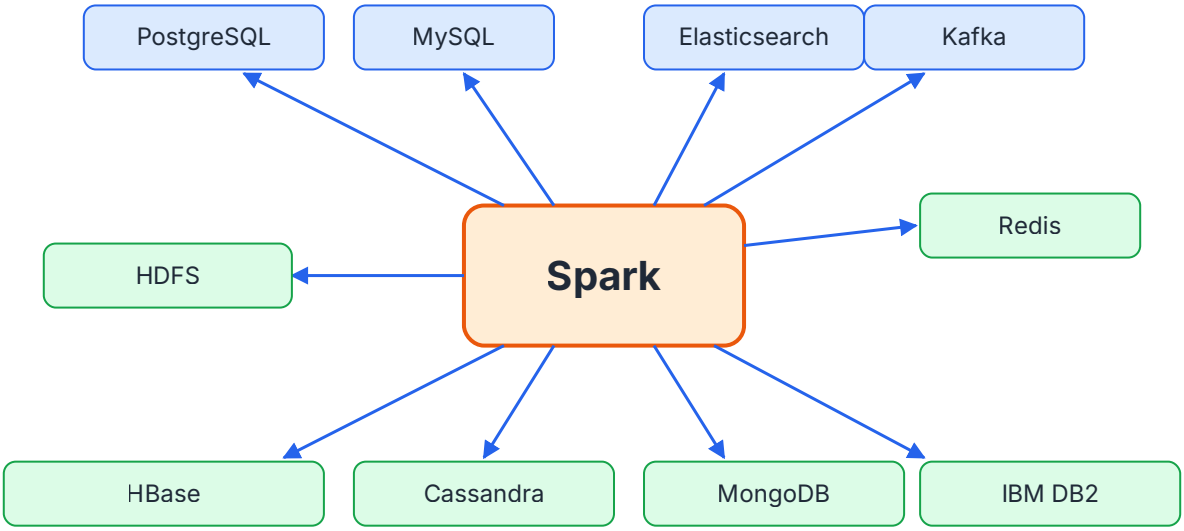
Data lakes: Spark connects to everything

Lecture 3 · Slide 9

Slide 9 is titled **"Data Lakes"** and shows the Spark logo in the centre with ten arrows leading out to logos. The slide never defines the term.

DEFINITION: DATA LAKE
Beyond the slides: a data lake is a central repository that keeps large amounts of data in its original (raw) format, structured and unstructured, from many different sources, to be processed later. Compare it with a data warehouse, which stores cleaned data in a fixed schema.

The picture's message: Spark is **not tied to one storage system**. It has connectors to all of these, so it can serve as the common compute layer over a company's data lake.



Blue: relational / search / messaging. Green: NoSQL / Hadoop.

Figure 3.5. Slide 9: Spark as the common compute layer over many storage systems (colours are our grouping, not the slide's).

Kind of system	Logos on the slide
Relational databases (tables, SQL)	PostgreSQL, MySQL, IBM DB2
NoSQL stores (flexible, scale out)	MongoDB, Cassandra, HBase, Redis
Search engine	"elastic" (Elasticsearch)
Messaging / streaming platform (not a database)	Kafka
Distributed file system (Hadoop)	HDFS

HDFS and HBase belong to the Hadoop ecosystem, which links to the bullet "Compatible with Hadoop" on the next slide. The slide draws the arrows one-way (from Spark to the system), but Spark both reads from and writes to these systems. You are not expected to know details of each product; know that the list shows *variety*.

Spark: the most popular data-flow system

Lecture 3 · Slide 10

Slide 10 collects the answer. Spark is described as an **expressive computing system, not limited to the map-reduce model**, and "the most popular data-flow system" (a claim with no source or date). Its **additions to the MapReduce model** are:

Addition on the slide	Meaning	Which MapReduce weakness it fixes
Fast data sharing: avoids saving intermediate results to disk	Results pass from one step to the next in memory.	Performance bottleneck (Slides 5–6)

Fast data sharing: caches data for queries (e.g. for machine learning)	Data that will be used again is kept in memory. Repeated queries and iterative learning algorithms re-read it from RAM, not from disk. (The slide mixes "queries" and "machine learning" in one bullet; they are two different uses.)	Performance bottleneck, especially for iteration
General execution graphs (DAGs)	Any acyclic graph of tasks, not just map then reduce. (The slide writes "DAGs=directed acyclic graph", a plural/singular mismatch.)	"Does not compose": no more hand-built job chains (Slide 6)
Richer functions than just map and reduce	Many operators are built in (filter, join, group, sort ...).	Difficulty of programming (Slide 6)
Compatible with Hadoop	Beyond the slides: Spark can read data from HDFS and run on existing Hadoop clusters, so Hadoop data and infrastructure are reused.	Not a fix; it makes adoption easy

QUICK RECAP: PROBLEM → FIX

- Disk I/O between steps → **fast data sharing** in memory, plus caching.
- Only map and reduce, jobs chained by hand → **general DAGs** and **richer functions**.
- Existing Hadoop world → **compatible with Hadoop**.

EXAM TIP

"What does Spark add to MapReduce?" has a three-part model answer: fast data sharing (in-memory, no intermediate disk writes, caching), general execution graphs (DAGs), richer functions than map and reduce; and it stays compatible with Hadoop. Also be able to say *why* each addition helps.

COMMON MISTAKE

Do not write "Spark is always faster because it does not use disk." Spark can use disk (for example when data does not fit in memory) and still reads its input from and writes its final result to disk-based storage. The gain comes from not touching disk *between* steps (Figure 3.2).

The key construct: the RDD

Lecture 3 · Slide 11

Slide 11 ("Spark Overview") gives the **key construct/idea**: the **Resilient Distributed Dataset (RDD)**. Its two sub-bullets: RDDs are a data abstraction in Apache Spark that allow **in-memory processing of large datasets**; and they provide **transformations** (e.g. map, filter) and **actions** (e.g. count, collect) through an **API** (application programming interface, the set of functions you may call), enabling distributed computation on data loaded into memory.

DEFINITION: RDD

A **Resilient Distributed Dataset** is Spark's central data type: a large collection of records, split into **partitions** that live on different machines of the cluster and are kept, as far as possible, in memory.

The three words in the name

Lecture 3 · Slide 11

Word	Meaning
Dataset	A collection of records (lines of text, numbers, key-value pairs ...).
Distributed	Its partitions are spread over many machines, so the machines work on their own pieces in parallel.

Resilient	It survives machine failures: a lost partition can be rebuilt. (The slide does not explain this word. Beyond the slides: Spark remembers how each RDD was derived from earlier ones, its **lineage**, and replays those steps for the lost partition only.)

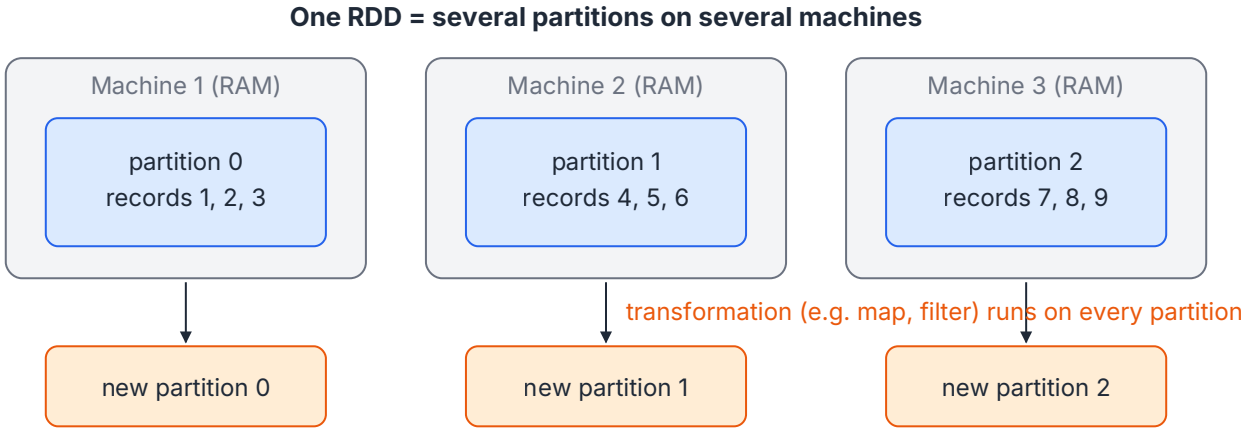


Figure 3.6. An RDD (illustrative, not on the slide): one logical dataset, cut into partitions held in the memory of different machines. A transformation produces a new RDD, partition by partition.

Transformations and actions

Lecture 3 · Slide 11

You work with an RDD through two kinds of operations, both called through the API:

	Transformation	Action
Slide's examples	`map`, `filter`	`count`, `collect`
What it does	Builds a *new* RDD from an existing one	Returns a result to the driver program (the program you run); for example a number (count) or the data itself (collect)
Output type	An RDD	A value, not an RDD

Worked example — which is which? Lecture 3 · Slide 11

An RDD holds the numbers 1, 2, 3, 4, 5, 6. Apply `filter` keeping even numbers, then `map` doubling each, then `count`. (Practice example, not from the slides.)

1. `filter` (transformation): gives a new RDD {2, 4, 6}.
2. `map` (transformation): gives a new RDD {4, 8, 12}.
3. `count` (action): returns the number 3 to the driver.

Answer: two transformations produce RDDs; one action produces the value 3.

EXAM TIP

The slide's own examples are the safest memory aid: **map, filter = transformations; count, collect = actions**. Rule of thumb: if it returns an RDD, it is a transformation; if it returns a result to you, it is an action.

BEYOND THE SLIDES: WHAT THE SLIDE LEAVES OUT

Three properties are not on Slide 11 but are central to how RDDs behave, and are the usual follow-up questions. RDDs are **immutable**: a transformation never changes an RDD, it creates a new one. They are **fault tolerant through lineage** (the recorded chain of transformations that produced them). Transformations are **lazy**: nothing is computed when you call them; Spark only records the plan, and the whole chain actually runs when an action asks for a result. Laziness is what lets Spark see the whole DAG before running it.

NOT EXAMINED — CODE (UNDERSTAND, DON'T MEMORISE)

The slides later show Spark code for these operations. In words: you load data as an RDD, call `filter` and `map` to describe the steps (nothing runs yet), then call `count` to trigger the work. You will not be asked to write such code.

SLIDE ERRATUM

Slide 11's sentence "through API, enabling distributed computation on data loaded into memory" is missing an article ("through an API") but the content is right. "Resilient" is never explained on the slide (see the table above).

Higher-level APIs: DataFrames and Datasets

Lecture 3 · Slide 12

Slide 12 (still "Spark Overview") names the **higher-level APIs: DataFrames and Datasets**. They were introduced in **more recent versions** of Spark, and are **different APIs for structured data, which allowed SQL support to be introduced** (the slide's wording "allowed to introduce SQL support" is ungrammatical, not wrong).

DEFINITION: DATAFRAME, DATASET, STRUCTURED DATA

Structured data has a fixed shape: rows made of named, typed columns (a schema), like a database table or spreadsheet. A **DataFrame** is a distributed table of rows with named columns. A **Dataset** is a related typed interface for the same kind of data. Both are built on the Spark SQL engine (Slide 18) and can be converted back to an RDD. (The slide does not define "structured data"; this is the standard meaning.)

BEGINNER INTUITION

An RDD is a pile of sealed boxes: Spark can move them around but does not know what is inside, so you must say in code what to do with every box. A DataFrame is a spreadsheet: Spark can see the column names and types, so you can say "select the name column where age is over 30" (or write it in SQL) and Spark works out an efficient way to do it.

	RDD (Slide 11)	DataFrame / Dataset (Slide 12)
Level	Low-level, the core abstraction	Higher-level, built on the core
Data it suits	Any records, Spark treats them as opaque	Structured data (rows with a schema)
Spark's knowledge	Knows nothing about the contents	Knows columns and types, so it can optimise
SQL	No	Yes (the SparkSQL box of Slide 8)
Introduced	From the start	"More recent versions"

Beyond the slides: DataFrames appeared in Spark 1.3 and Datasets in Spark 1.6, and the two were unified in Spark 2.0. The slide's "more recent versions" is vague; do not memorise version numbers for the exam.

EXAM TIP

Remember the one-line contrast: **RDD is the low-level core; DataFrames and Datasets are higher-level, for structured data, and are what makes SQL support possible.**

Part 1 in one table: problem and fix

Lecture 3 · Slides 3–12

MapReduce (Slides 3–7)	Spark (Slides 8–12)
Intermediate results written to disk, read back by the next step	Fast data sharing: results kept in memory; data cached for reuse
Exactly two ranks: Map then Reduce	General execution graphs (any DAG)
Only two functions; hand-chained jobs; "does not compose well"	Richer functions; one program describes the whole graph
Separate systems for streaming, graphs, ML, SQL	One engine plus Spark Streaming, GraphX, MLlib, SparkSQL (Slide 8)
Hadoop file system as the data home	Connectors to many systems (Slide 9); compatible with Hadoop (Slide 10)
(no equivalent single type)	RDD: partitioned, in-memory, transformations and actions (Slide 11); DataFrames/Datasets for structured data and SQL (Slide 12)

QUICK RECAP

- Big data forced **scale out**; MapReduce hid storage, coordination and data-locality problems and simplified programming.
- Two limitations: **performance** (disk I/O between steps) and **programmability** (does not compose; chains of jobs).
- **Data-flow systems** generalise MapReduce: any number of ranks, any functions, one-directional DAG.
- **Spark** adds fast in-memory data sharing and caching, general DAGs, richer functions, and stays compatible with Hadoop.
- Ecosystem: core engine plus Spark Streaming, GraphX, MLlib, SparkSQL; connectors to many data stores.
- Key type: **RDD** (Resilient Distributed Dataset), with **transformations** (map, filter) and **actions** (count, collect); higher-level **DataFrames/Datasets** for structured data and SQL.

The RDD: Spark's basic data container

Lecture 3 · Slides 13–14

Slides 1–12 explained why Spark exists: Hadoop MapReduce writes to disk after every step and composes badly, so Spark adds fast data sharing in memory, general DAGs (directed acyclic graphs: networks of steps with arrows and no cycles), richer functions, and Hadoop compatibility. Slide 13 now names the single idea that makes this possible.

BEGINNER INTUITION

Think of a huge deck of exam papers to mark. You cut the deck into piles and hand one pile to each of many markers. You never scribble on the original papers; each marker produces a *new* pile of marked sheets. That is an RDD: a big collection, cut into pieces, spread over many workers, never edited in place, only turned into new collections.

DEFINITION: RDD (RESILIENT DISTRIBUTED DATASET), SLIDE 13

A **partitioned collection of records (objects)**, **spread across the cluster**, **read-only**, which Spark can **cache in memory** (with fallback to disk). **Distributed** = its pieces live on many machines. **Dataset** = it is a collection of records.

Reading the slide's bullets one by one

Lecture 3 · Slide 13

Slide wording	Plain meaning	Why it matters
Partitioned collection of records/objects	The dataset is chopped into **partitions**; a **record** is one item (a line, a number, a pair, any object).	Each partition can be processed on a different machine at the same time: partitions = parallelism.
Generalizes (key-value) pairs	MapReduce only allowed (key, value) pairs. An RDD's records may be any object; (key, value) pairs are just one special case.	More natural programs; no need to force every item into a pair.
Spread across the cluster, read-only	Partitions sit on many machines. **Read-only (immutable)**: you never change an RDD; you create a new one.	No update conflicts between machines, and a lost piece can be rebuilt (see "Resilient" below).
Caching dataset in memory; fallback to disk possible	Spark keeps partitions in RAM when it can, and uses disk only if RAM is too small.	Re-using an RDD is then fast; this is the answer to MapReduce's disk-I/O problem.
Created from Hadoop, or by transforming other RDDs (you can stack RDDs)	Two ways to get an RDD: load external data, or apply a transformation to an existing RDD. Steps can be chained ("stacked").	Programs become pipelines of RDDs, each built from the previous.
Best suited for applying the same operation to all elements	Bulk operations (map, filter-style) that treat every record alike.	Not designed for fine-grained updates to one record at a time.

COMMON MISTAKE

"Created from Hadoop" does not mean the data must come through MapReduce. It means data held in Hadoop-supported storage, typically files in HDFS (the Hadoop Distributed File System). Also, "read-only" does not mean you cannot compute with an RDD; it means each computation yields a *new* RDD.

Beyond the slides: why "Resilient"? The slide never explains this word. Because an RDD is read-only and every transformation is deterministic (same input, same output), Spark only needs to remember the *recipe* (the chain of transformations, called the **lineage**) that produced each RDD. If a machine fails and a partition is lost, Spark re-runs the recipe for that partition alone. Resilient = able to recover from failure this way.

Figure 3.7: data in, RDDs stacked, one action out

Lecture 3 · Slide 13

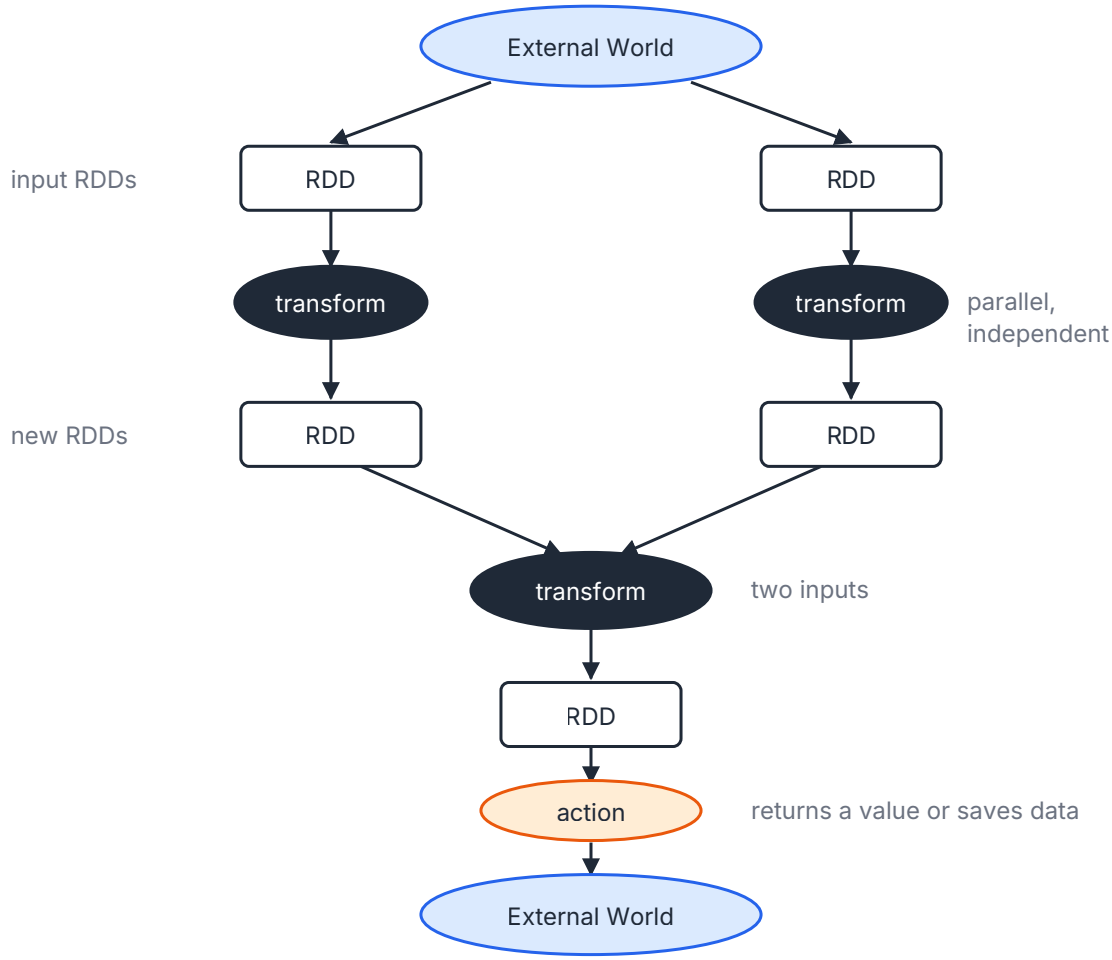


Figure 3.7 (redrawn from Slide 13). Data enters from the outside world as RDDs; transformations (black) turn RDDs into new RDDs and may take two inputs; only the action (orange) sends a result back out.

Read it top to bottom. The two clouds are the "external world": any source or sink such as HDFS or a local file. The figure has 5 RDD boxes, 3 transformations (two running in parallel on independent RDDs, one merging two RDDs into one, for example a join or union) and exactly 1 action. Notice that nothing leaves the Spark world until the action at the bottom.

Figure 3.8: one RDD, many partitions, memory backed by disk

Lecture 3 · Slide 14

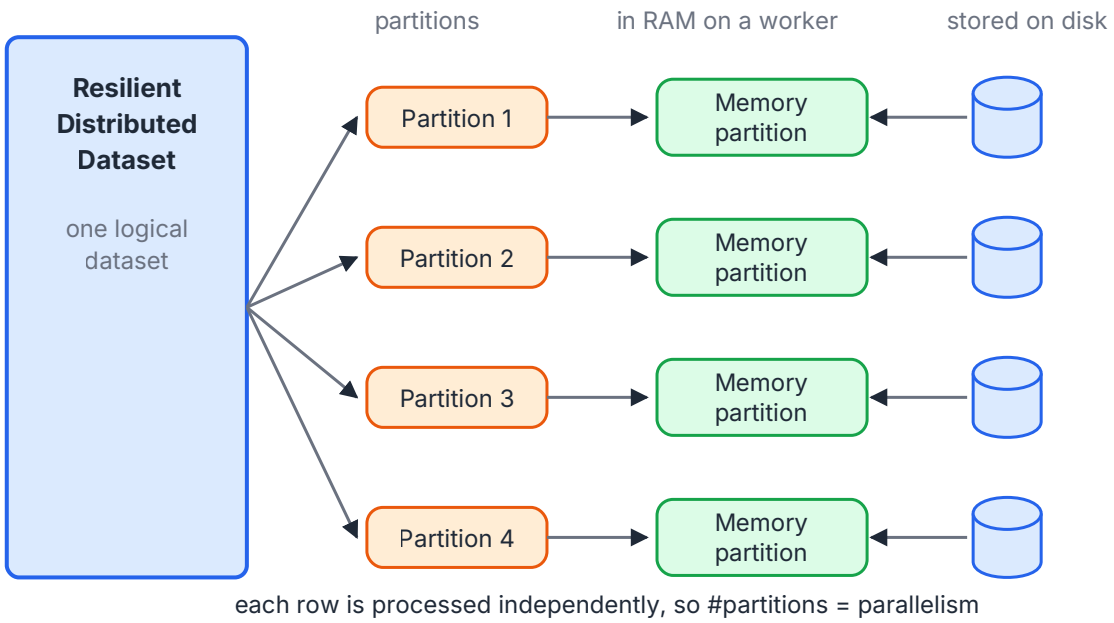


Figure 3.8 (redrawn from Slide 14, four rows as on the slide). One logical RDD is split into partitions; each partition is held as an in-memory partition on a worker, loaded from the stored data on disk.

Slide 14 shows the same RDD at three levels: **logical** (one blue box), **split into partitions** (orange), and **physical** (a green in-memory partition on the worker that processes it, filled from stored data, the blue cylinders). Both arrows point into the memory partition: one from the partition's definition and one from the data on disk. The slide does not label the arrows; this is the natural reading and it matches "cache in memory, fall back to disk". Each row is independent, so the number of partitions is the amount of parallel work.

Worked example: partitions and immutability Lecture 3 · Slides 13–14

Take the records 1, 2, 3, 4, 5, 6 as one RDD `R` with 3 partitions: `[1,2]`, `[3,4]`, `[5,6]`, on workers W1, W2, W3. Apply the transformation "multiply each record by 2".

1. Each worker doubles its own partition independently: W1 gives `[2,4]`, W2 gives `[6,8]`, W3 gives `[10,12]`. No data moves between machines.
2. The result is a *new* RDD `R2` with 3 partitions; `R` is unchanged (still 1,2,3,4,5,6).
3. If W2 crashes, Spark re-runs "double" on partition `[3,4]` only and recovers `[6,8]`.

EXAM TIP

If asked "What is an RDD?", give five facts: partitioned collection of records; spread across the cluster; read-only; cached in memory with disk fallback; created from stored data or by transforming other RDDs. Add "best for applying the same operation to all elements" and, for extra credit, "recoverable through lineage".

RDD operations: transformations, actions, lazy evaluation

Lecture 3 · Slides 15–16

Spark offers exactly two kinds of operations on RDDs. Slide 15 defines them.

	Transformation	Action
Purpose	Build a **new RDD** from other RDD(s) through deterministic operations	**Return a value** to the program or **export data**

Examples on slide	*map, filter, join, union, intersection, distinct*	*count, collect, reduce, save*
Evaluation	**Lazy: only records the step; computes nothing yet**	**Forces the computation of every pending transformation it depends on**
Output type	an RDD	a number, a list, a file (not an RDD)

The words, defined

Lecture 3 · Slide 15

- **Deterministic:** the same input always gives the same output, so a lost piece can be recomputed exactly.
- **map:** apply a function to every record, one output per input. **filter:** keep only records that satisfy a condition. **union:** all records of two RDDs. **intersection:** only records in both. **distinct:** remove duplicates. **join:** combine records of two RDDs that share a key.
- **count:** number of records. **reduce:** combine all records into one value with a function (for example sum). **collect:** bring all records to the driver program as a list. **save:** write the records to storage.

KEY IDEA: LAZY EVALUATION (SLIDE 15)

Nothing is computed until an action requires it. A program made only of transformations does nothing; as soon as an action is called, the whole chain behind it runs. *Why?* Spark sees the entire chain first, so it can pipeline steps, avoid storing intermediate RDDs, and compute only what the action needs.

Worked example: lazy chain with numbers Lecture 3 · Slide 15

Start from the RDD 1, 2, 3, 4, 5, 6.

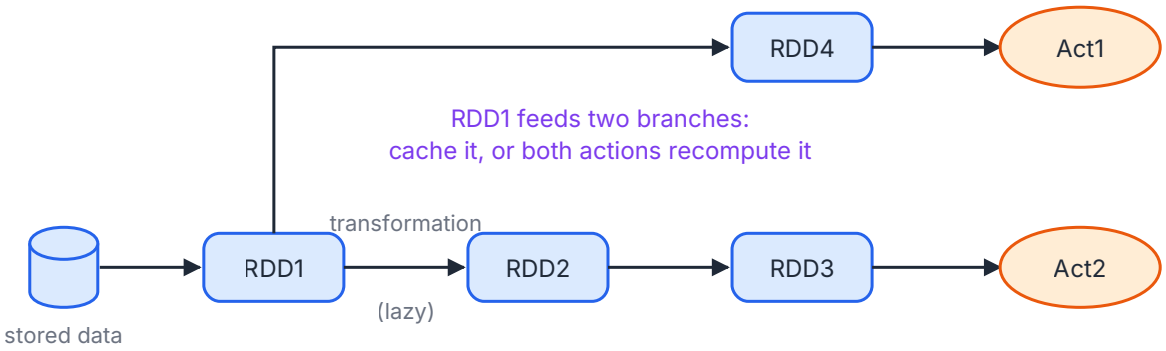
1. `map`: double each record. Recorded only. (Nothing computed.)
2. `filter`: keep values greater than 6. Recorded only.
3. `count`: an action, so Spark now runs both steps. After map: 2, 4, 6, 8, 10, 12. After filter: 8, 10, 12.
4. Answer: **3**. Without step 3 the program would return nothing and read no data.

COMMON MISTAKE

Thinking *filter* or *map* "runs" when written. They only extend the recipe. Also, *collect* is an action that pulls every record onto one machine (Beyond the slides: so use it only on small results). Slide 15's phrase "actions force calculations and return values" is a little circular; remember it as "an action triggers the computation and delivers a result".

Lineage graph (Slide 16)

Lecture 3 · Slide 16



Act1 runs: data → RDD1 → RDD4. Act2 runs: data → RDD1 → RDD2 → RDD3.
Until an action is called, the arrows are only a recipe (nothing is computed).

Figure 3.9 (redrawn from Slide 16). Boxes are RDDs, arrows between RDDs are transformations, ellipses are actions. RDD1 is shared by two branches.

Slide 16 draws a **lineage graph** (a DAG): stored data gives RDD1; RDD2 and RDD3 follow by successive transformations; RDD4 is a different transformation of RDD1. The slide has no text besides the labels; the take-aways below are the intended reading.

1. While the arrows are being *defined*, nothing is computed (lazy).
2. Calling **Act1** runs only data → RDD1 → RDD4. Calling **Act2** runs only data → RDD1 → RDD2 → RDD3.
3. RDD1 lies on both paths. Unless it is **cached** (kept in memory, Slides 13–14), each action recomputes it. Count: with no cache, calling both actions computes RDD1 twice; with a cache, once.

EXAM TIP

Typical question: "Explain lazy evaluation and why it helps." Answer: transformations only record lineage; an action triggers execution; Spark can optimise the whole chain (pipelining, no intermediate storage, compute only what is needed), and lineage also enables recovery.

The task scheduler: general DAGs, stages and shuffles

Lecture 3 · Slide 17

When an action is called, the **task scheduler** turns the lineage into an execution plan. Slide 17 lists four properties: it **supports general directed acyclic task graphs** (not just map then reduce); it **pipelines functions where possible**; it is **cache-aware** (reuses data, respects locality); and it is **partitioning-aware to avoid shuffles**.

TERMS THE SLIDE DOES NOT DEFINE (BEYOND THE SLIDES: STANDARD SPARK VOCABULARY)

- **Narrow dependency:** each output partition is computed from a small, fixed set of input partitions (typically one), as in map and filter. No data moves across the cluster; can be pipelined.
- **Wide dependency / shuffle:** each output partition needs data from many input partitions (groupBy, join). Data is exchanged across the network and written to disk; this is expensive.
- **Stage:** a group of pipelined narrow steps. A new stage starts at every shuffle.
- **Locality:** running a task on the machine that already holds its data (in cache or on disk) so nothing is copied.

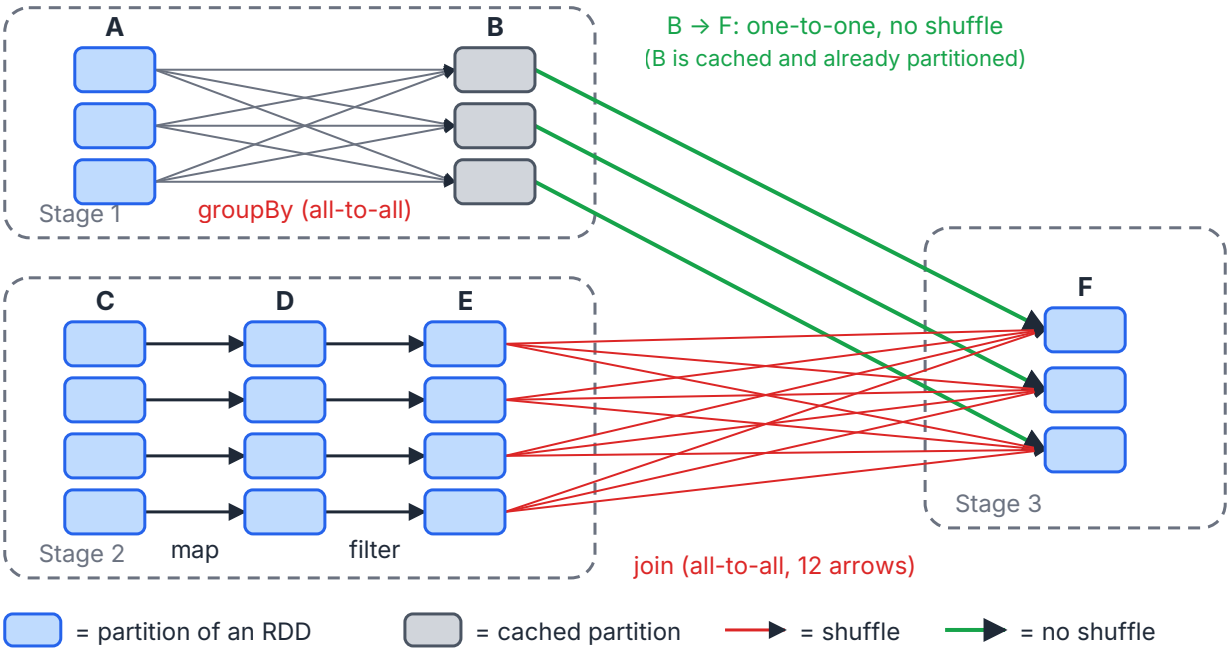


Figure 3.10 (redrawn from Slide 17). Six RDDs A–F. Red arrows: all-to-all shuffles. Green arrows: one-to-one, no shuffle. Grey partitions: cached.

Reading the figure

Lecture 3 · Slide 17

Step	Arrows	Type	Consequence
A → B, *groupBy*	3 × 3 = 9	all-to-all (shuffle)	Ends Stage 1
C → D, *map*	4, one-to-one	**narrow**	Pipelined: each partition flows through map then filter on one machine (Stage 2)
D → E, *filter*	4, one-to-one	**narrow**	
B → F	3, one-to-one	**narrow**	B is already partitioned as the join needs, so it is not shuffled again
E → F, *join*	4 × 3 = 12	all-to-all (shuffle)	Only E moves; Stage 3 produces F

Arithmetic check: 9 + 4 + 4 + 3 + 12 = 32 arrows in the picture. If B were not already partitioned for the join, B → F would also need 3 × 3 = 9 arrows, giving 38; this is what "partitioning-aware to avoid shuffles" saves. Likewise B is drawn grey (cached), so Stage 1 (A → B) does not run again when F is recomputed or reused.

The four bullets, matched to the figure: (1) a general DAG with six RDDs and two branches; (2) map and filter share Stage 2 (pipelining); (3) grey B partitions are reused (cache-aware); (4) one-to-one B → F avoids a second shuffle.

SLIDE ERRATUM

(a) The legend explains only "RDD" and "cached partition"; the **black** and **orange** partitions of RDD D are unexplained on the slide. (Beyond the slides: in the Spark research paper this figure comes from, black marks partitions already in memory, which is why those need no recomputation.) (b) The large dashed "Stage 3" box visually encloses Stages 1 and 2 as well; logically Stage 3 is only the join that produces F from B and E. Figure 3.10 draws the logical version. On the slide B sits outside the small Stage 1 box; in Figure 3.10 we draw B inside Stage 1 because it is the output of the Stage 1 groupBy. (c) The slide says "task graphs" while it draws RDD partitions: same idea at a different level of detail.

EXAM TIP
Know the four bullets by heart and be able to say what a shuffle is and why stage boundaries sit at shuffles. A map/filter chain is cheap (pipelined); groupBy/join are costly (shuffle).

Higher-level APIs: DataFrame and Dataset

Lecture 3 · Slides 18–19

An RDD is a bag of objects whose contents Spark cannot see. Slide 12 said newer Spark versions added DataFrames and Datasets, which enabled SQL support. Slide 18 defines them.

DATAFRAME (SLIDE 18)
Unlike an RDD, data is **organised into named columns**, like a **table in a relational database**. It imposes a **structure** on a distributed collection, allowing a higher-level abstraction.

DATASET (SLIDE 18)
An extension of the DataFrame API providing a **type-safe, object-oriented programming interface**, with **compile-time error detection** (mistakes such as using a field that does not exist are caught before the job runs).

Both are built on the **Spark SQL engine** and **both can be converted back to an RDD**.

BEGINNER INTUITION
An RDD is a box of unlabelled items. A DataFrame is a spreadsheet: columns have names, so you can say "the age column" and Spark understands and can optimise. A Dataset is a spreadsheet where each row is also a checked object, so the compiler warns you about a wrong column or type before you waste a long run.

	RDD	DataFrame	Dataset
Structure	none; any objects	**named columns (table)**	**named columns plus object types**
You describe	arbitrary functions (black boxes)	what you want, by column name	same, with typed objects
Error detection	at run time	mostly at run time	**compile time (type-safe)**
Engine	Spark core	Spark SQL engine; convertible back to an RDD	

Beyond the slides: typed Datasets exist in the compiled JVM languages (Scala, Java); PySpark (Python) uses DataFrames. A DataFrame is a Dataset of generic rows, and the SQL engine's optimiser plans the query for you. "Compile-time" only makes sense in compiled languages, which the slide does not say.

The one-line comparison (Slide 19)

Lecture 3 · Slide 19

NOT EXAMINED — CODE (UNDERSTAND, DON'T MEMORISE)
RDD style: `rdd.map(lambda x: ...).filter(lambda x: ...)`
DataFrame / Dataset style: `df.select("age").filter(df.age > 30)`

In words: the RDD line hands Spark arbitrary functions, applied record by record, which Spark cannot look inside. The DataFrame line states *what* is wanted: pick the column "age", keep rows where age is greater than 30. Because that is declarative, Spark can optimise it. Both lines are chains of transformations, so both are lazy. As written, neither ends with an action and the "..." is a placeholder, so neither would actually run.

EXAM TIP

Expected answer on "why DataFrame over RDD": structure (named columns), higher-level abstraction, SQL support and optimisation by the engine. Dataset adds type safety and compile-time errors. Both convert to an RDD.

Spark's libraries

Lecture 3 · Slide 20

Spark is a platform: the core (RDDs plus the DAG scheduler) and four libraries for common jobs.

Library	Slide's one-liner	What it means
Spark SQL	scalable processing of relational data	Query structured data with SQL or DataFrames across a cluster.
Spark Streaming	stream processing of live data streams	Handle data that arrives continuously, not stored files.
MLlib	scalable machine learning	ML algorithms on a cluster; they loop over the same data many times, so in-memory caching helps.
GraphX	graph manipulation	Graph computation (Beyond the slides: for example PageRank-style algorithms).

GRAPHX GRAPH (SLIDE 20)

GraphX **extends the Spark RDD with a Graph abstraction: a directed multigraph with properties attached to each vertex and edge**. **Directed:** every edge has a direction. **Multigraph:** several parallel edges may join the same pair of vertices. **Properties:** user-defined attributes on every vertex and every edge. Example: vertices = people (property: name), edges = "follows" or "emailed" (property: date); two edges A → B are allowed. Because it builds on RDDs, a graph is also distributed and immutable.

SLIDE ERRATUM

Typo: "datastreams" should read "data streams".

Spark vs Hadoop MapReduce

Lecture 3 · Slide 21

Slide 21 compares the two along three dimensions.

Dimension	Hadoop MapReduce	Spark
Performance	Persists back to disk after every map or reduce action; runs easily alongside other services with minor performance differences; works well for the **1-pass jobs** it was designed for.	**Processes data in memory, so normally faster. Caveat: often needs lots of memory; if other resource-demanding services compete or the data cannot fit in memory, Spark degrades.**
Ease of use	Lower-level programming model (Map and Reduce functions)	**Easier to program: higher-level APIs**
Data processing	Map then reduce	**More general: general DAGs, many operators, SQL, streaming, ML, graphs**

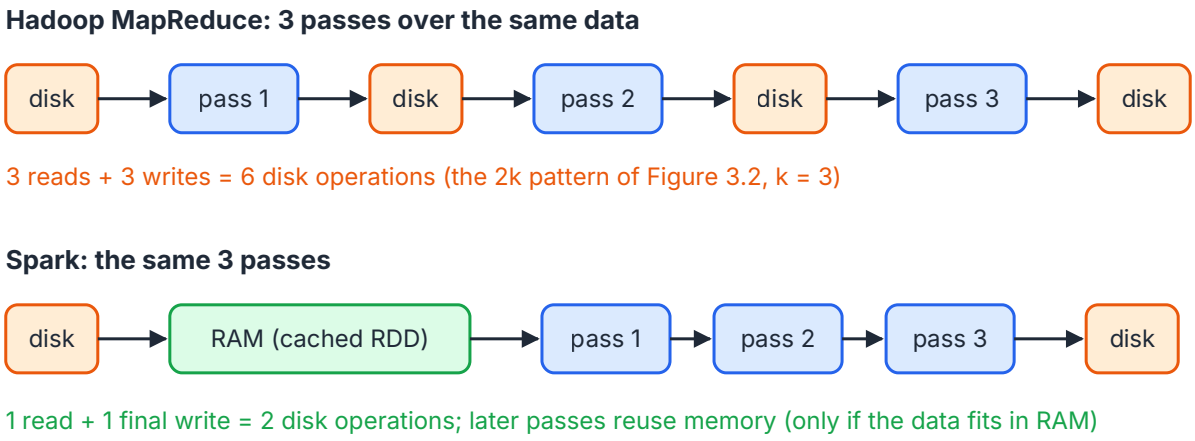


Figure 3.11 (illustration of the performance bullet). Three passes: MapReduce touches disk between every pass (6 operations), Spark reads once, keeps the RDD in RAM and writes the final result once (2 operations).

Counting the figure: 3 passes under MapReduce means 3 reads and 3 writes (6 disk operations) against 1 read and 1 final write (2 disk operations) for Spark, the 2k versus 2 pattern of Figure 3.2. This is an illustration of the principle, not a measured speed-up; the slide gives no "10x" or "100x" numbers and says nothing about fault tolerance or cost.

COMMON MISTAKE

Writing "Spark is always faster". The slide says **normally** faster, **with caveats**: it needs lots of memory, and for a simple one-pass job on a shared cluster MapReduce is perfectly adequate.

EXAM TIP

"Compare Spark and MapReduce": answer on performance (with the memory caveat), ease of use, and generality, and name MapReduce's strengths (one-pass jobs, coexistence with other services).

Which problems suit MapReduce?

Lecture 3 · Slides 22–23

Slide 22 is a section divider titled "Problems Suited for Map Reduce"; it carries no printed number (it sits between printed slides 21 and 23). Slide 23, titled "Problems NOT suitable for MapReduce", answers both sides.

MapReduce is great for	MapReduce is inefficient when random or irregular access is required
Sequential data access: scan the files start to end, each record handled independently (word count, filtering, simple aggregation). Splits easily across machines.	• **Interdependent data**, for example **comparisons of many pairs of items** (every item must meet every other) and **machine learning** (many repeated passes over the same data). • **Graphs**: a vertex needs its scattered neighbours, and iterative algorithms need many passes.

Beyond the slides: each extra MapReduce round costs a shuffle and a disk write, so iterative or interdependent jobs pay that cost again and again; Spark's caching and general DAGs soften it. Number check: comparing all pairs among n items needs $n(n-1)/2$ comparisons: each of the n items is paired with the other $n-1$, and each pair is counted twice, so divide by 2 (1,000 × 999 / 2 = 499,500).

SLIDE ERRATUM

The divider says "Suited" while the next slide is titled "NOT suitable"; the divider also writes "Map Reduce" with a space and has no number. Slide 23 has a grammar slip: "access to data required" should be "is required".

Decision checklist: MapReduce or Spark?

Lecture 3 · Slides 21, 23

This checklist combines Slides 21 and 23. It is a study aid, not a slide.

Question	If yes, lean to
One pass over the data, sequential scan, read-map-reduce-write once?	MapReduce is fine
Must it coexist with other resource-hungry services, with little memory to spare?	MapReduce
Does the job loop over the same data many times (machine learning)?	Spark (cache in memory)
Graphs, or records that depend on each other, or comparisons of many pairs?	Spark (DAGs, GraphX), though still costly
Need SQL, streaming, ML or graph libraries in one system?	Spark
Want shorter programs with higher-level APIs?	Spark
Is there enough memory for the working set?	If no, Spark degrades; reconsider

Tutorial and further reading

Lecture 3 · Slide 24

Slide 24 only points to hands-on material, adds no new concepts, and no code from Lectures 1–5 is examined, so use it for practice only. It lists a Spark programming tutorial (a Google Colab notebook, which may become unavailable) and the official guides: the Spark RDD Programming Guide, the DataFrame and SQL guide, the MLlib Guide, the GraphX Guide, and Spark Streaming. They map onto Slides 13–20 in that order.

SLIDE ERRATUM

The "DataFrame and SQL Guide" link points to the PySpark DataFrame API reference page, not to the Spark SQL programming guide the link text suggests.

QUICK RECAP OF SLIDES 13–24

- RDD = partitioned, read-only collection spread over the cluster, cached in memory with disk fallback; made from stored data or from other RDDs; best for the same operation on every element.
- Transformations (map, filter, join, union, intersection, distinct) are deterministic and lazy; actions (count, collect, reduce, save) trigger execution and return or export.
- Scheduler: general DAGs, pipelining of narrow steps, cache-aware reuse and locality, partitioning-aware shuffle avoidance; shuffles (groupBy, join) end stages.
- DataFrame = named columns; Dataset = type-safe with compile-time checks; both on Spark SQL and convertible to RDDs.
- Libraries: Spark SQL, Streaming, MLlib, GraphX (directed multigraph with vertex and edge properties).
- Spark vs MapReduce: faster in memory but memory-hungry; easier; more general. MapReduce suits sequential, one-pass work; it struggles with random access, interdependent data, ML and graphs.

Theory Q&A bank

Lecture 3 · Slides 1–24

Cover the answer, write your own, then compare. Numbered points are marking points. These questions are self-made practice except where a slide poses the question itself (for example "Scale up or scale out?").

Q1. Scale up or scale out? (the slide's own question)

Lecture 3 · Slide 3

1. **Scale out.** Scale up means one bigger machine; scale out means many commodity machines in a cluster.
2. Scaling up is costly and has a hard ceiling; a search engine crawling billions of pages cannot fit on one machine.
3. Scaling out creates storage, coordination and programming problems, which the MapReduce model hides.

Q2. What did the MapReduce programming model give us?

Lecture 3 · Slide 3

1. It handles **persistent storage** (distributed file system, replicated chunks).
2. It handles **distributed worker coordination** (the master assigns tasks and restarts failed ones).
3. It brings **computation closer to the data** (data locality).
4. It is **simpler for the programmer**: only map and reduce functions are written.

Q3. Describe the execution of a MapReduce job (six steps).

Lecture 3 · Slide 4

1. **Fork:** the user program starts a master and workers.
2. **Assign:** the master gives map tasks and reduce tasks to idle workers.
3. **Read:** a map worker reads its input split.
4. **Local write:** it writes intermediate pairs to its own local disk.
5. **Remote read:** each reduce worker pulls its share from every map worker (the shuffle).
6. **Write:** the reduce worker writes an output file.

Q4. What are the two major limitations of MapReduce? Explain "does not compose well".

Lecture 3 · Slides 5–6

1. **Performance bottlenecks:** mapper output is saved to disk and read again by reducers, and each job writes to HDFS; disk is much slower than memory.
2. **Programming difficulty:** many problems are not easily described as one map plus one reduce.
3. Many applications need **several chained steps**. Each step is a separate job linked only through files on disk, so each link pays the disk cost and the programmer must manage the chain by hand: MapReduce "does not compose well for all large applications".

Q5. What is a data-flow system, and how does it generalise MapReduce?

Lecture 3 · Slide 7

1. MapReduce has exactly two ranks of tasks (Map, then Reduce).
2. A data-flow system allows **any number of ranks** and **functions other than Map and Reduce**.
3. Data flows in **one direction only**, so the task network is a **DAG** (directed acyclic graph: directed edges, no cycles).

Q6. What does Spark add to the MapReduce model?

Lecture 3 · Slide 10

1. **Fast data sharing:** avoids saving intermediate results to disk, and caches data for repeated queries or machine learning (fixes the performance limitation).

2. **General execution graphs (DAGs)** (fixes composability).
3. **Richer functions** than map and reduce (fixes programming difficulty).
4. It remains **compatible with Hadoop**, so existing data and clusters can be reused.

Q7. Draw and describe the Spark software stack. Why is it useful?

Lecture 3 · Slides 8–9

1. Bottom: the **Spark core**, a fast memory-optimised execution engine with Python, Java and Scala APIs.
2. On top: **Spark Streaming** (stream processing), **GraphX** (graph computation), **MLlib** (machine learning), **SparkSQL** (SQL API).
3. One engine serves all workloads, and connectors (data lakes, slide 9: HDFS, HBase, Cassandra, MongoDB, MySQL, PostgreSQL, Kafka and others) let it read many stores.

Q8. What is an RDD? Why is it called resilient and distributed?

Lecture 3 · Slides 11, 13–14

1. A **Resilient Distributed Dataset**: a partitioned collection of records, spread across the cluster, **read-only**.
2. It can be **cached in memory** with fallback to disk, and is created from Hadoop-supported data or by transforming other RDDs.
3. **Distributed:** its partitions live on many machines and are processed in parallel.
4. **Resilient** (beyond the slides): Spark records the lineage of transformations, so a lost partition is recomputed from its recipe.
5. It is best for applying the same operation to all elements.

Q9. Compare transformations and actions. What is lazy evaluation and why is it useful?

Lecture 3 · Slides 11, 15

1. **Transformation** (map, filter, join, union, intersection, distinct): builds a new RDD through a deterministic operation. **Action** (count, collect, reduce, save): returns a value or exports data.
2. **Lazy evaluation:** transformations only record the step; nothing is computed until an action requires it.
3. Useful because Spark sees the whole chain first: it can pipeline steps, avoid storing intermediate RDDs and compute only what the action needs.

Q10. Explain the lineage graph of Slide 16: what runs when Act1 and Act2 are called?

Lecture 3 · Slide 16

1. Defining the transformations computes nothing.
2. Act1 runs stored data → RDD1 → RDD4. Act2 runs stored data → RDD1 → RDD2 → RDD3.
3. RDD1 is on both paths: without caching it is computed twice; if cached, once.
4. The lineage also lets Spark rebuild a lost partition.

Q11. What does Spark's task scheduler do? (four properties)

Lecture 3 · Slide 17

1. Supports **general directed acyclic task graphs**.
2. **Pipelines functions** where possible (map then filter in one pass, in one stage).
3. **Cache-aware**: reuses cached data and respects locality.
4. **Partitioning-aware to avoid shuffles**: if data is already partitioned as needed, it is not moved again. A shuffle (all-to-all, for example groupBy or join) ends a stage.

Q12. Compare RDD, DataFrame and Dataset.

Lecture 3 · Slides 12, 18–19

1. **RDD:** low-level, unstructured collection of objects; you give functions Spark cannot look inside.

2. **DataFrame:** data organised into named columns, like a relational table; imposes structure, higher-level abstraction, enables SQL support.
3. **Dataset:** extension of the DataFrame API with a type-safe, object-oriented interface and compile-time error detection.
4. Both are built on the Spark SQL engine and both convert back to an RDD.

Q13. Name the Spark libraries and describe GraphX's graph.

Lecture 3 · Slide 20

1. **Spark SQL** (scalable relational data), **Spark Streaming** (live data streams), **MLlib** (scalable machine learning), **GraphX** (graph manipulation).
2. GraphX extends the RDD with a Graph abstraction: a **directed multigraph with properties attached to each vertex and edge**.
3. Directed = edges have direction; multigraph = parallel edges between the same two vertices allowed; properties = user-defined attributes.

Q14. Compare Spark and Hadoop MapReduce.

Lecture 3 · Slide 21

1. **Performance:** MapReduce persists to disk after every map or reduce action and works well for 1-pass jobs and alongside other services; Spark works in memory so is normally faster, but needs lots of memory and degrades if memory is short or other services compete.
2. **Ease of use:** MapReduce has a lower-level model; Spark has higher-level APIs.
3. **Data processing:** MapReduce is map then reduce; Spark is more general (DAGs, SQL, streaming, ML, graphs).

Q15. Which problems are suited, and which not suited, to MapReduce?

Lecture 3 · Slides 22–23

1. **Suited:** sequential data access, scanning files with each record handled independently (word count, filtering, simple aggregation).
2. **Inefficient** where random or irregular access is needed: **interdependent data** (comparisons of many pairs of items, machine learning) and **graphs**.
3. Reason: such jobs need many rounds, and each MapReduce round pays a shuffle and disk write (beyond the slides).

One-page cheat sheet

Lecture 3 · Slides 1–24

Definitions

Term	One-line meaning
Scale up / scale out	One bigger machine / many commodity machines (choose scale out)
Data-flow system	Network of tasks, any number of ranks, any functions, one direction (DAG)
DAG	Directed acyclic graph: arrows, no loops
RDD	Partitioned, read-only collection of records across the cluster; cached in memory, disk fallback
Transformation	RDD to new RDD; deterministic; lazy
Action	Returns a value or exports data; forces computation
Lineage	Recorded chain of transformations; enables recovery (beyond slides)
Shuffle / stage	All-to-all data exchange / pipelined group of steps ended by a shuffle (beyond slides)
DataFrame / Dataset	Named columns (table) / typed, compile-time-checked extension; both on Spark SQL engine
GraphX graph	Directed multigraph with vertex and edge properties

Comparison tables

	Hadoop MapReduce	Spark
Intermediate data	Disk after every step	Memory (cache), disk fallback
Model	Map then reduce, low level	General DAGs, many operators, higher-level APIs
Best for	1-pass, sequential jobs; sharing a cluster	Iterative, interactive, multi-stage work (beyond the slides)
Weak spot	Chains of jobs, ML, graphs	Needs lots of memory; degrades without it

	Transformation	Action
Examples	map, filter, join, union, intersection, distinct	count, collect, reduce, save
Returns	New RDD	Value or exported data
Runs when?	Lazy: recorded only	Immediately; triggers the whole chain

	RDD	DataFrame	Dataset
Structure	None	Named columns	Columns plus object types
SQL support	No	Yes	Yes
Errors caught	Run time	Mostly run time	Compile time

Procedures in a few steps

- **MapReduce execution:** fork; assign map/reduce; read split; local write; remote read (shuffle); write output.
- **Spark program:** create RDD from stored data; chain lazy transformations; call an action; scheduler builds DAG, cuts stages at shuffles, pipelines, reuses cache.
- **Problem to fix (Slides 5–10):** disk I/O to fast data sharing; two ranks to general DAGs; few functions to richer functions.
- **Stack (Slide 8):** core + Streaming, GraphX, MLlib, SparkSQL.
- **Scheduler (Slide 17):** general DAGs; pipelines; cache-aware and locality; partitioning-aware to avoid shuffles.

Numbers to recompute

- k chained MapReduce jobs: 2k disk touches; minimum needed 2 (one read, one write).
- Figure of Slide 17: 3 × 3 = 9 (groupBy), 4 + 4 + 3 one-to-one, 4 × 3 = 12 (join); total 32 arrows.
- All pairs among n items: n(n−1)/2 (1,000 items: 499,500).

Common mistakes & traps

Lecture 3 · Slides 1–24

- Writing "Spark is always faster". The slide says **normally**, with a memory caveat; one-pass jobs suit MapReduce fine.
- Saying Spark never uses disk. Input and final output are on disk, and RDDs fall back to disk; the gain is no disk *between* steps.
- Saying map output goes to HDFS. It goes to the mapper's **local disk**; only final reducer output goes to the distributed file system.
- Mixing up control flow (fork, assign) with data flow (read, local write, remote read, write) on Slide 4.
- Calling `collect` or `count` a transformation. They are **actions**; map and filter are transformations.
- Believing a transformation runs when written. It only records the step (lazy); an action triggers it.
- Thinking "read-only" means an RDD cannot be used. Each operation yields a **new** RDD; the old one is unchanged.
- Assuming a DAG may contain loops. It is acyclic; iterations are written as a longer acyclic graph.
- Treating DataFrames and Datasets as the same: only the Dataset is type-safe with compile-time error detection.

- Describing a GraphX graph as undirected or simple: it is a **directed multigraph with properties**.

LECTURE 4

Frequent Itemsets & Association Rules (A-Priori, PCY, SON)

ROADMAP — WHAT THIS LECTURE IS ABOUT

- **The problem (Slides 1–18):** given a huge file of baskets (small sets of items), find itemsets that occur together in at least *s* baskets (**frequent itemsets**) and turn them into if-then **association rules**, judged by **support**, **confidence** and **interest**.
- **The hard part (Slides 19–28):** finding frequent itemsets. The cost is the number of **passes** over a disk-resident file; the scarce resource is **main memory**; the bottleneck is counting **pairs**. The naive method needs about 4 bytes × n(n−1)/2 of memory.
- **Exact multi-pass algorithms (Slides 29–51): A-Priori** (monotonicity, C_k/L_k), **PCY** (hash pairs into buckets in the idle Pass 1 memory, bitmap, 2/3 rule), and its refinements **Multistage** (3 passes) and **Multihash** (2 passes).
- **At most two passes (Slides 52–69): random sampling** (approximate: false positives and false negatives) and **SON** (exact), with SON as two MapReduce rounds and a five-basket worked example.
- **Practice (Slides 70–84):** the cat-and-dog baskets (pairs, triples, confidence, interest), A-Priori on integer baskets, the PCY bitmap, and the multistage idea. Slides 85–86 are the summary and the reading list.

Exam checklist — by the end you can:

- compute **support**, list all frequent itemsets level by level for a given s, and state **monotonicity**;
- compute **confidence** and **interest** of a rule, say when a rule is interesting, and generate all rules above a confidence threshold from the frequent itemsets (including the boundary case conf = c);
- explain why **passes** and **main memory** are the cost model and why **pairs** are the bottleneck;
- compute the memory of the naive method, the triangular matrix (4 bytes per pair) and the triples method (12 bytes per occurring pair), and state the **1/3 break-even**;
- run **A-Priori** by hand (Pass 1, frequent items, Pass 2, C_k, L_k) and explain the renumbering trick;
- describe **PCY** Pass 1, the bit-vector, the Pass 2 candidate test (both conditions), why no frequent pair is lost, why a bitmap, and the **2/3 rule**;
- describe and compare **Multistage** and **Multihash** (passes, filters, risks, limits);
- explain **random sampling** (scaled threshold, false positives vs false negatives) and **SON** (chunks, candidates, second pass, why it is exact) and contrast them;
- describe SON in **MapReduce** (Map 1, Reduce 1, Map 2, Reduce 2) and work the five-basket example, including the correct list of 9 candidates.

Slide map

Lecture 4 · Slides 1–86

For Lecture 4 the PDF page equals the printed slide number for slides 1–47. Printed slide 48 does not exist in the PDF, so for printed slides 49–86 the PDF page is the printed number minus 1 (PDF has 85 pages). Slide 1 and the section dividers (Slides 19, 29, 37 and 52) show no number in the slide corner; this guide cites them by their position in the sequence, and the PDF page is given in the second column.

Slide(s)	PDF page(s)	Topic	In this guide
1–2	1–2	Title; motivation: supermarket shelf management, beer and diapers	(p. 152)

3–4	3–4	The market-basket model: items, baskets, rules as output	(p. 152)
5–6	5–6	Applications: market baskets, Amazon, plagiarism, drug side-effects	(p. 154)
7	7	Outline of the lecture	(p. 154)
8–9	8–9	Support, support threshold, frequent itemsets; eight-basket example	(p. 155)
10	10	Association rules and confidence	(p. 163)
11–15	11–15	Interesting rules, interest; confidence and interest example	(p. 164)
16–17	16–17	Mining problem; Step 1 (frequent itemsets) and Step 2 (rule generation)	(p. 170)
18	18	Worked example: generating rules with s = 3, c = 0.75	(p. 172)
19–22	19–22	Divider; computation model: flat files, passes, main-memory bottleneck	(p. 180)
23	23	Why frequent pairs are the hard part	(p. 182)
24–28	24–28	Naive algorithm; triangular matrix vs triples; the 1/3 rule	(p. 183)
29–36	29–36	A-Priori: monotonicity, two passes, renumbering, C_k and L_k, all itemsets	(p. 192)
37–45	37–45	PCY: idle memory, hash buckets, bitmap, Pass 2 test, memory and the 2/3 rule	(p. 210)
46–47	46–47	Multistage: three passes, second hash function	(p. 222)
49–50	48–49	Multihash: several hash tables in one pass (Slide 48 is absent)	(p. 222)
51	50	Extensions of PCY and their limits; comparison of the four algorithms	(p. 223)
52–53	51–52	Divider; frequent itemsets in at most 2 passes: the idea	(p. 235)
54–55	53–54	Random sampling; false positives and false negatives	(p. 236)
56–57	55–56	SON algorithm: chunks, candidates, second pass, monotonicity idea	(p. 238)
58–60	57–59	SON on MapReduce: Map Phase 1, Reduce Phase 1	(p. 240)
61–64	60–63	SON example: dataset, chunks, Map and Reduce Phase 1 on the example	(p. 241)
65–66	64–65	SON MapReduce: Map Phase 2, Reduce Phase 2	(p. 241)
67–69	66–68	SON example: Phase 2 counts, global support, failures struck out	(p. 241)
53–60 (summary)	52–59	Guide-only comparison table: sampling versus SON (revision aid, not a slide)	(p. 252)
70–77	69–76	Quiz slides: cat-and-dog baskets (pairs, triples, confidence, interest)	(p. 253)
78–80	77–79	Quiz slides: A-Priori on four integer baskets	(p. 260)
81–82	80–81	Quiz slides: PCY bitmap, how it is used and why	(p. 261)
83–84	82–83	Quiz slides: the key idea of multistage	(p. 261)
85–86	84–85	Summary and actions (read Chapter 6 of the course book)	(p. 262)

Where this lecture starts: finding what goes together

Lecture 4 · Slides 1–2

Slide 1 is the title card: **"Large-Data Frequent Pattern Analysis, SON MapReduce"** (instructor Dr. Jarek Szlichta). Read it as a one-line summary of the lecture. *Frequent pattern analysis* means finding things that occur together often. *Large-data* means the data is far too big to handle naively. *SON MapReduce* is the final algorithm of the lecture, a way to do the job on a cluster. The slide does not spell out "SON" (Beyond the slides: it stands for Savasere, Omiecinski and Navathe, the authors of the algorithm).

Slide 2 supplies the motivation, **supermarket shelf management**:

Slide 2 bullet	Plain meaning
Goal: identify items that are bought together by sufficiently many customers	Find product combinations that many shoppers buy in the same visit. "Sufficiently many" will become the *support threshold* (Slide 8).
Approach: process the sales data to find dependencies among items	Read through the receipts and look for "if they buy this, they also buy that" patterns.
A classic rule: if someone buys diaper and milk, then they are likely to buy beer	The famous beer-and-diapers rule. Its message: such rules are surprising, so nobody would guess them, and a program must find them in the data. "Do not be surprised if you find six-packs next to diapers!"

BEGINNER INTUITION

Imagine you could read every receipt a supermarket printed this year. You would notice that some groups of products keep turning up on the same receipt. The shop can put those products next to each other, advertise them together, or discount one and not the other. Everything in this lecture is about finding such groups quickly when there are billions of receipts.

COMMON MISTAKE

The beer-and-diapers story on Slide 2 is folklore. The slide gives no data to back it up, and the words "likely" and "sufficiently many" are informal. Do not quote it as a proven fact. What you must know are the precise definitions that replace these vague words: **support**, **confidence** and **interest** (Slides 8–11).

The market-basket model

Lecture 4 · Slides 3–4

Input: items and baskets

Lecture 4 · Slide 3

DEFINITION: ITEM, BASKET, MARKET-BASKET MODEL

The model has two ingredients. A large set of **items** (for example, everything a supermarket sells). A large set of **baskets**, where each basket is a **small subset of items** (for example, the things one customer buys on one day). The model is the picture "many small sets drawn from one big catalogue".

The slide shows the **input** as a table with columns **TID** (transaction ID, simply the basket number) and **Items**. This guide uses it as the 5-basket running table for Slides 3, 4 and 8:

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Five baskets, five distinct items (Beer, Bread, Coke, Diaper, Milk). Three facts about baskets matter later:

- A basket is a **set**. Order is irrelevant and an item appears at most once in it, so {Beer, Bread} = {Bread, Beer}.
- Baskets are **small** compared with the item catalogue: a customer buys a handful of products out of tens of thousands.
- "Basket" is only a name. Anything that groups things together can be a basket (Slides 5–6 show this).

BEGINNER INTUITION

One basket = one shopping receipt. The table is the pile of receipts, reduced to "which products were on each receipt". Prices, quantities and times are thrown away on purpose.

Output: association rules

Lecture 4 · Slide 4

On Slide 4 the same table is the **Input**, and the **Output** is a box titled "Rules Discovered" containing two rules: `{Milk} → {Coke}` and `{Diaper, Milk} → {Beer}` (the slide writes the arrow as "⟶"). The text on the left says what a rule means: **association rules** are statements of the form "people who bought {x, y, z} tend to buy {v, w}".

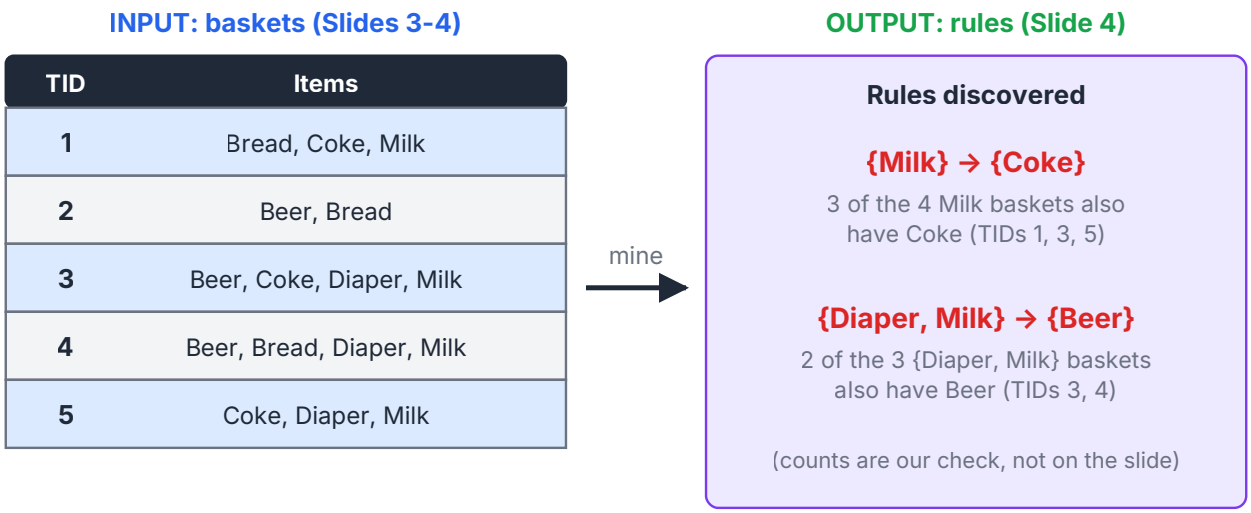


Figure L4p1-1. The market-basket task: a table of baskets goes in, if-then rules come out.

You can check the two rules against the table. Milk is in baskets 1, 3, 4, 5 and Coke is in 1, 3, 5 of those, so Coke accompanies Milk in 3 of 4 baskets. {Diaper, Milk} is in baskets 3, 4, 5 and Beer is in 3 and 4 of those, so 2 of 3. (These two counts are our own check; the slide prints no numbers.) Slide 10 will turn "tend to" into a precise number called confidence.

SLIDE ERRATUM

Slide 4 writes a rule as `{x,y,z} → {v,w}`, with a *set* on the right, but Slide 10 formally defines a rule with *one* item j on the right. Slides 17–18 then use several items on the right again (for example b → c,m). Treat "one item on the right" as the simple textbook case and "a set on the right" as the general case; the confidence formula is the same: support of the whole itemset divided by support of the left side.

Where the model is used

Lecture 4 · Slides 5–6

The model is abstract, so it fits many problems. The slides give four uses. The only thing that changes between them is what you call a "basket" and what you call an "item".

Application	Basket =	Item =	What a frequent pattern tells you
Real market baskets (Slide 5)	the set of products bought in one trip	a product	How shoppers move through a store (position tempting items), and tie-in "tricks", e.g. run a sale on diapers and raise the price of beer. The rule must occur *frequently* "or no $$'s", i.e. no money to be made.
"People who bought X also bought Y" (Slide 5, Amazon)	one customer's purchases	a product	Recommendations: if many baskets contain X and Y together, suggest Y to a buyer of X.
Plagiarism detection (Slide 6)	a **sentence**	a **document** that contains that sentence	Two documents that appear together in many baskets share many sentences. "Items that appear together too often could represent plagiarism."
Drug side-effects (Slide 6)	a **patient**	a drug taken, or a side-effect suffered	A frequent itemset such as {drug A, drug B, side-effect X} flags a drug combination that leads to a particular side-effect.

COMMON MISTAKE

In the plagiarism example the roles look reversed on purpose: **baskets are sentences and items are documents**. Read it as "this sentence appears in these documents". A basket then lists the documents containing one sentence. If you swap the roles you would be counting sentences that co-occur in documents, which answers a different question.

KEY IDEA

Slide 5's last sub-bullet is the reason this lecture talks about *frequent* patterns, not just any pattern. A pattern seen in two receipts out of a million cannot change how a store is run or a recommendation is made. Only patterns that occur often enough are worth acting on. "Often enough" is the support threshold, defined next.

Roadmap of the lecture

Lecture 4 · Slide 7

The roadmap (Slide 7)

Lecture 4 · Slide 7

Slide 7 is a black "Outline" slide. It splits the lecture into two parts:

1. **First, define** frequent itemsets and association rules (Slides 8–18, this part of the guide).
2. **Then, algorithms for finding frequent itemsets:** the A-Priori algorithm, the PCY algorithm "+ 2 refinements", and MapReduce (SON, from the title slide).

The two PCY refinements are not named on the slide. (Beyond the slides: in the textbook *Mining of Massive Datasets* they are the Multistage and Multihash algorithms.)

Support and frequent itemsets

Lecture 4 · Slides 8–9

The definitions (Slide 8)

Lecture 4 · Slide 8

Slide 8 begins with the "simplest question": **find sets of items that appear together "frequently" in baskets.** A set of items is called an **itemset** (a basket is one particular itemset; so is {Beer, Bread}). The slide defines three terms.

DEFINITION: SUPPORT, SUPPORT THRESHOLD, FREQUENT ITEMSET

Support of an itemset I: the **number of baskets that contain all items of I**. Written support(I).

Support threshold s: a number of baskets chosen by the analyst.

Frequent itemset: an itemset with support(I) ≥ s, i.e. it appears in **at least s** baskets.

BEGINNER INTUITION

Support is a **tally**. Go through the receipts. For the itemset you care about, make one tally mark for every receipt that has all its items. The final number of marks is the support. The threshold s is a cut-off line: "I only care about combinations seen at least this many times".

The slide's example uses the table above and states **support of {Beer, Bread} = 2**. The figure shows where the 2 comes from.

Support of {Beer, Bread}: count the rows where BOTH columns are ticked

	Beer	Bread	Coke	Diaper	Milk	
TID 1		✓	✓		✓	
TID 2	✓	✓				counts
TID 3	✓		✓	✓	✓	
TID 4	✓	✓		✓	✓	counts
TID 5			✓	✓	✓	
support	3	3	3	3	4	

{Beer, Bread}: rows 2 and 4 have both ticks, so support = 2. Column sums = support of single items.

Figure L4p1-2. Support is a tally of rows (baskets). A row counts for {Beer, Bread} only if both columns are ticked.

Method, in three steps: (1) for each basket ask "does it contain *every* item of the itemset?"; (2) make a tally mark for each yes; (3) the number of marks is the support. Baskets 2 and 4 contain both Beer and Bread, so the support is 2. With $s = 2$ the itemset would be frequent; with $s = 3$ it would not.

COMMON MISTAKES ABOUT SUPPORT

- **Support is a count of baskets here**, not a fraction or percentage. (Beyond the slides: many textbooks and tools divide by the number of baskets; if an exam gives a percentage threshold, convert with s = fraction × number of baskets.)
- A basket may contain **extra items** and still counts: basket 4 has Diaper and Milk as well, and it still counts for {Beer, Bread}.
- You count **baskets**, not purchases. Buying three Beers in one basket adds one, not three.
- "At least s" means **≥**. An itemset with support exactly s is frequent.
- Always state s. The same data gives different frequent itemsets for different thresholds.

Worked example: eight baskets, s = 3 (Slide 9)

Lecture 4 · Slide 9

Slide 9 changes data and uses short letters: items = {milk, coke, pepsi, beer, juice}, written `m, c, p, b, j`. The support threshold is **3 baskets**. In these letters `b` means **beer** (on Slides 3, 4 and 8 the letter B would suggest Bread).

Basket	Items	Basket	Items
B_1	{m, c, b}	B_2	{m, p, j}
B_3	{m, b}	B_4	{c, j}
B_5	{m, p, b}	B_6	{m, c, b, j}
B_7	{c, b, j}	B_8	{b, c}

The slide lists the answer: **frequent itemsets: {m}, {c}, {b}, {j}, {m,b}, {b,c}, {c,j}**. It does not show the counts, so here they are, level by level:

1. **Single items.** m = 5 (B1, B2, B3, B5, B6); c = 5 (B1, B4, B6, B7, B8); b = 6 (B1, B3, B5, B6, B7, B8); j = 4 (B2, B4, B6, B7); p = 2 (B2, B5). Keep those ≥ 3: m, c, b, j. **Pepsi is not frequent**, although it is on the item list.
2. **Pairs.** Only {m,b} = 4 (B1, B3, B5, B6), {b,c} = 4 (B1, B6, B7, B8) and {c,j} = 3 (B4, B6, B7) reach 3. Everything else has support 2 or less; for example {m,c} = 2 (B1, B6).
3. **Triples.** No triple reaches 3. The best is {m,c,b} with 2 (B1, B6). So there are no frequent triples and nothing larger.

Slide 9, redrawn: which baskets contain each frequent pair? (s = 3)

Basket	Items	{m,b}	{b,c}	{c,j}
B1	{m, c, b}	●	●	
B2	{m, p, j}			
B3	{m, b}	●		
B4	{c, j}			●
B5	{m, p, b}	●		
B6	{m, c, b, j}	●	●	●
B7	{c, b, j}		●	●
B8	{b, c}		●	
Support (number of dots)		4 ≥ 3 ✓	4 ≥ 3 ✓	3 ≥ 3 ✓

Every other pair has support 2 or less, so it is NOT frequent (for example {m,c}: only B1 and B6).

Figure L4p1-3. The coloured arrows of Slide 9 as a clean table: a dot means the basket contains the pair.

The slide's eleven coloured arrows say exactly what the dots say: red for {m,b}, green for {b,c}, dark blue for {c,j}. The slide does not explain the colours and the arrows overprint the text, so use the figure instead.

KEY IDEA: A PATTERN YOU WILL SEE AGAIN

Every frequent pair above is made only of frequent single items (m, c, b, j), and pepsi, the one infrequent item, appears in no frequent pair. This is the **monotonicity** property: if a set is frequent, so are all its subsets. Equivalently, a basket that contains a bigger set also contains every smaller set, so the smaller set's support can only be larger. The A-Priori algorithm (later slides) is built on this one fact, and Slide 17 already uses it.

COMMON MISTAKE

Do not assume two frequent items make a frequent pair. m and c are both frequent (5 each), but {m,c} has support only 2. The converse direction is the safe one: a frequent pair always has frequent members.

EXAM TIP

For "find all frequent itemsets" questions, build a tick table (rows = baskets, columns = items) and count level by level: singles, then pairs, then triples, until a level has no frequent set. Write each count with the basket numbers that support it; that earns the method marks even if one count is off. Quiz L4-Q1 does this in full.

Try it yourself first: cover the solution, verify that support({Beer, Bread}) = 2, then find every frequent itemset of the eight baskets with s = 3 and compare with the slide's list.

Quiz L4-Q1 **Support and frequent itemsets (two examples)** Lecture 4 · Slides 8–9

QUESTION (AS ON THE SLIDES)

Slide 8 — Frequent Itemsets. "Simplest question: find sets of items that appear together 'frequently' in baskets." *Support* for an itemset I is the number of baskets containing all items in I. Given a *support threshold* s, the sets of items that appear in **at least** s baskets are called *frequent itemsets*. The slide shows the five baskets below (the same table as on Slides 3–4) and states: support of `{Beer, Bread}` = 2.

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

(a) Verify that the support of `{Beer, Bread}` is 2. *Practice* (not printed on the slide): compute the support of **every** single item, **every** pair and **every** triple of items in this table, and list the frequent itemsets. Slide 8 gives no threshold, so the practice uses $s = 3$ and $s = 2$ (our choice, not the slide's).

Slide 9 — Example: Frequent Itemsets. Items = {milk, coke, pepsi, beer, juice} (written m, c, p, b, j). Support threshold = **3 baskets**. The eight baskets are:

Basket	Items	Basket	Items
B_1	`{m, c, b}`	B_2	`{m, p, j}`
B_3	`{m, b}`	B_4	`{c, j}`
B_5	`{m, p, b}`	B_6	`{m, c, b, j}`
B_7	`{c, b, j}`	B_8	`{b, c}`

(b) Find **all frequent itemsets**. Show the full count table for all single items, all pairs and all triples (and say what happens for bigger sets). The slide's answer: `{m}, {c}, {b}, {j}, {m,b}, {b,c}, {c,j}`.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

A **basket** is one shopping receipt. The **support** of a set of items is just a **tally**: go through the pile of receipts and make one tally mark for every receipt that has *all* the items of the set on it. Nothing else: no averages, no percentages, just a count. A set is **frequent** when its tally reaches the threshold s. Think of s as a cut-off line: "I only care about combinations that happen at least this many times".

FORMULAS USED IN THIS QUIZ

Support: support(I) = number of baskets B with $I \subseteq B$ ("$\subseteq$" = every item of I is also in B). **Frequent:** I is frequent $\Leftrightarrow$ support(I) $\geq s$. **How many itemsets:** from n items there are $C(n, k) = n! \div (k!(n-k)!)$ itemsets of size k, and $2^n - 1$ non-empty itemsets in total.

Part (a) — the five baskets of Slide 8

1. **Formula / idea:** support(I) = the number of baskets that contain *every* item of I. Three consequences that beginners often miss: (i) order does not matter, `{Beer, Bread}` = `{Bread, Beer}`; (ii) a basket may contain *extra* items and still counts (basket 4 also has Diaper and Milk); (iii) what is counted is **baskets**, not how many times an item is bought.
2. **Verify the slide's claim, support of `{Beer, Bread}` = 2.** Look at each basket and ask "does it have Beer *and* Bread?".

TID	Basket	has Beer?	has Bread?	has both?
1	Bread, Coke, Milk	no	yes	no
2	Beer, Bread	yes	yes	**yes**
3	Beer, Coke, Diaper, Milk	yes	no	no
4	Beer, Bread, Diaper, Milk	yes	yes	**yes**
5	Coke, Diaper, Milk	no	no	no
Number of baskets with both				**2**

   Only baskets **2** and **4** contain both, so support(`{Beer, Bread}`) = **2**. This agrees with the slide. (Beer alone is in baskets 2, 3, 4 and Bread alone in baskets 1, 2, 4, but they are *together* only in 2 and 4: basket 1 has Bread without Beer, basket 3 has Beer without Bread.)
3. **Practice: build a tick table so every later count is just reading a column.** One row per basket, one column per item, a tick where the basket contains the item. The **column sum** is the support of that single item.

TID	Beer	Bread	Coke	Diaper	Milk	Basket (slide 8)
1		✓	✓		✓	Bread, Coke, Milk
2	✓	✓				Beer, Bread
3	✓		✓	✓	✓	Beer, Coke, Diaper, Milk
4	✓	✓		✓	✓	Beer, Bread, Diaper, Milk
5			✓	✓	✓	Coke, Diaper, Milk
support	**3**	**3**	**3**	**3**	**4**	= column sum

   Check: ticks counted by rows = 3 + 2 + 4 + 4 + 3 = 16, by columns = 3 + 3 + 3 + 3 + 4 = 16. The two totals agree, so no tick was lost. Single-item supports: Beer 3, Bread 3, Coke 3, Diaper 3, Milk 4.
4. **Practice: support of every pair.** There are $C(5, 2) = 5! \div (2! \cdot 3!) = 10$ pairs. For each pair, take the baskets of one item and check whether the other item is also there (shortcut: start from the item with fewer baskets).

Itemset	Baskets (TIDs) containing all of it	Support	frequent if s = 3?	frequent if s = 2?
`{Beer, Bread}` (the slide's example)	2, 4	**2**	no	**yes**
`{Beer, Coke}`	3	**1**	no	no
`{Beer, Diaper}`	3, 4	**2**	no	**yes**
`{Beer, Milk}`	3, 4	**2**	no	**yes**
`{Bread, Coke}`	1	**1**	no	no
`{Bread, Diaper}`	4	**1**	no	no
`{Bread, Milk}`	1, 4	**2**	no	**yes**
`{Coke, Diaper}`	3, 5	**2**	no	**yes**
`{Coke, Milk}`	1, 3, 5	**3**	**yes**	**yes**
`{Diaper, Milk}`	3, 4, 5	**3**	**yes**	**yes**

Highlights: `{Coke, Milk}` and `{Diaper, Milk}` have support 3; `{Beer, Bread}` has support 2 (as the slide says); `{Bread, Coke}` and `{Bread, Diaper}` have support only 1. Check: a basket with k items contains C(k, 2) pairs, so the five baskets contain 3 + 1 + 6 + 6 + 3 = 19 pairs; the ten supports add to 2 + 1 + 2 + 2 + 1 + 1 + 2 + 2 + 3 + 3 = 19. ✓

5. **Practice: support of every triple (and larger sets).** There are C(5, 3) = 10 triples. A triple needs all three items in one basket, so only baskets with at least 3 items (baskets 1, 3, 4, 5) can contribute.

Itemset	Baskets (TIDs) containing all of it	Support	frequent if s = 3?	frequent if s = 2?
`{Beer, Bread, Coke}`	none	**0**	no	no
`{Beer, Bread, Diaper}`	4	**1**	no	no
`{Beer, Bread, Milk}`	4	**1**	no	no
`{Beer, Coke, Diaper}`	3	**1**	no	no
`{Beer, Coke, Milk}`	3	**1**	no	no
`{Beer, Diaper, Milk}`	3, 4	**2**	no	**yes**
`{Bread, Coke, Diaper}`	none	**0**	no	no
`{Bread, Coke, Milk}`	1	**1**	no	no
`{Bread, Diaper, Milk}`	4	**1**	no	no
`{Coke, Diaper, Milk}`	3, 5	**2**	no	**yes**

The largest triple support is 2 (`{Beer, Diaper, Milk}` in baskets 3 and 4; `{Coke, Diaper, Milk}` in baskets 3 and 5). Check: the baskets contain 1 + 0 + 4 + 4 + 1 = 10 triples, and the ten supports add to 0 + 1 + 1 + 1 + 1 + 2 + 0 + 1 + 1 + 2 = 10. ✓ Bigger sets: only `{Beer, Bread, Diaper, Milk}` (basket 4) and `{Beer, Coke, Diaper, Milk}` (basket 3) occur at all, each with support 1; the other three 4-itemsets and the 5-itemset have support 0.

6. **Practice: apply the threshold rule "support ≥ s".** Slide 8 gives no s, so practise with two values:
   - **s = 3**: frequent = all five singles (Beer, Bread, Coke, Diaper, Milk) plus the pairs `{Coke, Milk}` and `{Diaper, Milk}`; no triple. That is **7 frequent itemsets**. `{Beer, Bread}` (support 2) is **not** frequent here.
   - **s = 2**: frequent = the 5 singles; the 7 pairs `{Beer, Bread}`, `{Beer, Diaper}`, `{Beer, Milk}`, `{Bread, Milk}`, `{Coke, Diaper}`, `{Coke, Milk}`, `{Diaper, Milk}`; and the 2 triples `{Beer, Diaper, Milk}` and `{Coke, Diaper, Milk}`. That is **14 frequent itemsets** (5 + 7 + 2). Now `{Beer, Bread}` is frequent.

   Lesson: the same data gives different frequent itemsets for different thresholds, so always state s. Also notice that every subset of a frequent itemset is frequent: `{Beer, Diaper, Milk}` is frequent at s = 2, and so are `{Beer, Diaper}`, `{Beer, Milk}`, `{Diaper, Milk}`, `{Beer}`, `{Diaper}` and `{Milk}` (this is the *monotonicity* property of Slide 30, used again in part (b)).

7. **Cross-check by counting the other way round (basket by basket).** A basket with k items contains $2^k - 1$ non-empty sub-itemsets (basket 3 with 4 items has $2^4 - 1 = 15$: 4 singles + 6 pairs + 4 triples + 1 four-set). Adding 1 to the counter of each of them, for every basket, gives exactly the supports above. For these five baskets that is $(2^3 - 1) + (2^2 - 1) + (2^4 - 1) + (2^4 - 1) + (2^3 - 1) = 7 + 3 + 15 + 15 + 7 = 47$ counter increments, which equals the totals by size, 16 singles + 19 pairs + 10 triples + 2 four-sets + 0 five-sets = 47. ✓ This basket-by-basket view is what a program does, and it shows why counting *all* itemsets explodes for big baskets (the reason for the clever algorithms later in Lecture 4).

PREVIEW: WHY SUPPORT MATTERS (SLIDES 4 AND 10)

The two rules on Slide 4 are built from these counts. Confidence (Slide 10) = support($I \cup j$) ÷ support(I). For `{Milk} → {Coke}`: support(`{Milk, Coke}`) = 3 and support(`{Milk}`) = 4, so confidence = 3/4 = 0.75. For `{Diaper, Milk} → {Beer}`: support(`{Beer, Diaper, Milk}`) = 2 and support(`{Diaper, Milk}`) = 3, so confidence = 2/3 ≈ 0.667. You do not need these for this quiz; they only show that every association rule starts from support counts.

Part (b) — the eight baskets of Slide 9, threshold $s = 3$

1. **Plan and size of the problem.** With $n = 5$ items the possible non-empty itemsets are C(5,1) + C(5,2) + C(5,3) + C(5,4) + C(5,5) = 5 + 10 + 10 + 5 + 1 = $2^5 - 1$ = **31**. We count singles, then pairs, then triples, then see what is left. The rule applied to every count is the one from Slide 8: **frequent ⇔ support ≥ 3**. (We write itemsets in the order m, b, c, j, p so that the pairs read exactly as on the slide: `{m,b}`, `{b,c}`, `{c,j}`.)
2. **Tick table.** Rows = baskets, columns = items; the column sum is the support of the single item.

Basket	m milk	b beer	c coke	j juice	p pepsi	Items
B1	✓	✓	✓			`{m, b, c}`
B2	✓			✓	✓	`{m, j, p}`
B3	✓	✓				`{m, b}`
B4			✓	✓		`{c, j}`
B5	✓	✓			✓	`{m, b, p}`
B6	✓	✓	✓	✓		`{m, b, c, j}`
B7		✓	✓	✓		`{b, c, j}`
B8		✓	✓			`{b, c}`
support	**5**	**6**	**5**	**4**	2	= column sum

   Check: ticks by rows = 3 + 3 + 2 + 2 + 3 + 4 + 3 + 2 = 22; by columns = 5 + 6 + 5 + 4 + 2 = 22. The totals agree.
3. **Single items (size 1).**

Itemset	Baskets containing all of it	Support	≥ 3 ?
`{m}` (milk)	B_1, B_2, B_3, B_5, B_6	**5**	**frequent**
`{b}` (beer)	$B_1, B_3, B_5, B_6, B_7, B_8$	**6**	**frequent**
`{c}` (coke)	B_1, B_4, B_6, B_7, B_8	**5**	**frequent**
`{j}` (juice)	B_2, B_4, B_6, B_7	**4**	**frequent**
`{p}` (pepsi)	B_2, B_5	**2**	NOT frequent

   `{m}` = 5, `{b}` = 6, `{c}` = 5 and `{j}` = 4 are frequent. **Pepsi** appears in only B_2 and B_5 (support 2 < 3), so `{p}` is **not** frequent.
4. **Pairs (size 2).** C(5, 2) = 10 pairs. For each, list the baskets that contain both items:

Itemset	Baskets containing all of it	Support	≥ 3 ?
`{m,b}`	B_1, B_3, B_5, B_6	**4**	**frequent**
`{m,c}`	B_1, B_6	**2**	no
`{m,j}`	B_2, B_6	**2**	no
`{m,p}`	B_2, B_5	**2**	no
`{b,c}`	B_1, B_6, B_7, B_8	**4**	**frequent**
`{b,j}`	B_6, B_7	**2**	no
`{b,p}`	B_5	**1**	no
`{c,j}`	B_4, B_6, B_7	**3**	**frequent**
`{c,p}`	none	**0**	no
`{j,p}`	B_2	**1**	no

Frequent pairs: `{m,b}` (4: B_1, B_3, B_5, B_6), `{b,c}` (4: B_1, B_6, B_7, B_8) and `{c,j}` (3: B_4, B_6, B_7). Two things to observe: `{c,j}` has support *exactly* 3 and is frequent because the rule is "at least s" (≥, not >); and `{m,c}` is **not** frequent (support 2) even though m and c are both frequent on their own. Every pair containing pepsi has support ≤ 2. Check: the baskets contain 3 + 3 + 1 + 1 + 3 + 6 + 3 + 1 = 21 pairs; the ten supports add to 4 + 2 + 2 + 2 + 4 + 2 + 1 + 3 + 0 + 1 = 21. ✓

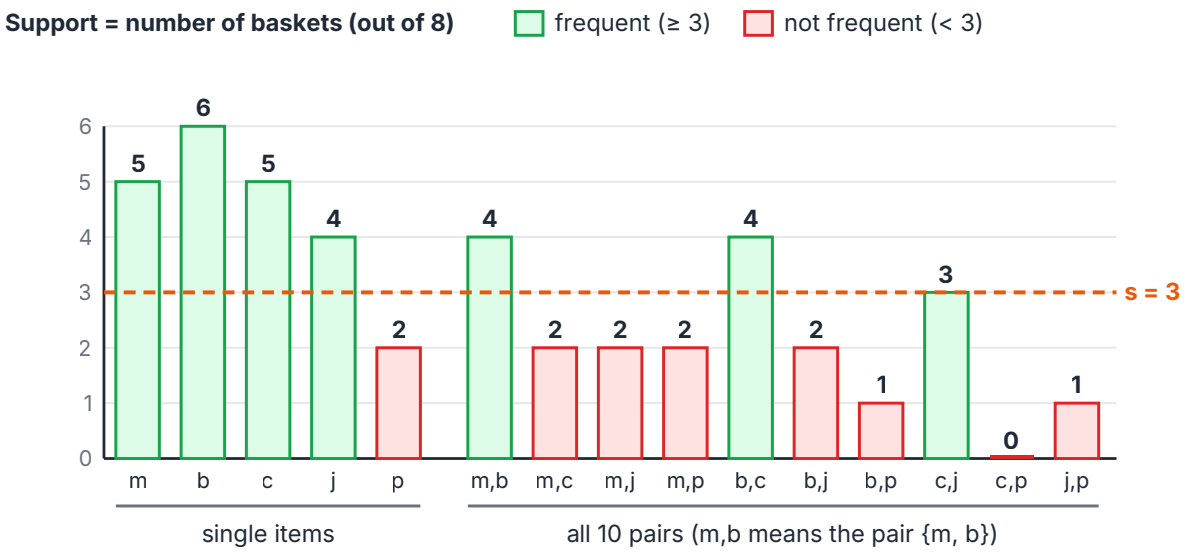


Figure L4-Q1: support of the 5 single items and the 10 pairs of Slide 9. Bars that reach the dashed line s = 3 are frequent (green): m, b, c, j, m,b, b,c, c,j. Pepsi (p) and the pairs m,c, m,j, m,p, b,j, b,p, c,p, j,p stay below the line. (All triples have support ≤ 2.)

5. **Triples (size 3).** C(5, 3) = 10 triples:

Itemset	Baskets containing all of it	Support	≥ 3 ?
`{m,b,c}`	B_1, B_6	**2**	no
`{m,b,j}`	B_6	**1**	no
`{m,b,p}`	B_5	**1**	no
`{m,c,j}`	B_6	**1**	no
`{m,c,p}`	none	**0**	no
`{m,j,p}`	B_2	**1**	no
`{b,c,j}`	B_6, B_7	**2**	no
`{b,c,p}`	none	**0**	no
`{b,j,p}`	none	**0**	no
`{c,j,p}`	none	**0**	no

The largest triple support is 2 (`{m,b,c}` in B_1, B_6; `{b,c,j}` in B_6, B_7), which is below 3, so **no triple is frequent**. Check: the baskets contain 1 + 1 + 0 + 0 + 1 + 4 + 1 + 0 = 8 triples (B_1, B_2, B_5 and B_7 one each, B_6 four); the ten supports (table order) add to 2 + 1 + 1 + 1 + 0 + 1 + 2 + 0 + 0 + 0 = 8. ✓

6. **Bigger itemsets (sizes 4 and 5).** The only basket with 4 items is B_6 = `{m,c,b,j}`, so `{m,b,c,j}` has support 1; the other four 4-itemsets and the 5-itemset `{m,b,c,j,p}` have support 0. None is frequent. We did not even need to count them: if a set appears in at least 3 baskets then **every subset of it** does too (*monotonicity*, Slide 30). No triple is frequent, so no 4-set or 5-set (each contains triples) can be frequent.
7. **Collect the answer.** Frequent singles: `{m}`, `{c}`, `{b}`, `{j}`. Frequent pairs: `{m,b}`, `{b,c}`, `{c,j}`. No frequent triples or larger. In total **7 frequent itemsets out of the 31 possible**, exactly the list on Slide 9.

EXAM TIP: A SHORTCUT THAT NEEDS NO TRIPLE COUNTING (MONOTONICITY, SLIDE 30)

Two facts save work. (1) Since `{p}` is not frequent, no pair containing p can be frequent (Slide 30, "contrapositive for pairs"), so only 6 of the 10 pairs even needed counting (those made of m, b, c, j). (2) A triple can be frequent only if **all three** of its pairs are frequent. The frequent pairs `{m,b}`, `{b,c}`, `{c,j}` form a chain m – b – c – j, never a "triangle": `{m,b,c}` would need `{m,c}` (support 2) and `{b,c,j}` would need `{b,j}` (support 2). So there is no candidate triple and we could have stopped after the pairs. You will use exactly this reasoning in the A-Priori algorithm.

READING THE COLOURED ARROWS ON SLIDE 9

The slide draws arrows from each frequent pair to the baskets that contain it: **red** arrows from `{m,b}` to B_1, B_3, B_5, B_6 (4 baskets); **green** arrows from `{b,c}` to B_1, B_6, B_7, B_8 (4 baskets); **blue** arrows from `{c,j}` to B_4, B_6, B_7 (3 baskets). The number of arrows ending at baskets is the support. These match the pair table above (the slide has no colour legend; the colour-to-itemset reading is ours).

COMMON MISTAKES

- **Counting baskets with Beer *or* Bread** (4 baskets) instead of Beer *and* Bread (2), or adding the two single supports (3 + 3 = 6). Only baskets with *both* count.
- **Counting only baskets that are *exactly* {Beer, Bread}** (basket 2 only, giving 1). A basket with extra items, like basket 4, still counts.
- **Using "> s" instead of "≥ s".** `{c,j}` has support exactly 3 = s and *is* frequent.
- **Assuming two frequent items make a frequent pair.** m and c are both frequent, but `{m,c}` has support 2 and is not.
- **Mixing up count and fraction.** On the slides support is a *count of baskets* (support 2, threshold "3 baskets"). Beyond the slides: some textbooks divide by the number of baskets (here 2/5 = 0.4); when a question gives the threshold as a number of baskets, compare counts.
- **Forgetting that order does not matter** and that you must state s: the frequent list changes with the threshold (7 itemsets at s = 3 but 14 at s = 2 in part (a)).

Final answer:
(a) support(`{Beer, Bread}`) = 2 (baskets 2 and 4), as the slide says. Singles: Beer 3, Bread 3, Coke 3, Diaper 3, Milk 4. Pairs: Coke–Milk 3 and Diaper–Milk 3; Beer–Bread, Beer–Diaper, Beer–Milk, Bread–Milk, Coke–Diaper 2 each; Beer–Coke, Bread–Coke, Bread–Diaper 1 each. Triples: `{Beer, Diaper, Milk}` 2, `{Coke, Diaper, Milk}` 2, six triples 1, and `{Beer, Bread, Coke}`, `{Bread, Coke, Diaper}` 0. Practice thresholds: s = 3 gives 7 frequent itemsets (5 singles + `{Coke, Milk}`, `{Diaper, Milk}`); s = 2 gives 14 (5 singles, 7 pairs, 2 triples).
(b) Supports: m 5, b 6, c 5, j 4, p 2; pairs mb 4, bc 4, cj 3, mc 2, mj 2, mp 2, bj 2, bp 1, jp 1, cp 0; no triple above 2; no 4- or 5-itemset above 1. Frequent itemsets (s = 3): `{m}`, `{c}`, `{b}`, `{j}`, `{m,b}`, `{b,c}`, `{c,j}`: 7 of the 31 possible itemsets.

Where the marks come from: (1) state the definition: support = number of *baskets* containing *all* items of the set, and frequent = support ≥ s; (2) show the check for `{Beer, Bread}` by naming baskets 2 and 4; (3) correct single-item counts (the tick table), discarding `{p}` in part (b); (4) correct pair counts, with the baskets named, in particular `{m,b}` = 4, `{b,c}` = 4, `{c,j}` = 3 and rejecting `{m,c}`, `{m,j}`, `{b,j}` (each 2); (5) correct triple counts, or a valid monotonicity argument that no triple can be frequent; (6) apply the "≥ 3" threshold correctly, including the boundary case `{c,j}` = 3; (7) state the final list of 7 frequent itemsets. Even with a counting slip, a clearly stated method keeps most of the marks.

Slide check: matches the slide. Slide 8's "support of {Beer, Bread} = 2" is correct, and Slide 9's list of seven frequent itemsets is exactly what the full count gives. Notes (not errors): Slide 8 states no threshold, so the s = 3 and s = 2 results in part (a) are our practice choices; Slide 9 shows the answer without the count table (supplied above) and its arrow colours have no legend.

Association rules and confidence

Lecture 4 · Slide 10

Frequent itemsets say *which items occur together*. They do not say *which item leads to which*. A rule adds direction, in the form "if you see this, expect that".

DEFINITION: ASSOCIATION RULE AND CONFIDENCE (SLIDE 10)

An **association rule** is an if-then rule about the contents of baskets. The rule $\{i_1, i_2, \ldots, i_k\} \to j$ means: **"if a basket contains all of $i_1, \ldots, i_k$, then it is *likely* to contain j"**. Write $I = \{i_1, \ldots, i_k\}$ for the left side.

The **confidence** of the rule is the probability of j given I:

FORMULA

$$\text{conf}(I \to j) = \frac{\text{support}(I \cup \{j\})}{\text{support}(I)}$$

Numerator: baskets containing *all* of I and also j. Denominator: baskets containing I. The slide writes "support($I \cup j$)"; it means the itemset I with the item j added.

BEGINNER INTUITION

Pick out only the receipts that contain the "if" items. Of those, what fraction also contain the "then" item? That fraction is the confidence. The left side defines the population you look at, which is why it is the **denominator**.

Facts to remember:

- Confidence lies between 0 and 1 (0 = never together, 1 = always together).
- It is **not symmetric**: conf($I \to j$) and conf($j \to I$) have the same numerator but different denominators. Slide 4's {Milk} → {Coke} has 3/4, whereas {Coke} → {Milk} has 3/3 = 1 (Coke is in baskets 1, 3, 5 and all three have Milk).
- Slide 10 warns: **"in practice there are many rules; we want to find the significant/interesting ones!"** Confidence alone is not enough, as the next slide shows.

COMMON MISTAKE

Putting the wrong support in the denominator. The denominator is the support of the **left side only** (the "if" part), never of the whole rule and never of the right side.

Interesting rules and the interest measure

Lecture 4 · Slides 11–15

Why high confidence can mislead (Slide 11)

Lecture 4 · Slide 11

Slide 11's headline: **not all high-confidence rules are interesting.** The example: the rule $X \to$ milk may have high confidence for many itemsets X, "because milk is just purchased very often (independent of X) and the confidence will be high".

BEGINNER INTUITION

Suppose 95% of all receipts contain milk. Then "if the basket has batteries, it has milk" is true about 95% of the time, and so is the same rule for shoelaces, tyres and anything else. High confidence, zero information: milk would be there anyway. To find a *real* association, ask whether knowing the "if" part **improves on the baseline** chance of the "then" item.

DEFINITION AND FORMULA: INTEREST (SLIDE 11)

The **interest** of a rule $I \to j$ is the difference between its confidence and the fraction of baskets that contain j:

Interest($I \to j$) = conf($I \to j$) − Pr[j], where Pr[j] = (number of baskets containing j) ÷ (total number of baskets).

The slide says interesting rules are those with **high interest values (usually above 0.5)**. How to read the number:

Interest	Meaning
≈ 0	Knowing I tells you nothing about j: the rule just reflects how popular j is.
large positive	I makes j much more likely than usual: a real association.
negative	I makes j *less* likely than usual.

SLIDE ERRATUM / OMISSION

Slide 11 mentions only *high* interest. Beyond the slides: the textbook (*Mining of Massive Datasets*) counts a rule as interesting when the interest is high in **absolute value**, positive or negative, because a strongly negative value is also informative. The slide also never gives a formula for Pr[j]. The threshold "usually above 0.5" is only a rule of thumb.

Example: confidence and interest of {m,b} → c (Slides 12–15)

Lecture 4 · Slides 12–15

Slides 12–15 are one slide built up in four steps, on the same eight baskets as Slide 9, for the rule **{m, b} → c** ("if milk and beer, then coke"). Each step reveals one more line:

Slide	What it shows	How to get it
12	Question: Confidence = ?	conf = support({m,b,c}) / support({m,b})
13	Answer: **2/4 = 1/2**	Numerator 2 (B1, B6 hold m, b *and* c); denominator 4 (B1, B3, B5, B6 hold m and b)
14	Question: Interest = ?	Interest = conf − Pr[c]
15	Answer: **1/2 − 5/8 < 0.5**; "item c appears in 5/8 of the baskets"; "rule is not very interesting!"	c is in B1, B4, B6, B7, B8, so Pr[c] = 5/8 = 0.625

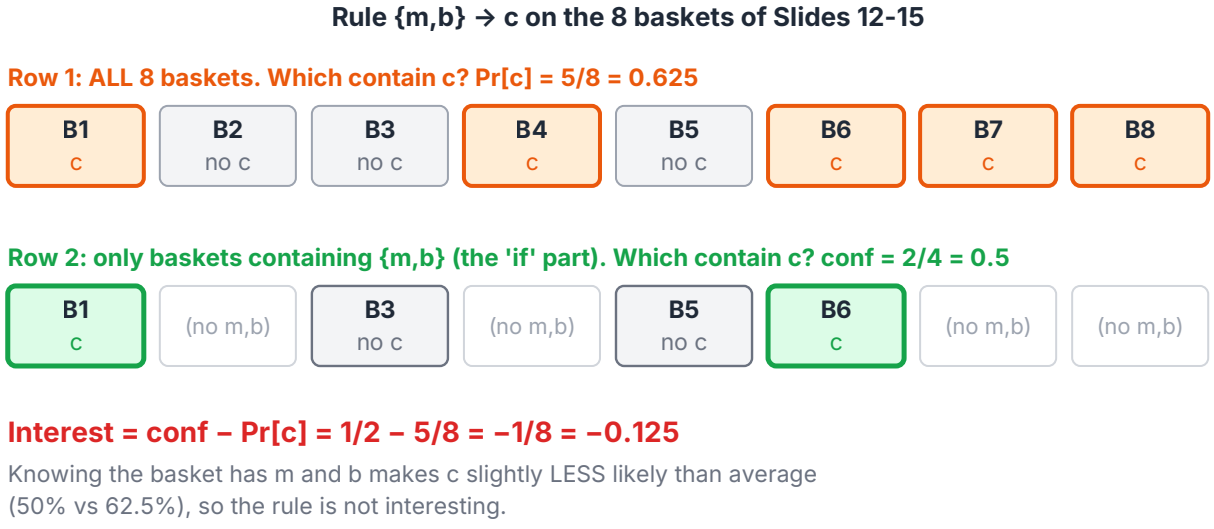


Figure L4p1-4. Confidence looks only at row 2; Pr[c] looks at row 1. Interest is the gap between the two.

Reading the figure: confidence uses only the green row (baskets that satisfy the "if" part); the baseline Pr[c] uses the orange row (all baskets). Coke appears in 50% of the baskets that have milk and beer, but in 62.5% of *all* baskets. Milk and beer do not help predict coke; they slightly hurt. The rule has respectable confidence (0.5) and is still not interesting. That is exactly the situation Slide 11 warns about.

SLIDE ERRATUM

Slide 15 writes "Interest = 1/2 − 5/8 < 0.5" and never evaluates the difference. The value is **1/2 − 5/8 = 4/8 − 5/8 = −1/8 = −0.125**. The "< 0.5" is trivially true for a negative number; the meaningful statement is that $|-0.125| = 0.125$ is far below 0.5. The same slides also contain the typo "B4=" (missing space). Neither changes the conclusion.

EXAM TIP

Write the three counts (support of the left side, support of the whole itemset, number of baskets containing the right-hand item) before dividing. Then give confidence, Pr[j], the signed interest, and one sentence of interpretation. Each is a separate mark.

Try it yourself first: compute the confidence and the interest of {m,b} → c from the eight baskets, then decide whether the rule is interesting.

Quiz L4-Q2 **Confidence and interest of an association rule** Lecture 4 · Slides 10–15

QUESTION (AS ON THE SLIDES)

There are eight shopping baskets over the items *m* = milk, *c* = coke, *p* = pepsi, *b* = beer, *j* = juice (the item list and the same baskets as on Slide 9; they are repeated on Slides 12–15):

B_1	{m, c, b}	B_2	{m, p, j}
B_3	{m, b}	B_4	{c, j}
B_5	{m, p, b}	B_6	{m, c, b, j}
B_7	{c, b, j}	B_8	{b, c}

Consider the association rule {m, b} → c ("if a basket contains milk and beer, it is likely to contain coke").

(a) What is the **confidence** of the rule? (Slide 12: "Confidence = ?")

(b) What is the **interest** of the rule? (Slide 14: "Interest = ?")

(c) Is the rule interesting? Justify your answer with the interest value. (Slide 15 states the conclusion; Slide 11 says interesting rules have high interest, usually above 0.5.)

The slides ask only "Confidence = ?" and "Interest = ?". Part (c) is added from the closing remark of Slide 15 ("Rule is not very interesting!") so that you also practise justifying the verdict.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Imagine a school lunch line in which almost everybody takes a water bottle. The rule "if you took a sandwich, then you took water" is right nearly every time (high confidence), yet it is useless, because water would have been taken anyway. **Interest** removes that illusion by subtracting how often water is taken *anyway*. Equivalently, ask: "does learning that the basket has milk and beer improve on a blind guess about coke?" Blind guess: coke is in 62.5% of all baskets. After learning about milk and beer: only 50%. So no, it does not help. Coke is fairly common (62.5% of baskets), and baskets with milk and beer contain it even less often (50%), so the rule does worse than a blind guess. The same logic as the water-bottle case of Slide 11, with a milder base rate.

FORMULAS USED (LECTURE 4 · SLIDES 8, 10, 11)

Support of an itemset S: support(S) = the number of baskets that contain *every* item of S (Slide 8).

Confidence of the rule I → j (Slide 10; it is the probability of j given I): $\text{conf}(I \to j) = \frac{\text{support}(I \cup \{j\})}{\text{support}(I)}$.

Here I ∪ {j} means the itemset I with the item j added, so a basket must contain *all* of those items.

Baseline probability of the right-hand item (Slide 11): Pr[j] = (number of baskets containing j) ÷ (total number of baskets).

Interest (Slide 11): Interest(I → j) = conf(I → j) − Pr[j]

1. **Name the pieces of the rule.** In {m, b} → c the "if" part (left side) is I = {m, b}, the "then" item (right side) is j = c, so I ∪ {j} = {m, b, c}, and the total number of baskets is 8. *Why:* every formula above is written in terms of I and j, so we must say which is which before we count anything.
2. **Check every basket once.** Support is only counting, so go through B_1 to B_8 and record three yes/no questions: does the basket contain both *m* and *b*? does it contain *c*? does it contain all of *m*, *b* and *c*? *Why:* the three totals at the bottom of the table are exactly the three numbers we need.

Basket	Items	contains *m* and *b*?	contains *c*?	contains *m*, *b* and *c*?
B_1	{m, c, b}	✓	✓	✓
B_2	{m, p, j}	× (no *b*)	×	×
B_3	{m, b}	✓	×	×
B_4	{c, j}	× (no *m*, no *b*)	✓	×
B_5	{m, p, b}	✓	×	×
B_6	{m, c, b, j}	✓	✓	✓
B_7	{c, b, j}	× (no *m*)	✓	×
B_8	{b, c}	× (no *m*)	✓	×
Total		**4** (B_1, B_3, B_5, B_6)	**5** (B_1, B_4, B_6, B_7, B_8)	**2** (B_1, B_6)

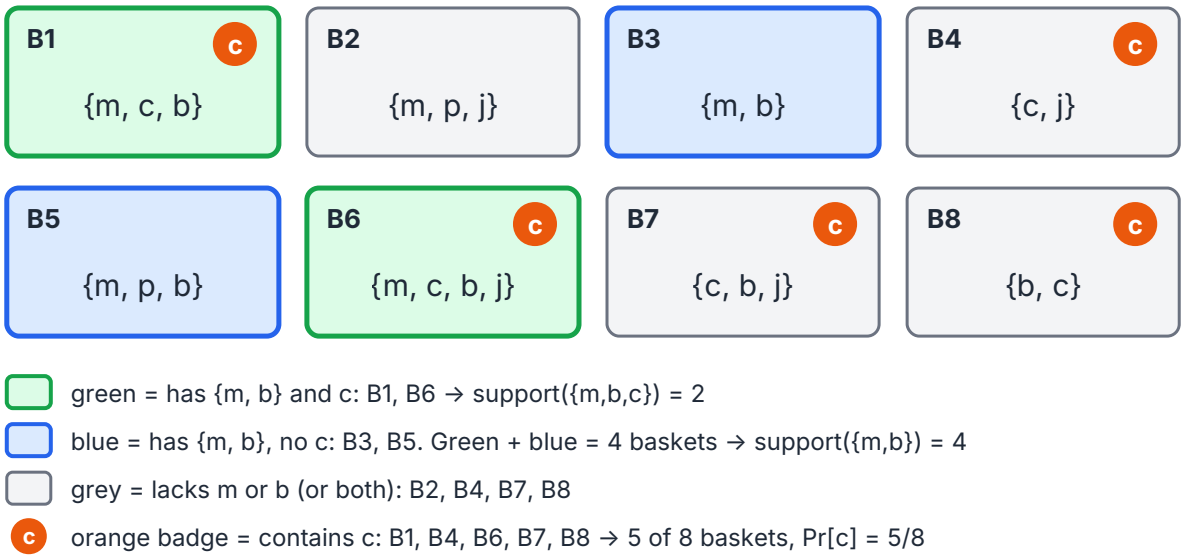


Figure L4-Q2-1. The eight baskets, coloured by what we must count. Confidence looks only at the green and blue baskets (those where the "if" part holds); Pr[c] looks at all eight baskets (orange badges).

3. **Count the "if" part: support(I) = support({m, b}) = 4.** The baskets containing both milk and beer are B_1 = {m, c, b}, B_3 = {m, b}, B_5 = {m, p, b} and B_6 = {m, c, b, j}. B_2 has milk but no beer, and B_7, B_8 have beer but no milk, so they do not count (all items of I must be present). *Why:* this is the **denominator** of confidence: "in how many baskets does the 'if' part happen?".
4. **Count the "if and then" part: support(I ∪ {c}) = support({m, b, c}) = 2.** Look only at the four baskets of step 3 and keep those that also contain coke: B_1 and B_6. B_3 and B_5 have no coke. *Why:* this is the **numerator**: "in how many of those baskets did the 'then' part happen too?".
5. **(a) Confidence.** conf({m, b} → c) = support({m, b, c}) ÷ support({m, b}) = 2/4 = 1/2 = **0.5**. *Meaning:* among the baskets that already contain milk and beer, exactly half also contain coke.
6. **Count the baseline: Pr[c] = 5/8 = 0.625.** Count the baskets containing coke among *all eight* baskets, ignoring milk and beer: B_1, B_4, B_6, B_7, B_8 = 5 (B_2, B_3, B_5 have none). So Pr[c] = 5/8 = 0.625. *Why:* interest compares the rule with "how often does coke appear anyway, whatever else is in the basket?". This is the "Item c appears in 5/8 of the baskets" line of Slide 15. Note that here we count over **every** basket, not just the four that contain {m, b}.
7. **(b) Interest.** Interest({m, b} → c) = conf − Pr[c] = 1/2 − 5/8. Use the common denominator 8: 4/8 − 5/8 = **−1/8 = −0.125**. *Meaning:* knowing that the basket has milk and beer moves the chance of coke from 62.5% down to 50%, a change of −12.5 percentage points.
8. **(c) Decide.** Slide 11 says interesting rules have a high interest value, usually above 0.5. Here the interest is −0.125, far below 0.5 (and even its size, |−0.125| = 0.125, is below 0.5), so the rule **{m, b} → c is not interesting**. *Reason in words:* coke is fairly common (in 5 of the 8 baskets, 62.5%), so a confidence of

0.5 is no better than what we would get without looking at the left side; the rule does even slightly worse than the baseline. This agrees with the slide's "Rule is not very interesting!".

9. **Side note: the support of the rule (Slide 16).** Slide 16 defines the support of an association rule as the support of the set of items on the *left* side, so here it is support({m, b}) = 4. That is at least the threshold s = 3 of Slide 9, so the rule passes the support filter; it is the *interest* test that rejects it. *Beyond the slides:* many other books define a rule's support as support(I ∪ {j}) (here 2). In this course, use the Slide 16 convention.

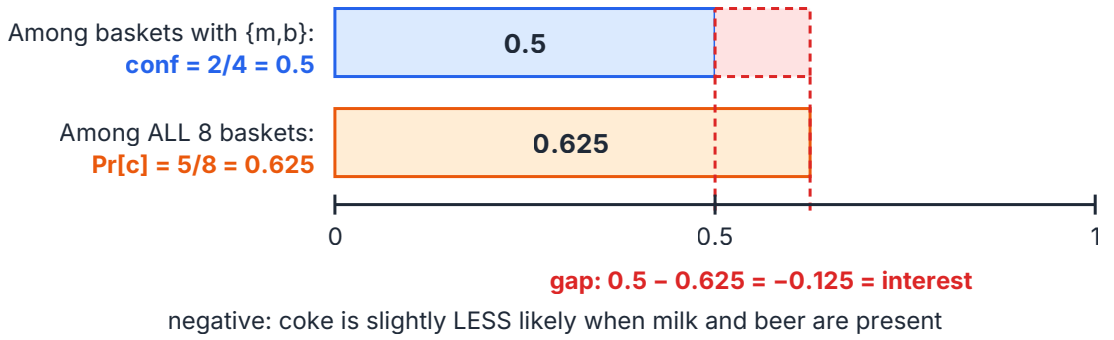


Figure L4-Q2-2. Interest is the gap between the two bars: the rule's confidence minus the baseline frequency of the right-hand item. The bars are drawn on a 0-to-1 scale.

SLIDE ERRATUM (MINOR OMISSION, NO ANSWER CHANGES)

Slide 15 stops at "Interest = 1/2 − 5/8 < 0.5" and never evaluates the subtraction. The value is −1/8 = −0.125, and it is *negative* (the inequality on the slide is true, so the conclusion stands). Slide 11 also says interesting rules have "high interest values (usually above 0.5)" without saying how large *negative* values are treated. *Beyond the slides:* in the textbook (*Mining of Massive Datasets*, Section 6.1.3) a rule is interesting when its interest is large positive *or* large negative, i.e. when the absolute value is large; a strongly negative interest means the left side makes the right side much *less* likely. For this rule |−0.125| = 0.125 < 0.5, so the verdict is the same under both readings.

What each number means in plain English

Quantity	Value	Plain-English meaning
support({m,b})	4	4 of the 8 baskets contain both milk and beer: the "if" part occurs in half of the baskets.
support({m,b,c})	2	2 baskets contain milk, beer *and* coke together.
conf({m,b} → c)	2/4 = 0.5	"If a basket has milk and beer, it also has coke" is true for 50% of such baskets, i.e. the rule is right half of the time.
Pr[c]	5/8 = 0.625	Coke is in 62.5% of *all* baskets: the chance that a random basket has coke, with no other information.
Interest	−1/8 = −0.125	Confidence minus baseline. Zero would mean "the left side tells us nothing about coke"; a large positive value would mean the left side strongly boosts coke. −0.125 is a small *drop* in the chance of coke: the rule is not a useful signal.

KEY IDEA — A POPULAR RIGHT-HAND ITEM CAPS THE INTEREST

Confidence can be at most 1, so Interest ≤ 1 − Pr[j]. For j = c this bound is 1 − 5/8 = 3/8 = 0.375, already below 0.5, so *no* rule ending in `c` could pass the "above 0.5" rule of thumb on these baskets. Our own exhaustive check (not on the slides): of the 60 rules with 1 to 4 items on the left, one item on the right, and a left side that occurs in at least one basket, the largest interest is 3/8 (for example {b, j} → c, whose confidence is 1), so on this tiny data set no rule reaches 0.5. The 0.5 of Slide 11 is only a rule of thumb meant for large data.

COMMON MISTAKES

- **Wrong denominator for confidence.** Divide by support(I) = 4 (baskets with the *left* side), not by 8 (all baskets). 2/8 = 0.25 is wrong.
- **Wrong Pr[c].** Pr[c] = 5/8 counts *every* basket with coke. It is not 2/8, and it is not computed only inside the four {m, b} baskets (that would just give the confidence again, and the interest would always be 0).
- **Reading I ∪ {j} as "or".** It is the itemset {m, b, c}: a basket must contain all three items, so support({m, b, c}) = 2, not the number of baskets containing m, b or c.
- **Direction matters.** conf({m, b} → c) = 2/4 = 0.5, but conf(c → {m, b}) = support({m,b,c}) ÷ support({c}) = 2/5 = 0.4. The denominator is always the support of the *left* side.
- **Do not call a rule interesting just because the confidence looks fairly high.** Always subtract Pr[j] and compare with the threshold.
- **Keep the sign.** The result is −1/8, not +1/8. A negative interest says the left side makes the right side *less* likely than normal.

EXAM TIP

Write the answer in this order so that every step earns marks: (1) both formulas, (2) support(I), (3) support(I ∪ {j}), (4) conf = (3) ÷ (2), (5) Pr[j], (6) interest = conf − Pr[j], (7) one sentence of conclusion that mentions the 0.5 threshold and *why* the rule is or is not interesting. List the basket numbers you counted, so a counting slip can still earn the method marks. Writing the slide's own line "Interest = 1/2 − 5/8 < 0.5" is acceptable, but evaluating it to −1/8 is better.

PRACTICE (SELF-MADE, NOT FROM THE COURSE): COVER THE ANSWERS AND TRY FIRST

Use the same eight baskets. For each rule compute the confidence, Pr[right side] and the interest, and say whether the rule is interesting (threshold 0.5).

Step	Practice 1: `{m} → b`	Practice 2: `{j} → c`
support of left side	m is in B_1, B_2, B_3, B_5, B_6 → 5	j is in B_2, B_4, B_6, B_7 → 4
support of left side + right item	{m, b} in B_1, B_3, B_5, B_6 → 4	{j, c} in B_4, B_6, B_7 → 3
confidence	4/5 = 0.8	3/4 = 0.75
Pr[right item]	b is in B_1, B_3, B_5, B_6, B_7, B_8 → 6/8 = 3/4	c is in B_1, B_4, B_6, B_7, B_8 → 5/8
interest	4/5 − 3/4 = 16/20 − 15/20 = 1/20 = 0.05	3/4 − 5/8 = 6/8 − 5/8 = 1/8 = 0.125
verdict	Not interesting: high confidence, but beer is in 3/4 of all baskets anyway (the situation described on Slide 11, with beer in the role of milk).	Not interesting: slightly positive (juice buyers are a bit more likely than average to buy coke), but far below 0.5.

Final answer: (a) confidence = 2/4 = 1/2 = 0.5, from support({m, b}) = 4 and support({m, b, c}) = 2. (b) Pr[c] = 5/8 = 0.625, so interest = 1/2 − 5/8 = −1/8 = −0.125. (c) −0.125 is far below 0.5 (and |−0.125| < 0.5 too), so the rule {m, b} → c is NOT interesting: coke is simply a very common item.

Where the marks come from: (1) correct confidence formula support(I ∪ {j}) ÷ support(I); (2) support({m, b}) = 4 with the right baskets (B_1, B_3, B_5, B_6); (3) support({m, b, c}) = 2 (B_1, B_6); (4) conf = 2/4 = 1/2; (5) correct interest formula conf − Pr[j]; (6) Pr[c] = 5/8 (c is in B_1, B_4, B_6, B_7, B_8); (7) interest = 1/2 − 5/8 = −1/8 with the sign kept; (8) the conclusion "not interesting" with the reason (well below 0.5, because c appears in 5/8 of all baskets). The exact split of marks on a real exam may differ.

Slide check: matches the slide's answer. Slide 13 gives confidence = 2/4 = 1/2 and Slide 15 gives confidence = 2/4 = 0.5 (same value, different notation), "Interest = 1/2 − 5/8 < 0.5", "Item c appears in 5/8 of the baskets" and "Rule is not very interesting!". Every count and fraction here was re-computed by program over the eight baskets: support({m,b}) = 4, support({m,b,c}) = 2, conf = 1/2, Pr[c] = 5/8, interest = −1/8. Slide erratum (omission only):

Slide 15 never evaluates the interest, which is −1/8 = −0.125 (negative); see the erratum box above. The slide's conclusion is unchanged.

The mining problem and the two-step recipe

Lecture 4 · Slides 16–17

The problem statement (Slide 16)

Lecture 4 · Slide 16

PROBLEM (SLIDE 16)

Find all association rules with support ≥ s and confidence ≥ c.

Two thresholds are chosen by the analyst: s (a count of baskets: how common must the pattern be?) and c (a number between 0 and 1: how reliable must the if-then be?). The rules that pass both tests are the output.

The slide's **Note**: "Support of an association rule is the support of the set of items on the left side." The slide's second bullet gives the focus of the whole lecture: **"Hard part: finding the frequent itemsets!"**

SLIDE ERRATUM / FLAG: SUPPORT OF A RULE

The slide's note is non-standard. Beyond the slides: the usual definition (Agrawal et al.; the textbook) takes the support of a rule $A \to B$ as support($A \cup B$), the *whole* itemset, which is also what Slides 17–18 effectively do (they start from frequent itemsets I = left ∪ right). The two definitions can give different rule sets in general; on the Slide 18 data with $s = 3$ and $c = 0.75$ they happen to give the same six rules. **If an exam asks "what is the support of a rule on these slides", quote the slide (support of the left side); when you compute rules, follow Slides 17–18: find frequent itemsets first.**

COMMON MISTAKE: TWO MEANINGS OF THE LETTER C

On Slides 16 and 18, c is the **confidence threshold**, but "c" is also the item **coke** in the baskets, and "c=4/6" on Slide 18 means "confidence = 4/6". Read each c from its context. This guide writes the item as `c` inside baskets and rules and the threshold as $c = 0.75$ with its value.

Step 1 and Step 2 (Slide 17)

Lecture 4 · Slide 17

Slide 17 splits the problem into two steps:

1. **Step 1: find all frequent itemsets I** ("we will explain this next": the rest of Lecture 4).
2. **Step 2: rule generation.** For every subset A of I, generate the rule $A \to I \setminus A$ (where "\" means "set minus": everything in I except A). Compute its confidence and **output the rules above the confidence threshold**.

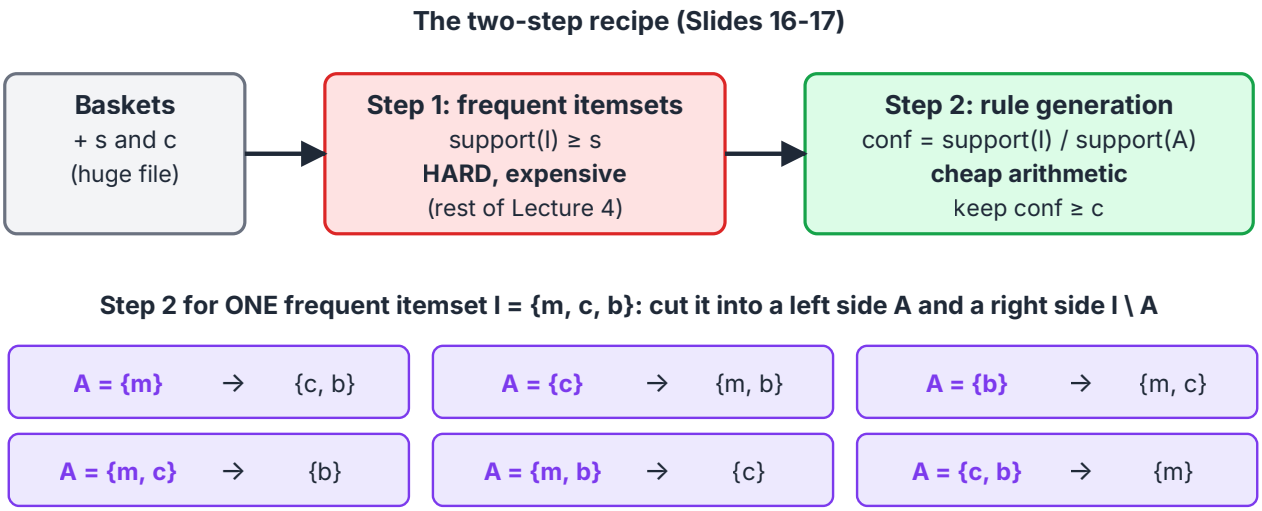


Figure L4p1-5. Step 1 is the bottleneck; Step 2 only divides counts that Step 1 already produced.

FORMULA (SLIDE 17)

$$\text{confidence}(A, B \rightarrow C, D) = \frac{\text{support}(A, B, C, D)}{\text{support}(A, B)}$$

General form: conf(A → I \ A) = support(I) / support(A). Here A, B, C, D are individual items: the left side is {A, B}, the right side {C, D}, and their union is the frequent itemset I.

The two sub-bullets under Step 2 explain why this step is cheap:

- **"Since I is frequent, A is also frequent."** Every basket that contains I also contains A, so support(A) ≥ support(I) ≥ s. This is monotonicity again. It means the denominator support(A) was already counted in Step 1 (when all frequent itemsets and their counts were found).
- **"Single pass to compute the rule confidence."** Our reading: at most one extra pass over the data is needed to fetch any count that is missing; no repeated searching is required. In the worked example (Quiz L4-Q3) every needed count is already in the Step 1 table, so no extra pass is needed.

How many rules to try? From a frequent itemset of size k there are $\mathbf{2^k - 2}$ candidate rules (every non-empty proper subset can be the left side): 2 rules for a pair (A → B and B → A), 6 for a triple, 14 for a set of 4.

SLIDE ERRATUM

(1) "For every subset A of I" should read "every **non-empty proper** subset": A = ø or A = I gives meaningless rules. (2) The letters A, B, C, D in the formula are items, while A in "$A \rightarrow I \setminus A$" is a set: do not mix them. (3) The right side may now hold several items, a generalisation of Slide 10's single j.

BEYOND THE SLIDES: PRUNING IDEA

The Stanford CS246 slides this lecture is adapted from add a shortcut that this slide omits: if A,B,C → D fails the confidence test, then so does A,B → C,D. The numerator support(A,B,C,D) is the same, but the denominator support(A,B) can only be as large or larger than support(A,B,C), so the confidence can only drop. You can see it on Slide 18: b,c → m has 3/5, and moving c to the right gives b → c,m with 3/6, which is lower.

WHY THE HARD PART IS STEP 1

Step 2 divides numbers that Step 1 already produced. Step 1 must count how often *every candidate itemset* occurs in a huge file, and the number of candidate itemsets grows explosively with basket size. That is why A-Priori, PCY and SON exist.

Worked example: generating the rules

Lecture 4 · Slide 18

Slide 18 runs the whole recipe by hand. **Support threshold $s = 3$ and confidence threshold $c = 0.75$.** The baskets are almost those of Slide 9, but **B_3 is different**:

Basket	Items	Basket	Items
B_1	{m, c, b}	B_2	{m, p, j}
B_3	{m, c, b, n}	B_4	{c, j}
B_5	{m, p, b}	B_6	{m, c, b, j}
B_7	{c, b, j}	B_8	{b, c}

SLIDE ERRATUM / FLAG

B_3 is {m, c, b, n} here but {m, b} on Slides 9 and 12–15, and the item n (not on Slide 9's item list) appears only once. The change is intentional but unannounced: with the old B_3 the itemsets {c,m} and {m,c,b} would have support 2 and fail the threshold. Do not mix the two data sets.

Step 1: frequent itemsets

Lecture 4 · Slide 18

The slide lists **{b,m}, {b,c}, {c,m}, {c,j}, {m,c,b}**. These are the frequent itemsets of size 2 or more (single items are frequent too, but a rule needs at least two items). Their supports:

Itemset	Baskets	Support
{b}, {c}, {m}, {j} (singles)	—	6, 6, 5, 4
{b,c}	B1, B3, B6, B7, B8	5
{b,m}	B1, B3, B5, B6	4
{c,m}	B1, B3, B6	3
{c,j}	B4, B6, B7	3
{m,c,b}	B1, B3, B6	3

Step 2: generate and test the rules

Lecture 4 · Slide 18

For each rule, confidence = support(I) ÷ support(left side). The slide shows these (struck-out rules fail 0.75):

Rule	Calculation	Confidence	Verdict
b → m	support{b,m} / support{b} = 4/6	0.667	struck out (< 0.75)
m → b	4/5	0.8	**kept**
b → c	support{b,c} / support{b} = 5/6	0.833	**kept**
b,c → m	support{m,c,b} / support{b,c} = 3/5	0.6	struck out
b,m → c	support{m,c,b} / support{b,m} = 3/4	0.75	**kept (equal to 0.75 passes)**
b → c,m	support{m,c,b} / support{b} = 3/6	0.5	struck out

The slide's "..." hides the other candidates. All 14 candidate rules (2 from each of the 4 pairs, plus 6 from the triple) are below; the slide's six are included.

Itemset	Rule	Confidence	≥ 0.75?
{b,m}	b → m	4/6 = 0.667	no
	m → b	4/5 = 0.8	**yes**
{b,c}	b → c	5/6 = 0.833	**yes**
	c → b	5/6 = 0.833	**yes (hidden by "...")**
{c,m}	c → m	3/6 = 0.5	no
	m → c	3/5 = 0.6	no
{c,j}	c → j	3/6 = 0.5	no
	j → c	3/4 = 0.75	**yes (hidden by "...")**
{m,c,b}	b,m → c	3/4 = 0.75	**yes**
	c,m → b	3/3 = 1.0	**yes (hidden by "...")**
	b,c → m	3/5 = 0.6	no
	m → b,c	3/5 = 0.6	no
	b → c,m	3/6 = 0.5	no
	c → b,m	3/6 = 0.5	no

Slide 18: the six rules cut from {m, c, b} (support 3)

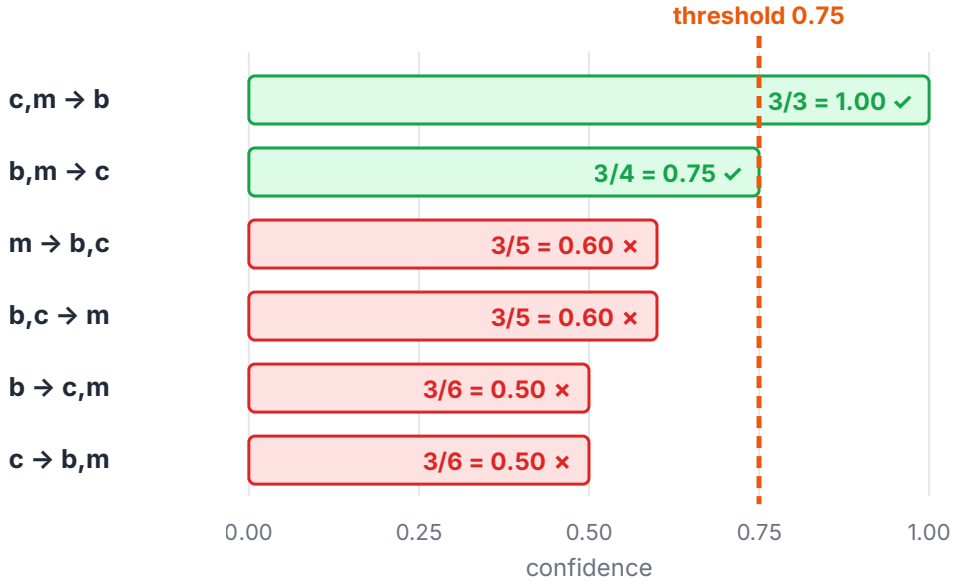


Figure L4p1-6. Four of the six rules fall below the dashed line. Slide 18 strikes out two of them (b,c → m and b → c,m) and hides the others behind the "...".

FINAL ANSWER FOR THE SLIDE 18 DATA

Exactly **six** rules pass: b → c, c → b, m → b, b,m → c, c,m → b, j → c. Slide 18 shows only three of them as kept (b → c, m → b, b,m → c) and hides the other three behind "...".

SLIDE ERRATUM / WHAT THE SLIDE LEAVES OUT

(1) The "..." hides three accepted rules (c → b, c,m → b, j → c), so do not copy the slide's list as the complete answer to "find *all* rules". (2) The three red strike-throughs are the only sign that a rule failed; the slide never says why (confidence below 0.75). (3) The frequent-itemset list omits single items, which are frequent here too, because rules need at least two items. (4) "c=4/6" uses c for confidence while "b→c" uses c for coke.

EXAM TIP

Organise rule generation by itemset: for each frequent set of size ≥ 2, list *every* split into left and right side, write support(I) / support(left) with the actual numbers, then compare with the threshold using "≥". Check the boundary case b,m → c: 3/4 = 0.75 passes. Quiz L4-Q3 gives the full, step-by-step version.

COMMON MISTAKES

- Dividing by support(I) – or by the right side's support – instead of by the support of the **left** side.
- Forgetting the reverse direction: each pair gives **two** rules, a triple gives **six**.
- Rejecting a rule with confidence exactly equal to the threshold (the condition is ≥).
- Skipping single-item left sides such as b → c,m (the left side may be one item with a two-item right side).

Try it yourself first: for the Slide 18 baskets with s = 3 and c = 0.75, find all frequent itemsets and all rules that pass, before opening the solution.

Quiz L4-Q3 **Generating association rules from frequent itemsets** Lecture 4 · Slides 16–18

QUESTION (AS ON THE SLIDES)

Slide 18 — Example. Eight baskets over the items *m* (milk), *c* (coke), *p* (pepsi), *b* (beer), *j* (juice) and one extra item *n* that occurs only once. (Careful: this is *not* exactly the data of Slides 9–15 — here B_3 = `{m, c, b, n}`.)

B_1	{m, c, b}	B_2	{m, p, j}
B_3	{m, c, b, n}	B_4	{c, j}
B_5	{m, p, b}	B_6	{m, c, b, j}
B_7	{c, b, j}	B_8	{b, c}

Support threshold **s = 3** baskets; confidence threshold **c = 0.75**. Task (Slides 16–17): find all association rules with support ≥ *s* and confidence ≥ *c* — Step 1: frequent itemsets; Step 2: for every subset *A* of a frequent itemset *I* generate the rule $A \to I \setminus A$ and output those above the confidence threshold.

(a) Step 1. Find **all** frequent itemsets and give their support counts.

(b) Step 2. For **every** frequent itemset *I* of size ≥ 2, generate **every** rule $A \to I \setminus A$ and compute its confidence.

(c) List the rules whose confidence is ≥ 0.75.

(d) Explain why every subset *A* of a frequent itemset *I* is itself frequent, and why the confidences therefore need only supports that were already computed in Step 1.

The parts (a)–(d) are our split of Slides 16–18. The slide itself prints only the start of the answer: its frequent itemsets of size ≥ 2 and six of the rules, the last line being "…". The complete answer is worked out below.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Picture a supermarket. **Step 1** asks: which groups of products are bought together often enough to matter (here: in at least 3 of the 8 baskets)? **Step 2** takes each such group, say {beer, coke}, and asks the question in *both* directions: "of the baskets that contain beer, what fraction also contain coke?" and "of the baskets that contain coke, what fraction also contain beer?". Each answer is a fraction, the **confidence**. We keep a rule only if its fraction is at least 3/4. The expensive part is Step 1 (Slide 16: "Hard part: finding the frequent itemsets!"); once it is done, Step 2 is just dividing numbers we already have.

FORMULAS USED (LECTURE 4 · SLIDES 8, 10, 16, 17)

Support of an itemset S: support(S) = number of baskets that contain *every* item of S (Slide 8). **Frequent** means support ≥ s = 3.

Confidence of a rule $A \to B$ (Slides 10 and 17): $\text{conf}(A \to B) = \frac{\text{support}(A \cup B)}{\text{support}(A)}$. For a rule cut out of a frequent itemset I we have $B = I \setminus A$, so $A \cup B = I$ and **$\text{conf}(A \to I \setminus A) = \frac{\text{support}(I)}{\text{support}(A)}$**. The **left side** goes in the denominator. Keep the rule if conf ≥ 0.75 ("≥": exactly 0.75 passes).

Number of rules from an itemset of size k: $2^k - 2$ (every non-empty, proper subset A of I is a left side).

1. **Plan: the two-step recipe of Slide 17.** Step 1 = find all frequent itemsets I, counting level by level (single items, then pairs, then triples ...). Step 2 = for each such I cut it into a left side A and a right side $I \setminus A$, compute the confidence and keep the rule if it is ≥ 0.75. *Why this order:* first find which item sets occur often (support), then ask how reliable each "if ... then ..." is (confidence). *Notation warning:* on Slide 18 the letter *c* means both the item "coke" and the confidence threshold; here the item is written `c` and the threshold "0.75".
2. **Part (a), Step 1a — count every single item.** Tick each item in each basket (✓), then add up the row. We need these counts twice: to find the frequent items and, later, as denominators of confidences. *Why:* an item that is in fewer than 3 baskets can never be part of a frequent set (justified in the part (d) step below), so we can ignore it from now on.

Item	B_1	B_2	B_3	B_4	B_5	B_6	B_7	B_8	Support	≥ 3 ?
`m (milk)`	✓	✓	✓	–	✓	✓	–	–	**5**	**frequent**
`c (coke)`	✓	–	✓	✓	–	✓	✓	✓	**6**	**frequent**
`b (beer)`	✓	–	✓	–	✓	✓	✓	✓	**6**	**frequent**
`j (juice)`	–	✓	–	✓	–	✓	✓	–	**4**	**frequent**
`p (pepsi)`	–	✓	–	–	✓	–	–	–	**2**	not frequent (2 < 3)
`n (extra item)`	–	–	✓	–	–	–	–	–	**1**	not frequent (1 < 3)

   Frequent items: `b` (6), `c` (6), `m` (5), `j` (4). The items `p` (2) and `n` (1) are dropped.
3. **Part (a), Step 1b — count the pairs.** By monotonicity (Slide 30: if a set appears at least *s* times, so does every subset of it) a pair can be frequent only if *both* its items are frequent. So only the 4 frequent items need to be paired: C(4,2) = 6 pairs. (The other 9 pairs contain `n` or `p` and have support at most 2 — e.g. `{m, p}` is only in B_2 and B_5 — so they are infrequent anyway.)

Pair	Baskets that contain both items	Support	≥ 3 ?
`{b,c}`	B_1, B_3, B_6, B_7, B_8	**5**	**frequent**
`{b,m}`	B_1, B_3, B_5, B_6	**4**	**frequent**
`{c,j}`	B_4, B_6, B_7	**3**	**frequent**
`{c,m}`	B_1, B_3, B_6	**3**	**frequent**
`{b,j}`	B_6, B_7	**2**	not frequent (2 < 3)
`{j,m}`	B_2, B_6	**2**	not frequent (2 < 3)

   Frequent pairs: `{b,c}` (5), `{b,m}` (4), `{c,j}` (3), `{c,m}` (3). Note that `{c,j}` and `{c,m}` sit exactly on the threshold, and "at least *s*" includes them.
4. **Part (a), Step 1c — triples, then bigger sets.** A triple can be frequent only if all three of its pairs are frequent (same monotonicity argument). The infrequent pairs `{b,j}` and `{j,m}` kill every triple that contains `j` together with `b` or `m`:

Triple	Are all three of its pairs frequent?	Support	Result
{b,c,m}	{b,c} ✓; {b,m} ✓; {c,m} ✓	**3 (B_1, B_3, B_6)**	**✓ frequent**
{b,c,j}	{b,c} ✓; {b,j} ×; {c,j} ✓	no need to count (a check gives 2)	× cannot be frequent
{b,j,m}	{b,j} ×; {b,m} ✓; {j,m} ×	no need to count (a check gives 1)	× cannot be frequent
{c,j,m}	{c,j} ✓; {c,m} ✓; {j,m} ×	no need to count (a check gives 1)	× cannot be frequent

So only {b,c,m} survives, and it is contained in B_1, B_3, B_6: support 3 ≥ 3, frequent. A 4-set can be frequent only if all four of its triples are frequent, but there is just one frequent triple; directly, only B_6 contains {b,c,j,m} (support 1). So the search stops: **no frequent itemset has size 4**. (Beyond the slides: this "count only what could still be frequent" idea is the A-Priori algorithm that Slides 30–31 explain later; with 8 baskets we simply did it by hand.)

5. **Part (a) result — all frequent itemsets with their counts.** There are 9 of them. Slide 18 lists only the five of size ≥ 2 — {b,m} {b,c} {c,m} {c,j} {m,c,b} (the slide writes the triple as {m,c,b}; it is the same set as our {b,c,m}) — because only these can produce rules (a rule needs at least one item on each side). **Those five match our table exactly.** We keep the single-item counts, because they are the denominators in Step 2.

Size	Frequent itemset	Support (count)	In baskets	On Slide 18?
1	{b}	**6**	B_1, B_3, B_5, B_6, B_7, B_8	not listed (no rule can come from one item)
1	{c}	**6**	B_1, B_3, B_4, B_6, B_7, B_8	not listed (no rule can come from one item)
1	{j}	**4**	B_2, B_4, B_6, B_7	not listed (no rule can come from one item)
1	{m}	**5**	B_1, B_2, B_3, B_5, B_6	not listed (no rule can come from one item)
2	{b,c}	**5**	B_1, B_3, B_6, B_7, B_8	yes
2	{b,m}	**4**	B_1, B_3, B_5, B_6	yes
2	{c,j}	**3**	B_4, B_6, B_7	yes
2	{c,m}	**3**	B_1, B_3, B_6	yes
3	{b,c,m}	**3**	B_1, B_3, B_6	yes

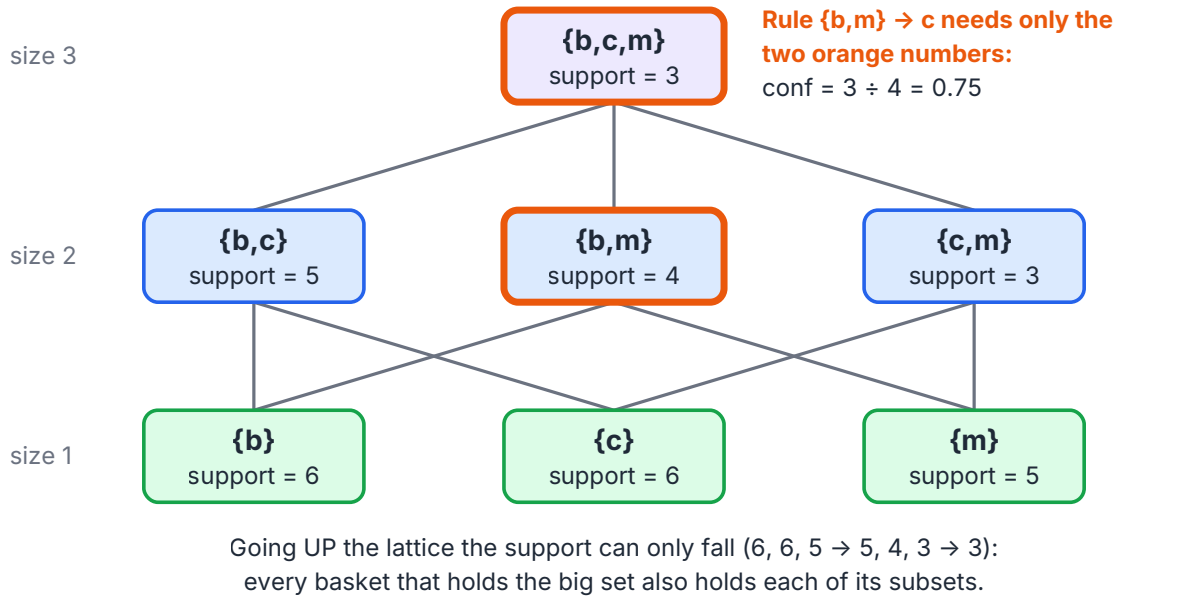


Figure L4-Q3-1 — The frequent subsets of {b, c, m} arranged by size, with their supports. Every set sits below the bigger sets that contain it and its support is never smaller than theirs. A rule's confidence is "support of the big set ÷ support of the left-hand subset", i.e. a ratio of two numbers on this picture.

6. **Part (d) — why every subset of a frequent itemset is frequent (placed here because Step 2 relies on it).** Let I be frequent, so support(I) ≥ s, and let $A \subseteq I$. A basket that contains *all* items of I automatically

contains all items of A, because the items of A are among those of I. Hence the set of baskets that contain I is a *subset* of the set of baskets that contain A, and therefore

support(A) ≥ support(I) ≥ s

so A is frequent too (Slide 17: "Since I is frequent, A is also frequent"; Slide 30: monotonicity). *Concrete check:* `{b,c,m}` is in B_1, B_3, B_6; its subset `{b,m}` is in B_1, B_3, B_5, B_6 — all three baskets reappear, plus B_5 — so 4 ≥ 3. (The same argument read backwards says: an item or set with support below *s* can never be inside a frequent set, which is why `p` and `n` could be dropped.)

Consequence for Step 2 (no new counting). Step 1 produces a table of *every* frequent itemset *with its count* (the table above). For a rule $A \to I \setminus A$ from a frequent I, the numerator support(I) is in the table because I is frequent, and the denominator support(A) is in the table because A is frequent by the argument above. So computing a confidence is one table look-up and one division (Slide 17 calls this a "single pass"). Two bonus facts: support(A) ≥ support(I) means every confidence is ≤ 1, and support(A) ≥ s ≥ 1 means we never divide by zero. Also, the rule's own support (= support of its left side, Slide 16) is support(A) ≥ 3, so **every** rule cut from a frequent itemset already meets the support requirement; only the confidence test can reject a rule.

7. **Part (b), Step 2a — list every rule $A \to I \setminus A$ (do not skip any).** An itemset of size k gives $2^k - 2$ rules: every subset A *except* the empty set (a rule with nothing on the left says nothing) and I itself (nothing left to predict). The single items give no rules. Count: four pairs × 2 rules + one triple × 6 rules = 8 + 6 = **14 rules**. A pair `{x, y}` gives `x → y` and `y → x` (two different rules with different denominators). The triple `{b,c,m}` gives three rules with one item on the left (`b → c,m`, `c → b,m`, `m → b,c`) and three with two items on the left (`b,c → m`, `b,m → c`, `c,m → b`).
8. **Part (b), Step 2b — compute all 14 confidences** with conf = support(I) ÷ support(A), taking the numbers from the table of part (a), and compare each with 0.75 (equal also passes).

Itemset I	#	Rule $A \to I \setminus A$	support(A)	conf = support(I) / support(A)	≥ 0.75 ?	On Slide 18?
`{b,m}` support 4	1	`b → m`	6	$\frac{4}{6} \approx 0.667$	× reject	shown, crossed out
	2	`m → b`	5	**$\frac{4}{5} = 0.8$**	**✓ keep**	shown
`{b,c}` support 5	3	`b → c`	6	**$\frac{5}{6} \approx 0.833$**	**✓ keep**	shown
	4	`c → b`	6	**$\frac{5}{6} \approx 0.833$**	**✓ keep**	hidden in the "…"
`{c,m}` support 3	5	`c → m`	6	$\frac{3}{6} = 0.5$	× reject	hidden in the "…"
	6	`m → c`	5	$\frac{3}{5} = 0.6$	× reject	hidden in the "…"
`{c,j}` support 3	7	`c → j`	6	$\frac{3}{6} = 0.5$	× reject	hidden in the "…"
	8	`j → c`	4	**$\frac{3}{4} = 0.75$**	**✓ keep**	hidden in the "…"
`{b,c,m}` support 3	9	`b → c,m`	6	$\frac{3}{6} = 0.5$	× reject	shown, crossed out
	10	`c → b,m`	6	$\frac{3}{6} = 0.5$	× reject	hidden in the "…"
	11	`m → b,c`	5	$\frac{3}{5} = 0.6$	× reject	hidden in the "…"
	12	`b,c → m`	5	$\frac{3}{5} = 0.6$	× reject	shown, crossed out
	13	`b,m → c`	4	**$\frac{3}{4} = 0.75$**	**✓ keep**	shown
	14	`c,m → b`	3	**$\frac{3}{3} = 1$**	**✓ keep**	hidden in the "…"

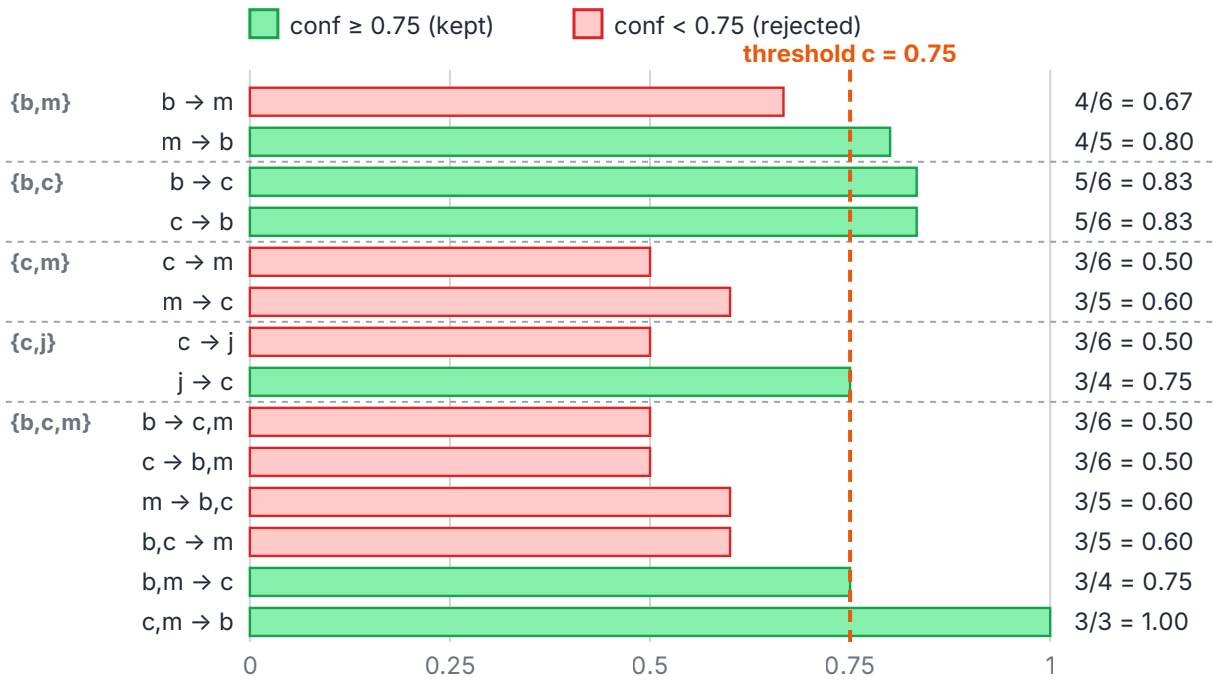


Figure L4-Q3-2 — Confidence of all 14 candidate rules (each bar is support(*I*)/support(*A*)); only the bars that reach the dashed 0.75 line survive. The bars for `j → c` and `b,m → c` end exactly on the line and are kept because the test is "≥".

9. **Part (c) — output the rules above the confidence threshold.** Keep only the rows marked ✓ (compare each fraction with 3/4):

Rule	support(I) / support(A)	Confidence	From itemset
`b → c`	5/6	≈ 0.833	`{b,c}`
`c → b`	5/6	≈ 0.833	`{b,c}`
`m → b`	4/5	= 0.8	`{b,m}`
`j → c`	3/4	= 0.75	`{c,j}`
`b,m → c`	3/4	= 0.75	`{b,c,m}`
`c,m → b`	3/3	= 1	`{b,c,m}`

Six rules survive. The other 8 fail, with confidences 4/6 (`b → m`), 3/6 (`c → j`), 3/6 (`c → m`), 3/5 (`m → c`), 3/6 (`b → c,m`), 3/6 (`c → b,m`), 3/5 (`m → b,c`), 3/5 (`b,c → m`) — all below 0.75. Their left-side supports (6, 6, 5, 4, 4, 3) are all ≥ 3, so each also meets the support requirement of Slide 16. (A brute-force check over *all* possible rules of this data set, including ones cut from infrequent itemsets, also gives exactly these six.)

10. **Cross-check with a shortcut (Beyond the slides).** For a fixed itemset I the numerator support(I) never changes, so a rule passes exactly when support(A) ≤ support(I) ÷ 0.75:

Itemset I	support(I)	Largest allowed support(A) = support(I) ÷ 0.75	Left sides that pass (their support)
`{b,m}`	4	4 ÷ 0.75 = 5.33	`{m}` (5)
`{b,c}`	5	5 ÷ 0.75 = 6.67	`{b}` (6), `{c}` (6)
`{c,m}`	3	3 ÷ 0.75 = 4.00	none
`{c,j}`	3	3 ÷ 0.75 = 4.00	`{j}` (4)
`{b,c,m}`	3	3 ÷ 0.75 = 4.00	`{b,m}` (4), `{c,m}` (3)

The passing left sides are exactly those of the six rules above. The same idea gives a pruning rule: a *smaller* left side has a larger or equal support, so its confidence can only be lower or equal. Hence if `b,c → m` fails (3/5 = 0.6), then `b → c,m` and `c → b,m` (left sides inside `{b,c}`) must fail too, with no need to test them; conversely, moving items from the right side to the left side can only raise the

confidence (`c → b,m` has 1/2, but `c,m → b` has 3/3 = 1). On an exam still write the fraction for every rule you consider, because each computed confidence is a mark.

COMMON MISTAKE

- **Wrong denominator or direction.** `b → m` and `m → b` share the numerator 4 but have different denominators (support of `b` = 6, of `m` = 5): 4/6 = 0.67 fails, 4/5 = 0.80 passes. Always divide by the support of the *left* side, never by the right side or by 8.
- **Dropping the boundary.** 0.75 ≥ 0.75, so `j → c` and `b,m → c` are kept.
- **Stopping at the slide.** The slide's "…" hides three more valid rules: `c → b` (5/6), `j → c` (3/4) and `c,m → b` (3/3).
- **Using the counts of Slides 9–15.** There B_3 = `{m, b}`, so `{b,m} → c` had confidence 2/4 on Slide 13, and `{c,m}` and `{b,c,m}` had support 2 (not frequent). On Slide 18, B_3 = `{m, c, b, n}`: confidence 3/4, and both sets have support 3.
- **Forgetting cases.** Only itemsets of size ≥ 2 give rules; the left side must be a non-empty proper subset; every such subset (not just single items) must be tried; and do not leave out a rule just because its reverse failed.

EXAM TIP

Layout that earns marks: (i) a table of all frequent itemsets with counts; (ii) for each itemset, write down its $2^k - 2$ rules; (iii) show every confidence as a fraction support(*I*)/support(*A*) *before* the decimal; (iv) state the threshold test explicitly; (v) list the surviving rules at the end. Even if one count is wrong, the method marks remain.

BONUS (NOT ASKED ON SLIDE 18) — ARE THE SIX RULES "INTERESTING"?

Slide 11: Interest(I → j) = conf − Pr[j], where Pr[j] is the fraction of baskets containing j (all six rules have a single item on the right, so the definition applies); interesting rules are usually above 0.5. Passing the confidence filter does not make a rule interesting: `b` and `c` each occur in 6 of the 8 baskets, so even a confidence of 0.75 is no better than the baseline 6/8 = 0.75.

Rule	Confidence	Pr[right item]	Interest = conf − Pr	Above 0.5 ?
`b → c`	5/6 ≈ 0.833	6/8 = 0.75	**0.08** (1/12)	no
`c → b`	5/6 ≈ 0.833	6/8 = 0.75	**0.08** (1/12)	no
`m → b`	4/5 = 0.8	6/8 = 0.75	**0.05** (1/20)	no
`j → c`	3/4 = 0.75	6/8 = 0.75	**0.00** (0)	no
`b,m → c`	3/4 = 0.75	6/8 = 0.75	**0.00** (0)	no
`c,m → b`	3/3 = 1	6/8 = 0.75	**0.25** (1/4)	no

All six interest values are far below 0.5: the confidences are high mostly because `b` and `c` are in 75 % of all baskets anyway.

Final answer —
(a) Frequent itemsets (s = 3): size 1: `{b}`:6, `{c}`:6, `{m}`:5, `{j}`:4; size 2: `{b,c}`:5, `{b,m}`:4, `{c,j}`:3, `{c,m}`:3; size 3: `{b,c,m}`:3; none of size 4 (9 in all).
(b) 14 rules: `{b,m}`: b→m 4/6, m→b 4/5; `{b,c}`: b→c 5/6, c→b 5/6; `{c,m}`: c→m 3/6, m→c 3/5; `{c,j}`: c→j 3/6, j→c 3/4; `{b,c,m}`: b→c,m 3/6, c→b,m 3/6, m→b,c 3/5, b,c→m 3/5, b,m→c 3/4, c,m→b 3/3.
(c) The 6 rules with confidence ≥ 0.75: `b → c` (5/6 ≈ 0.83), `c → b` (5/6 ≈ 0.83), `m → b` (4/5 = 0.80), `j → c` (3/4 = 0.75), `b,m → c` (3/4 = 0.75), `c,m → b` (3/3 = 1). The other 8 fail.
(d) Every basket containing *I* also contains each subset *A*, so support(*A*) ≥ support(*I*) ≥ *s*; hence *A* is frequent, was counted in Step 1, and conf = support(*I*)/support(*A*) uses only stored counts.

Where the marks come from: (1) support counts of the single items, the rule "frequent = support ≥ 3", dropping `n` and `p`; (2) the pair counts and the four frequent pairs (bc 5, bm 4, cj 3, cm 3); (3) the triple `{b,c,m}` = 3 and the statement that nothing larger is frequent; (4) the complete list of frequent itemsets with counts; (5) the formula conf($A \to I \setminus A$) = support(I)/support(A) with the *left* side in the denominator; (6) listing all 14 rules (2 per pair, 6 for the triple, both directions); (7) each correct confidence (the individual fractions are the marks if you slip somewhere); (8) the comparison with 0.75 including the boundary case, and the final list of six rules; (9) the part (d) explanation: baskets containing $I \subseteq$ baskets containing A, so support(A) ≥ support(I) ≥ s, and the confidence reuses stored counts. (The course has published no rubric; this is the natural split.)

SLIDE ERRATUM

No numerical error: every frequent itemset and every rule confidence printed on Slide 18 is correct. Three wording or completeness points: (1) Slide 17 says "for every subset A of I"; read literally this includes A = ∅ and A = I, which are not rules. The correct statement is "every **non-empty proper** subset A of I". (2) Slide 18 lists only the frequent itemsets of size ≥ 2 (singletons are omitted although their counts are needed as denominators) and shows only 6 of the 14 rules, hiding the rest behind "…"; the crossing-out is not explained on the slide (it marks the rules with confidence below 0.75). (3) Slide 16 defines the support of a rule as the support of its left side. Beyond the slides: many textbooks use the support of the whole itemset I instead. Here it makes no difference: a rule with support(A) ≥ 3 and confidence ≥ 0.75 has support(I) ≥ 0.75 × 3 = 2.25, hence ≥ 3 because supports are whole numbers; all six rules also have support(I) ≥ 3.

Slide check: matches the slide's answer. Slide 18's frequent itemsets of size ≥ 2 are exactly ours. Its printed rules `b → m` 4/6, `b → c` 5/6, `b,c → m` 3/5, `m → b` 4/5, `b,m → c` 3/4, `b → c,m` 3/6 all agree with the table; the three crossed out (`b → m`, `b,c → m`, `b → c,m`) are below 0.75. The slide is incomplete rather than wrong: its "…" hides 8 rules, among them three more that qualify (`c → b` 5/6, `j → c` 3/4, `c,m → b` 3/3), so the full answer has 6 rules, not the 3 uncrossed ones visible. See the Slide erratum box above for wording points.

QUICK RECAP OF SLIDES 1–18

- **Model:** items and baskets (small sets drawn from a big catalogue); input is a TID table, output is if-then rules. Applications: store layout, tie-in sales, recommendations, plagiarism (baskets = sentences, items = documents), drug side-effects (baskets = patients).
- **Support** of an itemset = number of baskets containing all its items; **frequent** = support ≥ s.
- **Confidence** of I → j = support(I ∪ {j}) / support(I); it is asymmetric.
- **Interest** = conf − Pr[j]; removes the effect of a popular right-hand item. Example: {m,b} → c has conf 1/2, interest −1/8, not interesting.
- **Problem:** find all rules with support ≥ s and confidence ≥ c. The slide defines rule support as the support of the left side; the textbook uses the whole itemset.
- **Recipe:** Step 1 find frequent itemsets (the hard part); Step 2 for every non-empty proper subset A of each frequent I, test A → I \ A with conf = support(I) / support(A). Monotonicity guarantees support(A) is known.

How frequent itemsets are found at scale: the computation model

Lecture 4 · Slides 19–22

Slide 19 is a black divider titled "Finding Frequent Itemsets" (it has no printed number). Everything before it said *what* we want: itemsets that occur in at least s baskets, and the association rules built from them. From here to slide 28 the lecture asks *how* to find them when the data are huge. Before any algorithm, it fixes three things: where the data live, what a computation costs, and what is scarce.

Where the data live: flat files, basket by basket

Lecture 4 · Slide 20

BEGINNER INTUITION

Picture a very long roll of till receipts. Each receipt (basket) is short, but the roll is kilometres long and sits in a warehouse (the disk), not on your desk. You can only read it from one end to the other. To find which *pairs* of products appear on many receipts, you must walk along the roll, and for each receipt write down every pair of products printed on it.

DEFINITION: FLAT FILE

A plain file on disk with no index and no database structure. Here it stores the baskets one after another ("basket-by-basket"); each basket is a short list of items, and every item is a **positive integer** (Milk = 1, Beer = 2, ... on the slide; the names are only for human readers). Integers matter because they can later be used as array positions (Slides 25–27).

The slide's key points:

- Baskets are **small** (a handful of items), but there are **many baskets** and **many different items**.
- As each basket is read, **expand it into pairs, triples, ...** The slide says to use **nested loops** to generate all sets of size k: two loops for pairs, three for triples, k loops for k-sets.
- Green note: *"We want to find frequent itemsets. To find them, we have to count them. To count them, we have to generate them."* Counting and generating are the same bottleneck.

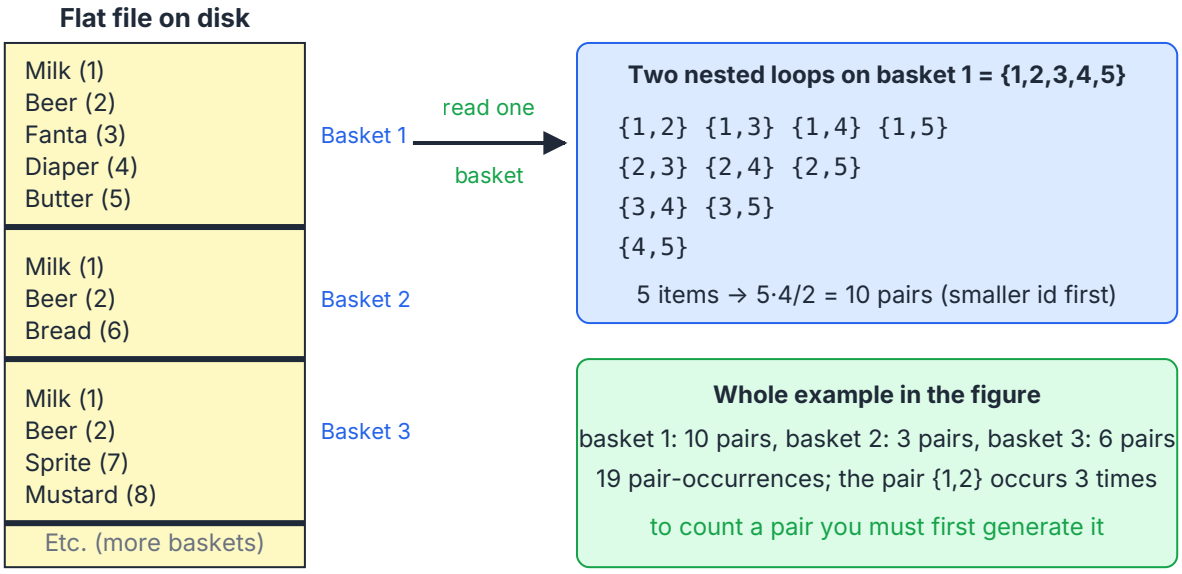


Figure L4p2-1. The data model of Slide 20: one long file, one basket after another, thick lines between baskets; pairs are generated on the fly with nested loops (the counts are our own check of the slide's figure).

Worked example: how many pairs does a scan generate? Lecture 4 · Slide 20

A basket with m items contains $m(m-1)/2$ pairs (choose 2 of m). Basket 1 has 5 items: 5·4/2 = 10. Basket 2 has 3 items: 3·2/2 = 3. Basket 3 has 4 items: 4·3/2 = 6. Total: 10 + 3 + 6 = 19 pair-occurrences. The pair {Milk, Beer} = {1,2} occurs in all three baskets, so its count is 3. Because ids are sorted inside a basket, "smaller id first" generates each pair exactly once.

SLIDE ERRATUM (MINOR)

The green caption under the figure, "Items are positive integers,", ends in a comma and is cut off. Beyond the slides: the original Stanford CS246 slide continues "and boundaries between baskets are −1"; here the thick horizontal lines in the figure play that role.

What a computation costs: passes over the data

Lecture 4 · Slide 21

Because the baskets sit on disk, and disk is far slower than memory, the real cost of mining disk-resident data is usually the **number of disk I/Os** (input/output operations). In practice, frequent-itemset algorithms read the data in **passes**.

> **DEFINITION: PASS**
> One sequential reading of the whole file, all baskets read in turn from first to last. **Cost of an algorithm = number of passes it makes over the data.** Fewer passes means a faster algorithm.

So the yardstick is "how many times do I have to read the file?", not "how many CPU operations?". Beyond the slides: whole-file sequential reads are far cheaper than random access, and CPU work on in-memory counters is cheap compared with disk reads, which is why this simple model works. Preview of what follows: A-Priori reads the file once per itemset size (two passes to get the frequent pairs); PCY also needs two passes for pairs but keeps fewer pair counters; SON needs two passes in total for all sizes.

What is scarce: main memory

Lecture 4 · Slide 22

For many frequent-itemset algorithms, **main memory is the critical resource**. While scanning, you must keep a counter for each itemset you are counting (for example, each pair). All those counters must sit in RAM at the same time, so memory limits how many different things you can count in one pass.

The slide asks: **"Swapping in/out is a disaster (why?)"** and leaves the answer to you.

> **ANSWER: WHY SWAPPING IS A DISASTER**
> If the counters do not fit in RAM, the operating system moves memory pages to and from disk ("swapping" or "paging"). Every basket increments counters at essentially random places, so almost every increment lands on a page that is currently on disk. Each such access costs milliseconds instead of about a hundred nanoseconds (thousands of times slower; this is called thrashing). In the worst case you pay one disk access for every pair occurrence, and the run time explodes. Design rule: count only as many things per pass as fit in memory, even if that costs extra passes.

Why frequent pairs are the hard part

Lecture 4 · Slide 23

Slide 23: "The hardest problem often turns out to be finding the frequent pairs of items $\{i_1, i_2\}$", followed by two "Why?" prompts: *"Freq. pairs are common, freq. triples are rare"* and *"Probability of being frequent drops exponentially with size"*. The slide then says: concentrate on pairs first, then extend to larger sets.

- **Single items** are few and cheap: one counter per item.
- **Pairs** are the first level where the candidate space is huge (about $n^2/2$ possible pairs) *and* many pairs really are frequent, so the counters strain memory.
- **Triples and larger** are much rarer. For a k-set to be frequent, all k items must appear together in at least s baskets, and the more items you demand, the less likely that is.

Worked example: why the probability drops exponentially Lecture 4 · Slide 23

Beyond the slides, a toy model. Suppose each item appears in a random basket with probability $p = 0.01$ (1 %), independently. A pair appears together with probability about $p^2 = 10^{-4}$; a triple about $p^3 = 10^{-6}$; a 4-set about 10^{-8}. Each extra item multiplies the probability by 0.01: exponential decay in the size k (p^k).

COMMON MISTAKE

"Drops exponentially" is a heuristic about typical data, not a theorem. With strongly correlated items (for example, always-bought-together bundles) large frequent itemsets can exist. Also, do not say pairs are hard because they are "complicated": they are hard because there are so many of them to count in memory. The slide gives only the assertion; the memory argument is on Slides 24–27.

Beyond the slides: a triple can only be frequent if all three of its sub-pairs are frequent, so once the frequent pairs are known the triples to count are few. That is the idea of A-Priori (Slide 30).

Counting pairs in memory: the naive algorithm

Lecture 4 · Slides 24–28

The naive algorithm and its memory bill

Lecture 4 · Slide 24

Naive approach: read the file once; keep in main memory one counter for each possible pair; from each basket generate its pairs by two nested loops and add 1 to each pair's counter. With n items there are $n(n-1)/2$ possible pairs ({i,j} is the same as {j,i}, and an item is not paired with itself, hence not n^2).

WHEN THE NAIVE ALGORITHM FAILS

It fails if (#items)^2 exceeds main memory. The slide reminds you that #items can be **100K** (Wal-Mart) or **10B** (Web pages).

Worked example: 10^5 items, 4-byte counters Lecture 4 · Slide 24

1. Pairs: $10^5(10^5-1)/2 = 4{,}999{,}950{,}000 \approx \mathbf{5 \times 10^9}$.
2. Memory: 4 bytes × 4,999,950,000 = 19,999,800,000 bytes ≈ **20 GB** (decimal gigabytes).
3. That is more than a typical machine's RAM, so swapping (Slide 22) would follow.
4. For 10B = 10^{10} items (our calculation): pairs ≈ 5×10^{19}, memory ≈ 2×10^{20} bytes (200 exabytes): hopeless.

Memory grows with the *square* of the number of items: ten times more items means a hundred times more memory. The slide's rule "(#items)^2 exceeds memory" is a rule of thumb; the exact need is $4 \cdot n(n-1)/2 \approx 2n^2$ bytes.

Two ways to store the counters

Lecture 4 · Slides 25–27

	Approach 1: triangular matrix	Approach 2: triples [i, j, c]
What is stored	One counter for **every possible** pair {i,j} with i < j, in a flat array; the position of the cell tells you which pair it is.	One record [i, j, c] ("the count of the pair {i,j} is c") for each pair that **actually occurred** (count > 0).
Bytes	**4 bytes per pair**, for all $n(n-1)/2$ pairs, occurring or not.	About **12 bytes per occurring pair** (three 4-byte numbers); pairs that never occur cost nothing.
Total	$4 \cdot T$, fixed, where $T = n(n-1)/2$.	$12 \cdot O$, where O = number of distinct pairs that occur.
Extra needs	Item ids must be 1..n with no gaps.	A lookup structure (for example a hash table) to find a pair's record quickly.

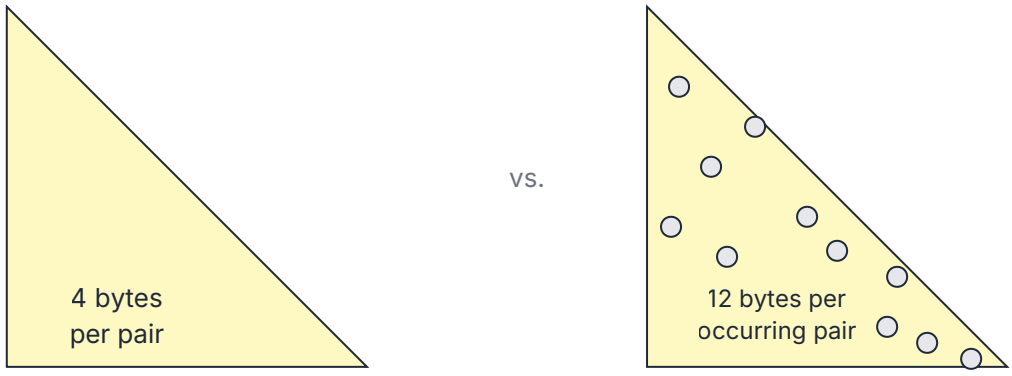


Figure L4p2-2. Slide 26, redrawn. The triangle is the space of all pairs {i,j} with i < j. Left: every cell is stored. Right: only the occupied cells (dots) are stored, each at triple the price.

BREAK-EVEN RULE (SLIDE 27)

Approach 2 is smaller when $12 \cdot O < 4 \cdot T \Leftrightarrow O < T/3$. **Triples beat the triangular matrix only if fewer than 1/3 of the possible pairs actually occur.**

Worked example: the 1/3 rule with the Slide 24 numbers Lecture 4 · Slides 24–27

$T = 4{,}999{,}950{,}000 \approx 5 \times 10^9$; $T/3 \approx 1.67 \times 10^9$. Matrix: always $4 \cdot T \approx 20$ GB. Triples: if 10^9 distinct pairs occur, $12 \times 10^9 = 12$ GB (better); if 3×10^9 occur, 36 GB (worse). Real baskets are small, so usually far fewer than a third of all pairs occur, and triples win, but see Slide 28.

SLIDE ERRATUM (OMISSION): "MATRIX" MEANS TRIANGULAR MATRIX

Slide 25 says only "matrix", but "4 bytes per pair" holds only for a **triangular** matrix (each pair {i,j} stored once, i < j). A full $n \times n$ matrix stores every pair twice plus the diagonal, about 8 bytes per pair. Beyond the slides: with items numbered 1..n, the pair (i, j), i < j, goes to cell $k = (i - 1)(n - i/2) + j - i$ of a one-dimensional array of $n(n-1)/2$ cells. For n = 4: (1,2)→1, (1,4)→3, (2,3)→4, (3,4)→6. The 12 bytes of Approach 2 also ignore the overhead of the lookup structure.

"Can we do better?"

Lecture 4 · Slide 28

Slide 28 states the problem: *"Problem is if we have too many items so the pairs do not fit into memory. Can we do better?"* Both storage schemes need memory proportional to the number of possible (or occurring) pairs, so with 10^5 items or more the one-pass naive algorithm fails. The answer, starting at Slide 29, is A-Priori: it avoids counting most pairs at the price of an extra pass.

QUICK RECAP: SLIDES 19–28

- Data: flat file on disk, basket by basket, items are positive integers; generate pairs/triples with nested loops. To find frequent sets you count them; to count them you generate them.
- Cost = number of passes (disk I/Os). Critical resource = main memory; swapping is a disaster (random increments cause a disk access each).
- Pairs are the bottleneck: frequent pairs are common, frequent triples rare (probability drops exponentially with size).
- Naive: $n(n-1)/2$ counters; 10^5 items need about 5×10^9 counters = 20 GB. Triangular matrix = 4 bytes per possible pair; triples = 12 bytes per occurring pair; triples win only if fewer than 1/3 of the pairs occur.

Try it yourself first: cover the solution and answer the memory questions, the "why?" prompts of Slides 21–23, and the 1/3 break-even.

Quiz L4-Q4 **Memory needed to count pairs: matrix vs triples** Lecture 4 · Slides 20–28

QUESTION (AS ON THE SLIDES)

Slides 20–28 ("Itemsets: Computation Model", "Computation Model", "Main-Memory Bottleneck", "Finding Frequent Pairs", "Naïve Algorithm", "Counting Pairs in Memory", "Comparing the 2 Approaches" and two "Comparing the two approaches" slides) build up one story. They contain one calculation and several "why?" prompts. In the order they appear:

- **Slide 20.** The data sit on disk in a flat file, stored basket by basket; items are positive integers. The picture shows three baskets, `{Milk 1, Beer 2, Fanta 3, Diaper 4, Butter 5}`, `{Milk 1, Beer 2, Bread 6}` and `{Milk 1, Beer 2, Sprite 7, Mustard 8}`, followed by "Etc.". Baskets are small, but there are many baskets and many items. As baskets are read they are expanded into pairs, triples, etc. with nested loops. Slide note: "We want to find frequent itemsets. To find them, we have to count them. To count them, we have to generate them."
- **Slide 21.** The true cost of mining disk-resident data is usually the number of disk I/Os. Algorithms read the data in *passes* (all baskets read in turn). "We measure the cost by the number of passes an algorithm makes over the data."
- **Slide 22.** For many frequent-itemset algorithms main memory is the critical resource: we need to count something (e.g. occurrences of pairs of items) and the number of different things we can count is limited by main memory. "Swapping in/out is a disaster (**why?**)".
- **Slide 23.** The hardest problem is often finding the frequent *pairs* $\{i_1, i_2\}$. "**Why?** Freq. pairs are common, freq. triples are rare. **Why?** Probability of being frequent drops exponentially with size." Concentrate on pairs first, then extend to larger sets.
- **Slide 24 (naïve algorithm).** Read the file once, counting in main memory the occurrences of each of the $n(n-1)/2$ possible pairs (generate each basket's pairs with two nested loops). Fails if $(\#\text{items})^2$ exceeds main memory; #items can be 100K (Wal-Mart) or 10B (Web pages). Suppose 10^5 items and 4-byte integer counts: number of pairs $10^5(10^5-1)/2 = 5\times10^9$; therefore 20 gigabytes of memory are needed.
- **Slides 25–27 (two approaches). Approach 1:** count all pairs using a matrix (a triangular matrix, 4 bytes per pair). **Approach 2:** keep triples $[i, j, c]$ = "the count of the pair of items $\{i, j\}$ is c"; if integers and item ids are 4 bytes we need approximately 12 bytes for each pair with count > 0. Slide 27: Approach 2 beats Approach 1 if less than 1/3 of the possible pairs actually occur. (Slide 26 is a picture only: a triangle labelled "4 bytes per pair" next to a triangle with a few dots labelled "12 per occurring pair".)
- **Slide 28.** "Problem is if we have too many items so the pairs do not fit into memory. Can we do better?"

(a) Naïve counting (slide 24). For n items, how many possible pairs are there? How much main memory does the naïve algorithm need for $n = 10^5$ items with 4-byte counts? (Slide answer: 5×10^9 pairs, 20 GB.)

(b) Matrix vs triples (slides 25–27). Write the memory needed by Approach 1 (triangular matrix) and by Approach 2 (triples). Show why Approach 2 wins only when fewer than 1/3 of the possible pairs occur, and evaluate both for 10^5 items.

(c) The "why?" questions (slides 21–23). Explain (i) why the cost is measured in passes over the data; (ii) why main memory is the bottleneck and why swapping in/out is a disaster; (iii) why finding frequent *pairs* is the hardest part, i.e. why frequent pairs are common but frequent triples are rare.

(d) Slide 28. "Can we do better?" What does the lecture do next?

(e) Practice (our own, not on the slides). Repeat (a) and (b) for $n = 20{,}000$ items; say which approach needs less memory if 30 million, and if 100 million, distinct pairs occur; and find the largest number of items whose triangular matrix of 4-byte counts fits in 8 GB.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

> **BEGINNER INTUITION**

Imagine n students and you want to know, for every two students, how many days they sat together. You need one tally box **per pair of students**, and the number of pairs grows with the *square* of the number of students (100 students: 4,950 pairs; 1,000 students: 499,500 pairs). In a data-mining run the "students" are items, the "days" are baskets and the tally boxes must live in **main memory** (RAM, random-access memory), because we add 1 to some box every time we see a pair — billions of times.

Approach 1 is a printed table with a pre-made box for every pair: boxes are small (4 bytes) and the box's position tells you which pair it is, but boxes for pairs that never sit together are wasted. **Approach 2** is a notebook where you write a line "student *i*, student *j*, count" only when a pair really occurs: a line is bigger (three numbers, 12 bytes, because you must write down *who* the pair is), but pairs that never occur cost nothing. Which is smaller depends on how many pairs actually occur — the break-even is exactly one third.

FORMULAS USED IN THIS QUIZ

Number of possible pairs: $N = n(n-1)/2 = C(n, 2)$. **Approach 1 memory:** $4 \times N$ bytes. **Approach 2 memory:** $12 \times x$ bytes, where x = number of distinct pairs that occur at least once ($0 \le x \le N$). **Break-even:** $12x = 4N \leftrightarrow x = N/3$. **Units:** 1 GB = 10^9 bytes, 1 TB = 10^{12}, 1 EB = 10^{18} (the slide's "20 gigabytes" uses decimal GB).

Part (a) — the naïve algorithm needs 20 GB for 10^5 items (slide 24)

1. **Formula / idea — count the possible pairs:** a pair is an *unordered* set {*i*, *j*} of two different items. There are n choices for the first item and $n-1$ for the second, so $n(n-1)$ ordered pairs; but {*i*, *j*} and {*j*, *i*} are the same pair, so every pair was counted twice and we divide by 2: $N = n(n-1)/2$. *Why we need it:* the naïve algorithm reads each basket once, generates all its pairs with two nested loops, and adds 1 to a counter for each pair. Before it knows which pairs will occur, it must keep **one counter for every possible pair**, all in main memory. So the number of counters is N.
2. **Substitute $n = 10^5$:** $N = 10^5 \times (10^5 - 1)/2 = 100{,}000 \times 99{,}999 / 2 =$ **4,999,950,000 ≈ $5{\times}10^9$** (the slide rounds because $10^5 - 1 \approx 10^5$, and $10^5 \times 10^5 / 2 = 5{\times}10^9$).
3. **Convert counters to bytes:** memory = (number of counters) × (bytes per counter) = 4,999,950,000 × 4 = 19,999,800,000 bytes ≈ $2{\times}10^{10}$ bytes = **about 20 GB**. *Why ×4:* the question says counts are 4-byte integers. (In binary units, 1 GiB = 2^{30} bytes, it is 18.6 GiB; either way it is far more than the 8–16 GB of RAM of a typical machine — *Beyond the slides:* typical RAM sizes.) So the naïve algorithm **fails** (slide 24).
4. **See how fast it grows:** memory = $4 \times n(n-1)/2 = 2n(n-1) \approx 2n^2$ bytes. It grows with the *square* of the number of items: 10 times more items means 100 times more memory (this is the slide's "(#items)² exceeds main memory"; the slide drops the constant 2). All values below were computed with Python:

Items n	Pairs $n(n-1)/2$	Memory at 4 bytes per pair
10,000 (10^4)	49,995,000 ≈ $5{\times}10^7$	≈ 0.2 GB
50,000	1,249,975,000 ≈ $1.25{\times}10^9$	≈ 5 GB
100,000 (10^5, Wal-Mart, slide 24)	**4,999,950,000 ≈ $5{\times}10^9$**	**≈ 20 GB**
1,000,000 (10^6)	499,999,500,000 ≈ $5{\times}10^{11}$	≈ 2,000 GB = 2 TB
10^{10} (Web pages, slide 24)	≈ $5{\times}10^{19}$	≈ $2{\times}10^{20}$ bytes = 200 EB

COMMON MISTAKE

(1) Forgetting the "÷ 2": using n^2 pairs gives 40 GB instead of 20 GB. (2) Forgetting "× 4 bytes": answering "$5{\times}10^9$" for the memory (that is the number of *counters*, not bytes). (3) Writing $n(n-1)/2$ but then using $10^5 \times 10^5$ without the 1/2.

Part (b) — triangular matrix (Approach 1) versus triples (Approach 2) (slides 25–27)

1. **Approach 1 — triangular matrix (4 bytes per possible pair):** a table with one 4-byte counter per pair. We keep only the cells with $i < j$, because $\{i, j\} = \{j, i\}$ and $\{i, i\}$ is not a pair. A full square $n \times n$ table would store every pair twice plus a useless diagonal: for $n = 10^5$ that is $4n^2$ = 40 GB, twice as much. **Memory = $4 \times N = 4 \times n(n-1)/2$ bytes** — the same for every dataset, even if most pairs never occur. The counter's *position* tells us which pair it belongs to, so no item ids are stored. Items are positive integers (slide 20), so they can be used directly as positions.

   NOT EXAMINED — HOW THE TRIANGLE IS STORED (UNDERSTAND, DON'T MEMORISE)
   Beyond the slides: with items numbered 1…n, the triangle is stored as one long array and the pair $\{i, j\}$, $i < j$, sits at position $k = (i-1)(n - i/2) + j - i$. For $n = 5$ the ten pairs {1,2}, {1,3}, {1,4}, {1,5}, {2,3}, …, {4,5} land in positions 1, 2, …, 10 without gaps (checked with Python).

2. **Approach 2 — triples [*i*, *j*, *c*] (12 bytes per occurring pair):** there is now no fixed position, so each entry must say *which* pair it is. We store the item id i (4 bytes), the item id j (4 bytes) and the count c (4 bytes): 3 × 4 = **12 bytes** per entry. An entry is created only when a pair is seen for the first time (so every stored count satisfies $c > 0$); pairs that never occur take no space. Let x = the number of distinct pairs that occur in the data. **Memory = $12 \times x$ bytes.** It depends on the data. (The slide says "approximately": a real program also needs a lookup structure such as a hash table to find the entry of a pair, which adds overhead.)

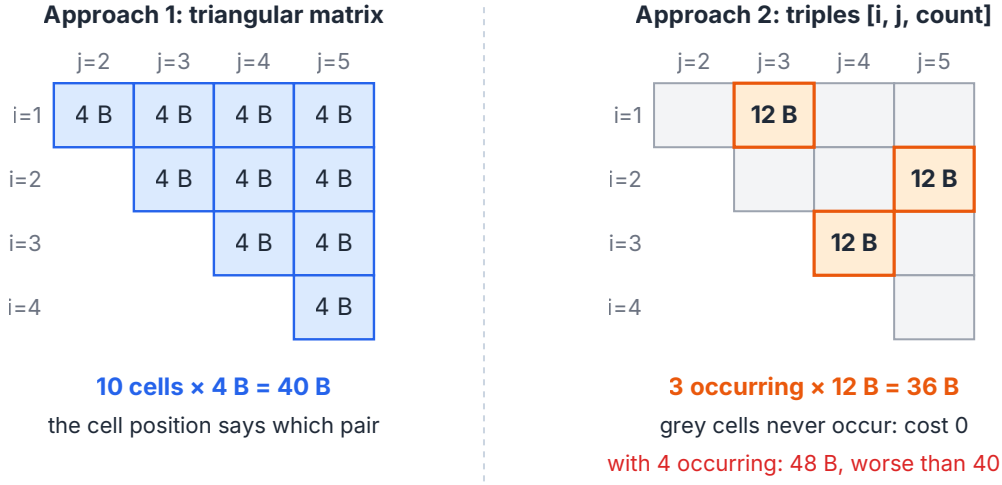


Figure L4-Q4-1 (mirrors slide 26, with a 5-item toy case). Left: every one of the 5·4/2 = 10 possible pairs owns a 4-byte cell (40 B). Right: only the 3 pairs that actually occur are stored, 12 B each (36 B). With 4 occurring pairs the triples would cost 48 B and lose; the break-even is 10/3 ≈ 3.33 pairs.

3. **Compare — derive the 1/3 rule:** Approach 2 is smaller exactly when $12x < 4N$. Divide both sides by 12: $x < 4N/12 = N/3$. Divide by N: $x/N < 1/3$. So **Approach 2 beats Approach 1 if fewer than 1/3 of the possible pairs actually occur** (slide 27); at exactly $x = N/3$ the two use the same memory, and above 1/3 the matrix wins. The threshold 1/3 does not depend on n: it is just the ratio 4 : 12 of the entry sizes.
4. **Evaluate for $n = 10^5$:** N = 4,999,950,000. Approach 1 = 4 × 4,999,950,000 = 19,999,800,000 bytes ≈ **20 GB** (fixed). Break-even $x = N/3$ = 1,666,650,000 ≈ **1.67×10^9 pairs**, where Approach 2 also needs 12 × 1,666,650,000 = 19,999,800,000 bytes ≈ 20 GB. Other cases (Python):

Share of pairs that occur	Occurring pairs x	Approach 2 = $12x$	Approach 1 = $4N$	Smaller
1%	49,999,500	0.6 GB	20 GB	**triples**
10%	499,995,000	6.0 GB	20 GB	**triples**
20%	999,990,000	12.0 GB	20 GB	**triples**
1/3 (≈ 33.3%)	1,666,650,000	20.0 GB	20 GB	tie
50%	2,499,975,000	30.0 GB	20 GB	matrix
100%	4,999,950,000	60.0 GB	20 GB	matrix (triples 3× worse)

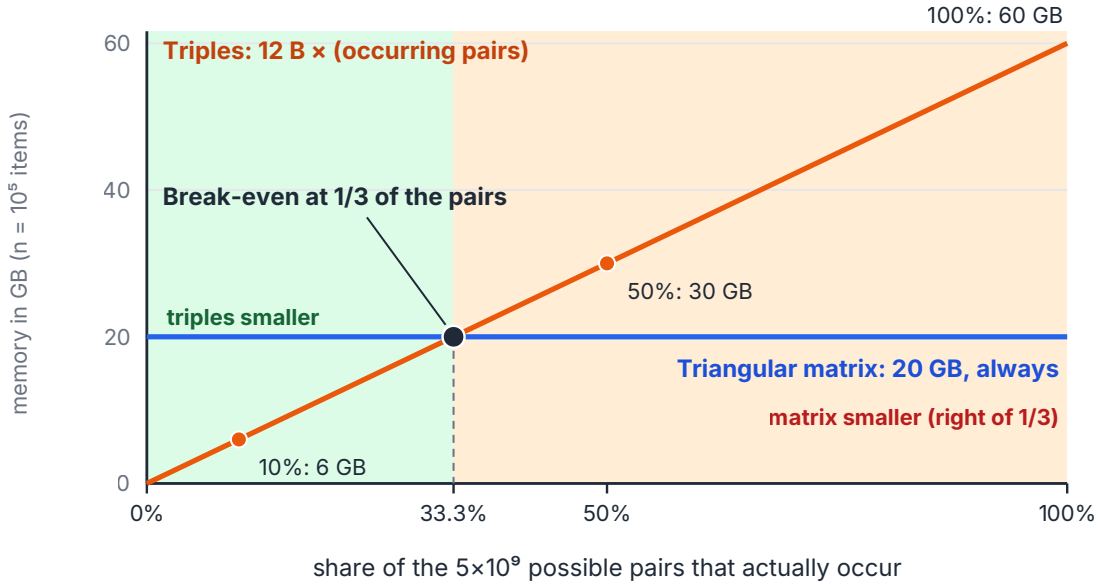


Figure L4-Q4-2. Memory for n = 10^5 items as a function of how many of the 5×10^9 possible pairs occur. The matrix is a flat line at 20 GB; the triples grow linearly and cross it at exactly 1/3 (slide 27). Green region: triples are smaller; orange region: the matrix is smaller.

5. **Two sanity checks on when each approach wins (Practice, our own numbers).** *(1) Toy case from the slide-20 baskets.* The three baskets {1,2,3,4,5}, {1,2,6}, {1,2,7,8} use items 1–8, so n = 8 and N = 8 × 7 / 2 = 28. They make 10 + 3 + 6 = 19 pair increments, but only x = **17 distinct pairs**, because {1,2} (Milk, Beer) occurs in all three baskets and ends with count 3. Matrix = 4 × 28 = 112 B; triples = 12 × 17 = 204 B. Here x/N = 17/28 ≈ 61% > 1/3, so the **matrix** is smaller, as the rule predicts (the file continues with "Etc." on the slide, so this is a toy only). *(2) Realistic scale.* Baskets are small, so x is limited by the pairs the baskets contain. 10^6 baskets of 10 items give at most 10^6 × C(10, 2) = 10^6 × 45 = 4.5×10^7 distinct pairs, i.e. at most 0.9% of the 5×10^9 possible pairs, so the triples need at most 12 × 4.5×10^7 = 540 MB instead of 20 GB. So on sparse data Approach 2 wins; on dense data (many pairs occur) the matrix wins.

EXAM TIP

To answer "when is Approach 2 better?" write the one-line inequality and then simplify: $12x < 4 \cdot n(n-1)/2 \Rightarrow x < n(n-1)/6 \Rightarrow$ fewer than one third of all pairs. State the two per-entry sizes (4 B vs 12 B = three 4-byte numbers) first; that alone usually earns a mark. Always quote the slide numbers: 5×10^9 pairs and 20 GB for 10^5 items.

COMMON MISTAKE

(1) Saying "triples are always 3 times worse (12 vs 4 bytes)": the 12 bytes are paid only for pairs that *occur*, the 4 bytes for *all possible* pairs. (2) Mixing up x (pairs that occur) and N (pairs that could occur), or turning the inequality round: triples win when x is *less* than $N/3$. (3) Forgetting units (bytes vs GB). (4) Forgetting that the matrix cost is the same whatever the data.

Part (c) — the "why?" questions (slides 21–23)

(c)(i) Why the cost is measured in passes (slide 21)

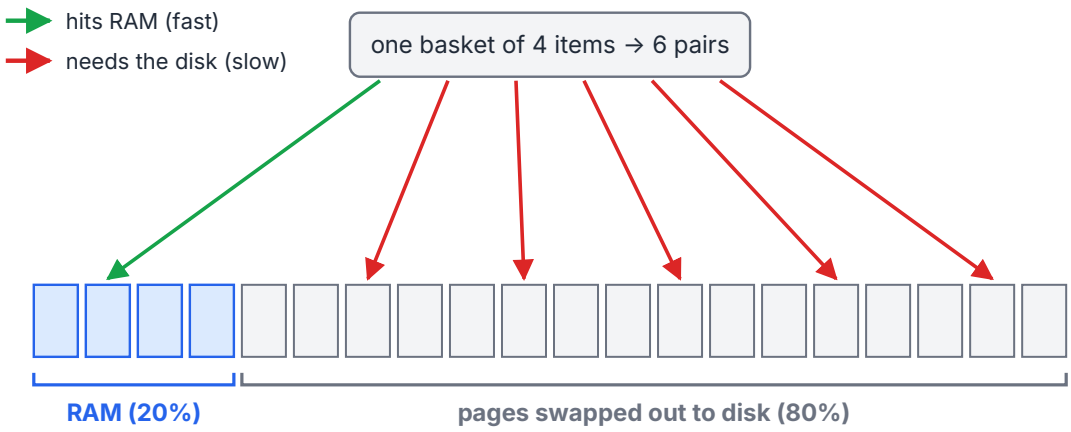
1. **The data live on disk:** the file of baskets is huge (gigabytes to terabytes) and cannot be held in main memory, so it stays on disk. Moving data from disk is far slower than computing on data already in RAM, so the time is dominated by **disk I/Os** (I/O = input/output, i.e. disk reads and writes), not by arithmetic. That is the slide's statement: "the true cost of mining disk-resident data is usually the number of disk I/Os".
2. **The data are read in passes:** the efficient way to use a disk is to read it in order, big block after big block. One **pass** = reading *all* baskets once, in turn, from the first to the last. Every extra pass re-reads the entire file. *Beyond the slides:* an illustration, order of magnitude only. A 1 TB file read sequentially at 100 MB/s takes 10^{12} / (100×10^{6}) = 10,000 s ≈ 2.8 hours per pass.
3. **So we count passes:** total I/O ≈ (number of passes) × (cost of one read of the file). What happens inside a pass — expanding a basket into pairs and adding 1 to counters in RAM — is cheap by comparison. Counting passes is simple and does not depend on the particular machine. Slide 21: "We measure the cost by the number of passes an algorithm makes over the data."
4. **It is a trade-off with memory:** the naïve algorithm uses 1 pass but needs 20 GB (part (a)); A-Priori uses 2 passes but far less memory (slide 30). Algorithms often spend an extra pass to save memory.

(c)(ii) Why main memory is the bottleneck, and why swapping is a disaster (slide 22)

1. **We must count while reading:** as every basket is read we update counters ("occurrences of pairs of items"). An update (add 1) is fast only if the counter is in main memory. The number of counters we can keep is therefore (RAM size) ÷ (bytes per counter) — "the number of different things we can count is limited by main memory". Part (a) showed that the table of pair counters needs about $2n^2$ bytes, which quickly exceeds RAM.
2. **What swapping is:** if the table is larger than RAM, the operating system keeps only some pages of it in RAM and parks the rest on disk (virtual memory). When the program touches a page that is not in RAM, the page must first be read from disk (a *page fault*) and often another page must be written out to make room.
3. **Why it is a disaster — (A) no locality:** the pairs of one basket land in scattered cells of the table, and the next basket hits other scattered cells. The updates jump around the whole table at random, so a page just loaded is rarely used again soon. *Illustration (our own numbers):* with a 20 GB table and 4 GB of RAM, if updates were spread evenly, only about 1/5 of the table is in RAM at any moment, so about 4 of every 5 updates wait for the disk (Figure L4-Q4-3).
4. **(B) Disk is about 10^5 times slower than RAM per access** (*Beyond the slides:* typical orders of magnitude for a hard disk, RAM ≈ 100 ns, random disk access ≈ 10 ms: 10 ms / 100 ns = 10^5). A solid-state drive (SSD) is faster, roughly 100 µs per random access, i.e. still 10^3 times slower than RAM; the conclusion does not change.
5. **(C) There are billions of updates.** A basket of k items makes $k(k-1)/2$ pair updates. *Illustration (our own numbers):* 10^8 baskets of 10 items give $10^8 \times 45 = 4.5\times10^9$ updates:

Situation	Time per update	Total time for 4.5×10^9 updates
Every counter is in RAM	100 ns	**450 s = 7.5 minutes**
Every update goes to disk (worst case, hard disk)	10 ms	4.5×10^7 s ≈ 521 days ≈ 1.4 years
80% of updates go to disk (20 GB table, 4 GB RAM)	mixed	still ≈ 417 days
Every update goes to an SSD	100 µs	4.5×10^5 s ≈ 5.2 days

   The machine spends its time moving pages instead of counting (**thrashing**). Moreover these are *random* I/Os, which the "number of passes" measure of slide 21 does not even see, so swapping silently ruins the cost model.
6. **The cure:** design the algorithm so that every counter fits in RAM — use less memory per count (Approach 2 when few pairs occur) or count fewer things (A-Priori, slide 30) — even if it costs another pass.



20 GB table, 4 GB RAM, even spread: about 4 of every 5 updates wait for the disk

Figure L4-Q4-3. Why swapping thrashes. One basket updates 6 pair counters that sit on scattered pages of a table that is larger than RAM. Only the updates that happen to land on the few pages in RAM are fast; the others each trigger a slow disk access.

(c)(iii) Why frequent *pairs* are the hard problem (slide 23)

1. **What the slide claims:** "Freq. pairs are common, freq. triples are rare" because "probability of being frequent drops exponentially with size". Frequent means support ≥ s: the set must appear together in at least s baskets.
2. **Why frequent sets get rarer as they grow:** every extra item is an extra condition that a basket must satisfy. *Beyond the slides:* an illustration that assumes items are independent (real data are not exactly independent). If an item is in 10% of the baskets, a given pair is in about 0.1^2 = 1% of them, a triple in 0.1^3 = 0.1%, a 4-set in 0.01%. Each added item multiplies by the same factor, so the chance falls **exponentially** with the size. Real data are correlated, so the decrease is slower, but the trend is the same.
3. **Monotonicity makes it a chain:** if an itemset is frequent then every subset of it is frequent too (slide 30). So a frequent triple can only be built from frequent pairs, and a frequent pair only from frequent items: pairs are the necessary stepping stone to everything larger.
4. **Check on a real example — the 8 baskets of slide 18 with $s = 3$** (we counted them with Python; slide 18 lists the frequent itemsets of size 2 and 3):

Size	Possible sets (6 items)	Sets that occur at least once	Frequent sets (support ≥ 3)	Share of possible sets that are frequent
1 (items)	6	6	4: m, c, b, j	4/6 ≈ 67%
2 (pairs)	15	12	**4: {b,m} {b,c} {c,m} {c,j}**	4/15 ≈ 27%
3 (triples)	20	9	**1: {m,c,b}**	1/20 = 5%
4	15	2	0	0%

   The share of possible sets that are frequent falls 67% → 27% → 5% → 0% as the size grows, as the slide says (the exponential drop is much clearer on large data; with only 8 baskets the numbers are tiny).
5. **Why that makes pairs the memory bottleneck:** before any counting we cannot know which pairs will turn out frequent, so a naïve counter must be ready for *all* $N = n(n-1)/2$ pairs — 5×10^9 for 10^5 items. Single items are cheap: 10^5 counters × 4 B = 400 KB, about 50,000 times less than the 20 GB for pairs (slide 31: pass 1 needs "memory proportional to #items"). For triples and larger sets, monotonicity lets us count *only* sets whose smaller subsets are all frequent, and by step 2 there are few of those. (Counting *every* triple naïvely would be far worse: $C(10^5, 3) \approx 1.67\times10^{14}$ counters, about 667 TB at 4 bytes — so

the saving from pruning is essential.) Hence the pair-counting table is the largest table the algorithm has to hold, and slide 23 concludes: "Let's first concentrate on pairs, then extend to larger sets."

KEY IDEA

Three facts, one story. (1) Only passes over the disk-resident file are charged (slide 21). (2) Everything that is counted must fit in RAM, otherwise swapping makes the run impossibly slow (slide 22). (3) The thing that threatens to overflow RAM is the table of *pair* counters, which grows like n^2 (slides 23–24). The two ways of storing it (matrix 4 B per possible pair, triples 12 B per occurring pair) only shift the problem by a constant factor, so we need a smarter algorithm (A-Priori, slide 30).

Part (d) — "Can we do better?" (slide 28)

1. **The problem:** both approaches break down when there are too many items: the matrix needs $4N$ bytes (2 TB for 10^6 items, 200 EB for 10^{10}, part (a)), and the triples may need up to $12N$ if many pairs occur. Counting *every* possible pair is simply too much.
2. **The answer the lecture gives:** the two-pass **A-Priori** algorithm (Lecture 4 · Slides 30–33). It uses **monotonicity** (slide 30): if item *i* does not appear in *s* baskets, then no pair containing *i* can appear in *s* baskets. Pass 1 counts the individual items (memory proportional to #items) and finds the frequent items; pass 2 counts *only* pairs whose two items are both frequent (memory proportional to the square of the number of *frequent* items only; slide 31).
3. **The trade-off:** one extra pass over the data buys a much smaller counting table. Slide 33 then reuses the triangular-matrix idea of this quiz with n = number of *frequent* items, after re-numbering the frequent items 1, 2, … .

Part (e) — Practice: n = 20,000 items (our own item count, not from the slides)

1. **Possible pairs:** $N = n(n-1)/2 = 20{,}000 \times 19{,}999 / 2 =$ **199,990,000** $\approx 2\times10^8$.
2. **Approach 1 (matrix):** 4 × 199,990,000 = **799,960,000 bytes ≈ 0.8 GB** (800 MB), whatever the data.
3. **Break-even for Approach 2:** $x < N/3 = 66{,}663{,}333.3$, i.e. at most 66,663,333 pairs ($\approx 6.67\times10^7$, 1/3 of all pairs); at that point 12 × 66,663,333 = 799,959,996 bytes ≈ 0.8 GB, the same as the matrix.
4. **Scenarios (Python):**

Occurring pairs x	Share of N	Triples = $12x$	Matrix	Smaller
19,999,000	10%	239,988,000 B ≈ 0.24 GB	≈ 0.8 GB	**triples (3.3× smaller)**
30,000,000	15%	360,000,000 B = 0.36 GB	≈ 0.8 GB	**triples (55% less memory)**
66,663,333	1/3	799,959,996 B ≈ 0.8 GB	≈ 0.8 GB	tie
100,000,000	≈ 50%	1,200,000,000 B = 1.2 GB	≈ 0.8 GB	matrix (triples 1.5× larger)

   So with 30 million occurring pairs the **triples** need less memory; with 100 million the **matrix** does.
5. **Largest n whose triangular matrix fits in 8 GB:** need $4 \times n(n-1)/2 = 2n(n-1) \le 8\times10^9 \Rightarrow n(n-1) \le 4\times10^9 \Rightarrow n \approx \sqrt{(4\times10^9)} \approx 63{,}246$. Check with Python: n = 63,246 needs 7,999,986,540 bytes ≤ 8×10^9, but n = 63,247 needs 8,000,239,524 bytes > 8×10^9. So **63,246 items**. Sanity check by scaling: memory grows like n^2, and 20 GB corresponds to 10^5 items, so 8 GB corresponds to $10^5 \times \sqrt{(8/20)} \approx 63{,}246$ ✓. (If "8 GB" means 8 GiB = 2^{33} bytes, the answer is exactly n = 65,536 = 2^{16}.)

Final answer:

(a) Number of pairs = n(n−1)/2. For n = 10^5: 4,999,950,000 ≈ 5×10^9 pairs × 4 bytes ≈ 2×10^{10} bytes = 20 GB (grows like 2n²).

(b) Triangular matrix = 4 bytes × n(n−1)/2 (fixed; 20 GB for 10^5 items). Triples = 12 bytes × x, with x = number of occurring pairs. 12x < 4·n(n−1)/2 ↔ x < n(n−1)/6, so triples win only if fewer than 1/3 of the possible pairs occur (for 10^5 items: x < 1.67×10^9; at 10% occurring 6 GB, at 50% 30 GB, at 100% 60 GB).

(c)(i) Cost = number of passes: the file is on disk, disk I/O dominates, and one pass reads every basket once, sequentially.

(c)(ii) Counters must fit in RAM; if swapped, almost every random update needs a disk access (about 10^5 times slower than RAM, times billions of updates), so the run thrashes.

(c)(iii) Pairs are the bottleneck: there are n(n−1)/2 of them to count (5×10^9 for 10^5 items), while the chance of being frequent drops exponentially with set size (monotonicity), so far fewer triples and larger sets need counting.

(d) It leads to the two-pass A-Priori algorithm (slides 30–33): pass 1 finds the frequent items, pass 2 counts only pairs of frequent items; one extra pass to save memory.

(e) n = 20,000: 199,990,000 pairs; matrix ≈ 0.8 GB; break-even x ≈ 6.67×10^7 pairs (1/3); 30 million occurring pairs: triples 0.36 GB win; 100 million: triples 1.2 GB lose; largest n that fits 8 GB = 63,246.

Where the marks come from: (1) formula n(n−1)/2 for the number of pairs, with the reason (÷2 because {i,j} = {j,i}); (2) substitution to 5×10^9; (3) ×4 bytes to reach 20 GB, with units; (4) triangular matrix = 4 bytes per possible pair, triples [i, j, c] = 3 × 4 = 12 bytes per occurring pair; (5) the inequality 12x < 4·n(n−1)/2 and the conclusion "fewer than 1/3 of the possible pairs"; (6) the passes argument: data on disk, disk I/O dominates, a pass = one sequential read of all baskets; (7) main memory limits what can be counted; swapping means random accesses to disk (very slow) for billions of updates; (8) pairs are hardest: n(n−1)/2 candidates, and frequency drops exponentially with set size (monotonicity), so triples are rare; (9) slide 28 leads to the two-pass A-Priori algorithm that counts only pairs of frequent items; (10) the practice numbers (199,990,000 pairs, 0.8 GB, 1/3 threshold, 63,246 items).

Slide check: matches the slide's answers — $10^5(10^5-1)/2 = 4.99995\times10^9 \approx 5\times10^9$ pairs, 19.9998 GB ≈ 20 GB, 4 bytes vs 12 bytes, break-even 1/3 (all verified with Python). **No numerical erratum.** Omissions and loose wording on the slides (none changes an answer): (i) slide 24's "fails if (#items)² exceeds main memory" is shorthand — the exact need is 2n(n−1) bytes, the constant 2 is dropped; (ii) slide 25 says "count all pairs using a matrix" before slides 26–27 make it a *triangular* matrix, and "4 bytes per pair" means per *possible* pair; (iii) slide 25's "approximately 12 bytes" ignores the lookup-structure (hash table) overhead of a real implementation, which would lower the 1/3 threshold a little; (iv) slides 22 and 23 pose "why?" but print no answer — the answers above are standard course background (textbook *Mining of Massive Datasets*, Chapter 6), with the order-of-magnitude timings marked "Beyond the slides"; (v) slide 20's last line "Items are positive integers," is cut off on the slide (*Beyond the slides:* the usual version continues "and boundaries between baskets are −1"); this does not affect any answer.

The A-Priori algorithm

Lecture 4 · Slides 29–36

Slide 29 is another black divider ("A-Priori Algorithm", no printed number). Slides 30–36 give the algorithm for pairs, then for all itemset sizes. The name is Latin for "from what comes before": use what you already know about smaller sets to decide what to count among bigger ones.

The key idea: monotonicity and its contrapositive

Lecture 4 · Slide 30

BEGINNER INTUITION

A band cannot fill a stadium more often than any single member fills a stadium. In the same way, a bundle of products cannot be bought together more often than each product is bought at all. So if a product is rarely bought, every bundle containing it is rarely bought, and you can stop worrying about all of those bundles.

DEFINITION: MONOTONICITY (SLIDE 30)

If a set of items I appears at least s times, so does every **subset** J of I.

CONTRAPOSITIVE FOR PAIRS (THE FORM YOU ACTUALLY USE)

If item i does **not** appear in s baskets, then **no pair including i** can appear in s baskets. (Read "s baskets" as "at least s baskets", as in the first statement.) The same holds for any set that contains an infrequent subset.

Why it is true: every basket containing I also contains each subset J of it, so support(J) ≥ support(I). Support can only go down as sets grow.

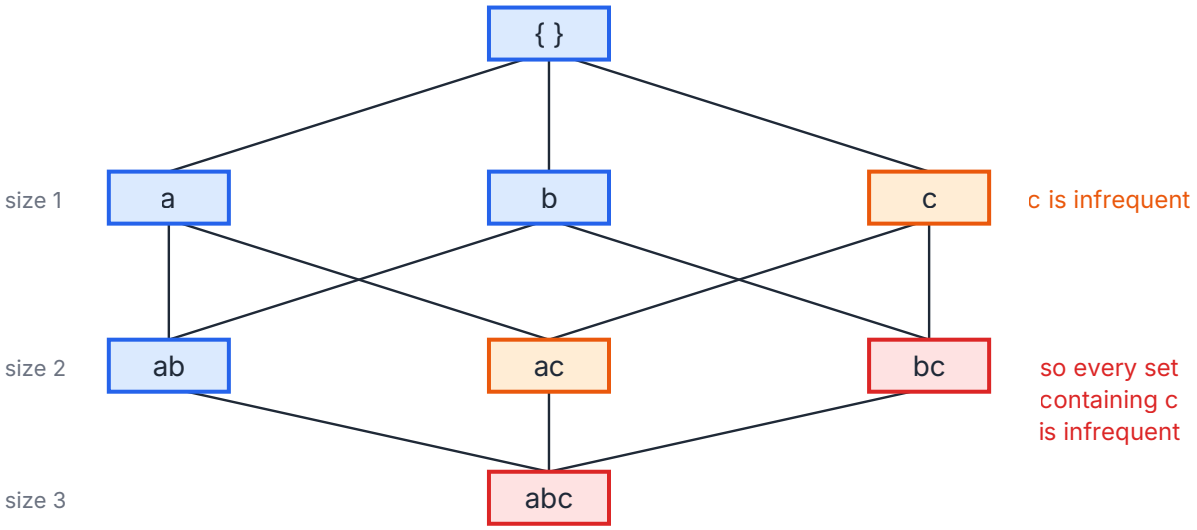


Figure L4p2-3. The itemset lattice of Slide 30 (the slide gives no legend; this colouring is our reading). Lines join a set to a set one item larger. If c is infrequent, c, ac, bc and abc (orange/red) can all be skipped; only {}, a, b, ab (blue) remain possible.

Worked example: monotonicity on the Slide 9 baskets (s = 3) Lecture 4 · Slides 9, 30

Slide 9's baskets: B_1={m,c,b}, B_2={m,p,j}, B_3={m,b}, B_4={c,j}, B_5={m,p,b}, B_6={m,c,b,j}, B_7={c,b,j}, B_8={b,c}. Item `p` appears only in B_2 and B_5: support 2 < 3. So {p,m}, {p,b}, {p,j}, {p,m,b}, ... cannot reach 3 and need never be counted. (Check: support{m,p} = 2.)

SLIDE ERRATUM (WORDING)

Slide 30 calls A-Priori a "two-pass approach". That is true for finding frequent **pairs**. For larger itemsets A-Priori needs one pass per itemset size (Slide 36).

Pass 1 and Pass 2

Lecture 4 · Slides 31–32

1. **Pass 1.** Read the baskets and count the occurrences of each **individual item**. Memory needed: proportional to the number of items (one counter per item).
2. **Between passes.** Items with count ≥ s are the **frequent items**. Keep a list of them; drop the item counters.

3. **Pass 2.** Read the baskets again. For each basket, keep only its frequent items, generate all pairs among them, and add 1 to each pair's counter. These pairs are the **candidate pairs**: both elements are frequent. Memory needed: proportional to the *square of the number of frequent items* (counters), plus the list of frequent items so that you can test membership.
4. **End.** Pairs whose counter is ≥ s are the frequent pairs.

SLIDE ERRATUM (OMISSION)
"Memory proportional to square of frequent items" means the square of the *number* of frequent items. The slide also never states the final filter after Pass 2 (keep pairs with count ≥ s); it appears on Slide 34.

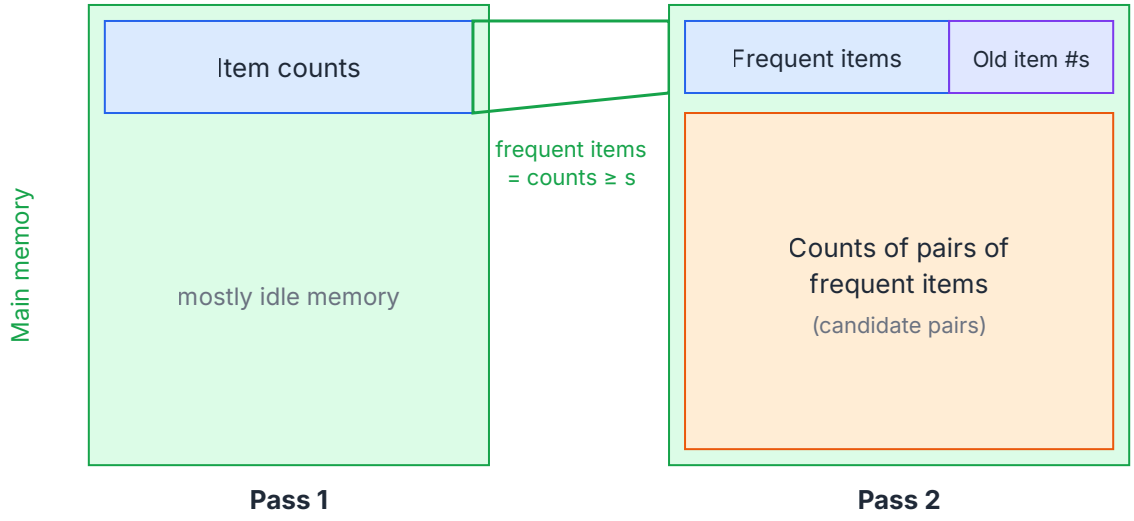


Figure L4p2-4. Main memory in A-Priori (Slides 32–33; box sizes are schematic). Pass 1 uses a small block; most memory is idle. Pass 2 keeps the list of frequent items (with the "Old item #s" table of Slide 33) and the big table of pair counters, which is the memory bottleneck.

Slide 32's picture carries two messages: Pass 2 needs far more memory than Pass 1, and Pass 1 leaves most of memory empty (PCY will use that, Slide 38).

Detail: triangular matrix and renumbering

Lecture 4 · Slide 33

Slide 33: you can use the triangular-matrix method with n = **number of frequent items** (note: here n no longer means all items, as on Slide 24). This may save space compared with storing triples. The catch: a triangular matrix needs items numbered 1..n with no gaps, but frequent items keep their original ids (say 5, 17, 903, 12004).

TRICK (SLIDE 33)
Re-number the frequent items 1, 2, … and keep a table relating the new numbers to the original ("Old item #s" in the figure). Then the triangular array has only $n(n-1)/2$ cells for the *frequent* items.

Worked example: the saving Lecture 4 · Slide 33

Beyond the slides, illustrative numbers. With 10^5 items the naive matrix needs about 20 GB (Slide 24). If only 10^4 items are frequent: $10^4(10^4-1)/2$ = 49,995,000 pairs × 4 bytes ≈ 200 MB. With 1000 frequent items: 4 × 1000·999/2 = 1,998,000 bytes ≈ 2 MB. If instead all 10^6 items were numbered: about 2×10^{12} bytes (2 TB).

General k: candidates C_k and frequent sets L_k

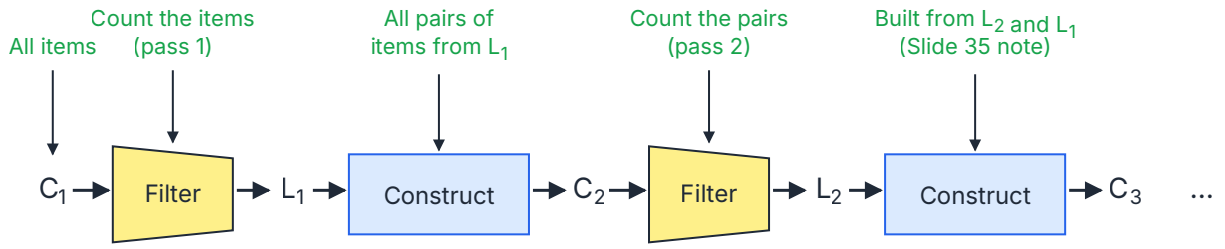
Lecture 4 · Slides 34–35

DEFINITION: C_K AND L_K (SLIDE 34)

C_k = the **candidate** k-itemsets: those that *might* be frequent (support ≥ s) based on information from the pass for $k-1$. **L_k** = the set of **truly frequent** k-itemsets (the slide says "k-tuples"; they are sets).

COMMON MISTAKE

A candidate has *not* been shown to be frequent; it only means no subset has been ruled out. Counting decides. Frequent means support **≥ s**, not > s (the slide draws "≥" as an underlined ">").



Filter = needs a pass over the data (count). Construct = in memory only (no pass).

Figure L4p2-5. The A-Priori loop of Slide 34: construct candidates, then filter by counting. Order to remember: C_k → (count in pass k) → L_k → (construct) → C_{k+1}.

C_1 is all items; counting them (Filter) gives L_1, the frequent items. Construct forms C_2 = all pairs of items from L_1. A second Filter counts those pairs and keeps the ones reaching s (L_2). Slide 34 marks the second Construct "To be explained"; Slide 35 explains it.

Worked example: the hypothetical steps of Slide 35 Lecture 4 · Slide 35

Step	Set	Size
C_1	`{b} {c} {j} {m} {n} {p}`	6
count, prune non-frequent	L_1 = `{b, c, j, m}` (n and p dropped)	4
generate C_2	`{b,c} {b,j} {b,m} {c,j} {c,m} {j,m}` (all pairs of L_1)	6
count, prune	L_2 = `{b,m} {b,c} {c,m} {c,j}`	4
generate C_3	`{b,m,c} {b,m,j} {b,c,j} {c,m,j}`	4
count, prune	L_3 = `{b,c,m}`	1

The callout on the slide (green "**") says: **generate C_k from L_{k-1} and L_1**, i.e. take each frequent pair and add one frequent item. {b,m}+c = {b,m,c}; {b,m}+j = {b,m,j}; {b,c}+j = {b,c,j}; {c,m}+j = {c,m,j}. The other extensions ({b,c}+m, {c,m}+b, {c,j}+b, {c,j}+m) only repeat triples already listed. This reproduces exactly the slide's four triples (checked by computer).

SLIDE ERRATUM (OMISSION): THE SLIDE'S RULE OVER-GENERATES

The slide's rule does not check that *every* 2-subset of a new triple is frequent. Beyond the slides (standard A-Priori, textbook *Mining of Massive Datasets*): a triple is a candidate only if **all** its (k–1)-subsets are in L_{k-1}. Here {b,m,c} has {b,m}, {b,c}, {c,m} all in L_2 and stays; {b,m,j} lacks {b,j} and {m,j}; {b,c,j} lacks {b,j}; {c,m,j} lacks {m,j}. Those three can be pruned *before* counting, so C_3 = {{b,c,m}} and L_3 is unchanged. The slide's version is not wrong, only wasteful.

Also, the data are made up: no baskets or threshold are given, so supports cannot be recomputed. With Slide 9's baskets and s = 3, the true frequent pairs are {b,m}, {b,c}, {c,j} ({c,m} has support 2) and there is no frequent triple. Quiz L4-Q5 runs A-Priori with real counts on the Slide 18 baskets.

A-Priori for all frequent itemsets

Lecture 4 · Slide 36

- **One pass for each k** (itemset size). Total passes = size of the largest itemset you ask for.
- **Needs room in main memory to count each candidate** of the current size.
- For typical market-basket data and reasonable support (e.g. 1 %), **k = 2 requires the most memory**: there are very many candidate pairs (quadratic in the number of frequent items), while frequent triples are rare (Slide 23), so from k = 3 on the candidate sets shrink sharply. This is exactly why PCY attacks the Pass 2 memory.
- **Extension:** lower the support s as the itemset gets bigger. Big sets are naturally rarer, so a lower threshold keeps interesting large sets from being missed. (Beyond the slides: with a different s per level, monotonicity pruning must be applied carefully, since thresholds differ by level.)

SLIDE ERRATUM (UNIT)

"1 %" is a fraction; elsewhere support is an absolute count (Slide 8). s = 1 % of the baskets means s = 0.01 × (number of baskets).

EXAM TIP

A-Priori answers to know cold: (1) the property used is monotonicity; (2) Pass 1 counts items, Pass 2 counts only pairs of frequent items; (3) memory in Pass 2 is about $m(m-1)/2$ counters for m frequent items; (4) it needs one pass per itemset size; (5) it is exact: no frequent set is lost, since every discarded set had an infrequent subset.

Try it yourself first: reproduce the whole run $C_1 \rightarrow L_3$ with real counts, then do the integer-basket exercise.

Quiz L4-Q5 A-Priori step by step: C_1, L_1, C_2, L_2, C_3, L_3 Lecture 4 · Slides 29–36

QUESTION (AS ON THE SLIDES)

Slide 35 – "Example: Hypothetical steps of the A-Priori algorithm". The slide runs A-Priori with support threshold **s = 3** on the eight baskets of **Slide 18** and prints only the sets C_1, L_1, C_2, L_2, C_3, L_3 (no counts, and it does not reprint the baskets). Reproduce the whole run yourself, with the support count of every candidate, and explain why each step is allowed. The baskets (single-letter item names, as on Slide 18):

Basket	Items	Basket	Items
B_1	{m, c, b}	B_2	{m, p, j}
B_3	{m, c, b, n}	B_4	{c, j}
B_5	{m, p, b}	B_6	{m, c, b, j}
B_7	{c, b, j}	B_8	{b, c}

(a) Pass 1. Write C_1, count the support of every item and give L_1.

(b) Pass 2. Construct C_2 from L_1 (Slide 35 lists 6 pairs), count the support of every candidate pair and give L_2.

(c) Pass 3. Construct C_3 (Slide 35 note: "generate C_k from L_{k-1} and L_1"), count the supports and give L_3. *Practice add-on:* which candidates would the monotonicity idea (Slide 30) remove from C_3 before counting?

(d) *Practice follow-up:* what must main memory hold in Pass 1 and in Pass 2 (Slides 31–33), and why may A-Priori ignore the items `n` and `p` from Pass 2 onwards?

Slide 35 is a worked example, not a question: parts (a)–(c) restate the slide's run as a task; the last sentence of (c) and all of (d) are our own Practice questions on Slides 30–33. **Careful with the data:** here B_3 = {m, c, b, n} (Slide 18), whereas Slides 12–15 use B_3 = {m, b}. Slide 35 does not reprint the baskets, but its C_1 contains `n`, and only the Slide-18 version of B_3 contains `n`.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

1. **Tools 1 – support and the threshold.** The *support* of an itemset I is the number of baskets that contain **every** item of I. With $s = 3$, an itemset is **frequent** exactly when support(I) ≥ 3 (Slide 31: "appear ≥ s times"). "At least" matters: a support of exactly 3 *is* frequent. (Slide 34 prints the symbol as an underlined ">", which means ≥; the data of Slide 35 confirm it, because {c,j}, {c,m} and {b,c,m} have support exactly 3 and are kept.)
2. **Tools 2 – C_k, L_k and the loop (Slide 34).** C_k = the *candidates*: size-k sets that *might* be frequent, decided only from the information of the pass for k-1. L_k = the size-k sets that are *truly* frequent, i.e. the candidates whose counted support is ≥ s. The pipeline of Slide 34 is repeated for k = 1, 2, 3, ...: **Construct** C_k (no data needed) → **one pass** over all baskets counts every candidate → **Filter** (keep count ≥ s) → L_k → construct C_{k+1} from L_k. For k = 1 the candidates are simply "All items"; C_2 = all pairs of items from L_1; for C_3 the slide only says "To be explained" and gives the footnote of Slide 35 ("generate C_k from L_{k-1} and L_1").
3. **Tools 3 – monotonicity (Slide 30).** If an itemset I appears in at least s baskets, then every subset $J \subseteq I$ also appears in at least s baskets, because every basket that contains I also contains J: support(J) ≥ support(I). We use the **contrapositive**: if some subset of I is *not* frequent, then I cannot be frequent. For pairs (Slide 30): if item i is in fewer than s baskets, no pair containing i can be in s baskets.

> **BEGINNER INTUITION**
> Three friends can have lunch together 3 times only if every two of them have had lunch together at least 3 times (each shared lunch of the three also counts for every pair inside the group). So you never need to check a group of three unless all of its pairs have already passed the test. A-Priori builds bigger groups only from groups that already passed, level by level; the shortlist at each level is C_k, the ones that pass are L_k.

4. **Part (a), Pass 1 – C_1 = every item that occurs.** Nothing has been learned yet, so every single item is a candidate (Slide 34: "All items" feeds the first Filter). Scanning B_1–B_8 gives six items:
   C_1 = { {b}, {c}, {j}, {m}, {n}, {p} } (Slide 35, line 1).

5. **Count the support of each item** (read the baskets once; keep one counter per item – memory proportional to the number of items, Slide 31). Add 1 to an item's counter for every basket that contains it:

Item	Baskets that contain it	Support	≥ 3 ?
`b`	B_1, B_3, B_5, B_6, B_7, B_8	**6**	**yes**
`c`	B_1, B_3, B_4, B_6, B_7, B_8	**6**	**yes**
`j`	B_2, B_4, B_6, B_7	**4**	**yes**
`m`	B_1, B_2, B_3, B_5, B_6	**5**	**yes**
`n`	B_3	**1**	no
`p`	B_2, B_5	**2**	no

   Sanity check: every basket entry is counted once, so the six supports must add up to the total number of items in all baskets: 6 + 6 + 4 + 5 + 1 + 2 = 24 = 3 + 3 + 4 + 2 + 3 + 4 + 3 + 2 ✓.

6. **Filter → L_1 = the items with support ≥ 3.** `b` (6), `c` (6), `m` (5) and `j` (4) pass; `n` (1) and `p` (2) fail.
   L_1 = { b, c, j, m } (Slide 35: "Prune non-frequent").

7. **Why dropping n and p is safe (monotonicity).** Any pair containing `n` occurs only in baskets that contain `n`, so its support is ≤ support(`n`) = 1 < 3; any pair containing `p` has support ≤ 2 < 3. The data confirm it: {b,n}, {c,n}, {m,n}, {b,p}, {j,p} occur once, {m,p} twice, and {c,p}, {j,n}, {n,p} never. The same bound holds for every larger set that contains `n` or `p`, so these two items never have to be looked at again.

8. **Part (b), Pass 2 – construct C_2 = all pairs of items from L_1.** With 4 frequent items there are $n(n-1)/2$ = 4·3/2 = **6** pairs (the pair-counting formula of Slide 24):
   C_2 = { {b,c}, {b,j}, {b,m}, {c,j}, {c,m}, {j,m} } (Slide 35).

   The nine pairs that contain `n` or `p` ({b,n}, {b,p}, {c,n}, {c,p}, {j,n}, {j,p}, {m,n}, {m,p}, {n,p}) never get a counter. Without Pass 1 we would have to count all 6·5/2 = 15 pairs of the six items. This is the A-Priori saving.

9. **Count the candidate pairs (second pass over the baskets).** For each basket, first ignore the items that are not in L_1 (here n and p; the basket itself is *not* thrown away), then add 1 to every pair of the items that remain (Slide 31: count "only those pairs where both elements are frequent"; the pairs are generated by two nested loops, Slide 20).

Basket	Items	Kept (in L_1)	Candidate pairs it adds 1 to
B_1	`m, c, b`	`b, c, m`	`bc, bm, cm`
B_2	`m, p, j`	`j, m` (p ignored)	`jm`
B_3	`m, c, b, n`	`b, c, m` (n ignored)	`bc, bm, cm`
B_4	`c, j`	`c, j`	`cj`
B_5	`m, p, b`	`b, m` (p ignored)	`bm`
B_6	`m, c, b, j`	`b, c, j, m`	`bc, bj, bm, cj, cm, jm`
B_7	`c, b, j`	`b, c, j`	`bc, bj, cj`
B_8	`b, c`	`b, c`	`bc`

Tally the pairs:

Pair	Baskets that contain it	Support	≥ 3 ?
`{b,c}`	B_1, B_3, B_6, B_7, B_8	**5**	**yes**
`{b,j}`	B_6, B_7	**2**	no
`{b,m}`	B_1, B_3, B_5, B_6	**4**	**yes**
`{c,j}`	B_4, B_6, B_7	**3**	**yes (exactly s)**
`{c,m}`	B_1, B_3, B_6	**3**	**yes (exactly s)**
`{j,m}`	B_2, B_6	**2**	no

Sanity check: the supports add up to 5 + 2 + 4 + 3 + 3 + 2 = 19, which equals the number of pairs added per basket: 3 + 1 + 3 + 1 + 1 + 6 + 3 + 1 = 19 ✓.

10. **Filter → L_2 = the pairs with support ≥ 3.** {b,c} (5), {b,m} (4), {c,j} (3) and {c,m} (3) pass; {b,j} (2) and {j,m} (2) fail.
L_2 = { {b,m}, {b,c}, {c,m}, {c,j} } (Slide 35 lists them in this order; they are also the 2-item sets printed on Slide 18).

11. **Part (c), Pass 3 – construct C_3 the way Slide 35 does it.** The footnote on the slide says: generate C_k from L_{k-1} and L_1. So take every frequent pair of L_2 and add one frequent item of L_1 that it does not contain. *Why this reaches every frequent triple:* if {x,y,z} is frequent, then by monotonicity {x,y} is in L_2 and z is in L_1, so the triple is produced from the pair {x,y} plus the item z. A triple can be reached from several pairs, so write each distinct triple once (the order inside a set does not matter):

Pair from L_2	Add an item of L_1	Triples obtained
`{b,m}`	`c` or `j`	`{b,m,c}`, `{b,m,j}`
`{b,c}`	`m` or `j`	`{b,m,c}` (again), `{b,c,j}`
`{c,m}`	`b` or `j`	`{b,m,c}` (again), `{c,m,j}`
`{c,j}`	`b` or `m`	`{b,c,j}` (again), `{c,m,j}` (again)

8 extensions, but only 4 different triples:
C_3 = { {b,m,c}, {b,m,j}, {b,c,j}, {c,m,j} } (exactly the list on Slide 35). With only 4 frequent items these are simply all C(4,3) = 4 triples of L_1.

12. **What the monotonicity prune would remove from C_3 (practice add-on to part c).** Slide 35 keeps all 4 triples and lets the count in Pass 3 reject the bad ones. *Beyond the slides – the textbook A-Priori rule* (*Mining of Massive Datasets*): keep a triple in C_3 only if **all three of its pairs are in L_2** (in general: all its (k-1)-subsets are in L_{k-1}). The test uses only numbers we already have from Pass 2:

Triple	Its three pairs (support from Pass 2)	All in L_2 ?	Verdict
{b,m,c}	bm (4) ✓, bc (5) ✓, cm (3) ✓	**yes**	**keep**
{b,m,j}	bm (4) ✓, bj (2) ×, jm (2) ×	no	prune
{b,c,j}	bc (5) ✓, bj (2) ×, cj (3) ✓	no	prune (bj)
{c,m,j}	cm (3) ✓, cj (3) ✓, jm (2) ×	no	prune (jm)

So the pruned C_3 is just **{ {b,c,m} }**: one counter instead of four. The prune never loses a truly frequent triple (every frequent triple has all its pairs frequent), so the final L_3 is identical.

> **SLIDE NOTE — THE SLIDE'S C_3 IS NOT PRUNED (SAME FINAL ANSWER)**
> Slide 35's rule "C_3 = (pairs of L_2) + (one item of L_1)" is not wrong, because C_3 only has to contain every frequent triple, but it is looser than necessary: here it equals all C(4,3) = 4 triples of L_1, so the information in L_2 did not shrink it at all. In an exam write the slide's 4 candidates and mention the prune (or the other way round); both are acceptable.

13. **Count the triples (third pass).** A basket can contain a triple of C_3 only if it has at least 3 frequent items. That rules out B_2, B_4, B_5 and B_8 (two frequent items each). For the others, list the candidate triples contained in the basket:

Basket	Kept (in L_1)	Candidate triples it adds 1 to
B_1	b, c, m	bmc
B_3	b, c, m (n ignored)	bmc
B_6	b, c, j, m	bmc, bmj, bcj, cmj
B_7	b, c, j	bcj

Tally the triples:

Triple	Baskets that contain all three items	Support	≥ 3 ?
{b,m,c}	B_1, B_3, B_6	**3**	**yes (exactly s)**
{b,m,j}	B_6	**1**	no
{b,c,j}	B_6, B_7	**2**	no
{c,m,j}	B_6	**1**	no

Sanity check: 3 + 1 + 2 + 1 = 7 = 1 + 1 + 4 + 1 (triples added per basket) ✓. The three triples the textbook prune would have removed have supports 1, 2, 1, so they fail the count anyway – confirming that the prune would have been safe.

14. **Filter → L_3 = the triples with support ≥ 3.** Only {b,m,c} (3) survives.
L_3 = { {b,c,m} } (Slide 35, last line; Slide 18 writes the same set as {m,c,b}). Together, $L_2 \cup L_3$ are exactly the five frequent itemsets of size ≥ 2 listed on Slide 18.

15. **When does A-Priori stop?** The loop ends when a level has no candidates. Slide 35 stops at L_3 and does not say what comes next. *Beyond the slides:* with the slide's rule (extend each set of L_3 by an item of L_1), {b,c,m} + j gives the single candidate C_4 = { {b,c,j,m} }. Counting it needs a fourth pass: it occurs only in B_6, support 1 < 3, so L_4 = ø and the algorithm stops after 4 passes. With the textbook prune, {b,c,j,m} is not even a candidate, because three of its four 3-subsets ({b,c,j}, {b,j,m}, {c,j,m}) are not in L_3; then C_4 = ø and 3 passes are enough (Slide 36: "one pass for each k"). Either way L_4 = ø. In total there are 4 + 4 + 1 = **9 frequent itemsets** (4 single items, 4 pairs, 1 triple).

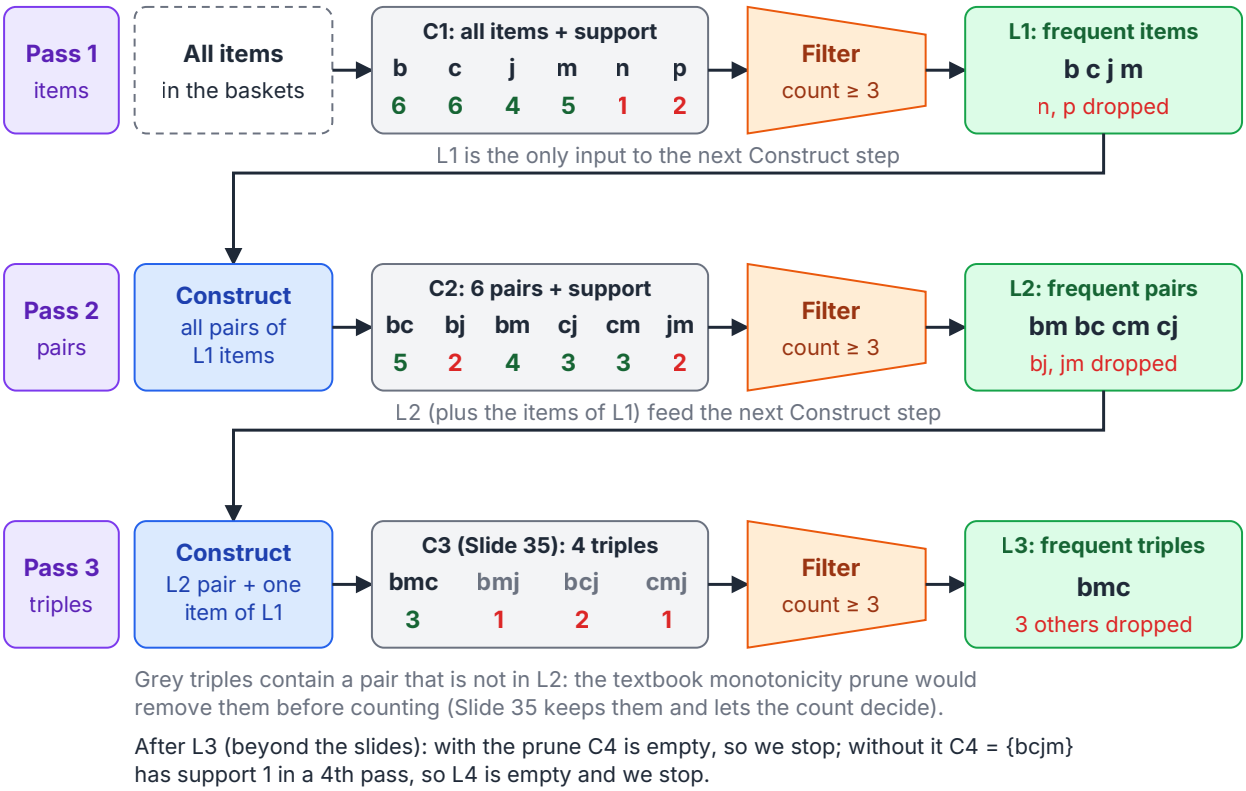


Figure L4-Q5-1. The A-Priori pipeline of Slide 34 filled in with the numbers of Slide 35. Green numbers: support ≥ 3 (survive); red: support < 3 (dropped). Each row is one pass over the baskets; the Construct steps use only L_k and need no data pass.

16. **Summary of the whole run (s = 3).**

k	Candidates C_k	Supports counted in pass k	Frequent L_k (support ≥ 3)
1	`{b} {c} {j} {m} {n} {p}`	b 6, c 6, j 4, m 5, n 1, p 2	**`{b} {c} {j} {m}`**
2	`{b,c} {b,j} {b,m} {c,j} {c,m} {j,m}`	bc 5, bj 2, bm 4, cj 3, cm 3, jm 2	**`{b,m} {b,c} {c,m} {c,j}`**
3	`{b,m,c} {b,m,j} {b,c,j} {c,m,j}` (slide); with textbook prune only `{b,m,c}`	bmc 3, bmj 1, bcj 2, cmj 1 (pruned version counts only bmc = 3)	**`{b,c,m}`**
4	none with the prune; `{b,c,j,m}` (support 1) without it	–	∅ – stop

17. **Part (d), memory – Pass 1 (Slides 31–32).** Main memory holds only the *item counters*: one per item (6 here), i.e. memory proportional to the number of items. When the pass ends, the counters are reduced to the *list of frequent items* {b, c, j, m}, and the rest of the memory is idle in Pass 1 (Slide 38 asks whether that idle memory can be used; that is the next topic of the lecture).
18. **Pass 2 memory (Slides 31–32).** Main memory holds (i) the *list of frequent items* – "so you know what must be counted" – and (ii) counters *only for pairs of frequent items*: memory proportional to the *square of the number of frequent items*, not of all items. Here that is 6 pair counters instead of 15. Pairs containing `n` or `p` get no counter, because by the contrapositive of monotonicity (Slide 30, see "Why dropping n and p is safe" above) they can never reach support 3, so counting them would waste memory and could not change L_2.

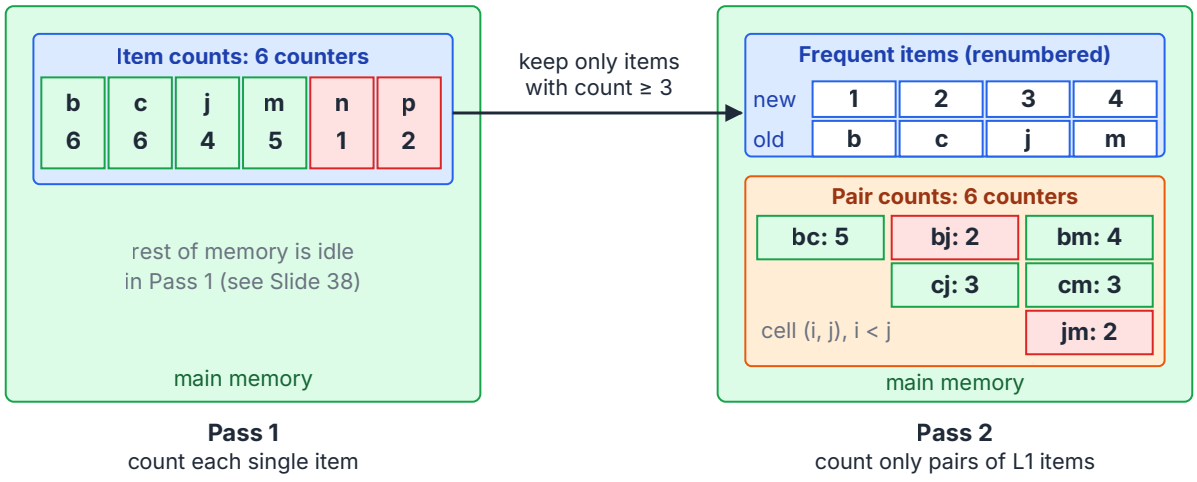


Figure L4-Q5-2. Slides 32–33 filled in with this example's numbers. Pass 1 needs only 6 item counters. Pass 2 stores the 4 frequent items (renumbered 1–4, with a table of the old item names) and a triangular matrix of 6 pair counters instead of 15.

19. **The triangular-matrix trick (Slide 33) and the byte count.** To store the pair counts in a compact triangular matrix, *re-number* the frequent items 1, 2, 3, … (here b → 1, c → 2, j → 3, m → 4) and keep a small table "new number → old item" (the "Old item #s" box on the slide). The matrix then has $n(n-1)/2$ = 6 cells for n = 4 frequent items, as in the figure. Slide 33 says this "may save space compared with storing triples" [i, j, count]. With the byte sizes of Slides 24–27 (4-byte counts, 12 bytes per *occurring* pair in the triples method) the toy numbers are:

Memory for the pair counters	Count all pairs of all 6 items	A-Priori Pass 2 (4 frequent items)
Triangular matrix, 4 bytes per possible pair	15 × 4 = 60 bytes	**6 × 4 = 24 bytes**
Triples [i, j, count], 12 bytes per occurring pair	12 occurring pairs × 12 = 144 bytes	**6 occurring pairs × 12 = 72 bytes**

(Slide 27: the triples method wins only if fewer than 1/3 of the possible pairs occur; here 12 of 15 and 6 of 6 occur, so the triangular matrix is the better choice both times.) These numbers are tiny, but the saving is the point for big data: with 10^5 items Slide 24 needs 5×10^9 pair counters = 20 GB if every pair is counted. A-Priori's price is an extra pass over the data; its gain is memory.

20. **Cost summary (Slide 36).** A-Priori makes **one pass for each k** (itemset size) and needs room in main memory to count every candidate of C_k (6 counters in Pass 2 here; 4, or 1 with the prune, in Pass 3). For typical market-basket data and a reasonable support (e.g. 1%), k = 2 requires the most memory, which is why Slides 31–33 concentrate on pairs. The slide lists one extension: lower the support s as the itemset gets bigger.

COMMON MISTAKES

- **Using > 3 instead of ≥ 3.** {c,j}, {c,m} and {b,c,m} have support exactly 3 and **are** frequent; dropping them gives a wrong L_2 and an empty L_3.
- **Building C_2 from all six items** (15 pairs) or putting `n` or `p` into any pair: C_2 must come from L_1 only.
- **Mixing up C_k and L_k.** C_k = "might be frequent, not yet counted"; L_k = "counted and passed". Do not read the 4 triples of C_3 as 4 frequent triples.
- **Building C_{k+1} from C_k instead of from L_k.**
- **Deleting a basket when one of its items is ignored.** B_3 = {m,c,b,n} still adds bc, bm, cm in Pass 2; it just adds nothing involving `n`.
- **Using B_3 = {m, b} from Slides 12–15** instead of Slide 18's {m,c,b,n}: that gives c = 5, {b,c} = 4, {c,m} = 2, {b,c,m} = 2, so L_3 = ø – a different answer.

EXAM TIP

Lay your work out as the slide does: C_1 → counts → L_1 → C_2 → counts → L_2 → C_3 → counts → L_3. Write the **count next to every candidate** together with the test "count ≥ 3", including the failing ones (2, 2, 2, 1, 1, ...), and check each pass with a sum (24 for items, 19 for pairs, 7 for triples). If you slip on one count, the steps that are correct *given your own earlier numbers* can still earn partial credit, but only if the numbers are written down. Always say which pass builds which set: Pass k counts C_k, the Filter gives L_k, the Construct step turns L_k into C_{k+1}.

Final answer (s = 3): C_1 = {b, c, j, m, n, p} with supports b 6, c 6, j 4, m 5, n 1, p 2, so L_1 = {b, c, j, m}. C_2 = {bc, bj, bm, cj, cm, jm} with supports 5, 2, 4, 3, 3, 2, so L_2 = {{b,m}, {b,c}, {c,m}, {c,j}}. C_3 = {{b,m,c}, {b,m,j}, {b,c,j}, {c,m,j}} (Slide 35 method: L_2 + L_1) with supports 3, 1, 2, 1, so L_3 = {{b,c,m}}; with the textbook monotonicity prune C_3 = {{b,m,c}} only, same L_3. L_4 = ø (3 passes with the prune, 4 without). 9 frequent itemsets in total. Memory: Pass 1 = item counters only (6); Pass 2 = list of frequent items + counters only for pairs of frequent items (6 instead of 15); n and p are ignored because, by monotonicity, no pair containing them can reach support 3.

Where the marks come from: (1) Definitions: support = number of baskets containing the set; frequent iff ≥ s = 3 (ties kept); meaning of C_k and L_k; the construct → count → filter loop; monotonicity and its contrapositive. (2) Pass 1: C_1 with all six counts (6, 6, 4, 5, 1, 2) and L_1 = {b,c,j,m}. (3) C_2 = the 6 pairs of L_1, justified by monotonicity (no pair with n or p). (4) The six pair counts (5, 2, 4, 3, 3, 2) and L_2 with {b,j} and {j,m} dropped. (5) C_3 built from L_2 and L_1 (4 triples) and, if you add it, the "all pairs in L_2" prune down to {b,c,m}. (6) The triple counts (3, 1, 2, 1) and L_3 = {b,c,m}. (7) Part (d): Pass 1 = item counters; Pass 2 = frequent-item list + counters only for pairs of frequent items (memory ∝ square of the number of frequent items); the re-numbering trick; why n and p are ignored; one pass per k.

Slide check: matches the slide's answer. Slide 35 prints C_1, L_1, C_2, L_2, C_3 and L_3; every set above is identical (only the order of the elements may differ), and every support count was recomputed by program (the slide prints no counts). No error in the results; slide notes: (i) Slide 35 does not reprint the baskets; they are those of Slide 18, whose B_3 differs from Slides 12–15. (ii) The slide's C_3 (4 triples) is not reduced by the monotonicity test; the textbook version would keep only {b,c,m}. Both give the same L_3. (iii) Slide 34 leaves the construction of C_3 "to be explained"; the only explanation is the one-line footnote of Slide 35, which does not mention the prune. (iv) Slide 34 prints "support > s" with an underlined >, which must be read as ≥ (as on Slide 31).

Now apply A-Priori to a fresh data set (cover the solution first).

Quiz L4-Q8 A-Priori on integer baskets (threshold 3) Lecture 4 · Slides 78–80

QUESTION (AS ON THE SLIDES)

Slide 78 — "Quiz: Apriori Algorithm". Compute frequent pairs for the baskets below with the Apriori algorithm. Assume threshold $s = 3$.

Basket	Items
a)	{1, 2, 4, 5, 8, 9}
b)	{1, 4, 7, 8, 9}
c)	{1, 2, 5, 9}
d)	{1, 2, 5, 7, 8}

Watch out: the letters a)–d) on the slide label the four **baskets**. They are not four separate sub-questions: there is **one** question about one data set of 4 baskets. (Slides 79 and 80 show the same slide again with the slide's answer added step by step: Pass 1 on Slide 79, Pass 2 on Slide 80. We compare with it at the end, so try the question first.)

(a) Pass 1 — count every single item and find the frequent items.

(b) Pass 2 — count only the candidate pairs (both items frequent) and find the frequent pairs. This is what the slide asks for.

(c) Practice (our extension, not on the slides): show that no pair containing item 4 or item 7 is ever counted (the memory saving), then run **pass 3**: build the candidate triples from the frequent pairs, prune them with monotonicity, count the survivors and list the frequent triples.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Imagine a club that met 4 times, and you want to know which *pairs of members* turn up together at least 3 times. Before you tally every possible pair of members (there could be millions), ask one cheap question first: **did this person even come to 3 meetings?** Someone who came to only 2 meetings can never be seen with *anybody* at 3 meetings, so every pair containing that person can be thrown away without counting it. That is the whole idea of A-Priori: **pass 1** asks the cheap question about single items, **pass 2** tallies only the pairs made of items that passed the test.

FORMULAS AND RULES USED IN THIS QUIZ

Count (support) of an itemset I: the number of baskets that contain *every* item of I (Slide 8). **Frequent:** I is frequent $\Leftrightarrow$ count(I) $\geq s$; here $s = 3$ and there are 4 baskets, so an itemset must lie in at least 3 of the 4 baskets (Slide 79: "count greater or equal 3").
Monotonicity (Slide 30): if an itemset appears in at least s baskets, so does every subset of it. Contrapositive for pairs: if item i appears in fewer than s baskets, no pair containing i can appear in s baskets.
A-Priori (Slide 31): pass 1 counts single items and keeps those with count $\geq s$. Pass 2 counts *only* the pairs whose two items are both frequent.
Counting pairs (Slide 24): n items give $C(n, 2) = n(n-1) \div 2$ pairs; a basket with k items generates $C(k, 2) = k(k-1) \div 2$ pairs.
Notation (Slide 34): C_k = candidate k-itemsets (*might* be frequent), L_k = the truly frequent k-itemsets.

Part (a) — Pass 1: count the single items Lecture 4 · Slides 31, 79

1. **Formula / idea:** for every item i, count(i) = the number of baskets that contain i. One counter per distinct item, so the memory needed is proportional to the number of items (Slide 31). The items that reach count ≥ 3 form L_1, the frequent items.
2. **Read the baskets once and tick the items.** One row per basket, one column per item; the column sum is the count of that item. The distinct items in the data are 1, 2, 4, 5, 7, 8, 9 (seven items).

Basket	1	2	4	5	7	8	9	Items in the basket	size
a	✓	✓	✓	✓		✓	✓	{1,2,4,5,8,9}	6
b	✓		✓		✓	✓	✓	{1,4,7,8,9}	5
c	✓	✓		✓			✓	{1,2,5,9}	4
d	✓	✓		✓	✓	✓		{1,2,5,7,8}	5
count	**4**	**3**	2	**3**	2	**3**	**3**	= column sum	20

Check: ticks by rows 6 + 5 + 4 + 5 = 20 and by columns 4 + 3 + 2 + 3 + 2 + 3 + 3 = 20. The totals agree, so no tick was lost.

3. **Apply the rule "count ≥ 3" to every item.**

Item	Baskets containing it	Count	≥ 3 ?
1	a, b, c, d	**4**	**frequent**
2	a, c, d	**3**	**frequent (exactly 3)**
4	a, b	**2**	not frequent
5	a, c, d	**3**	**frequent (exactly 3)**
7	b, d	**2**	not frequent
8	a, b, d	**3**	**frequent (exactly 3)**
9	a, b, c	**3**	**frequent (exactly 3)**

Items 2, 5, 8, 9 have count exactly 3 = s. The slide says "greater or equal 3", so they **are** frequent.

4. **Result of pass 1.** L_1 = {1, 2, 5, 8, 9} (five frequent items). Items 4 and 7 each appear in only 2 baskets and are dropped. Pass 1 hands **one thing** to pass 2: the list of the five frequent items (the "Frequent items" box in the memory picture on Slide 32). This agrees with the slide's pass-1 answer.

Part (b) — Pass 2: count only the candidate pairs Lecture 4 · Slides 31, 80

1. **Formula / idea:** in pass 2 the candidate pairs C_2 are all pairs whose *two* items are in L_1. With $n = 5$ frequent items there are C(5, 2) = 5 × 4 ÷ 2 = **10** candidate pairs (Slide 31: memory proportional to the square of the number of frequent items only). The ten candidates are `{1,2}`, `{1,5}`, `{1,8}`, `{1,9}`, `{2,5}`, `{2,8}`, `{2,9}`, `{5,8}`, `{5,9}`, `{8,9}`. A pair is only a *candidate*: it "might be frequent". Its real count decides.
2. **Re-read the baskets; in each basket strike out the non-frequent items first (4 and 7), then generate pairs from what is left.**

Basket	Original	After striking out 4 and 7	pairs generated	Those pairs (all counted)
a	{1,2,4, 5,8,9}	{1,2,5,8,9}	C(5,2) = 10	{1,2} {1,5} {1,8} {1,9} {2,5} {2,8} {2,9} {5,8} {5,9} {8,9}
b	{1,4,~~7~~,8,9}	{1,8,9}	C(3,2) = 3	{1,8} {1,9} {8,9}
c	{1,2,5,9}	{1,2,5,9}	C(4,2) = 6	{1,2} {1,5} {1,9} {2,5} {2,9} {5,9}
d	{1,2,5,~~7~~,8}	{1,2,5,8}	C(4,2) = 6	{1,2} {1,5} {1,8} {2,5} {2,8} {5,8}
Total counter increments in pass 2			**10 + 3 + 6 + 6 = 25**	

Every pair generated here has both items in L_1, so it is one of the 10 candidates. No pair containing 4 or 7 appears anywhere in this table.

3. **Add up the tallies.** For each candidate pair, list the baskets whose pair list contains it (equivalently: the baskets that contain both items).

Candidate pair	Baskets containing both items	Count	≥ 3 ?
`{1,2}`	a, c, d	**3**	**frequent**
`{1,5}`	a, c, d	**3**	**frequent**
`{1,8}`	a, b, d	**3**	**frequent**
`{1,9}`	a, b, c	**3**	**frequent**
`{2,5}`	a, c, d	**3**	**frequent**
`{2,8}`	a, d	**2**	no
`{2,9}`	a, c	**2**	no
`{5,8}`	a, d	**2**	no
`{5,9}`	a, c	**2**	no
`{8,9}`	a, b	**2**	no
Sum of all ten counts (must equal the 25 increments of step 2)		5 × 3 + 5 × 2 = 25	

The check works: 15 + 10 = 25 increments counted, and step 2 generated 25.

4. **Keep the pairs whose count is ≥ 3.** L_2 = { `{1,2}`, `{1,5}`, `{1,8}`, `{1,9}`, `{2,5}` }: five frequent pairs out of ten candidates. Every one of them has count exactly 3, so the "≥" (not ">") matters again.
5. **Compare with the slide.** Slide 80 lists exactly `{1,2}`, `{1,5}`, `{1,8}`, `{1,9}`, `{2,5}`. They match. As an independent check, a brute-force count of *all* pairs in the data (no A-Priori, done by computer) gives the same five frequent pairs and no others, so A-Priori is *exact*: it only avoids counting pairs that cannot possibly be frequent.

COMMON MISTAKE

Both items frequent does not mean the pair is frequent. Items 8 and 9 are each frequent (count 3), but `{8,9}` appears together only in baskets a and b (count 2). The 10 candidate pairs are only the pairs that *might* be frequent; pass 2 exists to find out which ones really are. The same happens for `{2,8}`, `{2,9}`, `{5,8}` and `{5,9}`.

Part (c) — Practice: why 4 and 7 are never counted, and pass 3 Lecture 4 · Slides 24–27, 30–36

1. **Idea (monotonicity, Slide 30):** if an item appears in fewer than 3 baskets, then every pair containing it appears in fewer than 3 baskets too, because a pair can only be in a basket where each of its items is. Item 4 is in 2 baskets (a, b), so any pair with 4 is in at most 2 baskets. Item 7 is in 2 baskets (b, d), so any pair with 7 is in at most 2 baskets. Hence these pairs cannot be frequent and A-Priori never spends memory on them.
2. **Find the pairs that A-Priori never counts.** There are 7 distinct items, so C(7, 2) = 7 × 6 ÷ 2 = 21 pairs exist in principle. Only C(5, 2) = 10 of them have two frequent items. The other 21 − 10 = **11** pairs contain 4 or 7 (or both). Check by counting: 6 pairs contain 4, 6 pairs contain 7, and `{4,7}` is in both lists, so 6 + 6 − 1 = 11. The figure shows all 21 pairs as cells: green and orange cells get a counter, grey cells do not. The number in brackets in a grey cell is that pair's true count, computed separately (by program) only to prove that skipping it was safe. The largest is 2, which is below 3, exactly as monotonicity predicts.

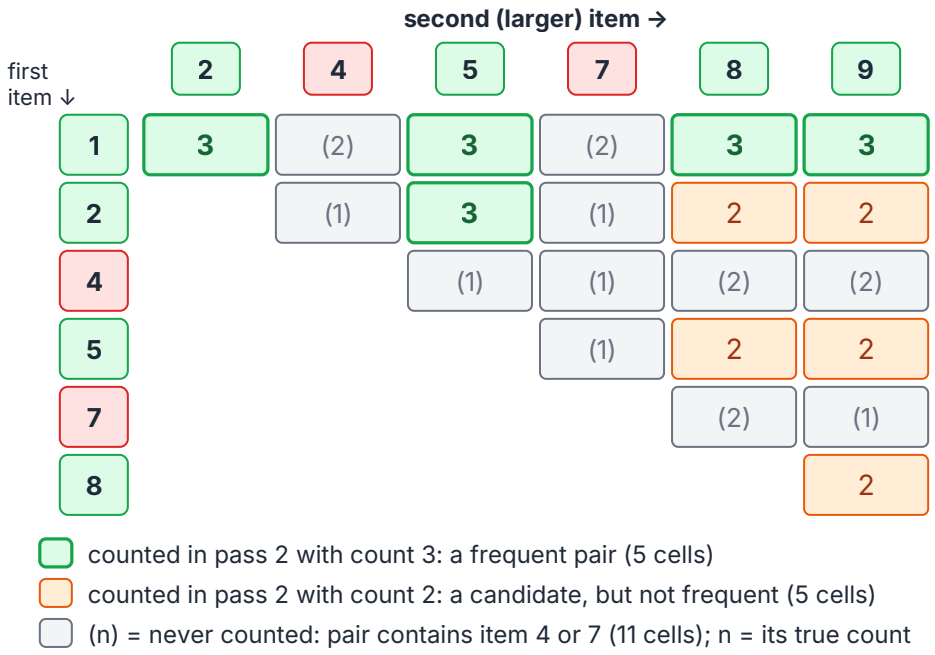


Figure — all 21 possible pairs of the 7 items (row = smaller item, column = larger item; green item labels are frequent, red ones are 4 and 7). Only the 10 cells where both items are frequent get a counter in pass 2; the 11 grey cells are never touched (their true counts in brackets were computed only to prove that skipping them was safe).

3. **Measure the saving on this tiny data set.**

	Count every pair	A-Priori pass 2
Pair counters that must exist	21	**10**
Counter increments while reading the 4 baskets (C(6,2) + C(5,2) + C(4,2) + C(5,2) = 15 + 10 + 6 + 10)	41	**25 (10 + 3 + 6 + 6)**
Memory at 4 bytes per counter (triangular-matrix method, Slides 25–27 and 33; the frequent items renumbered 1–5)	21 × 4 = 84 bytes	**10 × 4 = 40 bytes**

More than half of the counters are saved even here. On real data (Slide 24: 10^5 items give 5×10^9 pairs, about 20 GB) the saving is what makes pass 2 fit in main memory: the number of pairs grows with the *square* of the number of items, so removing infrequent items shrinks the pair table quadratically. The trick from Slide 33 for the compact triangular matrix: re-number the frequent items 1, 2, 5, 8, 9 as 1, 2, 3, 4, 5 and keep a table of the old numbers.



Figure — A-Priori main memory with the numbers of this quiz (compare the empty picture on Slide 32). Pass 1 keeps 7 item counters; pass 2 keeps the list of 5 frequent items and only 10 pair counters. The 11 pairs containing 4 or 7 have no counter at all.

4. **Pass 3: build the candidate triples C_3.** Idea (Slide 35: "generate C_k from L_{k-1} and L_1"): take each frequent pair and add one frequent item that is not already in it. L_2 has 5 pairs and 3 items can be added to each, which gives 15 additions; they collapse to **8 different triples** (for example `{1,2}` + 5 and `{1,5}` + 2 are the same triple `{1,2,5}`): `{1,2,5}`, `{1,2,8}`, `{1,2,9}`, `{1,5,8}`, `{1,5,9}`, `{1,8,9}`, `{2,5,8}`, `{2,5,9}`. (Of the C(5,3) = 10 triples of frequent items, the two missing ones, `{2,8,9}` and `{5,8,9}`, contain no frequent pair at all.)
5. **Prune with monotonicity (Slide 30, applied to triples).** If a triple is frequent, *all three* of its 2-element subsets must be frequent. So look up the three sub-pairs of each candidate in L_2 = { `{1,2}`, `{1,5}`, `{1,8}`, `{1,9}`, `{2,5}` } and cross out any triple with a sub-pair that is not there:

Candidate triple	Its three sub-pairs (✓ = in L_2)	Verdict
`{1,2,5}`	`{1,2}` ✓ `{1,5}` ✓ `{2,5}` ✓	**keep: counted in pass 3**
`{1,2,8}`	`{1,2}` ✓ `{1,8}` ✓ `{2,8}` × (count 2)	pruned
`{1,2,9}`	`{1,2}` ✓ `{1,9}` ✓ `{2,9}` × (count 2)	pruned
`{1,5,8}`	`{1,5}` ✓ `{1,8}` ✓ `{5,8}` × (count 2)	pruned
`{1,5,9}`	`{1,5}` ✓ `{1,9}` ✓ `{5,9}` × (count 2)	pruned
`{1,8,9}`	`{1,8}` ✓ `{1,9}` ✓ `{8,9}` × (count 2)	pruned
`{2,5,8}`	`{2,5}` ✓ `{2,8}` × `{5,8}` ×	pruned
`{2,5,9}`	`{2,5}` ✓ `{2,9}` × `{5,9}` ×	pruned

   Only C_3 = **{ `{1,2,5}` }** survives. Pass 3 therefore needs just **one** counter. The candidate list shrinks step by step: 35 possible triples among the 7 items → 10 among the 5 frequent items → 8 that extend a frequent pair → **1** after pruning. Beyond the slides: textbooks call this "all sub-pairs frequent" test the *prune step* of A-Priori candidate generation, and a textbook-style join that combines only pairs sharing their first item produces 6 candidates (`{1,2,5}`, `{1,2,8}`, `{1,2,9}`, `{1,5,8}`, `{1,5,9}`, `{1,8,9}`); the same pruning leaves the same single survivor. Slide 35's own example (for instance `{b,m,j}`) shows only the looser extension step.
6. **Count the survivor.** Read the baskets a third time and count the baskets containing 1, 2 and 5 together: basket a `{1,2,4,5,8,9}` ✓; basket b `{1,4,7,8,9}` × (no 2, no 5); basket c `{1,2,5,9}` ✓; basket d `{1,2,5,7,8}` ✓. Count(`{1,2,5}`) = **3** ≥ 3, so it is frequent: L_3 = { `{1,2,5}` }.

7. **Stop, and check.** A candidate 4-itemset needs *all four* of its 3-element subsets to be frequent, which would need at least four frequent triples; we have only one, so C_4 = ø and A-Priori stops after 3 passes (one pass per itemset size, Slide 36). Sanity checks by program: (i) the 7 pruned triples really have count 2 (nothing was lost); (ii) no 4-itemset reaches even 3 (the best have count 2); (iii) every subset of `{1,2,5}` is frequent, as monotonicity demands.
   Frequent itemsets overall: 5 single items + 5 pairs + 1 triple = 11.

EXAM TIP

Write the answer in the same two-pass layout as the slide: (1) a table of item counts and the list L_1; (2) the list of candidate pairs (say how many: C(n, 2)); (3) a table of their counts; (4) L_2. Always write the **counts**, not just the final list, so partial marks are available if one count is wrong. State "frequent means count ≥ 3" once at the top. A neat sentence to earn the "why" mark: "items 4 and 7 occur in only 2 baskets, so by monotonicity no pair containing them can be frequent and they need no counters."

COMMON MISTAKES

- **Treating a)–d) as four sub-questions.** They are the four baskets of one data set; the support of an itemset is counted over all four.
- **Using "> 3" instead of "≥ 3".** With "> 3" only item 1 (count 4) would survive pass 1 and no pair at all would be frequent. The slide says "greater or equal 3".
- **Counting pairs with 4 or 7 in pass 2.** The final list of frequent pairs would still be correct, but the point of A-Priori (the memory saving) is lost and the method mark for pass 2 may be lost with it.
- **Listing the 10 candidate pairs as the answer.** Candidates are what you *count*; the answer keeps only those with count ≥ 3, i.e. 5 pairs (see the warning after part (b)).
- **Forgetting a basket while counting pairs**, for example missing that basket b contributes `{1,8}`, `{1,9}` and `{8,9}`. Use the sum check (25 increments) to catch this.
- **Treating `{1,2}` and `{2,1}` as different pairs.** Itemsets are unordered, so each pair is counted once per basket.

Final answer:
Pass 1: counts 1→4, 2→3, 4→2, 5→3, 7→2, 8→3, 9→3. Frequent items L_1 = {1, 2, 5, 8, 9} (items 4 and 7 are dropped).
Pass 2: the 10 candidate pairs have counts `{1,2}` 3, `{1,5}` 3, `{1,8}` 3, `{1,9}` 3, `{2,5}` 3, `{2,8}` 2, `{2,9}` 2, `{5,8}` 2, `{5,9}` 2, `{8,9}` 2. Frequent pairs L_2 = `{1,2}`, `{1,5}`, `{1,8}`, `{1,9}`, `{2,5}` (this is the slide's answer).
Memory saving (practice): the 11 pairs containing 4 or 7 are never counted (10 pair counters instead of 21; 25 increments instead of 41).
Practice, pass 3: C_3 = { `{1,2,5}` } after pruning 7 of the 8 candidate triples; count(`{1,2,5}`) = 3, so L_3 = { `{1,2,5}` }; C_4 = ø, so the algorithm stops. All frequent itemsets for s = 3: 5 single items, 5 pairs, 1 triple (11 in all).

Where the marks come from: (1) state the rule "frequent = count ≥ 3" and the A-Priori idea (monotonicity: an item in fewer than 3 baskets cannot be in a frequent pair); (2) pass 1: correct single-item counts (1:4, 2:3, 4:2, 5:3, 7:2, 8:3, 9:3); (3) L_1 = {1, 2, 5, 8, 9}, with 4 and 7 removed; (4) pass 2: use only pairs of frequent items, i.e. the 10 candidates (C(5,2) = 10); (5) correct counts of those pairs (five with 3, five with 2); (6) the final list of five frequent pairs; (7) the memory-saving remark (pairs with 4 or 7 are never counted); (8) for the practice: candidate triples built from L_2, pruned by monotonicity to `{1,2,5}`, counted as 3, and the stop because C_4 is empty. The slides give no mark scheme, so this split is our suggestion; a clearly shown method keeps most of the marks even if one count slips.

Slide check: matches the slide's answer. Slide 79 (pass 1: frequent items 1, 2, 5, 8, 9) and Slide 80 (pass 2: frequent pairs {1,2}, {1,5}, {1,8}, {1,9}, {2,5}) are exactly what the full computation gives; every number here was recomputed by program. Notes (not errors): the slides print only the final lists, not the counts, so the count tables and the whole of part (c) (memory saving, pass 3) are our additions; the letters a)–d) on Slide 78 are the baskets;

Slide 34 describes C_k with "support > s", while Slides 8, 31 and 79 and this answer use "≥ s" (a minor wording slip on Slide 34).

The PCY (Park-Chen-Yu) algorithm

Lecture 4 · Slides 37–45

Slide 37 is the divider "PCY (Park-Chen-Yu) Algorithm" (no printed number). PCY is named after its three authors, Park, Chen and Yu. It is an improvement of A-Priori that makes Pass 2 smaller by using memory that A-Priori leaves idle in Pass 1.

The observation: idle memory in Pass 1

Lecture 4 · Slide 38

Observation: in Pass 1 of A-Priori most memory is idle: only the individual item counts are stored (compare Figure L4p2-4). Slide 38's question: **can we use the idle memory to reduce the memory required in Pass 2?** Slides 39–45 answer yes.

Pass 1: hash pairs into buckets, keeping only counts

Lecture 4 · Slides 39–40

BEGINNER INTUITION

You cannot give every pair its own tally sheet, so you set out, say, 100 million mailboxes. A fixed rule (the hash function) drops every pair into one mailbox, and the mailbox only keeps a tally of how many pairs were dropped in, not which pairs. Many different pairs share a mailbox. Later, a mailbox whose tally is small proves that *none* of the pairs in it can be popular.

DEFINITIONS (SLIDE 39)

Hash function h: a fixed rule that maps a pair {i,j} to a **bucket** number 0 … $B-1$; the same pair always lands in the same bucket. **Bucket**: just a counter (a few bytes, typically a 4-byte integer). **Collision**: two different pairs in the same bucket. A bucket count is the *sum* of the counts of all pairs that hash to it.

Pass 1 of PCY: in addition to the item counts, maintain a hash table with as many buckets as fit in memory. For each pair of items in each basket, hash the pair to a bucket and add 1 to that bucket's count. **For each bucket, keep just the count, not the actual pairs that hash to the bucket.**

NOT EXAMINED — CODE (UNDERSTAND, DON'T MEMORISE)

Slide 40 gives the Pass 1 pseudo-code, explained in words: for each basket, for each item in the basket add 1 to the item's count; then for each pair of items in the basket, hash the pair to a bucket and add 1 to that bucket's count. The last loop is the part marked "New in PCY"; the rest is A-Priori's Pass 1. The slide's note: pairs must be **generated from the input file**, because the file stores baskets, not pairs. A basket with b items produces $b(b-1)/2$ pairs: extra CPU work, but no extra disk pass.

SLIDE ERRATUM (DRAWING)

On Slide 40 the green curly brace marked "New in PCY" is misaligned: it touches the old line "add 1 to item's count" and stops short of the new line "add 1 to the count for that bucket". The new code is the pair loop and both lines of its body.

Worked example: PCY Pass 1 on the Slide 9 baskets (s = 3) Lecture 4 · Slides 9, 39–40

Beyond the slides, a toy hash with **5 buckets**. Number the items b=1, c=2, j=3, m=4, p=5 and use $h(\{i,j\}) = (i + j) \bmod 5$. Baskets: B_1={m,c,b}, B_2={m,p,j}, B_3={m,b}, B_4={c,j}, B_5={m,p,b}, B_6={m,c,b,j}, B_7={c,b,j}, B_8={b,c}. The 8 baskets contain 3+3+1+1+3+6+3+1 = 21 pair-occurrences.

1. **Item counts:** b=6, m=5, c=5, j=4, p=2. With s = 3, frequent items = {b, c, j, m}; p is out.

2. **True pair counts:** {b,c}=4, {b,m}=4, {c,j}=3, {c,m}=2, {j,m}=2, {b,j}=2, {m,p}=2, {b,p}=1, {j,p}=1.
3. **Hash:** bucket 0 gets {b,m} and {c,j} (1+4=5→0, 2+3=5→0); bucket 1 gets {c,m} and {b,p}; bucket 2 gets {j,m}; bucket 3 gets {b,c} and {j,p}; bucket 4 gets {m,p} and {b,j}.
4. **Bucket counts** (sum of its pairs): bucket 0 = 4+3 = 7; bucket 1 = 2+1 = 3; bucket 2 = 2; bucket 3 = 4+1 = 5; bucket 4 = 2+2 = 4. Total 21, as expected.

Observations about buckets

Lecture 4 · Slide 41

This slide is the whole logic of PCY. A bucket's count is the sum of the counts of all pairs that share it, so it is at least as large as the count of any single pair in it.

Slide 41 statement	Why	Example above
If a bucket contains a **frequent pair**, the bucket is surely **frequent** (count ≥ s).	bucket count ≥ that pair's count ≥ s.	Bucket 0 (7) holds {b,m}=4 and {c,j}=3.
Even without any frequent pair, a bucket can still be frequent (sad face).	Many infrequent pairs collide and their counts add up.	Bucket 1 = 3 ≥ s, but {c,m}=2 and {b,p}=1 are both infrequent. Bucket 4 = 4, pairs {m,p}=2, {b,j}=2.
For a bucket with total count < s, none of its pairs can be frequent (smiley).	Each pair's count ≤ bucket count < s. This is the useful direction.	Bucket 2 = 2 < 3: {j,m} is eliminated, *even though j and m are both frequent items*.

Pairs that hash to such an infrequent bucket "can be eliminated as candidates (even if the pair consists of 2 frequent items)". **Pass 2:** only count pairs that hash to frequent buckets. No truly frequent pair is ever lost (no false negatives); frequent buckets may only cause some unnecessary counting (false positives). Here a false negative means a truly frequent pair that gets wrongly thrown away, which must never happen, and a false positive means a pair that looks like a candidate but is not frequent, which is harmless because it is simply counted and later discarded; see also Slide 55 in the sampling section.

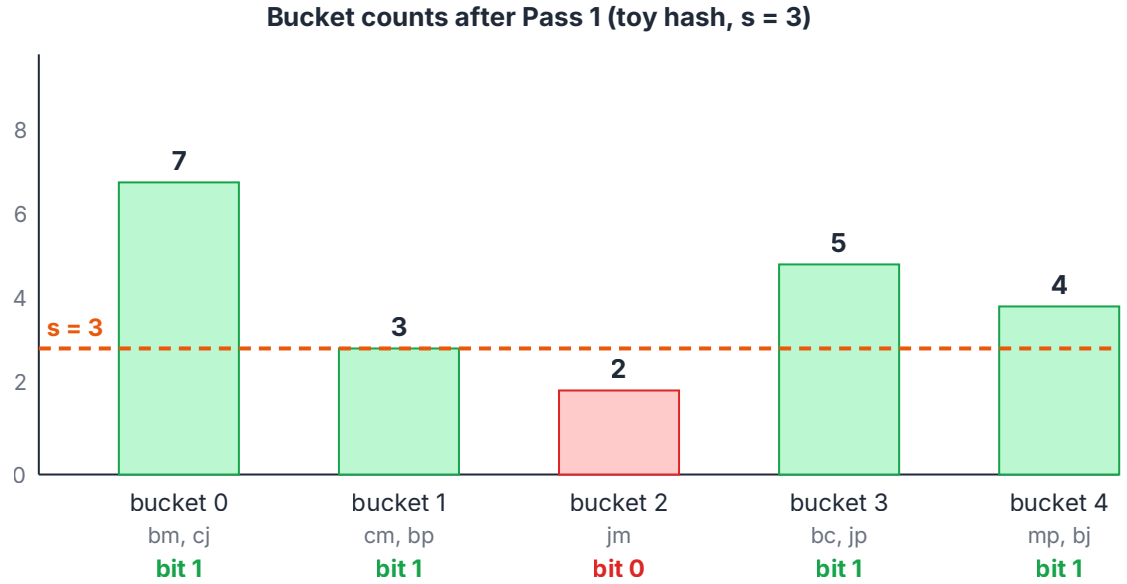


Figure L4p2-6. The worked example: five bucket counters (green = reaches s, red = below s) and the resulting bit-vector 1 1 0 1 1. Letters under each bar are the pairs sharing that bucket. Bucket 1 and 4 are frequent without containing a frequent pair; bucket 2 wipes out {j,m}.

Between the passes: buckets become a bit-vector

Lecture 4 · Slide 42

- **Replace the buckets by a bit-vector (bitmap):** 1 means the bucket count reached the support s (a **frequent bucket**); 0 means it did not.
- A 4-byte integer is 4 × 8 = 32 bits, so the bit-vector needs **1/32 of the memory** of the counters it replaces (1/32 = 0.03125). For 10^8 buckets: 400 MB as 4-byte integers, 12.5 MB as bits (illustration, Beyond the slides).
- Also decide which items are frequent (count ≥ s) and **list them for the second pass**, exactly as in A-Priori.

SLIDE ERRATUM (BOUNDARY)

Slide 42 says 1 means the count "exceeded" s, which suggests > s. The correct rule, consistent with Slide 41 ("less than s" is eliminated) and the definition of frequent, is **count ≥ s**: a bucket with count exactly s gets bit 1, because one pair with exactly s occurrences would produce it. Also, "1/32" assumes 4-byte counters.

Pass 2: the candidate-pair test

Lecture 4 · Slide 43

DEFINITION: CANDIDATE PAIR IN PCY (SLIDE 43)

In Pass 2 count all pairs {i, j} that meet **both** conditions:
1. Both i and j are **frequent items** (from Pass 1's item counts);
2. The pair {i, j} hashes (with the same hash function as in Pass 1) to a bucket whose bit in the bit-vector is **1** (a **frequent bucket**).

Both conditions are necessary for the pair to have a chance of being frequent. Each condition can only discard pairs that cannot be frequent, so no truly frequent pair is lost. Pairs that pass are only candidates; their counts still decide. A-Priori uses condition 1 alone, so PCY counts fewer pairs.

Worked example: PCY Pass 2 on the same data Lecture 4 · Slides 41–43

1. Frequent items: {b, c, j, m}. A-Priori would count all C(4,2) = 6 pairs: bc, bj, bm, cj, cm, jm.
2. Bit-vector (buckets 0..4) = 1 1 0 1 1. Test each pair's bucket: bc→3 (1) keep; bj→4 (1) keep; bm→0 (1) keep; cj→0 (1) keep; cm→1 (1) keep; jm→2 (0) **drop**.
3. PCY counts 5 pairs instead of 6. Their true counts: bc=4, bm=4, cj=3, bj=2, cm=2. Frequent pairs: {b,c}, {b,m}, {c,j}, the same answer as A-Priori.

With only 5 buckets and 21 increments the average bucket count is 4.2, close to s, so pruning is weak. Real PCY uses as many buckets as memory allows (millions), giving small average counts and strong pruning.

COMMON MISTAKE

Applying only the bucket test. Slide 41 states only the bucket rule, but Slide 43 adds that both items must also be frequent. Also, do not store the pairs themselves in Pass 1: only counts.

Main-memory picture and memory details

Lecture 4 · Slides 44–45

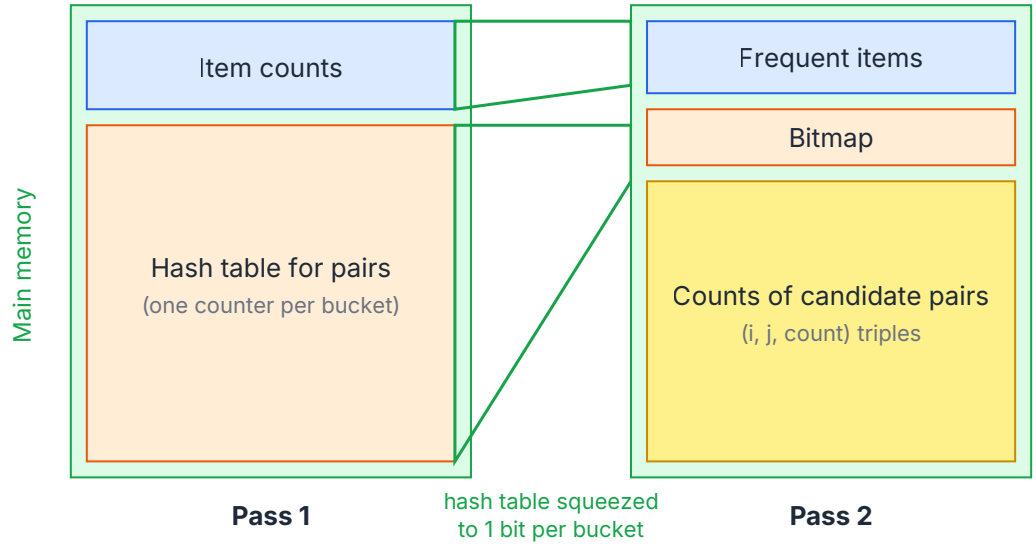


Figure L4p2-7. Main memory in PCY (Slide 44, schematic; the bitmap is drawn larger than its true 1/32). Pass 1 fills the idle memory with a hash table of bucket counts. Between the passes item counts shrink to a list of frequent items and the hash table to a bitmap. Pass 2 holds the list, the bitmap and counters for only the surviving candidate pairs (a new structure).

Slide 45 gives two practical points.

1. **Buckets require a few bytes each; the number of buckets is O(main-memory size).** A bucket is only a counter and holds no pair identity, so the number of buckets grows in proportion to the memory you have. Use all spare memory for as many buckets as possible: a small average count per bucket gives better pruning.
2. **On the second pass a table of (item, item, count) triples is essential.** The surviving candidate pairs are scattered, so the compact triangular matrix (4 bytes per pair) no longer works: the matrix needs a cell for *every* pair, including the eliminated ones. You must store triples, about 12 bytes per candidate pair.

THE 2/3 RULE (SLIDE 45)

The hash table must eliminate approximately **2/3 of the candidate pairs** for PCY to beat A-Priori. Reason: let C be the number of A-Priori's candidate pairs (matrix, 4 bytes each = $4C$). If PCY keeps a fraction f, it stores $12 \cdot f \cdot C$ bytes (triples). PCY wins when $12f \cdot C < 4C \Leftrightarrow f < 1/3$: it must **keep fewer than one third**, i.e. eliminate more than two thirds (the same 1/3 break-even as Slide 27). Check: $12 \times (1 - 2/3) = 4$ bytes per original candidate pair.

Worked example: the 2/3 rule with numbers Lecture 4 · Slide 45

Beyond the slides, illustrative. A-Priori has $C = 10^9$ candidate pairs: matrix $4 \times 10^9 = 4$ GB. PCY eliminates 50 %: 5×10^8 pairs × 12 = 6 GB, worse than A-Priori. PCY eliminates 80 %: $2 \times 10^8 \times 12 = 2.4$ GB, better. The break-even is at 2/3 eliminated (about 3.33×10^8 pairs kept × 12 = 4 GB).

SLIDE ERRATUM (PRECISION)

"Approx. 2/3" is exact only for 4-byte vs 12-byte sizes, and it ignores the extra memory taken by the bitmap and the frequent-item list, so the correct reading of "to beat A-Priori" is "more than 2/3". The slide does not explain where 2/3 comes from; it is the 12-vs-4 byte ratio.

QUICK RECAP: PCY

- Pass 1: count items *and* hash every pair to a bucket counter (counts only).
- Between: counters → bit-vector (1 = count ≥ s, 1/32 of memory); items → list of frequent items.
- Pass 2: count a pair only if both items are frequent *and* its bucket bit is 1; store (i, j, count) triples.
- Frequent pair ⇒ frequent bucket; frequent bucket ⇏ frequent pair; bucket count < s ⇒ none of its pairs is frequent.
- Needs to eliminate more than 2/3 of A-Priori's candidates to win.

EXAM TIP

If asked "why is the hash table converted to a bitmap?" answer: after Pass 1 only the yes/no fact "count ≥ s" matters; a bit replaces a 32-bit counter (1/32 of the memory), freeing memory for the Pass 2 candidate counters. If asked "why can PCY not lose a frequent pair?" answer: a pair's count is at most its bucket's count.

Try it yourself first: explain how the bitmap is used and why the hash map becomes a bitmap.

Quiz L4-Q9 PCY: the bitmap — how it is used and why

Lecture 4 · Slides 81–82 (theory on Slides 39–45)

QUESTION (AS ON THE SLIDES)

Slide 81 — "Quiz: PCY algorithm" (the slide's own answer is on Slide 82). PCY = Park–Chen–Yu, the three authors of the algorithm; it is a smarter version of A-Priori for finding frequent *pairs*.

(a) Describe how the bitmap is used in PCY algorithm.

(b) Why is the hash map in main memory from Pass 1 transformed into a bitmap in PCY algorithm?

The slide also shows a main-memory figure. **Pass 1** holds "Item names to integers", a table 1 … n of "Item counts" and a large "Hash table for bucket counts". **Pass 2** holds "Item names to integers", a table 1 … n of "Frequent items", a thin "Bitmap" strip and a "Data structure for counts of pairs". No numbers are given: this is a pure **explain / theory** question, so every correct idea you write earns part of the marks. ("Hash map" on the slide means the hash table of bucket counts.)

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION — MAILBOXES AND YES/NO FLAGS

A library wants to know which *pairs* of books are borrowed together at least s times. There are far too many possible pairs to give each one its own tally, so the librarian uses a rule (a **hash function**) that drops every pair into one of a fixed number of **mailboxes** and keeps only a **tally per mailbox** — not the pairs themselves. A mailbox with fewer than s tally marks cannot contain even one pair that reached s, so **all pairs of that mailbox are cleared** without ever being counted one by one. When the day is over, the tally *numbers* have done their job: one **yes/no flag per mailbox** ("could something frequent be in here?") is all that is needed. That row of flags is the **bitmap**. A tally needs a whole 32-bit number, a flag needs 1 bit — that is the "1/32". In the second pass you look at the flag before you give a pair its own counter.

DEFINITION — THE WORDS USED IN THIS ANSWER

- **Hash function** h: a fixed rule that turns a pair $\{i, j\}$ into a **bucket** number 0, 1, …, $B - 1$. The same pair always lands in the same bucket. A bucket is just a counter; **different pairs can share a bucket** (a *collision*).
- **Frequent** (Slides 8, 31): count ≥ s. A **frequent bucket** (Slides 42–43) is a bucket whose total count is ≥ s.
- **Bitmap** (= **bit-vector**): one bit per bucket, in bucket order. **Candidate pair** (Slide 43): a pair that PCY will give a counter to in pass 2.

FORMULAS AND RULES USED IN THIS QUIZ

Bucket count (Slides 39–40): count(b) = number of pair-occurrences hashed to bucket b = $\sum$ count({i, j}) over all distinct pairs with $h(\{i, j\}) = b$. Hence for every pair: **count({i, j}) ≤ count(h({i, j}))**.
Bitmap (Slide 42): bit[b] = 1 if count(b) ≥ s, otherwise 0.
Candidate pair (Slide 43): {i, j} is counted in pass 2 ↔ (1) i and j are both frequent items **and** (2) bit[h({i, j})] = 1.
Space (Slides 42, 82): 1 bit ÷ 32 bits (a 4-byte integer) = $\frac{1}{32}$. **Break-even (Slide 45):** the hash table must eliminate about $\frac{2}{3}$ of the candidate pairs for PCY to beat A-Priori.

1. **Idea — what problem does the bitmap solve? (Slides 36, 38, 46).** Main memory is the bottleneck of frequent-itemset mining (Slide 46). In pass 1 of A-Priori only one counter per item is stored, so most of the memory sits **idle** (Slide 38), yet pass 2 needs a counter for every pair of frequent items, and for typical market-basket data the case $k = 2$ needs the most memory (Slide 36). PCY asks: *can the idle memory of pass 1 reduce the memory needed in pass 2?* Yes: it uses that memory for a **hash table of buckets** that acts as a cheap pre-filter on pairs, and the **bitmap** is the compact form in which this pre-filter is carried from pass 1 into pass 2.
2. **Pass 1 of PCY (Slides 39–40).** Read every basket once and, for each basket, do two things (Slide 40 shows this as pseudo-code; here it is in words):
   - add 1 to the counter of every **item** in the basket (exactly as in A-Priori);
   - *new in PCY:* generate every **pair** of items of the basket (m items give $m(m-1)/2$ pairs, e.g. 6 items give 15 pairs; the pairs are not in the input file, they must be generated), apply h to the pair and add 1 to the counter of that **bucket**.

   The hash table has "as many buckets as fit in memory", and for each bucket we keep **only the count, never the pairs** that hashed there (Slide 39). All pairs are hashed because, during pass 1, we do not yet know which items will turn out to be frequent.
3. **Why a bucket count is useful: three observations (Slide 41).** Because a bucket count is the *sum* of the counts of all pairs in it (see the formula box):

Statement	True?	Why
If a bucket contains a **frequent pair**, the bucket is surely **frequent**.	**always**	Bucket count ≥ that pair's count ≥ s.
If a bucket is frequent, it contains a frequent pair.	not necessarily (the "sad face" on the slide)	Several small pairs, each below s, collide and add up to ≥ s. Such a "false-alarm" bucket still gets bit 1.
If a bucket's total count is **less than s**, **none of its pairs can be frequent** (the "happy face").	**always**	Contrapositive of the first row. Those pairs are **eliminated as candidates, even if both items are frequent**.

   Only the third statement is used to prune, so PCY errs only on the safe side: it may keep a pair that is not frequent, but it never throws away one that is.
4. **Between the passes: build the bitmap (Slide 42).** Wait until the **last** basket of pass 1 has been read (a bucket count can still grow until then), then scan the hash table once and replace it by a **bit-vector**, one bit per bucket: **1** if count ≥ s (a **frequent bucket**), **0** if count < s. In the same scan, look through the item counters and write down the list of **frequent items** (count ≥ s) "for the second pass" (last bullet of Slide 42). The item counters and the 32-bit bucket counters can now be thrown away; only the frequent-item list and the bitmap travel to pass 2. (Slide 42 words the 1-bit as "exceeded the support"; the correct condition is "reached", count ≥ s — see the erratum box.)
5. **(a) Pass 2 — how the bitmap is used (Slide 43).** Read all baskets a second time. For every pair {i, j} in a basket, give it a counter and count it **only if it is a candidate pair**, i.e. only if **both** conditions hold:
   1. **Condition 1:** both i and j are **frequent items** (look them up in the list of frequent items);

2. **Condition 2:** the pair **hashes to a bucket whose bit is 1**, i.e. compute $b = h(\{i, j\})$ and test bit[b] = 1 (a frequent bucket). Pass 2 must use **the same hash function** as pass 1, otherwise the bits would describe different groups of pairs.

So the bitmap is a **constant-time look-up table** ("is this pair's bucket frequent?") placed in front of the pair counters. A pair that fails either test gets no counter and uses no memory (Figure L4-Q9-1). At the end of pass 2 the counters of the candidates are exact, and the candidates with count ≥ s are the frequent pairs.

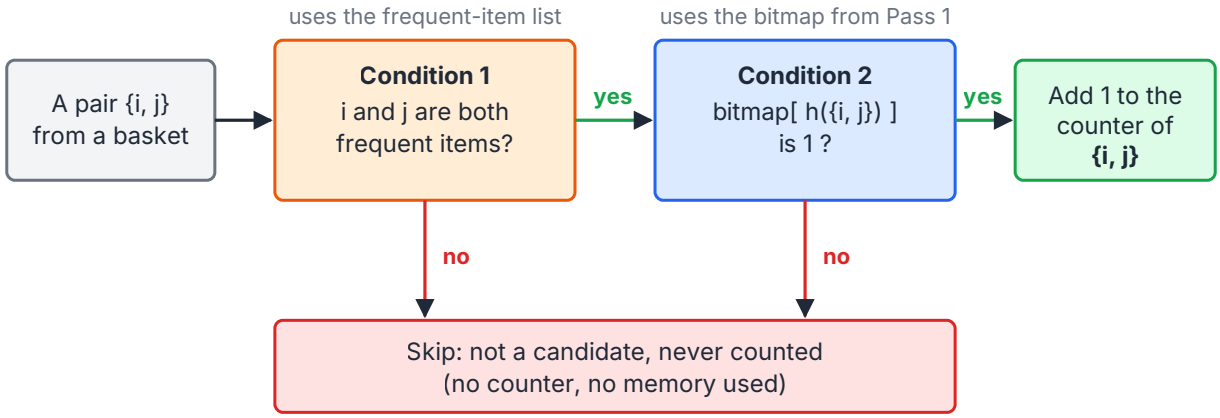


Figure L4-Q9-1 — What pass 2 does with every pair {i, j} of every basket (Slide 43). Both tests must be passed before a counter is used.

"Both conditions are necessary for the pair to have a chance of being frequent" (Slide 43): a frequent pair has frequent items (monotonicity, Slide 30) and lies in a frequent bucket (statement 1 of step 3). They are **not sufficient**: a candidate can still end with a count below s, which is why pass 2 still counts candidates exactly.

6. **(b) Why the pass-1 hash table is turned into a bitmap (Slides 42, 82).**
   - **Only a yes/no fact is needed.** After pass 1 the only question asked about a bucket is "was its count ≥ s?" (the pruning rule is statement 3 of step 3). The exact count is never used again, so **one bit is enough** and nothing that pass 2 needs is lost.
   - **It shrinks the table 32 times.** A bucket count is a 4-byte integer = 4 × 8 = **32 bits**; its bit replaces it. With B buckets the counts need $32B$ bits, the bitmap needs B bits: $B \div 32B = \frac{1}{32}$ (about 3.1 %; a saving of 31/32 ≈ 96.9 %). Example (our numbers): $B = 10^8$ buckets need $10^8 \times 4$ bytes = 400 MB as counts, but only 10^8 bits ÷ 8 = 12.5 MB as a bitmap (400 ÷ 12.5 = 32). The "1/32" assumes 4-byte counts, as on the slide.
   - **Memory is the bottleneck, and pass 2 needs the room.** In pass 2 main memory must hold the list of frequent items, the bitmap and — the big part — the **counters of the candidate pairs** (Slide 44). Keeping the full 32-bit table would take 32 times more memory for information that is no longer needed and would steal space from those counters. The freed memory is what lets PCY count pairs that A-Priori could not fit.
   - **Why the table can be so large in pass 1 in the first place:** the memory is otherwise idle (Slide 38) and a bucket needs only a few bytes, so the number of buckets is O(main-memory size) (Slide 45) — many buckets, few collisions, good pruning. The bitmap then compresses that big table to a size pass 2 can afford. Big table in pass 1 + tiny summary in pass 2 is the whole design.
7. **The memory picture (Slide 44, and the figure on Slide 81).** Read it left to right: the **item counts** of pass 1 become the **list of frequent items** of pass 2; the big **hash table** becomes the tiny **bitmap**; and the space saved holds the **counts of candidate pairs**. (The figure on Slide 81 also shows a small "Item names to integers" table that turns item names into integers 1 … n so that they can index the arrays; it is present in both passes.) The bitmap is the only thing that carries information about *pairs* from pass 1 into pass 2.

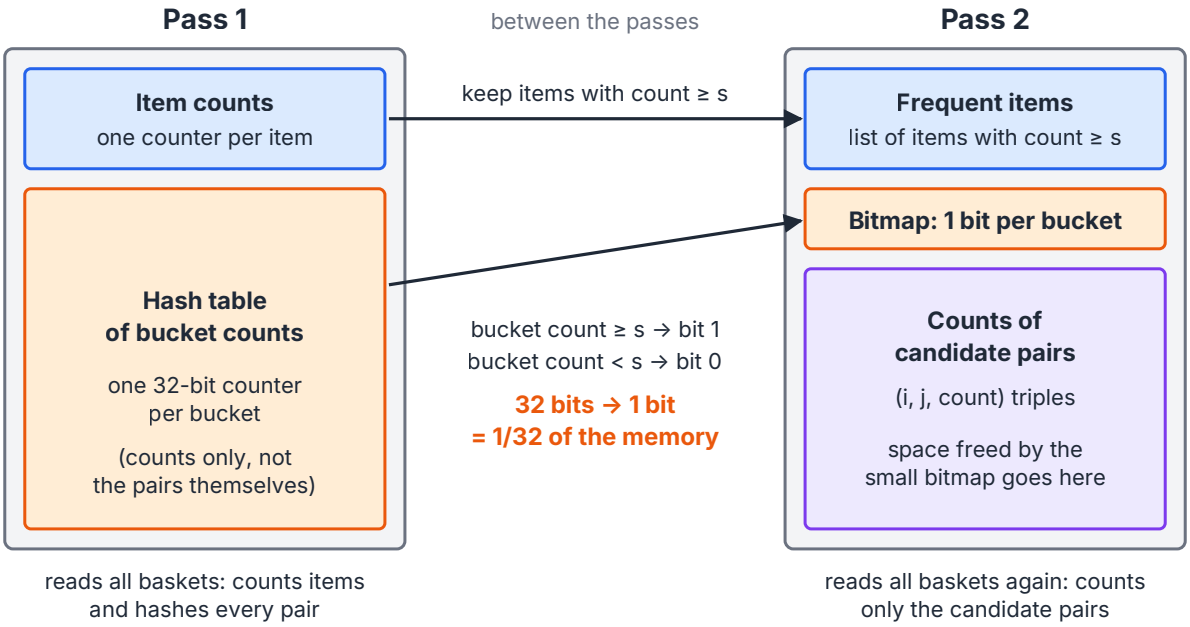


Figure L4-Q9-2 — Main memory in PCY (redrawn from Slides 44 and 81). Between the passes the hash table is summarised as a bitmap and the item counts as a list of frequent items.

	A-Priori (Slides 30–33)	PCY (Slides 38–45)
Pass 1	count items	count items **and** hash every pair into a bucket, counting buckets
Between passes	list the frequent items	list the frequent items **and** turn the bucket counts into a bitmap
Pass 2: pair {i, j} is counted if	both items are frequent	both items are frequent **and** its bucket bit is 1
Pass 2 counters	triangular matrix possible (Slide 33)	(item, item, count) triples are essential (Slide 45)
Frequent pairs found	exactly the same set (PCY only counts fewer useless pairs)	

8. **Memory details: triples and the "2/3" rule (Slide 45).**
   - **Why (item, item, count) triples are essential.** A-Priori counts *all* pairs of the n frequent items, so it can use the **triangular matrix** (Slide 33): the slot of $\{i, j\}$ is computed from i and j, so only the 4-byte count is stored. PCY's candidates are an **irregular subset** picked by the hash function. A formula-based array would still reserve a slot for every eliminated pair and save nothing, so PCY stores only the surviving pairs, and each counter must say *which* pair it belongs to: **(item, item, count)**.
   - **Beyond the slides — where "2/3" comes from** (4-byte integers). A triple costs 3 × 4 = 12 bytes, a triangular-matrix slot only 4 bytes, so a PCY counter is 3 times as expensive. Let P = number of pairs of frequent items (what A-Priori counts; this is what Slide 45 calls "candidate pairs") and C = number left after the bitmap. A-Priori needs about $4P$ bytes, PCY about $12C$ bytes.

     FORMULA — WHEN DOES PCY BEAT A-PRIORI?

     $12C < 4P \leftrightarrow C < \frac{P}{3} \leftrightarrow$ fraction eliminated $1 - \frac{C}{P} > \frac{2}{3}$. This is the slide's "the hash table must eliminate approx. 2/3 of the candidate pairs for PCY to beat A-Priori".

Example: $n = 10^5$ frequent items, so $P = n(n-1)/2 \approx 5 \times 10^9$ pairs	Candidates left	Pass-2 counter memory
A-Priori, triangular matrix (4 bytes × P)	5×10^9	≈ 20 GB
PCY eliminates 0 % (12 bytes × 5×10^9)	5×10^9	≈ 60 GB (worse)
PCY eliminates 50 %	2.5×10^9	≈ 30 GB (worse)
PCY eliminates 2/3 (break-even)	$\approx 1.67 \times 10^9$	≈ 20 GB (equal)
PCY eliminates 90 %	5×10^8	**≈ 6 GB (better)**

9. **Correctness — the bitmap never loses a frequent pair.** Suppose $\{i, j\}$ is truly frequent (count ≥ s). Then i and j each appear in at least s baskets, so both are frequent items (condition 1 holds). Its bucket count is at least the pair's own count, hence ≥ s, so the bit is 1 (condition 2 holds). So every frequent pair is a candidate: PCY has **no false negatives**. Candidates that are not frequent ("false alarms", caused by collisions) are counted exactly in pass 2 and then discarded. The output is therefore exactly the A-Priori result.

COMMON MISTAKES

- **"The bit says the pair is frequent."** No. The bit belongs to a **bucket**: a 1 only says the bucket *might* contain a frequent pair (nothing is proved), a 0 proves that every pair in it is infrequent.
- **"Frequent bucket ⇒ frequent pair."** Only the opposite holds: frequent pair ⇒ frequent bucket (Slide 41). Collisions can make a bucket frequent with no frequent pair inside (Practice: buckets 0 and 4).
- **Using only one of the two conditions** in pass 2. Counting every pair of a frequent bucket would count pairs with infrequent items; counting every pair of frequent items would ignore the bitmap. Slide 43 requires **both**.
- **Using a different hash function in pass 2**; the bit positions only make sense for the function of pass 1.
- **Saying the bitmap stores the counts or the pairs, or "1/32 of the data".** It stores one bit per bucket, and it is 1/32 of the memory of the *hash table of counts* (32-bit counts → 1-bit flags), not of all main memory.
- **Thinking PCY needs fewer passes or finds different pairs.** It still takes two passes for pairs and returns the same frequent pairs as A-Priori; only the memory (the number of counted pairs) changes.

SLIDE ERRATUM

(1) Wording of the bit rule. Slide 42 says a bit is 1 when the bucket count "exceeded the support s". Read literally this means count > s, which would wrongly give bit 0 to a bucket whose count equals s exactly, although such a bucket can hold a frequent pair of count exactly s (frequent means ≥ s: Slides 8, 31, 41). The correct rule is **bit = 1 if and only if count ≥ s**. **(2) Shorthand.** The last line of Slide 41 ("Pass 2: only count pairs that hash to frequent buckets") is only half of the rule; the full candidate test has two conditions (Slide 43).

EXAM TIP

The question has two halves — **how the bitmap is used** and **why it is made** — and the slide's own answer (Slide 82) covers mostly how it is *built* and the 1/32. For full marks write both halves: **Use:** built between the passes, 1 = frequent bucket (count ≥ s), and in pass 2 a pair is counted only if both items are frequent *and* its bucket bit is 1 (same hash function). **Why:** 32-bit count → 1 bit = 1/32 of the memory, which frees main memory (the bottleneck) for the counts of candidate pairs. Add one line on correctness (a 0 bit proves infrequency, so no frequent pair is lost). Use the exact words **"frequent bucket"**, **"candidate pair"** and **"same hash function"**.

Practice — a full PCY run on the Slide-78 baskets (self-made, not from the course)

Lecture 4 · Slide 78 (data) · Slides 39–45 (method)

Data. Baskets a) {1, 2, 4, 5, 8, 9}, b) {1, 4, 7, 8, 9}, c) {1, 2, 5, 9}, d) {1, 2, 5, 7, 8}; support threshold $s = 3$ (the baskets of the A-Priori quiz; its answer is on Slides 79–80). **Hash table:** $B = 5$ buckets numbered 0–4 and a deliberately simple hash function **$h(\{i, j\}) =$ (smaller item of the pair) mod 5**, e.g. $h(\{2,8\}) = 2$ mod 5 = 2 and $h(\{8,9\}) = 8$ mod 5 = 3. (A poor hash function for real data, chosen because it is easy to evaluate by hand; see the last bullet below.)

Pass 1 (i) — item counts. Count how many baskets contain each item.

Item	1	2	4	5	7	8	9
Count	4	3	2	3	2	3	3
Frequent (≥ 3)?	**yes**	**yes**	no	**yes**	no	**yes**	**yes**

Frequent items: `{1, 2, 5, 8, 9}` (the same as on Slide 79).

Pass 1 (ii) — hash every pair of every basket. The baskets have 6, 5, 4 and 5 items, i.e. C(6,2) + C(5,2) + C(4,2) + C(5,2) = 15 + 10 + 6 + 10 = **41 pair-occurrences** (21 distinct pairs); each adds 1 to one bucket. (A real run keeps only the five counters; the pair lists below just show why each total is what it is.)

Bucket	Pairs that hash here (pair: number of baskets)	Bucket count	≥ 3 ?	Bit
0	{5,7}: 1, {5,8}: 2, {5,9}: 2	**5**	**yes**	**1**
1	{1,2}: 3, {1,4}: 2, {1,5}: 3, {1,7}: 2, {1,8}: 3, {1,9}: 3	**16**	**yes**	**1**
2	{2,4}: 1, {2,5}: 3, {2,7}: 1, {2,8}: 2, {2,9}: 2, {7,8}: 2, {7,9}: 1	**12**	**yes**	**1**
3	{8,9}: 2	**2**	no	**0**
4	{4,5}: 1, {4,7}: 1, {4,8}: 2, {4,9}: 2	**6**	**yes**	**1**

Check: 5 + 16 + 12 + 2 + 6 = 41 ✓.

Between the passes. The five 32-bit counts (5 × 32 = 160 bits) become the **bitmap** `[1, 1, 1, 0, 1]` (bucket 0 … bucket 4), 5 bits: 5 ÷ 160 = 1/32. The frequent items are listed: {1, 2, 5, 8, 9}. The counts are thrown away.

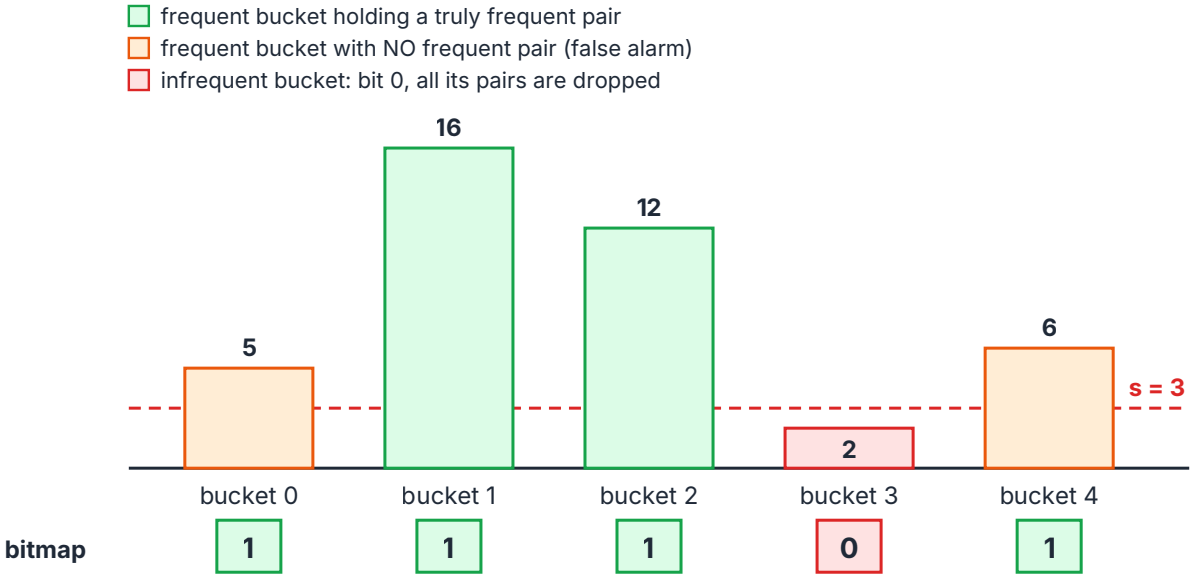


Figure L4-Q9-3 — Practice example: the pass-1 bucket counts against the threshold s = 3, and the bitmap (one bit per bucket) that replaces them between the passes. Only bucket 3 falls below the line.

Pass 2 — test every pair. A pair with item 4 or 7 fails condition 1 and is never counted. The 10 pairs of frequent items {1, 2, 5, 8, 9} pass condition 1; condition 2 looks up the bit of their bucket, and the candidates are counted exactly (the counts equal those of A-Priori on Slide 80):

Pair	Cond. 1: both items frequent?	Bucket h	Cond. 2: bit	Candidate?	Pass-2 count	Frequent (≥ 3)?
{1,2}	yes	1	1	yes	3	**yes**
{1,5}	yes	1	1	yes	3	**yes**
{1,8}	yes	1	1	yes	3	**yes**
{1,9}	yes	1	1	yes	3	**yes**
{2,5}	yes	2	1	yes	3	**yes**
{2,8}	yes	2	1	yes (false alarm)	2	no
{2,9}	yes	2	1	yes (false alarm)	2	no
{5,8}	yes	0	1	yes (false alarm)	2	no
{5,9}	yes	0	1	yes (false alarm)	2	no
{8,9}	yes	3	0	no: eliminated by the bitmap	not counted	—

Result: frequent pairs `{1,2}, {1,5}, {1,8}, {1,9}, {2,5}` — exactly the answer on Slide 80, so PCY and A-Priori agree. PCY counted **9 candidate pairs** instead of A-Priori's 10 (and made 23 counter increments in pass 2 instead of 25: the two baskets containing {8,9} no longer touch a counter).

What the example shows (tie each point to Slide 41 and Slide 43).

- The five frequent pairs sit in buckets 1 and 2, whose bits are 1: **frequent pair ⇒ frequent bucket**.
- Buckets 0 and 4 have bit 1 although none of their pairs is frequent (5 = 1 + 2 + 2 and 6 = 1 + 1 + 2 + 2 are sums of small counts): **a frequent bucket need not contain a frequent pair**. Bucket 0's bit lets the false alarms {5,8} and {5,9} through; the pairs of bucket 4 all contain item 4, so condition 1 stops them anyway.
- Bucket 3 has count 2 < 3, so its only pair {8,9} is dropped **even though 8 and 9 are both frequent items** (and indeed count({8,9}) = 2).
- **Both conditions are needed.** {8,9} passes condition 1 but fails condition 2. Conversely all 11 pairs containing item 4 or item 7 (for instance {1,4}, bucket 1) land in buckets with bit 1, so the bitmap alone would not remove them; condition 1 does.
- **The 2/3 test on this toy.** Only 1 of 10 pairs (10 %) was eliminated, far below 2/3. With 4-byte integers A-Priori's triangular matrix needs 10 × 4 = 40 bytes, PCY's triples 9 × 12 = 108 bytes (plus 5 bits). So on this tiny input PCY does **not** win on memory: the example shows the *mechanism*, not the saving. The reason: 41 pair-occurrences fall into only 5 buckets, an average of 41/5 = 8.2 per bucket, far above $s = 3$. With the more mixing hash $(i + j) \bmod 5$ the bucket counts would be 9, 8, 8, 9, 7, the bitmap 1 1 1 1 1 and nothing would be eliminated. **Beyond the slides:** PCY pays off when there are so many buckets that most of them receive fewer than s pair-occurrences — which is why the table is made as large as memory allows (Slide 45) and why "halving the number of buckets doubles the average count" is a risk (Slide 49).

Final answer:
(a) How the bitmap is used. In pass 1 PCY counts the items and hashes every pair of every basket to a bucket, adding 1 to the bucket's counter (counts only, the pairs are not stored). A frequent pair makes its bucket frequent; a frequent bucket need not contain a frequent pair (collisions); a bucket with count < s contains no frequent pair. Between the passes the hash table is summarised as a bitmap: one bit per bucket, 1 if the bucket count is ≥ s (frequent bucket), 0 otherwise, and the frequent items are listed. In pass 2 a pair $\{i, j\}$ is counted only if it is a candidate: (1) both i and j are frequent items and (2) it hashes (same hash function as pass 1) to a bucket whose bit is 1; both conditions are necessary. A pair in a 0-bit bucket cannot be frequent and is never counted. The candidates are counted exactly, and the result is the same set of frequent pairs as A-Priori. Pass-2 memory = frequent items + bitmap + counts of candidate pairs, stored as (item, item, count) triples, so the bitmap must eliminate about 2/3 of the candidate pairs for PCY to beat A-Priori.
(b) Why a bitmap. Pass 2 needs only the yes/no fact "is the bucket frequent?", not the count. Each 32-bit (4-byte) integer count is replaced by a single bit, so the bitmap takes 1/32 of the memory of the table of counts. This frees main memory (the bottleneck) for the counters of candidate pairs in pass 2, and nothing needed is lost, because every frequent pair lies in a frequent bucket.
***(Practice result, not part of the exam question:* with s = 3, 5 buckets and h = (smaller item) mod 5 the bucket counts are 5, 16, 12, 2, 6, the bitmap is 1 1 1 0 1, {8,9} is removed, 9 candidates are counted and the frequent pairs are {1,2}, {1,5}, {1,8}, {1,9}, {2,5}.*)**

Where the marks come from (our estimate — the slides give no mark scheme): (1) pass 1 of PCY: item counts plus a hash table of pair counts, counts only and no pairs stored; (2) the observations: frequent pair ⇒ frequent bucket, frequent bucket does not imply a frequent pair, bucket count < s ⇒ none of its pairs is frequent; (3) between the passes: the table becomes a bit-vector, one bit per bucket, 1 = frequent bucket (count ≥ s), 0 = not, plus the list of frequent items; (4) pass 2: count a pair only if both items are frequent *and* it hashes to a bucket with bit 1, both necessary, same hash function; (5) the memory picture: item counts + hash table in pass 1; frequent items + bitmap + counts of candidate pairs in pass 2; (6) why: 32-bit integer counts become single bits, so 1/32 of the memory; (7) what the saved memory is for: counters of candidate pairs, memory is the bottleneck; (8) extra credit: triples (item, item, count) are essential and the hash table must eliminate about 2/3 of the candidate pairs to beat A-Priori; no frequent pair is lost; a correct small example (bucket counts 5, 16, 12, 2, 6; bitmap 1 1 1 0 1). Points (1)–(2) and (4)–(5) are the "how it is used" half; (3) and (6)–(7) are the "why" half and are what Slide 82 asks for.

Slide check: matches the slide's answer but the slide's answer is incomplete. Slide 82 says: "Between the passes of PCY, the hash table is summarized as a bitmap, with one bit for each bucket. The bit is 1 if the bucket is frequent and 0 if not. Thus integers of 32 bits are replaced by single bits, and the bitmap shown in the second pass in the figure takes up only 1/32 of the space that would otherwise be available to store counts." Everything in it is included above. It does **not** say how the bitmap is *used* in pass 2 (the two candidate conditions of Slide 43) nor what the saved memory is for; the answer above adds both, plus the observations (Slide 41), the memory picture (Slide 44) and the triples/"2/3" point (Slide 45). Slide erratum: Slide 42's "exceeded the support s" should read "reached" (count ≥ s), and Slide 41's last line is shorthand (see the erratum box). The Practice example, the 12-byte versus 4-byte derivation of "2/3" and the 10^8-bucket example are our additions, verified by computer.

Refinements of PCY: Multistage and Multihash

Lecture 4 · Slides 46–51

PCY's weakness: if there are too many pairs and too few buckets, infrequent pairs collide and make many buckets reach s. Too many pairs then survive Pass 2 and use scarce memory. Both refinements below add a second, independent filter. Slide 46's reminder: **memory is the bottleneck**, so we limit the number of candidates to be counted.

Multistage: three passes, a second hash function

Lecture 4 · Slides 46–47

BEGINNER INTUITION

Airport security with two different sieves in a row. The first sieve (PCY's bitmap) lets some innocent pairs through only because they shared a mailbox with popular pairs. The second sieve re-sorts the survivors into new mailboxes using a *different* rule. An innocent pair is unlikely to be unlucky twice, so the second sieve catches many of the first sieve's mistakes. The price: one more trip through the data.

KEY IDEA (SLIDE 46)

After Pass 1 of PCY, **rehash the pairs with a different hash function**. This **requires 3 passes over the data**.

1. **Pass 1** (as in PCY): count items; hash every pair into the **first hash table**. Between passes: items → list of frequent items; first table → **Bitmap 1** (B1).
2. **Pass 2:** memory holds the frequent items and B1. Read the data again and hash pairs {i,j} into a **second hash table** with a different (independent) hash function, Hash2. Between passes: second table → **Bitmap 2** (B2).
3. **Pass 3:** count pair {i,j} **iff** (a) i and j are both frequent, **and** (b) {i,j} hashes to a frequent bucket in B1, **and** (c) {i,j} hashes to a frequent bucket in B2. Memory holds the frequent items, B1, B2 and the counters of the (now fewer) candidate pairs.

Beyond the slides (textbook *Mining of Massive Datasets*, §6.3.2): in Pass 2 only the pairs that satisfy (a) and (b) are hashed into Hash2. So each Hash2 bucket receives fewer pairs and fewer buckets reach s: that is where the second round of pruning comes from. Slide 47's caption "Hash pairs {i,j} into Hash2" does not say this, and the three conditions in the Pass 3 caption are listed with no "and"; all three must hold (the word "iff").

Multihash: two passes, several tables at once

Lecture 4 · Slides 49–50

Key idea (Slide 49): use **several independent hash tables on the first pass**. A second opinion without an extra pass: split the spare memory into two (or more) tables, each with its own independent hash function, and hash every pair into every table during the same Pass 1. Between passes make one bitmap per table. In Pass 2, a pair is a candidate only if both items are frequent and the pair hits a frequent bucket in **every** table. (This Pass 2 rule is not written on Slides 49–50; it is only implied by the figure of Slide 50.)

RISK (SLIDE 49)

Halving the number of buckets doubles the average count. The total number of pair-increments is fixed, so with half as many buckets per table each bucket gets on average twice as many pairs. "We have to be sure most buckets will still not reach count s." If so, we get a benefit like multistage, but in only 2 passes.

Worked example: halving the buckets Lecture 4 · Slide 49

Beyond the slides, illustrative numbers. 10^9 pair-increments into 10^8 buckets: average $10^9/10^8 = 10$ per bucket. Split the same memory into two tables of 5×10^7 buckets each: average $10^9/(5 \times 10^7) = 20$ per bucket. If $s = 100$, almost no bucket is frequent either way and the double filter prunes very well. If $s = 20$, a large fraction of buckets reach s and the filter barely filters.

Rough pruning arithmetic (assumes independence and ignores the shrinking of the second table in multistage): if a fraction $f = 0.3$ of buckets are frequent, an infrequent pair slips through one table with probability 0.3, through two independent tables with probability $0.3^2 = 0.09$, through three with $0.3^3 = 0.027$.

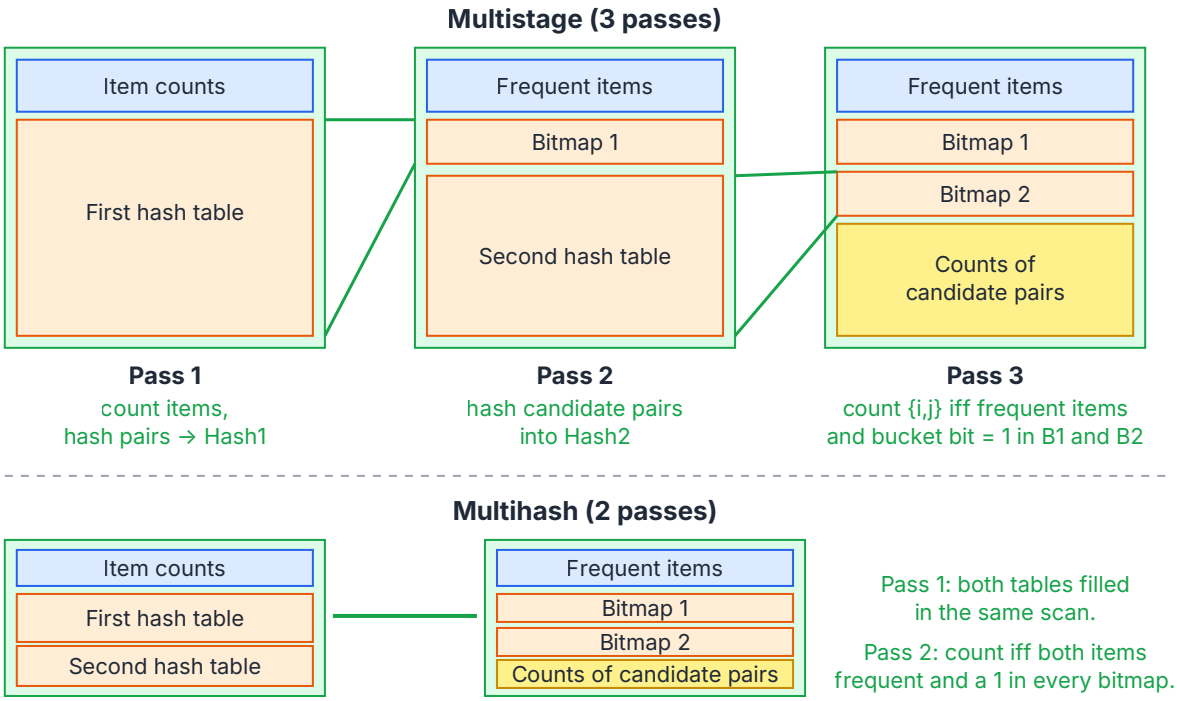


Figure L4p2-8. Slides 47 and 50 redrawn (schematic sizes). Multistage uses three passes, one hash table per pass, memory growing a bitmap per stage. Multihash splits Pass 1 memory into two tables (each about half the size of PCY's) and needs only two passes.

SLIDE ERRATUM (OMISSIONS)

Slide 49 claims "a benefit like multistage" without justification: it holds only if most buckets stay below s. Slide 50's connectors from the Item-counts and the First-hash-table boxes start at the same corner and look like one forked line; read them as two separate wedges (item counts → frequent items; each hash table → its bitmap).

Extensions and their limits

Lecture 4 · Slide 51

Either multistage or multihash can use more than two hash functions. Each extension has a cost that limits it:

- **Multistage: diminishing returns**, since the bit-vectors eventually consume all of main memory. Every added stage adds another bitmap that must stay in memory for all later passes (and costs another pass, which the slide does not say), crowding out the counters.
- **Multihash: too many hash functions makes all counts ≥ s.** Memory is split among more and more tables, so each has fewer buckets and a higher average count; eventually almost every bucket reaches s, every bucket is "frequent", and nothing is filtered. ("All" is true only in the limit; read "most or all".)

So there is an optimum number of hash functions, usually 2 or 3. Slide 52 is a black divider: "Frequent Itemsets in ≤ 2 Passes". All algorithms so far (A-Priori, PCY, multistage, multihash) need about one pass per itemset size or more; the next part (random sampling, SON; Slides 53 onward) reads the data at most twice for all sizes.

Side-by-side comparison

Lecture 4 · Slides 31–51

	A-Priori	PCY	Multistage	Multihash
Passes (for pairs)	2	2	3	2

Kept in memory, Pass 1	item counts	item counts + one hash table of bucket counts	item counts + first hash table	item counts + two (or more) hash tables, each about half the size
Kept in memory, later pass(es)	Pass 2: frequent-item list + pair counters (triangular matrix after renumbering)	Pass 2: frequent items + bitmap + (i,j,count) triples	Pass 2: frequent items + B1 + second hash table. Pass 3: frequent items + B1 + B2 + triples	Pass 2: frequent items + one bitmap per table + triples
Candidate condition for counting {i,j}	both items frequent	both items frequent **and** bucket bit = 1	both frequent **and** bit 1 in B1 **and** bit 1 in B2	both frequent **and** bit 1 in every bitmap
Benefit	simple; exact	uses idle Pass 1 memory; prunes pairs	second filter prunes more	second filter without an extra pass
Risk / limit	pair counters (about $m^2/2$) may not fit	must remove more than 2/3 of candidates (triples cost 12 bytes vs 4); too few buckets means most are frequent	extra pass; each stage adds a bitmap, diminishing returns	halving buckets doubles average count; too many tables makes all buckets frequent

All four are exact: none loses a frequent pair, because every discarded pair has an infrequent item or sits in a bucket whose total is below s.

EXAM TIP

Memorise the pattern: *PCY = A-Priori + one hash filter; Multistage = more passes, filters in sequence; Multihash = same pass, filters in parallel.* Typical questions: "describe the key idea of multistage" (second independent hash of the pairs that survived the first test, third pass counts only pairs passing both bitmaps), "what is the risk of multihash" (halving buckets doubles the average count), "why diminishing returns".

QUICK RECAP: SLIDES 19–52

- Cost = passes over the disk-resident basket file; scarce resource = main memory; pairs are the bottleneck.
- A-Priori: monotonicity; Pass 1 item counts; Pass 2 counts pairs of frequent items; renumber items for the triangular matrix; $C_k \rightarrow L_k$ one pass per k; k = 2 needs most memory.
- PCY: hash pairs into bucket counters in Pass 1, bitmap (1/32) between passes, candidate = both frequent and frequent bucket; buckets O(memory); needs to remove more than 2/3.
- Multistage: 3 passes, second hash function, B1 and B2. Multihash: 2 passes, several tables at once; halving buckets doubles the average count. Limits: bitmaps eat memory; too many tables makes everything frequent.

Try it yourself first: describe the key idea of multistage and compare it with multihash.

Quiz L4-Q10 Multistage and Multihash: key ideas — Lecture 4 · Slides 83–84 (theory: Slides 46–47, 49–51)

QUESTION (AS ON THE SLIDES)

Slide 83 — "Quiz: Multistage Algorithm". *Describe the key idea behind the multistage algorithm.* The slide also shows the textbook picture of main memory in three passes: **Pass 1** = "Item names to integers" + a table 1 … n of "Item counts" + one big "Hash table for bucket counts"; **Pass 2** = "Item names to integers" + "Frequent items" + a thin "Bitmap 1" + a "Second hash table for bucket counts"; **Pass 3** = "Item names to integers" + "Frequent items" + "Bitmap 1" + "Bitmap 2" + a "Data structure for counts of pairs".

Slide 84 — the slide's own answer. "The Multistage Algorithm uses additional hash tables to reduce the number of candidate pairs." (This sentence is the caption of the multistage figure in the textbook.)

No data is given: this is a pure **explain / theory** question, so the solution is a written answer. (Multistage is an improvement of **PCY** — Park–Chen–Yu, the three authors — which itself improves **A-Priori** for finding frequent *pairs*; s is the support threshold.) The slide's answer is only a one-line headline, so the solution first gives the **full-marks answer** (part a). Because an examiner can ask any follow-up on the same slides, it then covers the neighbouring points as parts (b)–(d) (multihash, comparison, diminishing returns — Slides 49–51), and ends with a small **Practice** run (part e) so you can see the mechanism with real numbers.

(a) Describe the key idea behind the multistage algorithm — what each of its three passes does and which test decides whether a pair gets a counter.

(b) Multihash (Slides 49–50): the same idea in two passes — how it works and what its risk is.

(c) Compare PCY, multistage and multihash in a table (passes, memory, risk, when to use).

(d) Why do more than two stages / hash tables give diminishing returns (Slide 51)?

(e) Practice (our own, not on the slides): run PCY, multistage and multihash by hand on the baskets of Slide 78.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Finding the few really popular pairs among millions of possible pairs is like screening a huge crowd at an airport. **PCY** is *one coarse sieve* (the bitmap): it throws away every pair that obviously cannot be popular. But some innocent pairs slip through, simply because they were dropped into the same "mailbox" (bucket) as popular pairs — or because several unpopular pairs shared a mailbox and their tallies added up. **Multistage** puts a *second sieve with a different hole pattern* (a different hash function) behind the first one. An innocent pair that slipped through the first sieve only because of its mailbox-mates is very unlikely to have popular mailbox-mates again, so the second sieve catches it. Only the survivors of sieve 1 are poured onto sieve 2 — and you know the survivors only after the first trip through the data is finished — so you need **one more trip (a third pass)**. **Multihash** puts *two smaller sieves side by side* in the very first trip: no extra pass, but each sieve is coarser (fewer mailboxes), so more mailboxes look popular.

FACTS AND FORMULAS USED IN THIS QUIZ

Frequent (Slides 8, 31): an itemset is frequent ⇔ its count ≥ s. **Bucket count (Slides 39–40):** count(b) = ∑ of the counts of all pairs hashed to bucket b; hence **count({i,j}) ≤ count(its bucket)**.
Bitmap (Slide 42): bit[b] = 1 ⇔ count(b) ≥ s (a *frequent bucket*); a 4-byte counter becomes 1 bit, so the bitmap needs $\frac{1}{32}$ of the memory of its hash table.
PCY candidate (Slide 43): {i,j} is counted ⇔ i, j both frequent **and** $\text{bit}_1[h_1(\{i,j\})] = 1$.
Multistage / multihash candidate (Slides 47, 50): {i,j} is counted ⇔ i, j both frequent **and** $\text{bit}_1 = 1$ **and** $\text{bit}_2[h_2(\{i,j\})] = 1$.
Average bucket count = N / B, where N = number of pair-occurrences hashed in pass 1 (∑ over baskets of C(basket size, 2)) and B = number of buckets. Halving B doubles it (Slide 49).

Part (a) — The key idea of the multistage algorithm — Lecture 4 · Slides 46–47, 83–84

KEY IDEA IN ONE SENTENCE (WHAT THE EXAMINER WANTS FIRST)

The multistage algorithm **uses additional hash tables** — a second, **different** hash function applied only to the pairs that survived PCY's first filter — **to reduce the number of candidate pairs** that must be counted, so their counters fit in main memory; it needs **3 passes**. The steps below give the detail that earns the remaining marks.

1. **Where it comes from: the weakness of PCY (Slides 38–45).** In PCY, pass 1 hashes every pair into a bucket and keeps only a count per bucket; between the passes the buckets become a **bitmap** (1 = frequent bucket). In pass 2 a pair gets a counter only if both items are frequent *and* its bucket bit is 1. But a bucket can be frequent although it contains **no frequent pair** — unpopular pairs collide and their counts add up to ≥ s (the "sad face" of Slide 41), or an unpopular pair simply shares a bucket with a popular one. Every such **false candidate** still receives a counter, and each counter is an (item, item, count) triple (Slide 45), which is why the slide demands that the hash table eliminate about 2/3 of the candidate pairs for PCY to beat A-Priori. *Beyond the slides:* a triple costs about 12 bytes against 4 bytes per pair in A-Priori's triangular array, so PCY wins only if at most about 1/3 of the pairs survive. **Memory is the bottleneck** (Slide 46): if too many false candidates survive, their counters do not fit in main memory. Multistage is the fix when one filter is not strong enough: use a second one.
2. **The key idea (Slide 46).** The slide says: "Limit the number of candidates to be counted" and "Key idea: After Pass 1 of PCY, **rehash pairs with a different hash function**". In other words: add a **second hash table** (and later a second bitmap) so that a pair must be "vouched for" by **two independent buckets** instead of one. A false candidate that was unlucky in table 1 is unlikely to be unlucky again in table 2. The price is **3 passes over the data** (Slide 46). This is exactly the slide-84 answer: *additional hash tables reduce the number of candidate pairs*.
3. **Pass 1 — identical to PCY (Slide 47, left).** Read all baskets once: count every item, and hash **every** pair $\{i,j\}$ of every basket with the first hash function h_1 into **Hash table 1** (add 1 to the bucket's counter; only the count is stored, not the pairs). At the end of the pass: item counts → **list of frequent items** (count ≥ s); Hash table 1 → **Bitmap 1** (1 bit per bucket, 1 = count ≥ s). The big counters are discarded, which frees the memory they occupied.
4. **Pass 2 — rehash the survivors with a *different* function (Slide 47, middle).** Main memory now holds the list of frequent items, Bitmap 1 and a **second hash table**. Read the baskets a second time. For each pair $\{i,j\}$: apply the PCY test (both items frequent **and** its bit in Bitmap 1 is 1). If it passes, hash the pair with a **second hash function h_2** into Hash table 2 (add 1 to that bucket); if it fails, ignore the pair. Nothing is counted exactly yet — Hash table 2 holds only bucket counts. At the end of the pass, Hash table 2 → **Bitmap 2** (1 = count ≥ s). Why does this second table help? **(i) The function is different** (independent of h_1): pairs that shared a bucket under h_1 are spread over different buckets under h_2, so an innocent pair no longer has the same popular neighbours. (If h_2 were the same function, the same pairs would collide again and little would be learned.) **(ii) Only candidates are hashed into it**, so table 2 is much less crowded than table 1 (in the practice example of part (e) table 1 receives all 41 pair-occurrences, table 2 only 21) and more of its buckets stay below s. Part (e), step 5, shows a pair that would survive if *all* pairs were hashed into table 2 but is removed because only candidates are.
5. **Pass 3 — count only the pairs that survive all tests (Slide 47, right).** Memory holds the frequent items, Bitmap 1, Bitmap 2 and the counters of candidate pairs. Read the baskets a third time and **count {i,j} if and only if**
   - **(1)** i and j are both frequent items, **and**
   - **(2)** $\{i,j\}$ hashes (by h_1) to a **frequent bucket in Bitmap 1**, **and**
   - **(3)** $\{i,j\}$ hashes (by h_2) to a **frequent bucket in Bitmap 2**.

   These counters are now exact; the pairs with count ≥ s are the frequent pairs.

6. **Why the result is still exactly right (no false negatives).** Let {i,j} be a truly frequent pair, i.e. it occurs in at least s baskets. Check the three tests of pass 3 one by one:
   - **(1) Both items are frequent:** each of i, j occurs in at least as many baskets as the pair does (monotonicity, Slide 30).
   - **(2) Bit in Bitmap 1 = 1:** a bucket count is the *sum* of the counts of all pairs hashed to it, so count(bucket) ≥ count({i,j}) ≥ s.
   - **(3) Bit in Bitmap 2 = 1:** because the pair passed (1) and (2), *every* one of its occurrences was hashed into Hash table 2 in pass 2; so that bucket count is also ≥ count({i,j}) ≥ s.

   So a frequent pair is never discarded. An infrequent pair *may* survive (a **false positive**), but then its **exact** count in pass 3 shows that it is below s and it is not reported. Hence multistage returns **exactly the same frequent pairs** as A-Priori and PCY; only the number of counters (the memory) changes. *Why fewer false positives:* an infrequent pair now has to be unlucky **twice**. *Beyond the slides (simple model):* if it lands in a frequent bucket with probability f_1 in table 1 and f_2 in table 2, independently, it survives with probability about $f_1 \cdot f_2$, which is smaller than either one (e.g. 0.2 × 0.2 = 0.04).
7. **The memory picture and the cost (Slide 47 = Figure 1; Slide 84).** Pass 1 looks like PCY. Pass 2 trades the big first table for the tiny Bitmap 1 (1/32 of its size) and uses the space it freed for the second table. Pass 3 keeps both bitmaps and spends the rest of memory on the counters of the few remaining candidates. (The textbook figure on Slide 84 also shows the "item names → integers" table at the top of each pass: the renumbering trick of Slide 33.) The cost is the **extra pass**: the whole data file must be read three times instead of two. *Beyond the slides:* by the same logic, with m hash tables (stages) multistage needs m + 1 passes (PCY = 1 table = 2 passes; two tables = 3 passes, as on Slide 46).

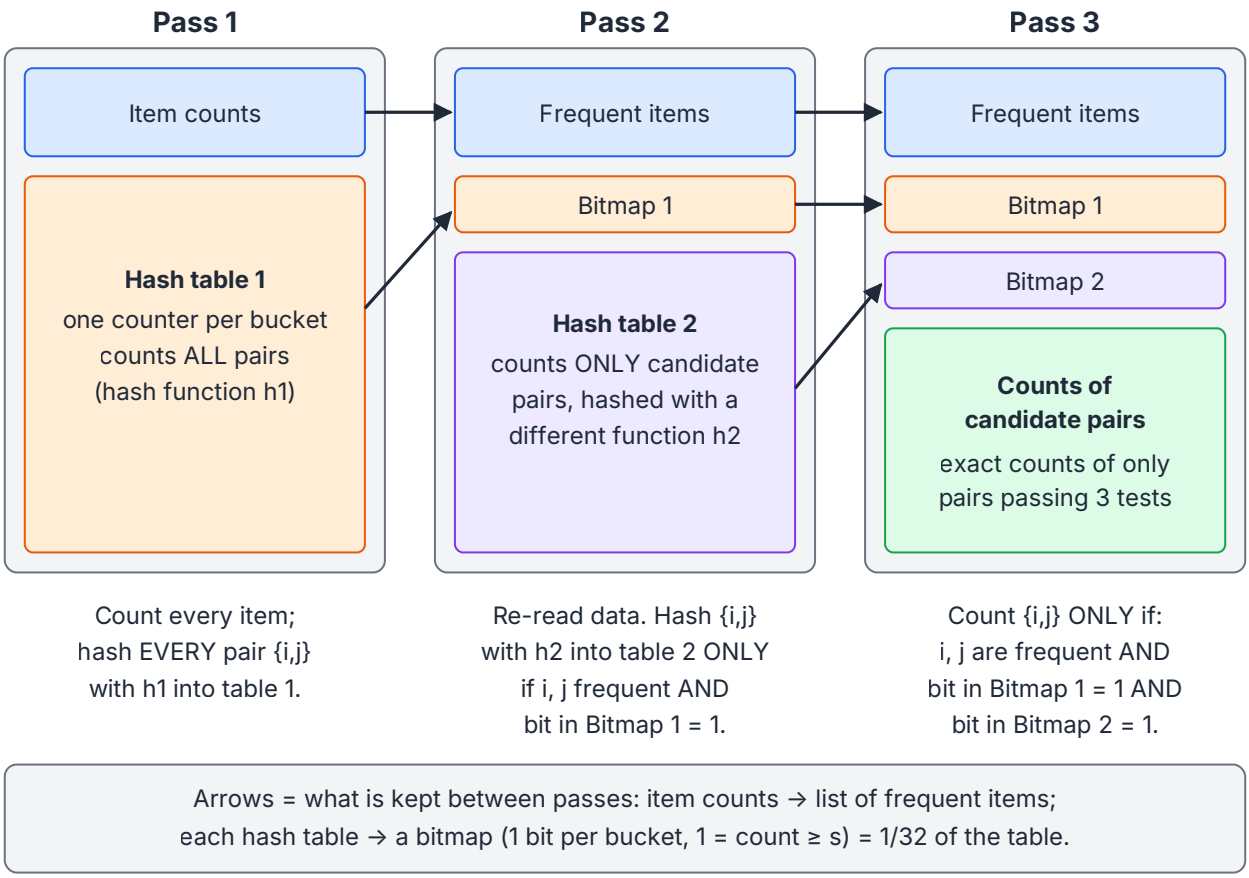


Figure 1 — Multistage main memory in its three passes (Slide 47; the same picture is on Slide 84). Orange = first hash function, purple = second hash function, green = the final counters.

SLIDE ERRATUM (OMISSION) — SLIDES 46–47

Slide 47 (pass 2) says only "Hash pairs {i,j} into Hash2", and Slide 46 says "rehash pairs with a different hash function". Read literally, this suggests that *all* pairs are rehashed. **The correct version:** in pass 2 only the pairs that already pass the PCY test (both items frequent **and** bit in Bitmap 1 = 1) are hashed into Hash table 2 (standard in *Mining of Massive Datasets*, §6.3.2). This is what makes table 2 less crowded, and it is why a multistage pass 2 needs the frequent-items list and Bitmap 1 in memory (as Slide 47's memory picture shows). Write the restriction in your answer.

Part (b) — Multihash: the same idea in two passes Lecture 4 · Slides 49–50

1. **Key idea (Slide 49).** "Use **several independent hash tables on the first pass**." Instead of building the second filter in a second pass (multistage), build both filters **at the same time** in pass 1. If it works, "we can get a benefit like multistage, but in only 2 passes".
2. **Pass 1 (Slide 50, left).** The spare memory that PCY gives to one hash table is **shared** by two tables (Hash table 1 and Hash table 2), each with $B/2$ buckets. Count every item; hash **every** pair twice — with h_1 into table 1 and with h_2 into table 2 (h_1, h_2 independent). Between the passes: each table → a bitmap (Bitmap 1, Bitmap 2), item counts → list of frequent items.
3. **Pass 2 (Slide 50, right).** Memory holds frequent items, Bitmap 1, Bitmap 2 and the counters. Count {i,j} iff i and j are frequent **and** $bit_1 = 1$ **and** $bit_2 = 1$ — the same three-part test as multistage's pass 3, but reached after only 2 passes.
4. **The risk (Slide 49).** "Halving the number of buckets doubles the average count." Every pair is hashed into *both* tables, so each table still receives all N pair-occurrences, but spread over only $B/2$ buckets: the average bucket count is $N/(B/2) = 2N/B$. "We have to be sure most buckets will still not reach count s." If too many do, both bitmaps are mostly 1s and almost nothing is filtered — multihash can then be *worse* than PCY with one big table. Slide 49 puts the condition the other way round: *if* most buckets still stay below s, "we can get a benefit like multistage, but in only 2 passes". Numbers for both cases are in part (d) and part (e).
5. **Difference from multistage in one line.** Multistage: the two filters are applied *one after the other* (stage 2 sees only survivors of stage 1, needs an extra pass, keeps full-size tables). Multihash: the two filters are applied *side by side* to all pairs (no extra pass, but each table is half-size, so each filter is weaker).

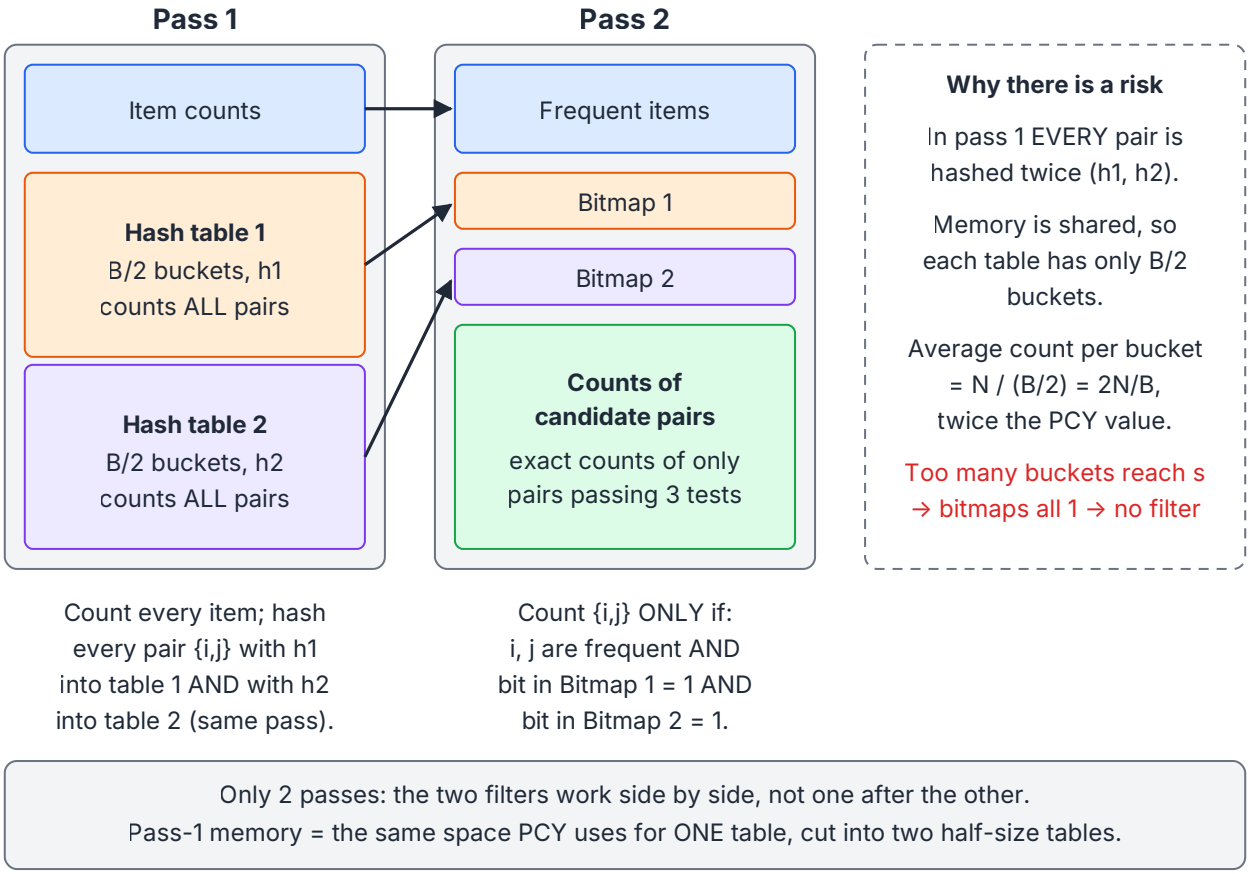


Figure 2 — Multihash main memory in its two passes (Slide 50). The two pass-1 tables together fill the memory that PCY gives to one table.

Part (c) — PCY, multistage and multihash compared Lecture 4 · Slides 38–50

Baseline, A-Priori (Slide 31): 2 passes; pass 1 stores only item counts (most memory idle); pass 2 counts *every* pair of frequent items. All three algorithms below are refinements that use the idle pass-1 memory to count fewer pairs in the last pass.

	PCY (Slides 38–45)	Multistage (Slides 46–47)	Multihash (Slides 49–50)
Passes (frequent pairs)	2	3	2
Pass-1 memory	item counts + **one** hash table using all spare memory (B buckets)	same as PCY: item counts + one table of B buckets	item counts + **several** hash tables **sharing** the spare memory (2 tables → $B/2$ buckets each)
Later memory	pass 2: frequent items + Bitmap + counters	pass 2: frequent items + Bitmap 1 + **second hash table** (about B buckets, only candidates hashed into it); pass 3: frequent items + Bitmap 1 + Bitmap 2 + counters	pass 2: frequent items + Bitmap 1 + Bitmap 2 + counters
A pair gets a counter iff	i, j frequent **and** $bit_1 = 1$	i, j frequent **and** $bit_1 = 1$ **and** $bit_2 = 1$	i, j frequent **and** $bit_1 = 1$ **and** $bit_2 = 1$
What the extra filter is	— (one filter)	a **second filter built only from the survivors** of the first, with a different hash function: less crowded, splits earlier collisions	a **second independent filter built from all pairs** in the same pass, but from a half-size table
Risk / cost	if most buckets are frequent the bitmap is mostly 1s and prunes little; may still leave too many candidates for memory	an **extra pass** over the data; each extra bitmap takes 1/32 of a table's memory; h_2 must be different from h_1	**halving the buckets doubles the average count**; if buckets reach s the bitmaps are all 1s and the filtering fails (can be worse than PCY)
When to use	spare memory in pass 1, and a large share of buckets stay below s; the table must remove about 2/3 of the candidate pairs to beat A-Priori (Slide 45)	PCY alone still leaves too many candidates for the counters to fit, and one more pass over the data is affordable	you want a multistage-like reduction but cannot afford a third pass, **and** bucket counts are well below s even after halving (many buckets, large s)
Output	**Exactly the same frequent pairs as A-Priori (no false negatives). The three algorithms differ only in how many counters they need and how many passes they use.**		

The "when to use" row combines the conditions stated on Slides 45, 46 and 49 with ordinary engineering reasoning; the slides do not give a formal rule.

Part (d) — Why more stages / more hash tables give diminishing returns Lecture 4 · Slide 51

Slide 51 ("PCY: Extensions") says three things: *either multistage or multihash can use more than two hash functions*; *in multistage there is a point of diminishing returns, since the bit-vectors eventually consume all of main memory*; *for multihash, too many hash functions makes all counts* ≥ s (the slide prints ">" with an underline, which means ≥). Here is the reasoning behind each statement.

1. **Multistage — the bitmaps eat the memory (Slide 51).** Every extra stage leaves one more bitmap in memory for the last pass. A bitmap is 1/32 of the table it summarises (Slide 42): a table of 10^8 four-byte buckets is 4×10^8 bytes = 400 MB, its bitmap is 10^8 bits = 12.5 MB. With m stages the last pass keeps m bitmaps, about $m/32$ of one table's memory (table below). That memory is taken from the space for the pair counters — exactly what the extra stage was supposed to protect. *Beyond the slides:* if each table were as large as all of main memory, 32 bitmaps would fill it completely, and long before that there would be no room left for the counters.
2. **Multistage — each stage helps less than the previous one.** The false candidates are removed by a *fraction* each time, so the absolute gain shrinks, while the cost of a stage (one more bitmap and, *beyond the slides*, one more pass) stays the same. Illustration with **our own idealised numbers**: suppose each independent table lets 20 % of the remaining false candidates through.

Hash tables (stages) m	Passes (m + 1)	Bitmap memory kept in last pass (fraction of one table)	False candidates still counted	Removed by the *last* table added
1 (= PCY)	2	1/32	20 %	80 percentage points
2	3	2/32	4 %	16 percentage points
3	4	3/32	0.8 %	3.2 percentage points
4	5	4/32	0.16 %	0.64 percentage points
5	6	5/32	0.032 %	0.128 percentage points

The first table removes 80 % of the false candidates, the second only 16 more, the fifth only 0.128. The truly frequent pairs can never be removed, so there is a floor below which no stage can go; once almost all false candidates are gone, another stage has almost nothing left to remove but still costs a bitmap and a pass.

3. **Multihash — too many tables make every bucket frequent (Slide 51).** With k tables sharing the memory of one PCY table (B buckets), each table has B/k buckets but still receives all N pair-occurrences, so the average bucket count is $k \cdot N/B$ — k times the PCY average. When this average reaches s, a typical bucket is frequent, every bitmap is almost all 1s, the test "bit = 1" removes (almost) nothing, and the bitmaps merely waste memory.
Beyond the slides (the textbook's reasoning, with our numbers; k = 1 is plain PCY): the bucket counts of one table add up to N, so at most N/s of its buckets can have a count ≥ s, i.e. at most a fraction $k \cdot N/(s \cdot B)$ of them. Take a typical case where the PCY average count is $s/10$ (so $N/(s \cdot B) = 1/10$). If the k hash functions are independent, an infrequent pair passes all k tests with probability at most $(k/10)^k$:

Tables k	Buckets per table	Average count per bucket	Share of buckets that can be frequent (at most)	Chance that an infrequent pair passes all k tests (at most)
1	B	0.1 s	10 %	10.0 %
2	$B/2$	0.2 s	20 %	**4.0 %**
3	$B/3$	0.3 s	30 %	**2.7 %**
4	$B/4$	0.4 s	40 %	**2.6 %**
5	$B/5$	0.5 s	50 %	3.1 %
6	$B/6$	0.6 s	60 %	4.7 %
8	$B/8$	0.8 s	80 %	16.8 %
10	$B/10$	1.0 s	no guarantee (up to 100 %)	no guarantee (up to 100 %)

Reading the table: the second table helps a lot (10 % → 4 %), the third and fourth help only a little (2.7 %, 2.6 %), and from the fifth table on the guarantee gets *worse*, because, as on Slide 49, every extra table has fewer buckets and so raises the average count, until it reaches s — the slide's "too many hash functions makes all counts ≥ s". (The bound is a worst-case model, not a measurement; real data behave better or worse depending on how the counts are spread.)

4. **Conclusion.** Two (or a few) hash tables capture most of the benefit; beyond that, multistage runs out of memory (and costs passes) and multihash runs out of "empty" buckets. The best number depends on the data and the memory — the slide only states that a point of diminishing returns exists.

Part (e) — Practice: the three algorithms on the baskets of Slide 78 (our own run)

Lecture 4 · Slide 78 (data)

1. **Setup.** Baskets (Slide 78): a = {1,2,4,5,8,9}, b = {1,4,7,8,9}, c = {1,2,5,9}, d = {1,2,5,7,8}; s = 3. The data are tiny (only 41 pair-occurrences), so a very small table would put almost everything into frequent buckets and show nothing; we therefore give the first table B = 26 buckets (numbered 0 … 25). Slide 78 gives no hash functions, so we choose our own (stated; deliberately simple so you can compute by hand — real systems use better ones, and ours leave many buckets empty): $h_1(\{i,j\}) = (i + j) \bmod 26$ and $h_2(\{i,j\}) = (i \times j) \bmod 26$ for PCY / multistage (26 buckets each); for multihash the same

memory is split into two tables of 13 buckets: table A = $(i + j) \bmod 13$, table B = $(i \times j) \bmod 13$. ("x mod 26" = the remainder of $x \div 26$; here $i + j \le 17$, so for h_1 the "mod 26" only fixes the table size.)

2. **Pass 1 — item counts and Hash table 1.** Item counts: 1 → 4, 2 → 3, 4 → 2, 5 → 3, 7 → 2, 8 → 3, 9 → 3, so the frequent items (≥ 3) are **1, 2, 5, 8, 9** (as on Slide 79). Pairs per basket: C(6,2) = 15, C(5,2) = 10, C(4,2) = 6, C(5,2) = 10, total $\mathbf{N = 41}$ pair-occurrences (21 distinct pairs). Hashing all of them with h_1 gives:

Bucket h_1	Pairs hashed to it (× number of baskets containing the pair)	Count	≥ 3 ?	Bit in Bitmap 1
3	`{1,2}` ×3	**3**	**yes**	**1**
5	`{1,4}` ×2	**2**	no	**0**
6	`{1,5}` ×3, `{2,4}` ×1	**4**	**yes**	**1**
7	`{2,5}` ×3	**3**	**yes**	**1**
8	`{1,7}` ×2	**2**	no	**0**
9	`{1,8}` ×3, `{2,7}` ×1, `{4,5}` ×1	**5**	**yes**	**1**
10	`{1,9}` ×3, `{2,8}` ×2	**5**	**yes**	**1**
11	`{2,9}` ×2, `{4,7}` ×1	**3**	**yes**	**1**
12	`{4,8}` ×2, `{5,7}` ×1	**3**	**yes**	**1**
13	`{4,9}` ×2, `{5,8}` ×2	**4**	**yes**	**1**
14	`{5,9}` ×2	**2**	no	**0**
15	`{7,8}` ×2	**2**	no	**0**
16	`{7,9}` ×1	**1**	no	**0**
17	`{8,9}` ×2	**2**	no	**0**
Check: the 14 non-empty counts add up to the number of pair-occurrences (the other 12 buckets — 0, 1, 2, 4 and 18–25 — stay empty: count 0, bit 0)		**3 + 2 + 4 + 3 + 2 + 5 + 5 + 3 + 3 + 4 + 2 + 2 + 1 + 2 = 41**		

   Bitmap 1 (buckets 0 … 25) = **00010011011111000000000000**: 8 of the 26 buckets are frequent (3, 6, 7, 9, 10, 11, 12, 13).

3. **PCY result.** The ten pairs of frequent items {1,2,5,8,9} are tested with Bitmap 1: only `{5,9}` (bucket 14, count 2) and `{8,9}` (bucket 17, count 2) fall in infrequent buckets. **PCY counts 8 pairs** (A-Priori would count all 10). Five of the eight are truly frequent; `{2,8}`, `{2,9}`, `{5,8}` are false candidates.

4. **Multistage, pass 2 — Hash table 2 receives only the 8 candidates.** Hash each candidate with h_2:

Bucket h_2	Candidate pairs hashed to it (× baskets)	Count	≥ 3 ?	Bit in Bitmap 2
2	`{1,2}` ×3	**3**	**yes**	**1**
5	`{1,5}` ×3	**3**	**yes**	**1**
8	`{1,8}` ×3	**3**	**yes**	**1**
9	`{1,9}` ×3	**3**	**yes**	**1**
10	`{2,5}` ×3	**3**	**yes**	**1**
14	`{5,8}` ×2	**2**	no	**0**
16	`{2,8}` ×2	**2**	no	**0**
18	`{2,9}` ×2	**2**	no	**0**
Check: counts add up to the occurrences of the 8 candidate pairs (all other buckets are empty: count 0, bit 0)		**3 + 3 + 3 + 3 + 3 + 2 + 2 + 2 = 21**		

   Bitmap 2 (buckets 0 … 25) = **00100100111000000000000000**: only 5 of 26 buckets are frequent (2, 5, 8, 9, 10). Table 2 received 21 pair-occurrences, table 1 received 41.

5. **Multistage, pass 3 and the verdict for every pair.** A pair is counted only if both items are frequent and its bit is 1 in *both* bitmaps. The next table also shows the multihash run (step 7). Cell colour: green = frequent bucket (bit 1), red = infrequent bucket (bit 0).

Pair	True count	PCY: h_1 bucket (count)	PCY	Multistage: h_2 bucket (count)	Multistage	Multihash: table A bucket (count)	Multihash: table B bucket (count)	Multihash
{1,2}	3	3 (3)	counted	2 (3)	counted	3 (4)	2 (4)	counted
{1,5}	3	6 (4)	counted	5 (3)	counted	6 (4)	5 (5)	counted
{1,8}	3	9 (5)	counted	8 (3)	counted	9 (5)	8 (4)	counted
{1,9}	3	10 (5)	counted	9 (3)	counted	10 (5)	9 (4)	counted
{2,5}	3	7 (3)	counted	10 (3)	counted	7 (3)	10 (5)	counted
{2,8}	2	10 (5)	counted	16 (2)	dropped	10 (5)	3 (2)	dropped
{2,9}	2	11 (3)	counted	18 (2)	dropped	11 (3)	5 (5)	counted
{5,8}	2	13 (4)	counted	14 (2)	dropped	0 (4)	1 (3)	counted
{5,9}	2	14 (2)	dropped	not hashed	dropped	1 (2)	6 (4)	dropped
{8,9}	2	17 (2)	dropped	not hashed	dropped	4 (2)	7 (5)	dropped
Counters needed			8		5			7

Multistage counts 5 pairs: {1,2}, {1,5}, {1,8}, {1,9}, {2,5}; their exact counts are all 3 ≥ s, so they are the frequent pairs — exactly the answer of Slide 80. The second table removed the three false candidates of step 3: `{2,8}` was only riding on the frequent pair `{1,9}` in bucket 10 (5 = 3 + 2); `{2,9}` (2) and `{4,7}` (1) reached 3 together in bucket 11 by collision; `{5,8}` (2) and `{4,9}` (2) reached 4 together in bucket 13. In table 2 each of them sits alone in a bucket with count 2 < 3.

Why "only candidates" matters: had we hashed *all* 41 pair-occurrences into table 2 (as a literal reading of Slide 47 suggests), bucket 14 would also contain `{2,7}` (count 1, a pair with the infrequent item 7), so its count would be 2 + 1 = 3 ≥ s, bit 1, and the false candidate `{5,8}` would survive: 6 counters instead of 5.

6. **Multistage needed 3 passes, but only 5 counters.** Pass 1: items + Hash table 1; pass 2: Hash table 2 (candidates only); pass 3: the 5 counters. In a real data set a few false candidates would usually remain; removing all of them here is a lucky property of a tiny example.
7. **Multihash (2 passes, 13 + 13 buckets).** Pass 1 hashes all 41 pair-occurrences into *both* tables. Bucket counts (an asterisk * marks a count < 3):
   Table A, (i + j) mod 13 — 0: 4, 1: 2*, 2: 2*, 3: 4, 4: 2*, 5: 2*, 6: 4, 7: 3, 8: 2*, 9: 5, 10: 5, 11: 3, 12: 3 (sum 41).
   Bitmap A = **1001001101111** → 8 of 13 buckets frequent.
   Table B, (i × j) mod 13 — 0: 0*, 1: 3, 2: 4, 3: 2*, 4: 4, 5: 5, 6: 4, 7: 5, 8: 4, 9: 4, 10: 5, 11: 1*, 12: 0* (sum 41).
   Bitmap B = **0110111111100** → 9 of 13 buckets frequent. Pass 2 counts the pairs whose bucket is frequent in **both** tables (last three columns above): **7 pairs** are counted; `{5,9}`, `{8,9}` (table A) and `{2,8}` (table B) are dropped, while `{2,9}` and `{5,8}` still get through. This is the **risk of Slide 49** in numbers: the average bucket count is 41/13 ≈ 3.2 (it was 41/26 ≈ 1.6 for PCY's single table), so 8/13 and 9/13 of the buckets are frequent instead of 8/26, the bitmaps filter much less, and multihash ends only one counter better than PCY and two worse than multistage. With real data where bucket counts stay well below s, the halving does not hurt this much (Slide 49).

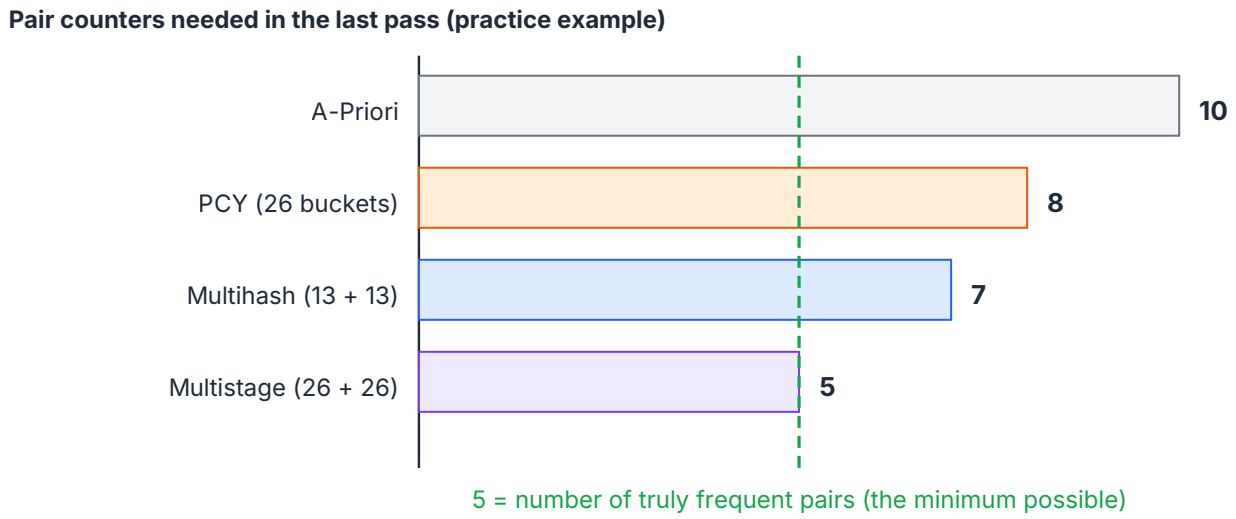


Figure 3 — Practice example: how many counters each algorithm needs in its last pass (bars are drawn to scale, 38 units per counter; the 10 pairs of frequent items are A-Priori's candidates). Fewer counters = less memory.

EXAM TIP

For "Describe the key idea behind the multistage algorithm" write, in this order: **(1) the purpose** — reduce the number of candidate pairs (memory is the bottleneck); **(2) the idea** — after PCY's pass 1, rehash the surviving pairs with a *different* hash function into a second hash table (and bitmap); **(3) the three passes** — pass 1 = PCY, pass 2 = rehash candidates, pass 3 = count; **(4) the pass-3 test** — both items frequent **and** frequent bucket in bitmap 1 **and** frequent bucket in bitmap 2. Add one line on multihash (same idea, several tables in pass 1, only 2 passes, risk = halved buckets double the average count) if the question asks for a comparison. A figure of the three memory layouts is worth drawing.

COMMON MISTAKES

- Saying multistage *saves* passes. It needs **3**; the two-pass relative is **multihash**.
- Mixing up the names: multi**stage** = filters in *stages*, one after another (extra pass); multi**hash** = several hash tables *at once* in the first pass.
- Forgetting that the second hash function must be **different**, or saying that *all* pairs go into the second table. Only the pairs that already passed the PCY test do.
- Thinking the bitmap stores counts. It stores **one bit per bucket** (1 = count ≥ s).
- Mixing up a **frequent bucket** (bucket count ≥ s) with a **frequent pair**. A frequent bucket may contain no frequent pair at all (several small pairs add up).
- Forgetting one of the **three** pass-3 tests (items frequent, Bitmap 1 bit = 1, Bitmap 2 bit = 1); PCY has only the first two.
- Believing that more stages or more hash functions are always better (Slide 51), or that the algorithm may **miss** frequent pairs. It cannot: it only removes pairs that provably cannot be frequent; the survivors are counted exactly.

Final answer: The key idea of the multistage algorithm is to use additional hash tables to reduce the number of candidate pairs, so that their counters fit in main memory. It needs 3 passes. Pass 1 = PCY: count items and hash every pair with h_1 into Hash table 1; afterwards Hash table 1 → Bitmap 1 and the item counts → list of frequent items. Pass 2: rehash, with a different hash function h_2 into a second hash table, only the pairs that pass the PCY test (both items frequent and bit in Bitmap 1 = 1); afterwards Hash table 2 → Bitmap 2. Pass 3: count {i,j} iff i, j are both frequent and {i,j} hashes to a frequent bucket in Bitmap 1 and to a frequent bucket in Bitmap 2. The result is exactly the frequent pairs of A-Priori (no false negatives), found with fewer counters.
Multihash = the same idea in 2 passes: several independent hash tables share the pass-1 memory (each one smaller) and both bitmaps are used in pass 2. Risk: halving the buckets doubles the average count, so many buckets may reach s and the filter fails.
Diminishing returns (Slide 51): multistage — the bitmaps (and, beyond the slides, the extra passes) eventually consume main memory while each stage removes fewer false candidates; multihash — too many tables make all counts ≥ s, so every bit is 1.
Practice (Slide 78 baskets): counters needed — A-Priori 10, PCY 8, multihash 7, multistage 5 (the five frequent pairs {1,2}, {1,5}, {1,8}, {1,9}, {2,5}).

Where the marks come from: (1) **Purpose / headline** — additional hash tables reduce the number of candidate pairs; memory is the bottleneck (this is the slide's own answer). (2) **Key mechanism** — after PCY pass 1, rehash with a different hash function into a second table. (3) **Three passes**, each described correctly (pass 1 = PCY with Bitmap 1; pass 2 = hash the surviving pairs into Hash table 2 → Bitmap 2; pass 3 = count). (4) **The pass-3 condition** with all three parts (items frequent, bucket frequent in B1, bucket frequent in B2). (5) **Bitmaps** replace the tables (1 bit per bucket, 1/32 of the memory). (6) **Correctness / effect** — no false negatives, fewer false positives, exact counts in the last pass. (7) **Multihash** — several independent tables in pass 1, 2 passes, risk (halved buckets double the average count). (8) **Comparison** — passes, memory, risk, when to use. (9) **Diminishing returns** — bitmaps consume memory (multistage), all counts ≥ s (multihash). (10) Optional extra: a small numeric run or a labelled figure.

Slide check: the slide's own answer (Slide 84: “uses additional hash tables to reduce the number of candidate pairs”) agrees with ours but is only the headline: it does not say that a *different* hash function is used, that a third pass is needed, or what the pass-3 test is. Our answer adds these from Slides 46–47. **Slide erratum (omission):** Slides 46–47 do not say that in pass 2 only the pairs that pass the PCY test (frequent items and frequent bucket in Bitmap 1) are hashed into Hash table 2; the correct version is given in the box above. Small notes: (1) printed slide 48 does not exist in the PDF, so the multihash slides are printed 49–51 and printed slide 52 is only the section divider “Frequent Itemsets in ≤ 2 Passes”; (2) Slide 51 prints “all counts > s” with an underlined “>”, which means ≥ s; (3) Slide 42 says a bit is 1 if the bucket count “exceeded” s, but the standard meaning (and the one used on Slide 31 for frequent items) is count ≥ s, which is what we use.

Fewer passes: the limited-pass idea

Lecture 4 · Slides 52–53

Slide 52 is a section divider titled **"Frequent Itemsets in ≤ 2 Passes"** (it carries no printed number; the "≤" is drawn as an underlined "<"). Slide 53 states the problem it solves.

BEGINNER INTUITION

A **pass** is one complete read of the basket file from disk. Reading terabytes from disk is slow, so the number of passes is the main cost. Imagine counting votes: A-Priori and PCY say "first count single items, then re-read everything to count pairs, then re-read again for triples ...". The slide asks: can we get the answer after reading the file at most twice, whatever the itemset size?

What Slide 53 says, in plain words:

- **A-Priori, PCY, etc., take k passes to find frequent itemsets of size k.** Read this as "k passes to reach itemsets up to size k": 5 passes to find frequent 5-itemsets. Every pass is another slow scan of the whole file.

- **"Can we use fewer passes?"** The slide's answer is **"use 2 or fewer passes for all sizes, but may miss some frequent itemsets"**, with two methods listed: **random sampling** and **SON** (Savasere, Omiecinski and Navathe, the three authors).

SLIDE ERRATUM

On Slide 53 the phrase "but may miss some frequent itemsets" (printed in pink) is true of **random sampling only**. SON is listed under the same bullet, but SON is **exact**: it misses nothing and, after its second pass, reports nothing false (Slide 57, explained below). Keep the two straight: *sampling = approximate, SON = exact*. Both use at most two passes.

Random sampling

Lecture 4 · Slides 54–55

The method

Lecture 4 · Slide 54

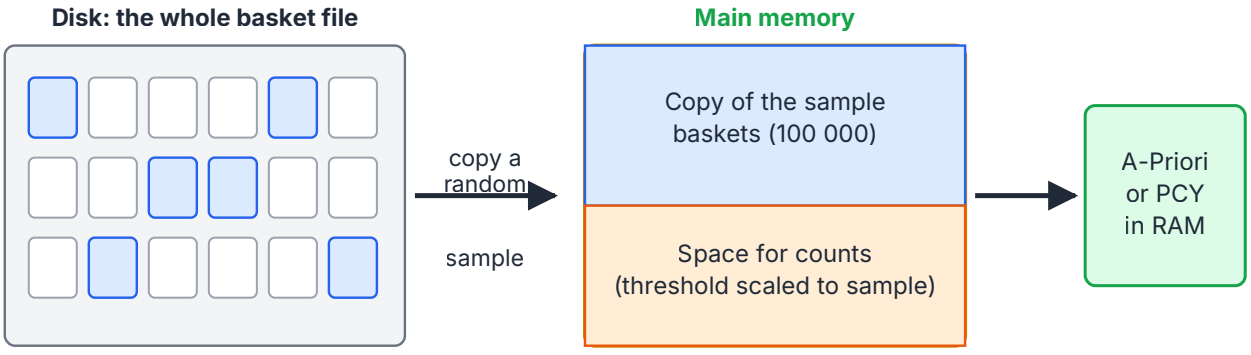
BEGINNER INTUITION

Pollsters do not ask every voter; they ask a random few thousand and scale the result. Random sampling does the same with baskets: read a small random subset that fits in RAM, mine that subset completely in memory, and treat the answer as the answer for the whole file.

DEFINITION: RANDOM SAMPLING FOR FREQUENT ITEMSETS (SLIDE 54)

Take a **random sample of the market baskets** and run A-Priori (or one of its improvements such as PCY) **in main memory** on that sample, with the support threshold **reduced proportionally** to the sample size.

1. **Sample:** pick a random subset of the baskets small enough to fit in main memory together with the counters (the slide's figure shows exactly two memory blocks: "Copy of sample baskets" and "Space for counts").
2. **Scale the threshold:** if the sample holds a fraction p of all baskets, use $p \times s$ as the threshold on the sample.
3. **Mine in memory:** run A-Priori / PCY on the sample. Moving from pairs to triples to bigger sets now costs no disk I/O, because every pass is a scan of RAM. This is the saving the slide names: "save disk I/O each time we increase the size of itemsets".
4. **Output** the itemsets found frequent in the sample (optionally verify them, next section).



Every pass over the sample is a memory scan, so pairs, triples, … cost no extra disk reads.

Figure L4p3-1. Random sampling (Slide 54, redrawn): the slide's "copy of sample baskets + space for counts" memory box, with the disk side added. Numbers are the Beyond-the-slides illustration in the text.

Worked example — scaling the threshold Lecture 4 · Slide 54

Beyond the slides: an illustration (the slide gives no numbers). A file has 10^7 baskets and the support threshold is s = 1 % of the baskets = 10^5 baskets.

1. Take a 1 % random sample: p = 0.01, so the sample has $0.01 \times 10^7 = 10^5$ baskets.
2. Scaled threshold = $p \times s = 0.01 \times 10^5$ = **1 000 baskets** of the sample.
3. As a fraction it is still "1 %" of the sample ($1\,000 / 10^5$ = 1 %), which is why the slide can write "threshold s = 1 %" for both. If you forgot to scale and used 10^5 on a sample of 10^5 baskets, an itemset would need to be in *every* sampled basket, and almost nothing would pass.

COMMON MISTAKE

Slide 8 defines support as an *absolute count* of baskets, but Slide 54 writes "s = 1 %". When a threshold is a percentage, always convert to a count of the data set you are actually counting in (sample or chunk). "Reduce proportionally" is about that count. Also note the sample must be **random**; the first n baskets of a file (Beyond the slides: often sorted by date or customer) can be badly biased.

Two kinds of mistake

Lecture 4 · Slide 55

Slide 55 has two bullets, one for each kind of error a sample can make.

DEFINITION: FALSE POSITIVE AND FALSE NEGATIVE

A **false positive** is an itemset that looks frequent in the sample but is *not* frequent in the whole data. A **false negative** is an itemset that *is* frequent in the whole data but is *not* frequent in the sample, so it is missed.

TRUTH (whole file)	frequent in sample	not frequent in sample
really frequent in the whole file	**CORRECT** found and truly frequent	**FALSE NEGATIVE** missed; a second pass over candidates can't recover it
really NOT frequent in the whole file	**FALSE POSITIVE** looks frequent but is not; the optional second pass removes it	**CORRECT** correctly ignored

Slide 55: the optional second pass fixes only the orange cell. The red cell stays.

Figure L4p3-2. The four outcomes of random sampling (Slide 55). Only the red cell is dangerous.

- **Bullet 1: "Optionally, verify that the candidate pairs are truly frequent in the entire data set by a second pass (avoid false positives)."** One more pass over the full file counts only the itemsets the sample reported. Those that fall below s are discarded. This removes false positives. It is optional because it costs a second disk pass; the slide's "candidate pairs" really means candidate itemsets of any size.
- **Bullet 2: "But you do not catch sets frequent in the whole but not in the sample."** These are the false negatives. The verification pass only recounts itemsets that already appeared in the sample, so it can never bring a missed itemset back. This is why Slide 53 says sampling "may miss some".

EXAM TIP

The classic one-line contrast: *sampling can give false positives (fixable with a second pass) and false negatives (not fixable); SON gives neither after its second pass.* If a question asks "how do you remove false positives?", the answer is "count the reported itemsets over the entire data in a second pass".

Beyond the slides: trading memory for fewer misses

Lecture 4 · Slide 55 (extension)

Beyond the slides: the textbook (*Mining of Massive Datasets*, Section 6.4.1) notes that false negatives can be reduced, though not eliminated, by **lowering the sample threshold a little below the proportional value**, for example to $0.9 \times p \times s$ or $0.8 \times p \times s$. In the example above that is 900 or 800 instead of 1 000. An itemset that is just above s in the whole file but unlucky in the sample then still has a chance. The price: more itemsets pass in the sample, so more memory is needed and more false positives appear (which the second pass can remove). Slide 55 itself does not mention this.

The SON algorithm

Lecture 4 · Slides 56–57

BEGINNER INTUITION

Instead of asking a few voters, SON asks *everyone* but in groups. A huge pile of exam papers is split into small piles that one marker can handle. Each marker lists the mistakes that are common enough in her own pile. Any mistake that is common in the whole pile must be common in at least one small pile, so the union of the lists contains every truly common mistake. A final recount over all papers removes the listed mistakes that were only common by chance. The data is read exactly twice.

Pass 1: find candidates chunk by chunk

Lecture 4 · Slide 56

DEFINITION: SON, PASS 1 AND CANDIDATE (SLIDE 56)

SON (Savasere, Omiecinski and Navathe) **repeatedly reads small subsets of the baskets into main memory** and runs an in-memory algorithm (A-Priori, PCY, ...) to find the frequent itemsets of each subset. An itemset becomes a **candidate** if it is found frequent in **any one or more** of the subsets.

1. Cut the entire basket file into **chunks**, each small enough to fit in memory with its counters.
2. For each chunk in turn: load it, run the in-memory algorithm with the **local** (lowered) threshold, and write out the itemsets that are frequent *in this chunk*.
3. The **candidate list** is the union of all those outputs (an itemset reported by several chunks is listed once).

COMMON MISTAKE

Slide 56 stresses: **"we are not sampling, but processing the entire file in chunks that fit into the memory."** Every basket is read and used; nothing is thrown away. That is the whole difference from Random Sampling, and the reason SON can be exact.

Pass 2: count the candidates over everything

Lecture 4 · Slide 57

On a second pass, SON counts all the candidate itemsets over the *entire* file and keeps those that are frequent in the entire set (count $\geq s$). Only the candidates are counted, not every possible itemset, so memory stays small.

KEY "MONOTONICITY" IDEA (SLIDE 57)

An itemset cannot be frequent in the entire set of baskets unless it is frequent in at least one subset. Consequence: the candidate list can never leave out a truly frequent itemset (**no false negatives**), and the second pass removes the itemsets that were frequent only locally (**no false positives** in the final output). So SON gives the **exact** answer, for all itemset sizes, with two passes.

COMMON MISTAKE

The word "monotonicity" here is a *different* property from Slide 30's A-Priori monotonicity ("every subset of a frequent itemset is frequent"). Slide 57's version is "frequent overall ⇒ frequent in at least one chunk". Beware of mixing them up when asked "what is the monotonicity idea in SON?".

Why it works: the pigeonhole argument

Lecture 4 · Slide 57 (proof sketch, Beyond the slides)

Beyond the slides: the slide states the idea without proof (the proof follows Mining of Massive Datasets, Section 6.4.3). Suppose the file is split into p equal chunks, and the global threshold is s. Use the local threshold s/p in each chunk. If an itemset has fewer than s/p occurrences in *every* chunk, its total is below $p \times (s/p) = s$, so it cannot be globally frequent. Taking the contrapositive: a globally frequent itemset reaches s/p in at least one chunk, so some chunk reports it, so it is a candidate.

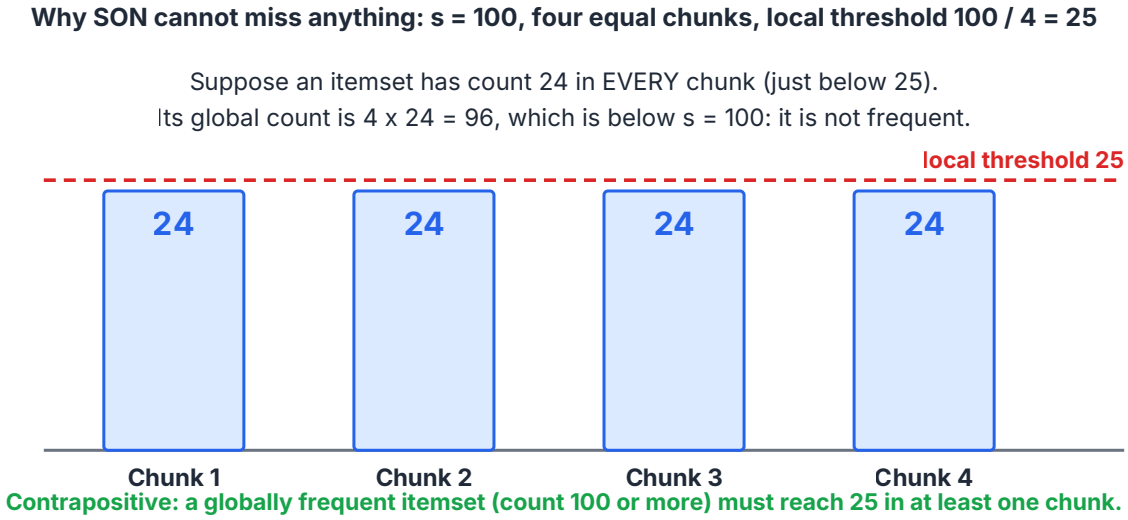


Figure L4p3-3. The pigeonhole argument behind Slide 57's "monotonicity" idea, with made-up numbers.

FORMULA: THE LOCAL THRESHOLD

local threshold of a chunk = $p_{chunk} \times s$, where p_{chunk} = (baskets in the chunk) ÷ (baskets in the whole file). For equal chunks this is s/(number of chunks). Because counts are whole numbers, use "count ≥ the next integer at or above" the value.

SLIDE ERRATUM

Slides 56–57 never say which threshold is used *inside* a chunk. If the full threshold s were applied to each small chunk, an itemset spread evenly across the chunks would be below s in every one of them and would be missed, and the "no false negatives" claim would be false. The threshold must be scaled down (the SON example on Slide 63 uses a local support of 2 against a global support of 4). Quiz L4-Q6 (below) checks this against the example's numbers.

SON on a cluster: two MapReduce rounds

Lecture 4 · Slides 58–60, 65–66

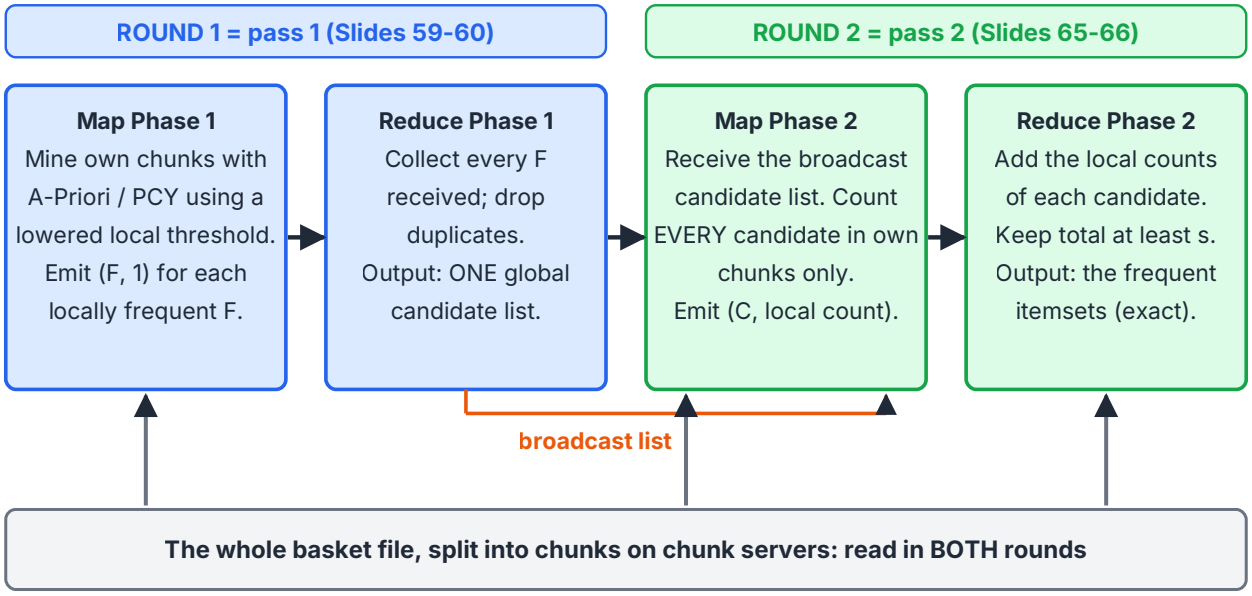
Why SON suits distributed data

Lecture 4 · Slide 58

Slide 58: **"SON lends itself to distributed big data analysis."** The baskets are spread over many nodes (machines). The slide's two sub-bullets are the entire algorithm: **(1) compute frequent itemsets at each node; (2) accumulate the counts of all candidates.** The reason it parallelises so well is that each chunk is mined *independently*: no machine needs to see another machine's chunk during Pass 1, and in Pass 2 each machine only counts on its own chunk, then the counts are added.

BEGINNER INTUITION

Recall MapReduce from the earlier lectures: **map** workers each process their own piece of the data and emit key-value pairs; the system groups pairs by key; **reduce** workers combine all values of one key. SON is "two MapReduce jobs back to back". Job 1 produces the candidate list. Job 2 uses that list to produce exact counts.



Example (Quiz L4-Q6): Reduce 1 gives 9 candidates, Reduce 2 keeps 3 frequent itemsets.

Figure L4p3-4. SON as two MapReduce jobs (Slides 58-60, 65-66). Round 1 produces candidates; round 2 counts them exactly.

Round 1: Map Phase 1 and Reduce Phase 1

Lecture 4 · Slides 59–60

Phase	What the slide says	Plain meaning
Map Phase 1: run algorithm on subsets (Slide 59)	The entire dataset is split into chunk servers; a subset of chunks is processed independently by each mapper. On each mapper, run a frequent-itemset algorithm (Apriori, PCY, FP-growth etc.). Each mapper outputs the candidate frequent itemsets from its subset of chunks.	Every mapper takes the chunks it is responsible for, loads them in memory, mines them with a single-machine algorithm and emits the itemsets that are frequent *there*. Together the mappers see **all** the baskets.
Reduce Phase 1: collect candidate itemsets (Slide 60)	Aggregate the candidates found in Phase 1 into a global list of candidate itemsets, including elimination of duplicates.	The same itemset may be frequent in several chunks. The reducers' only job is a **union without duplicates**. Output: one global candidate list.

Beyond the slides (Mining of Massive Datasets, Section 6.4.4): the map output is a key-value pair $(F, 1)$ for each locally frequent itemset F (the value 1 is meaningless; only the key matters), and the reducer simply outputs each key it receives once. The local threshold in Map Phase 1 is $p \times s$, as above.

"FP-growth" appears on Slide 59 as one more example of an in-memory algorithm. It is not taught anywhere in Lectures 1–5, so you only need to know it is an alternative to A-Priori and PCY for mining one chunk.

SLIDE ERRATUM

Slide 60 says the candidates "will be *optionally* evaluated against the entire dataset in Phase 2". For SON that is wrong. The second round is **required**: without it the output contains false positives (in the example on Slides 68–69, five of the eight listed candidates are thrown away). The word "optionally" belongs to Random Sampling (Slide 55), where verification really is optional. Slide 57 correctly says "on a second pass, count all the candidate itemsets". Also on Slide 59, "split into chunk servers" should read "split into chunks stored on chunk servers", and "candidate frequent itemsets that appear in subsets of chunks" should read "that are *frequent* in the subsets of chunks".

Round 2: Map Phase 2 and Reduce Phase 2

Lecture 4 · Slides 65–66

Phase	What the slide says	Plain meaning
Map Phase 2: count the support of candidate itemsets (Slide 65)	Take the global list of candidates from Phase 1 and **broadcast** it to all mappers. Each mapper checks the support of each candidate on its chunks.	This is the "second pass". Every mapper receives the same candidate list and counts *every* candidate in *its own* chunks, including candidates that were not frequent there (a count of 0 is reported as 0).
Reduce Phase 2: aggregate global support counts (Slide 66)	Aggregate the support counts for each candidate globally. Itemsets whose support meets the minimum threshold are considered frequent.	Each reducer receives, for one candidate, the per-chunk counts and **adds** them. Chunks do not overlap, so the sum is the exact global support. Keep the candidate if the total is **at least** s.

Beyond the slides: the Phase 2 map output is (C, v), where C is a candidate and v is its count in that mapper's chunks. "Broadcast" means the (small) candidate list is copied to every mapper; it is not shuffled by key.

EXAM TIP

If asked to "describe SON in MapReduce", answer with the four phases in order and say *what is emitted* at each: (1) Map 1 emits locally frequent itemsets; (2) Reduce 1 removes duplicates to form the candidate list; (3) Map 2 counts every broadcast candidate on its own chunks; (4) Reduce 2 sums the counts and applies the threshold. Mention "two passes over the data, whatever the itemset size" and "exact".

The worked example (five baskets, two chunks)

Lecture 4 · Slides 61–69

Slides 61–69 run the four phases on a tiny data set so you can see each list. The setting:

- **Slide 61:** five transactions T1–T5 over apple, banana, orange and pear, with **global support 4** (an itemset must be in at least 4 of the 5 baskets).
- **Slide 62:** Chunk 1 = T1–T3 (3 baskets), Chunk 2 = T4–T5 (2 baskets).
- **Slides 63–64:** Map Phase 1 and Reduce Phase 1 with **local support 2**; the slide shows a list of 8 candidate itemsets.
- **Slide 65–66** restate the Phase 2 rules; **Slide 67** shows each mapper's counts of the 8 candidates; **Slide 68** adds them up per candidate; **Slide 69** repeats Slide 68 with the failing itemsets struck through.

What to watch for as you work the example:

- An itemset such as `{pear}` is frequent *locally* in Chunk 2 (count 2) but its global count is only 2, below 4. It enters the candidate list and is eliminated in Phase 2: a textbook false positive.
- A chunk must count *every* candidate, not only the ones it found itself (for example Chunk 1 reports `{pear}` with count 0).
- The final answer must equal what plain A-Priori on the whole file would give. It does.

SLIDE ERRATUM

Slides 63–64 (and so Slides 65–69) leave out a candidate. In Chunk 2 the triple `{banana, orange, pear}` occurs in both T4 and T5, so it also has local count 2 and A-Priori with local support 2 would report it. The correct candidate list has **9** itemsets, not 8. The final answer is unchanged because its global count is 2, below 4 (it would simply be a sixth false positive). Slide 63 also has the typo "e.g," and writes "`{banana, pear} = 2 times`" with "=" where every other line has ":". Slide 67's second line is missing separators between entries, and Slide 68 never states the final answer in words (it appears only as strike-through on Slide 69): the survivors are `{banana}` (4), `{orange}` (5) and `{banana, orange}` (4).

NOT EXAMINED — LOCAL THRESHOLD IN THE EXAMPLE

The rule "local threshold = $p \times s$" would give 0.6 × 4 = 2.4 for Chunk 1 and 0.4 × 4 = 1.6 for Chunk 2. The slides simply use 2 for both, which is safe: it is no higher than needed for Chunk 2 and lower than needed for Chunk 1 (so it can only produce extra candidates, never miss one). Quiz L4-Q6 part (e) discusses this.

Try it yourself first: cover the solution of the quiz below, reproduce each of the four phases from the five baskets, then compare, including the missing ninth candidate and the three-part explanation of why SON is exact.

Quiz L4-Q6 SON algorithm: the two-chunk MapReduce example Lecture 4 · Slides 56–69

QUESTION (AS ON THE SLIDES)

SON (Savasere, Omiecinski and Navathe) MapReduce Example — Slides 61–69 (theory on Slides 56–60). The slides run the SON algorithm on a tiny data set of five baskets (transactions) split into two chunks. They present it as a worked example rather than a numbered question, so it is restated here as a task: reproduce every phase and explain why it works.

Basket	Items	Chunk
T1	`[apple, banana, orange]`	Chunk 1 (Slide 62)
T2	`[banana, orange]`	
T3	`[apple, orange]`	
T4	`[banana, orange, pear]`	Chunk 2 (Slide 62)
T5	`[banana, apple, orange, pear]`	

Global support = 4 (Slide 61): an itemset is frequent in the whole data set if it is contained in at least 4 of the 5 baskets. **Local support = 2** in each chunk (Slide 63). Mappers run Apriori on their chunk.

(a) Map Phase 1: find the locally frequent itemsets of Chunk 1 and of Chunk 2 (local support 2), with their local counts.

(b) Reduce Phase 1: form the global list of candidate itemsets (duplicates removed).

(c) Map Phase 2: broadcast the candidates and let each mapper count every candidate on its own chunk.

(d) Reduce Phase 2: add up the counts and keep the itemsets whose global support is at least 4. What are the frequent itemsets?

(e) Explain (i) why SON can have *no false negatives*, (ii) why the second pass removes the *false positives*, and (iii) how the local threshold relates to the global threshold.

(f) Describe the MapReduce view of SON: what do Map Phase 1, Reduce Phase 1, Map Phase 2 and Reduce Phase 2 each do (Slides 58–60, 65–66)?

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Two markers, one big pile of exam papers. A pile of 100 exam papers is split into two piles of 50. You want every mistake that occurs on at least 40 papers overall. Suppose mistake X really is on 40 papers. Could X be on fewer than 20 papers in pile A *and* on fewer than 20 papers in pile B? No: at most 19 + 19 = 38, which is less than 40. So X must be on at least 20 papers in *one* of the piles.

That is the whole trick of SON. **Pass 1:** each marker (mapper) writes down every mistake that is "common enough" in *her own* pile; this list can never miss a mistake that is common overall. But some listed mistakes were only common in one pile by chance. **Pass 2:** everybody recounts only the listed mistakes over *all* papers and throws away the ones that do not reach 40. Result: the exact answer, after reading the data just twice.

KEY IDEA

SON = **two passes**. Pass 1 (Map Phase 1 + Reduce Phase 1): find the frequent itemsets *inside each chunk*, using a lowered (local) threshold; every itemset found anywhere becomes a **candidate**. Pass 2 (Map Phase 2 + Reduce Phase 2): count *only the candidates* over the whole data and keep those with global support ≥ s. Note (Slide 56): this is **not sampling** — the entire file is processed, just in chunks that fit into main memory.

Step 0 — set-up and the numbers we will need

1. **Definitions (Lecture 4 · Slide 8):** the **support** of an itemset I is the *number of baskets that contain all items of* I; I is **frequent** if its support is at least the threshold s. Here the thresholds are counts, not percentages.

2. **Data facts:** N = 5 baskets; global threshold s = 4 (that is 80 % of the baskets); Chunk 1 holds 3 baskets (T1–T3), Chunk 2 holds 2 baskets (T4–T5). The chunks do not overlap and together contain every basket — a **partition** of the file, so per-chunk counts add up to the global count.
3. **Fraction of the file in each chunk:** p_1 = 3 ÷ 5 = 0.6 and p_2 = 2 ÷ 5 = 0.4, so $p_1 + p_2 = 1$. We will use these in part (e). The slides simply state local support = 2 for both chunks.
4. **Three words to keep straight:** a **candidate** is an itemset found frequent inside at least one chunk (Slide 56); a **false positive** is a candidate that is *not* frequent in the whole file; a **false negative** is an itemset that *is* frequent in the whole file but is missing from the candidate list. Pass 1 is built to give no false negatives, Pass 2 removes the false positives.

Part (a) — Map Phase 1: frequent itemsets inside each chunk (Slides 59, 63)

1. **Formula / idea:** local count of I in a chunk = number of baskets *of that chunk* containing every item of I; I is **locally frequent** if local count ≥ 2. Each mapper runs an ordinary in-memory algorithm on its chunk (the slide uses A-Priori; Slide 59 also names PCY and FP-growth). A-Priori works level by level: count single items, keep the frequent ones, build candidate pairs *only* from frequent items, count them, keep the frequent ones, build candidate triples *only* from frequent pairs, and so on.
2. **Chunk 1 (T1, T2, T3), level 1 — single items:** apple is in T1, T3 → 2; banana is in T1, T2 → 2; orange is in T1, T2, T3 → 3; pear is in none → 0. Keep those with count ≥ 2: apple, banana, orange.
3. **Chunk 1, level 2 — pairs** (all three items are frequent, so all three pairs are candidates): `{apple, banana}` is only in T1 → 1 (not frequent); `{apple, orange}` is in T1, T3 → 2 (frequent); `{banana, orange}` is in T1, T2 → 2 (frequent).
4. **Chunk 1, level 3 — triples:** the only possible triple `{apple, banana, orange}` needs all three of its pairs to be frequent, but `{apple, banana}` is not, so A-Priori does not even count it (inside Chunk 1 its true count is 1, from T1). **Chunk 1 output = 5 itemsets**: `{apple}`:2, `{banana}`:2, `{orange}`:3, `{apple, orange}`:2, `{banana, orange}`:2. This matches Slide 63.
5. **Chunk 2 (T4, T5), level 1 — single items:** apple is only in T5 → 1 (not frequent); banana in T4, T5 → 2; orange in T4, T5 → 2; pear in T4, T5 → 2. Frequent: banana, orange, pear.
6. **Chunk 2, level 2 — pairs** built only from the frequent items banana, orange, pear (apple is dropped, so no pair containing apple is ever counted): `{banana, orange}` in T4, T5 → 2; `{banana, pear}` in T4, T5 → 2; `{orange, pear}` in T4, T5 → 2. All three are frequent.
7. **Chunk 2, level 3 — triples:** the triple `{banana, orange, pear}` has all its pairs frequent, so it is a candidate; it is contained in T4 and in T5 → count 2 ≥ 2, so it **is locally frequent**. Level 4: only one frequent triple exists, so no 4-item candidate can be formed. **Chunk 2 output = 7 itemsets**: `{banana}`:2, `{orange}`:2, `{pear}`:2, `{banana, orange}`:2, `{banana, pear}`:2, `{orange, pear}`:2 **and** `{banana, orange, pear}`:2.

A-Priori level	Mapper 1 — Chunk 1 (T1–T3)	Mapper 2 — Chunk 2 (T4–T5)
1 (single items)	apple 2 ✓ · banana 2 ✓ · orange 3 ✓ · (pear 0)	apple 1 × · banana 2 ✓ · orange 2 ✓ · pear 2 ✓
2 (pairs)	{apple, banana} 1 × · {apple, orange} 2 ✓ · {banana, orange} 2 ✓	{banana, orange} 2 ✓ · {banana, pear} 2 ✓ · {orange, pear} 2 ✓
3 (triples)	{apple, banana, orange}: pruned, never counted	**{banana, orange, pear} 2 ✓ (missing on Slide 63)**
Output of Map Phase 1	**5 itemsets**	**7 itemsets (the slide shows 6)**

SLIDE ERRATUM

Slide 63 leaves out `{banana, orange, pear}` from the Chunk 2 list. It appears in both baskets of Chunk 2 (T4 and T5), so its local count is 2 and it meets the local support of 2; A-Priori does not stop at pairs. (The slide's list would be exactly right only if the course restricted itself to itemsets of size 2 or less — the slide does not say so.) Chunk 1 on the slide is complete. Consequences: the candidate list on Slide 64 should have **9** itemsets, not 8, and the Map Phase 2 lines on Slide 67 should also show `{banana, orange, pear}`: 0 times (Mapper 1) and 2 times (Mapper 2). The **final answer on Slide 69 does not change** because the total support of this triple is 2, below 4 (shown in part (d)). This is exactly the point of Pass 2: an extra candidate is harmless, it is simply filtered out. In an exam, write the triple and mention that the slide leaves it out.

BEGINNER INTUITION

Why does a mapper never count `{apple, banana}` in Chunk 2? Because A-Priori's rule says: *a set can only be frequent if all its smaller subsets are frequent*. In Chunk 2 apple is only 1 (below 2), so any set containing apple must be below 2 as well — no need to look. This is the "monotonicity" used *inside* a mapper; SON's own "monotonicity" (Slide 57) is a different idea that works *across* chunks (see part (e)).

Part (b) — Reduce Phase 1: the global candidate list (Slides 60, 64)

1. **Idea:** every locally frequent itemset is sent out by its mapper as a key. All copies of the same itemset arrive at the same reducer, which keeps it **once** (elimination of duplicates, Slide 60). The local counts are ignored here — they are only local facts. The output is the **global list of candidate itemsets**; Slide 60 calls them "potential frequent itemsets", to be checked against the entire data set in Phase 2.
2. **Count the outputs:** Mapper 1 sent 5 itemsets, Mapper 2 sent 7, so 5 + 7 = 12 items arrive. Three itemsets were sent by both mappers — `{banana}`, `{orange}` and `{banana, orange}` — so 12 − 3 = **9 unique candidates**.

Candidate itemset	From Chunk 1?	From Chunk 2?	On Slide 64?
`{apple}`	✓	—	✓
`{banana}`	✓	✓ (duplicate, merged)	✓
`{orange}`	✓	✓ (duplicate, merged)	✓
`{pear}`	—	✓	✓
`{apple, orange}`	✓	—	✓
`{banana, orange}`	✓	✓ (duplicate, merged)	✓
`{banana, pear}`	—	✓	✓
`{orange, pear}`	—	✓	✓
`{banana, orange, pear}`	—	✓	× omitted

Part (c) — Map Phase 2: count the candidates (Slides 65, 67)

1. **Idea:** the complete candidate list (9 itemsets) is **broadcast** to all mappers (Slide 65). Each mapper re-reads *its own* chunk and counts **every** candidate in it — also candidates that were *not* locally frequent in that chunk (for example {apple} in Chunk 2 is counted as 1, and {pear} in Chunk 1 as 0). These are now exact counts for that chunk.
2. **Counting** (which baskets contain the itemset):

Candidate	Mapper 1 (T1, T2, T3): baskets → count	Mapper 2 (T4, T5): baskets → count
`{apple}`	T1, T3 → **2**	T5 → **1**
`{banana}`	T1, T2 → **2**	T4, T5 → **2**
`{orange}`	T1, T2, T3 → **3**	T4, T5 → **2**
`{pear}`	none → **0**	T4, T5 → **2**
`{apple, orange}`	T1, T3 → **2**	T5 → **1**
`{banana, orange}`	T1, T2 → **2**	T4, T5 → **2**
`{banana, pear}`	none → **0**	T4, T5 → **2**
`{orange, pear}`	none → **0**	T4, T5 → **2**
`{banana, orange, pear}` (missing on slide)	none → **0**	T4, T5 → **2**

The eight rows printed on Slide 67 all agree with these numbers.

> **COMMON MISTAKE**
> Adding up only the counts from the chunks where the itemset was *locally frequent*. That undercounts. For `{apple}`, Chunk 2 contributes 1 (not 0), giving 2 + 1 = 3; for `{pear}`, Chunk 1 contributes 0, giving 0 + 2 = 2. In Pass 2 each mapper must report its exact count for every candidate, including small counts and zeros.

Part (d) — Reduce Phase 2: global support and the answer (Slides 66, 68–69)

1. **Formula:** global support of a candidate C = $\text{count}_1(C)$ + $\text{count}_2(C)$. This sum is the true global count *because the chunks are disjoint and cover every basket*. C is frequent if the sum ≥ s = 4.
2. **Aggregate and compare with 4:**

Candidate	Chunk 1 + Chunk 2 = total	≥ 4 ?	Verdict
`{apple}`	2 + 1 = 3	No	false positive (3 < 4)
`{banana}`	2 + 2 = **4**	Yes	**FREQUENT**
`{orange}`	3 + 2 = **5**	Yes	**FREQUENT**
`{pear}`	0 + 2 = 2	No	false positive (2 < 4)
`{apple, orange}`	2 + 1 = 3	No	false positive (3 < 4)
`{banana, orange}`	2 + 2 = **4**	Yes	**FREQUENT**
`{banana, pear}`	0 + 2 = 2	No	false positive (2 < 4)
`{orange, pear}`	0 + 2 = 2	No	false positive (2 < 4)
`{banana, orange, pear}` (missing on slide)	0 + 2 = 2	No	false positive (2 < 4)

1. **Result:** 3 of the 9 candidates survive: `{banana}` = 4, `{orange}` = 5, `{banana, orange}` = 4. The other 6 candidates (`{apple}`, `{pear}`, `{apple, orange}`, `{banana, pear}`, `{orange, pear}`, `{banana, orange, pear}`) are **false positives** that Pass 2 removes. On Slide 69 these are the struck-through lines (five of them; the sixth is the omitted triple).
2. **Sanity check directly on the whole file** (not part of SON, only to confirm): banana is in T1, T2, T4, T5 = 4; orange is in all five baskets = 5; apple in T1, T3, T5 = 3; pear in T4, T5 = 2; `{banana, orange}` in T1, T2, T4, T5 = 4; `{apple, orange}` in T1, T3, T5 = 3.
3. **Nothing was missed:** over the four items there are 15 non-empty itemsets; 9 are candidates. The other six — `{apple, banana}` (1 + 1 = 2 in total), `{apple, pear}` (0 + 1 = 1), `{apple, banana, orange}` (1 + 1 = 2), `{apple, banana, pear}` (0 + 1 = 1), `{apple, orange, pear}` (0 + 1 = 1) and `{apple, banana, orange, pear}` (0 + 1 = 1) — have a count of at most 1 in each chunk, so none of them can be locally frequent, and all have totals far below 4. A brute-force count of all 15 itemsets on the five baskets (checked by computer) gives exactly the same three frequent itemsets. So SON returned the exact answer: no false negatives and no false positives.

Part (e) — why SON is correct

(e-i) No false negatives (Slide 57, "monotonicity" idea). A *false negative* would be an itemset that is truly frequent but never reaches the candidate list.

1. **Concrete argument with our numbers:** an itemset that is *not* locally frequent in a chunk has a local count of at most 1 there (the local support is 2). If that happened in *both* chunks, its total would be at most 1 + 1 = 2, which is below 4. So an itemset with total ≥ 4 must have a local count ≥ 2 in at least one chunk, i.e. it is locally frequent there and is output by that mapper. Chunk 1 has 3 baskets (count ≤ 3) and Chunk 2 has 2 baskets (count ≤ 2), so there are only three ways to reach a total of 4 or more, and each is caught:

Chunk 1 count	Chunk 2 count	Total	Caught by (local count ≥ 2)
2	2	4	both chunks
3	1	4	Chunk 1 only (3 ≥ 2)
3	2	5	both chunks

1. **General claim:** if an itemset I has global support ≥ s, then in at least one chunk i its local count is ≥ $p_i \cdot s$, where p_i is the fraction of all baskets that lie in chunk i.
2. **Proof by contradiction:** suppose $\text{count}_i(I) < p_i \cdot s$ in *every* chunk. Add over all chunks (the chunks partition the file, so the local counts add up to the global count): $\text{count}(I) = \sum \text{count}_i(I) < \sum p_i \cdot s = s \cdot \sum p_i = s \cdot 1 = s$. So $\text{count}(I) < s$, which contradicts "I is frequent". Hence some chunk has count ≥ $p_i \cdot s$. This is the slide's wording: *an itemset cannot be frequent in the entire set unless it is frequent in at least one subset*. (It is the same pigeonhole argument as in the exam-paper story.)
3. **Consequence:** if each mapper uses a local threshold that is **at most** $p_i \cdot s$, every truly frequent itemset is output by at least one mapper in Phase 1 (each mapper uses an *exact* in-memory algorithm, so it finds *all* itemsets that are frequent in its chunk). The itemset therefore lands on the candidate list; Pass 2 then counts it exactly and keeps it. Nothing frequent is ever lost.
4. **In our data** ($p_1 \cdot s = 2.4$, $p_2 \cdot s = 1.6$): `{banana}` has local counts 2 and 2; 2 < 2.4 in Chunk 1 but 2 ≥ 1.6 in Chunk 2, so Chunk 2 catches it (2.4 + 1.6 = 4: it cannot be below the line in both). `{banana, orange}` (2 and 2) is caught the same way; `{orange}` (3 and 2) is caught by both chunks (Figure L4-Q6-1).

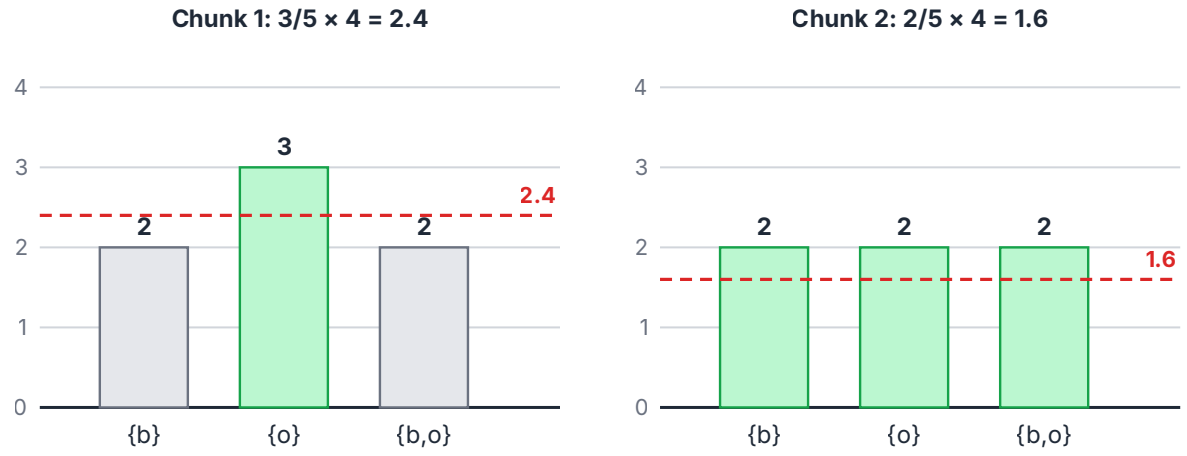


Figure L4-Q6-1. The pigeonhole idea behind "no false negatives", drawn with the proportional thresholds p·s. {banana} and {banana, orange} are below the line in Chunk 1 but above it in Chunk 2, so they are still caught. (The slide's local support 2 is even lower than 2.4 in Chunk 1, so it is safe too.)

KEY IDEA — TWO DIFFERENT "MONOTONICITY" RULES

(1) **A-Priori monotonicity** (inside a mapper): if an itemset is frequent, all its *subsets* are frequent. (2) **SON "monotonicity"** (Slide 57, across mappers): if an itemset is frequent in the whole file, it is frequent in at least one *chunk*. Rule (1) makes each mapper fast; rule (2) is what guarantees SON never misses anything.

(e-ii) Why Pass 2 removes false positives. A *false positive* is an itemset reported as frequent although it is not frequent in the whole data.

1. **Where false positives come from:** a candidate only needs to be frequent in *one* chunk — a local property (Slide 56: "any one or more subsets of the baskets"). It can be popular in one chunk and rare elsewhere. Example: `{pear}` is in 100 % of Chunk 2 (count 2) but 0 in Chunk 1, total 2 < 4; `{apple}` is 2 in Chunk 1 and 1 in Chunk 2, total 3 < 4.
2. **How Pass 2 fixes it:** Pass 2 computes the *exact global support* of each candidate (sum of exact chunk counts) and applies the global test "total ≥ 4". Here 6 of the 9 candidates fail it and are discarded. Without Pass 2 (Slide 60 says the candidates are "optionally evaluated"; skip it only if false positives are acceptable), SON would output all 9 itemsets, 6 of them wrong, with no true supports. So: Pass 1 guarantees *completeness* (no false negatives); Pass 2 guarantees *correctness* (no false positives).

(e-iii) Local threshold versus global threshold.

1. **Beyond the slides:** the textbook rule (*Mining of Massive Datasets*, Leskovec–Rajaraman–Ullman, Section 6.4.3). A mapper whose data holds the fraction p of all baskets uses local threshold $p \cdot s$ (rounded *up* when counts are integers). With k equal chunks this is $s \div k$. (Slide 59 lets a mapper process a *subset of chunks*; then p is the share of baskets that this mapper gets.) The slide uses 2 in both chunks; 2 = 4 ÷ 2 is the right value for two *equal* halves, and here it is also safe for the unequal chunks (table below). The slides do not say how the 2 was chosen.

Chunk	Baskets	Fraction p	$p \cdot s$	Smallest integer count that reaches it	Slide uses	Safe?
Chunk 1	3	3/5 = 0.6	0.6 × 4 = 2.4	3	2	**Yes: 2 ≤ 2.4 (more candidates than needed)**
Chunk 2	2	2/5 = 0.4	0.4 × 4 = 1.6	2	2	**Yes: exactly the tightest safe value**

1. **Integer version of the guarantee:** if the thresholds are t_1 and t_2, an itemset missed by both chunks has counts ≤ $t_1 - 1$ and ≤ $t_2 - 1$, so its total is ≤ $(t_1 - 1) + (t_2 - 1)$. No false negatives are possible as long as this is **below** $s = 4$.
2. **What happens when the local threshold is changed** (every row verified by running SON on the five baskets):

Local thresholds (Chunk 1, Chunk 2)	$(t_1-1)+(t_2-1)<4$?	Number of candidates	Final output	Correct?
(1, 1) — very low	0 + 0 = 0: yes	15	{banana}=4, {orange}=5, {banana, orange}=4	**Yes, but every itemset that occurs is a candidate (12 false positives, wasteful)**
(2, 2) — the slide	1 + 1 = 2: yes	9 (slide: 8)	{banana}=4, {orange}=5, {banana, orange}=4	**Yes (6 false positives)**
(3, 2) — tightest safe: 2.4 and 1.6 rounded up	2 + 1 = 3: yes	7	{banana}=4, {orange}=5, {banana, orange}=4	**Yes, and fewer false positives (4; {apple} and {apple, orange} are no longer candidates)**
(3, 3) — too high for Chunk 2	2 + 2 = 4: no	1	only {orange}=5	No: {banana} and {banana, orange} (both truly 4) are lost: false negatives
(4, 4) — copying the global threshold	3 + 3 = 6: no	0	nothing	No: nothing reaches 4 inside a small chunk

1. **Conclusion:** a **lower** local threshold is always safe but produces more candidates (more memory and more counting in Pass 2); a **higher** local threshold than $p \cdot s$ can break the no-false-negative guarantee. Use exactly (or just below) $p \cdot s$.

> **COMMON MISTAKE**
> **Using the global threshold 4 inside each chunk.** A chunk is only part of the data, so its counts are smaller; with threshold 4 almost nothing would be found in a chunk of 2 or 3 baskets (row (4, 4) above). Also do not confuse "candidate" with "frequent": a candidate is only *locally* frequent in some chunk; only the global sum ≥ s makes it frequent.

Part (f) — the MapReduce view (Slides 58–60, 65–66)

1. **Setting (Slide 58, "SON — Distributed Version"):** the baskets are spread over many nodes (chunk servers); each node computes frequent itemsets on its own data, and then the counts of all candidates are accumulated. SON is thus **two MapReduce jobs in sequence**: Job 1 = Map Phase 1 + Reduce Phase 1 (find candidates, Pass 1); Job 2 = Map Phase 2 + Reduce Phase 2 (count candidates, Pass 2).
2. **The four phases** (Figure L4-Q6-2 shows the data flow for our example; the table after it says what each phase does, and its slide):

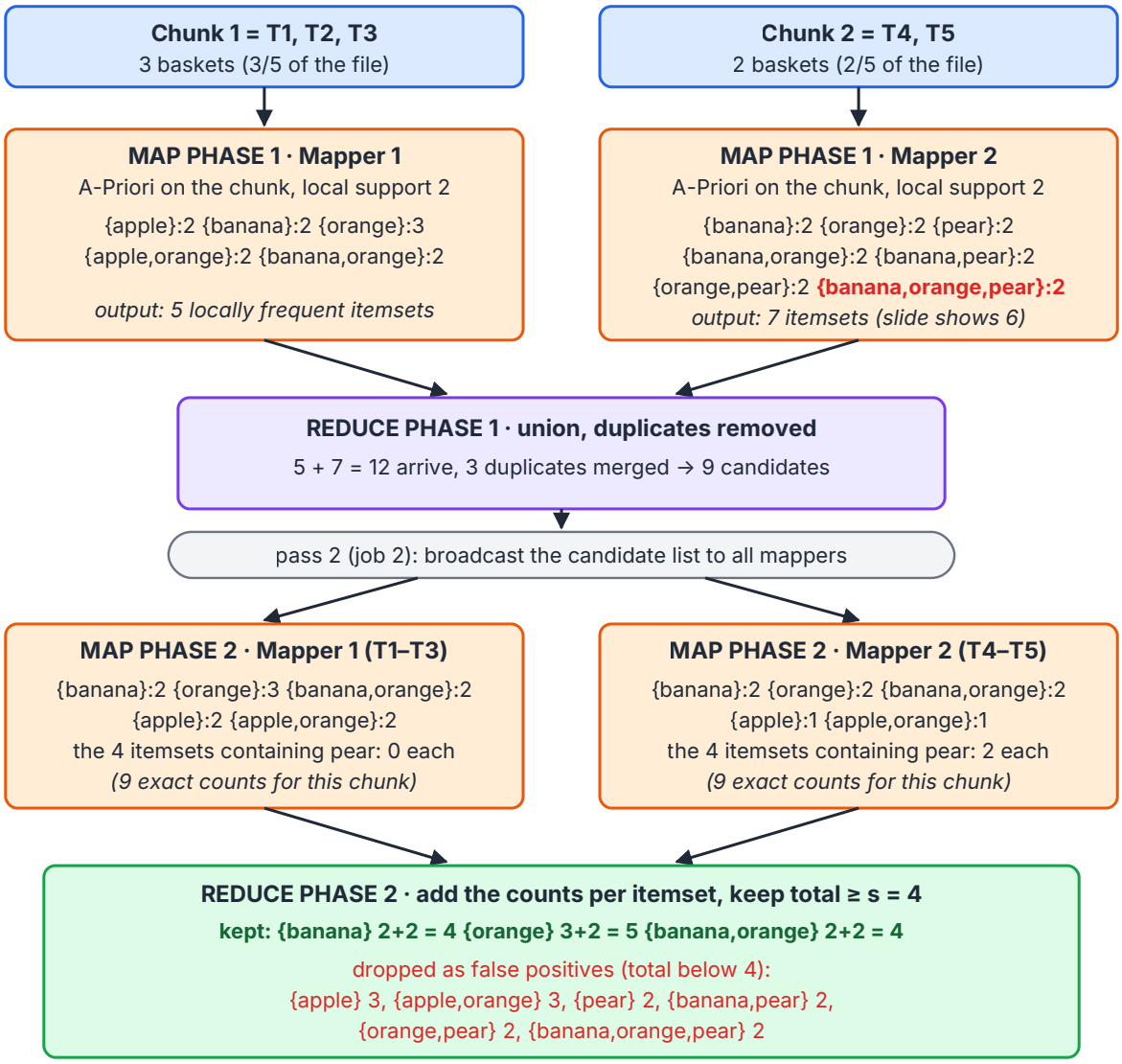


Figure L4-Q6-2. SON on the slide example, phase by phase. Everything above the grey broadcast bar is Pass 1 (MapReduce job 1), everything below it is Pass 2 (job 2). Orange = Map phases (run in parallel, one per chunk), purple = Reduce Phase 1 (union), green = Reduce Phase 2 (sum and filter). The red triple is the itemset missing on Slide 63.

Phase	Input	What it does	Output
Map Phase 1 Slide 59, 63	one or more chunks of baskets per mapper (mappers work independently, in parallel)	runs an in-memory frequent-itemset algorithm (A-Priori, PCY, FP-growth ...) on its chunk with the local threshold	the locally frequent itemsets (the "candidate frequent itemsets"). **Beyond the slides:** in the textbook (MMDS §6.4.4) they are emitted as key–value pairs (itemset, 1)
Reduce Phase 1 Slide 60, 64	all candidate itemsets from all mappers	gathers equal itemsets together and eliminates duplicates; counts are ignored	the global list of candidate itemsets (potential frequent itemsets)
Map Phase 2 Slide 65, 67	the candidate list (broadcast to every mapper) + the mapper's chunk	each mapper counts the support of every candidate in its own chunk, zeros included	(candidate, local count) for each candidate and chunk
Reduce Phase 2 Slide 66, 68–69	all local counts of each candidate	adds the local counts to get the global support and keeps those that meet the threshold	the frequent itemsets with their global supports: {banana}=4, {orange}=5, {banana, orange}=4

1. **Why it suits big data:** each mapper only needs its chunk to fit in main memory; chunks are independent, so the work parallelises over many nodes; the data is read exactly twice (once per job), independent of the itemset size, whereas A-Priori needs k passes for itemsets of size k (Slide 53).

Compare	Random sampling (Slides 54–55)	SON (Slides 56–60)
Which data is mined	a random sample only	the whole file, chunk by chunk (Slide 56: "we are not sampling")
Threshold used	s scaled to the sample size	s scaled to each chunk size ($p \cdot s$)
False negatives	possible: "you do not catch sets frequent in the whole but not in the sample"	**none (monotonicity argument)**
False positives	possible unless a second pass verifies the candidates	**removed by Pass 2 (Reduce Phase 2)**
Passes over the full data	none for mining (one optional pass to verify)	exactly 2

SLIDE ERRATUM (IMPRECISION ON SLIDE 53)

Slide 53 lists SON together with random sampling under "Use 2 or fewer passes for all sizes, *but may miss some frequent itemsets*". That description is true of random sampling, but SON with both passes does **not** miss any frequent itemset (proved in (e-i)); it is an exact algorithm. Write the correct statement in the exam: *SON finds exactly the frequent itemsets in two passes.*

EXAM TIP

For "explain SON" answers, write in this order: (1) split the file into chunks that fit in memory; (2) Pass 1: find frequent itemsets in each chunk with a *lowered local support $p{\cdot}s$*, union them as candidates; (3) Pass 2: count the candidates over all data, keep those ≥ s; (4) *why correct:* frequent overall ⇒ frequent in at least one chunk (no false negatives); Pass 2 removes false positives; (5) MapReduce: Map 1 / Reduce 1 = candidates, Map 2 / Reduce 2 = counts. For a numeric example, show the table of local counts, the candidate list, the per-chunk counts and the sums — each is a mark.

Final answer:
(a) Map Phase 1 (local support 2). Chunk 1: {apple}:2, {banana}:2, {orange}:3, {apple, orange}:2, {banana, orange}:2. Chunk 2: {banana}:2, {orange}:2, {pear}:2, {banana, orange}:2, {banana, pear}: 2, {orange, pear}:2, {banana, orange, pear}:2 (the last one is missing on Slide 63).
(b) Reduce Phase 1: 9 unique candidates = the 8 on Slide 64 plus {banana, orange, pear}.
(c)–(d) Map Phase 2 counts (Chunk 1 | Chunk 2) and Reduce Phase 2 totals: {apple} 2|1 → 3; {banana} 2|2 → 4; {orange} 3|2 → 5; {pear} 0|2 → 2; {apple, orange} 2|1 → 3; {banana, orange} 2|2 → 4; {banana, pear} 0|2 → 2; {orange, pear} 0|2 → 2; {banana, orange, pear} 0|2 → 2. Frequent itemsets (global support ≥ 4): {banana} = 4, {orange} = 5, {banana, orange} = 4. The other 6 candidates are false positives removed in Pass 2.
(e) No false negatives: frequent overall ⇒ local count ≥ p·s in some chunk (else total < s·∑p = s; here: not locally frequent in both chunks ⇒ total ≤ 1 + 1 = 2 < 4). Pass 2 gives exact counts, so no false positives. Local threshold ≤ p·s (2.4 and 1.6 here, i.e. 3 and 2 as integers; the slide uses 2 and 2); never the global 4.
(f) Map 1 = local frequent itemsets per chunk; Reduce 1 = union without duplicates; Map 2 = broadcast candidates, count them per chunk; Reduce 2 = sum the counts, keep ≥ s.

Where the marks come from: (1) stating the support definition and threshold s = 4 on 5 baskets, local support 2; (2) Chunk 1: 3 singletons + 2 pairs with correct counts and the triple correctly excluded; (3) Chunk 2: 3 singletons + 3 pairs and the triple {banana, orange, pear} (the slide's list leaves it out — you still get the final answer without it); (4) Reduce Phase 1: union with duplicates merged, 9 candidates (8 as on the slide is accepted); (5) Map Phase 2: every candidate counted in every chunk, including 0s and the small counts such as {apple}:1; (6) Reduce Phase 2: sums and comparison with 4; (7) the three frequent itemsets with supports 4, 5, 4; (8) no false negatives: argument that frequent overall ⇒ frequent in at least one chunk; (9) Pass 2 removes false positives, with an example such as {pear} or {apple}; (10) local threshold ≈ p·s and why not 4; (11) the roles of the four MapReduce phases.

Slide erratum: Slide 63 omits `{banana, orange, pear}` (local count 2 in Chunk 2), so Slide 64 lists 8 candidates instead of 9 and Slide 67 omits its counts (0 in Chunk 1, 2 in Chunk 2); Slides 68–69 lack its row (0 + 2 = 2, a false positive; the slide strikes out five lines, the corrected list has six). Everything else on Slides 63–69 re-computed correctly (Chunk 1 list, all counts on Slide 67, all sums on Slide 68, the struck-through lines on Slide 69) and the slide's final answer {banana}=4, {orange}=5, {banana, orange}=4 is correct and unchanged. Minor: on Slide 67 the Mapper 2 line lacks separators between items; Slide 69 is Slide 68 repeated with strike-throughs; Slide 53 wrongly groups SON with "may miss some frequent itemsets" (see the imprecision box above).

Sampling versus SON

Lecture 4 · Slides 53–60 (comparison)

	Random sampling (Slides 54–55)	SON (Slides 56–60)
Data used	Only a random sample of the baskets.	**All baskets, in chunks that fit in memory (not sampling).**
Passes over the file	1 (read the sample); 2 with optional verification.	Exactly 2: Pass 1 finds candidates, Pass 2 counts them.
Threshold inside memory	Scaled to the sample size ($p \times s$).	Scaled to each chunk's share of the file ($p_{chunk} \times s$).
False positives	Possible; removed only by the optional second pass.	**Possible in the candidate list; always removed by the (required) second pass.**
False negatives	Possible, and not recoverable.	**None (monotonicity idea, Slide 57).**
Result	Approximate.	**Exact: same as A-Priori on the whole file.**

Parallelism	The slides do not present it as a distributed design (one in-memory sample); SON is the one built for MapReduce.	**Chunks mined independently: maps naturally onto MapReduce (Slides 58–66).**
Cost / weak point	Cheapest. Risk: unlucky sample.	A second full pass; the candidate list can be large if the local threshold is low.

EXAM TIP

"Why does SON never miss a frequent itemset but sampling can?" Sampling looks at only part of the data, so a globally frequent itemset can look rare in the sample. SON looks at every chunk, and a globally frequent itemset must be locally frequent in at least one of them.

Putting it all together: the cat-and-dog baskets

Lecture 4 · Slides 70–77

Slides 70–77 are a pair-by-pair practice session on **one data set**: eight "baskets", each of which is a sentence (a list of words). The words are the items. It reuses everything from the first half of the lecture: support and frequent itemsets (Slides 8, 30), A-Priori (Slides 30–36), confidence (Slide 10) and interest (Slide 11). The questions are posed on Slides 70, 72, 74 and 76; the answers are on Slides 71, 73, 75 and 77.

Slides	Question (threshold s = 3)	Tool you need
70–71	Compute the **frequent pairs**.	A-Priori: Pass 1 counts words, Pass 2 counts pairs of frequent words.
72–73	Are there frequent **triples, quadruples** or larger sets?	Monotonicity: a triple needs all three of its pairs frequent; a quadruple all four triples.
74–75	Confidence of the rule {cat, dog} → and, and of the rule {cat} → kitten.	conf(I → j) = support(I ∪ {j}) ÷ support(I).
76–77	Interest of {dog} → cat and of {cat} → kitten; "interesting"?	Interest = conf − Pr[j], Pr[j] = share of all baskets containing j.

Ground rules the slides leave unsaid

Lecture 4 · Slides 70–71

- **A basket is a set.** A word repeated in one basket ("a … a" in basket 2, "training … training" in basket 4, "and … and" in basket 5) counts **once** for that basket.
- **Capital letters are ignored:** "Cat" and "cat" are the same item. Without this, the answer to Slide 70 would shrink to just two pairs.
- The symbol "&" in basket 6 is just another item, not the word "and".
- Support is a **count of baskets**, and a count of exactly 3 *is* frequent (≥ s).

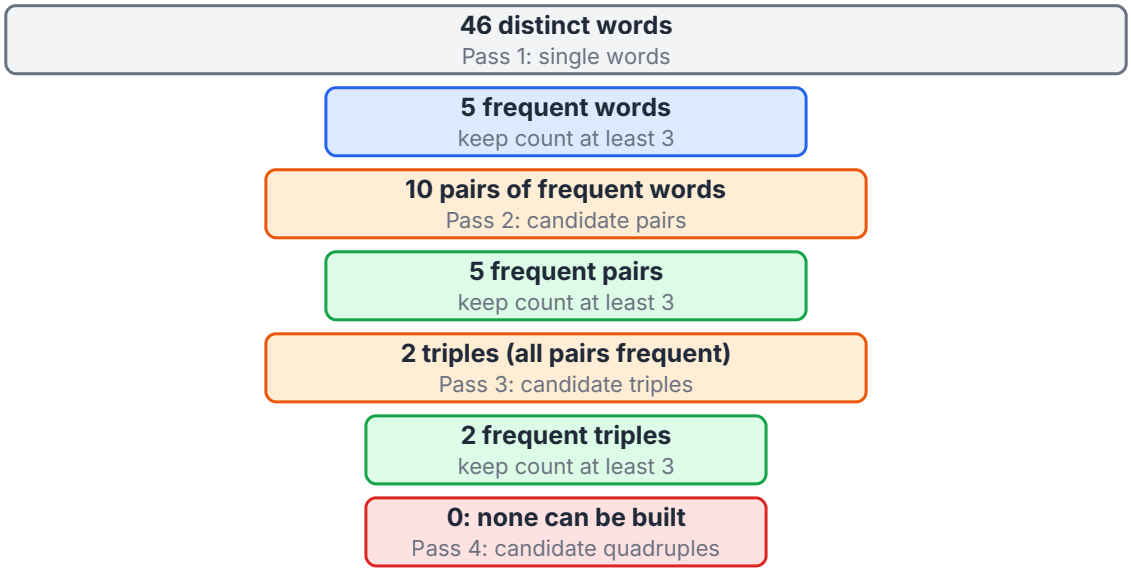


Figure L4p3-5. How the number of survivors shrinks on the cat-and-dog baskets of Slide 70 (s = 3). Quiz L4-Q7 shows every count.

KEY IDEA

This is A-Priori in miniature. Pass 1 shrinks 46 distinct words to 5 frequent ones. Pass 2 only needs the 10 pairs among those 5 (not the roughly one thousand possible pairs of all 46 words). The pairs that survive decide which triples are even worth counting, and so on. Each level uses the previous level's survivors, which is the A-Priori pruning principle on Slide 30.

Errors on the answer slides

Lecture 4 · Slides 71 and 73

SLIDE ERRATUM

Slide 71 prints a table of which baskets contain each pair. Two cells omit basket 7: {dog, and} should list baskets **1, 2, 7, 8** (support 4, slide: 1, 2, 8) and {cat, and} should list **1, 2, 5, 7** (support 4, slide: 1, 2, 5). The five frequent pairs the slide lists are still right, because both 3 and 4 are at least 3.

Slide 73 answers the triples question with only "{dog, cat, a} is a frequent triple". There are **two** frequent triples: the second, {dog, cat, and} (baskets 1, 2, 7), is missing, a knock-on effect of the Slide 71 table error. It also says "There are not frequent quadruples" (should be "no"); the conclusion, no frequent quadruple, is right. The slide shows no reasoning, so a copied answer would earn no method marks. Quiz L4-Q7 gives the full working.

Confidence and interest on real data

Lecture 4 · Slides 74–77

Two lessons come out of Slides 74–77, both worth remembering for the exam:

- **Confidence divides by the support of the *left* side** only (the number of baskets containing everything on the left), not by 8 and not by the support of the right side. Slide 75 reads: of the five baskets with cat and dog, three also have "and".
- **A high-looking confidence can still be uninteresting.** "Cat" is in three quarters of all baskets, so finding it in 5/7 of the "dog" baskets means that dog adds nothing. Interest, being confidence minus the baseline probability, exposes this. Slide 77 concludes that both rules have interest close to 0 and are therefore "not very interesting".

Beyond the slides: in the textbook an association rule counts as interesting if its interest is large in *absolute value* (strongly positive or strongly negative). Slide 11 only counts high positive values. Also, {cat, kitten} has

support 1, below s = 3, so the lecture's pipeline (rules are built from *frequent* itemsets, Slides 16–18) would never even generate the rule cat → kitten. Slide 77 evaluates it anyway as a formula exercise.

Try it yourself first: with the eight baskets in front of you and the ground rules above, work out all four parts before opening the solution.

Quiz L4-Q7 **Cat-and-dog baskets: pairs, triples, confidence, interest** Lecture 4 · Slides 70–77

QUESTION (AS ON THE SLIDES)

Consider the following set of eight baskets. Each basket is a list of words (the words are the items). Assume we set our threshold at s = 3. (Slides 70, 72, 74 and 76 pose the questions; Slides 71, 73, 75 and 77 print the slide answers.)

Basket	Words in the basket (exactly as on Slide 70)
1	{Cat, and, dog, bites}
2	{Yahoo, news, claims, a, cat, mated, with, a, dog, and, produced, viable, offspring}
3	{Cat, killer, likely, is, a, big, dog}
4	{Professional, free, advice, on, dog, training, puppy, training}
5	{Cat, and, kitten, training, and, behavior}
6	{Dog, &, Cat, provides, dog, training, in, Eugene, Oregon}
7	{Dog, and, cat, is, a, slang, term, used, by, police, officers, for, a, male, female, relationship}
8	{Shop, for, your, show, dog, grooming, and, pet, supplies}

(a) Compute the **frequent pairs** of items. Lecture 4 · Slides 70–71

(b) Are there any frequent **triples**, **quadruples** or larger sets? Lecture 4 · Slides 72–73

(c) What is the **confidence** of the rule {cat, dog} → and? What is the confidence of the rule {cat} → kitten? Lecture 4 · Slides 74–75

(d) What is the **interest** of the rule {dog} → cat? What is the interest of the rule {cat} → kitten? Are these rules "interesting"? Lecture 4 · Slides 76–77

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

1. **Tools: the definitions we need** (Lecture 4 · Slides 8, 10, 11, 16, 30, 31).
   - **Support** of an itemset I = the number of baskets that contain *all* items of I (Slide 8). I is **frequent** if it appears in *at least* s baskets, i.e. support(I) ≥ 3. A support of exactly 3 *is* frequent.
   - **Confidence** (Slide 10): $\text{conf}(I \to j) = \frac{\text{support}(I \cup \{j\})}{\text{support}(I)}$, the probability of j given that the basket contains all of I. The denominator is the support of the *left* side (Slide 16 calls this the support of the rule).
   - **Interest** (Slide 11): Interest(I → j) = conf(I → j) − Pr[j], where Pr[j] = (number of baskets containing j) ÷ (total number of baskets, here 8). "Interesting" rules are those with *high* interest, usually above 0.5.
   - **Monotonicity** (Slide 30): if a set is frequent, every subset of it is frequent. Contrapositive: a set with an infrequent subset cannot be frequent. So a pair can be frequent only if both its words are frequent, and a triple only if all three of its pairs are frequent.
   - **How to read the baskets.** A basket is a *set* of words, so a word repeated inside one basket counts once. Capital letters are ignored ("Cat" = "cat", "Dog" = "dog"). The symbol "&" in basket 6 is just another token, not the word "and".

> **BEGINNER INTUITION**
> Think of each basket as one shopping trip and each word as an item bought on that trip. **Support** = "on how many trips did these words show up together?". **Frequent** = "on at least 3 trips". **Confidence** of "left side → right word" = "of the trips that had the whole left side, what fraction also had the right word?". **Interest** asks whether that fraction beats the right word's everyday popularity: if a word is in 75 % of all trips anyway, finding it in 71 % of the trips that contain another word tells you nothing new.

2. **Clean the data: each basket becomes a set of lower-case words.** *Why:* support counts *baskets* (Slide 8), so a word written twice in one basket counts once, and "Cat"/"cat" (also "Dog"/"dog") must be the same word. Repeats that disappear: "a" in baskets 2 and 7, "training" in basket 4, "and" in basket 5, "dog" in basket 6. The "&" in basket 6 stays as its own token (it is *not* "and"). After cleaning, the eight baskets contain **46 different words** (checked by program).
3. **(a) Pass 1 of A-Priori: count every single word** (Slide 31). *Why:* by monotonicity a pair can be frequent only if *both* of its words are frequent, so the single-word counts tell us which pairs are worth counting at all.

Word	Baskets containing it	Support	≥ 3 ?
`dog`	1, 2, 3, 4, 6, 7, 8	7	**frequent**
`cat`	1, 2, 3, 5, 6, 7	6	**frequent**
`and`	1, 2, 5, 7, 8	5	**frequent**
`a`	2, 3, 7	3	**frequent**
`training`	4, 5, 6	3	**frequent**
`for`	7, 8	2	no
`is`	3, 7	2	no
each of the other 39 words (e.g. `kitten`, `bites`, `puppy`, `&`)	one basket each	1	no

   The frequent words are L_1 = {dog, cat, and, a, training}; the nearest misses "for" and "is" have support 2. So we count only C(5, 2) = 10 candidate pairs instead of C(46, 2) = 1035.
4. **(a) Pass 2 of A-Priori: count the 10 candidate pairs** (Slide 31). First, for every basket list which of the five frequent words it contains; a basket with n frequent words adds $C(n, 2)$ pairs:

Basket	Frequent words it contains	How many	Pairs it adds, C(n, 2)
1	`dog, cat, and`	3	3
2	`dog, cat, and, a`	4	6
3	`dog, cat, a`	3	3
4	`dog, training`	2	1
5	`cat, and, training`	3	3
6	`dog, cat, training`	3	3
7	`dog, cat, and, a`	4	6
8	`dog, and`	2	1
Total			**26**

   Now mark, for each candidate pair, the baskets that contain both words:

	Basket number (✓ = the basket contains both words)									
Candidate pair	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**Support**	**≥ 3 ?**
`{dog, cat}`	✓	✓	✓			✓	✓		**5**	**yes**
`{dog, and}`	✓	✓					✓	✓	**4**	**yes**
`{cat, and}`	✓	✓			✓		✓		**4**	**yes**
`{dog, a}`		✓	✓				✓		**3**	**yes**
`{cat, a}`		✓	✓				✓		**3**	**yes**
`{a, and}`		✓					✓		2	no
`{dog, training}`				✓		✓			2	no
`{cat, training}`					✓	✓			2	no
`{and, training}`					✓				1	no
`{a, training}`									0	no
Column total									**26**	

Sanity check: the pairs added per basket total 26, and the supports 5 + 4 + 4 + 3 + 3 + 2 + 2 + 2 + 1 + 0 also total 26. Baskets 2 and 7 are the busiest: each contains dog, cat, and, a, so each counts for six pairs.

5. **(a) Keep the pairs with support ≥ 3.** L_2 = {dog, cat} (5; baskets 1, 2, 3, 6, 7), {dog, and} (4; baskets 1, 2, 7, 8), {cat, and} (4; baskets 1, 2, 5, 7), {dog, a} (3; baskets 2, 3, 7) and {cat, a} (3; baskets 2, 3, 7). The other five fail: {a, and} 2, {dog, training} 2, {cat, training} 2, {and, training} 1, {a, training} 0. *Double-check:* a brute-force count over every pair that occurs anywhere in the baskets (271 distinct pairs, not only pairs of frequent words) finds no other pair with support 3; the best misses, such as {dog, is} and {a, and}, have support 2. So **there are exactly five frequent pairs**.
6. **(b) Frequent triples.** *Idea:* by monotonicity a triple can be frequent only if *all three* of its pairs are in L_2. Of the four triples that can be made from {dog, cat, and, a}, two pass this test; we count all four to be sure. Note that {a, and} is *not* in L_2 (support 2).

Triple	Its three pairs (in L_2?)	Baskets with all 3 words	Support	Verdict
`{dog, cat, a}`	{dog,cat} ✓, {dog,a} ✓, {cat,a} ✓	2, 3, 7	3	**frequent**
`{dog, cat, and}`	{dog,cat} ✓, {dog,and} ✓, {cat,and} ✓	1, 2, 7	3	**frequent**
`{dog, and, a}`	{dog,and} ✓, {dog,a} ✓, {and,a} × (2)	2, 7	2	pruned
`{cat, and, a}`	{cat,and} ✓, {cat,a} ✓, {and,a} × (2)	2, 7	2	pruned
the 6 triples containing `training`	each contains a pair with “training”, and no such pair is frequent	–	at most 1	pruned

So **two triples are frequent**: {dog, cat, a} (baskets 2, 3, 7) and {dog, cat, and} (baskets 1, 2, 7), each with support exactly 3 = s. A brute-force count over every possible triple of words confirms that these are the only two. The slide builds C_3 a little more loosely (Slide 35: C_k from L_{k-1} and L_1, giving 9 distinct candidate triples); their supports are 3, 3, 2, 2, 1, 1, 0, 0, 0, so L_3 is the same two triples. (Slide 73 lists only the first triple: see the Slide erratum below.)

7. **(b) Quadruples and larger sets.** A quadruple needs all four of its sub-triples to be frequent. Among the five frequent words, four of the five 4-sets contain `training` (excluded as before), leaving only {dog, cat, and, a}. Its sub-triples are {dog,cat,and} ✓, {dog,cat,a} ✓, {dog,and,a} × and {cat,and,a} ×, so it is pruned. Counting it anyway: it is in baskets 2 and 7 only → support 2 < 3. **No frequent quadruple**, and therefore no frequent set of 5 or more words (such a set would contain a frequent quadruple, by monotonicity). The largest frequent itemsets have size 3. In total there are 5 + 5 + 2 = **12 frequent itemsets** (5 words, 5 pairs, 2 triples).
8. **(c) Confidence of {cat, dog} → and.** Formula: conf = support({cat, dog, and}) ÷ support({cat, dog}).
   - support({cat, dog}) = 5: baskets 1, 2, 3, 6, 7 contain both cat and dog.

- support({cat, dog, and}) = 3: of those five, only baskets 1, 2, 7 also contain “and” (basket 3 has no “and”; basket 6 has only the symbol “&”).
- conf = $\frac{3}{5}$ = 0.6, i.e. 60 % of the baskets that contain cat and dog also contain “and”.

9. **(c) Confidence of {cat} → kitten.** Here I = {cat} and j = kitten.
   - support({cat}) = 6: baskets 1, 2, 3, 5, 6, 7.
   - support({cat, kitten}) = 1: only basket 5 contains “kitten” (it appears in no other basket).
   - conf = $\frac{1}{6}$ ≈ 0.167 (about 17 %).

   Link to Slides 16–17: because {cat, kitten} is not a frequent itemset (support 1 < 3), the two-step method of Slide 17 would never generate this rule; the quiz simply asks us to compute its confidence.

10. **Picture of the counts used in (a)–(c).** Each row is a word or itemset; a filled square means that basket contains *every* word of the row. Support is the number of filled squares. Reading downwards, adding a word can only *remove* filled squares (monotonicity). Confidence will be a ratio of two rows.

Basket number	1	2	3	4	5	6	7	8	support	s = 3 ?
dog	■	■	■	■		■	■	■	7	frequent
cat	■	■	■		■	■	■		6	frequent
and	■	■			■		■	■	5	frequent
a		■	■				■		3	frequent
kitten					■				1	not frequent
{dog, cat}	■	■	■			■	■		5	frequent
{cat, kitten}					■				1	not frequent
{dog, cat, and}	■	■					■		3	frequent
{dog, cat, a}		■	■				■		3	frequent
{dog, cat, and, a}		■					■		2	not frequent

Filled square = this basket contains every word in the row label.
Support = number of filled squares.
Adding a word to a row can only remove filled squares.

Figure L4-Q7-a: basket-membership strips. Green numbers = frequent (support ≥ 3), red = not frequent. E.g. conf({cat, dog} → and) = filled squares in row {dog, cat, and} ÷ filled squares in row {dog, cat} = 3 ÷ 5 (Lecture 4 · Slides 8, 10, 30).

11. **(d) Interest of {dog} → cat.** Formula: Interest = conf − Pr[cat].
    - conf({dog} → cat) = support({dog, cat}) ÷ support({dog}) = $\frac{5}{7}$ ≈ 0.714 (dog is in baskets 1, 2, 3, 4, 6, 7, 8; five of them, 1, 2, 3, 6, 7, also contain cat).
    - Pr[cat] = fraction of *all* 8 baskets containing cat = $\frac{6}{8} = \frac{3}{4}$ = 0.75.
    - Interest = $\frac{5}{7} - \frac{3}{4} = \frac{20}{28} - \frac{21}{28} = -\frac{1}{28}$ ≈ **−0.036**.

12. **(d) Interest of {cat} → kitten.**
    - conf({cat} → kitten) = $\frac{1}{6}$ (from part (c)).
    - Pr[kitten] = $\frac{1}{8}$ = 0.125 (only basket 5 of the 8 baskets contains kitten).
    - Interest = $\frac{1}{6} - \frac{1}{8} = \frac{4}{24} - \frac{3}{24} = \frac{1}{24}$ ≈ **0.042**.

Rule	Confidence	Pr[right side]	Interest	Interesting?
{dog} → cat	5/7 ≈ 0.714	6/8 = 0.75	−1/28 ≈ −0.036	no
{cat} → kitten	1/6 ≈ 0.167	1/8 = 0.125	1/24 ≈ 0.042	no

13. **(d) Are the rules "interesting"?** Slide 11: interesting rules are those with a *high* interest value, usually above 0.5. Here the interest values are −0.036 and +0.042, both essentially 0 and far below 0.5, so **neither rule is interesting** (the slide says "not very interesting"). In words: {dog} → cat has a fairly high confidence (0.714), but cat is already in 75 % of all baskets, so learning that a basket has "dog" does not make "cat" any more likely (71 % versus 75 %). This is exactly the "high confidence is not enough" warning of Slide 11. For {cat} → kitten, "kitten" is rare (1 of 8 baskets) and having "cat" lifts its chance only from 12.5 % to 16.7 %, based on a single basket, which is no real evidence of a link.

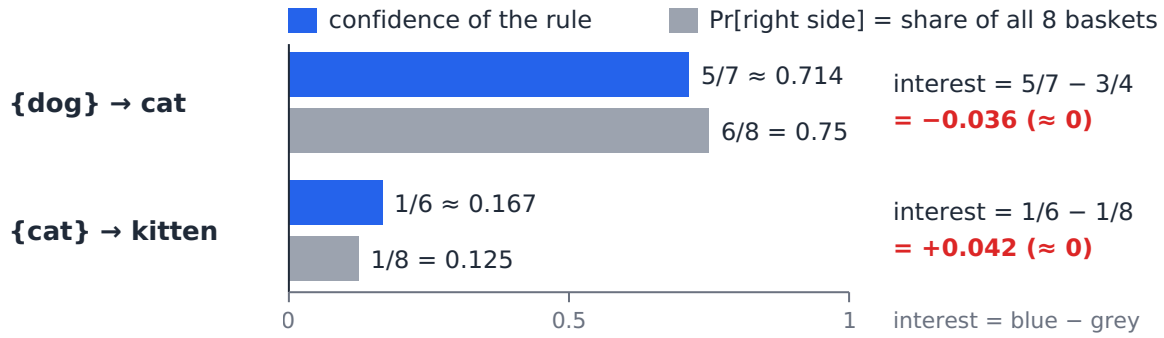


Figure L4-Q7-b: interest is the gap between the blue bar (confidence) and the grey bar (how common the right-hand word is anyway). Both gaps are tiny, so both rules are uninteresting (Lecture 4 · Slide 11).

Beyond the slides: the textbook (*Mining of Massive Datasets*) also counts a very *negative* interest (the left side makes the right side much *less* likely) as notable. −0.036 is not that either, so the conclusion is the same.

COMMON MISTAKES

- **Counting occurrences instead of baskets.** "training" is written 4 times but lives in only 3 baskets (4, 5, 6); "a" is written 5 times but is in only 3 baskets; "dog" 8 times in 7 baskets; "and" 6 times in 5 baskets. Support counts baskets, so always use the *set* of words of a basket.
- **Treating "&" (basket 6) as "and".** That would give {cat, dog, and} support 4 and confidence 4/5 instead of 3/5. Likewise do not treat "Cat" and "cat" as different words.
- **Using "> 3" instead of "≥ 3".** {dog, a}, {cat, a}, {dog, cat, a} and {dog, cat, and} all have support exactly 3 and are frequent (Slide 8: "at least s").
- Calling {a, and} frequent: it appears together only in baskets 2 and 7, support 2. That is why {dog, and, a} and {cat, and, a} fail.
- **Stopping at the slide's single triple** {dog, cat, a} and missing {dog, cat, and} (see the Slide erratum).
- **Dividing by the wrong number.** Confidence divides by the support of the *left side* (5 for {cat, dog}, 6 for {cat}), not by 8; Pr[j] divides by 8, the number of all baskets. And Interest = conf − Pr[j], not Pr[j] − conf.

EXAM TIP

Write the basket numbers next to every count (for example "baskets with both cat and dog: 1, 2, 3, 6, 7 → 5; of these, the ones with the word and: 1, 2, 7 → 3"). It makes each step checkable, so partial marks are easy to award; method marks go to the formula, the two supports and the final fraction. For (b) name *both* triples and say why {dog, and, a} and {cat, and, a} fail ({and, a} is not frequent). For (d) always give the number, compare it with the "usually above 0.5" guide and add one sentence of reason (the base rate Pr[j]): "near 0, far below 0.5, so not interesting".

SLIDE ERRATUM

- **Slide 71, table of pairs:** it lists {dog, and} in baskets 1, 2, 8 and {cat, and} in baskets 1, 2, 5. Basket 7 ("Dog, and, cat, ...") contains dog, and, cat, so the correct lists are **1, 2, 7, 8** and **1, 2, 5, 7**, and both pairs have support **4**, not 3. The slide contradicts itself, because Slide 75 uses basket 7 for "and". The list of five frequent pairs is *not* affected, and every other cell of the slide's table is correct.
- **Slide 73:** it names only {dog, cat, a} as a frequent triple. **{dog, cat, and} is also frequent** (baskets 1, 2, 7 → support 3 = s; its pairs have supports 5, 4, 4). Slide 75 itself proves it: it says "and" appears with cat and dog in baskets (1), (2) and (7). A likely cause of the slip (our inference): with basket 7 missing from Slide 71's table, {dog, cat, and} would look like it has support 2. "No frequent quadruples" on Slide 73 is correct.

Final answer: (a) five frequent pairs: {dog, cat} = 5, {dog, and} = 4, {cat, and} = 4, {dog, a} = 3, {cat, a} = 3 (frequent words: dog 7, cat 6, and 5, a 3, training 3). (b) Two frequent triples: {dog, cat, a} (support 3, baskets 2, 3, 7) and {dog, cat, and} (support 3, baskets 1, 2, 7); no frequent quadruples (the only candidate {dog, cat, and, a} has support 2), hence nothing larger. (c) conf({cat, dog} → and) = 3/5 = 0.6; conf({cat} → kitten) = 1/6 ≈ 0.167. (d) Interest({dog} → cat) = 5/7 − 3/4 = −1/28 ≈ −0.036; Interest({cat} → kitten) = 1/6 − 1/8 = 1/24 ≈ 0.042. Both are close to 0 (far below 0.5), so neither rule is interesting.

Where the marks come from: (1) Correct definitions: support counts baskets, frequent means support ≥ 3, baskets treated as sets and words case-insensitively. (2) Single-word counts and the five frequent words (dog 7, cat 6, and 5, a 3, training 3), with the monotonicity reason for counting only pairs of frequent words. (3) The ten pair counts and the list of the five frequent pairs. (4) Triples: candidates chosen by monotonicity, counted (baskets 2, 3, 7 and 1, 2, 7), both frequent triples named, {and, a} named as the reason the other two fail; quadruple {dog, cat, and, a} shown to have support 2, so none and nothing larger. (5) Confidence: formula, support({cat, dog}) = 5 and support({cat, dog, and}) = 3 giving 3/5; support({cat}) = 6 and support({cat, kitten}) = 1 giving 1/6. (6) Interest: formula conf − Pr[j]; conf 5/7 and Pr[cat] = 3/4 giving −0.036; conf 1/6 and Pr[kitten] = 1/8 giving 0.042. (7) Conclusion "not interesting" with the reason (values near 0, not above 0.5; base rate).

Slide check: (a) matches the slide's five frequent pairs, but Slide 71's table omits basket 7 for {dog, and} and {cat, and} (true supports are 4 and 4, not 3 and 3); the answer is unchanged. (b) Slide erratum: Slide 73 lists only {dog, cat, a}; {dog, cat, and} (baskets 1, 2, 7) is also a frequent triple. "No frequent quadruples" matches. (c) matches (3/5 and 1/6). (d) matches (−0.036 and 0.042; "not very interesting"). Every number above was recomputed by program from the eight baskets.

More practice slides: A-Priori, PCY bitmap, Multistage

Lecture 4 · Slides 78–84

The last three quizzes revisit algorithms from earlier parts of the lecture. This section says what each slide asks and what a full-mark answer must contain. The worked, step-by-step solutions are the solved quizzes L4-Q8, L4-Q9 and L4-Q10 in this chapter; they are not repeated here.

A-Priori on four integer baskets

Lecture 4 · Slides 78–80

Worked solution: Quiz L4-Q8, (p. 204).

Question (Slide 78): find the frequent pairs with the A-Priori algorithm, threshold s = 3, for the four baskets a) {1, 2, 4, 5, 8, 9}, b) {1, 4, 7, 8, 9}, c) {1, 2, 5, 9}, d) {1, 2, 5, 7, 8}. The labels a)–d) name the four *baskets*; they are not four sub-questions.

- **Slide 79 (Pass 1):** count every single item; the items with count at least 3 are 1, 2, 5, 8, 9. Items 4 and 7 (count 2) are dropped and never used again.
- **Slide 80 (Pass 2):** count only pairs of frequent items (10 candidate pairs); the frequent pairs are {1,2}, {1,5}, {1,8}, {1,9}, {2,5}.

SLIDE ERRATUM

The slide answers are correct but show **no counts** (not the Pass 1 item counts and not the supports of the 10 candidate pairs), no list of candidate pairs and no mention that pairs containing 4 or 7 are skipped. In an exam those intermediate steps carry the method marks, so write them down. The slide also spells it "Apriori" and "greater or equal 3" (should be "greater than or equal to 3"). See Quiz L4-Q8.

PCY: the bitmap, how it is used and why it exists

Lecture 4 · Slides 81–82

Worked solution: Quiz L4-Q9, (p. 214).

Questions (Slide 81): (1) describe how the bitmap is used in PCY; (2) why is the Pass-1 hash table transformed into a bitmap? The slide shows PCY's two memory layouts.

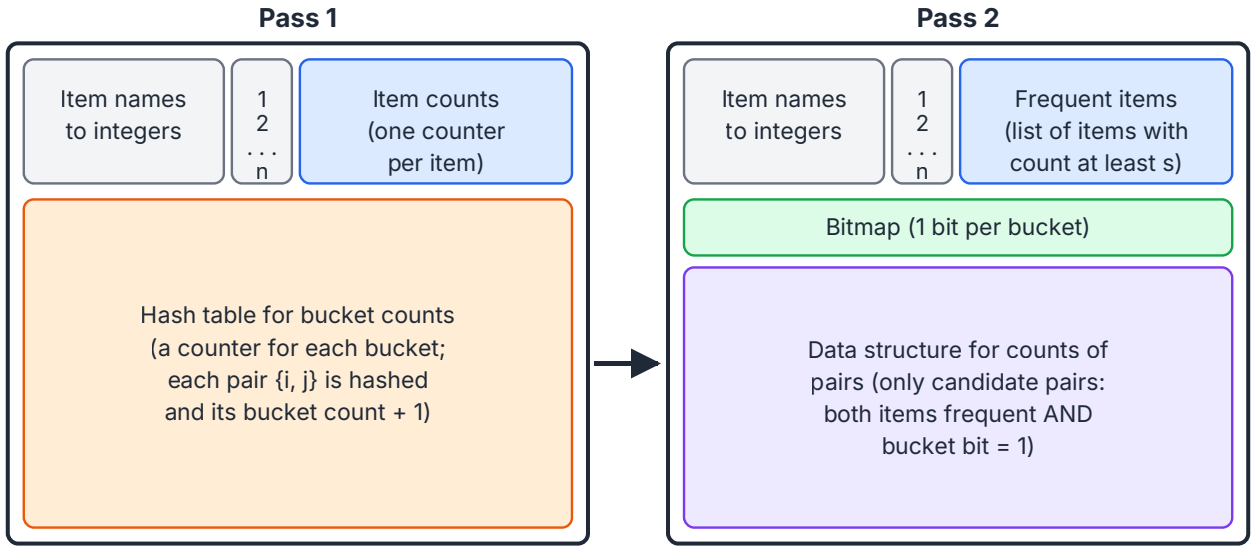


Figure L4p3-6. PCY's two memory layouts (Slides 81 and 44). The big hash table is squeezed into a thin bitmap, freeing room for pair counters. Not to scale.

Question	Full-mark answer
How is the bitmap used?	Between the passes the hash table is summarised as a bitmap with **one bit per bucket**: 1 if the bucket's count reached the threshold s (a *frequent bucket*), 0 if not. In Pass 2 a pair $\{i, j\}$ is counted **only if** both i and j are frequent items **and** the pair hashes (with the same hash function as in Pass 1) to a bucket whose bit is 1 (Slide 43).
Why a bitmap?	A counter is a 32-bit integer, but we only need to know "frequent or not". One bit replaces 32, so the bitmap takes **1/32 of the space** and leaves the memory free for the counters of candidate pairs, which is PCY's bottleneck.

SLIDE ERRATUM

The answer on Slide 82 explains how the bitmap is *built* (one bit per bucket, 1 if frequent) and how small it is (1/32), but never says how it is *used* in Pass 2. The use is the candidate-pair test of Slide 43 above. "The figure" mentioned in the answer is on Slide 81, not Slide 82. The question's "hash map" means the hash table. The slide's bitmap box is drawn much larger than 1/32 of the table; the figure is only schematic. See Quiz L4-Q9. **Beyond the slides** (a sanity check): 10^8 buckets need 4×10^8 bytes = 400 MB as 4-byte counters, but only 10^8 bits = 12.5 MB as a bitmap.

Multistage: the key idea

Lecture 4 · Slides 83–84

Worked solution: Quiz L4-Q10, (p. 225).

Question (Slide 83): describe the key idea behind the multistage algorithm. The slide shows three memory layouts. **Slide 84's answer:** "The Multistage Algorithm uses additional hash tables to reduce the number of candidate pairs."

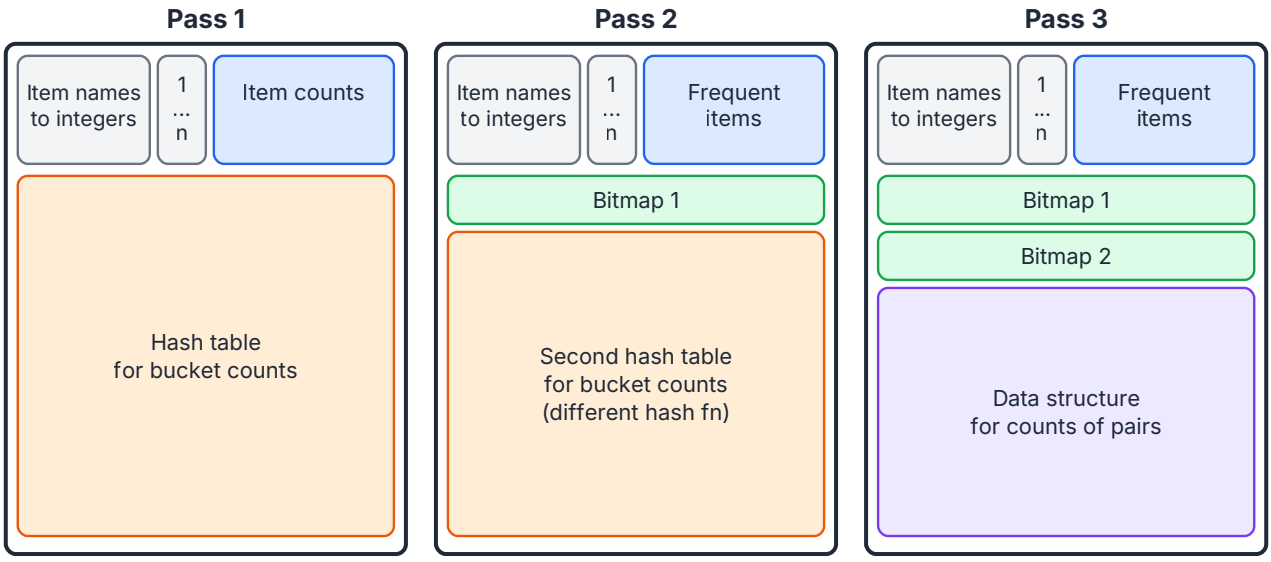


Figure L4p3-7. Multistage's three memory layouts (Slides 83-84). Each hash table shrinks to a bitmap; only pairs passing every bitmap get counted in Pass 3.

For full marks expand the one-line answer with what the figure shows:

1. **Pass 1** is PCY's Pass 1: count items and hash all pairs into hash table 1.
2. **Between passes:** items become a list of frequent items; hash table 1 becomes Bitmap 1.
3. **Pass 2** (the extra pass): hash the candidate pairs again with a **different** hash function into a second hash table; afterwards it becomes Bitmap 2.
4. **Pass 3:** count a pair only if both items are frequent **and** it hits a frequent bucket in **Bitmap 1 and Bitmap 2**.
5. **Why:** memory for the pair counters is the bottleneck, and more filters mean fewer candidate pairs. **Cost:** one extra pass (three passes in total).

SLIDE ERRATUM

The one-line answer on Slide 84 omits the "different hash function", the "frequent in *every* bitmap" condition and the extra pass. "Ans" there also lacks the colon used on the other answer slides. For Multihash (the other PCY refinement, Slides 49–51) the extra tables are built *together* in the first pass, so no extra pass is needed, at the price of smaller tables; Quiz L4-Q10 covers both.

Chapter summary and what to read next

Lecture 4 · Slides 85–86

The summary slide

Lecture 4 · Slide 85

Slide 85 states the lecture in two bullets:

- **Frequent itemset mining algorithms.** We explored in-memory algorithms like **Apriori, PCY, Sampling and SON**, which find frequent itemsets in large datasets by reducing the search space with candidate generation and pruning.
- **MapReduce SON algorithm.** SON adapts frequent-itemset mining to the distributed framework **MapReduce**.

SLIDE ERRATUM

The summary is incomplete as a revision list. It leaves out association rules, confidence and interest (the first half of the lecture) and PCY's Multistage and Multihash refinements. It also lists SON among "in-memory" algorithms and again as the MapReduce algorithm, and it does not say Sampling is approximate. Use the table below instead.

Algorithm	Key trick	Passes	Exact?
A-Priori	Monotonicity: only count itemsets whose subsets are all frequent.	k for size k	Yes
PCY	Use idle memory in Pass 1 for a hash table of pair counts; summarise as a bitmap; skip pairs in infrequent buckets.	2 (pairs)	Yes
Multistage	Extra hash tables with different hash functions, one extra pass each.	3 (pairs)	Yes
Multihash	Several independent hash tables in Pass 1.	2 (pairs)	Yes
Random sampling	Mine a random sample in memory with a scaled threshold.	1 (+1 to verify)	No (false negatives)
SON	Mine each chunk locally, union = candidates, count candidates over everything.	2	**Yes**
SON on MapReduce	Map 1 / Reduce 1 build candidates; Map 2 / Reduce 2 count them.	2 rounds	**Yes**

Actions

Lecture 4 · Slide 86

Slide 86 is the instructor's to-do list: **review the slides, quizzes and examples**, and **read Chapter 6 of the course book**, whose electronic version is on eClass (York's learning platform). **Beyond the slides:** the book is *Mining of Massive Datasets* by Leskovec, Rajaraman and Ullman, and its Chapter 6, "Frequent Itemsets", covers the market-basket model, A-Priori, PCY, Multistage, Multihash, the limited-pass algorithms (sampling and SON) and the SON MapReduce formulation. Sections 6.4.1 (simple randomized algorithm) and 6.4.3–6.4.4 (SON and its MapReduce form) are the ones behind this part of the guide.

QUICK RECAP

- A-Priori and PCY need about k passes for size-k itemsets; sampling and SON need at most 2 (Slide 53).
- **Sampling:** scaled threshold, in-memory run; false positives fixable with a second pass, false negatives not (Slides 54–55).
- **SON:** not sampling; chunks, candidate = locally frequent anywhere; second pass counts candidates; frequent overall ⇒ frequent in at least one chunk (Slides 56–57).
- **SON MapReduce:** Map 1 mines chunks; Reduce 1 unions candidates; Map 2 counts broadcast candidates; Reduce 2 sums and filters (Slides 58–60, 65–66). Phase 2 is required, not optional (Slide 60 erratum).
- The worked SON example needs 9 candidates (the slides list 8) and ends with {banana}, {orange}, {banana, orange} (Slides 61–69).
- Slide 71's table and Slide 73's answer have errors (basket 7, second frequent triple); confidence divides by the left-side support; interest = confidence − Pr[j] (Slides 70–77).

Theory Q&A bank

Lecture 4 · Slides 1–86

Cover the answer, write your own, then compare. Numbered points are marking points. These questions are self-made practice except where a slide poses the question itself (Q4, Q10 and Q12 come from Slides 22, 81–82 and 83–84). Lecture 4 contains no BFR material (BFR belongs to Lecture 5), so no BFR question appears here.

Q1. Define support, support threshold, frequent itemset and confidence.

Lecture 4 · Slides 8, 10

1. **Support** of an itemset I = the number of baskets that contain all items of I.
2. **Support threshold** s = a basket count chosen by the analyst; I is **frequent** if support(I) ≥ s ("at least", so equality counts).
3. **Confidence** of the rule I → j = support(I ∪ {j}) ÷ support(I): of the baskets containing I, the fraction that also contain j.
4. Example (Slide 9 data, s = 3): support({m,b}) = 4, so {m,b} is frequent.

Q2. Why is a high-confidence rule not necessarily interesting? What is the interest measure?

Lecture 4 · Slides 11–15

1. If j is bought very often anyway (for example milk), X → j has high confidence for almost every X, although X tells us nothing about j.
2. **Interest(I → j) = conf(I → j) − Pr[j]**, where Pr[j] = fraction of all baskets that contain j. It measures how much knowing I changes the chance of j.
3. Interest near 0 means no real association; large positive means a real one. The slide treats high values (usually above 0.5) as interesting. Beyond the slides: the textbook counts large negative interest as interesting too (the left side makes j much less likely), so judge by the size of |interest|.
4. Example: {m,b} → c has conf 1/2, Pr[c] = 5/8, interest = −1/8: not interesting.

Q3. State the association-rule mining problem and the two-step recipe. Which step is hard and why?

Lecture 4 · Slides 16–17

1. **Problem:** find all rules with support ≥ s and confidence ≥ c.
2. **Step 1:** find all frequent itemsets I.
3. **Step 2:** for every non-empty proper subset A of each frequent I, form A → I \ A, compute conf = support(I) ÷ support(A), and keep it if ≥ c. The needed supports are already known because A is a subset of a frequent set, hence frequent (monotonicity).
4. **Step 1 is the hard part**: it must count a huge number of candidate itemsets over a file too large for memory; Step 2 only divides numbers Step 1 produced.

Q4. Swapping in and out of memory is a disaster. Why? (the slide's own question)

Lecture 4 · Slide 22

1. Counting increments counters at essentially random positions, one per pair occurrence.
2. If the counters do not fit in RAM, most increments hit a page that is on disk, so each costs a disk access (milliseconds) instead of a memory access (about 100 nanoseconds): thousands of times slower (thrashing).
3. Design rule: only count as many things per pass as fit in main memory, even if that needs more passes.

Q5. Why is finding frequent pairs the hardest part of the problem?

Lecture 4 · Slide 23

1. Frequent pairs are common; frequent triples are rare, because the probability that k items appear together drops exponentially with k.

2. Single items need only n counters, but there are about $n^2/2$ possible pairs, so pair counters are the first level that strains memory.
3. Once the frequent pairs are known, only triples whose sub-pairs are frequent need counting, which is a small set. So algorithms concentrate on pairs first.

Q6. Compare the triangular matrix and the triples method for counting pairs. When is each better?

Lecture 4 · Slides 24–27

1. **Triangular matrix:** a flat array with one 4-byte counter for each of the T = n(n−1)/2 possible pairs; needs items numbered 1..n; memory 4·T whether or not the pair occurs.
2. **Triples [i, j, c]:** one 12-byte record for each pair that actually occurs (O of them); memory 12·O; needs a lookup structure such as a hash table.
3. **Break-even:** $12 \cdot O < 4 \cdot T \iff O < T/3$. Triples win only if fewer than 1/3 of all possible pairs occur.
4. Example: $n = 10^5$ gives $T \approx 5 \times 10^9$ pairs and $4 \cdot T \approx 20$ GB for the matrix.

Q7. State the A-Priori property and explain the two passes for pairs.

Lecture 4 · Slides 30–32

1. **Monotonicity:** if a set appears in at least s baskets, so does every subset. Contrapositive: if item i is infrequent, no pair containing i can be frequent.
2. **Pass 1:** count each item (memory proportional to the number of items). Keep the items with count ≥ s.
3. **Pass 2:** for each basket generate only pairs of frequent items and count them (memory about m(m−1)/2 counters for m frequent items). Pairs with count ≥ s are the frequent pairs.
4. It is exact: every discarded pair had an infrequent item.

Q8. Explain C_k and L_k and how A-Priori finds all frequent itemsets. Why renumber the items?

Lecture 4 · Slides 33–36

1. $\mathbf{C_k}$ = candidate k-itemsets (not yet ruled out by the pass for k−1); $\mathbf{L_k}$ = the truly frequent ones. C_1 = all items → count → L_1 → construct C_2 → count → L_2 → … until L_k is empty.
2. Construct C_{k+1} from L_k (and L_1); the standard rule also requires every k-subset of a candidate to be in L_k.
3. One pass for each k, so the number of passes equals the size of the largest itemset. k = 2 needs the most memory.
4. **Renumbering:** the triangular matrix needs ids 1..n with no gaps. Renumber the m frequent items 1..m and keep a table new number → old item number; the matrix then has only m(m−1)/2 cells.

Q9. Describe the PCY algorithm. Why can it never lose a frequent pair?

Lecture 4 · Slides 38–43

1. **Observation:** in A-Priori Pass 1 most memory is idle. PCY uses it for a hash table of buckets.
2. **Pass 1:** count items and hash every pair of every basket to a bucket; each bucket stores only a count, not the pairs.
3. **Between passes:** bucket → bit (1 if count ≥ s); items with count ≥ s are the frequent items.
4. **Pass 2:** count a pair {i,j} only if (a) i and j are both frequent and (b) it hashes to a bucket with bit 1. Store counts as (i, j, count) triples.
5. **No loss:** a bucket count is the sum of the counts of the pairs in it, so it is at least each pair's count. A frequent pair therefore sits in a frequent bucket. Buckets with count < s hold no frequent pair. (A frequent bucket may still contain only infrequent pairs; that only causes extra counting.)

Q10. How is the bitmap used in PCY, and why is the hash table transformed into a bitmap? (the slide's own questions)

Lecture 4 · Slides 42, 81–82

1. **Use:** in Pass 2, hash each pair with the same function; if its bucket's bit is 0 the pair is not counted, if 1 (and both items are frequent) it is a candidate.
2. **Why:** after Pass 1 only the yes/no fact "count ≥ s" matters. A 4-byte counter (32 bits) is replaced by 1 bit, so the bitmap takes 1/32 of the memory.
3. The memory saved goes to the candidate-pair counters of Pass 2, which are the bottleneck. Example: 10^8 buckets are 400 MB as counters and 12.5 MB as bits.

Q11. Explain the 2/3 rule for PCY.

Lecture 4 · Slide 45

1. After PCY the surviving pairs are scattered, so the triangular matrix no longer works; they must be stored as triples of about 12 bytes, versus 4 bytes per pair in A-Priori's matrix.
2. If A-Priori has C candidate pairs it needs 4C bytes; if PCY keeps a fraction f it needs 12·f·C bytes.
3. PCY wins when $12f \cdot C < 4C \Leftrightarrow f < 1/3$, so the hash table must eliminate more than about 2/3 of the candidate pairs.
4. The number of buckets is O(main memory), so use all spare memory for buckets: a smaller average bucket count gives better pruning.

Q12. Describe the key idea of the multistage algorithm. (the slide's own question)

Lecture 4 · Slides 46–47, 83–84

1. Multistage adds a second hash filter to PCY, using a **different hash function** on the pairs, to reduce the number of candidate pairs.
2. **Pass 1** = PCY Pass 1 (item counts, hash table 1); between passes: frequent items list and Bitmap 1.
3. **Pass 2:** rehash pairs (those passing items-frequent and Bitmap 1) with a second hash function into table 2; then Bitmap 2.
4. **Pass 3:** count {i,j} only if both items are frequent *and* it hits a frequent bucket in Bitmap 1 *and* in Bitmap 2.
5. Fewer candidates mean less counter memory; the cost is an extra pass (three passes in total).

Q13. Describe Multihash. Compare it with Multistage and give the risks and limits of each.

Lecture 4 · Slides 49–51

1. **Multihash:** split the Pass 1 memory into two (or more) hash tables with independent hash functions and hash every pair into all of them in the same pass. A bitmap per table. Pass 2 counts a pair only if both items are frequent and it hits a frequent bucket in every bitmap. Two passes in total.
2. **Risk:** halving the buckets per table doubles the average bucket count, so more buckets may reach s and the filter weakens; most buckets must still stay below s.
3. **Multistage vs Multihash:** multistage = three passes, filters applied in sequence, each table gets the full memory; multihash = two passes, filters in parallel, each table is smaller.
4. **Limits:** multistage has diminishing returns (bitmaps eventually use all memory); multihash with too many tables makes (almost) all buckets frequent. In practice 2 or 3 hash functions.

Q14. Explain random sampling for frequent itemsets. What are false positives and false negatives, and which can be removed?

Lecture 4 · Slides 53–55

1. Take a random sample of the baskets that fits in memory, run A-Priori or PCY on it in memory, with the threshold reduced proportionally (sample fraction p, threshold p × s). Fewer disk passes: no extra disk I/O when moving to bigger itemsets.

2. **False positive:** frequent in the sample but not in the whole data. **False negative:** frequent in the whole data but not in the sample.
3. An optional second pass over the full data counts the reported itemsets and discards those below s: this removes false positives.
4. False negatives cannot be recovered, since only itemsets that appeared in the sample are rechecked. So sampling is approximate.

Q15. Describe the SON algorithm and explain why it is exact (no false negatives, no false positives).

Lecture 4 · Slides 56–57

1. **Pass 1:** split the whole file into chunks that fit in memory (this is not sampling: every basket is used). On each chunk run an in-memory algorithm with a threshold scaled to the chunk ($p_{chunk} \times s$). An itemset frequent in any chunk is a **candidate**; the candidate list is the union.
2. **Pass 2:** count all candidates over the entire file; keep those with count ≥ s.
3. **No false negatives:** an itemset cannot be frequent in the whole file unless it is frequent in at least one chunk (if it were below its scaled threshold in every chunk, its total would be below s), so it is in the candidate list.
4. **No false positives:** Pass 2 counts exactly over everything.
5. Two passes for itemsets of all sizes; result identical to A-Priori on the whole file.

Q16. Describe SON as two MapReduce rounds, and contrast SON with random sampling.

Lecture 4 · Slides 58–60, 65–66

1. **Map Phase 1:** each mapper runs an in-memory frequent-itemset algorithm on its chunks and emits the locally frequent itemsets.
2. **Reduce Phase 1:** union of all of them with duplicates removed: the global candidate list.
3. **Map Phase 2:** the candidate list is broadcast to all mappers; each counts every candidate on its own chunks.
4. **Reduce Phase 2:** add the counts per candidate; keep those with total ≥ s. The second round is required, not optional (Slide 60 says "optionally": erratum).
5. **Contrast:** sampling uses only part of the data, is approximate (false negatives possible) and its second pass is optional; SON uses all the data, is exact, and always makes exactly two passes. SON also parallelises because chunks are mined independently.

One-page cheat sheet

Lecture 4 · Slides 1–86

Definitions and formulas

Term	Definition / formula	Slide
Support of I	number of baskets containing all items of I; **frequent** iff support ≥ s	8
Monotonicity (A-Priori)	I frequent ⇒ every subset of I frequent; item infrequent ⇒ no pair containing it is frequent	30
Confidence	conf(I → j) = support(I ∪ {j}) ÷ support(I) (denominator = left side only)	10, 17
Interest	conf(I → j) − Pr[j], Pr[j] = (baskets with j) ÷ (all baskets); interesting if high (usually above 0.5)	11
Rule mining	find rules with support ≥ s and confidence ≥ c: Step 1 frequent itemsets (hard), Step 2 test A → I \ A for every non-empty proper subset A; a k-set gives $2^k - 2$ candidate rules	16–17
Cost model	number of passes over the disk file; scarce resource = main memory; swapping is a disaster	21–22
Pairs in a basket of m items	m(m−1)/2; n items give T = n(n−1)/2 possible pairs (10^5 items: about 5 × 10^9)	20, 24
Triangular matrix	4 bytes × T, ids must be 1..n; 10^5 items: about 20 GB	25–26
Triples [i, j, c]	12 bytes × O (occurring pairs); better iff O < T/3	25–27
C_k, L_k	candidate k-sets; truly frequent k-sets; C_k → count → L_k → construct C_{k+1}	34–35
Bit-vector	1 bit per bucket (1 iff count ≥ s) = 1/32 of 4-byte counters	42
PCY 2/3 rule	triples cost 12 bytes vs 4: must eliminate more than 2/3 of A-Priori's candidate pairs	45
Sample threshold	sample is a fraction p of baskets ⇒ threshold p × s; SON chunk threshold = p_{chunk} × s	54, 57
False positive / negative	frequent in sample (or candidate) but not overall / frequent overall but missed	55

Algorithms in a few steps

Algorithm	Steps
A-Priori	(1) Pass 1: count items. (2) Keep items with count ≥ s (frequent items), renumber them 1..m. (3) Pass 2: count only pairs of frequent items. (4) Keep pairs ≥ s. (5) For size k: build C_k from L_{k-1} (all subsets frequent), count in pass k, keep L_k. One pass per k.
PCY	(1) Pass 1: count items and hash every pair to a bucket counter (counts only). (2) Between: buckets → bit-vector (1 iff ≥ s); list frequent items. (3) Pass 2: count {i,j} iff both items frequent and bucket bit = 1; store triples. (4) Keep pairs ≥ s.
Multistage	(1) Pass 1 as PCY → Bitmap 1. (2) Pass 2: rehash surviving pairs with a second hash function → Bitmap 2. (3) Pass 3: count {i,j} iff both frequent and bit 1 in B1 and B2.
Multihash	(1) Pass 1: item counts plus two or more hash tables (each about half size) with independent hash functions. (2) Between: one bitmap per table. (3) Pass 2: count {i,j} iff both frequent and bit 1 in every bitmap.
Random sampling	(1) Pick a random sample that fits in RAM. (2) Run A-Priori/PCY in memory with threshold p × s. (3) Output the frequent itemsets of the sample. (4) Optional: second pass counts them in the full data to drop false positives.
SON	(1) Split the whole file into chunks. (2) Pass 1: mine each chunk with a scaled threshold; candidates = union. (3) Pass 2: count all candidates over the entire file. (4) Keep count ≥ s. Exact.
SON MapReduce	Map 1: locally frequent itemsets. Reduce 1: union, no duplicates = candidates. Map 2: counts of broadcast candidates on own chunks. Reduce 2: sum, keep ≥ s.

Comparison table

	A-Priori	PCY	Multistage	Multihash	Sampling	SON
Passes (pairs)	2	2	3	2	1 (+1 optional)	2
All sizes	k passes	— (slides cover pairs only)	— (slides cover pairs only)	— (slides cover pairs only)	at most 2	2
Exact?	yes	yes	yes	yes	no (false negatives)	yes
Candidate test for a pair	both items frequent	+ bucket bit 1	+ bit 1 in B1 and B2	+ bit 1 in all tables	frequent in sample	frequent in some chunk
Weak point	$m^2/2$ counters	must cut more than 2/3	extra pass, bitmaps use memory	smaller tables, too many make all buckets frequent	unlucky sample	large candidate list

Common mistakes & traps

Lecture 4 · Slides 1–86

- **Wrong denominator for confidence.** Divide by support of the left side only. Not by the number of baskets, not by the right side's support. Slide 75: 3 of the 5 baskets with cat and dog, so 3/5.
- **Confusing confidence with interest.** A rule with confidence 1/2 can have negative interest (Slide 15: 1/2 − 5/8 = −1/8). Always give Pr[j] and the signed interest.
- **Strict versus non-strict.** Frequent and "passes the confidence threshold" use ≥. Support exactly s is frequent; confidence exactly c is kept (b,m → c with 3/4 = 0.75 on Slide 18).
- **Missing rule directions.** A pair gives two rules, a triple gives six; also a one-item left side with a two-item right side. Slide 18's "..." hides three accepted rules (c → b, c,m → b, j → c).
- **Counting purchases, not baskets.** Support counts baskets. A word repeated inside one basket counts once; capital letters are ignored on Slides 70–77.
- **Assuming frequent items make a frequent pair.** m and c are frequent but {m,c} is not. Monotonicity works downward only: frequent set ⇒ frequent subsets, never the reverse.
- **PCY: using only the bucket test.** A pair must have both items frequent and a frequent bucket. A frequent bucket does not imply a frequent pair (collisions); an infrequent bucket proves none of its pairs is frequent. Bit = 1 means count ≥ s, not "greater than".
- **PCY: forgetting triples.** After PCY the triangular matrix no longer works, so counters are 12-byte triples; hence the 2/3 rule (not 1/3: it is the fraction to eliminate).
- **Mixing up Multistage and Multihash.** Multistage = 3 passes, one table per pass; Multihash = 2 passes, several smaller tables in the same pass. Multihash's risk is that halving the buckets doubles the average count.
- **Treating SON like sampling.** SON reads every basket and is exact; Slide 53 lumps them together ("may miss some") and Slide 60 says "optionally": both misleading. SON's second pass is mandatory.
- **Not scaling the threshold.** Using the full s inside a chunk or sample misses itemsets spread across chunks. Use p × s (percentages must be turned into counts). Two different "monotonicity" ideas exist: A-Priori's (subsets of a frequent set are frequent) and SON's (frequent overall ⇒ frequent in some chunk).
- **Trusting slide answers blindly.** Known errata: Slide 15's interest value is never computed; Slide 71 and Slide 73 miss basket 7 and the second triple {dog, cat, and}; the SON example (Slides 63–69) misses the ninth candidate {banana, orange, pear}. Show your own counts.

LECTURE 5

Clustering (Hierarchical, k-means, CURE, BFR, k-means on Map-Reduce)

ROADMAP — WHAT THIS LECTURE IS ABOUT

- **The problem (Slides 1–14):** given points and a distance measure, group them into **clusters** so that members of a cluster are similar and members of different clusters are not. The measure (Euclidean, cosine, Jaccard, edit) defines "similar". Clustering is hard because real clusters have odd shapes and sizes, and because in 10 or 10,000 dimensions many pairs of points are about equally far apart. Two families exist: **hierarchical** and **point assignment**.
- **Hierarchical clustering (Slides 16–21, quiz Slides 69–75):** repeatedly merge the two nearest clusters; a cluster is represented by its **centroid** (Euclidean) or its **clustroid** (non-Euclidean). It costs $O(N^3)$, too much for big data.
- **k-means (Slides 22–34, quiz Slides 76–77):** choose k, pick k seeds, then assign, update, reassign until convergence; choose k at the elbow of the average-distance curve.
- **Scalable versions (Slides 35–67): CURE** (sample, hierarchical clustering, representatives moved 20% toward the centroid, one rescan; any shape), **BFR** (read data in memory loads and keep only N, SUM, SUMSQ per cluster in the discard set, plus compression and retained sets; normal, axis-aligned clusters; theory only) and **k-means on MapReduce** (mapper assigns, combiner pre-sums, reducer averages).
- **Quizzes (Slides 69–84)** are solved in this guide; Slides 86–88 are the summary and the reading list (book Chapter 7).

Exam checklist — by the end you can:

- define clustering and name the four distance measures with the data each suits; explain why clustering is hard and what the curse of dimensionality means;
- contrast **hierarchical** (agglomerative, divisive) with **point assignment**, and explain a **dendrogram**;
- run agglomerative clustering by hand with centroids (weighted-average update, distances, stopping at k) and draw the dendrogram; explain the **clustroid** and why it is needed; explain why the cost is $O(N^3)$;
- run **k-means** by hand (farthest-first seeds, two rounds with every distance and every new centroid), state convergence, and explain how to pick k (elbow, too few / too many);
- describe **CURE** in both passes, apply the 20% move to representatives, and say why representatives allow any shape;
- explain **BFR** conceptually: assumptions (normal, axis-aligned; which ellipses are allowed), DS / CS / RS, why N, SUM and SUMSQ suffice and are updated by addition, the five steps per memory load, the Mahalanobis test, and how two CS sets are merged (no BFR numericals are examined);
- describe **k-means on MapReduce** (mapper, combiner, reducer, iteration) and work the small numeric examples;
- compare k-means, CURE, BFR and MapReduce k-means in one table.

Slide map

Lecture 5 · Slides 1–88

For Lecture 5 the PDF has 75 pages but the printed slide numbers run from 1 to 88, because some printed numbers are absent: 3, 7, 10, 11, 13, 15, 19, 20, 28–30, 37 and 85. The table gives the PDF page for each group of printed slides. Always look slides up by their **printed** number (bottom-right corner).

Slide(s)	PDF page(s)	Topic	In this guide
1–2	1–2	Title: Scalable Clustering; course map (clustering sits under high-dimensional data)	(p. 272)
4	3	The problem of clustering; distance measures (Euclidean, cosine, Jaccard, edit)	(p. 272)
5–6	4–5	Examples: dog breeds as clusters; clusters and outliers	(p. 273)
8–9	6–7	Why clustering is hard: messy data; curse of dimensionality	(p. 275) (p. 275)
12	8	Application: documents as 0/1 word vectors, topics as clusters	(p. 275)
14	9	Two families: hierarchical (agglomerative, divisive) and point assignment	(p. 276)
16–18	10–12	Hierarchical clustering: centroid, worked picture and dendrogram, clustroid (non-Euclidean); Quiz L5-Q1	(p. 277)
21	13	Cost of hierarchical clustering: $O(N^3)$, too slow for big data	(p. 288)
22	14	Divider: k-means clustering	(p. 297)
23	15	k-means setup: Euclidean, k, seeds (random, farthest-first)	(p. 297)
24	16	k-means loop: assign, update, reassign, convergence (Quiz L5-Q3 on Slides 76–77 follows below)	(p. 298)
25–27	17–19	Three-snapshot example: clusters after round 1, round 2 and at the end	(p. 300)
31	20	Getting k right: average distance to centroid versus k (elbow)	(p. 308)
32–34	21–23	Picking k: too few, just right, too many	(p. 309)
35–36	24–25	CURE: divider; assumptions, representatives, any shape	(p. 310)
38	26	Starting CURE (Pass 1): sample, hierarchical clustering, dispersed points moved 20% toward the centroid	(p. 310)
39–41	27–29	CURE example: initial clusters, pick dispersed points, move them 20%	(p. 311)
42	30	Finishing CURE (Pass 2): assign each point to the cluster of the closest representative; Quiz L5-Q6	(p. 312)
43–45	31–33	BFR: divider; what it is, assumptions (normal, axis-aligned), memory loads, first k centroids	(p. 321)
46–47, 52	34–35, 40	Three classes of points: discard set DS, compression set CS, retained set RS (Slide 52 repeats Slide 47)	(p. 323)
48–49	36–37	Summarising points: N, SUM, SUMSQ; centroid and variance formulas	(p. 325)
50–51	38–39	The memory-load loop: five steps	(p. 326)
53–56	41–44	A few details: Q1 how close is close enough; Mahalanobis distance and the threshold	(p. 328)
57–58	45–46	Q2: should two CS clusters be combined (variance, Mahalanobis between centroids)	(p. 330)
59	47	Divider: k-means MapReduce	(p. 346)
60–63	48–51	k-means MapReduce: initialisation, mapper, reducer, combiner, iteration	(p. 346)
64–67	52–55	Worked example: 8 numbers, k = 2, 3 nodes; mapper and reducer phases; Quiz L5-Q7	(p. 347)
68	56	Summary of the lecture	(p. 366)

69–75	57–63	Quiz: hierarchical clustering of 12 points to k = 6 (question, answer, dendrogram); Quiz L5-Q2	(p. 288)
76–77	64–65	Quiz: k-means initialisation and two rounds, question and answer; Quiz L5-Q3	(p. 300)
78–79	66–67	Quiz: BFR steps and allowed cluster shapes; Quiz L5-Q4	(p. 332)
80–81	68–69	Quiz: SUM, SUMSQ, variance (BFR numerical, not examined); Quiz L5-Q5	(p. 332)
82–83	70–71	Quiz: initialisation in CURE, 20% move; Quiz L5-Q6	(p. 312)
84	72	Quiz: k-means MapReduce homework (17 numbers, k = 3, 3 nodes); Quiz L5-Q8	(p. 354)
86	73	Summary: clustering and the two algorithm families	(p. 366)
87	74	Summary: BFR and CURE	(p. 367)
88	75	Actions: review the slides, read book Chapter 7	(p. 367)

Printed slides 3, 7, 10, 11, 13, 15, 19, 20, 28, 29, 30, 37 and 85 are not in the PDF; nothing is missing from this guide.

Where clustering fits, and what the lecture covers

Lecture 5 · Slides 1–2

Lecture 5 is called **Scalable Clustering: BFR, CURE, MapReduce** (Slide 1). The title promises three things: two algorithms for clustering data that is too big for memory (BFR and CURE), and a way to run clustering on a cluster of machines (MapReduce). This first part of the chapter builds the foundations: what clustering is, why it is hard, and the oldest family of methods, **hierarchical clustering**. The slides never expand the acronyms, so here they are once: **BFR** = Bradley–Fayyad–Reina (the three authors), **CURE** = Clustering Using REpresentatives. Both come later in the lecture.

Slide 2 is the course map. Its first column, "High dim. data", lists three topics: locality-sensitive hashing (LSH), **clustering** (the green box, this lecture) and dimensionality reduction. The other columns (graph data, infinite data, machine learning, apps) belong to other parts of the course. The message of the slide is simply that clustering is a tool for *high-dimensional* data, and that is exactly the difficulty that shapes the rest of the lecture.

SLIDE NUMBERING

The printed slide numbers in this deck skip some values (3, 7, 10, 11, 13, 15, 19, 20 do not exist in the PDF; the deck jumps 2→4, 9→12, 12→14, 14→16, 18→21). That is how the instructor's deck is; nothing is missing from this guide. Always use the printed number in the bottom-right corner of a slide.

The problem of clustering

Lecture 5 · Slide 4

BEGINNER INTUITION

You are given a huge pile of unlabelled photos and asked to sort them into piles so that photos in one pile look alike and photos in different piles look different. Nobody tells you the pile names or even exactly how many piles there should be. That is clustering. The only thing you need is a way to say how "alike" two photos are, which is a *distance*.

DEFINITION — CLUSTERING (SLIDE 4)

Given a **set of points** and a notion of **distance** between points, **group the points** into some number of **clusters** so that (i) members of a cluster are **similar** to each other (small distance) and (ii) members of different clusters are **dissimilar** (large distance).

The slide adds what is **usually** true in practice: the points live in a **high-dimensional space** (many coordinates per point), and similarity is defined by a **distance measure**. The slide lists four: Euclidean, cosine, Jaccard and edit distance. It does not define them (they come from earlier lectures), so here is a one-line refresher of each with a tiny example.

Distance	Works on	Definition	Tiny example
Euclidean	numeric vectors	straight-line distance: $\sqrt{(\sum(x_i - y_i)^2)}$	(0,0) and (3,4): $\sqrt{(9+16)} = 5$
Cosine	vectors (direction matters, length does not)	$1 - \cos\theta$, where $\cos\theta = (x \cdot y)/(\|x\|\|y\|)$; equivalently the angle θ itself	x=(1,0,1,1), y=(1,1,0,1): $x \cdot y=2$, $\|x\|=\|y\|=\sqrt{3}$, $\cos\theta=2/3$, distance = 1 − 2/3 = 0.333 ($\theta \approx 48.2°$)
Jaccard	sets	$1 - \|A \cap B\| / \|A \cup B\|$	A={a,b,c}, B={b,c,d}: $\|A \cap B\|=2$, $\|A \cup B\|=4$, distance = 1 − 2/4 = 0.5
Edit distance	strings	minimum number of single-character insertions and deletions that turn one string into the other (some books also allow substitutions)	"cat" → "cart": insert r, distance 1

EXAM TIP

The choice of distance *is* the definition of "similar". Euclidean for numeric measurements (height, weight), Jaccard or cosine for documents seen as sets of words, edit distance for strings such as names or DNA. Euclidean is special for this lecture because only in Euclidean space can you **average** points (Slides 16 and 18).

Notice two things the slide leaves open: it does not say how many clusters to look for ("some number"), and it gives no threshold for "similar". Different algorithms answer these differently, and choosing the number of clusters (called *k*) becomes an issue in the k-means part of the lecture.

Pictures of clusters, outliers and difficulty

Lecture 5 · Slides 5, 6 and 8

Example: clusters of dogs

Lecture 5 · Slide 5

Slide 5 is the "easy" picture of what clustering should find. Each dog is a point in a 2-D space with **weight** on the horizontal axis and **height** on the vertical axis. The dogs fall into three visibly separate groups, labelled **Beagles**, **Chihuahuas** and **Dachshunds**. The algorithm is never told the breed names; it only sees the coordinates and discovers that the three groups exist. Chihuahuas are low and light, dachshunds are as low but heavier, beagles are tallest and heaviest. Beyond the slides: this is the classic dog example of the textbook, *Mining of Massive Datasets* (MMDS), chapter 7.

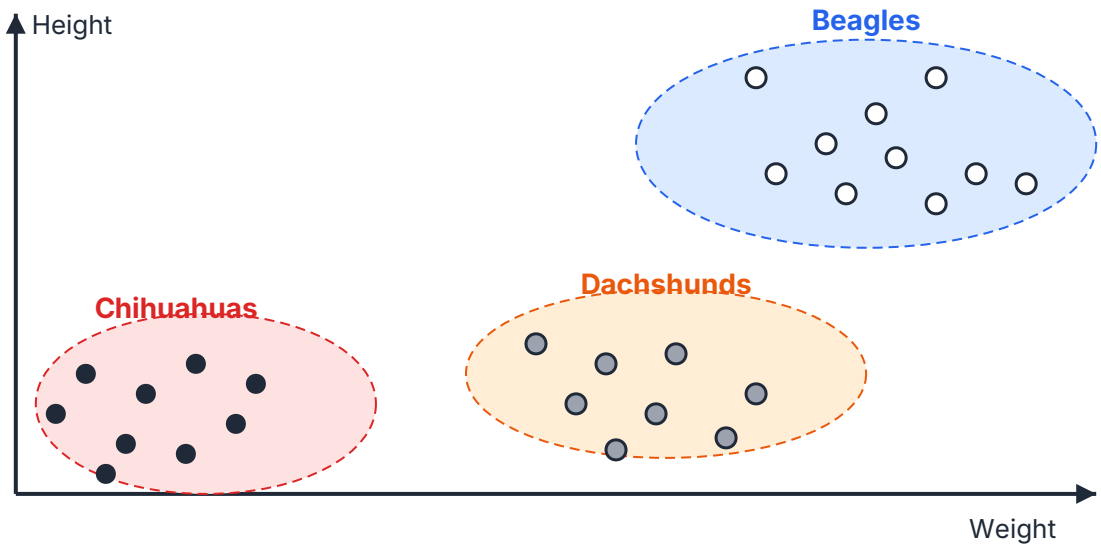


Figure 1 — Redrawn from Slide 5 (positions schematic). Each dog is a point (weight, height); three well-separated groups are the three clusters. Dashed outlines are the clusters a good algorithm should find.

Example: clusters and outliers

Lecture 5 · Slide 6

Real data is rarely this tidy. Slide 6 shows three clusters (dense groups of × marks, drawn as coloured ellipses; 15, 14 and 10 points) plus a few isolated points that belong to no ellipse. The slide labels one of them **Outlier** and one ellipse **Cluster**. (There are in fact two isolated points; only the lower one is labelled, the upper one is an outlier too.)

DEFINITION — OUTLIER

An **outlier** (noise point) is a point that is far from every cluster and does not belong to any of them. A **cluster** is a dense group of mutually close points.

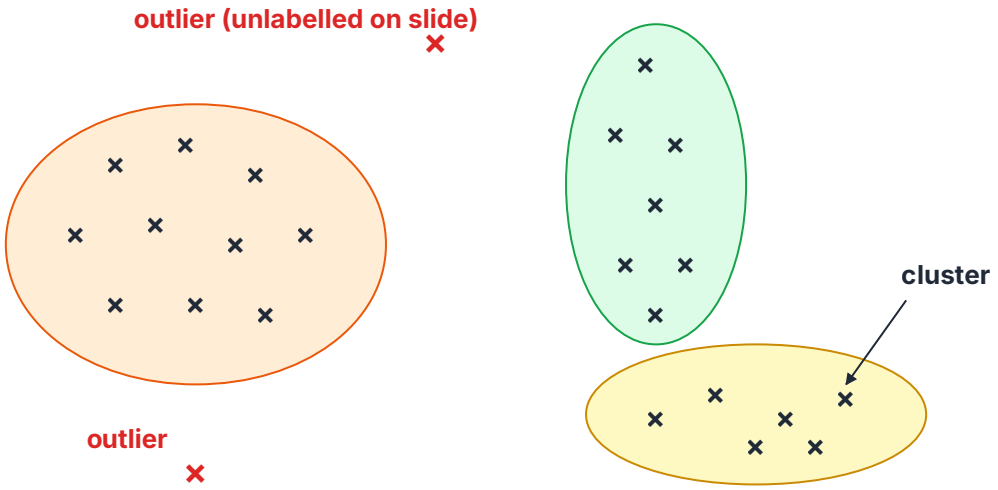


Figure 2 — Slide 6 redrawn schematically: three clusters and two outliers (red ×) lying outside every ellipse.

WHY OUTLIERS MATTER

A good algorithm must not let an outlier drag a cluster's centre towards it, and it should either give outliers a cluster of their own or leave them out. This returns in BFR, where a special "retained set" collects exactly such stragglers.

Clustering is a hard problem

Lecture 5 · Slide 8

Slide 8 contains only a picture: thousands of coloured dots forming about eleven groups. The groups have very different **sizes, densities and shapes**, they **overlap** and touch, stray dots of one colour sit inside the region of another, and there is no legend. Even a person could not draw the "right" boundaries. The point of the slide is that real clusters have fuzzy edges; it prepares the next slide.

Why clustering is hard: high dimensions

Lecture 5 · Slide 9

Slide 9 makes one argument in two steps. First: clustering in **two dimensions** looks easy, clustering **small amounts of data** looks easy, and in most cases looks are *not* deceiving, so your eyes find the right clusters. Second: **many applications involve not 2 but 10 or 10,000 dimensions**. You cannot look at 10,000-dimensional data, and its geometry behaves unlike the plane. The slide's key sentence:

> **KEY IDEA — HIGH-DIMENSIONAL SPACES LOOK DIFFERENT**
> **Many pairs of points are at about the same distance.** When nearly all pairs are about equally far apart, "near" and "far" lose meaning, so distance-based clustering has little to grab on to. Beyond the slides: the textbook calls this the **curse of dimensionality**. It is why the course also has a topic called dimensionality reduction (Slide 2).

> **BEGINNER INTUITION**
> On a street (1-D) your neighbour is much closer than a person across town. With many independent coordinates, two random points differ a little in each coordinate, and those many small differences add up to nearly the same total for every pair. Nobody is clearly "close".

Beyond the slides: a quick numerical check. The table shows 100 random points with every coordinate drawn uniformly from [0,1] (a computer experiment with a fixed random seed; the exact numbers vary from run to run, the trend does not). For each dimension d it gives the smallest and the largest of the 4,950 pairwise Euclidean distances.

Dimensions d	smallest distance	largest distance	largest ÷ smallest
2	0.016	1.128	70
10	0.404	2.016	5.0
100	3.12	5.06	1.6
10,000	40.0	41.6	1.04

In 2-D the farthest pair is 70 times farther than the nearest. In 10,000-D every pair is within 4% of every other: all points look equally far apart.

> **EXAM TIP**
> If asked "why is clustering hard?", give both reasons: (1) real clusters differ in size, density and shape, overlap, and come with outliers (Slides 6 and 8); (2) in high dimensions many pairs of points are at about the same distance (Slide 9). Say that both make "nearest cluster" unreliable.

Application: clustering documents to find topics

Lecture 5 · Slide 12

To cluster documents you must first turn each one into a *point*. Slide 12 does it with the simplest scheme. Let the vocabulary have *k* distinct words. Represent a document by the vector $(x_1, x_2, \ldots, x_k)$ where $\mathbf{x_i = 1}$ **if and only if the i-th word appears in the document**, and 0 otherwise. Then: **documents with similar sets of words may be about the same topic**. A cluster of documents is a topic.

Worked example — three tiny documents as 0/1 vectors Lecture 5 · Slide 12

Vocabulary (k = 6): `goal, match, team, vote, party, law`. Doc 1 "the team won the match" contains match, team (and the slide's scheme ignores everything not in the vocabulary). Doc 2 "goal in the match for the team" contains goal, match, team. Doc 3 "the party passed the law in a vote" contains vote, party, law.

	goal	match	team	vote	party	law
Doc 1	0	1	1	0	0	0
Doc 2	1	1	1	0	0	0
Doc 3	0	0	0	1	1	1

Jaccard distance (words as sets): d(Doc1, Doc2) = 1 − 2/3 = 0.333; d(Doc1, Doc3) = 1 − 0/5 = 1. Docs 1 and 2 cluster together (sport), Doc 3 is far away (politics).

Two remarks. A real vocabulary has tens of thousands of words, so this is exactly the high-dimensional situation of Slide 9. And the slide does not say which distance to use; Jaccard (sets) or cosine (vectors) from the Slide 4 list are the natural choices.

Overview: two families of clustering methods

Lecture 5 · Slide 14

Slide 14 splits all methods into two families.

Family	Idea	Output
Hierarchical (a) **Agglomerative** (bottom-up) (b) **Divisive** (top-down)	(a) Initially every point is its own cluster; repeatedly **combine the two "nearest" clusters** into one. (b) Start with **one** cluster holding everything and recursively **split** it.	A tree of nested clusters, the **dendrogram**. Choose the number of clusters afterwards by cutting the tree.
Point assignment	Maintain a set of clusters; each point belongs to its **nearest** cluster.	A flat partition into a fixed number of clusters.

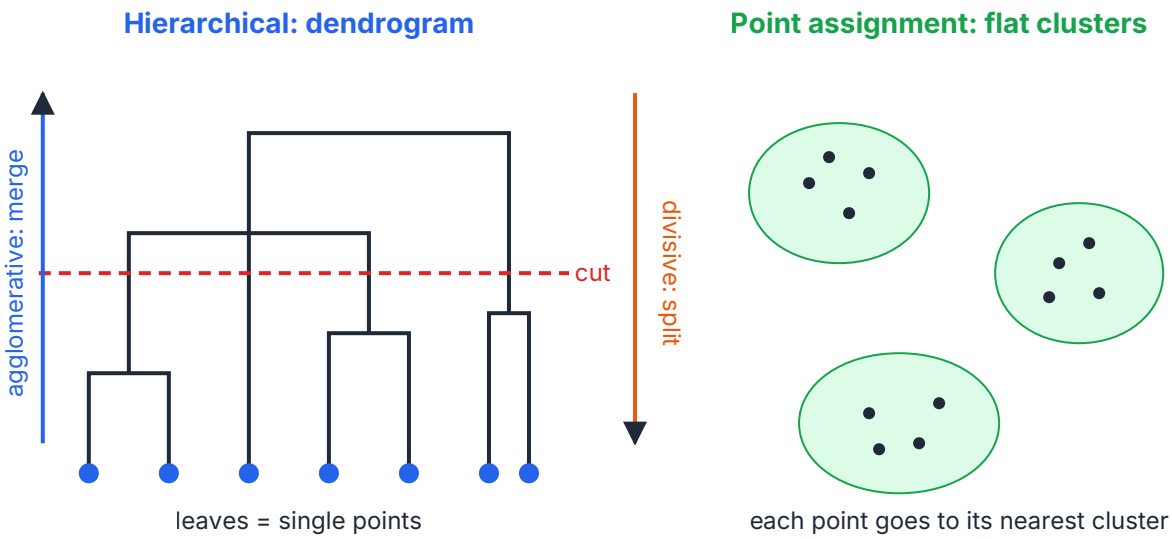


Figure 3 — Left: a dendrogram (Slide 14, top right); agglomerative reads it bottom-up, divisive top-down, and a horizontal cut (dashed) gives a flat clustering (here 4 clusters). Right: point assignment (Slide 14, bottom right).

Reading a dendrogram

Lecture 5 · Slide 14

The picture on Slide 14 (top right) is a dendrogram of 30 items, coloured into four groups. The leaves along the bottom are the individual points. Each horizontal bar joins two clusters, and the **height of the bar is the distance at which they were merged**: low bars are very similar items, the top bar joins the most different groups. Slide 14 does not label the vertical axis; read it (0.2 to 0.9) as merge distance. **Cutting** the tree with a horizontal line at some height leaves as many clusters as bars the line crosses; the four colours on the slide are one such cut. Because one tree contains the answer for every possible number of clusters, you can decide "how many?" *after* clustering.

WHICH FAMILY ARE BFR AND CURE?

Slide 14 does not say, but the point-assignment family is where k-means, BFR and CURE belong (they keep clusters and put each point into the nearest). Hierarchical clustering is the subject of the next slides. Do not mix the two: **hierarchical = build a tree of merges; point assignment = fixed set of clusters that points are assigned to**.

Hierarchical (agglomerative) clustering

Lecture 5 · Slides 16–17

The key operation and the two decisions

Lecture 5 · Slide 16

KEY OPERATION

Repeatedly **combine the two nearest clusters**. **Key problem:** after you merge clusters, how do you represent the "location" of a cluster of many points, so that you can tell which pair of clusters is now closest?

Slide 16 answers with two decisions for the **Euclidean case** (where averages make sense):

1. **How to represent a cluster of many points?** By its **centroid**, the **average of its (data)points**, coordinate by coordinate.
2. **How to determine "nearness" of clusters?** Measure the distance between clusters by the **distance between their centroids**.

DEFINITION — CENTROID

The centroid of a cluster of n points is the point whose every coordinate is the average of that coordinate over the n members: centroid = (1/n) ∑ points. It need not be one of the data points.

The algorithm, step by step:

1. Start with every point as its own cluster (its centroid is the point itself).
2. Compute the distance between the centroids of every pair of clusters.
3. Merge the pair with the smallest distance; compute the centroid of the new cluster.
4. Repeat steps 2–3 until the stopping rule is met (see below).

FORMULA — CENTROID AFTER A MERGE

Clusters P (n_P points, centroid c_P) and Q (n_Q points, centroid c_Q) merge into a cluster with centroid $\frac{n_P \cdot c_P + n_Q \cdot c_Q}{n_P + n_Q}$, a **weighted average**. It equals the plain average of all the original member points.

Worked example — why the update is weighted Lecture 5 · Slide 16

Cluster P = {(0,0), (2,0)} has centroid (1,0) and $n_P = 2$. Cluster Q = {(4,3)} has $n_Q = 1$. Merge them.

1. Correct (average of all three points): ((0+2+4)/3, (0+0+3)/3) = (2, 1).
2. Same thing from the centroids: (2·(1,0) + 1·(4,3)) / 3 = (6,3)/3 = (2, 1). ✓
3. Wrong (midpoint of the two centroids): ((1+4)/2, (0+3)/2) = (2.5, 1.5). This treats the pair as if it were one point and is the classic mistake.

COMMON MISTAKE

Averaging the two *centroids* instead of averaging all the *points* (or using the weights n_P, n_Q). A big cluster must pull the new centroid harder than a single point.

When to stop, and other cluster distances

Lecture 5 · Slide 16

Slide 16 gives no stopping rule. Beyond the slides: common choices are (a) stop when k clusters remain (used in Quiz L5-Q2, "stopping point k = 6"; going from N clusters to k takes N − k merges, because every merge removes exactly one cluster); (b) stop when the closest pair is farther apart than a threshold; (c) merge until one cluster is left and keep the whole tree. Beyond the slides: other ways to measure the distance between clusters exist (smallest, largest or average distance between members, called single, complete and average linkage); they can give a different tree. In this course use only the centroid rule of Slide 16.

The slide's worked picture

Lecture 5 · Slide 17

Slide 17 shows a complete run on six points, drawn as `o` (data point) with centroids drawn as `x`. The points on the slide are (0,0), (1,2), (2,1), (4,1), (5,0), (5,3); the slide prints no letters, so Figure 4 and the quiz below use A–F in that order. Read the slide in this order:

1. The closest pairs are (1,2)–(2,1) and (4,1)–(5,0), both at √2 ≈ 1.414 (a tie). Their centroids are the blue **x** (1.5, 1.5) and the green **x** (4.5, 0.5), and the small blue and green circles enclose them.
2. Next (0,0) joins the blue pair; the new (dark red) centroid is the average of all three points, (1, 1). Likewise (5,3) joins the green pair, new centroid (14/3, 4/3) ≈ (4.7, 1.3). The slide prints it rounded.
3. Finally the two big clusters merge. The **dendrogram** at the bottom right records the order: blue pair first, green pair, then the single points, then the root.

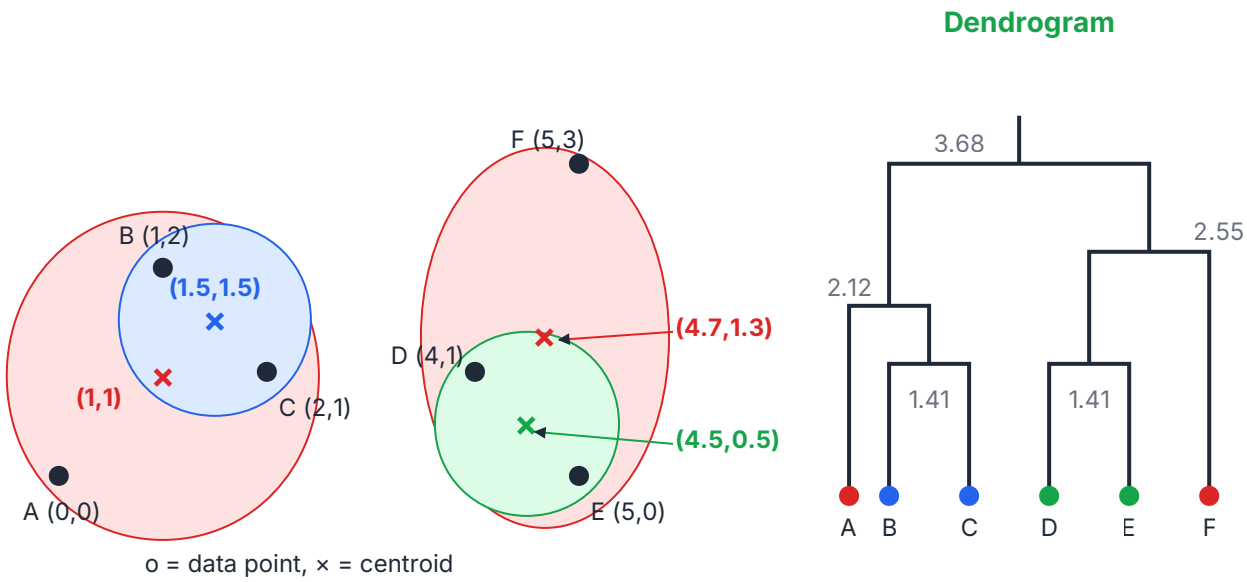


Figure 4 — Slide 17 redrawn. Left: points (o), centroids (×) and the circles of the four merges. Right: the dendrogram; the grey numbers are the merge distances computed in Quiz L5-Q1 (the slide itself shows no scale; its two √2 merges are drawn at slightly different heights only for looks).

EXAM TIP

To find the closest pair by hand, compare *squared* distances and skip square roots; take a root only for the pair you merge. After each merge, only the distances involving the new cluster change, so recompute only those. When two pairs tie, break the tie arbitrarily and say so.

The non-Euclidean case: clustroid

Lecture 5 · Slide 18

Slide 18 asks: what if the space is non-Euclidean? Then **the only "locations" you can talk about are the points themselves**: there is **no "average" of two points**. What is the average of two strings, or of two sets? The slide's approach has the same two decisions as Slide 16.

1. **Representing a cluster:** by a **clustroid**, the (data)point **"closest"** to the other points of the cluster.
2. **Nearness of clusters:** treat the clustroid as if it were the centroid when computing inter-cluster distances.

DEFINITION — CLUSTROID

The clustroid of a cluster is one of its **own data points**, chosen as the one most central under the given distance. A centroid is generally *not* a data point; a clustroid always is.

The slide emphasises "closest" but never says what it means. **Beyond the slides:** the textbook (MMDS 7.2.3) lists the usual choices. The clustroid is the point that minimises (a) the **sum of its distances** to the other points, (b) its **maximum distance** to another point, or (c) the **sum of the squares** of its distances to the other points.

Worked example — the three rules can disagree Lecture 5 · Slide 18 (numbers are ours)

Four points P, Q, R, S with the distances shown in Figure 5 (they satisfy the triangle inequality). For each point, list its distances to the other three.

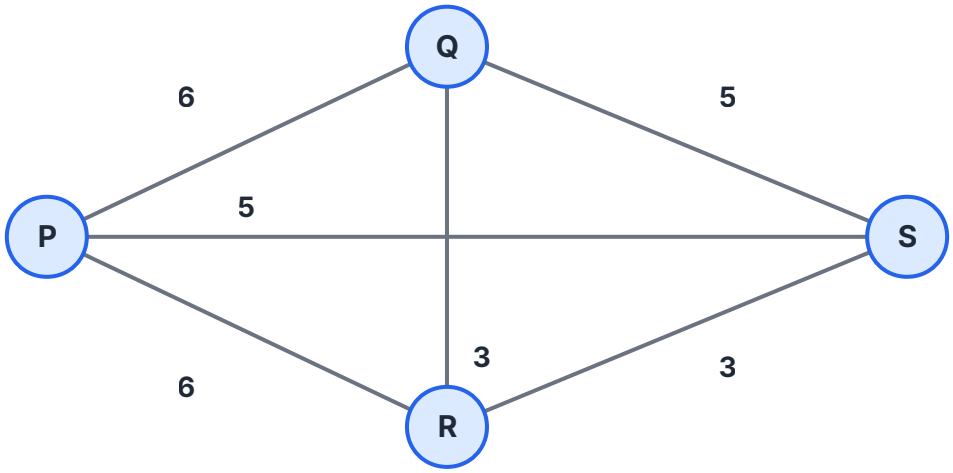


Figure 5 — Four points with the distances written on the edges: PQ = 6, PR = 6, PS = 5, QR = 3, QS = 5, RS = 3.

Point	distances to the others	sum	maximum	sum of squares
P	6, 6, 5	17	6	97
Q	6, 3, 5	14	6	70
R	6, 3, 3	**12**	6	**54**
S	5, 5, 3	13	**5**	59

Minimum sum: R (12). Minimum sum of squares: R (54). Minimum maximum: S (5). So "closest" depends on the rule; always state which one you use. The sum of squares punishes a single large distance more than the plain sum does.

Two properties to remember: with a clustroid every inter-cluster distance is just a distance between two real points (so it works with any distance function, including Jaccard and edit distance), and the clustroid must be recomputed after every merge (Beyond the slides: you compare the member points of the merged cluster, not only the two old clustroids). The two parts of Slide 18 are tested together with Slide 17 in the quiz below.

	Euclidean case (Slide 16)	Non-Euclidean case (Slide 18)
Cluster representative	**Centroid** = average of the points	**Clustroid** = data point "closest" to the others
Is it a data point?	Usually not	Always
Cluster distance	distance between centroids	distance between clustroids
Needs an "average"?	Yes	No (works with any distance)

Try it yourself first: work through the six-point run of Slide 17, cover the solution, and compare. It also asks you to read a dendrogram and to explain the clustroid.

Quiz L5-Q1 Hierarchical clustering example and dendrogram — Lecture 5 · Slides 17–18

QUESTION (AS ON THE SLIDES)

Slide 17 (*Example: Hierarchical clustering*) is a picture with no printed question. It shows six data points (drawn as `o`), four centroids (drawn as `x`), circles and ellipses round the clusters, and a **dendrogram**. It illustrates the two rules of Slide 16: a cluster is represented by its **centroid** (the average of its points) and the nearness of two clusters is the distance between their centroids. Slide 18 then asks what to do in the **non-Euclidean** case, where no average exists. The parts below turn the slides into an exam-style question.

Point (my label)	A	B	C	D	E	F
Coordinates (x, y)	(0, 0)	(1, 2)	(2, 1)	(4, 1)	(5, 0)	(5, 3)
Leaf colour in the slide's dendrogram	dark red	blue	blue	green	green	red

The letters A–F are my own labels (the slide prints only coordinates). Centroids printed on the slide: (1.5, 1.5) blue, (4.5, 0.5) green, (1, 1) red on the left, (4.7, 1.3) red on the right.

(a) Run agglomerative hierarchical clustering on the six points, using the Euclidean distance between cluster centroids, until one cluster is left. Show **every merge**: which two clusters are merged, at what distance, and the new centroid.

(b) Explain how the four centroids printed on Slide 17 arise.

(c) Draw the dendrogram.

(d) What does a dendrogram show, and how do you cut it to obtain k clusters (for example k = 2, 3, 4)?

(e) Non-Euclidean case (Slide 18): what is a **clustroid**, how does it differ from a centroid, and how is the nearness of two clusters then measured?

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

1. **Formula / idea — what agglomerative clustering does (Slides 14 and 16).**
   Slide 14 lists two families of clustering methods: *hierarchical* (agglomerative = bottom-up, divisive = top-down) and *point assignment*. This example is **agglomerative**: at the start every point is a cluster of its own; in every round the two **nearest** clusters are combined into one; repeat until one cluster is left (or until k clusters are left). By hand you only need the two rules of Slide 16: (1) a cluster is represented by its **centroid** = the average of its points; (2) the nearness of two clusters is the distance between their centroids.

   FORMULAS USED

   Centroid of a cluster of n points: $\frac{\sum x}{n}$ for the x-coordinate and $\frac{\sum y}{n}$ for the y-coordinate (the average of each coordinate).
   Distance between clusters P and Q with centroids (x_P, y_P) and (x_Q, y_Q): $d(P, Q) = \sqrt{(x_P - x_Q)^2 + (y_P - y_Q)^2}$.
   Centroid after a merge, P with n_P points and Q with n_Q points: $c_{new} = (n_P \cdot c_P + n_Q \cdot c_Q) / (n_P + n_Q)$. This is a **weighted** average; it equals the average of all the original member points.

   BEGINNER INTUITION

   Six houses are marked on a map and you want neighbourhoods. First glue together the two nearest houses. The glued pair is now one blob whose location is pinned at its middle (the centroid). Next round look again for the two nearest things on the map (single houses or blobs) and glue them. Keep going until everything is one big blob. The written record of who was glued to whom, and at what distance, is the **dendrogram**.

2. **Part (a), start — distances between all pairs of single points.** At the start each cluster is one point, so its centroid is the point itself. Use $d = \sqrt{((x_1 - x_2)^2 + (y_1 - y_2)^2)}$. Example: $d(B, C) = \sqrt{((1 - 2)^2 + (2 - 1)^2)} = \sqrt{2} = 1.414$. All 15 distances:

d	A (0,0)	B (1,2)	C (2,1)	D (4,1)	E (5,0)	F (5,3)
A (0,0)	0	2.24	2.24	4.12	5.00	5.83
B (1,2)		0	**1.41**	3.16	4.47	4.12
C (2,1)			0	2.00	3.16	3.61
D (4,1)				0	**1.41**	2.24
E (5,0)					0	3.00
F (5,3)						0

The smallest distance is 1.41 and it occurs **twice**: B–C and D–E (both √2). A tie is harmless here: the two pairs share no point, so merging one does not change the other, and both will be merged anyway (D–E is still the smallest distance after B–C is merged, see the round-2 row of the table below). Ties are broken arbitrarily (Slide 71 does the same in another example); I merge B–C first.

> **EXAM TIP**
> To find the closest pair quickly, compare **squared** distances and skip the square roots: B–C = 2, D–E = 2, C–D = 4, A–B = 5, A–C = 5, D–F = 5, E–F = 9, B–D = 10, C–E = 10, C–F = 13, A–D = 17, B–F = 17, B–E = 20, A–E = 25, A–F = 34. The square root keeps the order, so the smallest d^2 is the smallest d. Take the root only for the pair you actually merge.

3. **Part (a), merge 1 — B and C at distance √2 = 1.414.** New centroid = average of the two points = ((1 + 2)/2, (2 + 1)/2) = **(1.5, 1.5)**. This is the blue x and the small blue circle on Slide 17. Clusters now: A, {B,C}, D, E, F (5 clusters).
4. **Part (a), rounds 2 to 5 — recompute the distances between the current centroids.** After every merge, the distances involving the merged points are replaced by distances to the new centroid; the others are unchanged. The full list for each round (smallest first, the merged pair in bold):

Round	Clusters present (centroid)	Distance between every pair of centroids, smallest first (d = √ of the squared distance)	Merged
2	A (0, 0), BC (1.5, 1.5), D (4, 1), E (5, 0), F (5, 3)	**D–E: √2 = 1.414**; A–BC: √4.5 = 2.121; D–F: √5 = 2.236; BC–D: √6.5 = 2.550; E–F: √9 = 3.000; BC–E: √14.5 = 3.808; BC–F: √14.5 = 3.808; A–D: √17 = 4.123; A–E: √25 = 5.000; A–F: √34 = 5.831	**D + E**
3	A (0, 0), BC (1.5, 1.5), DE (4.5, 0.5), F (5, 3)	**A–BC: √4.5 = 2.121**; DE–F: √6.5 = 2.550; BC–DE: √10 = 3.162; BC–F: √14.5 = 3.808; A–DE: √20.5 = 4.528; A–F: √34 = 5.831	**A + BC**
4	ABC (1, 1), DE (4.5, 0.5), F (5, 3)	**DE–F: √6.5 = 2.550**; ABC–DE: √12.5 = 3.536; ABC–F: √20 = 4.472	**DE + F**
5	ABC (1, 1), DEF (4.67, 1.33)	**ABC–DEF: √(122/9) = 3.682**	**ABC + DEF**

Work done in total: 15 + 10 + 6 + 3 + 1 = 35 distance computations for 6 points. This growth is why the naive method does not scale (see part (d), Slide 21).

5. **Part (a), merge 2 — D and E at distance √2 = 1.414.** D–E is still the smallest distance in round 2 (next: A–{B,C} = 2.121). New centroid = ((4 + 5)/2, (1 + 0)/2) = **(4.5, 0.5)**, the green x and the small green circle. If you had merged D–E before B–C the tree would be identical. Clusters now: A, {B,C}, {D,E}, F (4 clusters).
6. **Part (a), merge 3 — A joins {B, C} at distance √4.5 = 2.121.** Round 3 has A (0,0), {B,C} (1.5,1.5), {D,E} (4.5,0.5), F (5,3). The smallest distance is A–{B,C}: d = √((1.5 − 0)² + (1.5 − 0)²) = √4.5 = 2.121 (the runner-up is F–{D,E} = 2.550). New centroid, weighted by the number of points (1 for A, 2 for {B,C}): x = (1·0 + 2·1.5)/3 = 1 and y = (1·0 + 2·1.5)/3 = 1, so **(1, 1)**. Check with the raw points: ((0 + 1 + 2)/3, (0 + 2 + 1)/3) = (1, 1). This is the red x on the left of Slide 17. Clusters now: {A,B,C}, {D,E}, F (3 clusters).

> **COMMON MISTAKE**
> Do not average the two *centroids* as if they counted equally: ((0, 0) + (1.5, 1.5))/2 = (0.75, 0.75) is wrong. The cluster {B,C} stands for two points, so it must count twice (weights 1 and 2), or simply re-average the original points.

BEGINNER INTUITION

Notice that A is closer to the *middle* of {B,C} (2.121) than to B or to C separately (2.236 each). Measuring to a centroid is not the same as measuring to the nearest member: the centroid sits between the members.

7. **Part (a), merge 4 — F joins {D, E} at distance √6.5 = 2.550.** Round 4 has {A,B,C} (1,1), {D,E} (4.5,0.5), F (5,3). F–{D,E}: $\sqrt{((5-4.5)^2 + (3-0.5)^2)} = \sqrt{(0.25 + 6.25)} = \sqrt{6.5} = 2.550$. {A,B,C}–{D,E}: $\sqrt{(3.5^2 + 0.5^2)} = \sqrt{12.5} = 3.536$. {A,B,C}–F: $\sqrt{(4^2 + 2^2)} = \sqrt{20} = 4.472$. The smallest is F–{D,E}. New centroid: x = (2·4.5 + 1·5)/3 = 14/3 = 4.667 and y = (2·0.5 + 1·3)/3 = 4/3 = 1.333, so **(14/3, 4/3) ≈ (4.67, 1.33)**. Slide 17 prints this red x as (4.7, 1.3), i.e. rounded to one decimal. Clusters now: {A,B,C}, {D,E,F} (2 clusters), exactly the state drawn on Slide 17.

COMMON MISTAKE

F is merged with the *cluster* {D,E} (distance 2.550), not with D alone, although the point-to-point distance d(D, F) = 2.236 is smaller. Once D has been merged, D no longer exists as a separate cluster; the only "location" of that cluster is its centroid (4.5, 0.5).

8. **Part (a), merge 5 — the last two clusters at distance √(122/9) = 3.682.** Only {A,B,C} with centroid (1, 1) and {D,E,F} with centroid (14/3, 4/3) remain: $d = \sqrt{((14/3 - 1)^2 + (4/3 - 1)^2)} = \sqrt{((11/3)^2 + (1/3)^2)} = \sqrt{(122/9)} = 3.682$ (using the rounded (4.7, 1.3) would give 3.71, so keep exact fractions). New centroid = (3·(1, 1) + 3·(14/3, 4/3))/6 = ((3 + 14)/6, (3 + 4)/6) = **(17/6, 7/6) ≈ (2.83, 1.17)**, which is also the average of all six points: (0+1+2+4+5+5)/6 = 17/6 and (0+2+1+1+0+3)/6 = 7/6. Slide 17 stops one merge earlier (at two clusters), so this centroid is not drawn, but this merge is the **root** of the dendrogram.
9. **Part (a) summary and part (b) — how the slide's centroids arise.**

Merge	Clusters merged	Distance between centroids	Points in new cluster	New centroid	On Slide 17
1	B (1,2) + C (2,1)	√2 = 1.414	2	(1.5, 1.5)	blue x, small blue circle
2	D (4,1) + E (5,0)	√2 = 1.414	2	(4.5, 0.5)	green x, small green circle
3	A (0,0) + {B,C}	√4.5 = 2.121	3	(1, 1)	red x on the left, left red ellipse
4	F (5,3) + {D,E}	√6.5 = 2.550	3	(14/3, 4/3) ≈ (4.67, 1.33)	red x printed (4.7, 1.3), right red ellipse
5	{A,B,C} + {D,E,F}	√(122/9) = 3.682	6	(17/6, 7/6) ≈ (2.83, 1.17)	not drawn (the root)

Part (b). Every centroid on the slide is simply the average of the data points inside its circle: (1.5, 1.5) is the average of (1,2) and (2,1); (4.5, 0.5) is the average of (4,1) and (5,0); (1, 1) is the average of (0,0), (1,2), (2,1); (4.7, 1.3) is the average of (4,1), (5,0), (5,3), namely (14/3, 4/3) rounded to one decimal. The dashed lines on the slide join each centroid to the points it was averaged from. The picture shows the state after four merges, that is, the two clusters {A,B,C} and {D,E,F}. The colours tell the story: blue = merge 1, green = merge 2, red = the two 3-point clusters created by merges 3 and 4, each of which added one red "loner" point (A or F) to a pair. The same colours are used for the leaves of the slide's dendrogram.

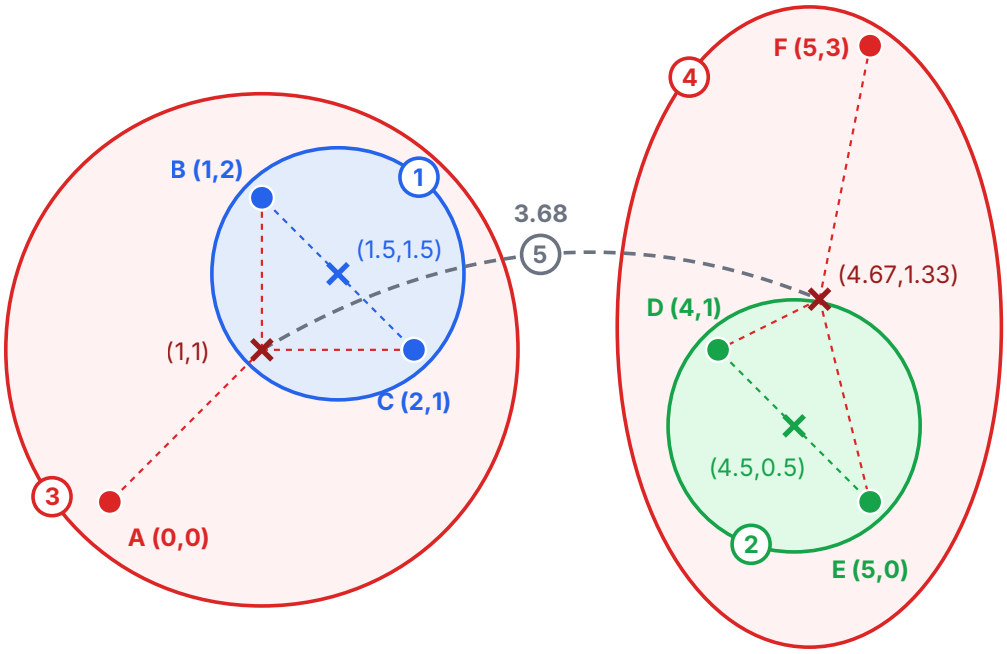


Figure 1 — The Slide-17 picture rebuilt with the merge order. Numbered tags 1–4 sit on the circles drawn on the slide; tag 5 is the last merge (distance between the centroids (1,1) and (4.67,1.33)), which the slide does not draw. x marks a centroid, dashed spokes join a centroid to the points it averages.

10. **Part (c) — draw the dendrogram.** Recipe: (i) put the six points as **leaves** along the bottom, ordered so that lines do not cross (A, B, C, D, E, F); (ii) take the merges in order and, for each, draw a horizontal bar at the height equal to the merge distance, joining the vertical lines of the two clusters merged (together they form a "U"); the vertical stem rising from the middle of the bar is the new cluster and is used by the next merge; (iii) the last bar is the **root**: all points in one cluster. Heights: 1.414, 1.414, 2.121, 2.550, 3.682.

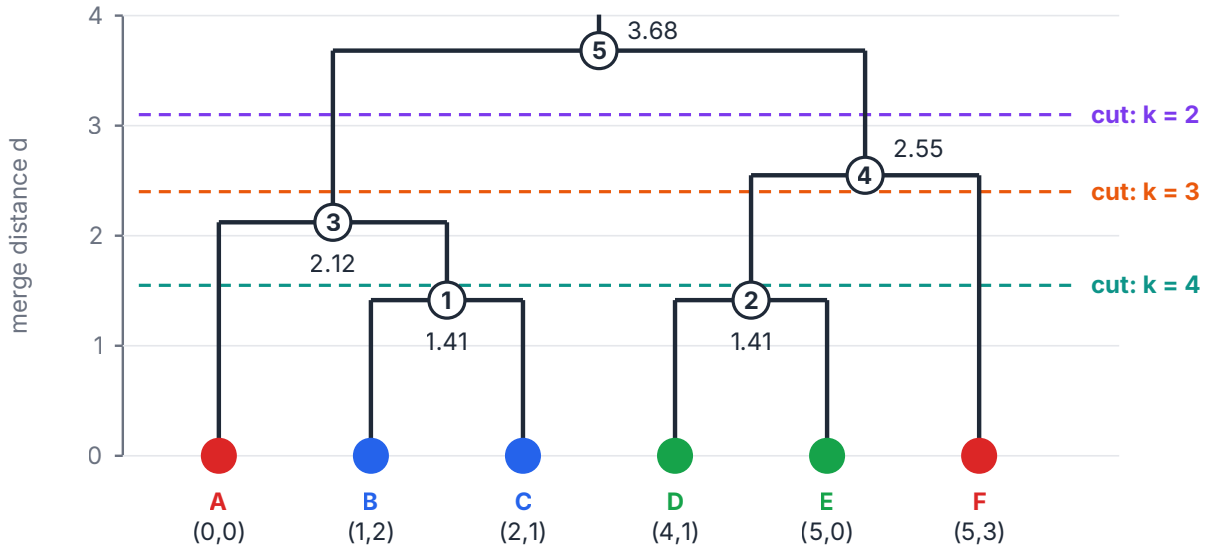


Figure 2 — Dendrogram of the six points, drawn to scale (vertical axis = distance between the centroids at the moment of the merge). Each numbered badge is one merge (1–5 = order) and the number next to it is the merge distance; the new centroids are in the table above. The dashed lines show where to cut for k = 2, 3 and 4. Leaf colours are those of Slide 17.

In bracket form the tree is **((A, (B, C)), ((D, E), F))**: the left sub-tree (A with the pair B–C) is the left ellipse of the slide, the right sub-tree (the pair D–E with F) is the right ellipse.

The slide's dendrogram (bottom right of Slide 17) has the same shape but no distance axis: the blue pair is joined first, then the green pair, then the red leaf (0,0) joins the blue pair, the red leaf (5,3) joins the green pair, and the two sub-trees meet at the root. Its bar heights are only schematic: the two bars of merges 1 and 2 are really at the same height, 1.41, although the slide draws the green one a little higher.

> **SLIDE ERRATUM (MINOR — PRESENTATION ONLY, NO NUMERICAL ERROR)**
> All four printed centroids agree with the computation; (4.7, 1.3) is (14/3, 4/3) = (4.67, 1.33) rounded to one decimal. What the slide leaves out or draws loosely: (1) it prints no question and no merge distances; (2) its dendrogram is not to scale (no axis; the two merges at √2 are drawn at different heights); (3) it stops at two clusters, so the last merge (distance 3.682, centroid (2.83, 1.17)) and the root height are not shown. Figure 2 supplies the correct to-scale version.

11. **Part (d) — what a dendrogram shows.**
    - The whole merge history as a **nested (hierarchical) structure**: every cluster contains the smaller clusters below it. Leaves are the data points; each U is one merge.
    - The **order** of the merges (bottom to top) and the **distance** at which each happened (the height of the bar). Low bars = very similar points; a long vertical stretch without any merge means a big gap between groups.
    - You can read off a clustering for *any* number of clusters without re-running the algorithm. (Read top-down the tree looks like repeated splitting, the idea of the *divisive* methods of Slide 14; this example is agglomerative, built bottom-up.)

> **COMMON MISTAKE**
> The left-to-right order of the leaves carries no meaning: at every U the two branches may be swapped, like a hanging mobile. Only the tree structure and the heights matter. Also, the height is the *distance between centroids at the merge*, not the size of the cluster.

12. **Part (d) — how to cut the dendrogram to get k clusters.** Draw a **horizontal line** at a chosen height h. Every vertical line it crosses is one cluster, made of all the leaves hanging below that line; the number of crossings is k. A lower cut gives more, tighter clusters; a higher cut gives fewer, looser ones. Equivalent view: stop the algorithm after n − k merges (here n = 6).

k	Cut height h	Merges done	Clusters obtained
6	h < 1.41	0	A, B, C, D, E, F (every point alone)
5	no clean cut: merges 1 and 2 are tied at 1.41, so stop after merge 1	1	{B,C}, A, D, E, F
4	1.41 < h < 2.12	2	A, {B,C}, {D,E}, F
3	2.12 < h < 2.55	3	{A,B,C}, {D,E}, F
2	**2.55 < h < 3.68**	**4**	**{A,B,C}, {D,E,F}: exactly the two big ellipses of Slide 17**
1	h > 3.68	5	all six points

Unlike k-means (Slide 23: "start by picking k"), hierarchical clustering does **not** need k in advance: you build the whole tree and choose k afterwards.

Beyond the slides: a common rule for choosing the cut is to look for the longest vertical stretch with no merge, i.e. the biggest jump between successive merge heights. Here the jumps are 0.71 (1.41 to 2.12), 0.43 (2.12 to 2.55) and 1.13 (2.55 to 3.68); the largest is just below the root, so k = 2 is the natural choice, in agreement with the slide. Other stopping rules in the textbook (*Mining of Massive Datasets*, chapter 7, hierarchical clustering): stop at k clusters, or stop when the next merge would create a cluster that is too spread out (for example its diameter exceeds a threshold).

> **WHY THIS METHOD DOES NOT SCALE (SLIDE 21)**
> The naive implementation computes the pairwise distances between all pairs of clusters at every step and then merges, which costs $O(N^3)$ for N points (Slide 21), too expensive for big data sets that do not fit in memory. Beyond the slides: keeping the distances in a priority queue reduces this to about $O(N^2 \log N)$, which is still too slow for very large N. This is the motivation for the point-assignment methods (k-means and relatives) that follow.

13. **Part (e) — non-Euclidean case: why the centroid breaks down (Slide 18).** In a Euclidean space the points are vectors of numbers, so we can add them and divide by n; the average is again a point of the space. In a non-Euclidean space the objects may be, say, words compared by edit distance or sets compared by Jaccard distance (Beyond the slides: these are examples, the slide names none). Then "the only locations we can talk about are the points themselves": there is no "average" of two words or two sets. Slide 18 therefore changes the two answers of Slide 16:
    - **(1) Representing a cluster:** use the **clustroid** = the data point of the cluster that is "closest" to the other points of the cluster.
    - **(2) Nearness of clusters:** treat the clustroid as if it were the centroid when computing inter-cluster distances, i.e. d(P, Q) = distance between the clustroid of P and the clustroid of Q. The rest of the algorithm is unchanged.

    Slide 18 leaves "closest" informal. Beyond the slides (textbook *Mining of Massive Datasets*, chapter 7): "closest" is usually made precise as the point with the **smallest sum of distances** to the other points of the cluster; two other common choices are the smallest *maximum* distance and the smallest *sum of squares* of distances. I use the sum of distances and show all three.
14. **Part (e) — worked demonstration of a clustroid.** The Slide-17 data is Euclidean, so a centroid exists, but we can still compute the clustroids of the two final clusters to see the difference. For {D, E, F}:

Candidate in {D, E, F}	Distances to the other points	Sum	Max	Sum of squares
D (4,1)	d(D,E) = 1.414 + d(D,F) = 2.236	**3.650**	2.236	7
E (5,0)	d(E,D) = 1.414 + d(E,F) = 3.000	4.414	3.000	11
F (5,3)	d(F,D) = 2.236 + d(F,E) = 3.000	5.236	3.000	14

    The clustroid is **D (4, 1)** under all three criteria (smallest sum 3.650, smallest max 2.236, smallest sum of squares 7). The centroid (4.67, 1.33) is *not* one of the data points; the clustroid always is. For {A, B, C}:

Candidate in {A, B, C}	Distances to the other points	Sum	Max	Sum of squares
A (0,0)	d(A,B) = 2.236 + d(A,C) = 2.236	4.472	2.236	10
B (1,2)	d(B,A) = 2.236 + d(B,C) = 1.414	**3.650**	2.236	7
C (2,1)	d(C,A) = 2.236 + d(C,B) = 1.414	**3.650**	2.236	7

    B and C tie (sum 3.650 each, sum of squares 7 each; the maximum criterion cannot separate any of the three, all 2.236), so the clustroid is **B (1, 2) or C (2, 1)**, chosen arbitrarily; the centroid (1, 1) is not a data point. Treating clustroids as the cluster locations, the distance between the two clusters would be d(C, D) = 2 or d(B, D) = √10 = 3.162, instead of the centroid distance 3.682. So the clustroid distance is only a stand-in for "where the cluster is", and the numbers (and in general the merge order) can differ from the centroid version. Beyond the slides (checked on this data): if the clustroids of the pairs {B,C} and {D,E} were chosen as C (2,1) and D (4,1), those two pairs would be only 2 apart and would merge *before* A joins (A is 2.236 from any of its candidates), whereas the centroids (1.5,1.5) and (4.5,0.5) are 3.162 apart.

    A truly non-Euclidean mini-example: words compared by edit distance (number of letters inserted or deleted). For the cluster {car, cart, carts}: d(car, cart) = 1, d(cart, carts) = 1, d(car, carts) = 2. Sums of distances: car 1 + 2 = 3, cart 1 + 1 = 2, carts 2 + 1 = 3. The clustroid is **cart**; there is no "average of car and carts" to use as a centroid.
15. **Part (e) — centroid versus clustroid at a glance.**

	Centroid (Slide 16, Euclidean case)	Clustroid (Slide 18, non-Euclidean case)
Needs	points that can be averaged (vectors of numbers)	only a distance between any two objects
Definition	average of the points of the cluster	the data point "closest" to the other points (e.g. smallest sum of distances)
A real data point?	usually not, e.g. (1, 1) is not A, B or C	always, by definition
Distance between clusters	distance between the two centroids	treat the clustroid as the centroid: distance between the two clustroids
After a merge	weighted average of the two old centroids (cheap)	Beyond the slides: no simple formula from the two old clustroids; the new clustroid has to be picked from the points of the merged cluster (needs distances, costlier)

Beyond the slides: other definitions of cluster nearness exist (for example the smallest distance between any two members of the two clusters); this course uses centroid distance (Euclidean) and clustroid distance (non-Euclidean).

BEGINNER INTUITION

A centroid is the group's imaginary centre of gravity (it may fall in empty space, like the middle of a ring of people). A clustroid is the real person standing closest to everyone else: the group's "delegate". When you cannot compute a centre, you send the delegate instead.

EXAM TIP

For a "run hierarchical clustering" question: (1) state the rule (Euclidean distance between centroids); (2) write the pairwise distance table; (3) for each merge give the pair, the distance and the new centroid computed from **all original points** of the merged cluster; (4) say how you broke any tie; (5) stop at the required k or when one cluster is left; (6) draw the dendrogram with the heights written on it. If asked "what changes in the non-Euclidean case?", answer in two lines: no averages exist, so a cluster is represented by a **clustroid** (an actual data point closest to the others), and the distance between clusters is computed between clustroids "as if they were centroids"; add one example (strings with edit distance).

Final answer:
(a) Merges (centroid distance): 1) B + C at √2 = 1.414 → (1.5, 1.5); 2) D + E at √2 = 1.414 → (4.5, 0.5); 3) A + {B,C} at √4.5 = 2.121 → (1, 1); 4) F + {D,E} at √6.5 = 2.550 → (14/3, 4/3) ≈ (4.67, 1.33); 5) {A,B,C} + {D,E,F} at √(122/9) = 3.682 → (17/6, 7/6) ≈ (2.83, 1.17).
(b) Each printed centroid is the average of the points inside its circle; (4.7, 1.3) is (14/3, 4/3) rounded to one decimal.
(c) Tree ((A,(B,C)),((D,E),F)) with bars at 1.41, 1.41, 2.12, 2.55, 3.68 (Figure 2).
(d) A dendrogram records the merge order and the distance of every merge as a nested tree. Cut it with a horizontal line: k = number of vertical lines crossed (or stop after n − k merges). k = 2: {A,B,C} and {D,E,F}; k = 3: {A,B,C}, {D,E}, {F}; k = 4: A, {B,C}, {D,E}, F.
(e) Clustroid = the data point of the cluster that is closest to the other points (no averaging needed, always a real point); cluster distance = distance between clustroids, used as if they were centroids.

> **Where the marks come from:** (1) stating the method: agglomerative = start with singletons and repeatedly merge the two nearest clusters; centroid = average of the points; cluster distance = Euclidean distance between centroids; (2) the distances at the start and at each round, identifying the closest pair (tie B–C / D–E noted and broken); (3) each of the five merges: the pair, its distance, and the correct new centroid (weighted, not the plain average of two centroids); (4) explaining that the slide's centroids are averages of the points in each circle, with (4.7, 1.3) rounded from (14/3, 4/3); (5) the dendrogram: correct nesting ((A,(B,C)),((D,E),F)), merge order, labelled leaves and heights; (6) saying what a dendrogram shows and the cutting rule (horizontal line, count the crossed vertical lines, or stop after n − k merges), with the clusters for k = 2, 3, 4; (7) the non-Euclidean variant: why no centroid exists, clustroid = data point closest to the others, and distances between clustroids used in place of centroid distances.

Slide check: matches the slide's answer. The slide prints no numeric answer, only the picture, and the computation reproduces all four printed centroids ((1.5, 1.5), (4.5, 0.5), (1, 1) exact; (4.7, 1.3) is 14/3 and 4/3 rounded to one decimal) and the same merge order as the slide's dendrogram (blue pair, green pair, then (0,0) joins the blue pair, (5,3) joins the green pair, then the root). Slide erratum (minor, presentation only, no answer changes): the slide's dendrogram is schematic (no distance axis; the two √2 merges are drawn at different heights), it does not show the last merge (distance 3.682, centroid (2.83, 1.17)) and it prints no distances or question; Slide 18 says "closest" without defining it (the three precise definitions above are Beyond the slides).

Why hierarchical clustering does not scale

Lecture 5 · Slide 21

Slide 21 describes the **naïve implementation**: at each step compute the pairwise distances between all pairs of clusters, then merge the closest pair. The slide states its cost as **$O(N^3)$** and concludes it is **too expensive for really big datasets that do not fit in memory**. N is the number of points (the slide does not define it).

Where the N^3 comes from Lecture 5 · Slide 21

1. There are about N merges (N clusters down to 1).
2. With m clusters left, a step looks at $m(m-1)/2$ pairs, about $N^2/2$ at the start.
3. Total work: $\sum_{m=2..N} m(m-1)/2 = (N+1)N(N-1)/6 \approx N^3/6$, which is $O(N^3)$.
4. Check with tiny numbers: N = 6 gives 15+10+6+3+1 = 35 distance computations (the table in Quiz L5-Q1 does exactly this). N = 12 gives 286 (the six rounds of Quiz L5-Q2 up to k = 6 use 251 of them). N = 10^6 gives about 1.7×10^{17}.

There is a second problem besides time: the method needs all the data, and all pairwise distances, in memory at once. That is why the rest of the lecture introduces algorithms that read the data in chunks (BFR, CURE) and ones that run in parallel (MapReduce).

BEYOND THE SLIDES: THE FASTER VERSION

Keep all pairwise distances in a **priority queue** (a heap). Building it costs $O(N^2)$. Each merge then deletes the entries of the two merged clusters and inserts the O(N) distances to the new cluster, each operation costing O(log N), so one merge costs O(N log N) and all N merges cost **$O(N^2 \log N)$**. This is better than $O(N^3)$ but still far too slow, and it still needs $O(N^2)$ memory, for really big data.

COMMON MISTAKE

Writing "$O(N^2)$" for the naïve method. The slide says $O(N^3)$: N rounds, each with about N^2 pairs. The priority-queue version is $O(N^2 \log N)$.

Worked example: hierarchical clustering of 12 points (k = 6)

Lecture 5 · Slides 69–75

These slides come later in the deck (they are the lecture's closing quiz), but they belong to the same topic, so the full worked solution is placed here. Slide 69, "Quiz: Hierarchical Clustering", gives twelve 2-D points and

says: *apply hierarchical clustering with Euclidean distance; assume the stopping point is k = 6*. Slides 70–75 then walk through the answer: the starting state, the first merge (Slide 71), the next merges (Slides 72–74), the six final clusters (Slide 74) and the final tree (Slide 75).

What the solution needs from this chapter and what to watch for:

- **Stopping rule:** k = 6 means stop when 6 clusters remain, so 12 − 6 = 6 merges.
- **Centroid rule:** every merged cluster is represented by the average of all its points (use the weighted update), and the distance between clusters is the distance between centroids (Slide 16).
- **Ties:** the slides break ties arbitrarily (Slide 71 says so). Several pairs are equally close at some rounds; any consistent tie-break is acceptable, but you must say you chose one.
- **Speed trick:** compare squared distances and compute the full table only where points look close in the picture.

Try it yourself first: cover the solution, do the six merges on the twelve points, and only then compare with the worked answer below.

Quiz L5-Q2 **Hierarchical clustering of 12 points down to k = 6** Lecture 5 · Slides 69–75

QUESTION (AS ON THE SLIDES)

Slide 69, "Quiz: Hierarchical Clustering": "Apply hierarchical clustering on the following data in a 2-dimensional Euclidean space. Assume stopping point is k = 6."

The data are a scatter plot of **twelve points**; every dot is printed with its coordinates and the slide gives no table. Read from the figure:

Region of the plot	Points (x, y)	Count
upper left	`(4,10)` `(7,10)` `(4,8)` `(6,8)`	4
right	`(12,6)` `(10,5)` `(11,4)` `(9,3)` `(12,3)`	5
lower left	`(3,4)` `(2,2)` `(5,2)`	3

The slide asks one thing and then walks through the answer on slides 70–75 (every merge, the state at k = 6, and the final tree). The sub-tasks below follow that walk-through so that every step can earn partial marks; the letters (a) to (c) are ours, not the slide's.

(a) Run agglomerative hierarchical clustering (each cluster represented by its centroid, Euclidean distance between centroids). List every merge: the two clusters, their distance and the new centroid. Stop when k = 6 clusters remain.

(b) Give the six final clusters (members and centroids).

(c) Draw the tree (dendrogram) of the complete grouping (slide 75) and show where the k = 6 cut falls.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

1. **Formula / idea: what "agglomerative hierarchical clustering with centroids" means (Lecture 5 · Slides 14 and 16).**
   Slide 14 splits clustering methods into *hierarchical* (agglomerative = bottom-up, divisive = top-down) and *point assignment*. This quiz is **agglomerative**: at the start every point is a cluster by itself, and in every round the two **nearest** clusters are combined into one. Slide 16 gives the two rules you need to do this by hand: (1) in the Euclidean case a cluster is represented by its **centroid** = the average of its data points; (2) the nearness of two clusters is the **distance between their centroids**. (In a non-Euclidean space there is no average of points, so slide 18 uses a *clustroid* instead; not needed here.) The **stopping point k = 6** means: stop as soon as **6 clusters** are left.

FORMULAS USED

Euclidean distance between centroids (x_1, y_1) and (x_2, y_2): $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

Centroid of a cluster with n points: $\frac{\sum x}{n}$ and $\frac{\sum y}{n}$ (the average of each coordinate).

Centroid after a merge (cluster P with n_P points and centroid c_P, cluster Q likewise): $c_{new} = (n_P \cdot c_P + n_Q \cdot c_Q) / (n_P + n_Q)$. It is a **weighted** average and equals the plain average of all the original points inside the new cluster.

BEGINNER INTUITION

Twelve pins on a map. Find the two closest pins and tie them together; from now on the tied pair is one blob that sits at its middle point (its centroid). Look for the closest two things again (single pins or blobs, always measured from centre to centre), tie them, and repeat until only six things are left. The record of who was tied to whom, and how far apart they were, is the tree on slide 75.

EXAM TIP

To find the closest pair, compare **squared** distances $d^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$ and skip the square root: the smallest d^2 belongs to the smallest d, and d^2 is a whole number for the raw points. Take the root only for the pair you actually merge. Keep centroids as exact fractions (14/3, 26/3) and round only at the end.

Beyond the slides: this way of measuring the distance between clusters is usually called *centroid linkage*. Other rules (single, complete, average linkage) measure cluster distance differently and can give a different tree; in this course use only the centroid rule of slide 16.

2. **Data, initial state and number of merges (Lecture 5 · Slide 70).** The 12 points are the ones read from the figure of slide 69. Initially each point is a cluster by itself and is the centroid of that cluster, so there are 12 clusters and their centroids are simply the 12 points: (2,2), (3,4), (4,8), (4,10), (5,2), (6,8), (7,10), (9,3), (10,5), (11,4), (12,3), (12,6). Every merge replaces two clusters by one, so the number of clusters drops by exactly 1 per merge. Going from 12 clusters to k = 6 needs 12 - 6 = **6 merges**.
3. **Round 1: all distances between the 12 points (Lecture 5 · Slide 71).** With 12 clusters there are 12·11/2 = 66 pairs. All 66 squared distances are in the table (a cell is d^2 between the row point and the column point; for example (10,5) and (11,4): $(10 - 11)^2 + (5 - 4)^2 = 1 + 1 = 2$).

d^2	(3,4)	(4,8)	(4,10)	(5,2)	(6,8)	(7,10)	(9,3)	(10,5)	(11,4)	(12,3)	(12,6)
(2,2)	**5**	40	68	9	52	89	50	73	85	101	116
(3,4)		17	37	8	25	52	37	50	64	82	85
(4,8)			**4**	37	**4**	13	50	45	65	89	68
(4,10)				65	8	9	74	61	85	113	80
(5,2)					37	68	17	34	40	50	65
(6,8)						**5**	34	25	41	61	40
(7,10)							53	34	52	74	41
(9,3)								**5**	**5**	9	18
(10,5)									**2**	8	**5**
(11,4)										**2**	**5**
(12,3)											9

Reading the table: the smallest value is **$d^2 = 2$** (green), which occurs **twice**: (10,5)–(11,4) and (11,4)–(12,3), each at $d = \sqrt{2} \approx 1.414$. This is exactly what slide 71 says. The next values are $d^2 = 4$ (bold; two pairs: (4,8)–(4,10) and (4,8)–(6,8)) and $d^2 = 5$ (bold; six pairs: (2,2)–(3,4), (6,8)–(7,10), (9,3)–(10,5), (9,3)–(11,4), (10,5)–(12,6), (11,4)–(12,6)). The other 56 pairs all have $d^2 \geq 8$.

> **EXAM TIP**
> In the exam you do not have to compute all 66 distances. Look at the picture: only pairs that look close can be the minimum. Compute those few and compare; for every other pair just make sure it is clearly farther.

4. **Merge 1 (M1): (11,4) + (12,3) at d = √2 ≈ 1.414 (Lecture 5 · Slide 71).** The two pairs at $d^2 = 2$ share the point (11,4), so there is a **tie**. Slide 71 says "let us break ties arbitrarily" and combines (11,4) with (12,3): $d = \sqrt{((11 - 12)^2 + (4 - 3)^2)} = \sqrt{2}$. New centroid = average of the two points = ((11 + 12)/2, (4 + 3)/2) = **(11.5, 3.5)**. We call the new cluster C1 = {(11,4), (12,3)}. **11 clusters** are left.
5. **Merge 2 (M2): (4,8) + (4,10) at d = 2 (Lecture 5 · Slide 72).** Only the distances to the new centroid (11.5, 3.5) are new; all the others are as in the table. Round 2, closest candidates:

Pair of clusters (centroid)	d^2 (squared distance)	d	Verdict
(4,8) – (4,10)	4	2.000	**merged (slide's choice)**
(4,8) – (6,8)	4	2.000	tie at the minimum, not chosen
(10,5) – C1 (11.5, 3.5)	9/2 = 4.5	2.121	farther
(2,2) – (3,4)	5	2.236	farther
(6,8) – (7,10)	5	2.236	farther
(9,3) – (10,5)	5	2.236	farther
(10,5) – (12,6)	5	2.236	farther

   The smallest is $d^2 = 4$, i.e. d = 2, and it occurs for two pairs: (4,8)–(4,10) with $d = \sqrt{(0^2 + 2^2)} = 2$ and (4,8)–(6,8) with $d = \sqrt{(2^2 + 0^2)} = 2$ (this second tie is not mentioned on the slide, see the erratum at the end). Slide 72 combines (4,8) and (4,10). New centroid = ((4 + 4)/2, (8 + 10)/2) = **(4, 9)**. We call it C2 = {(4,8), (4,10)}. **10 clusters** are left.
6. **Merge 3 (M3): (10,5) + C1 at d = √4.5 ≈ 2.121 (Lecture 5 · Slide 72).** Round 3:

Pair of clusters (centroid)	d^2 (squared distance)	d	Verdict
(10,5) – C1 (11.5, 3.5)	9/2 = 4.5	2.121	**merged (unique closest pair)**
(2,2) – (3,4)	5	2.236	farther
(6,8) – C2 (4, 9)	5	2.236	farther
(6,8) – (7,10)	5	2.236	farther
(9,3) – (10,5)	5	2.236	farther
(10,5) – (12,6)	5	2.236	farther
(9,3) – C1 (11.5, 3.5)	13/2 = 6.5	2.550	farther
(12,6) – C1 (11.5, 3.5)	13/2 = 6.5	2.550	farther

   Now there is a **unique** closest pair: (10,5) and the centroid of C1, (11.5, 3.5): $d = \sqrt{((10 - 11.5)^2 + (5 - 3.5)^2)} = \sqrt{(1.5^2 + 1.5^2)} = \sqrt{4.5} \approx 2.121$. This matches "the two closest centroids are (10,5) and (11.5, 3.5)" on slide 72. C1 has 2 points and (10,5) has 1, so the new centroid is a **weighted** average: x = (1·10 + 2·11.5)/3 = 33/3 = 11 and y = (1·5 + 2·3.5)/3 = 12/3 = 4, i.e. **(11, 4)**. Check with the raw points: ((10 + 11 + 12)/3, (5 + 4 + 3)/3) = (11, 4). We call the new cluster C3 = {(10,5), (11,4), (12,3)}. **9 clusters** are left.

> **COMMON MISTAKE**
> Two traps in this merge. (1) **Do not take the midpoint of the two old centroids.** The midpoint of (10,5) and (11.5, 3.5) is (10.75, 4.25), which is wrong because C1 contains *two* points and must count twice; the weighted average (11, 4) is the same as averaging the three original points. (2) **Measure to the centroid, not to a member.** In round 1 the point (10,5) was only √2 ≈ 1.414 away from the single point (11,4), but (11,4) is no longer a cluster of its own: C1 sits at its centroid (11.5, 3.5), which is 2.121 away from (10,5). Distances between clusters can grow when clusters merge. (Beyond the slides: measuring to the nearest member would be a different method, called single linkage.)

7. **Merge 4 (M4): (6,8) + C2 at d = √5 ≈ 2.236 (Lecture 5 · Slides 73–74).** Round 4, the state shown on slide 73:

Pair of clusters (centroid)	d^2 (squared distance)	d	Verdict
(6,8) – C2 (4, 9)	5	2.236	**merged (slide's choice)**
(2,2) – (3,4)	5	2.236	tie at the minimum, not chosen
(6,8) – (7,10)	5	2.236	tie at the minimum, not chosen
(9,3) – C3 (11, 4)	5	2.236	tie at the minimum, not chosen
(12,6) – C3 (11, 4)	5	2.236	tie at the minimum, not chosen
(3,4) – (5,2)	8	2.828	farther
(2,2) – (5,2)	9	3.000	farther

The smallest d^2 is 5 (d = √5 ≈ 2.236) and it is shared by **five** pairs: (6,8)–C2, (6,8)–(7,10), (2,2)–(3,4), (9,3)–C3 and (12,6)–C3. This is slide 73's "several pairs of centroids that are at distance √5". For example, (6,8) to the centroid (4,9) of C2: $\sqrt{(6-4)^2 + (8-9)^2} = \sqrt{4+1} = \sqrt{5}$. Slide 74 combines (6,8) with the cluster of two elements having centroid (4,9). Weights 1 and 2: x = (1·6 + 2·4)/3 = 14/3 and y = (1·8 + 2·9)/3 = 26/3, so the new centroid is **(14/3, 26/3) ≈ (4.67, 8.67)**; the slide prints it rounded as (4.7, 8.7). Check with the raw points: ((6 + 4 + 4)/3, (8 + 8 + 10)/3) = (14/3, 26/3). We call it C4 = {(4,8), (4,10), (6,8)}. **8 clusters** are left.

Slides 73–74 show the three √5 merges together. In reality they happen one after the other, and you re-check the closest pair after each one (the next two tables do exactly that). We follow the slide's order: M4, M5, M6.

8. **Merge 5 (M5): (2,2) + (3,4) at d = √5 ≈ 2.236 (Lecture 5 · Slide 74).** Round 5:

Pair of clusters (centroid)	d^2 (squared distance)	d	Verdict
(2,2) – (3,4)	5	2.236	**merged (slide's choice)**
(9,3) – C3 (11, 4)	5	2.236	tie at the minimum, not chosen
(12,6) – C3 (11, 4)	5	2.236	tie at the minimum, not chosen
(7,10) – C4 (4.67, 8.67)	65/9 ≈ 7.22	2.687	farther
(3,4) – (5,2)	8	2.828	farther

Three pairs are still tied at $d^2 = 5$. The old tie (6,8)–(7,10) has disappeared because (6,8) is no longer a cluster of its own: (7,10) is now √(65/9) ≈ 2.687 from the centroid of C4. Slide 74 merges (2,2) with (3,4): $d = \sqrt{(3-2)^2 + (4-2)^2} = \sqrt{5}$. New centroid = ((2 + 3)/2, (2 + 4)/2) = **(2.5, 3)**. We call it C5 = {(2,2), (3,4)}. **7 clusters** are left.

9. **Merge 6 (M6): (9,3) + C3 at d = √5 ≈ 2.236 (Lecture 5 · Slide 74), and STOP.** Round 6:

Pair of clusters (centroid)	d^2 (squared distance)	d	Verdict
(9,3) – C3 (11, 4)	5	2.236	**merged (slide's choice)**
(12,6) – C3 (11, 4)	5	2.236	tie at the minimum, not chosen
(7,10) – C4 (4.67, 8.67)	65/9 ≈ 7.22	2.687	farther
(5,2) – C5 (2.5, 3)	29/4 = 7.25	2.693	farther

Two pairs are still tied at $d^2 = 5$: (9,3)–C3 and (12,6)–C3. Slide 74 merges (9,3) with the cluster of three elements having centroid (11,4): $d = \sqrt{(11-9)^2 + (4-3)^2} = \sqrt{5}$. Weights 1 and 3: x = (1·9 + 3·11)/4 = 42/4 = 10.5 and y = (1·3 + 3·4)/4 = 15/4 = 3.75, so the new centroid is **(10.5, 3.75)** (the slide prints (10.5, 3.8), which is 3.75 rounded to one decimal). Check with the raw points: ((9 + 10 + 11 + 12)/4, (3 + 5 + 4 + 3)/4) = (42/4, 15/4). We call it C6 = {(9,3), (10,5), (11,4), (12,3)}.

Count the clusters: 12 - 6 merges = 6 clusters, which is the stopping point k = 6, so the algorithm stops here. (Had we continued, the next closest pairs would be at 2.687, 2.693 and 2.704, all larger than √5.)

COMMON MISTAKE

"Stopping point k = 6" means *stop when 6 clusters are left*, not "do 6 merges". Here the two coincide only because 12 - 6 = 6. With the same data and k = 3 you would need 9 merges.

10. **Part (a): the merge log (summary of M1 to M6).**

Merge	Clusters combined (centroids)	d^2	d	New cluster	New centroid	Left
M1	(11,4) + (12,3)	2	1.414	C1 = {(11,4), (12,3)}	(11.5, 3.5)	11
M2	(4,8) + (4,10)	4	2.000	C2 = {(4,8), (4,10)}	(4, 9)	10
M3	(10,5) + C1 (11.5, 3.5)	9/2 = 4.5	2.121	C3 = {(10,5), (11,4), (12,3)}	(11, 4)	9
M4	(6,8) + C2 (4, 9)	5	2.236	C4 = {(4,8), (4,10), (6,8)}	(14/3, 26/3) ≈ (4.67, 8.67)	8
M5	(2,2) + (3,4)	5	2.236	C5 = {(2,2), (3,4)}	(2.5, 3)	7
M6	(9,3) + C3 (11, 4)	5	2.236	C6 = {(9,3), (10,5), (11,4), (12,3)}	(21/2, 15/4) = (10.5, 3.75)	6

C1 to C6 are our names for the clusters created by merges 1 to 6 (the slides do not name them). "Left" = number of clusters remaining after the merge. d = √(d^2).

11. **The state after every merge (Lecture 5 · Slides 70–74).** Single points never change, so only clusters with two or more points are listed with their centroid. The cluster created by each merge is in bold.

State	Clusters	Clusters with 2 or more points: members → centroid	Single-point clusters (each is its own centroid)
start (slide 70)	12	none	(2,2), (3,4), (4,8), (4,10), (5,2), (6,8), (7,10), (9,3), (10,5), (11,4), (12,3), (12,6)
after M1 (slide 71)	11	**C1 = {(11,4), (12,3)} → (11.5, 3.5)**	(2,2), (3,4), (4,8), (4,10), (5,2), (6,8), (7,10), (9,3), (10,5), (12,6)
after M2 (slide 72)	10	C1 = {(11,4), (12,3)} → (11.5, 3.5) **C2 = {(4,8), (4,10)} → (4, 9)**	(2,2), (3,4), (5,2), (6,8), (7,10), (9,3), (10,5), (12,6)
after M3 (slide 72)	9	C2 = {(4,8), (4,10)} → (4, 9) **C3 = {(10,5), (11,4), (12,3)} → (11, 4)**	(2,2), (3,4), (5,2), (6,8), (7,10), (9,3), (12,6)
after M4 (slide 74)	8	C3 = {(10,5), (11,4), (12,3)} → (11, 4) **C4 = {(4,8), (4,10), (6,8)} → (4.67, 8.67)**	(2,2), (3,4), (5,2), (7,10), (9,3), (12,6)
after M5 (slide 74)	7	C3 = {(10,5), (11,4), (12,3)} → (11, 4) C4 = {(4,8), (4,10), (6,8)} → (4.67, 8.67) **C5 = {(2,2), (3,4)} → (2.5, 3)**	(5,2), (7,10), (9,3), (12,6)
after M6 (slide 74)	6	C4 = {(4,8), (4,10), (6,8)} → (4.67, 8.67) C5 = {(2,2), (3,4)} → (2.5, 3) **C6 = {(9,3), (10,5), (11,4), (12,3)} → (10.5, 3.75)**	(5,2), (7,10), (12,6)

12. **Part (b): the six final clusters (Lecture 5 · Slide 74).**

#	Cluster	Members	Size	Centroid (exact)	Centroid (decimal)	Slide 74 prints
1	C5	{(2,2), (3,4)}	2	(5/2, 3)	(2.5, 3)	(2.5, 3)
2	C4	{(4,8), (4,10), (6,8)}	3	(14/3, 26/3)	(4.67, 8.67)	(4.7, 8.7)
3	single point	{(5,2)}	1	(5, 2)	(5, 2)	—
4	single point	{(7,10)}	1	(7, 10)	(7, 10)	—
5	C6	{(9,3), (10,5), (11,4), (12,3)}	4	(21/2, 15/4)	(10.5, 3.75)	(10.5, 3.8)
6	single point	{(12,6)}	1	(12, 6)	(12, 6)	—
	Total		**12**			

Cross-check: 3 + 1 + 2 + 1 + 4 + 1 = 12 points, none lost and none counted twice. The slide rounds the centroids to one decimal, so (14/3, 26/3) is printed as (4.7, 8.7) and 3.75 as 3.8; this is only rounding. The picture shows the clusters, their centroids (+) and the order of the merges (numbered dashed links).

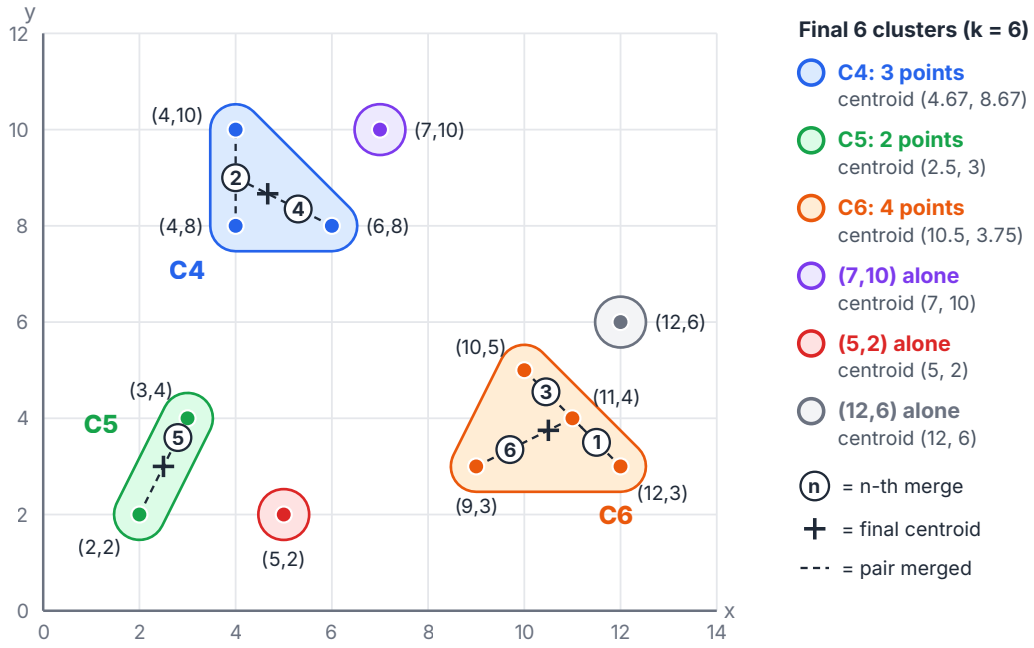


Figure L5-Q2-1. The twelve points, the six clusters at k = 6 (shaded) and the order of the six merges (numbered dashed links). Equal scale on both axes, so distances can be compared by eye. The slide-74 picture shows the same grouping.

13. **Part (c): the tree (dendrogram) and the cut at k = 6 (Lecture 5 · Slide 75).** A dendrogram records the merges: the leaves are the 12 points, every horizontal bar is one merge, and the height of the bar is the distance between the two centroids at that moment (the vertical axis below; slide 75 draws the same tree without a scale). A horizontal cut through the tree leaves as many clusters as vertical lines it crosses. A cut at about 2.46 (anywhere between 2.236, the last merge of the six, and 2.687, merge M7) crosses **6** lines, and these six lines are the six clusters of part (b). Merges M7 to M11 lie above the cut and are not performed when k = 6.

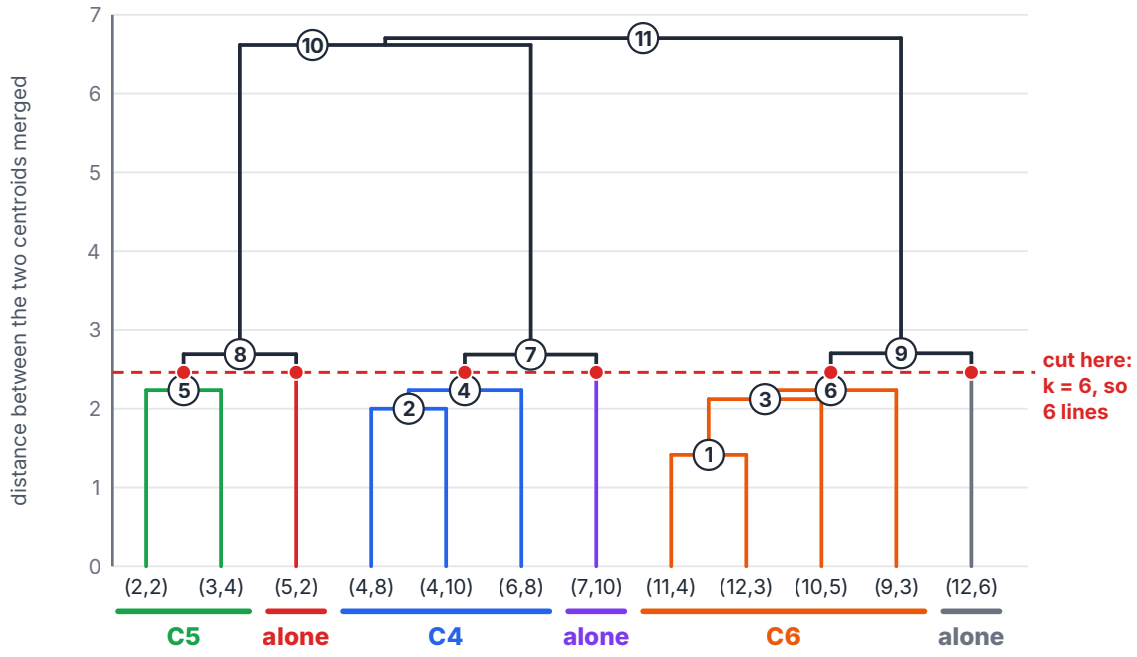


Figure L5-Q2-2. Dendrogram of the complete grouping with merge distances to scale. Leaf order follows slide 75. Coloured bars = merges M1 to M6 (kept for k = 6); dark bars = M7 to M11 (the rest of the tree on slide 75). The red dashed line is the k = 6 cut (at d = 2.46).

Beyond k = 6 (not asked, but drawn on slide 75), carrying the same centroid method on with unique closest pairs gives (Cn = the cluster created by merge Mn):

Merge	Clusters combined (centroids)	d between centroids	New cluster	New centroid	Clusters left
M7	(7,10) + C4 (4.67, 8.67)	2.687	C7 = 4 points	(5.25, 9)	5
M8	(5,2) + C5 (2.5, 3)	2.693	C8 = 3 points	(3.33, 2.67)	4
M9	(12,6) + C6 (10.5, 3.75)	2.704	C9 = 5 points	(10.8, 4.2)	3
M10	C7 (5.25, 9) + C8 (3.33, 2.67)	6.617	C10 = 7 points	(4.43, 6.29)	2
M11	C9 (10.8, 4.2) + C10 (4.43, 6.29)	6.704	C11 = 12 points	(7.08, 5.42)	1

Slide 75's tree has exactly this shape, with the leaves in the same left-to-right order (2,2), (3,4), (5,2), (4,8), (4,10), (6,8), (7,10), (11,4), (12,3), (10,5), (9,3), (12,6). Left half: (2,2) and (3,4) join, then (5,2) joins them; (4,8) and (4,10) join, then (6,8), then (7,10) join them; these two groups meet at M10. Right half: (11,4) and (12,3) join, then (10,5), then (9,3), then (12,6). The left and right halves meet last (M11, the root). Merge distances only go up as you climb the tree (1.414, 2, 2.121, 2.236, 2.236, 2.236, 2.687, 2.693, 2.704, 6.617, 6.704), so there are no "inversions".

> **KEY IDEA: READING THE HEIGHTS**
> The merges M1 to M9 all happen at small distances (1.41 to 2.70), while M10 and M11 happen at about 6.6 and 6.7. Beyond the slides: such a big jump in merge distance is the usual hint for where to cut a dendrogram, in the same spirit as looking for the "right k" in k-means (slide 31). Here the jump points to k = 3: the groups {(7,10), (4,8), (4,10), (6,8)}, {(5,2), (2,2), (3,4)} and {(12,6), (9,3), (10,5), (11,4), (12,3)}. This quiz asks for k = 6, which simply means that the three stragglers (7,10), (5,2) and (12,6) have not yet been absorbed into their groups (that would be M7 to M9). Also beyond the slides: with centroid distances the heights of a tree can in general go *down* at some merge (an inversion); that does not happen for this data.

14. **Ties: why the answer is not unique, and what to write in the exam (Lecture 5 · Slides 71–74).** Ties occurred several times (M1: two pairs at √2; M2: two pairs at 2; M4 to M6: up to five pairs at √5), and the slides break them "arbitrarily". We tried every possible way of breaking the ties by computer. There are exactly **four** different sets of six clusters, and the slide's answer is one of them:

#	The six clusters	What differs
1	**{(4,8),(4,10),(6,8)}, {(7,10)}, {(2,2),(3,4)}, {(5,2)}, {(9,3), (10,5),(11,4),(12,3)}, {(12,6)}**	**the slide's answer: (6,8) joins (4,9); (9,3) joins the cluster at (11,4)**
2	{(4,8),(4,10),(6,8)}, {(7,10)}, {(2,2),(3,4)}, {(5,2)}, {(10,5), (11,4),(12,3),(12,6)}, {(9,3)}	the 3-point cluster around (11,4) takes (12,6) instead of (9,3)
3	{(4,8),(4,10)}, {(6,8),(7,10)}, {(2,2),(3,4)}, {(5,2)}, {(9,3), (10,5),(11,4),(12,3)}, {(12,6)}	(6,8) joins (7,10) instead of (4,9)
4	{(4,8),(4,10)}, {(6,8),(7,10)}, {(2,2),(3,4)}, {(5,2)}, {(10,5), (11,4),(12,3),(12,6)}, {(9,3)}	both of the alternatives above

The four results come from two independent √5 choices: does (6,8) join (4,9) or (7,10), and does the three-point cluster around (11,4) take (9,3) or (12,6)? The left group {(2,2),(3,4)} and the single point (5,2) are the same in all four. The tie at M1 changes nothing (all four results stay reachable); the tie at M2 matters only a little: if (4,8) + (6,8) were merged first, (4,10) would always end up in the same cluster, and results 3 and 4 would become impossible.

EXAM TIP

When two distances tie, say so ("(6,8) is √5 from both (7,10) and (4,9); I break the tie arbitrarily by ...") and continue consistently. Making the same choices as slides 71 to 74 is the safest way to match the marking key. Also remember that the naive method (recompute all pairwise distances at every step) costs $O(N^3)$ and is too expensive for really big data (slide 21); for these 12 points the six rounds already scan 66 + 55 + 45 + 36 + 28 + 21 = 251 pair distances.

SLIDE ERRATUM (OMISSIONS ONLY; NO NUMBER ON THE SLIDES IS WRONG)

(1) Slide 72 says the two closest clusters are (4,8) and (4,10); that is a correct minimum (d = 2), but (4,8) and (6,8) are also exactly 2 apart, which the slide does not say. (2) Slides 73 and 74 present three merges as if they all happened at once at √5. In fact **five** pairs are tied at √5, the merges happen one after the other (each changes a centroid), and two tied pairs, (6,8)–(7,10) and (12,6)–C3, are simply not chosen; other choices give the other three valid answers in the table above. (3) Slide 74 shows rounded centroids: (4.7, 8.7) is (14/3, 26/3) and (10.5, 3.8) is (10.5, 3.75). Every distance and centroid printed on slides 70 to 75 was recomputed in exact fractions and is correct.

PRACTICE (SELF-MADE, NOT FROM THE SLIDES)

- Continue the same run to k = 3 (three more merges, M7 to M9). Answer: {(7,10), (4,8), (4,10), (6,8)} with centroid (5.25, 9); {(5,2), (2,2), (3,4)} with centroid (10/3, 8/3) ≈ (3.33, 2.67); {(12,6), (9,3), (10,5), (11,4), (12,3)} with centroid (10.8, 4.2). The merge distances are 2.687, 2.693 and 2.704.

Final answer: k = 6 is reached after 6 merges. Merges (distance between centroids): M1 (11,4) + (12,3) at √2 → centroid (11.5, 3.5); M2 (4,8) + (4,10) at 2 → (4, 9); M3 (10,5) + {(11,4),(12,3)} at √4.5 ≈ 2.121 → (11, 4); M4 (6,8) + {(4,8),(4,10)} at √5 → (14/3, 26/3) ≈ (4.67, 8.67); M5 (2,2) + (3,4) at √5 → (2.5, 3); M6 (9,3) + {(10,5),(11,4),(12,3)} at √5 → (10.5, 3.75). The six clusters: {(4,8), (4,10), (6,8)}; {(7,10)}; {(2,2), (3,4)}; {(5,2)}; {(9,3), (10,5), (11,4), (12,3)}; {(12,6)}.

Where the marks come from: (1) name the method: agglomerative = bottom-up, cluster = centroid (average of its points), cluster distance = Euclidean distance between centroids, and k = 6 means stop with 6 clusters (12 - 6 = 6 merges); (2) read all 12 points correctly and start with 12 single-point clusters; (3) find the closest pair by computing distances, note the tie at √2, and merge (11,4) with (12,3) into (11.5, 3.5); (4) merge (4,8) with (4,10) into (4, 9); (5) merge (10,5) with the centroid (11.5, 3.5) (d = √4.5) into (11, 4), using a weighted average; (6) the √5 round: (6,8) joins (4,9) giving (4.67, 8.67), (2,2) joins (3,4) giving (2.5, 3), and (9,3) joins (11,4) giving (10.5, 3.75); (7) stop at 6 clusters and list them with members and centroids (3 + 1 + 2 + 1 + 4 + 1 = 12 points); (8) the dendrogram with the cut at k = 6 (slide 75); (9) a remark that ties were broken arbitrarily, so other answers are possible.

Slide check: matches the slides' answer. The twelve points, all merges, all distances (√2, 2, √4.5, √5), all centroids ((11.5, 3.5), (4, 9), (11, 4), (4.7, 8.7), (2.5, 3), (10.5, 3.8, which is 3.75 rounded)), the six clusters on slide 74 and the tree on slide 75 were recomputed in exact arithmetic and agree. Slide erratum (omissions only, no wrong number): the second tie on slide 72 ((4,8)–(6,8) is also at distance 2), the five (not three) pairs tied at √5 on slides 73–74, and the fact that other tie-breaks give other valid 6-cluster answers (four in total); see the box in the last step.

QUICK RECAP — PART 1

- Clustering = group points so members are similar and different clusters are dissimilar, under a distance measure (Euclidean, cosine, Jaccard, edit); outliers belong to no cluster (Slides 4–6).
- It is hard because clusters differ in size, density and shape, overlap, and in high dimensions many pairs of points are at about the same distance (Slides 8–9). Documents are clustered as 0/1 word vectors (Slide 12).
- Two families: hierarchical (agglomerative bottom-up or divisive top-down, output a dendrogram) and point assignment (Slide 14).
- Agglomerative: repeatedly merge the two nearest clusters; Euclidean: cluster = centroid (average), nearness = distance of centroids, new centroid = weighted average. Non-Euclidean: clustroid = actual point "closest" to the others (Slides 16–18).
- Naïve cost $O(N^3)$ (priority queue $O(N^2 \log N)$, beyond the slides): too expensive for big data (Slide 21).

Next: from a tree of merges to k-means

Lecture 5 · Slide 22

Slide 22 is a black section divider titled *k-means clustering*; it carries no content and no number. It marks the switch from hierarchical clustering to the point-assignment family of Slide 14: you fix the number of clusters k in advance, keep k "centre" points and assign every point to the nearest centre. That material is the next part of this chapter.

k-means: point-assignment clustering

Lecture 5 · Slides 22–27

Slide 22 is a black divider titled "*k*-means clustering". It marks a change of family. The hierarchical methods you just met start with every point alone and merge clusters step by step. **Point-assignment** methods work differently: you decide *in advance* how many clusters you want, put that many "centre" points on the map, attach every data point to a centre, and keep adjusting the centres until nothing changes. *k*-means is the standard example, and the building block that CURE and BFR extend later in this lecture.

BEGINNER INTUITION

Think of opening *k* pizza shops in a city. Every household orders from the nearest shop. After a week each shop manager notices that the customers are not centred on the shop, so the shop is moved to the middle of its own customers. Now some households are closer to a different shop, so they switch. Repeat until nobody switches. The shops are the **centroids**, the households are the **data points**.

Setup: Euclidean space, choosing k, choosing the seeds

Lecture 5 · Slide 23

Slide 23 lists three things *k*-means needs before it can start:

1. **Euclidean space and distance.** Points must have coordinates so that "the middle of a group of points" (their average) exists and distance is the straight-line distance.
2. **The number *k* of clusters**, picked by you. (How to choose it well is the topic of Slides 31–34, below.)
3. **One starting point per cluster**, called a **seed**. Slide 23 gives two ways: pick *k* points **at random**, or pick **one point at random, then *k*−1 other points, each as far away as possible from the previous points**.

DEFINITIONS

Euclidean distance between $p = (x_1, y_1)$ and $q = (x_2, y_2)$: $d(p, q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$; in more dimensions add one squared difference per coordinate.

Centroid of a cluster: the coordinate-wise average (mean) of its points. It is generally *not* one of the data points.

Seed (initial centroid): the starting position of a centroid; with the methods of Slide 23 each seed is one of the data points.

Why "Euclidean" matters. The slide only says "assumes Euclidean", but the reason is the centroid: an average of points exists only where points can be added and divided by a count. For non-Euclidean data (for example sets or strings) there is no average; Slide 18 handled that for hierarchical clustering with a *clustroid*, an actual data point chosen to represent the cluster.

Spreading the seeds out (farthest-first)

Lecture 5 · Slide 23

Random seeds can land in the same natural group, which wastes a centroid. The second recipe avoids this. Slide 23 says "as far away as possible from the previous points".

FARTHEST-FIRST RULE, MADE PRECISE (BEYOND THE SLIDES:)

Seed 1: any point (random, or given). To choose each further seed: for every candidate point compute its distance to *each* seed already chosen and keep the **smallest** of those distances (its "score": how close it is to the nearest existing seed). Pick the candidate with the **largest score**. This is the same rule that Quiz L5-Q3 below uses and the one the answer on Slide 77 follows.

Worked example — farthest-first on six points in one dimension Practice, Slide 23 rule

Points on a line: 1, 2, 3, 6, 9, 10. Suppose seed 1 is the point 1 and $k = 3$.

Step	Candidates and their score (distance to nearest chosen seed)	Choice
Seed 2	2: 1 3: 2 6: 5 9: 8 **10: 9**	10
Seed 3	2: min(1, 8) = 1 3: min(2, 7) = 2 **6: min(5, 4) = 4** 9: min(8, 1) = 1	6

The seeds 1, 6, 10 are spread over the line. Choosing 9 as seed 3 (largest distance to seed 1 only) would be wrong: it sits next to seed 10.

COMMON MISTAKE

With $k > 2$, do not measure "far away" from the *last* seed only, and do not add distances. Use the smaller of the distances to all chosen seeds, then take the largest.

SLIDE ERRATUM (OMISSIONS)

Slide 23 is titled "*k*-means Algorithm(s)" in the plural but shows one algorithm with two initialisation options. It also never says that the final clusters depend on the seeds. **Beyond the slides:** *k*-means always stops, but it can stop at a **local optimum**, a stable clustering that is not the best possible one. Different seeds can give different answers; that is why spread-out seeds (or several runs) are preferred.

The loop: assign, update, reassign

Lecture 5 · Slide 24

Slide 24 ("Populating Clusters") is the whole algorithm. Note that the slide numbers its steps 1, 2, 3 and then says "repeat 2 and 3". Steps 1 and 3 are the *same operation* (send each point to its nearest centroid); step 1 uses the seeds, step 3 uses the updated centroids.

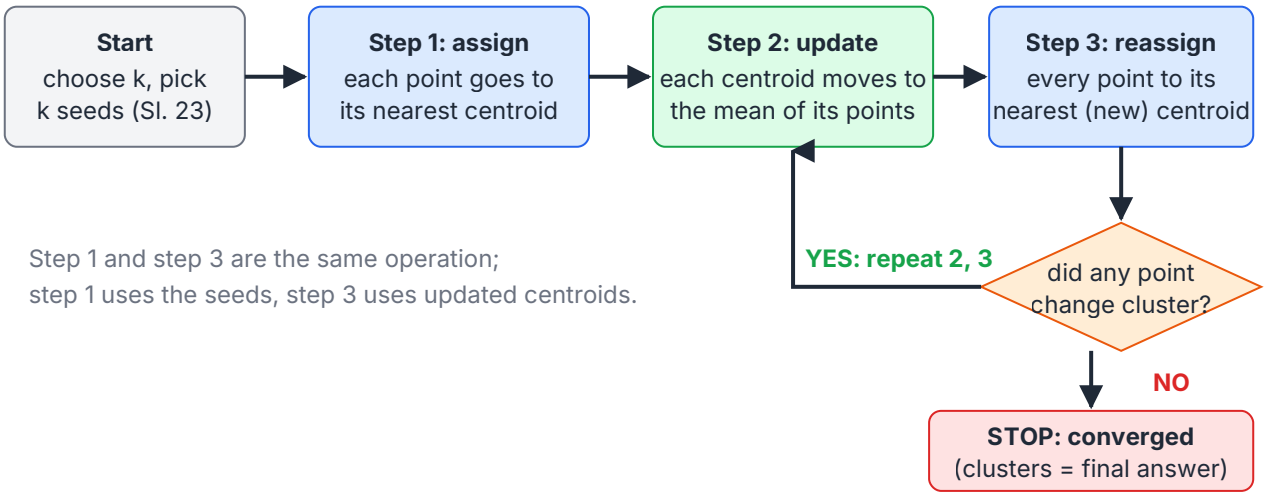


Figure 1 — The *k*-means loop (Slide 24).

1. **Assign.** For each point, find the centroid it is nearest to and place the point in that cluster.
2. **Update.** After *all* points are assigned, move each of the *k* centroids to the mean of the points now in its cluster.
3. **Reassign.** Send all points to their closest (new) centroid. The slide notes this "sometimes moves points between clusters": a point near a boundary can be captured by the other centroid once the centroids have moved.
4. **Repeat 2 and 3 until convergence.**

DEFINITION: CONVERGENCE (SLIDE 24)

Convergence means that **points do not move between clusters and the centroids stabilise**. The two conditions go together: if no point changes cluster, the means cannot change, and if the means do not change, no point will change cluster.

Worked example — the loop on six points, with deliberately bad seeds Practice, Slide 24 loop

Points 1, 2, 3, 6, 9, 10 (one dimension, so distance = |difference|). k = 2, seeds 1 and 2.

Stage	Centroids	Clusters	What happens
1. Assign to seeds	1 and 2	{1} and {2, 3, 6, 9, 10}	3 is 1 away from 2 but 2 away from 1, and so on: seed 2 grabs almost everything.
2. Update	1 and 6		mean of {2, 3, 6, 9, 10} = 30/5 = 6
3. Reassign	1 and 6	{1, 2, 3} and {6, 9, 10}	2: distance 1 to centroid 1, 4 to centroid 6, so it moves. 3: 2 against 3, so it moves too.
2. Update	2 and 8.33		6/3 = 2 and 25/3 = 8.33
3. Reassign	2 and 8.33	{1, 2, 3} and {6, 9, 10}	nobody moves (for example 6: 4 from 2 but 2.33 from 8.33)

Converged: clusters {1, 2, 3} and {6, 9, 10}. Starting from the spread-out seeds 1 and 10 (see the farthest-first rule) the very first assignment already gives this answer, so good seeds save rounds. Both runs were checked by computer.

EXAM TIP

For every point write the distance to *every* centroid (compare squared distances if you like; the square root never changes which is smallest) and underline the smallest. Then compute each new centroid as $(\sum x/n, \sum y/n)$ with the count n written out. State clearly which round your answer refers to. Quiz L5-Q3 below does exactly this.

NOT EXAMINED — WHY IT ALWAYS STOPS (BEYOND THE SLIDES:)

Each assign step and each update step can only keep or reduce the sum of squared distances from points to their centroid, and there are finitely many ways to split the points, so the loop must stop. The slide itself states no iteration cap, tie-breaking rule or proof.

The three-snapshot picture

Lecture 5 · Slides 25–27

Slides 25, 26 and 27 show one data set of 11 points (marked "x") in three snapshots, captioned "Clusters after round 1", "Clusters after round 2" and "Clusters at the end". The legend marks data points as "x" and centroids as hollow squares; the shaded ovals show which points belong together. The slides do not print *k*, but there are two centroids, so *k* = 2. The points form an "L": six lie along a flat row on the left (P1–P6), five climb diagonally to the right (P7–P11).



Figure 2 — Slides 25–27 redrawn: the same 11 points, k = 2, in three snapshots.

- **Slide 25, round 1.** The seeds sit on two data points, P1 and P7. Each point joins the nearer seed: P1–P3 are pink, P4–P11 are blue. The split is lopsided (3 against 8), and neither seed is in the middle of its group, so the algorithm must continue.
- **Slide 26, round 2.** The centroids have slid to the middle of their groups: pink moved right, blue moved up and to the right. Because of that, P4 (the point at the junction) is now closer to pink and switches. This is the sentence "sometimes moves points between clusters" on Slide 24 in action.
- **Slide 27, at the end.** The centroids moved again and two more junction points, P5 and P6, joined pink. Final clusters: pink {P1–P6}, blue {P7–P11}. When no point changes cluster in a pass, the algorithm has converged (Slide 24).

Two lessons: points near the boundary can migrate over several rounds, and the final split (flat group, diagonal group) is the natural one even though the seeds were poor.

SLIDE ERRATUM (MINOR, FIGURE ONLY)

The legend square on Slides 25–27 is green, but the centroids are drawn pink and blue. More importantly, the squares are drawn only *roughly* at the cluster means, and the slides do not say how many rounds lie between "round 2" and "the end". Do not compute from this figure. (Beyond the slides: measuring the 11 point positions from the picture (P5 is almost equidistant from the two centroids, so this depends on reading accuracy) and running *k*-means exactly from the same two seeds gives the round-1 and round-2 clusters shown above but then *stops* with {P1–P4} and {P5–P11}, which is also a stable state. The "at the end" picture is a different stable state with a smaller total squared distance. This is a small example of the local-optimum remark above: the end result depends on the exact centroid positions.)

QUICK RECAP: *K*-MEANS

- Needs Euclidean points, a chosen *k*, and *k* seeds (random, or farthest-first).
- Loop: assign each point to its nearest centroid; recompute each centroid as the mean of its points; reassign; repeat.
- Stops when no point changes cluster and the centroids stabilise.
- Centroid = mean, so it may not be a data point. Result can depend on the seeds.

Try it yourself first. Cover the solution and choose the two farthest seeds for the twelve points below, then carry out two rounds of the loop on paper.

Quiz L5-Q3 **k-means on 12 points (k = 3): farthest-point start, then two rounds** Lecture 5 · Slides 76–77

QUESTION (AS ON THE SLIDES)

Let us consider the twelve points presented below. K-Means clustering algorithm for $k = 3$ is used. Assume the initial choice of point is (6,8). What are the other two initial points? Pick points that are as far away as possible. Provide result of K-Means alg. after 2nd round.

Data (slide 76 draws the twelve points as a scatter plot with each point's coordinates printed beside it; read from that figure): `(4,10)` `(7,10)` `(4,8)` `(6,8)` `(12,6)` `(10,5)` `(11,4)` `(3,4)` `(9,3)` `(12,3)` `(2,2)` `(5,2)`. Distances are Euclidean.

(a) The initial choice of point is (6,8). What are the other two initial points? Pick points that are as far away as possible.

(b) Provide the result of K-Means after the 2nd round.

Slide 77 prints the answer to (a) only and ends with "Note: compute the result of K-Means after 2nd round on your own." So part (b) is solved in full below and carried on to convergence.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION — K-MEANS AS THREE PIZZA SHOPS

Think of the 12 points as 12 houses and the 3 centroids as 3 pizza shops. **Start:** open the three shops at three starting spots (here: at three of the houses). **Assign:** every house orders from its nearest shop. **Update:** every shop moves to the average position of the houses it serves. **Reassign:** every house re-checks which shop is now nearest. Repeat until nobody switches shop and the shops stop moving. Part (a) asks *where to open the shops at the start* (far apart from each other); part (b) asks *what the map looks like after the second round*.

FORMULAS USED (LECTURE 5 · SLIDES 23–24)

Euclidean distance between $p = (x_1, y_1)$ and $q = (x_2, y_2)$: $d(p, q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

Shortcut: to decide *which* centroid is nearest it is enough to compare the squared distances d^2 (the square root never changes which number is smallest); take $\sqrt{}$ only when you report a distance. In the tables below each cell shows d^2 and, in brackets, d.

Centroid of a cluster of n points = coordinate-wise mean = $(\sum x / n, \sum y / n)$.

k-means loop (slide 24): (1) place each point in the cluster whose current centroid is nearest; (2) after all points are assigned, update the centroids of the k clusters; (3) reassign all points to their closest centroid; repeat 2 and 3 until **convergence** = points do not move between clusters and the centroids stabilise.

1. **[Part (a): choosing the two other initial points] Idea (Lecture 5 · Slide 23):** "Pick one point at random, then $k-1$ other points, each as far away as possible from the previous points." The first point (6,8) is given. When more than one point is already chosen, "as far away as possible from the previous points" means: for every candidate compute its distance to *each* already-chosen point, keep the **smaller** of these (the distance to its nearest chosen point; slide 77 calls it the candidate's "score"), and pick the candidate with the **largest** score. (This is the farthest-point or "max–min" rule.) We apply it twice: first for seed 2 (only one chosen point so far, so the score is just the distance to (6,8)) and then for seed 3.

2. **Seed 2: the point farthest from (6,8).** Compute $d^2 = \Delta x^2 + \Delta y^2$ from (6,8) to each of the other 11 points (table sorted from the largest d^2):

Candidate point	$\Delta x^2 + \Delta y^2$ from (6,8)	d^2	$d = \sqrt{d^2}$
(12,3)	**$6^2 + (-5)^2$**	**61**	**7.81**
(2,2)	$(-4)^2 + (-6)^2$	52	7.21
(11,4)	$5^2 + (-4)^2$	41	6.40
(12,6)	$6^2 + (-2)^2$	40	6.32
(5,2)	$(-1)^2 + (-6)^2$	37	6.08
(9,3)	$3^2 + (-5)^2$	34	5.83
(10,5)	$4^2 + (-3)^2$	25	5.00
(3,4)	$(-3)^2 + (-4)^2$	25	5.00
(4,10)	$(-2)^2 + 2^2$	8	2.83
(7,10)	$1^2 + 2^2$	5	2.24
(4,8)	$(-2)^2 + 0^2$	4	2.00

   The largest is (12,3): $d^2 = 6^2 + (-5)^2 = 36 + 25 = 61$, so $d = \sqrt{61} = 7.81$. The runner-up is (2,2) with $d^2 = 52$ ($d = \sqrt{52} = 7.21$), which is smaller, so there is no tie. **Seed 2 = (12,3)** (matches slide 77).

3. **Seed 3: the candidate whose *nearest* chosen seed is farthest away.** Now two points are chosen, (6,8) and (12,3). For each of the remaining 10 points compute d^2 to (6,8) and d^2 to (12,3); the score is the smaller of the two (table sorted by score):

Candidate	d^2 to (6,8)	d^2 to (12,3)	score^2 = smaller d^2	nearest chosen seed	score = √
(2,2)	**52**	**101**	**52**	**(6,8)**	**7.21**
(5,2)	37	50	37	(6,8)	6.08
(3,4)	25	82	25	(6,8)	5.00
(12,6)	40	9	9	(12,3)	3.00
(9,3)	34	9	9	(12,3)	3.00
(4,10)	8	113	8	(6,8)	2.83
(10,5)	25	8	8	(12,3)	2.83
(7,10)	5	74	5	(6,8)	2.24
(4,8)	4	89	4	(6,8)	2.00
(11,4)	41	2	2	(12,3)	1.41

   Worked line for (2,2): d^2 to (6,8) $= (-4)^2 + (-6)^2 = 16 + 36 = 52$, so $d = \sqrt{52} = 7.21$; d^2 to (12,3) $= (-10)^2 + (-1)^2 = 100 + 1 = 101$, so $d = \sqrt{101} = 10.05$. The smaller is 52, so its score is $\sqrt{52} = 7.21$. Every other candidate has a smaller score (the best of the rest is (5,2) with $\sqrt{37} = 6.08$, then (3,4) with 5.00), so **seed 3 = (2,2)**. This is exactly the claim on slide 77 that any other point is closer than 7.21 to at least one of (6,8) and (12,3): Figure 1 shows it as two discs of radius 7.21 that together cover every remaining point except (2,2), which lies exactly on the edge of the first disc.

4. **Starting centroids (part (a) is now complete):** c_1 = (6,8) (given), c_2 = (12,3), c_3 = (2,2).

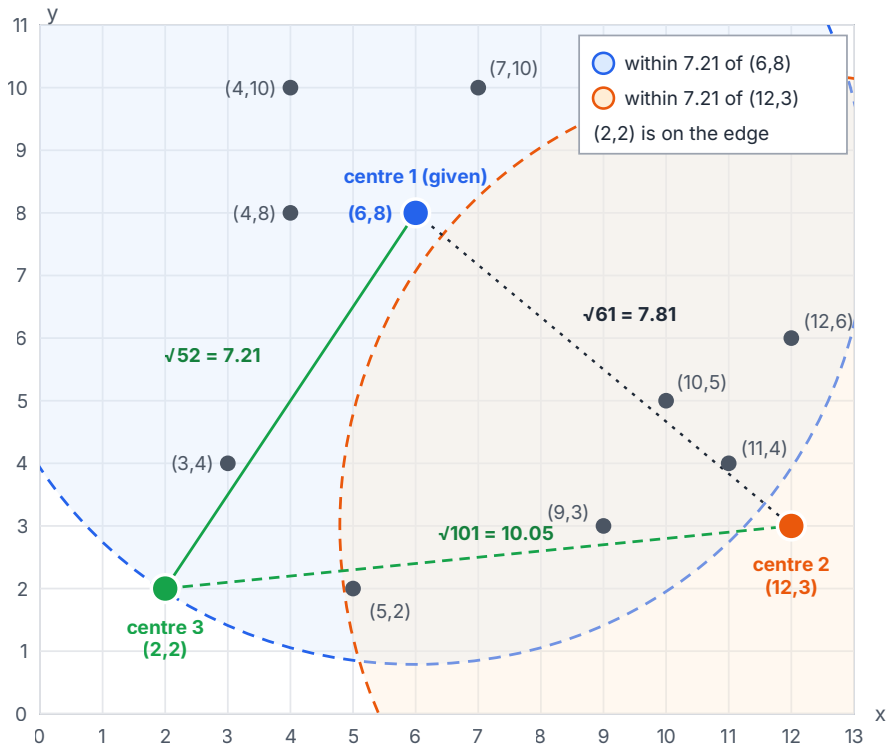


Figure 1 — Choosing the starting points. Seed 1 (6,8) is given; seed 2 (12,3) is the point farthest from it (√61 = 7.81); seed 3 (2,2) has score min(√52, √101) = 7.21. Both dashed discs have radius √52 = 7.21: every other point lies inside at least one of them (score below 7.21), but (2,2) sits exactly on the edge of the blue disc and far outside the orange one, so it has the largest score.

> **BEGINNER INTUITION — WHY THE *SMALLER* OF THE TWO DISTANCES?**
> Imagine placing three coffee shops so that they do not compete. The first is at (6,8); the second goes as far from it as possible, at (12,3). The third must be far from *both*. A site like (4,10) is √113 = 10.63 from the shop at (12,3) but only √8 = 2.83 from the shop at (6,8): it would sit right next to a shop that already exists, so its score is only 2.83. Taking the *smaller* distance catches this; looking at one shop only would not. (For this data, adding the two distances, 7.21 + 10.05 = 17.26, would also pick (2,2), but the rule on slide 77 is the smaller-distance score.)

5. **[Part (b): the k-means rounds] Round 1: assign every point to its nearest seed.** (Counting convention: "round 1" = assign every point to the nearest initial centroid and then recompute the centroids; "round 2" = reassign every point to the nearest new centroid and recompute again. The slides label the pictures "Clusters after round 1 / round 2 / at the end" (Slides 25–27) without defining a round more precisely. The answer is the same under every reading, see the Exam tip below.) For each point compute d^2 to c_1 = (6,8), c_2 = (12,3), c_3 = (2,2). Worked example for (4,10): to (6,8): $(-2)^2 + 2^2 = 4 + 4 = 8$; to (12,3): $(-8)^2 + 7^2 = 64 + 49 = 113$; to (2,2): $2^2 + 8^2 = 4 + 64 = 68$; the smallest is 8, so (4,10) joins C_1. Full table (smallest d^2 in each row highlighted; no ties occur):

Point	d^2 (d) to c_1 = (6,8)	d^2 (d) to c_2 = (12,3)	d^2 (d) to c_3 = (2,2)	Nearest
(4,10)	**8 (2.83)**	113 (10.63)	68 (8.25)	**c_1**
(7,10)	**5 (2.24)**	74 (8.60)	89 (9.43)	**c_1**
(4,8)	**4 (2.00)**	89 (9.43)	40 (6.32)	**c_1**
(6,8)	**0 (0.00)**	61 (7.81)	52 (7.21)	**c_1**
(12,6)	40 (6.32)	**9 (3.00)**	116 (10.77)	**c_2**
(10,5)	25 (5.00)	**8 (2.83)**	73 (8.54)	**c_2**
(11,4)	41 (6.40)	**2 (1.41)**	85 (9.22)	**c_2**
(3,4)	25 (5.00)	82 (9.06)	**5 (2.24)**	**c_3**
(9,3)	34 (5.83)	**9 (3.00)**	50 (7.07)	**c_2**
(12,3)	61 (7.81)	**0 (0.00)**	101 (10.05)	**c_2**
(2,2)	52 (7.21)	101 (10.05)	**0 (0.00)**	**c_3**
(5,2)	37 (6.08)	50 (7.07)	**9 (3.00)**	**c_3**

Clusters after round 1: C_1 = {(4,10), (7,10), (4,8), (6,8)} (4 points); C_2 = {(12,6), (10,5), (11,4), (9,3), (12,3)} (5 points); C_3 = {(3,4), (2,2), (5,2)} (3 points). Check: 4 + 5 + 3 = 12, so every point is in exactly one cluster. The closest call is (10,5): $\sqrt{8} = 2.83$ from c_2 against $\sqrt{25} = 5.00$ from c_1.

6. **Update the centroids** (mean of each cluster; slide 24, step 2): the starting points were only a first guess, so every centroid moves to the average of the points that now belong to it.

Cluster	Members	$\sum x$	$\sum y$	n	New centroid ($\sum x$ / n, $\sum y$ / n)
C_1	(4,10), (7,10), (4,8), (6,8)	4+7+4+6 = 21	10+10+8+8 = 36	4	**(21 / 4, 36 / 4) = (5.25, 9)**
C_2	(12,6), (10,5), (11,4), (9,3), (12,3)	12+10+11+9+12 = 54	6+5+4+3+3 = 21	5	**(54 / 5, 21 / 5) = (10.8, 4.2)**
C_3	(3,4), (2,2), (5,2)	3+2+5 = 10	4+2+2 = 8	3	**(10 / 3, 8 / 3) = (3.33, 2.67)**

The centroids moved from the seeds (6,8), (12,3), (2,2) to **(5.25, 9), (10.8, 4.2), (3.33, 2.67)**, exactly (21/4, 9), (54/5, 21/5), (10/3, 8/3). A centroid is an average, so it need not coincide with any data point.

7. **Round 2: reassign every point to its nearest *new* centroid** (slide 24, step 3). Worked example for (9,3) (the closest call): to c_1 = (5.25, 9): $(9 - 5.25)^2 + (3 - 9)^2 = 14.06 + 36 = 50.06$, d = 7.08; to c_2 = (10.8, 4.2): $(9 - 10.8)^2 + (3 - 4.2)^2 = 3.24 + 1.44 = 4.68$, d = 2.16; to c_3 = (10/3, 8/3): $(9 - 10/3)^2 + (3 - 8/3)^2 = 289/9 + 1/9 = 290/9 = 32.22$, d = 5.68. The smallest is 4.68, so (9,3) stays in C_2. All twelve points (centroids used with their exact fractional values; nearest highlighted):

Point	d^2 (d) to c_1 = (5.25, 9)	d^2 (d) to c_2 = (10.8, 4.2)	d^2 (d) to c_3 = (3.33, 2.67)	Nearest	Was in	Moved?
(4,10)	**2.56 (1.60)**	79.88 (8.94)	54.22 (7.36)	**c_1**	C_1	no
(7,10)	**4.06 (2.02)**	48.08 (6.93)	67.22 (8.20)	**c_1**	C_1	no
(4,8)	**2.56 (1.60)**	60.68 (7.79)	28.89 (5.37)	**c_1**	C_1	no
(6,8)	**1.56 (1.25)**	37.48 (6.12)	35.56 (5.96)	**c_1**	C_1	no
(12,6)	54.56 (7.39)	**4.68 (2.16)**	86.22 (9.29)	**c_2**	C_2	no
(10,5)	38.56 (6.21)	**1.28 (1.13)**	49.89 (7.06)	**c_2**	C_2	no
(11,4)	58.06 (7.62)	**0.08 (0.28)**	60.56 (7.78)	**c_2**	C_2	no
(3,4)	30.06 (5.48)	60.88 (7.80)	**1.89 (1.37)**	**c_3**	C_3	no
(9,3)	50.06 (7.08)	**4.68 (2.16)**	32.22 (5.68)	**c_2**	C_2	no
(12,3)	81.56 (9.03)	**2.88 (1.70)**	75.22 (8.67)	**c_2**	C_2	no
(2,2)	59.56 (7.72)	82.28 (9.07)	**2.22 (1.49)**	**c_3**	C_3	no
(5,2)	49.06 (7.00)	38.48 (6.20)	**3.22 (1.80)**	**c_3**	C_3	no

No point changes cluster in round 2 ("Moved?" is "no" in all 12 rows). The smallest difference between the nearest and the second-nearest centroid is about 3.5 (for (9,3): 2.16 against 5.68), so no reassignment was even close. Hence the clusters after round 2 are the same as after round 1.

8. **Update the centroids again (end of round 2).** Because the clusters are unchanged, the sums are unchanged: C_1: (21/4, 36/4) = (5.25, 9); C_2: (54/5, 21/5) = (10.8, 4.2); C_3: (10/3, 8/3) = (3.33, 2.67). The centroids did not move.
9. **Convergence test (slide 24):** "Points do not move between clusters and centroids stabilize." Both conditions hold after round 2 (0 points moved, 0 centroids moved), so k-means stops. A 3rd round would use the same centroids, compute the same distances and give the same clusters. Hence the result after round 2 is also the **final** result.
10. **Summary of the run:**

Stage	Centroids used	Clusters formed	Points that moved
Round 1: assign to the seeds	(6,8), (12,3), (2,2)	C_1 = {(4,10), (7,10), (4,8), (6,8)} C_2 = {(12,6), (10,5), (11,4), (9,3), (12,3)} C_3 = {(3,4), (2,2), (5,2)}	— (first assignment)
Update after round 1	new centroids (5.25, 9), (10.8, 4.2), (3.33, 2.67)		
Round 2: reassign	(5.25, 9), (10.8, 4.2), (3.33, 2.67)	C_1, C_2, C_3 exactly as above	**0**
Update after round 2	centroids (5.25, 9), (10.8, 4.2), (3.33, 2.67): unchanged, so the algorithm has converged		

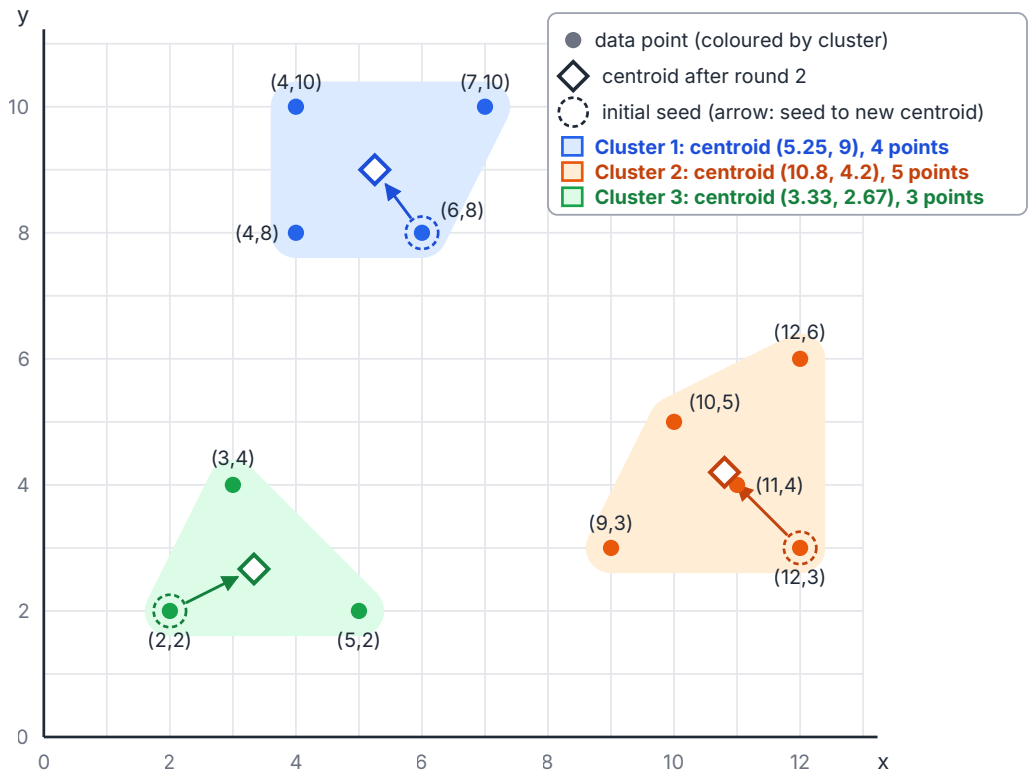


Figure 2 — Clusters after round 2 (also the final clusters). Dashed rings mark the initial seeds; arrows show how each centroid moved to the mean of its cluster in the round-1 update; diamonds are the centroids after round 2 (unchanged).

WHY DID IT CONVERGE SO FAST?

The data form three obvious blobs: a top group (y ≥ 8), a right group (x ≥ 9) and a bottom-left group. The farthest-point rule put exactly one seed in each blob, so the first assignment was already the "natural" grouping and the means merely moved to the middle of each blob. With a poor start (for example two seeds in the same blob) k-means usually needs more rounds and may end in a worse clustering. This is why slide 23 asks for seeds that are as far apart as possible.

EXAM TIP — WHAT IS A "ROUND"?

Write a table of distances for **every** point and every centroid (squared distances are fine), mark the smallest in each row, list the resulting clusters, then show ∑x, ∑y and n for each new centroid. State the convergence test in words ("no point moved and the centroids did not change, so stop"). State your round convention in one line. Here it does not matter: whether round 1 means "assign only" or "assign + update", the clusters never change after the first assignment and the centroids never change after the first update, so the state after round 2 is the same under every reading. If a point were ever exactly equidistant from two centroids (it does not happen here), the slides give no rule; say which tie-break you use.

COMMON MISTAKES IN THIS QUESTION

- Choosing seed 3 as the point farthest from the *latest* seed only: that gives (4,10), which is just 2.83 from (6,8). Always take the minimum over *all* chosen seeds, then the maximum of those minima.
- Reusing the *starting* centroids (6,8), (12,3), (2,2) for the round-2 assignment instead of the updated ones (5.25, 9), (10.8, 4.2), (3.33, 2.67).
- Stopping after the assignment and forgetting to recompute the centroids, or taking the centroid to be the "middle" data point instead of the average.
- Saying k-means runs a fixed number of rounds: the stopping rule on slide 24 is convergence (nothing moves), not a round count.
- Rounding too early, for example 3.3 and 2.7 for centroid 3: keep fractions (10/3, 8/3) or at least two decimals.

NOT EXAMINED — OPTIONAL SELF-CHECK (BEYOND THE SLIDES)

Beyond the slides: the standard k-means objective is the sum of squared distances from each point to its own centroid. With the seeds as centres and the round-1 clusters it is 17 + 28 + 14 = 59; with the updated centroids it is 43/4 + 68/5 + 22/3 = 1901/60 = 31.68, and it stays 31.68 in round 2. Every assignment step and every update step can only keep this number the same or lower; if your own value ever goes *up*, you have an arithmetic error. The average distance from a point to its centroid (the quantity slide 31 uses to choose k) is 1.55 for this final clustering. Different starting seeds can end in a different (local) result, which is why the choice of seeds matters.

Final answer:
(a) The other two initial points are (12,3) (farthest from (6,8): √61 = 7.81) and then (2,2) (score = min(√52, √101) = min(7.21, 10.05) = 7.21, the largest). Starting centroids: (6,8), (12,3), (2,2).
(b) After the 2nd round: Cluster 1 = {(4,10), (7,10), (4,8), (6,8)} with centroid (5.25, 9); Cluster 2 = {(12,6), (10,5), (11,4), (9,3), (12,3)} with centroid (10.8, 4.2); Cluster 3 = {(3,4), (2,2), (5,2)} with centroid (3.33, 2.67) = (10/3, 8/3). The clusters and centroids are the same as after round 1: no point moved in round 2 and the centroids are unchanged, so k-means has converged and this is also the final clustering.

Where the marks come from: (1) rule for the starting points stated (first given, each next as far as possible from the already-chosen ones, using the nearest-chosen-point score) and Euclidean distance and centroid-as-mean formulas stated; (2) seed 2 = (12,3), justified as the farthest from (6,8) with d = √61 = 7.81; (3) seed 3 = (2,2), justified with the score min(√52, √101) = 7.21 and the remark that every other point scores less; (4) round 1: all 12 points assigned correctly, with distances shown; (5) centroids after round 1 computed as means: (5.25, 9), (10.8, 4.2), (3.33, 2.67); (6) round 2: all 12 points reassigned using the *new* centroids, with distances shown; (7) clusters and centroids after round 2 stated; (8) convergence criterion stated and applied (no point moved, centroids stable).

Slide check: part (a) matches slide 77 exactly: seeds (6,8), (12,3), (2,2); √52 = 7.21; √101 = 10.05; score 7.21; and its claim that every other point is closer than 7.21 to at least one of (6,8) and (12,3) is correct (the strongest competitor is (5,2) with score √37 = 6.08). Slide 77 gives no answer for part (b) ("compute ... on your own"), so there is nothing to compare with; the clusters, centroids and convergence above were computed from scratch with exact fractions and checked by a program. No slide erratum. Two clarifications: the slide's phrase "the one whose distance to (6,8) and (12,3) is the furthest" means "the largest score", i.e. the largest value of the *smaller* of the two distances, and the slides do not define exactly what one "round" contains (see the Exam tip). The twelve coordinates were read from the labels on the slide-76 figure.

Choosing k

Lecture 5 · Slides 31–34

The printed numbering jumps from slide 27 to slide 31 in the lecture PDF (slides 28–30 are not in the file; nothing is missing from the sequence of ideas). Slide 31, "Getting the *k* right", answers the obvious question: *k*-means needs *k*, but how do you know it?

Average distance to the centroid versus k

Lecture 5 · Slide 31

DEFINITION: AVERAGE DISTANCE TO CENTROID

For a clustering, take each point's distance to the centroid of *its own* cluster and average these distances over all points. It measures how tight the clusters are: small means points sit close to their centre.

The slide's recipe: **try different *k*** and watch how this average changes as *k* grows. It **falls rapidly until the right *k*, then changes little**. The slide's picture is a curve (vertical axis "Average distance to centroid", horizontal axis *k*) with an arrow labelled "Best value of *k*" pointing at the bend.

BEGINNER INTUITION

Adding a centroid always helps a little, because some point gets a closer centre. In the extreme, *k* = number of points gives distance 0, so you cannot just minimise the average. But while you are adding centroids that the data really need, the gain is large. Once every natural group has its own centroid, extra centroids only split groups that were already tight, and the gain is tiny. The bend is where the big gains stop.

Worked example — an elbow with numbers Practice, Slide 31 idea

Nine points on a line forming three tight groups: 1, 2, 3, then 10, 11, 12, then 24, 25, 26. For each *k* the best split into *k* groups (found by trying all ways to cut the sorted list) and its average distance to centroid:

k	Best clusters	Average distance	Drop from the previous *k*
1	all nine points (centroid 12.67)	74 / 9 = 8.22	—
2	{1,2,3,10,11,12} and {24,25,26}	3.22	5.00
3	**{1,2,3} {10,11,12} {24,25,26}**	**6 / 9 = 0.67**	**2.56**
4	{1} {2,3} {10,11,12} {24,25,26}	0.56	0.11
5	{1} {2} {3} {10,11,12} {24,25,26}	0.44	0.11

The drops are 5.00, 2.56, then only 0.11 and 0.11: the curve bends at *k* = 3, matching the three obvious groups.

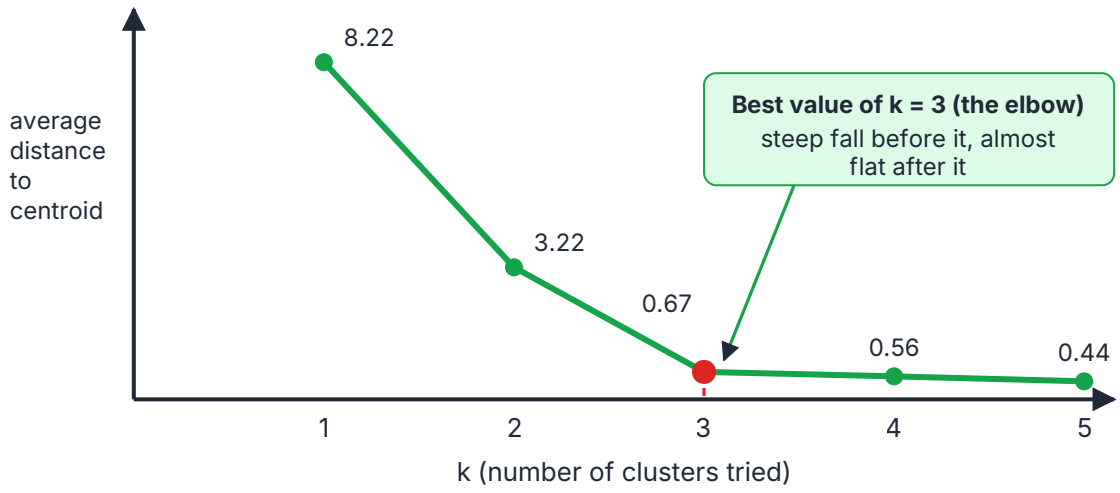


Figure 3 — Average distance to centroid against *k* for the nine-point example; the elbow is at *k* = 3 (idea of Slide 31).

KEY IDEA: THE ELBOW

Beyond the slides: this technique is commonly called the **elbow** (or knee) **method**; the slide does not name it. Its curve is schematic (no numbers on the axes), so the "best *k*" is read off visually as the bend, not computed. "Average" here is the mean distance to the centroid; some books use the sum of squared distances, which usually shows a bend at a similar *k*.

Too few, just right, too many

Lecture 5 · Slides 32–34

The three slides "Example: Picking *k*" use the same 41 points that fall into three natural groups (15 on the left, 15 on the upper right, 11 at the bottom). The short lines inside each cluster are samples of the distances from points to their centroid. The slides never print *k*; count the shapes.

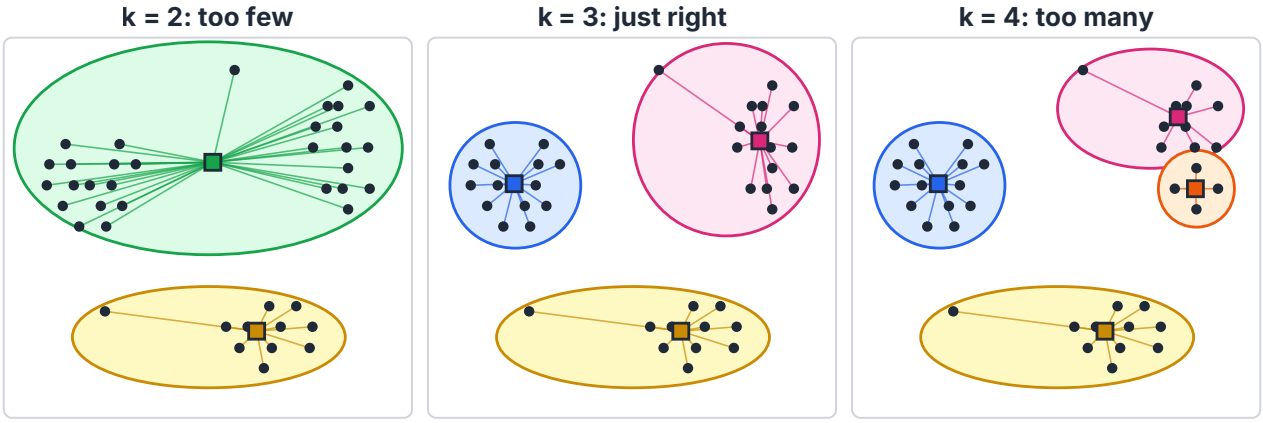


Figure 4 — Slides 32–34 redrawn: too few, just right, too many (41 points, positions copied from the slides).

Slide	*k*	Slide's words	What the picture shows	Position on the Slide 31 curve
32	2	"Too few; many long distances to centroid."	The left and upper-right groups are lumped into one big cluster whose centroid is in empty space between them.	steep part (large average)
33	**3**	**"Just right; distances rather short."**	**One centroid in the middle of each natural group.**	**the elbow**
34	4	"Too many; little improvement in average distance."	The 15-point upper-right group is split into 10 + 5 although it had no gap.	flat part

Measured from the slides' own picture (distances in pixels of the rendered slide, so only the ratios matter): average distance to centroid is about 280 for one cluster, 205 for $k = 2$, 85 for $k = 3$ and 72 for $k = 4$. Going 2 to 3 removes about 120; going 3 to 4 removes only about 13. That is the elbow, numerically.

EXAM TIP

"How would you pick *k*?" Model answer: run *k*-means for $k = 1, 2, 3, \ldots$; plot average distance to centroid against *k*; it drops rapidly until the right *k* and changes little afterwards; choose *k* at the bend. Give the reasons: too few = clusters merge natural groups and distances are long; too many = clusters split tight groups for little gain.

COMMON MISTAKE

Choosing the *k* with the smallest average distance. That is always the largest *k* tried (distance reaches 0 when every point is its own cluster). Look for where the improvement stops, not for the minimum.

The CURE algorithm: clusters of any shape, large data

Lecture 5 · Slides 35–42

Slide 35 is a black divider: "The CURE Algorithm — Extension of *k*-means to clusters of arbitrary shapes and large data". It promises two upgrades over *k*-means: (1) clusters need not be round blobs, and (2) the data may be too large for main memory. The next slides show how.

What CURE is, and why one centroid is not enough

Lecture 5 · Slide 36

(Printed Slide 37 is absent from the PDF; Slide 38 follows Slide 36 directly.)

> **DEFINITION: CURE (CLUSTERING USING REPRESENTATIVES)**
> A clustering algorithm that (Slide 36) **assumes a Euclidean distance**, **allows clusters to assume any shape**, and **uses a collection of representative points to represent clusters**. A **representative** is one of several stand-in points, spread over a cluster, that together describe it.

The picture in the corner of Slide 36 shows two interlocking half-moons of dots, one blue and one red. It is the motivating example: with *one* centroid per cluster the centre of each moon falls in empty space, and the "nearest centroid" boundary slices straight through both moons. Describing each moon by *several* points lying along it lets a point find the right moon.

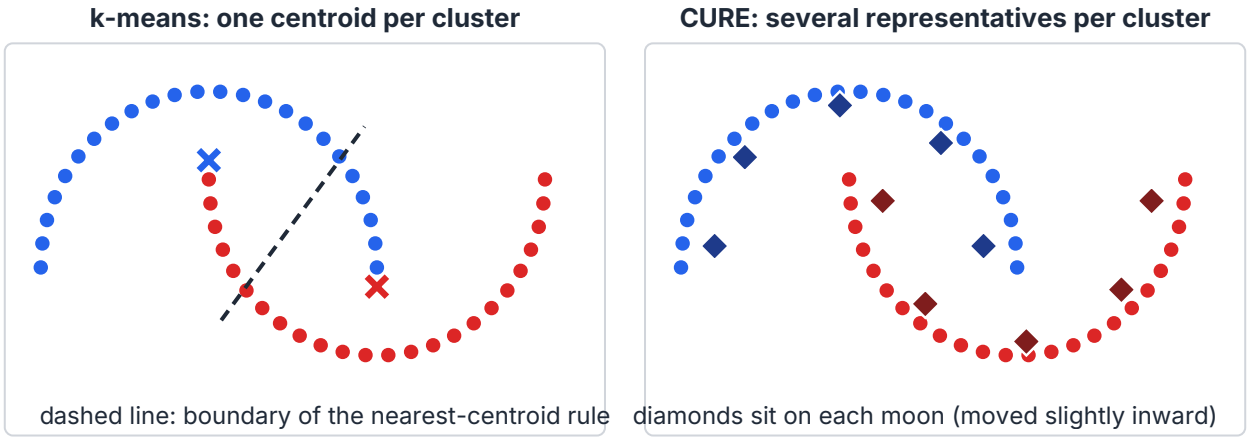


Figure 5 — Two interlocking half-moons as on Slide 36: why one centroid is not enough. Left: centroids (x) and the nearest-centroid boundary (dashed). Right: representatives (diamonds) along each moon. Illustration built for this guide.

In the illustration above (built for this guide, 24 dots per moon) nearest-centroid assignment gives the wrong label to 10 of the 48 points (about one in five); with representatives along each moon, a point is assigned to the moon whose representative is nearest, which follows the curve.

Why Euclidean? Representatives are moved toward a *centroid* (an average), so CURE needs a space where averages exist, just like *k*-means. **Beyond the slides:** "any shape" means non-spherical shapes such as ellipses and crescents; very complicated or touching shapes can still fail.

Pass 1: build clusters and representatives from a sample

Lecture 5 · Slide 38

Slide 38 ("Starting CURE") says CURE is a **2-pass algorithm** (one pass = one full read of the data from disk). Pass 1 works only on a small sample, so it fits in memory:

1. **Step 0, sample.** Pick a **random sample of points that fits in main memory**.
2. **Step 1, initial clusters. Cluster these points hierarchically**: start with every sampled point as its own cluster and repeatedly group (merge) the nearest points or clusters. Hierarchical clustering is affordable here because the sample is small (it is expensive on the whole data: Slide 21 gives its cost as $O(N^3)$). The slide does not say when to stop merging; stop when the wanted number of clusters is left. **Beyond the slides:** the

textbook suggests that two clusters are close if they contain a close pair of points, which lets clusters follow curved shapes.

3. **Step 2, representatives.** For each cluster, **pick a sample of points as dispersed as possible** (far apart, tracing the outline of the cluster). From that sample, **pick the representatives by moving them (say) 20% toward the centroid of the cluster**.

"(say)" means the numbers 4 points and 20% are examples, not fixed constants. The slide leaves open how many dispersed points to take (the next slides use 4), and "20%" means 20% of the distance to the centroid.

The salary-versus-age example

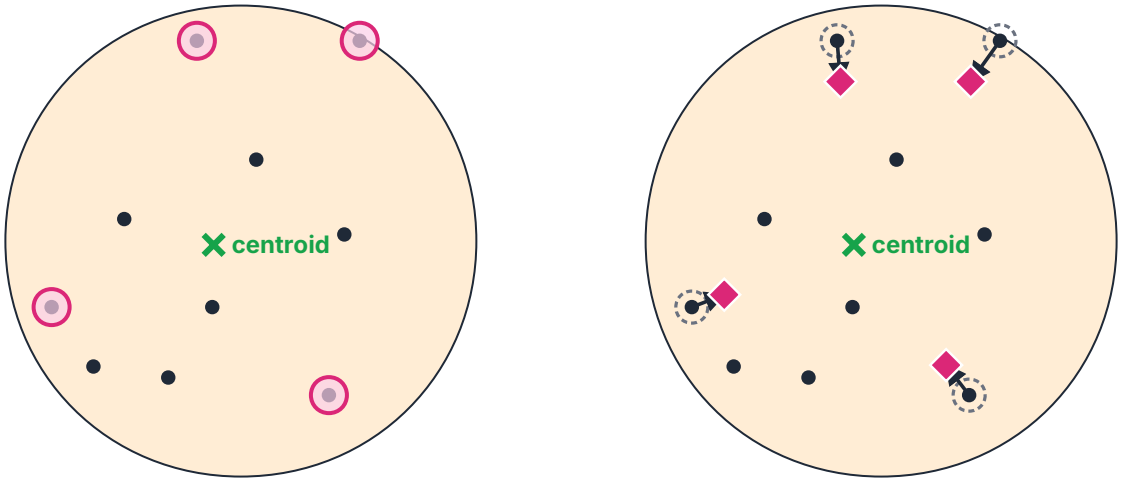
Lecture 5 · Slides 39–41

Slide 39, "Example: Initial Clusters". A scatter plot with axes "age" (right) and "salary" (up) and no numbers. 24 points are drawn as the letters "h" and "e"; the slide never explains these letters, they are just two kinds of point and play no part in the algorithm. The result of Pass 1 steps 0–1 is three clusters: a round peach circle (upper right, 10 points, a mix of h and e), a gold ellipse (middle left, 6 points) and a long thin purple ellipse (bottom, 8 points). The two ellipses are elongated and lie side by side. This is the point of the picture: clusters can have different shapes and sizes, which a single-centroid method handles badly.

Slide 40, "Example: Pick Dispersed Points". The green note says "Pick (say) 4 remote points for each cluster." Four pink circles mark four points on the rim of the peach cluster: top, top right, left and bottom right. "Remote" and "dispersed" mean the same thing: far from each other and from the middle, so that together they outline the cluster. Only one cluster is shown, but the rule is applied to *every* cluster. The slide does not say how to choose them; **Beyond the slides:** take the point farthest from the centroid first, then repeatedly the point whose distance to the nearest chosen point is largest (the farthest-first idea of Slide 23).

Slide 41. Same title, new note: "Move points (say) 20% toward the centroid." The four pink circles slide inward. These moved points are the cluster's **representatives**. They are artificial: generally not data points.

Slide 40: pick (say) 4 remote points | **Slide 41: move them (say) 20% inward**



pink ring = chosen remote point; dashed ring = old position; diamond = representative (artificial point)

Figure 6 — CURE Pass 1, step 2. Left (Slide 40): pink rings mark the chosen remote points. Right (Slide 41): each moves 20% toward the centroid (dashed ring = old position); the diamonds are the representatives.

FORMULA: MOVING A POINT TOWARD THE CENTROID

For a chosen point p and the centroid c of its cluster: new position = $p + 0.2 \times (c - p) = 0.8\,p + 0.2\,c$, applied to each coordinate. The point moves 20% of its distance to the centroid, so it ends up at 80% of its old distance from the centroid.

Worked example — one representative Practice, Slide 41 rule

Chosen point p = (10, 2), centroid c = (4, 6). c − p = (−6, 4); 0.2 × that = (−1.2, 0.8); new = (10 − 1.2, 2 + 0.8) = **(8.8, 2.8)**. Check: old distance √(36 + 16) = 7.21; new distance = 0.8 × 7.21 = 5.77. Quiz L5-Q6 below applies the same rule to a whole picture of representatives.

Why move inward at all? (Beyond the slides, standard reasoning.) A rim point can be an outlier; pulling every chosen point a fixed fraction toward the centroid lessens the effect of such points, while a *small* fraction keeps the spread, and so the shape information. Moving 100% would collapse all representatives onto the centroid, which is just *k*-means.

SLIDE ERRATUM (MINOR)

On Slide 41 the circles are drawn about one third of the way to the middle, not 20%. Measured from the picture, the drawing is only roughly to scale; use the 20% rule. Also, Slides 40 and 41 share a title, and the slides do not say that the representatives are new, synthetic points.

Pass 2: rescan and assign every point

Lecture 5 · Slide 42

After Pass 1 only the sample has been clustered; the representatives describe the clusters. Slide 42, "Finishing CURE", defines Pass 2: **rescan the whole data set and visit each point *p***, and **place it in the "closest cluster"**. The slide's definition: find the **closest representative** to *p* and assign *p* to **that representative's cluster**. Its picture shows one peach cluster with four representatives and a point *p* outside it.

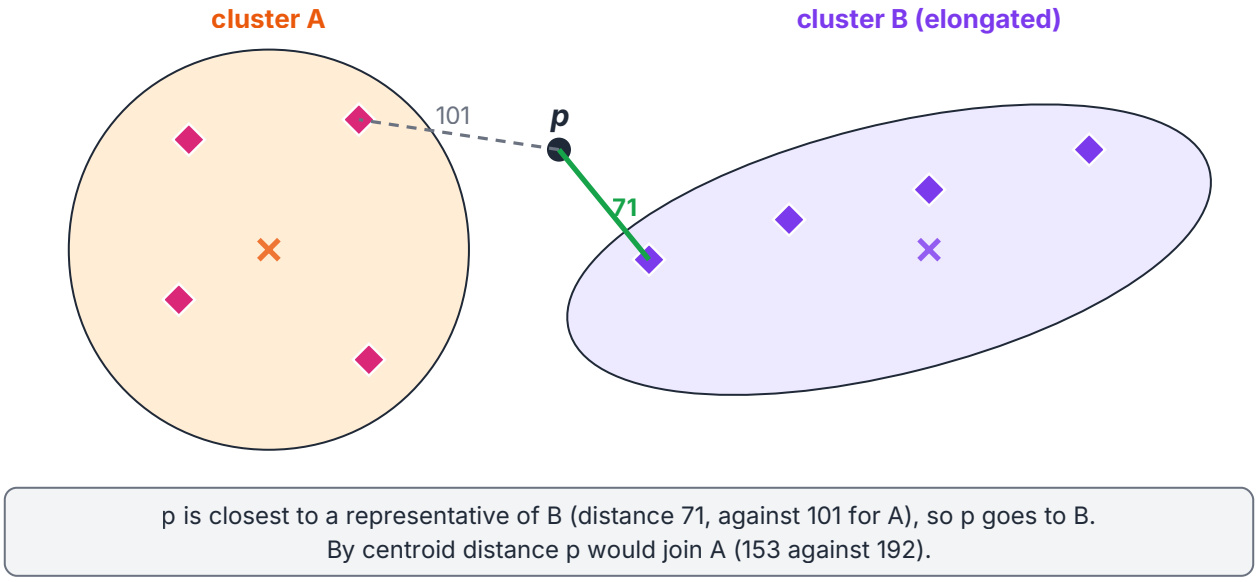


Figure 7 — CURE Pass 2: a point joins the cluster of its closest representative (Slide 42). Distances in arbitrary units.

1. Read the next point *p* from the data on disk.
2. Compute the distance from *p* to every representative of every cluster (Euclidean).
3. Find the nearest representative; give *p* that representative's cluster label.
4. Go to the next point. When the file is finished, every point has a label.

Only the few representatives (plus one point at a time) have to be in memory. Total data reads: two passes (one to take the sample, one to label everything).

KEY IDEA

"Closest cluster" is defined through the **representatives**, not through the centroid. The figure's elongated cluster B has its centroid far to the right, so *p* is nearer the centroid of A, yet it is nearer to a *representative* of B and belongs to B. Slide 42 calls this the "normal definition of closest" (closest representative); it implies that other definitions exist but gives none.

CURE versus k-means

Lecture 5 · Slides 23–27, 35–42

	k-means	CURE
Distance	Euclidean	Euclidean
Cluster summary	one point, the centroid (mean)	a collection of representative points
Cluster shape	compact, roundish blobs	any shape (ellipses, rings, crescents)
Needs *k* up front	yes, picked by the user (elbow method)	number of clusters at which Pass 1 stops merging (the slides do not give a rule)
Start	*k* seeds: random or farthest-first	a random sample clustered hierarchically
Data handled	the whole data set, re-read in every round	a memory-sized sample for the hard work, then one rescan
Assigning a point	to the nearest centroid	to the cluster of the nearest representative
Outliers	a mean is pulled by them (Beyond the slides)	representatives moved 20% inward damp them (Beyond the slides)
Iteration	repeat until convergence	no iteration: two passes

QUICK RECAP: CURE

- CURE = Clustering Using REpresentatives; Euclidean; any shape; large data.
- **Pass 1:** random sample that fits in memory → cluster hierarchically → per cluster pick (say) 4 dispersed points → move them (say) 20% toward the centroid = representatives.
- **Pass 2:** rescan all data; each point joins the cluster of its closest representative.
- Next in this lecture: BFR, the other scalable extension of *k*-means (it assumes normal clusters instead).

EXAM TIP

"Describe how CURE works": give Pass 1 steps 0, 1, 2 (sample, hierarchical clustering, dispersed points moved 20% to the centroid) and Pass 2 (rescan, nearest representative). Mention why: any shape (representatives) and large data (sample plus one rescan).

Try it yourself first. Write the steps of CURE from memory, then move a set of representatives 20% toward the centroid and compare with the solution.

Quiz L5-Q6 CURE: steps, and moving the representatives 20% toward the centroid

Lecture 5 · Slides 82–83 (method: Slides 35–42, 87)

QUESTION (AS ON THE SLIDES)

Quiz: Initialization in CURE (question on Slide 82; the slide's answer is on Slide 83). CURE = **C**lustering **U**sing **RE**presentatives.

(a) Describe in steps how CURE algorithm works.

(b) Assuming following selection of representative points from each cluster move the representative points 20% of the distance to the cluster's centroid.

Data (the picture on Slide 82; it has no numbers). Two clusters in a 2-D plane: a small round cluster (a *disc*) in the middle and a ring-shaped cluster (a grey *band*) around it, with an empty gap (the *hollow*) in between. The selected representative points are the black dots: **5 dots** on the small disc and **14 dots** on the ring (7 along its outer edge and 7 along its inner edge, alternating) — 19 in all. Slide 83 repeats the picture with the dots moved; its only written answer is "Moving the representative points 20% of the distance to the cluster's centroid". Because the slide gives no numbers, part (b) first reads the dot positions off the picture (Figure L5-Q6-2).

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Picture two countries on a map: a round island, and a ring-shaped island (an atoll) around it. To describe a country you could plant one flag in its middle (that is what k-means does with a centroid). For the atoll that fails: its middle is open water, and it is exactly the same spot as the round island's middle. CURE instead posts a handful of *border guards* (the **representatives**) spread around each country's edge. A newcomer simply asks "which guard is nearest to me?" and joins that guard's country. The "20% move" pulls every guard one fifth of the way back toward his country's centre, so a guard who wandered out to a far-off hamlet (an outlier) cannot stretch the country.

WORDS YOU NEED

Centroid = the average (mean) of all the points of a cluster, coordinate by coordinate (Lecture 5 · Slide 16); it need not be a data point. **Representative point** = one of a few stand-in points that together describe a cluster. **Euclidean distance** = ordinary straight-line distance, $d(p, q) = \sqrt{((p_1 - q_1)^2 + (p_2 - q_2)^2)}$ in 2-D. **Hierarchical (agglomerative) clustering** = start with every point as its own cluster and repeatedly merge the two nearest clusters (Slides 14 and 16).

Part (a) The steps of CURE Lecture 5 · Slides 35–42, 87

1. **What CURE is** Lecture 5 · Slides 35–36 CURE is an "extension of k-means to clusters of arbitrary shapes and large data" (Slide 35). It **assumes a Euclidean distance**, **allows clusters to have any shape** and **uses a collection of representative points to represent each cluster** instead of one centroid (Slide 36). It is a **2-pass algorithm**: Pass 1 builds the clusters and their representatives from a small sample; Pass 2 places every point of the whole data set (Slides 38 and 42).
2. **Pass 1, step 0 — pick a random sample of points that fits in main memory** Lecture 5 · Slide 38 *Why:* the full data set is too big for memory, but a sample can be clustered in memory by a method that would be far too slow on all the data; being random, the sample tends to show the same overall shapes.
3. **Pass 1, step 1 — initial clusters: cluster the sample hierarchically** Lecture 5 · Slides 38–39 Start with every sampled point as its own cluster and repeatedly "group nearest points/clusters" (merge the two closest clusters; Slide 14) until the wanted clusters are left; Slide 39 shows three initial clusters in a salary-versus-age plot. *Why hierarchical:* merging nearby points can follow a curved or elongated group, so clusters are not forced to be round blobs. *Beyond the slides:* merging stops when the wanted number of clusters is left (the slides give no stopping rule for CURE; compare "stopping point k" on Slide 69), and the textbook (*Mining of Massive Datasets*) suggests merging clusters that contain a close pair of points, which lets clusters follow curved shapes.

4. **Pass 1, step 2a — in each cluster pick a sample of points that is as dispersed as possible** Lecture 5 · Slides 38, 40 Slide 40: "pick (say) 4 remote points for each cluster". *Why:* points spread over the cluster outline its shape and extent: for a ring, points all around the ring; for a long cluster, points at both ends and in between. *Beyond the slides:* a standard way is "farthest first" (the idea of Slide 23 for starting k-means): take one point, then repeatedly add the point whose distance to the nearest already-chosen point is largest, until you have (say) 4. The quiz picture uses 14 dots for the ring and 5 for the disc; "4" is only an example.
5. **Pass 1, step 2b — move each chosen point (say) 20% toward the centroid of its cluster; the moved points are the representatives** Lecture 5 · Slides 38, 41 Slide 38: "pick representatives by moving them (say) 20% toward the centroid of the cluster"; Slide 41: "Move points (say) 20% toward the centroid". The moved points are artificial positions (not necessarily data points), and the data points themselves are not moved. A Euclidean space is needed, otherwise there is no centroid and no "straight line" to move along (this is why Slide 36 says CURE assumes a Euclidean distance). This is the step part (b) asks you to carry out.
6. **Pass 2 — rescan the whole data set and put every point in its "closest cluster"** Lecture 5 · Slides 42, 87 Read the entire data set once from disk. For each point p, find the **closest representative** to p (smallest Euclidean distance, over the representatives of all clusters) and assign p to **that representative's cluster** (Slide 42). Only the few representatives and one point at a time have to be in memory, and no further iterations are needed. *Beyond the slides:* the textbook inserts an optional merge before this pass: two clusters are merged if a pair of their representatives (one from each) is closer than a user-chosen threshold.

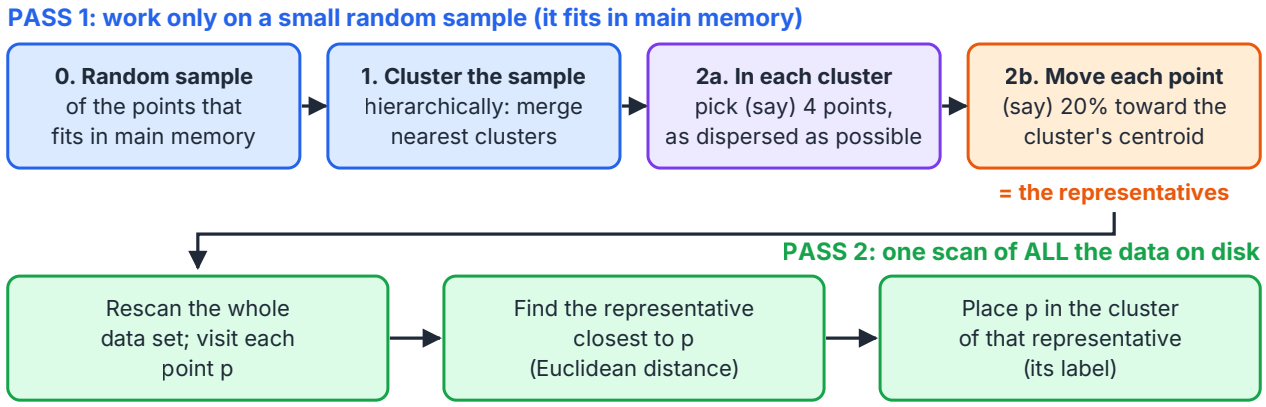


Figure L5-Q6-1. The two CURE passes (Lecture 5 · Slides 38, 40–42): the sample makes the representatives, the rescan uses them.

WHY REPRESENTATIVES ALLOW ARBITRARY SHAPES (SLIDE 36)

A cluster is described by *several* points spread over its outline, and a point belongs to the cluster of its nearest representative. So a cluster's territory is the union of the neighbourhoods of its representatives, which can bend along a ring or a crescent (Slide 36 shows two interlocking crescents). One centroid can only describe a compact blob. In this quiz the ring's centroid sits exactly where the small disc's centroid is, so no centroid-only method can tell the two clusters apart, but the ring's representatives lie on the ring and the disc's lie in the disc, so every point finds the right cluster (checked numerically in part (b), step 9). BFR (Bradley–Fayyad–Reina, Slide 44), by contrast, assumes normally distributed, axis-aligned elliptical clusters; CURE has no such assumption.

CURE COMPARED WITH K-MEANS, IN THREE BULLETS (K-MEANS: SLIDES 23–27)

- **Shape.** k-means summarises each cluster by ONE point (the centroid) and sends every point to the nearest centroid, so it finds compact, roundish blobs. CURE summarises each cluster by SEVERAL spread-out representatives, so a cluster can be a ring, a crescent or a long ellipse.
- **Data size and passes.** k-means repeats "assign all points, recompute centroids" until convergence (Slide 24), so it needs the whole data set again and again. CURE clusters only a memory-sized sample in Pass 1 and then needs just one more scan (Pass 2), which is why Slide 35 says "large data".
- **Start, summary and outliers.** k-means starts from k chosen points (Slide 23) and its centroid, a mean, is dragged by outliers. CURE starts from a hierarchical clustering of the sample, pulls its representatives 20% toward the centroid (*Beyond the slides:* this damps the pull of outlying points) and defines "closest cluster" as the cluster of the closest *representative*. Both assume a Euclidean distance.

Part (b) Move the representatives 20% toward the centroid **Lecture 5 · Slides 82–83 (method: 38, 41)**

1. **Formula (write it down first).** For a representative p and the centroid c of *its own* cluster, applied to each coordinate:

   MOVE 20% TOWARD THE CENTROID
   new position = $p + 0.2 \times (c - p) = 0.8 \times p + 0.2 \times c$
   distance moved = $0.2 \times d(p, c)$ and new distance to the centroid = $0.8 \times d(p, c)$

   Why it works: $(c - p)$ is the arrow from the point to the centroid; adding one fifth of that arrow to p slides the point one fifth of the way along the straight line to the centroid. "Move 20% of the distance to the centroid" means move *by* 20% of that distance (the point stays far out), not "move to a place 20% of the distance away" (that would be 80% of the way in). Slide 83's picture agrees: the dots only shift a little.
2. **Tiny numeric check of the formula.** For p = (8, 6) and c = (4, 2): new = (8 + 0.2 × (4 − 8), 6 + 0.2 × (2 − 6)) = (8 − 0.8, 6 − 0.8) = (7.2, 5.2). The old distance $\sqrt{(4^2 + 4^2)} = 5.657$ shrinks to $\sqrt{(3.2^2 + 3.2^2)} = 4.525 = 0.8 \times 5.657$.
3. **Find the centroids.** The centroid is the average of *all* points of the cluster (not of the representatives only). Both clusters are symmetric about the same middle of the picture, so both centroids are at that middle. The ring's centroid is therefore an empty spot in the hollow, not a data point; this is why CURE needs a Euclidean space, where averages exist. The slide marks no centroids, so we put the origin **(0, 0)** at the middle: c = (0, 0) for both clusters.

   COMMON MISTAKE
   Do not average the representatives to get the centroid. The 14 ring dots average to about (1.4, −7.3) only because they are unevenly spaced; the true centroid of the whole ring is its middle.
4. **Read the coordinates off Slide 82.** Slide 82 is a picture without numbers, so we measured it: *x* to the right, *y* upward, and the unit chosen so that the **outer radius of the ring is 100** (the ring band runs from radius about 74 to 100, so the hollow's edge is at about 74; the small disc has radius about 25). Readings are accurate to about ±1–2 units; any consistent reading earns the marks. Name the dots **O1–O7** (ring, outer edge), **I1–I7** (ring, inner edge), **D1–D5** (small disc). The small disc is drawn about 2.7 units left of the ring's middle; that is below reading accuracy (taking the disc's own middle as its centroid would change each new disc position by only 0.2 × 2.7 = 0.5 units), so both centroids are taken at (0, 0).
5. **Worked rows, in full.** With c = (0, 0) we have $c - p = -p$, so new = $p - 0.2p = 0.8p$.
   - **O1** (ring, outer edge), p = (80, 58): $c - p$ = (−80, −58); 0.2 × that = (−16, −11.6); new = (80 − 16, 58 − 11.6) = **(64.0, 46.4)**. Check: old distance $\sqrt{(80^2 + 58^2)} = \sqrt{9764} = 98.8$; new distance $\sqrt{(64^2 + 46.4^2)} = \sqrt{6248.96} = 79.1$, i.e. 80% of the old distance; distance moved $\sqrt{(16^2 + 11.6^2)} = 19.8$, i.e. 20% of the old distance.
   - **I5** (ring, inner edge), p = (0, −78): $c - p$ = (0, 78); 0.2 × that = (0, 15.6); new = (0 + 0, −78 + 15.6) = **(0.0, −62.4)**. Check: old distance $\sqrt{(0^2 + 78^2)} = \sqrt{6084} = 78.0$; new distance $\sqrt{(0^2 + 62.4^2)}$ =

√3893.76 = 62.4, i.e. 80% of the old distance; distance moved √(0² + 15.6²) = 15.6, i.e. 20% of the old distance.

- **D3** (small disc), $p = (-23, 10)$: $c - p = (23, -10)$; 0.2 × that = (4.6, −2); new = (−23 + 4.6, 10 − 2) = **(−18.4, 8.0)**. Check: old distance √(23² + 10²) = √629 = 25.1; new distance √(18.4² + 8²) = √402.56 = 20.1, i.e. 80% of the old distance; distance moved √(4.6² + 2²) = 5.0, i.e. 20% of the old distance.

6. **All 19 representatives.** Every row is new = 0.8 × old; the distances come from d = √(x² + y²).

Ring cluster (14 representatives), centroid (0, 0)					
Rep	**Slide 82 position (x, y)**	**distance d to centroid**	**moved 0.2 · d**	**new position (x, y)**	**new distance 0.8 · d**
O1	(80, 58)	98.8	19.8	**(64.0, 46.4)**	79.1
O2	(8, 98)	98.3	19.7	**(6.4, 78.4)**	78.7
O3	(−80, 57)	98.2	19.6	**(−64.0, 45.6)**	78.6
O4	(−95, −28)	99.0	19.8	**(−76.0, −22.4)**	79.2
O5	(−32, −93)	98.4	19.7	**(−25.6, −74.4)**	78.7
O6	(45, −90)	100.6	20.1	**(36.0, −72.0)**	80.5
O7	(88, −48)	100.2	20.0	**(70.4, −38.4)**	80.2
I1	(40, 65)	76.3	15.3	**(32.0, 52.0)**	61.1
I2	(−30, 70)	76.2	15.2	**(−24.0, 56.0)**	60.9
I3	(−78, 10)	78.6	15.7	**(−62.4, 8.0)**	62.9
I4	(−55, −55)	77.8	15.6	**(−44.0, −44.0)**	62.2
I5	(0, −78)	78.0	15.6	**(0.0, −62.4)**	62.4
I6	(53, −55)	76.4	15.3	**(42.4, −44.0)**	61.1
I7	(75, −13)	76.1	15.2	**(60.0, −10.4)**	60.9

Small disc cluster (5 representatives), centroid (0, 0)					
Rep	**Slide 82 position (x, y)**	**distance d to centroid**	**moved 0.2 · d**	**new position (x, y)**	**new distance 0.8 · d**
D1	(20, 5)	20.6	4.1	**(16.0, 4.0)**	16.5
D2	(5, 22)	22.6	4.5	**(4.0, 17.6)**	18.0
D3	(−23, 10)	25.1	5.0	**(−18.4, 8.0)**	20.1
D4	(−15, −20)	25.0	5.0	**(−12.0, −16.0)**	20.0
D5	(13, −18)	22.2	4.4	**(10.4, −14.4)**	17.8

7. **Draw it (this is what Slide 83 shows).** Mark the centroid, draw from every dot a straight line toward it, and put the new dot one fifth of the way along the line (equivalently at 80% of the old distance from the centre).

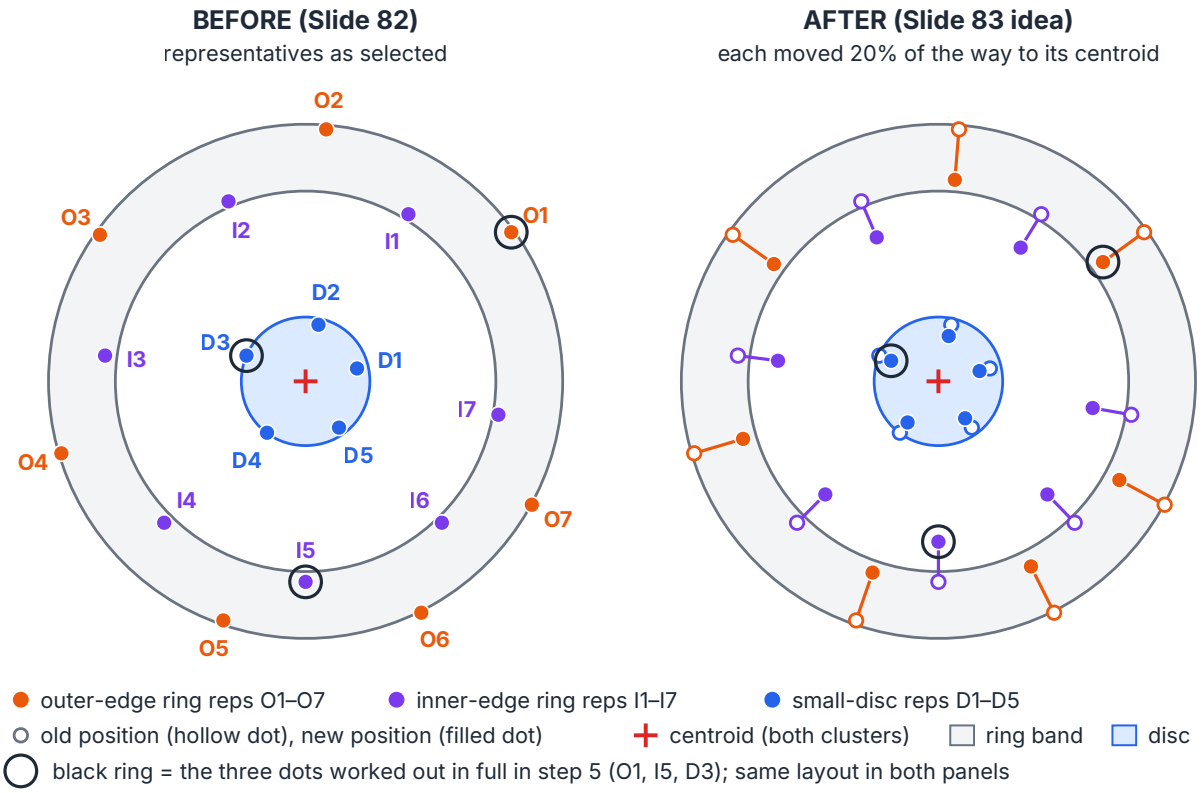
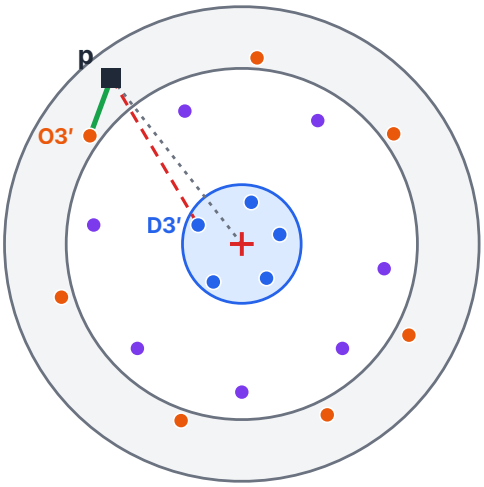


Figure L5-Q6-2. Before (Slide 82) and after (the exact 20% rule; Slide 83 is a hand-drawn version of this): every representative slides one fifth of the way along the straight line to its centroid, so the ring of dots and the disc of dots both shrink toward the middle. Coordinates from the tables above (outer radius of the ring = 100 units).

8. **Sanity checks and what the numbers say.** (i) Every new dot lies on the segment between its old position and the centroid (same direction, smaller radius). (ii) New distance = 0.8 × old; distance moved = 0.2 × old, so points farther out move farther. (iii) **Ring, outer-edge dots**: radius ≈ 98–101 becomes ≈ 79–80 (moved on average 19.8); they are still inside the grey band, whose inner edge is at about 74. **Ring, inner-edge dots**: ≈ 76–79 becomes ≈ 61–63 (moved on average 15.4), which is below 74, so they now lie **inside the empty hollow**. **Small disc**: ≈ 21–25 becomes ≈ 16.5–20.1 (moved on average 4.6), still inside the disc (radius about 25). (iv) The whole pattern is a shrunken copy (scaled by 0.8 about the same centre) of the old one. (v) No dot crosses the centroid, because each moves only 20% of the way. This is exactly the pattern of Slide 83: 7 dots in the band, 7 dots in the hollow, 5 dots pulled inward in the disc.
9. **Check that Pass 2 still works with the moved representatives.** Take the 19 moved dots as the final representatives and assign data points p by the nearest representative (Euclidean distance, Slide 42). Both clusters have the same centroid, so the distance to "the centroid" is the same for the ring and the disc (second column) and cannot decide; the nearest representative can.

point p	distance to the centroid (both clusters)	nearest small-disc rep, distance	nearest ring rep, distance	assigned to
`(10, −12)`	15.6	D5′ 2.4	I6′ 45.5	**small disc**
`(−55, 70)`	89.0	D3′ 72.0	O3′ 26.0	**ring**
`(70, −45)`	83.2	D5′ 67.0	O7′ 6.6	**ring**



Pass 2 for one data point p = (−55, 70)

(dots = the 19 moved representatives; p lies in the ring band)

- to nearest ring rep O3′: 26.0
- to nearest small-disc rep D3′: 72.0
- to the centroid (same spot for both): 89.0

Closest-representative rule (CURE, Slide 42):
26.0 < 72.0, so p joins the ring cluster.

Closest-centroid rule (k-means style):
89.0 = 89.0, a tie, so it cannot decide.

Figure L5-Q6-3. Pass 2 on the quiz picture (representatives after the 20% move): a point in the ring band is far closer to a ring representative than to any small-disc representative, although both centroids are at the same spot.

(Python check: of 300,000 random points in the ring band and 300,000 in the disc, none is closer to a representative of the other cluster.)

WHY MOVE THE REPRESENTATIVES TOWARD THE CENTROID?

Beyond the slides: the dispersed points are the most extreme points of their cluster, so they may be outliers or lie right at the fringe. Shrinking them toward the centroid pulls them in, and the farther out a point is, the more it moves (20% of a bigger distance). With a shrink of 100% every representative would collapse onto the centroid and CURE would behave like a centroid method such as k-means; with 0% it would use the raw boundary points. The slides only say "(say) 20%", so treat 20% as an example value, not a law.

PRACTICE (SELF-MADE, NOT FROM THE SLIDES): THE SAME MOVE WITH NUMBERS GIVEN

A cluster has the five points (1, 1), (3, 7), (5, 1), (9, 3), (7, 3), so its centroid is c = (5, 3) (average of each coordinate; not (0, 0)). Pick the four dispersed points (1, 1), (3, 7), (9, 3), (5, 1) and move each 20% toward c with new = $p + 0.2 \times (c - p)$:

p	c − p	0.2 × (c − p)	new position
(1, 1)	(4, 2)	(0.8, 0.4)	**(1.8, 1.4)**
(3, 7)	(2, −4)	(0.4, −0.8)	**(3.4, 6.2)**
(9, 3)	(−4, 0)	(−0.8, 0.0)	**(8.2, 3.0)**
(5, 1)	(0, 2)	(0.0, 0.4)	**(5.0, 1.4)**

EXAM TIP

Part (a) is "describe in steps": write the numbered steps in this order with the key words: random sample that fits in main memory → hierarchical clustering of the sample (merge nearest points/clusters) → pick dispersed points in each cluster (say 4) → move them (say) 20% toward the centroid = the representatives → Pass 2: rescan all points, assign each to the cluster of the closest representative. For part (b), draw both clusters, mark the centroid, show new = p + 0.2(c − p) once, and put every new dot one fifth of the way in. A clearly shown formula and line earn marks even if a coordinate is slightly off.

COMMON MISTAKE

- Moving to 20% of the distance (i.e. 80% of the way) instead of moving *by* 20% of the distance; or moving by a fixed amount ("20 units") instead of a fraction of each point's *own* distance.
- Using the wrong centroid (the middle of the whole data, the other cluster's, or the average of only the representatives), or thinking the ring's centroid must be a data point (it lies in the empty hollow).
- Moving all data points, or the representatives of only one cluster. Every representative of every cluster moves, and only the representatives.
- Forgetting Pass 2, or doing the final assignment by nearest centroid. In CURE a point goes to the cluster of its closest *representative*.

SLIDE ERRATUM (MINOR: DRAWING AND OMISSIONS ONLY; NO WRONG RULE)

(1) Slide 83 gives only a picture and one line of text, so the written steps of part (a) come from Slides 35–42 and 87. (2) The Slide 83 picture is hand-drawn and not to scale. Measured against Slide 82 (both pictures scaled to the same ring size), the ring dots moved only about 9–20% of their distance (outer edge from radius ≈ 99 to ≈ 80–90, inner edge from ≈ 77 to ≈ 62–68) and the disc dots about 30–45% (from ≈ 23 to ≈ 12–17), instead of exactly 20% for all. The direction (straight toward the common middle) and the pattern (7 dots in the band, 7 in the hollow, 5 in the disc) agree, so treat it as an imprecise drawing; for numbers use the exact rule above. (3) Omissions: the slides do not say when the hierarchical step stops, how "dispersed" points are chosen, or that the textbook merges close clusters before Pass 2 (Slide 87 omits it too); these are filled in above under "Beyond the slides". (4) Slide 40 says "(say) 4" remote points per cluster while the quiz picture has 14 and 5; the number is a free parameter.

Final answer:
(a) CURE has two passes. Pass 1: (0) take a random sample of points that fits in main memory; (1) cluster the sample hierarchically (repeatedly merge the nearest points/clusters) to get the initial clusters; (2) in each cluster pick (say) 4 points that are as dispersed as possible and move each (say) 20% toward the cluster's centroid, p′ = p + 0.2(c − p); the moved points are the representatives. Pass 2: rescan the whole data set and place each point in the cluster of its closest representative (Euclidean distance).
(b) Each representative moves straight toward the centroid of its own cluster by 20% of its own distance: new = p + 0.2(c − p), which with the centroid at the middle (0, 0) is 0.8 × p, i.e. to 80% of the old distance (outer radius of the ring = 100 units). Ring outer-edge dots: radius ≈ 99.1 → ≈ 79.3 (still in the band). Ring inner-edge dots: ≈ 77.1 → ≈ 61.6 (now in the hollow). Small-disc dots: ≈ 23.1 → ≈ 18.5 (still in the disc). All 19 coordinates are in the tables above, e.g. (80, 58) → (64, 46.4), (0, −78) → (0, −62.4), (−23, 10) → (−18.4, 8).

Where the marks come from: Part (a), roughly one mark per step: (1) CURE idea: Euclidean, representatives instead of one centroid, 2 passes; (2) random sample that fits in main memory; (3) hierarchical clustering of the sample (merge nearest points/clusters) gives the initial clusters; (4) in each cluster pick dispersed points (say 4); (5) move them (say) 20% toward the centroid, giving the representatives; (6) Pass 2: rescan all the data and assign each point to the cluster of its closest representative. Part (b): (7) correct formula or idea "one fifth of the way to the centroid", new = p + 0.2(c − p); (8) correct centroid for each cluster (both at the middle; the ring's is in the hollow); (9) every representative of both clusters moved straight toward its own centroid; (10) each move is 20% of that point's own distance (new distance = 80% of old; outer dots move more than inner and disc dots). This split is a sensible guess, not the official mark scheme.

Slide check: matches the slide's answer. Slide 83 shows every representative pulled toward the common middle, which is what "move 20% of the distance to the cluster's centroid" gives; our exact positions (new = 0.8 × old) were recomputed with Python. Slide erratum (minor): the Slide 83 picture is not drawn exactly to scale (ring dots ≈ 9–20%, disc dots ≈ 30–45% instead of 20%) and the slides omit a stopping rule, the farthest-first rule and the merge step; see the erratum box above. The coordinates were measured from the raster picture on Slide 82 (accuracy about ±1–2 units), so your own readings may differ by that much without being wrong.

BFR: k-means for data that does not fit in memory

Lecture 5 · Slides 43–45

Slide 43 is a black divider titled "The BFR Algorithm" with the subtitle "Extension of *k*-means to large data". Two scalable clustering ideas are in this lecture. CURE (the section before) keeps a memory-sized *sample*, builds representative points from it and then rescans the data once. **BFR** takes the opposite route: it reads *all* the data, chunk by chunk, but keeps almost none of the points: it keeps running totals (N, SUM, SUMSQ) per cluster and per compressed group, plus a few raw retained points (RS).

BEGINNER INTUITION

A market-research firm interviews a billion people. It cannot keep a billion questionnaires on one desk. Instead, for each customer segment it keeps a tally sheet: how many people so far, the running total of their ages, and the running total of the squares of their ages. From those three numbers it can always say "the average age is 41, and ages are spread by about 9 years", and when a new questionnaire arrives it simply adds to the totals and throws the paper away. BFR is that tally-sheet trick applied to clusters.

What BFR is, and what it assumes

Lecture 5 · Slide 44

DEFINITION: BFR (BRADLEY–FAYYAD–REINA)

BFR is a variant of *k*-means designed to handle **very large (disk-resident) data sets**. It **assumes** that clusters are **normally distributed around a centroid in a Euclidean space**, and that the dimensions are independent, so each cluster is an **axis-aligned ellipse** (Slide 44). Its trick is an **efficient way to summarise clusters**: the memory required should be **O(clusters), not O(data)**.

Take the three phrases one at a time.

- **"Very large, disk-resident".** The data sit on disk because they do not fit in main memory (RAM). Reading from disk is the slow part, so the sensible plan is to read each point from disk only once and extract everything we need from it at that moment. (**Beyond the slides:** that is what BFR does: one pass over the data. The slides do not say "one pass" in words; they say points are read "in chunks".)
- **"Euclidean".** A centroid is an average of points, and distances are straight-line distances. So BFR cannot be used with Jaccard, cosine or edit distance, exactly as for *k*-means itself (Slide 23).
- **"O(clusters), not O(data)".** Memory grows with the number of clusters k (and the dimension d), not with the number of points n. Doubling the data size does not change the memory needed. The arithmetic: each cluster is stored with $2d + 1$ numbers (Slide 49, explained below). For $k = 50$ clusters in $d = 100$ dimensions that is $50 \times (2 \cdot 100 + 1) = 10\,050$ numbers, whether the data hold 10^6 or 10^9 points. Keeping the 10^9 raw points would need $10^9 \times 100 = 10^{11}$ numbers.

The normal-distribution assumption

Lecture 5 · Slide 44 (bell-curve figure)

The picture in the top-right corner of Slide 44 is the bell curve of the **Gaussian (normal) distribution**: the density of points is highest at the centre (the mean, 0 on the axis) and falls away symmetrically. The horizontal axis is measured in **standard deviations σ** (sigma): σ is the typical distance of a point from the centre, along that axis. The numbers on the slide are the share of all points in each strip between the dashed lines. Check that they add up: $2 \times (0.3413 + 0.1359 + 0.0214 + 0.00135) = 0.9999$, which is 1 up to rounding.

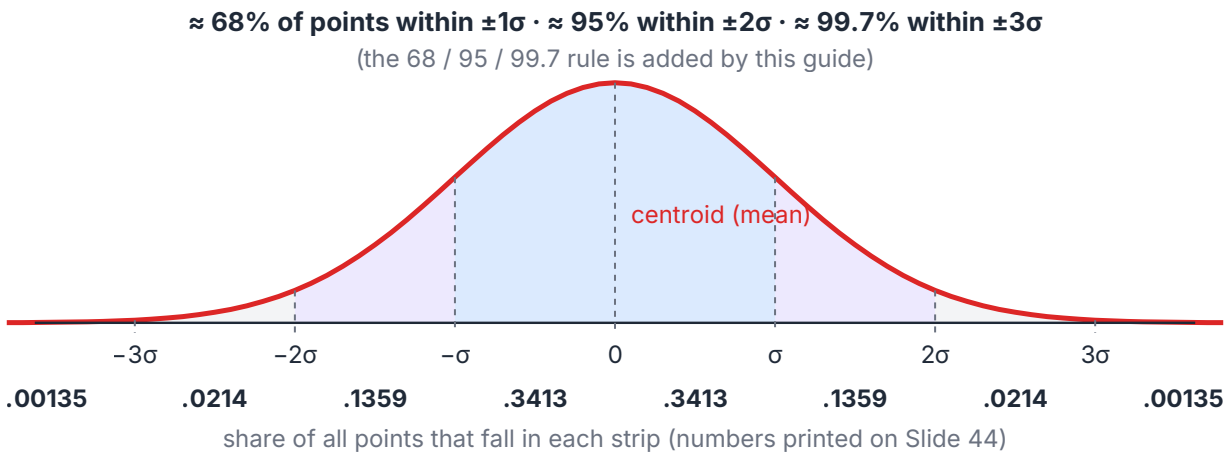


Figure 1: The Gaussian curve of Slide 44. Strip masses are the numbers printed on the slide; the 68 / 95 / 99.7 summary on top is simply 0.3413 × 2 = 68.3%, plus 0.1359 × 2 gives 95.4%, plus 0.0214 × 2 gives 99.7%.

Why this matters for clustering: **if a cluster really is normal, almost all of its points lie within 2 or 3 σ of its centroid**. That fact is what makes the later "is this point close enough?" test (Slides 54–56) meaningful: "within about 2 standard deviations" has a precise probabilistic meaning only because of this assumption.

DEFINITION: NORMAL CLUSTER IN D DIMENSIONS (AS BFR USES IT)

Each coordinate of the points in a cluster is bell-shaped around the cluster mean, with its own standard deviation $\sigma_1, \ldots, \sigma_d$, and the coordinates do not influence each other. The cloud of points is then an **ellipse** (a stretched circle) whose long and short directions are along the coordinate axes.

Why "axis-aligned" ellipses?

Lecture 5 · Slide 44 (three clusters figure); shapes quiz Slides 78–79

The coloured picture on Slide 44 shows three clusters: a circle (blue), a wide horizontal ellipse (magenta) and a tall vertical ellipse (green). All three have their long and short directions parallel to the page axes. That is the BFR assumption. The standard deviations may be **different in different dimensions** (that makes an ellipse rather than a circle), but the axes of the cluster must **align with the axes of the space** (Slide 79).

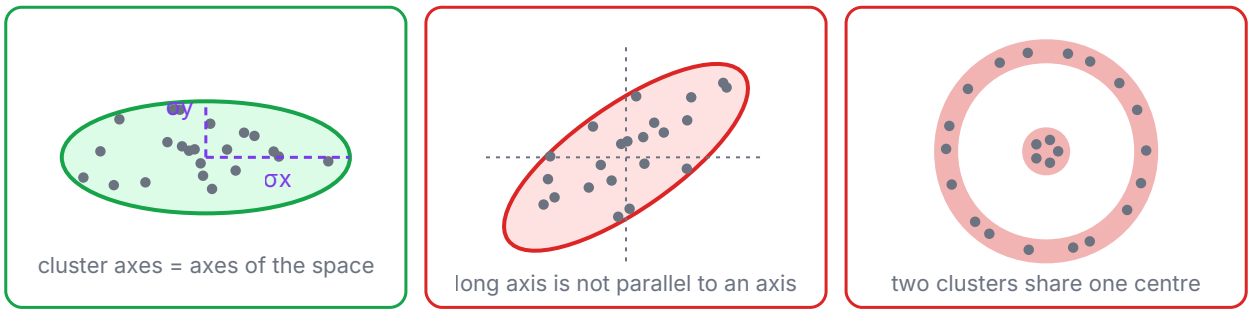


Figure 2: What BFR can and cannot describe. Left: allowed (Slides 44 and 79). Middle: the tilted ellipse of Slide 78, marked "Not OK" on Slide 79. Right: a ring around a core is not one bell-shaped cloud; this is the kind of shape CURE (Slides 35–42) was designed for.

Why does BFR need this? (**Beyond the slides** for the full reasoning; the slides only state the rule.) A cluster is summarised by *one* mean and *one* variance *per dimension* (Slides 48–49). That describes spread along each coordinate axis separately. A tilted ellipse means that when x is large, y also tends to be large: the coordinates are **correlated**. Capturing that needs extra numbers (covariances, $d(d-1)/2$ of them), which BFR does not store.

A tilted cluster would be summarised by an axis-aligned ellipse that is too fat, and the closeness test would accept points that do not belong (see Figure 5 of Quiz L5-Q4 below).

EXAM TIP

The shape question is a standard one: **horizontal ellipse OK, vertical ellipse OK, tilted ellipse not OK** (circles are the special case of equal σ and are fine). Give the reason in one line: "standard deviations may differ along different axes, but the axes of the cluster must align with the axes of the space".

Memory loads and the first k centroids

Lecture 5 · Slide 45

Slide 45 sets the stage for the whole algorithm in three bullets.

1. **Points are read from disk into main memory in chunks.** One chunk is called a **memory load**. It is as many points as fit in RAM at once (together with the summaries).
2. **Most points from previous memory loads are summarised by simple statistics.** They are not kept. The word is "most", not "all": some points (the retained set, next section) are kept exactly as they are.
3. **To begin, from the initial load we select the initial *k* centroids "by some sensible approach".** The only one listed is "take *k* random points"; the second bullet is a literal "...".

SLIDE ERRATUM (OMISSION)

The "..." on Slide 45 stands for the other sensible approaches, which the slide never lists. **Beyond the slides:** (i) pick one random point, then repeatedly the point farthest from all points already chosen (the farthest-first rule of Slide 23, used in Quiz L5-Q3); (ii) cluster a small sample hierarchically and take the centroids. The slide also does not say how big a chunk is, and it takes *k* as known in advance, as in *k*-means.

The three sets of points: DS, CS and RS

Lecture 5 · Slides 46–47, 52

Slide 46 names the vocabulary that the rest of BFR is written in. At any moment each point that BFR has seen is in **exactly one** of three places (or has been merged into one of them).

BEGINNER INTUITION

A post office sorts letters. Letters for a known neighbourhood go in that neighbourhood's bin, where only a bundle count is recorded (**DS**). Letters from several senders in the same unknown town are tied into a bundle that waits to be assigned to a neighbourhood later (**CS**). A strange letter that fits nowhere stays loose on the table (**RS**).

DEFINITIONS (SLIDE 46)

Discard set (DS): points close enough to a centroid to be summarised. Only the summaries are kept in main memory; the points are discarded.

Compression set (CS): groups of points that are close together but not close to any existing centroid. These points are summarised, but not assigned to a cluster. Only the summaries are kept in main memory.

Retained set (RS): isolated points waiting to be assigned to a compression set. Held in main memory exactly as they are.

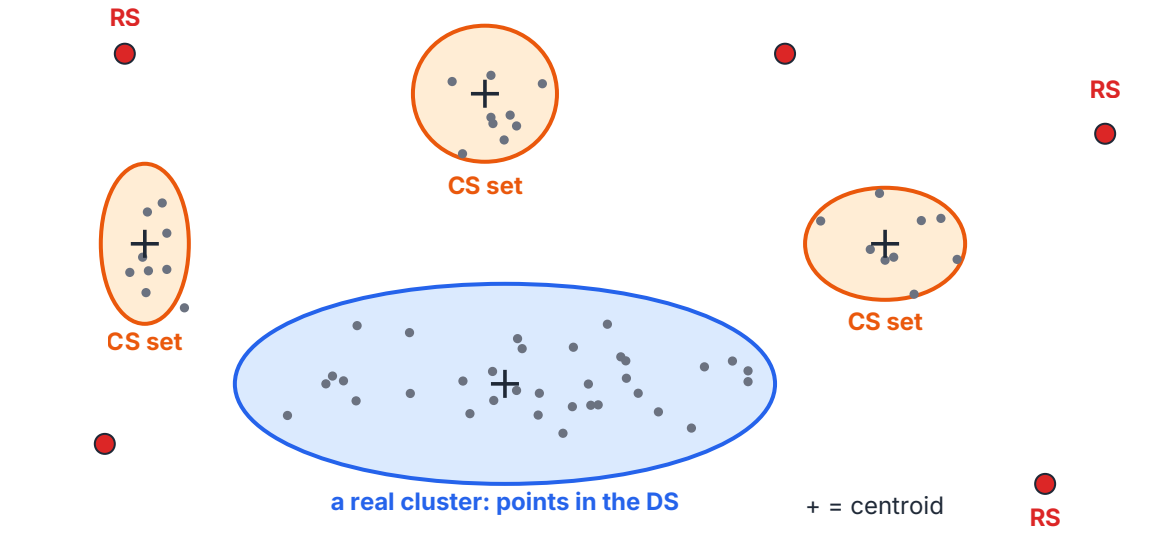


Figure 3: The "galaxies" picture of Slides 47 and 52, redrawn. Slide 47 draws a single real cluster to keep the picture simple; BFR actually keeps *k* of them. The small grey dots indicate points that were summarised and are no longer stored; only the red RS dots are real points in memory.

Read it like a night sky. The big blue ellipse is a real "galaxy": a cluster, whose size and shape reflect its standard deviations in each dimension. The small orange ellipses are tight little groups of stars that are not near a galaxy yet: they are summarised (each has its own centre, the +), but they do not belong to a cluster. The red dots are lone stars: neither near a galaxy nor near each other. Slide 52 is an exact repeat of Slide 47 (it is shown again just before the two "A few details" questions); nothing new is on it.

	DS	CS	RS
What it holds	points close to a cluster centroid	small tight groups far from every centroid	isolated points
In memory as	summary per cluster (N, SUM, SUMSQ)	summary per group (N, SUM, SUMSQ)	the raw points
Assigned to a cluster?	**yes**	no (not yet)	no
Points discarded?	yes	yes	no
How many of them	exactly *k* summaries	a modest number of groups	a modest number of points
Fate	stays; grows as points are added	merged with other CS (step 4) or, in the last round, into the nearest cluster / outlier	may join a new CS in a later load; last round: nearest cluster / outlier

Why bother with a CS? Suppose a group of 5 000 points lies far from every centroid. If we kept them as raw points, memory would grow with the data. If we threw them away, we would lose a possible cluster. Summarising them as one CS set costs only $2d + 1$ numbers and keeps the option open. **Why an RS?** A single point, or two far-apart points, give no useful spread estimate (one point has variance 0). They are cheaper and safer to keep as they are until more points arrive.

COMMON MISTAKES

- "The CS belongs to a cluster." No: the CS is summarised but **not assigned to a cluster**.
- "All points are discarded." No: the RS points are kept exactly as they are.
- "DS and CS are stored differently." Both are summaries; the difference is whether the group is attached to one of the *k* clusters.

(Slide 46 never says what the CS summary is. It is the same N, SUM, SUMSQ as for the DS; this is implied by Slides 48–51 and said outright on Slide 57.)

What is stored: N, SUM and SUMSQ

Lecture 5 · Slides 48–49

This is the technical heart of BFR, and it is short. The question: "what is the smallest thing we can keep about a cluster, so that we can still say where its centre is and how spread out it is, and so that we can add new points cheaply?"

DEFINITION: THE SUMMARY OF A CLUSTER (SLIDE 48)

For each cluster, the discard set (DS) is **summarised** by:

- **N**: the number of points;
- **SUM**: a vector whose *i*-th component is the sum of the coordinates of the points in the *i*-th dimension;
- **SUMSQ**: a vector whose *i*-th component is the sum of the squares of the coordinates in the *i*-th dimension.

With *d* dimensions that is 1 + *d* + *d* = **2*d* + 1 numbers, whatever the size of the cluster** (Slide 49). A cluster of 10 points and a cluster of 10 million points take the same space.

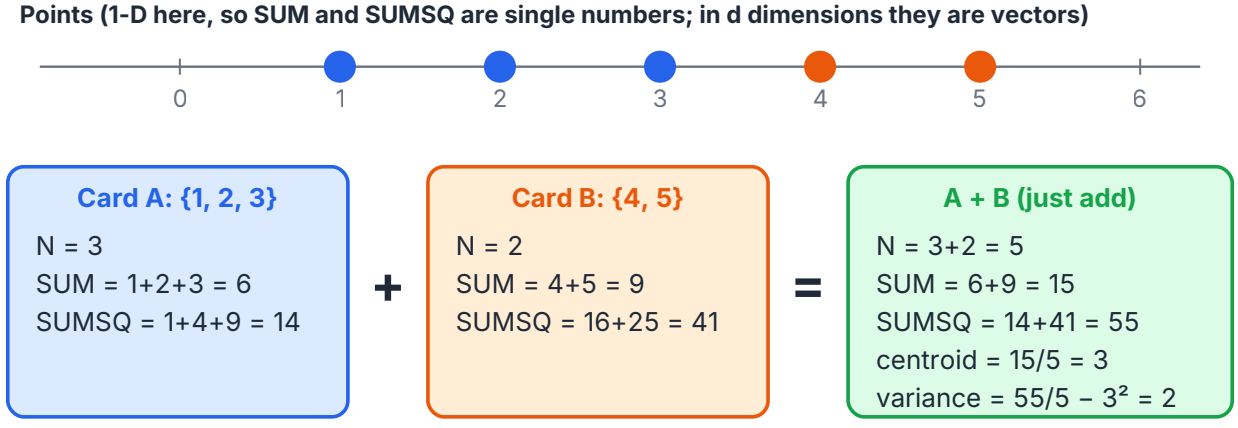
FORMULAS (SLIDE 49): READING THE CLUSTER BACK OUT OF ITS SUMMARY

Centroid in dimension *i*: $c_i = SUM_i / N$

Variance in dimension *i*: $\sigma_i^2 = SUMSQ_i / N - (SUM_i / N)^2$

Standard deviation in dimension *i*: $\sigma_i = \sqrt{(SUMSQ_i / N - (SUM_i / N)^2)}$

Why the variance formula works. (Beyond the slides, three lines of algebra; you do not need to reproduce it.) Variance is the average squared distance from the mean c: $(1/N) \Sigma (x - c)^2$. Expanding the square gives $(1/N) \Sigma x^2 - 2c \cdot (1/N) \Sigma x + c^2$. Since $(1/N) \Sigma x = c$, this is $(1/N) \Sigma x^2 - 2c^2 + c^2 = SUMSQ/N - c^2$. In words: **mean of the squares minus the square of the mean**.



Same as recomputing from the raw points 1 to 5 (mean 3, variance 2), without needing them.

Figure 4: A cluster summary is a small "tally card". Two cards are combined by plain addition, and centroid and variance can be read from the result at any time.

Why this summary is so convenient

- **It is incrementally updatable.** A new point x is absorbed by N ← N + 1, SUM ← SUM + x, SUMSQ ← SUMSQ + x^2 (coordinate by coordinate). The point can then be thrown away. This is exactly "Add Ns, SUMs, SUMSQs" on Slide 51.
- **Two summaries combine by addition** (Figure 4). That is how CS sets are merged (step 4) and how a CS is folded into a cluster in the last round. No old points are consulted.
- **It supports the closeness tests.** The centroid and the σ_i needed by the Mahalanobis test (Slide 55) come straight from N, SUM, SUMSQ.
- **It matches the axis-aligned assumption.** Only per-dimension totals are stored, so only per-dimension variances can be recovered; that is why the clusters must be axis-aligned.

This also answers the question of Slide 80: **the statistics do not have to be computed from scratch in each round; they are updated from their previous values** (see Quiz L5-Q5 below).

Worked example (BFR numbers not examined; understand the idea only) — a tally card in one dimension
Lecture 5 · Slides 48–49 (illustration added by this guide)

Points {1, 2, 3}. N = 3, SUM = 1 + 2 + 3 = 6, SUMSQ = 1 + 4 + 9 = 14. Centroid = 6/3 = 2. Variance = 14/3 − 2^2 = 4.667 − 4 = 0.667; σ = √0.667 = 0.816. Now the point 4 arrives: N = 4, SUM = 10, SUMSQ = 14 + 16 = 30, so the centroid is 2.5 and the variance is 30/4 − 2.5^2 = 7.5 − 6.25 = 1.25. Recomputing from {1, 2, 3, 4} gives the same answer, and no old point was needed.

COMMON MISTAKES

- **SUMSQ is not (SUM)².** For {1, 2, 3}: SUMSQ = 14 but (SUM)² = 36. Square each coordinate first, then add.
- SUM and SUMSQ are **vectors** (one entry per dimension), not single numbers; N is a single number.
- The variance divides by N (not N − 1), as on Slide 49.
- Don't average the standard deviations of two clusters to get the standard deviation of their union; add the N, SUM, SUMSQ and recompute.

Processing one memory load

Lecture 5 · Slides 50–51

Slide 49 ends with "Next step: actual clustering". Slides 50 and 51 give the loop that is run for **each memory load** (each chunk of points read from disk). There are five steps; the first four are repeated for every chunk and the fifth happens only once, after the last chunk.

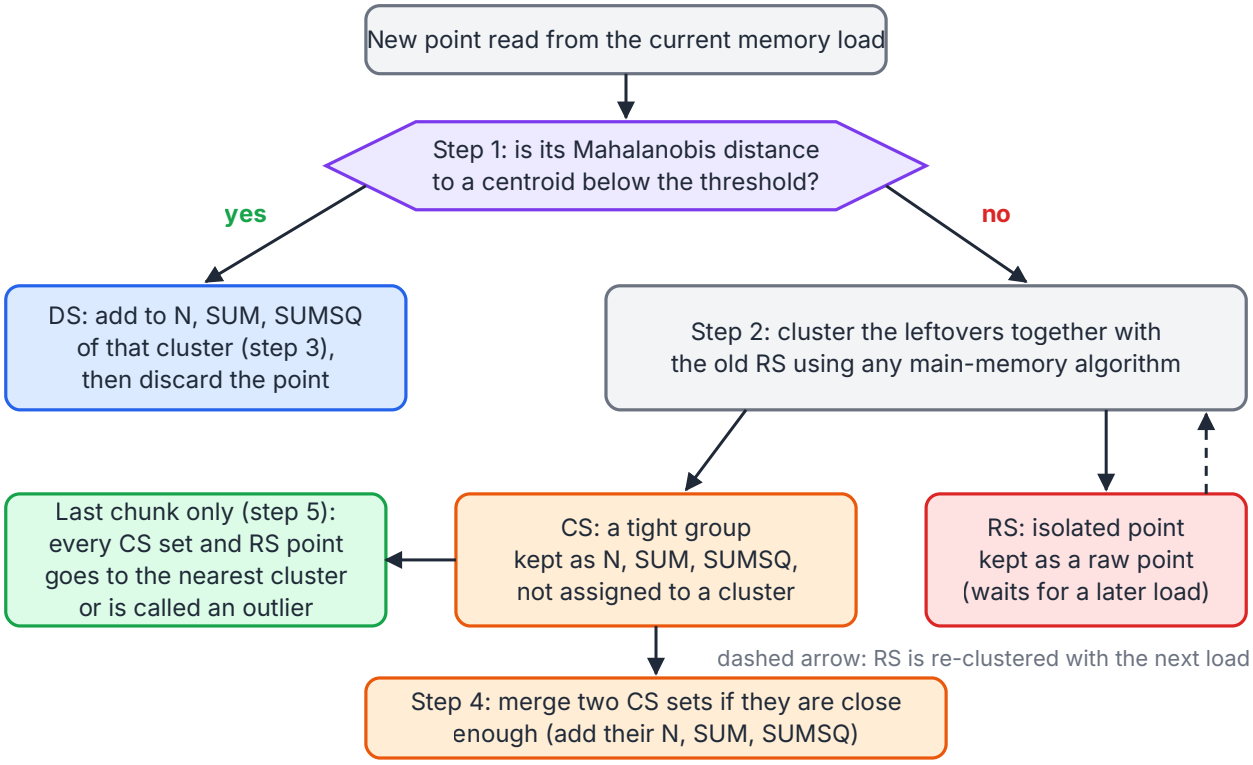


Figure 5: The journey of a point through one BFR round (Slides 50–51; the "step" numbers are the slides' own).

1. **Step 1: add sufficiently close points to the clusters (DS).** Find the points in the chunk that are "sufficiently close" to a cluster centroid, add them to that cluster and to the DS. They are so close to the centroid that they can be summarised and then discarded. (How "sufficiently close" is decided is question Q1 of Slide 53, answered by the Mahalanobis distance on Slides 54–56 below.)
2. **Step 2: cluster the rest.** Use **any main-memory clustering algorithm** (for example *k*-means or hierarchical clustering; BFR treats it as a black box) on the **remaining points together with the old RS**. Clusters go to the CS; outlying points go to the RS. (**Beyond the slides:** in the textbook a group of two or more points counts as a cluster for the CS, and a point left alone stays in the RS.)
3. **Step 3: update the DS statistics.** Adjust the statistics of the clusters to account for the new points: add the Ns, SUMs and SUMSQs. (The new CS groups from step 2 also get their own N, SUM, SUMSQ; the slide only mentions the DS.)
4. **Step 4: consider merging compressed sets in the CS.** If two CS groups are close enough, combine them by adding their statistics. (How to decide is question Q2 of Slide 53, answered on Slides 57–58 below.)
5. **Step 5: last round only.** If this is the last round (the last chunk of data), merge all compressed sets in the CS and all RS points into their nearest cluster, or treat them as outliers.

> **KEY IDEA**
> After every memory load, the only things in memory are: the *k* cluster summaries, the CS summaries, the RS points and the next chunk. All the points that were absorbed into the DS or the CS are gone, yet nothing has been lost that BFR needs.

A feel for the dynamics

The table below is a **made-up illustration** (not from the slides) of how the three sets might develop over a run with $k = 2$.

Moment	DS (two clusters)	CS	RS
Load 1	centroids chosen from this chunk; most points near them absorbed	1 small group found in the leftovers	a few stray points

Load 2	N, SUM, SUMSQ grow by addition; centroids shift slightly	a new group appears; two groups close together are merged	old strays are re-clustered with the new leftovers; two of them form a new CS group
Load 3, ...	same	same	same
Last load	receives the nearest CS sets and RS points	empty	empty (or declared outliers)

SLIDE ERRATUM / OMISSIONS (SLIDES 50–51)

- Slide 50 uses "sufficiently close" without defining it; the definition comes on Slides 54–56.
- Step 5 gives two options, "nearest cluster" or "outliers", without a criterion. **Beyond the slides:** the same distance tests apply: a CS or RS point is merged into the nearest cluster only if it is close enough (for example within the threshold); otherwise it is an outlier.
- On Slide 51 the legend overlaps the last line of step 5 ("as outliers)"); this is a layout flaw only. In practice the DS statistics are updated while points are absorbed in step 1, so "step 3" is bookkeeping done at that moment.

Q1: How close is "close enough"? The Mahalanobis distance

Lecture 5 · Slides 53–56

Slide 53, "A few details...", poses the two open questions of the loop:

- **Q1)** How do we decide if a point is "close enough" to a DS cluster that we will add the point to that cluster?
- **Q2)** How do we decide whether two compressed sets (CS) deserve to be combined into one?

Slide 54 answers Q1: **BFR suggests that the Mahalanobis distance is less than a threshold.** "Put into a cluster (and discard)" means: add the point's numbers into the cluster's N, SUM, SUMSQ, then throw the point away.

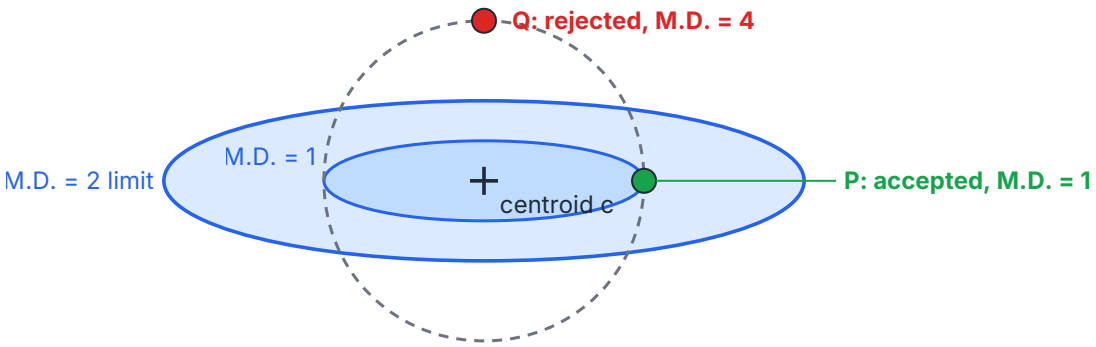
Why not ordinary Euclidean distance?

Lecture 5 · Slides 54–55

Consider a cluster that is long and thin: very spread out in x ($\sigma_x = 4$) and tight in y ($\sigma_y = 1$). A point 4 units away along the long direction is perfectly typical for this cluster. A point 4 units away along the thin direction is very unusual (4 standard deviations). Euclidean distance cannot tell them apart; both are at distance 4. The fix is to measure every direction in **units of that direction's standard deviation**.

One cluster with σx = 4, σy = 1 (axis-aligned)

P and Q are both at Euclidean distance 4 from c (dashed circle).



The accepted region (M.D. < 2) is an axis-aligned ellipse:
long where the cluster is spread out, thin where it is tight.

Figure 6: P and Q have the same Euclidean distance (4) from the centroid, but different Mahalanobis distances (1 and 4). Numbers computed in the table below.

DEFINITION: MAHALANOBIS DISTANCE AS USED IN BFR (SLIDE 55)

For a point $(x_1, \ldots, x_d)$ and a cluster centroid $(c_1, \ldots, c_d)$ ("normalised Euclidean distance from centroid"):

1. Normalise in each dimension: $y_i = (x_i - c_i) / \sigma_i$
2. Take the sum of the squares of the y_i
3. Take the square root: $d(x, c) = \sqrt{\Sigma_{i=1..d} ((x_i - c_i) / \sigma_i)^2}$

where σ_i is the standard deviation of the points in the cluster in the *i*-th dimension.

Reading it. A normalised difference $y_i = 1$ means "one standard deviation away from the centre in dimension *i*". In one dimension the Mahalanobis distance is simply $|x - c| / \sigma$ (how many standard deviations away you are). In several dimensions it is the Euclidean distance of the normalised point. A dimension in which the cluster is tight (small σ_i) counts a lot; a dimension in which it is spread out (large σ_i) counts little.

All the ingredients come from the summary: $c_i = SUM_i/N$ and $\sigma_i = \sqrt{(SUMSQ_i/N - (SUM_i/N)^2)}$ (Slide 49). So the test needs no raw points.

Worked example (BFR numbers not examined; understand the idea only) — Mahalanobis distance for understanding Lecture 5 · Slides 55–56 (numbers added by this guide)

Cluster centroid $c = (10, 5)$, $\sigma = (4, 1)$. (BFR numericals will not be asked; this only shows how the formula behaves.)

Point	Differences (x−c)	Normalised y	M.D. = $\sqrt{(y_1^2 + y_2^2)}$	Euclidean	M.D. < 2 ?
P = (14, 5)	(4, 0)	(1, 0)	1	4.00	**accept**
Q = (10, 9)	(0, 4)	(0, 4)	4	4.00	reject
R = (13, 6)	(3, 1)	(0.75, 1)	$\sqrt{1.5625}$ = 1.25	3.16	**accept**
S = (17, 5)	(7, 0)	(1.75, 0)	1.75	7.00	**accept**
T = (10, 7)	(0, 2)	(0, 2)	2	2.00	reject (not < 2)

Point S is 7 units from the centroid in the ordinary sense but accepted; point T is only 2 units away but rejected. That is the point of the normalisation.

The acceptance rule and its threshold

Lecture 5 · Slide 56

> **RULE (SLIDE 56)**
> Accept a point for a cluster if its M.D. is **<** some threshold; common practice is to specify **M.D. < 2**.

The intuition behind "2" comes from the bell curve of Figure 1: roughly 2 standard deviations covers about 95% of a normal cluster's points. A point beyond that is unusual for this cluster. Points that fail the test for every cluster are not added to the DS; they go on to step 2 (they may become part of a CS or stay in the RS). (**Beyond the slides:** if more than one cluster passes the test, the point is given to the nearest one.)

> **SLIDE ERRATUM (NAMING AND A CAVEAT)**
> - **Name.** The slide's heading is "Mahalanobis distance", but its subtitle says "normalized Euclidean distance". They coincide here because BFR assumes axis-aligned clusters (no correlation between dimensions). The general Mahalanobis distance uses the full covariance matrix, $\sqrt{(x-c)^T S^{-1} (x-c)}$. For BFR you use the slide's formula.
> - **σ = 0.** If a cluster has N = 1 or a constant coordinate, $\sigma_i = 0$ and the formula divides by zero. The slides do not address it.
> - **Dimension (Beyond the slides).** "M.D. < 2" is only sensible in low dimension. For a normal cluster the average squared M.D. of its members equals d, so a typical member has M.D. ≈ $\sqrt{d}$. The share of a true cluster's points that a fixed threshold of 2 would keep is 95.4% for d = 1, 86.5% for d = 2, 59.4% for d = 4 and only 5.3% for d = 10. A threshold that grows with the dimension, such as $2\sqrt{d}$, keeps 95.4%, 98.2%, 99.7% and about 100% for those four cases. (Computed with the chi-square distribution.) In the exam, quote the slide's "M.D. < 2".

Q2: Should two compressed sets be combined?

Lecture 5 · Slides 57–58

Step 4 of the loop asks whether two CS groups (the slides call them "CS clusters", "CS subclusters" and "compression sets" interchangeably) actually belong together. Slide 57 gives two ways to decide.

Way 1: variance of the combined subcluster

Lecture 5 · Slide 57

"Compute the variance of the combined subcluster. N, SUM and SUMSQ allow us to make that calculation quickly." Add the two summaries ($N = N_1 + N_2$, $SUM = SUM_1 + SUM_2$, $SUMSQ = SUMSQ_1 + SUMSQ_2$) and read the variance with the Slide 49 formula. If the merged group is still tight (variance not much larger than the parts), merge them. The slide gives no numeric threshold for this variance.

Worked example (BFR numbers not examined; understand the idea only) — merge or not?
Lecture 5 · Slide 57 (numbers added by this guide)

One dimension. CS_1 = {9, 10, 11}: N = 3, SUM = 30, SUMSQ = 302 (σ = 0.816). Option A, CS_2 = {11, 12, 13}: N = 3, SUM = 36, SUMSQ = 434. Merged: N = 6, SUM = 66, SUMSQ = 736; centroid 11; variance 736/6 − 11² = 122.667 − 121 = 1.667, σ = 1.29: still tight, so merge. Option B, CS_2 = {19, 20, 21}: N = 3, SUM = 60, SUMSQ = 1202. Merged: N = 6, SUM = 90, SUMSQ = 1504; centroid 15; variance 1504/6 − 15² = 250.667 − 225 = 25.667, σ = 5.07: far more spread out than either part, so do not merge.

Way 2: Mahalanobis distance between the two centroids

Lecture 5 · Slides 57–58

"When merging compression sets you would typically use the Mahalanobis distance between the centroids of the two compression sets to decide whether or not they should be merged. A threshold like MD < 2 ensures that the compression sets are close enough." Because there are two clusters, each with its own σ, the distance

needs a single σ per dimension; Slide 58 offers two choices for it (the two clusters are written with superscripts (1) and (2)).

FORMULAS (SLIDE 58)

(1) $d(c_1, c_2) = \sqrt{\Sigma_{i=1..d} ((c_{1i} - c_{2i}) / \sigma_i^{pool})^2}$

(2) $\sigma_i^{pool} = \sqrt{((\sigma_i^{(1)})^2 + (\sigma_i^{(2)})^2) / 2}$ (the root of the average of the two variances)

or

(3) $\sigma_i^{pool} = \max(\sigma_i^{(1)}, \sigma_i^{(2)})$ (the larger of the two standard deviations)

Merge if $d(c_1, c_2)$ is below the threshold (e.g. 2). The root-mean-square in (2) is never larger than the max in (3), so option (3) gives the **smaller** distance and merges **more** readily. The slide does not remark on this.

Worked example (BFR numbers not examined; understand the idea only) — the two pooled σ choices
Lecture 5 · Slide 58 (numbers added by this guide)

One dimension: CS_1 has centroid 10 and $\sigma^{(1)} = 2$; CS_2 has centroid 13.5 and $\sigma^{(2)} = 1$. Gap between centroids = 3.5.

Pooled σ	Value	Distance = 3.5 / σ^{pool}	Merge at threshold 2?
(2) root of the average variance	$\sqrt{(4 + 1)/2} = 1.581$	2.21	no
(3) maximum	max(2, 1) = 2	1.75	**yes**

SLIDE ERRATUM / OMISSIONS (SLIDES 57–58)

- The first bullet of Slide 57 (variance of the combined set) and the last two (Mahalanobis distance of the centroids) are two different criteria; the slide does not say which to use or whether they are alternatives. State both in an answer.
- No symbols on Slide 58 are defined in words (c_{1i}, $\sigma_i^{(1)}$, σ_i^{pool}). Also, the pooled variance in (2) is a plain average of the two variances; it ignores the sizes N_1, N_2 (the textbook pooled variance would weight by size). The slide's version is the one to use.
- σ = 0 (a CS with a single point or constant coordinate) is not handled, and "ensures ... close enough" is loose wording: the threshold *defines* "close enough".

BFR in one page

Lecture 5 · Slides 43–58, 78–81, 87

BFR SUMMARY (THEORY ONLY; NO BFR NUMERICALS ARE EXAMINED)

- **What:** Bradley–Fayyad–Reina, a variant of *k*-means for very large, disk-resident data. Memory O(clusters), not O(data).
- **Assumes:** each cluster is normally distributed around its centroid in Euclidean space = an axis-aligned ellipse (σ may differ per axis; axes must line up with the space's axes; tilted ellipses and rings are not allowed).
- **Data flow:** read the data in chunks (memory loads); choose *k* initial centroids from the first load (e.g. *k* random points); summarise old points by simple statistics.
- **Three sets:** DS (close to a centroid: summary only), CS (tight groups far from every centroid: summary only, not assigned), RS (isolated points: kept raw).
- **Summary of a cluster:** N, SUM, SUMSQ = 2*d* + 1 numbers. Centroid = SUM_i/N; variance = $SUMSQ_i/N - (SUM_i/N)^2$; σ = √variance. Updated by addition, never from scratch.
- **Per memory load:** (1) points close enough go to the DS; (2) cluster the rest + old RS: clusters → CS, outliers → RS; (3) update DS statistics (add Ns, SUMs, SUMSQs); (4) consider merging CS sets; (5) on the last load, merge CS and RS into the nearest cluster or call them outliers.
- **Q1 (close enough?):** Mahalanobis distance = $\sqrt{\Sigma((x_i - c_i)/\sigma_i)^2}$; accept if M.D. < threshold (common practice 2, about 2 standard deviations).
- **Q2 (merge two CS?):** variance of the combined set (from N, SUM, SUMSQ) is small, or M.D. between their centroids (pooled σ = RMS of the two, or the max) is below a threshold.

Try it yourself first. Close the notes, write the BFR steps from memory as a numbered list, and decide for each of the three drawn ellipses whether BFR may handle it; then compare with the model answer.

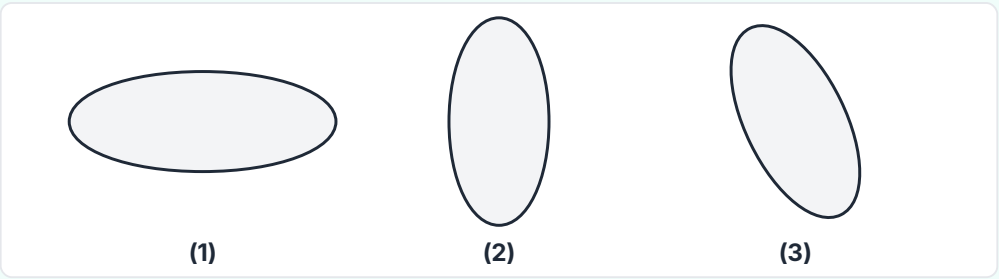
Quiz L5-Q4 BFR: steps of the algorithm and allowed cluster shapes (theory) Lecture 5 · Slides 78–79

QUESTION (AS ON THE SLIDES)

Quiz: BFR Algorithm (BFR = Bradley–Fayyad–Reina clustering algorithm). Slide 78 asks two questions and draws three ellipses; Slide 79 ("Ans") gives the slide's short answer.

(a) Describe in steps how the BFR algorithm works.

(b) Which of the following three shapes are allowed for clusters in the BFR algorithm?



The three shapes of Slide 78 (redrawn), numbered (1), (2), (3) by us from left to right: a wide ellipse, a tall ellipse and a tilted ellipse (top leaning left, bottom to the right).

Slide 79 says only: "For steps of BFR see lecture slides" (these are Slides 44–58) and "The clusters in data for which the BFR algorithm may be used can have standard deviations that differ along different axes, but the axes of the cluster must align with the axes of the space"; its picture labels the three shapes **OK, OK, Not OK**. Scope: this is a **theory** question (no code and no BFR numericals will be asked), so the answer below is written the way you should write it in the exam.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

> **BEGINNER INTUITION — SORTING A BILLION PHOTOS WITH A SMALL DESK**
> Imagine sorting a billion photos into k albums when your desk holds only a few thousand photos at a time, so you work chunk by chunk. For each album you keep a **tally card** (how many photos, plus a few running totals) instead of the photos themselves. A photo that clearly fits an album is added to that album's card and then thrown away (this is the **DS**). Photos that look alike but fit no album yet get their own "pile card" (the **CS**). Odd single photos stay loose on the desk (the **RS**). At the very end you decide where the pile cards and the loose photos go. That is BFR: the desk only ever holds the cards, a few loose photos and one chunk — never the whole collection.

Part (a) The steps of the BFR algorithm Lecture 5 · Slides 43–58

Plan of the answer: purpose and set-up (steps 1–4), then the loop that is run for every chunk (steps 5–9 are the slides' own "steps 1–4"), then the last round (step 10, the slides' "step 5"); steps 11–12 put it together and explain the memory saving.

1. **Purpose and assumptions (Slides 43–44). Idea:** ordinary k-means (Slides 23–24) assigns every point to its nearest centroid, recomputes the centroids and repeats, so it must look at *all* the points again and again. If the data are so large that they sit on disk and do not fit in main memory (RAM), that is impractical. **BFR** (Bradley–Fayyad–Reina; Slide 43 is the section title "Extension of k-means to large data") is *"a variant of k-means designed to handle very large (disk-resident) data sets"* (Slide 44). Say these things in your answer:
   - **Assumption:** clusters are **normally distributed around a centroid** in a **Euclidean space**, and the dimensions are independent, so each cluster is an **axis-aligned ellipse** (Slide 44; see part (b)).
   - **Goal:** an efficient way to **summarise** clusters so that the memory required is **O(clusters), not O(data)** (Slide 44).
2. **Read the data in chunks and choose the k initial centroids (Slide 45).** Points are read from disk into main memory **in chunks** (one chunk = a **memory load**); most points from previous memory loads are summarised by **simple statistics** instead of being kept. To begin, from the **initial (first) memory load** select the initial k centroids "by some sensible approach", for example **take k random points** (the "..." on the slide means that other approaches exist). **Why:** exactly as in k-means, the algorithm needs k starting centres; k is fixed in advance and each centre starts one cluster (in the DS, next step).
   Beyond the slides: another sensible choice is the farthest-point rule (pick one point at random, then $k-1$ others, each as far as possible from those already chosen: Slide 23, practised in the quiz on Slides 76–77); in practice the first load is often also clustered with an ordinary in-memory algorithm so that every cluster starts with real statistics (a single seed point has no spread yet). BFR makes **one pass** over the data: each point is read from disk once and, once absorbed into a summary, never needed again.
3. **Know the three sets of points: DS, CS, RS (Slides 46–47).** BFR keeps track of three sets. **Idea:** a point is either clearly part of a known cluster (DS), or part of a tight little group that belongs to no known cluster *yet* (CS), or a loner (RS).

Set	Meaning (slide wording)	What stays in main memory	Assigned to a cluster?
DS — Discard set	Points close enough to a centroid to be summarised.	Only the **summary** (N, SUM, SUMSQ) of each cluster; the points themselves are thrown away.	**Yes — to that cluster**
CS — Compression set	Groups of points that are close together but not close to any existing centroid.	Only a **summary** (N, SUM, SUMSQ) of each group.	No — summarised but not assigned to any cluster
RS — Retained set	Isolated points waiting to be assigned to a compression set.	The **points themselves**, exactly as they are (raw coordinates).	No

Figure 1 shows the three kinds of points in the style of the "galaxies" picture on Slides 47 and 52: the big ellipses are DS clusters (with their centroid +), the small ellipses are CS sets, the loose dots are RS points.

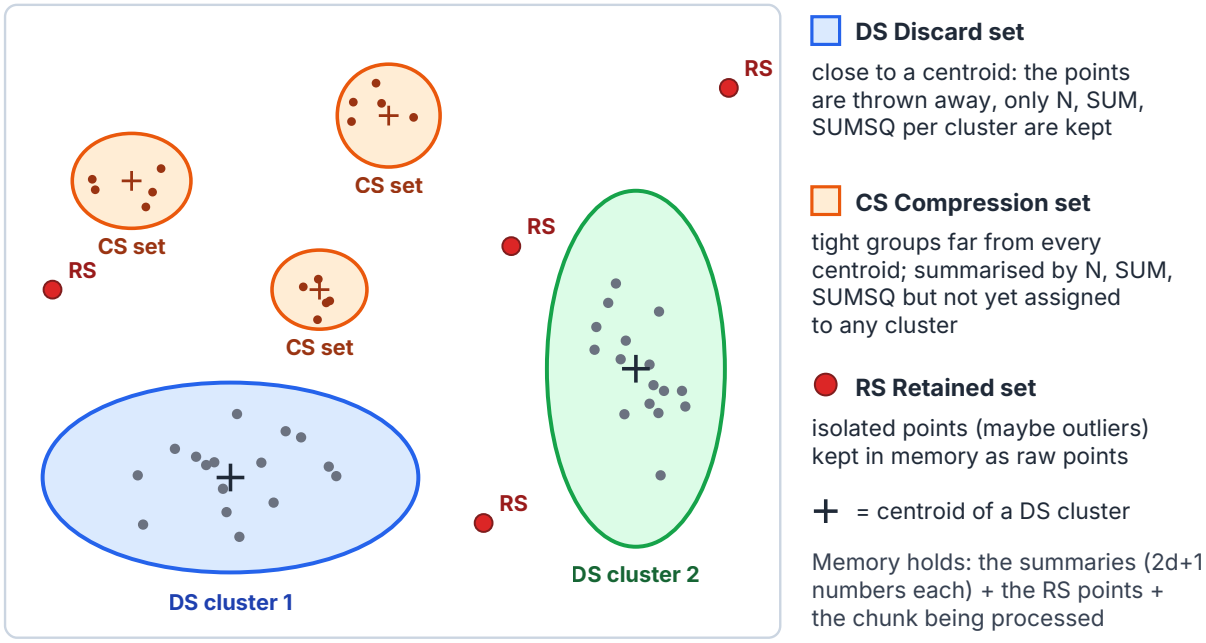


Figure 1. The three kinds of points in BFR, in the style of the "galaxies" picture of Slides 47 and 52 (the slide shows one DS cluster; two are drawn here because BFR keeps k of them). The grey dots inside the DS and CS ellipses stand for points that were summarised and are no longer stored; only the RS holds raw points.

4. **What is stored: the summary statistics N, SUM, SUMSQ (Slides 48–49).** For each DS cluster (and, in the same way, each CS group) BFR stores only: N, the number of points; the vector **SUM**, whose i-th component is the sum of the coordinates of the points in dimension i; the vector **SUMSQ**, whose i-th component is the sum of the squares of the coordinates in dimension i. That is $1 + d + d$ = **$2d + 1$ numbers** (d = number of dimensions) **for a cluster of any size** (Slide 49). From them we get, without any raw point:

> **FORMULA — WHAT N, SUM, SUMSQ GIVE US (SLIDE 49)**
> **Centroid** in dimension i: $c_i = SUM_i / N$. **Variance** in dimension i: $\sigma_i^2 = SUMSQ_i / N - (SUM_i / N)^2$.
> **Standard deviation**: $\sigma_i = \sqrt{(\sigma_i^2)}$.

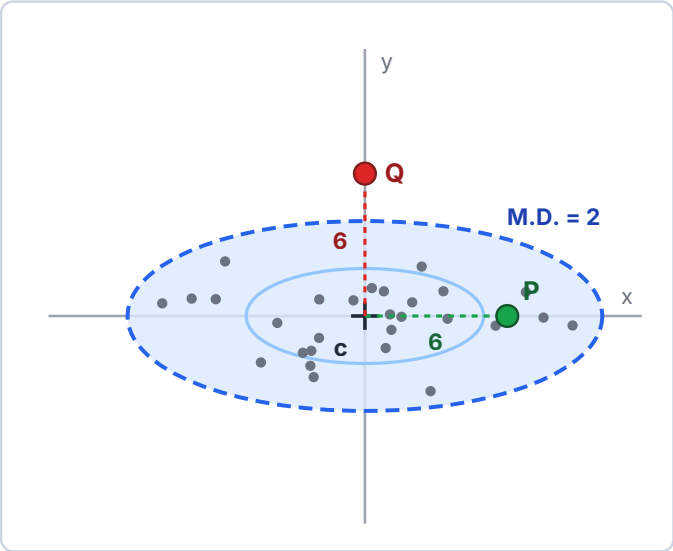
Why the variance formula is right: variance = average of $(x - c)^2$. Expanding, $(1/N)\sum(x - c)^2 = (1/N)\sum x^2 - 2c \cdot (1/N)\sum x + c^2 = SUMSQ/N - 2c^2 + c^2 = SUMSQ/N - c^2$, and $c = SUM/N$.

Why these three numbers are enough: they tell us *where* the cluster is (centroid) and *how spread out* it is along each axis (σ_i) — exactly what BFR needs for its closeness test. And they are **additive**: adding a point adds 1 to N, its coordinates to SUM and their squares to SUMSQ; merging two groups just adds their Ns, SUMs and SUMSQs. So the statistics are **never recomputed from scratch** (this is what Slide 81 stresses, in the quiz on Slides 80–81).
Sanity check of the idea (one dimension, not an exam numerical): group A = {1, 2, 3} has N = 3, SUM = 6, SUMSQ = 14; group B = {10, 12} has N = 2, SUM = 22, SUMSQ = 244. Adding the statistics gives N = 5, SUM = 28, SUMSQ = 258, so the merged centroid is 28/5 = 5.6 and the variance is 258/5 − 5.6² = 51.6 − 31.36 = 20.24 — exactly what the five raw numbers {1, 2, 3, 10, 12} give.

5. **Loop, slide step 1: absorb the points that are close enough into the DS (Slide 50).** Now process the data one memory load (chunk) at a time. **Slide step 1:** find those points that are "**sufficiently close**" to a cluster centroid and add them to that cluster and to the DS. These points are so close to the centroid that they can be summarised and then **discarded**: their values are added into the cluster's N, SUM, SUMSQ and the raw points disappear from memory. **Why:** this is how the data shrink — most points end here. (*Natural reading:* a point goes to the DS cluster with the smallest distance, provided that distance passes the test of the next step.)
6. **How "close enough" is decided: the Mahalanobis distance (Q1, Slides 53–56).** Slide 53 asks (Q1): how do we decide that a point is close enough to a DS cluster? Slide 54: BFR suggests that *the Mahalanobis distance (M.D.) is less than a threshold*. Slide 55 gives the formula and calls it the *normalised Euclidean distance* from the centroid.

> **FORMULA — MAHALANOBIS DISTANCE (SLIDES 55–56)**
> For a point $\mathbf{x} = (x_1, \ldots, x_d)$ and a cluster centroid $\mathbf{c} = (c_1, \ldots, c_d)$: (1) normalise in each dimension, $y_i = (x_i - c_i) / \sigma_i$; (2) take the sum of the squares of the y_i; (3) take the square root.
>
> $$\text{M.D.}(\mathbf{x}, \mathbf{c}) = \sqrt{\sum_{i=1}^{d} \left((x_i - c_i) / \sigma_i \right)^2}$$
>
> σ_i = standard deviation of the cluster's points in dimension i; both c_i and σ_i come from the stored N, SUM, SUMSQ (previous steps). **Rule (Slide 56): accept the point for the cluster if M.D. < some threshold; common practice is M.D. < 2.**

A point that is not accepted by any DS cluster is *not* thrown away: it goes on to the next step.
Why divide by σ_i? Clusters are not equally spread in every direction. M.D. measures the distance in units of that cluster's own standard deviation, one dimension at a time, so "close" means "within about 2 standard deviations" on the cluster's own scale. Figure 2 shows two points that are equally far from the centroid in plain Euclidean distance, yet only one is accepted — and that the accepted region (M.D. < 2) is itself an **axis-aligned ellipse**.



Illustration, not an exam numerical
Cluster centroid c, spread (σx, σy) = (5, 2): wide in x, narrow in y, axes aligned.

P = c + (6, 0): M.D. = 6/5 = 1.2 accept
Q = c + (0, 6): M.D. = 6/2 = 3.0 reject

P and Q are both 6 units from c in plain Euclidean distance, yet only P is 'close' for this cluster. Dividing by σ fixes that.

The region M.D. < 2 is an ellipse with half-axes 2σx = 10 and 2σy = 4, lined up with the axes of the space.

Figure 2. Why BFR divides each coordinate by its own standard deviation. The set of points with M.D. < 2 (dashed) is an ellipse whose axes line up with the axes of the space; the light inner ellipse is M.D. = 1.

How strict is "M.D. < 2"? In one dimension, Slide 44's bell curve shows 34.13% + 13.59% = 47.72% of a normal cluster on each side of its centre up to 2σ, so about **95%** of the cluster's own points pass. *Beyond the slides:* in more dimensions fewer pass (M.D.² of a normal point follows a chi-square law with d degrees of freedom, so a typical point has M.D. around $\sqrt{d}$): about 86.5% for $d = 2$, 73.9% for $d = 3$ and only about 5% for $d = 10$. So "2" is a rule of thumb and the threshold is a tuning choice: larger = more points absorbed into the DS, smaller = more points left for the CS/RS. Also *beyond the slides:* the textbook Mahalanobis distance uses a full covariance matrix; with only the per-axis σ_i (no covariances) it reduces to the formula of Slide 55 — which is exactly why BFR needs axis-aligned clusters (part (b)).

7. **Loop, slide step 2: cluster the leftovers together with the old RS (Slide 50).** Use **any main-memory clustering algorithm** (the slide leaves the choice open, e.g. hierarchical clustering or k-means) to cluster **the remaining points** (those not close enough to any DS centroid) **and the old RS**. **Clusters go to the CS** — each is summarised by N, SUM, SUMSQ and is *not* assigned to any of the k clusters — and **outlying points go to the RS** and are kept as raw points. **Why:** far-away points must not be forced into a wrong cluster, yet tight groups of them can already be compressed; and the old RS points get a fresh chance, because the new chunk may have brought their neighbours. *Beyond the slides:* a "cluster" of one single point is just an outlier (its variance is 0), so it stays in the RS; only groups of two or more points become CS sets.
8. **Loop, slide step 3: update the DS statistics (Slide 51).** "**DS set:** adjust statistics of the clusters to account for the new points — add Ns, SUMs, SUMSQs." **Why:** each DS cluster has just become bigger, so its N, centroid (SUM/N) and spreads σ_i change. Because the statistics are additive, the update only adds the new points' contributions — no old point is needed. (In practice the addition is done as each point is absorbed in slide step 1; the slide lists it separately. New CS groups from slide step 2 are stored with their own N, SUM, SUMSQ in the same way.)
9. **Loop, slide step 4: consider merging compressed sets in the CS (Q2, Slides 51, 53, 57–58).** Slide 53 asks (Q2): how do we decide whether two CS sets deserve to be combined? Small groups that now lie close together should become one group, otherwise the number of CS sets (and the memory used) would keep growing. **How to decide (Slide 57):** compute the **variance of the combined subcluster**; N, SUM and SUMSQ of the two sets simply add, so this can be done quickly without any raw point. In practice one typically uses the **Mahalanobis distance between the centroids** of the two compression sets, and a threshold like **M.D. < 2** ensures that the sets are close enough. Because the two sets have different spreads, Slide 58 uses a **pooled** standard deviation in each dimension. If the sets are merged, the new set's statistics are the sums of the two sets' statistics.

FORMULA — SHOULD TWO CS SETS BE COMBINED? (SLIDE 58)

$$d(c_1, c_2) = \sqrt{\sum_{i=1}^{d} \left((c_{1i} - c_{2i}) / \sigma_i^{pool} \right)^2}$$

$$\sigma_i^{pool} = \sqrt{\left((\sigma_i^{(1)})^2 + (\sigma_i^{(2)})^2 \right) / 2} \quad or \quad \sigma_i^{pool} = \max(\sigma_i^{(1)}, \sigma_i^{(2)})$$

c_1, c_2 = centroids of the two CS sets; $\sigma_i^{(1)}$, $\sigma_i^{(2)}$ = their standard deviations in dimension i. Merge if $d(c_1, c_2) <$ threshold (e.g. 2). The pooled σ lets the two sets share one yardstick.

10. **Loop, slide step 5: the last round (Slide 51).** "If this is the last round (last chunk of data), **merge all compressed sets in the CS and all RS points into their nearest cluster** (or treat them as outliers)." **Why:** after the last chunk no further points can arrive to give the CS sets and RS points new neighbours, so their fate must be decided: attach each to the nearest of the k clusters, or, if it is far from everything, declare it an outlier that belongs to no cluster. *Beyond the slides:* the points inside the DS were discarded, so the result of BFR is the k cluster summaries (N, SUM, SUMSQ, hence centroid and variance of each cluster); to find out which cluster each individual point belongs to, one more pass over the data would be needed.
11. **Putting the loop together (Slides 50–51).** Figure 3 shows what happens to one memory load. The whole algorithm is: **initialise** (steps 1–2), then **repeat the loop for every chunk** (steps 5–9), then **finish with the last round** (step 10).

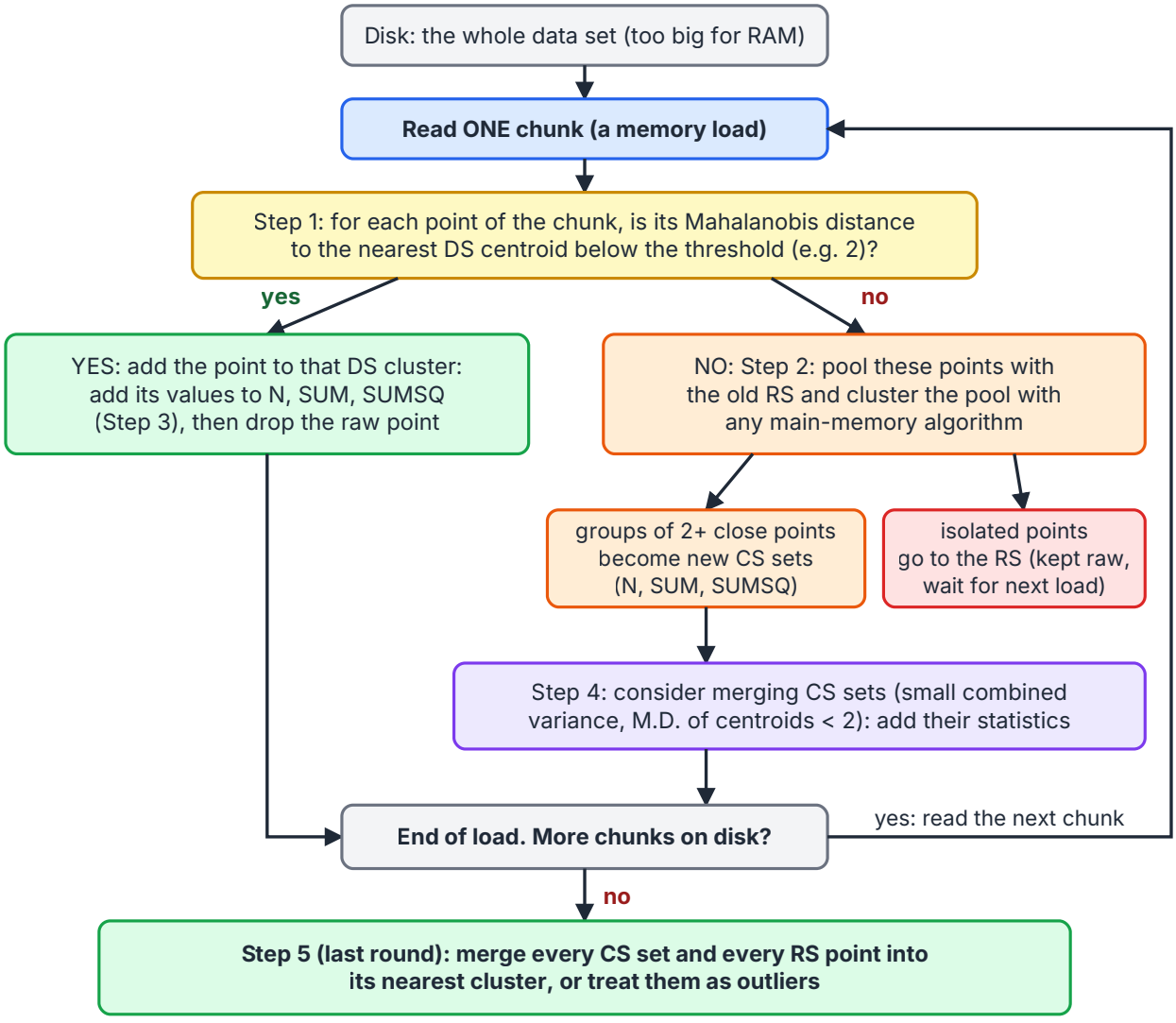


Figure 3. One round of BFR (Slides 50–51; the step numbers inside the boxes are the slides' own). Steps 1–4 are repeated for every memory load; step 5 happens once, after the last load.

12. **Why the memory is O(clusters), not O(data) (Slide 44).** After every memory load, main memory holds only the items in the table — none of which grows with the total number of points n, whereas plain k-means needs all $n \cdot d$ coordinates.

What BFR keeps in main memory	How many numbers	Grows with n?
DS: k clusters, each (N, SUM, SUMSQ)	$k \times (2d + 1)$	**no**
CS: m groups, each (N, SUM, SUMSQ)	$m \times (2d + 1)$	**no (merging keeps m small)**
RS: r isolated points, kept raw	$r \times d$	**no (r stays small)**
the memory load being processed: c points	$c \times d$	**no (the chunk size is our choice)**

The letters m, r, c are ours, not the slides'. *Illustration with made-up sizes (not an exam numerical):* k = 10 clusters, d = 100 dimensions: the DS needs only $10 \times (2 \cdot 100 + 1) = 2{,}010$ numbers whether the data set has 10^6 or 10^9 points, whereas storing 10^9 points of 100 numbers each would need 10^{11} numbers. This relies on the RS staying small and on the CS sets being merged, which is exactly what steps 7 and 9 do.

Part (b) Which shapes are allowed? Lecture 5 · Slides 44, 78–79

1. **The rule (Slides 44 and 79).** BFR assumes each cluster is **normally distributed around its centroid**. A normal distribution has ellipse-shaped contours, and its standard deviation **may be different in every dimension** (so the ellipse may be stretched), but the cluster's axes must **align with the axes of the space**: clusters are **axis-aligned ellipses**. A **circle** (equal standard deviation in every dimension) is also allowed — it is the special case where the axes are equal (the picture on Slide 44 even shows a roundish cluster beside two stretched ones).
2. **Test each of the three drawings (Slide 78 → Slide 79).** Look at the direction of the ellipse's own long and short axes and compare them with the horizontal and vertical axes of the space (Figure 4).

Shape on Slide 78	Its own axes versus the axes of the space	What it means for the data	Allowed?
(1) wide (horizontal) ellipse	long axis along x, short axis along y: both coincide with the axes of the space	$\sigma_x > \sigma_y$; the dimensions are independent	**OK ✓**
(2) tall (vertical) ellipse	long axis along y, short axis along x: both coincide with the axes of the space	$\sigma_y > \sigma_x$; the dimensions are independent	**OK ✓**
(3) tilted ellipse	long axis points diagonally: neither of its axes lines up with the axes of the space	x and y are correlated (one grows as the other shrinks)	NOT OK ×

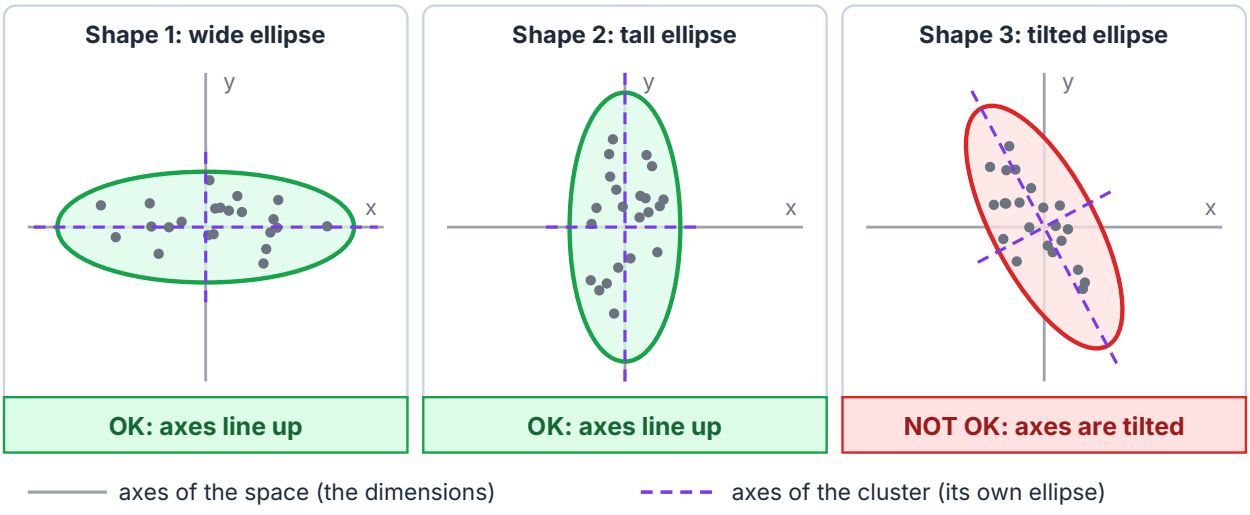


Figure 4. Allowed (shapes 1 and 2) and not allowed (shape 3) clusters in BFR: the cluster's own axes (purple, dashed) must coincide with the axes of the space (grey).

3. **Why a tilted cluster is not allowed (the reason that earns the mark).** The slide's reason: the standard deviations may differ along different axes, but the axes of the cluster must align with the axes of the space. *Beyond the slides* (why the algorithm needs this): the textbook (*Mining of Massive Datasets*, Chapter 7) says that the cluster's **dimensions must be independent**: a cigar-shaped cluster is fine, but the cigar must not be rotated off the axes. Two reasons follow directly from the algorithm:
   - **The stored summary cannot describe a tilt.** N, SUM, SUMSQ hold one centre and one spread *per dimension* ($2d + 1$ numbers). A tilted ellipse means that the dimensions are correlated, and describing that would need extra sums of products $x_i \cdot x_j$ for every pair of dimensions ($d(d-1)/2$ more numbers per cluster), which BFR does not keep.
   - **The Mahalanobis test would use the wrong shape.** The M.D. formula divides each coordinate by its own σ_i and has no cross terms, so its acceptance region is always an axis-aligned ellipse (Figure 2).

     Figure 5 shows what happens on a tilted cluster: the axis-aligned region is too "fat" and covers empty corners, so a point such as P is accepted (M.D. = 1.70) although it is 3.79 standard deviations away measured along the real tilted shape (fewer than 1 in 1000 genuine members lie that far out); and genuine members near the tips of the long axis fall outside the region and are wrongly rejected. The same argument holds in more dimensions: the ellipsoids must have their axes parallel to the coordinate axes.

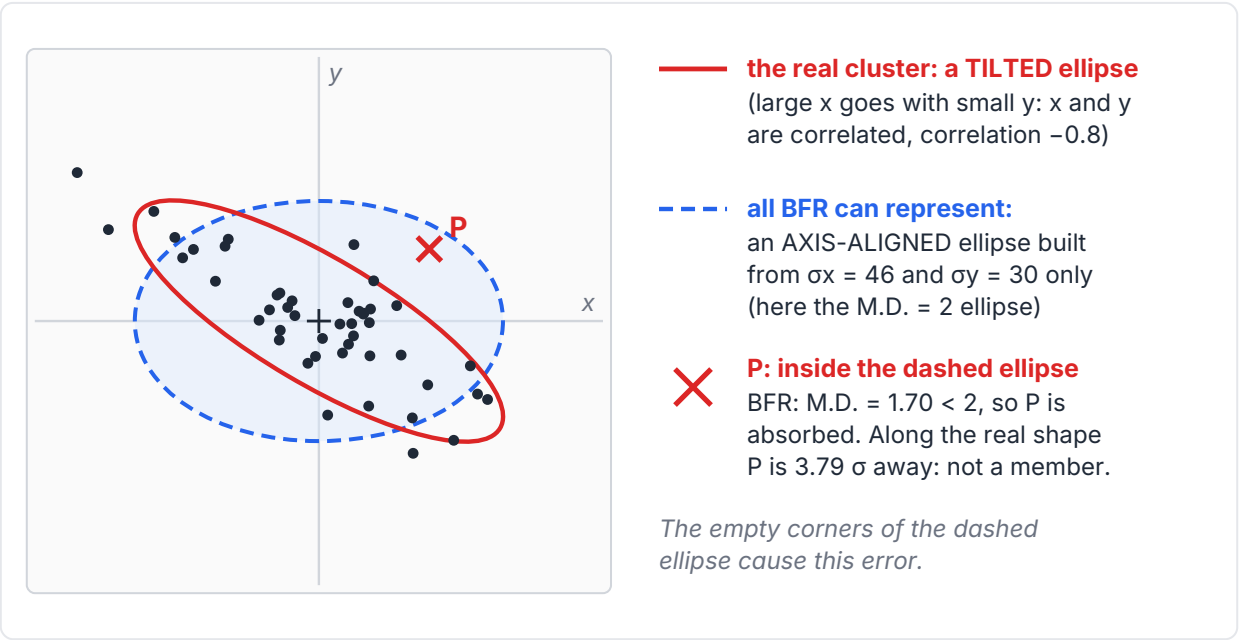


Figure 5. Why shape (3) is not allowed (illustration for understanding, not an exam numerical). The dashed axis-aligned ellipse is the best BFR can do for a tilted cluster: it wrongly accepts points such as P in the empty corners, and it leaves out genuine members near the two tips of the long axis.

COMMON MISTAKE

- Saying that all three sets are stored as raw points — **only the RS is raw**; DS and CS keep just N, SUM, SUMSQ.
- Saying that CS points belong to a cluster — they are summarised but **not assigned** to any cluster until they are merged or until the last round.
- Writing the variance as SUMSQ/N — you must **subtract (SUM/N)²**; and $SUMSQ_i$ is $\sum x_i^2$ (sum of squares), not $(SUM_i)^2$.
- Using plain Euclidean distance for "close enough" — BFR uses the **Mahalanobis (normalised) distance**; "M.D. < 2" means closer than 2 *standard deviations*, not 2 units.
- Forgetting the last round (what happens to the CS and RS at the end), or thinking that k can change during the run.
- Saying that BFR keeps all the points in memory or re-scans the data many times.
- Part (b): answering "all ellipses are allowed" or "only circles are allowed". Ellipses are allowed **only if axis-aligned**; circles are just the special case of equal spread.

EXAM TIP

For "describe in steps how BFR works" write a **numbered list** and use the lecture's keywords: *very large / disk-resident, chunks (memory loads), k initial centroids, DS / CS / RS, N, SUM, SUMSQ (2d + 1 numbers), Mahalanobis distance < threshold (e.g. 2), main-memory clustering of the remaining points + old RS, clusters to CS / outliers to RS, update statistics by adding, merge CS sets, last round: merge CS and RS into the nearest cluster or treat as outliers.* For the shapes question give the verdict for **each** drawing (OK, OK, Not OK) **and** a one-line reason (axes of the cluster must align with the axes of the space; standard deviations may differ per axis).

SLIDE ERRATUM (WORDING ON SLIDE 87)

The chapter summary on Slide 87 says BFR "assumes clusters are normally distributed about the axes". That is loosely worded (it echoes the textbook's chapter summary): the precise assumption (Slide 44) is that clusters are **normally distributed around a centroid** and are **axis-aligned ellipses**. Use the Slide 44/79 wording in the exam.

Final answer:
(a) BFR is a k-means variant for very large, disk-resident data; it assumes normally distributed, axis-aligned elliptical clusters in a Euclidean space and keeps only O(clusters), not O(data), in memory.
1. Read the data from disk in chunks (memory loads); take the k initial centroids from the first load (e.g. k random points).
2. Keep three sets: DS (points close enough to a centroid; only N, SUM, SUMSQ per cluster kept, the points are discarded), CS (tight groups far from every centroid; summarised by N, SUM, SUMSQ, not assigned to a cluster), RS (isolated points, kept raw). Each summary = $2d + 1$ numbers: centroid = SUM/N, variance = SUMSQ/N – (SUM/N)².
3. For each memory load: (i) every point whose Mahalanobis distance to a centroid is below a threshold (e.g. 2) joins that cluster (DS) and is discarded; (ii) cluster the remaining points together with the old RS using any main-memory algorithm: clusters go to the CS, outlying points to the RS; (iii) update the DS statistics by adding Ns, SUMs, SUMSQs; (iv) consider merging CS sets (small variance of the combined set / M.D. between their centroids < 2).
4. After the last chunk: merge all CS sets and all RS points into their nearest cluster, or treat them as outliers.
(b) Wide ellipse: OK. Tall ellipse: OK. Tilted ellipse: NOT OK. Reason: the standard deviations may differ along different axes, but the axes of the cluster must align with the axes of the space (BFR keeps only per-dimension statistics, no covariance).

Where the marks come from (our own breakdown of the separable ideas an examiner can credit; the course has not published a mark scheme): **(a)** (1) purpose and assumptions: k-means variant for very large data, normal / axis-aligned clusters, memory O(clusters); (2) chunk-by-chunk processing and the k initial centroids from the first load; (3) DS, CS and RS named and defined correctly; (4) the summary statistics N, SUM, SUMSQ, $2d + 1$ numbers, centroid and variance formulas; (5) close points (Mahalanobis distance < threshold) go to the DS and are discarded; (6) remaining points plus old RS clustered with a main-memory algorithm, clusters to the CS, outliers to the RS; (7) DS statistics updated by adding; (8) CS sets merged (variance of the combined set / M.D. between centroids); (9) last round: CS and RS merged into the nearest cluster or treated as outliers. **(b)** (10) shape 1 allowed; (11) shape 2 allowed; (12) shape 3 not allowed; (13) the reason: axes of the cluster must align with the axes of the space (standard deviations may differ per axis). A partly correct list still earns the marks for every idea it contains.

Slide check: part (b) matches Slide 79 exactly (OK, OK, Not OK, same reason; the three labels are in the picture, not in the slide text). Part (a): Slide 79 only says "For steps of BFR see lecture slides", so the steps above are assembled from Slides 44–58; nothing on those slides is contradicted. Omissions on the slides, filled in above and labelled "Beyond the slides": the other ways to pick the initial centroids (Slide 45 shows only "..."); which main-memory algorithm to use and that a one-point cluster stays in the RS (Slides 50–51); a numeric threshold for the combined variance (Slide 57; Slide 58 gives the pooled-σ M.D. test). Small notes: Slide 55 calls the formula "normalized Euclidean distance" while Slides 54 and 56 call it the Mahalanobis distance (the full Mahalanobis distance has covariance terms; BFR's version has none); Slides 50–51 list the DS-statistics update as step 3 although in practice it happens while points are absorbed in step 1. Slide erratum: only the loose wording on Slide 87 (see the box above); there is no numerical error on these slides.

Try it yourself first. Compute N, SUM, SUMSQ, the centroid, the variance and the standard deviation of a three-point cluster, and say whether the statistics must be recomputed from scratch in each round (this quiz is flagged as not examined; the last question is theory).

Quiz L5-Q5 **BFR summary statistics: N, SUM, SUMSQ, centroid, variance, standard deviation**
Lecture 5 · Slides 80–81

QUESTION (AS ON THE SLIDES)

NOT EXAMINED — BFR NUMERICAL
BFR numerical — you were told this will NOT be asked; solved only so no quiz is missing. You can skip the arithmetic. Do read part (b) and the "Why this is conceptually valuable" box at the end: the *idea* that a cluster is stored as just N, SUM and SUMSQ, which are simply added together round after round, is theory and can be asked.

Quiz: SUM, SUMSQ, Variance (Slide 80; the slide's own answer is on Slide 81). BFR is the Bradley–Fayyad–Reina clustering algorithm.

Suppose a cluster consists of the points `(5, 1)`, `(6, −2)` and `(7, 0)`.

(a) Compute each of the following statistics: **N**, **SUM**, **SUMSQ**, the **centroid**, the **variance** and the **standard deviation**.

(b) Do the statistics have to be computed from scratch in each round of the BFR algorithm?

Background (not part of the question; from Lecture 5 · Slides 48–49): BFR summarises the discard set (DS) of each cluster by N, SUM and SUMSQ. The variance in dimension i is $\mathrm{SUMSQ}_i / N - (\mathrm{SUM}_i / N)^2$ and the standard deviation is its square root.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION

Imagine a shopkeeper who, instead of keeping every receipt, keeps three running tallies: how many sales (N), the total of the amounts (SUM) and the total of the amounts each squared (SUMSQ). From just those tallies she can still say what the average sale was and how spread out the sales were. BFR does the same for a cluster of points: **throw the points away, keep the tally card, and read the centre and the spread off the card.** Our points have $d = 2$ coordinates, so the card holds only $2d + 1 = 5$ numbers (Slide 49): N, two SUM entries and two SUMSQ entries. Here the cluster has just three points so that you can see every number.

Part (a) The statistics of the cluster **Lecture 5 · Slides 48–49, 80–81**

1. **Formulas to use (write these down first; they earn marks).** The points live in $d = 2$ dimensions (first coordinate = dimension 1, second coordinate = dimension 2). Everything is computed *per dimension*.

   FORMULAS (LECTURE 5 · SLIDES 48–49)

   N = number of points in the cluster

   SUM_i = sum of the i-th coordinates of all points

   SUMSQ_i = sum of the *squares* of the i-th coordinates

   $\text{Centroid}_i = \text{SUM}_i / N$

   $$\text{Variance}_i = \frac{\text{SUMSQ}_i}{N} - \left(\frac{\text{SUM}_i}{N}\right)^2$$

   $\text{Standard deviation}_i = \sqrt{(\text{Variance}_i)}$

2. **Set up: list every point with its squared coordinates.** Squaring each coordinate is the only extra work needed for SUMSQ. Squaring happens *before* adding, and the square of a negative number is positive: $(-2)^2 = 4$.

Point	dim 1 value	dim 2 value	dim 1 squared	dim 2 squared
(5, 1)	5	1	$5^2 = 25$	$1^2 = 1$
(6, −2)	6	−2	$6^2 = 36$	$(-2)^2 = 4$
(7, 0)	7	0	$7^2 = 49$	$0^2 = 0$
Column total	**18**	**−1**	**110**	**5**

3. **N = the number of points.** Count the rows: **N = 3**. (It is stored because it is the divisor in every average below.)
4. **SUM = the vector whose i-th entry is the sum of the coordinates in dimension i (Slide 48).** Dimension 1: 5 + 6 + 7 = 18. Dimension 2: 1 + (−2) + 0 = −1. So **SUM = [18, −1]**. It is a vector with one entry per dimension, not a single number.
5. **SUMSQ = the vector whose i-th entry is the sum of the *squares* of the coordinates in dimension i (Slide 48).** Square first, then add. Dimension 1: 25 + 36 + 49 = 110. Dimension 2: 1 + 4 + 0 = 5. So **SUMSQ = [110, 5]**.

   COMMON MISTAKE

   SUMSQ is the sum of the squares, NOT the square of the sum. The square of SUM would be $[18^2, (-1)^2] = [324, 1]$, which is a different thing from SUMSQ = [110, 5]. Square each coordinate first, then add.

6. **Centroid = SUM_i / N in each dimension (Slide 49).** The centroid is the average position of the points, and an average is "total divided by count". Dimension 1: 18 / 3 = 6. Dimension 2: (−1) / 3 = −1/3 ≈ −0.333. So the **centroid = [6, −1/3]** ≈ [6, −0.333]. It is not one of the three given points; a centroid is just their average.

7. **Variance in dimension i = $SUMSQ_i$ / N − (SUM_i / N)² (Slide 49).** In words: "the average of the squares minus the square of the average". The second term is the square of the centroid coordinate from the previous step. Use exact fractions to avoid rounding errors.
   - Dimension 1: 110/3 − (18/3)² = 110/3 − 6² = 110/3 − 108/3 = 2/3 ≈ **0.667**.
   - Dimension 2: 5/3 − (−1/3)² = 5/3 − 1/9 = 15/9 − 1/9 = 14/9 ≈ **1.556** (the slide rounds this to 1.56).

   Check from the definition (the variance is the average squared distance to the centroid). Dimension 1: distances to 6 are −1, 0, 1; squares 1, 0, 1; total 2; divided by 3 gives 2/3 ✓. Dimension 2: distances to −1/3 are 4/3, −5/3, 1/3; squares 16/9, 25/9, 1/9; total 42/9; divided by 3 gives 14/9 ✓. Both routes agree, so the shortcut built only from the tally card is trustworthy.
   Beyond the slides: why the shortcut works. The variance is the average of $(x - \mu)^2$. Expanding the bracket, $(x - \mu)^2 = x^2 - 2\mu x + \mu^2$, and averaging gives (average of x^2) $- 2\mu^2 + \mu^2$ = (average of x^2) $- \mu^2$. The average of x^2 is SUMSQ/N and μ = SUM/N.
8. **Standard deviation = the square root of the variance (Slide 49).**
   - Dimension 1: √(2/3) = √0.6667 ≈ **0.816**.
   - Dimension 2: √(14/9) = √14 / 3 = 3.742 / 3 ≈ **1.247** (the slide rounds this to 1.25).

   So **SD = [0.816, 1.247]**: the cluster is a little more spread out in dimension 2 than in dimension 1.

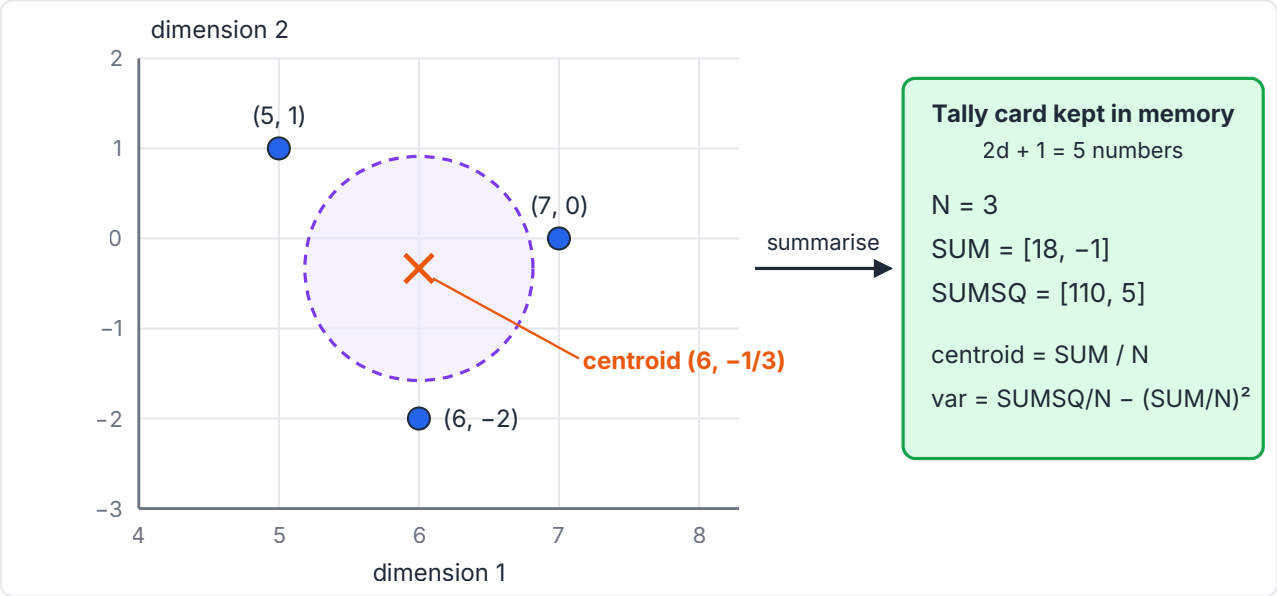


Figure — the three points of the quiz (blue), their centroid (orange cross) and, dashed, the axis-parallel ellipse that lies one standard deviation from the centroid in each dimension (0.816 along dimension 1, 1.247 along dimension 2; step 8). It is the set of points whose normalised distance from the centroid is 1 (Lecture 5 · Slides 54–56). The five numbers on the green card are all BFR keeps about this cluster. Axis-parallel spread is exactly the cluster shape BFR assumes (Lecture 5 · Slide 79).

Part (b) Do the statistics have to be recomputed from scratch in every round? **Lecture 5 · Slides 48–51, 57, 81**

1. **Answer: No.** They are **updated incrementally** from the previous values. The slide says this is "at the heart of BFR algorithm" (Slide 81).
2. **Why recomputing from scratch is impossible, not just slow.** BFR reads the data one memory-load (chunk of points) at a time. A point that is close enough to a cluster is added to that cluster's discard set (DS), summarised and then thrown away (Slide 50). The old points are therefore no longer available; the only way to know the new statistics is to start from the stored old ones.
3. **Why updating is easy: N, SUM and SUMSQ are plain totals, and totals can be added.** When a new memory-load puts extra points into a cluster, BFR does step 3 of its loop, "Add Ns, SUMs, SUMSQs" (Slide 51): $N_{new} = N_{old} + N_{added}$; $SUM_{new} = SUM_{old} + SUM_{added}$; $SUMSQ_{new} = SUMSQ_{old} + SUMSQ_{added}$. Afterwards the centroid, variance and standard deviation are read off the card again with the formulas of part (a). That costs a few operations per dimension, however many points have been seen so far.

4. **Practice — incremental update (my own illustration, not on the slides).** Suppose a new point `(8, −1)` is found close enough to this cluster. Its own tiny card is N = 1, SUM = [8, −1], SUMSQ = [64, 1]. We only add:

	N	SUM	SUMSQ	centroid	variance
Old card (this quiz)	3	[18, −1]	[110, 5]	[6, −1/3]	[0.667, 1.556]
New point (8, −1)	1	[8, −1]	[64, 1]	—	—
Updated card = old + new	**4**	**[26, −2]**	**[174, 6]**	**[6.5, −0.5]**	**[1.25, 1.25]**

   New centroid = [26/4, −2/4] = [6.5, −0.5]. New variances: 174/4 − 6.5² = 43.5 − 42.25 = 1.25 and 6/4 − (−0.5)² = 1.5 − 0.25 = 1.25. Recomputing from the four raw points gives exactly the same values, but the update never looked at the first three points again.
5. **The same addition merges two summaries.** When BFR considers combining two compressed sets (CS), it adds their cards and gets the variance of the combined subcluster "quickly" from the merged card (Slide 57: "N, SUM, and SUMSQ allow us to make that calculation quickly"). In the last round the same addition folds a CS into its nearest cluster (Slide 51). Practice (my own numbers): add this card to the card of the two points (8, −1) and (9, 3), which is N = 2, SUM = [17, 2], SUMSQ = [145, 10]. The result is N = 5, SUM = [35, 1], SUMSQ = [255, 15], so the combined variances are 255/5 − 7² = 2 and 15/5 − (1/5)² = 2.96.

WHY THIS IS CONCEPTUALLY VALUABLE

Because variance = SUMSQ/N − (SUM/N)², the stored $2d + 1$ numbers are enough to recover both the **centre** and the **spread** of a cluster of any size, so BFR can discard the points and still run its distance test: the normalised (Mahalanobis) distance on Slides 54–56 divides each coordinate difference by the cluster's standard deviation σ_i, which is exactly the square root of the variance computed in part (a).

Means and variances cannot simply be added across chunks, but N, SUM and SUMSQ can; that is why the card stores these three quantities rather than the mean and variance themselves.

EXAM TIP

Even though this numerical will not be asked, be ready to say in words: (1) what N, SUM and SUMSQ are; (2) that $2d + 1$ numbers describe a cluster of any size; (3) that the centroid is SUM/N and the variance in dimension i is $\text{SUMSQ}_i/N - (\text{SUM}_i/N)^2$; (4) that updates are done by adding N, SUM and SUMSQ, never by recomputing from old points.

COMMON MISTAKE

Do not forget the "− (SUM/N)²" term: 110/3 = 36.67 on its own is only the average of the squares, not a variance. Beyond the slides: some statistics courses divide by N − 1 (sample variance), which would give 1.0 and 2.333 here. That is NOT the slide's answer; the BFR formulas on Slide 49 divide by N, so always use SUMSQ/N − (SUM/N)².

Final answer:
(a) N = 3; SUM = [18, −1]; SUMSQ = [110, 5]; centroid = SUM/N = [6, −1/3] ≈ [6, −0.333]; variance = [2/3, 14/9] ≈ [0.667, 1.556] (110/3 − 6² and 5/3 − (−1/3)²); standard deviation = [√(2/3), √14/3] ≈ [0.816, 1.247].
(b) No. The statistics are not recomputed from scratch; they are updated incrementally by adding the new points' (or merged sets') N, SUM and SUMSQ to the stored values. This is at the heart of BFR.

Where the marks come from: (1) N = 3; (2) SUM = [18, −1], by adding coordinates dimension by dimension; (3) SUMSQ = [110, 5], by squaring before adding; (4) centroid = SUM/N = [6, −1/3] with the formula stated; (5) variance formula SUMSQ/N − (SUM/N)² stated and applied in both dimensions (2/3 ≈ 0.667 and 14/9 ≈ 1.556); (6) standard deviation = √variance (0.816 and 1.247); (7) part (b): "No" plus the reason (the statistics can be updated by adding N, SUM and SUMSQ of the new points to the previous values, so old points need not be revisited). Most of these marks survive an arithmetic slip in a later step, so show every line.

SLIDE ERRATUM (NOTATION AND ROUNDING ONLY; NO NUMERICAL RESULT ON THE SLIDE IS WRONG)
Slide 81 writes the first variance as "110/3 − 18/3²". Read literally that is 110/3 − 18/9 = 34.67; the intended expression is 110/3 − **(18/3)²** = 36.667 − 36 = 0.667, as the slide itself writes for dimension 2, (−1/3)². The slide also rounds 14/9 = 1.556 to 1.56 and √(14/9) = 1.247 to 1.25, and prints √0.667 = 0.816: taking the square root of the rounded 0.667 gives 0.8167, so use the exact fraction 2/3 (√(2/3) = 0.8165) to get 0.816.

Slide check: matches the slide's answer on every result (N = 3, SUM = [18, −1], SUMSQ = [110, 5], centroid [6, −1/3], variances 0.667 and 1.56, standard deviations 0.816 and 1.25, and "NO! They can be updated based on the previous values"). Slide erratum: notation and rounding only, as explained in the box above.

Three scalable ways to cluster: k-means, CURE and BFR

Lecture 5 · Slides 23–27, 35–42, 43–58, 86–87

	k-means	CURE	BFR
Meaning	point assignment to *k* centroids	Clustering Using REpresentatives	Bradley–Fayyad–Reina: *k*-means for large data
Data size handled	data fit in main memory	very large: a memory-sized sample plus one rescan	very large, disk-resident: read in memory loads
Cluster shape assumed	compact, roundish blobs around a centroid	**any shape (rings, crescents, elongated)**	normal, axis-aligned ellipses
Distance	Euclidean	Euclidean	Euclidean (Mahalanobis for closeness tests)
Passes over the data	many (repeat until convergence)	two: sample, then rescan to assign points	one (Beyond the slides): each chunk is read once
What is kept in memory	all points + *k* centroids	the sample, then a few representative points per cluster	N, SUM, SUMSQ per cluster and per CS group, the RS points and one chunk
Cluster represented by	one point, the centroid	several representatives, moved (say) 20% toward the centroid	the summary (N, SUM, SUMSQ)
How a new point is assigned	nearest centroid	cluster of the nearest representative	Mahalanobis distance below a threshold, else CS / RS
Extra structures	none	none	DS, CS, RS
Outlier handling	outliers pull the mean	representatives moved inward damp them	outliers sit in the RS until the end
Main limitation	whole data must fit in memory	needs a good sample	cannot handle tilted or non-normal shapes

EXAM TIP
"Why BFR, why CURE?" Both extend *k*-means to data that do not fit in memory. BFR gets there by **summarising** clusters (and so needs normal, axis-aligned clusters). CURE gets there by **sampling** and keeping representatives (and so handles any shape). When the question names a ring or crescent, answer CURE; when it names tilted ellipses, say BFR cannot, and that CURE can.

k-means on MapReduce

Lecture 5 · Slides 59–67, 84

Slide 59 is a divider titled "K-Means MapReduce" (it carries no printed number and no footer; 59 is inferred from the page order). The BFR and CURE part of the lecture is over. They cope with big data by **summarising** (BFR) or **sampling** (CURE). This last section shows the other standard way: leave *k*-means exactly as it is and **spread the work over many machines** with MapReduce.

> **BEGINNER INTUITION**
> Eight students stand in a corridor and two flags are planted in it. Every student walks to the nearest flag (each student can decide alone, so many helpers can work at the same time: this is the **map** step). Then each flag is moved to the average position of the students standing at it (to average a crowd you need the whole crowd in one place: this is the **reduce** step). Repeat until the flags stop moving.

The MapReduce vocabulary you need

Lecture 5 · Slides 61–63 (terms from the MapReduce lectures)

Term	Meaning here
Node	one machine of the cluster; the data are split across nodes (Slide 64: "on 3 nodes").
Mapper	a task that runs on a node over that node's share of the data and emits **key–value pairs**. "Emit" means "output one pair".
Shuffle (and sort)	the system step between map and reduce that sends all pairs with the same **key** to the same reducer.
Reducer	a task that receives one key together with all its values and outputs a combined result.
Combiner	an optional mini-reducer that runs on each mapper's node *before* the shuffle, to shrink the data sent over the network. (Slide 63: "It can use combiners since the average can be decomposed into sum and count!")
Driver (Beyond the slides)	the small program that starts one MapReduce job per iteration, hands the new centroids to the next job and decides when to stop.

The algorithm, step by step

Lecture 5 · Slides 60–63

Slides 60 to 63 are one slide built up in four steps (each slide adds a block to the previous one). Slide 63 shows the whole algorithm.

1. **Initialisation (Slide 60).** Start with **random initial centroids**. (**Beyond the slides:** the *k* centroids are tiny, so the driver makes a copy available to every mapper.)
2. **Mapper (Slide 61).** For each data point (x, y), find the nearest centroid and emit **(centroidID, (x, y))**. The key is the ID of the chosen centroid; the value is the point. This is the **assignment** step of *k*-means. A mapper needs no information from other mappers.
3. **Reducer (Slide 62).** For each centroid, compute the new centroid by **averaging the points assigned to it**. The shuffle has put all points of one centroid at one reducer. This is the **update** step of *k*-means.
4. **Combiner (Slide 63).** Because the average decomposes into a sum and a count, each node may first collapse its own pairs for a centroid into one pair (centroidID, (partial sum, count)). The reducer then adds up the partial sums, adds up the counts, and divides once.
5. **Iterations (Slide 63).** Repeat the MapReduce process until the centroids stop moving (or for a predefined number of iterations).

k-means step (Slide 24)	MapReduce role	Pairs exchanged
assign every point to its nearest centroid	Mapper	emits (centroidID, point)

(local pre-sum, optional)	Combiner	emits (centroidID, (partial sum, count))
move each centroid to the mean of its points	Reducer	outputs (centroidID, new centroid)
repeat until nothing moves	Driver starts the next job	new centroids sent to the mappers

One MapReduce job is exactly one *k*-means iteration. The whole algorithm is a sequence of such jobs.

COMMON MISTAKE: THE AVERAGE OF AVERAGES

A combiner must not send a node's *average*. Suppose node A holds the points {2, 3, 5} for a centroid and node B holds {100}. The true mean is (2 + 3 + 5 + 100) / 4 = 27.5. The average of the two local averages is (3.33 + 100) / 2 = 51.67, which is wrong because the groups have different sizes. Sending (sum, count) = (10, 3) and (100, 1) gives (10 + 100) / (3 + 1) = 27.5, correct. This is why Slide 63 stresses "sum and count".

WHY IT SCALES

The mappers work on different parts of the data in parallel, and with combiners only a few small (sum, count) pairs per centroid cross the network, never the bulky data. The price: each iteration is a separate job that reads all the data from disk again, so many iterations are slow. (**Beyond the slides:** this is the main reason in-memory frameworks such as Spark became popular for iterative algorithms.) The slides do not mention this cost.

SLIDE ERRATUM / OMISSIONS (SLIDES 59–63)

- The slides write points as (x, y) but the example (Slide 64 onward) uses single numbers; the distance (Euclidean, in one dimension just $|a - c|$) is never named.
- "Random initial centroids" and "stop moving" are not defined (exactly zero movement, or a small tolerance?). How the mappers receive the centroids is not stated.
- The Reducer bullet on Slide 62 repeats its own heading ("Reducer: Reducer: ..."); the slide's title on 59 says "K-Means" but on 60–63 "k-means". These are cosmetic.

The worked example (Slides 64–67)

Lecture 5 · Slides 64–67

Slide 64 sets the example: the data points (2, 21, 5, 8, 15, 3, 29, 25) with 2 clusters (k = 2) on 3 nodes (the parentheses are only a list of eight numbers, not a coordinate pair). Slide 65 gives the starting centroids (5 and 21) and the split of the data over the three nodes; Slide 66 ("Mapper Phase") lists the pairs each node emits; Slide 67 ("Reducer Phase") shows which points reach each reducer, the new centroids and the rule for iterating. The solved version, with every iteration, the combiner tables and the data-flow picture, is Quiz L5-Q7 below, so it is not repeated here.

What to look for in the quiz: (a) each mapper emits one pair per point it holds, with the ID of the nearest centroid as the key; (b) each reducer receives all points of one key and outputs their mean; (c) the driver compares the new centroids with the old and stops when they no longer change.

Try it yourself first. Without looking at the solution, work out what each of the three nodes emits in iteration 1 and what the new centroids are; then carry on until the centroids stop moving.

Quiz L5-Q7 k-means with MapReduce — worked example (2, 21, 5, 8, 15, 3, 29, 25)

Lecture 5 · Slides 59–67

QUESTION (AS ON THE SLIDES)

k-means MapReduce example (Slides 64–67). Run k-means with MapReduce on the data points `2, 21, 5, 8, 15, 3, 29, 25` with **2 clusters (k = 2)** on **3 nodes**. Start with **random initial centroids**: **Centroid 1 = 5** and **Centroid 2 = 21**. Assume the points are distributed over the nodes like this: **Node 1: [2, 21, 5]**, **Node 2: [8, 15]**, **Node 3: [3, 29, 25]**.

(a) Mapper phase, iteration 1. What does each node (mapper) emit, as pairs (centroidID, point)?

(b) Reducer phase, iteration 1. Which points reach each reducer, and what are the new centroids? (Also show what the optional combiners would send.)

(c) Iterate. The slide says "repeat the MapReduce process with the updated centroids (4.5 and 22.5) until convergence". Carry on, with the mapper, combiner and reducer tables of every iteration, until the centroids stop moving. State the final clusters and centroids.

(d) Roles. Explain what the mapper, the combiner, the reducer and the driver do, why the centroids are sent to every mapper, and draw the data flow of one iteration.

Slides 64–67 give the data and the answers to (a) and (b) only; Slides 60–63 give the algorithm (initialisation, mapper, reducer, combiner reminder, iterations). Parts (c) and (d) are our own completion of the example, written as practice for exam variants; every number was recomputed with Python.

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

1. **Formula / idea: one k-means round = one MapReduce job.** Ordinary k-means (Lecture 5 · Slide 24) repeats two things: *(i) assign* every point to the cluster whose centroid is nearest, then *(ii) update* every centroid to the average of the points assigned to it; it stops when the points no longer move between clusters and the centroids stabilise. The MapReduce version (Slides 60–63) does exactly one such round per job, so the **Mapper is the assignment step** and the **Reducer is the update step**.

   FORMULAS USED IN THIS QUIZ

   - **Distance** (1-dimensional Euclidean distance, because every point is a single number): $d(x, c) = |x - c|$. (For 2-D points (x, y), as the slides' notation suggests, it would be $\sqrt{((x - c_x)^2 + (y - c_y)^2)}$.)
   - **Mapper rule** (Slide 61): centroidID(x) = the j with the smallest $d(x, c_j)$; emit (centroidID, x).
   - **Reducer rule** (Slides 62 and 67): new $c_j = \frac{\sum x \text{ over the points of cluster } j}{n_j \text{ (how many points)}} = \frac{SUM_j}{COUNT_j}$.
   - **Combiner rule** (Slide 63: "the average can be decomposed into sum and count"): each node sends (partial SUM, partial COUNT) per centroidID; the reducer adds all the sums, adds all the counts, then divides once.
   - **Stopping rule** (Slide 63): repeat until the centroids stop moving, or for a predefined number of iterations.

   BEGINNER INTUITION

   Picture 8 students standing in a long corridor at the positions 2, 3, 5, 8, 15, 21, 25, 29, and 2 flags planted at positions 5 and 21. **Step A (mapper):** every student walks to the nearest flag. Each student can decide alone, so three helpers (the 3 nodes) can each handle their own few students at the same time. **Step B (reducer):** each flag is moved to the average position of the crowd standing at it. To compute an average you need the whole crowd of that flag in one place, which is why everybody with the same centroidID is sent to the same reducer. Repeat A and B until the flags no longer move.

2. **Set-up (Slide 65).** Number the centroids: **ID 1 = 5** and **ID 2 = 21**; with k = 2 there are exactly two keys, 1 and 2. Every mapper receives a copy of this tiny list of k = 2 numbers (Beyond the slides: the

"broadcast", explained in step 10). Each node keeps only its own slice of the data, 3 + 2 + 3 = 8 points, each point exactly once, and the data never move:

Node	Points stored on it (Slide 65)
Node 1	`2, 21, 5`
Node 2	`8, 15`
Node 3	`3, 29, 25`

3. **Iteration 1 — Mapper (assignment step), answer to (a).** For each point the mapper computes its distance to *both* centroids, finds the smaller one, and emits (centroidID, point). The smaller distance in each row is highlighted.

Node	Point x	$\lvert x - 5 \rvert$ (to ID 1)	$\lvert x - 21 \rvert$ (to ID 2)	Nearest	Mapper emits (centroidID, point)
1	2	**3**	19	ID 1	`(1, 2)`
1	21	16	**0**	ID 2	`(2, 21)`
1	5	**0**	16	ID 1	`(1, 5)`
2	8	**3**	13	ID 1	`(1, 8)`
2	15	10	**6**	ID 2	`(2, 15)`
3	3	**2**	18	ID 1	`(1, 3)`
3	29	24	**8**	ID 2	`(2, 29)`
3	25	20	**4**	ID 2	`(2, 25)`

So the three mappers emit: **Node 1:** `(1, 2)`, `(2, 21)`, `(1, 5)`; **Node 2:** `(1, 8)`, `(2, 15)`; **Node 3:** `(1, 3)`, `(2, 29)`, `(2, 25)` — exactly Slide 66 (written there as "centroidID=1, 2" etc.; 8 pairs in total). Read `(1, 2)` as "centroid ID 1, data point 2", not as a 2-D point. The only close call is **15**: 10 away from 5 but only 6 away from 21, so it goes to ID 2. Shortcut for checking in one dimension: the half-way point of the two centroids is (5 + 21) / 2 = 13; everything below 13 is nearer to 5 and everything above 13 is nearer to 21 (no point is exactly on 13, so there are no ties).

4. **Iteration 1 — Combiner (optional local pre-aggregation).** Slide 63 says a combiner may be used because an average is *sum ÷ count*. On each node the combiner merges that node's pairs with the same key into one pair (partial SUM, partial COUNT). It runs on the node, before the network transfer, so it shrinks the data that has to be shuffled.

Node	Mapper output on that node	Key 1: (sum, count)	Key 2: (sum, count)
1	`(1,2) (2,21) (1,5)`	(2 + 5, 2) = **(7, 2)**	**(21, 1)**
2	`(1,8) (2,15)`	**(8, 1)**	**(15, 1)**
3	`(1,3) (2,29) (2,25)`	**(3, 1)**	(29 + 25, 2) = **(54, 2)**
Totals after adding over the nodes		**(7 + 8 + 3, 2 + 1 + 1) = (18, 4)**	**(21 + 15 + 54, 1 + 1 + 2) = (90, 4)**

With the combiner only **6** records leave the nodes instead of 8 (2 keys × 3 nodes); with a huge data set the saving is enormous, because each node sends at most k records.

> **COMMON MISTAKE**
> Do **not** let the combiner send local *averages*. For key 1 the local averages are 3.5 (Node 1: points 2 and 5), 8 (Node 2) and 3 (Node 3). Averaging those three numbers gives (3.5 + 8 + 3) / 3 ≈ **4.83**, but the true average is **4.5**, because Node 1's average stands for two points and the others for one. (Key 2 would give (21 + 15 + 27) / 3 = 21 instead of the true 22.5.) A sum and a count can be added safely in any order and grouping, an average cannot. This is why the slide says "sum and count".

5. **Iteration 1 — Shuffle & sort, then Reducer (update step), answer to (b).** The system routes every pair with key 1 to Reducer 1 and every pair with key 2 to Reducer 2 ("group by key"). Each reducer then takes the average of its points:

Reducer for	Points that reach it (Slide 67 lists them sorted)	Sum	Count	New centroid = sum ÷ count
key 1 (old 5)	`2, 3, 5, 8` (arrive as 2, 5, 8, 3)	18	4	**(2 + 3 + 5 + 8) / 4 = 18 / 4 = 4.5**
key 2 (old 21)	`15, 21, 25, 29` (arrive as 21, 15, 29, 25)	90	4	**(15 + 21 + 25 + 29) / 4 = 90 / 4 = 22.5**

With the combiner the reducers receive `(7,2) (8,1) (3,1)` for key 1 and `(21,1) (15,1) (54,2)` for key 2, and get the same answer: (7 + 8 + 3) / (2 + 1 + 1) = 18 / 4 = 4.5 and (21 + 15 + 54) / (1 + 1 + 2) = 90 / 4 = 22.5. **After iteration 1 the new centroids are 4.5 and 22.5** (Slide 67).

6. **Driver decision after iteration 1.** Compare old and new centroids: ID 1 moved from 5 to 4.5 (change −0.5) and ID 2 moved from 21 to 22.5 (change +1.5). They moved, so we have **not converged**: start iteration 2 with the centroids **4.5 and 22.5** (Slide 67: "repeat the MapReduce process with the updated centroids … until convergence").
7. **Iteration 2 — Mapper, with the NEW centroids (4.5 and 22.5).** The data is the same, only the centroids changed. Every point is re-assigned from scratch (new half-way point (4.5 + 22.5) / 2 = 13.5):

Node	Point x	$\lvert x - 4.5\rvert$ (to ID 1)	$\lvert x - 22.5\rvert$ (to ID 2)	Nearest	Mapper emits
1	2	**2.5**	20.5	ID 1	`(1, 2)`
1	21	16.5	**1.5**	ID 2	`(2, 21)`
1	5	**0.5**	17.5	ID 1	`(1, 5)`
2	8	**3.5**	14.5	ID 1	`(1, 8)`
2	15	10.5	**7.5**	ID 2	`(2, 15)`
3	3	**1.5**	19.5	ID 1	`(1, 3)`
3	29	24.5	**6.5**	ID 2	`(2, 29)`
3	25	20.5	**2.5**	ID 2	`(2, 25)`

Mapper output: **Node 1:** `(1, 2)`, `(2, 21)`, `(1, 5)`; **Node 2:** `(1, 8)`, `(2, 15)`; **Node 3:** `(1, 3)`, `(2, 29)`, `(2, 25)` — **identical to iteration 1**. The reason: no data point lies between the old border 13 and the new border 13.5, so no point can change side (Figure L5-Q7-A in step 9).

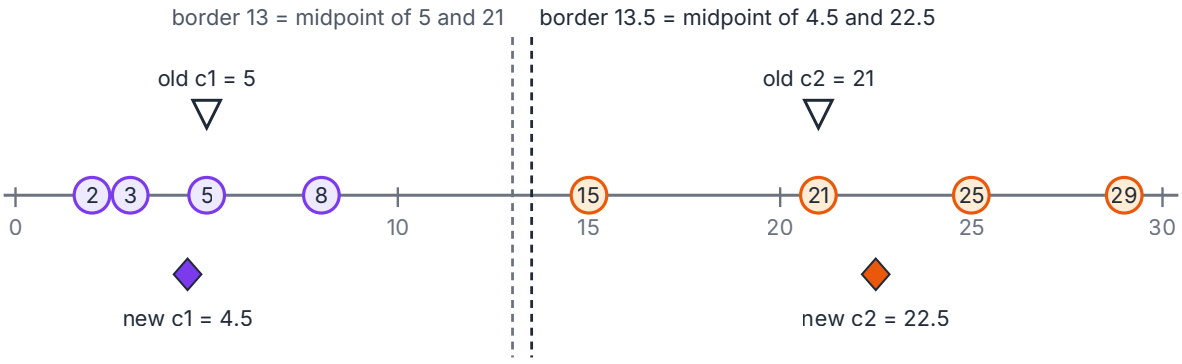
8. **Iteration 2 — Combiner and Reducer.** Because the pairs are the same, the tables are the same:

Node	Combiner key 1: (sum, count)	Combiner key 2: (sum, count)
1	(7, 2)	(21, 1)
2	(8, 1)	(15, 1)
3	(3, 1)	(54, 2)
Reducer adds up	**(7 + 8 + 3, 2 + 1 + 1) = (18, 4) → 18 / 4 = 4.5**	**(21 + 15 + 54, 1 + 1 + 2) = (90, 4) → 90 / 4 = 22.5**

(Without a combiner Reducer 1 would again receive the points 2, 3, 5, 8 and Reducer 2 the points 15, 21, 25, 29.) The reducers output **4.5** and **22.5** again.

9. **Convergence check, answer to (c).** New centroids (4.5, 22.5) = old centroids (4.5, 22.5): movement 0 and 0, and no point changed cluster. This meets the stopping rule of Slide 63 ("until the centroids stop moving") and the convergence definition of Slide 24 (points do not move between clusters and centroids stabilise), so the driver stops. A third iteration would only repeat iteration 2, so **two MapReduce jobs** are needed: iteration 1 moves the centroids, iteration 2 confirms that they no longer move.

Iteration	Centroids used by the mappers	Cluster of ID 1	Cluster of ID 2	Centroids produced by the reducers	Moved?
1	5, 21	`2, 3, 5, 8`	`15, 21, 25, 29`	4.5, 22.5	yes (−0.5, +1.5)
2	4.5, 22.5	`2, 3, 5, 8`	`15, 21, 25, 29`	4.5, 22.5	**no (0, 0): STOP**



No point lies between 13 and 13.5, so no point changes cluster: converged.

Figure L5-Q7-A. The 8 points on a number line (purple = cluster of ID 1, orange = cluster of ID 2). The centroids move from 5 and 21 (white triangles) to 4.5 and 22.5 (filled diamonds), and the border between the clusters moves from 13 to 13.5. Points 8 and 15 are the nearest to the border, but both stay on their side.

EXAM TIP

Lay your answer out in the order the framework runs it: centroids → mapper pairs (key = centroidID, value = point) → combiner (sum, count) per node → shuffle → reducer total sum ÷ total count → new centroids → compare with the old ones → repeat or stop. Each iteration must use the centroids produced by the *previous* iteration (iteration 2 uses 4.5 and 22.5, not 5 and 21). In 1-D, find the midpoint of the two centroids first (13, then 13.5) and use it to check every assignment quickly. Always finish with an explicit sentence such as "centroids unchanged, so k-means has converged after 2 MapReduce iterations". If the question only asks for "one iteration", stop at 4.5 and 22.5 as the slide does. The order of the points inside a reducer's list does not matter.

10. **Roles of every component, answer to (d).**

Component	Its job in k-means	Receives	Emits	Slides
Driver (the main program that submits the jobs; Beyond the slides)	Loop control: picks the initial centroids, sends the current centroids to every mapper, runs one MapReduce job per iteration, reads the new centroids back, compares old and new, and stops at convergence or after a fixed number of iterations	new centroids from the reducers	centroid list for the next job	60, 63, 67
Mapper	**Assignment step:** nearest centroid for each point. Works on one point at a time, so all nodes run in parallel on their own slice	one point + all k current centroids	(centroidID, point)	61, 66
Combiner (optional)	Local speed-up: partial (sum, count) per centroidID on one node, before the network	the mapper output of one node	(centroidID, (partial sum, partial count))	63
Shuffle & sort (done by the framework; Beyond the slides)	Groups by key so that all values of one centroidID meet at one reducer	all mapper / combiner pairs	(centroidID, list of values)	–
Reducer	**Update step:** new centroid = average of the points of that cluster = total sum ÷ total count	one centroidID + all its points (or partial (sum, count) pairs)	(centroidID, new centroid)	62, 63, 67

Why every mapper needs all the centroids: a mapper sees only its own few points but must compare each point with *every* centroid to find the nearest one. The k centroids are tiny (here 2 numbers), the data is huge, so it is cheap to copy the centroids to all mappers and never move the data (Beyond the slides: in Hadoop this is done with the distributed cache or the job configuration, in Spark with a broadcast variable). The mappers do not need to talk to each other. Every iteration the driver sends the *newest* centroids again.

Why a reducer needs the whole cluster: a centroid is an average over *all* points of the cluster, and these points are spread over all nodes. The key (centroidID) tells the framework where to send each point. With k = 2 there are only 2 keys, so at most 2 reducers are busy; this is exactly why the combiner is valuable: with it each node sends at most k records, so the shuffle carries at most k × (number of nodes) = 2 × 3 = 6 records, however many points there are.

11. **Data-flow diagram of one iteration (iteration 1 with the real numbers), part of (d).**

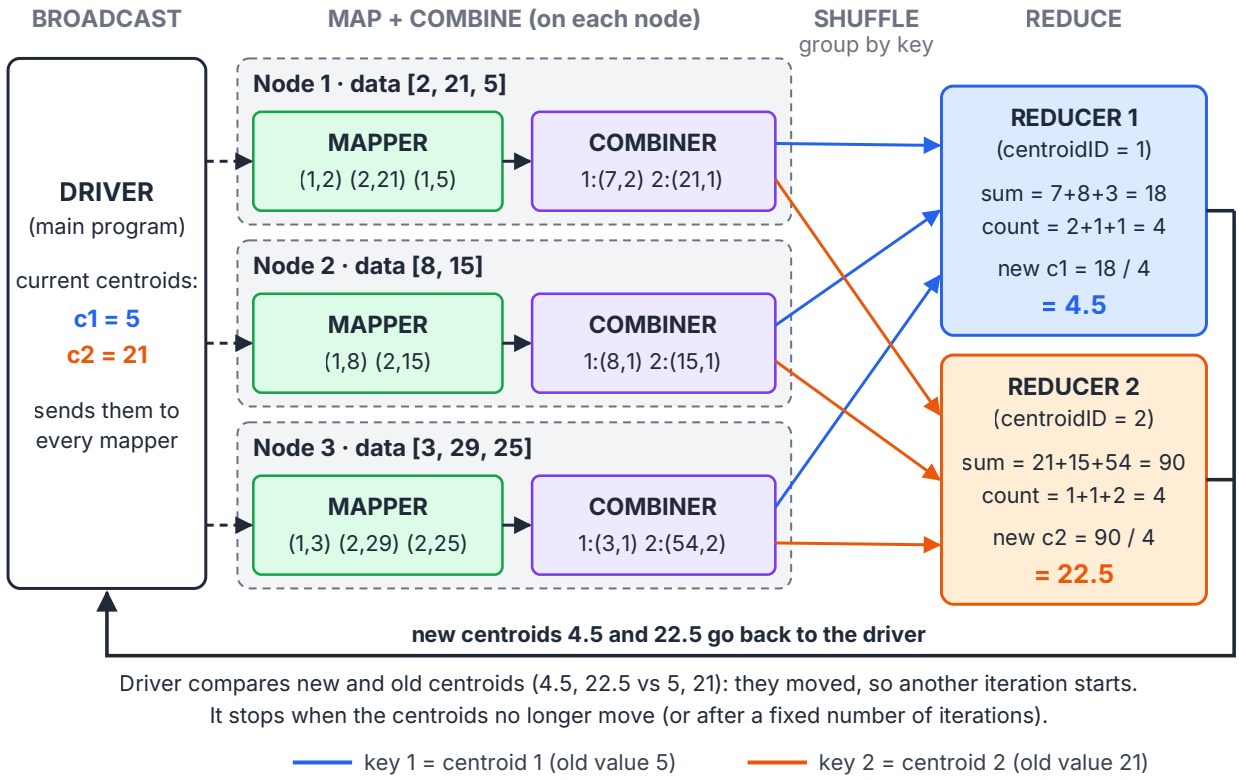


Figure L5-Q7-B. One k-means iteration as a MapReduce job, with the numbers of iteration 1. Dashed arrows: (1) the driver broadcasts the current centroids to every mapper. (2) Each mapper assigns its own points to the nearest centroid and emits (centroidID, point). (3) Each combiner collapses its node's pairs into one (sum, count) per key. (4) Shuffle & sort sends all pairs with the same key to the same reducer (blue = key 1, orange = key 2); only the small (sum, count) pairs cross the network. (5) Each reducer divides the total sum by the total count to get the new centroid. (6) The new centroids go back to the driver, which compares them with the old ones and either stops or starts the next iteration.

12. **Beyond the slides: extra facts that explain and check the result.**
    - **Combiner typing.** A real combiner may run zero, one or several times, so it must produce the same kind of record that the reducer expects. That is why implementations let the mapper emit (centroidID, (point, 1)) and let the combiner and reducer just add the pairs. Slide 66 shows the simplified form without the "1". The idea is identical. The combiner is only an optimisation: the centroids are the same with or without it.
    - **Ties and empty clusters.** A point exactly half-way between two centroids needs a fixed tie-break rule (for example the smaller centroidID); no ties occur here. If a centroid receives no point, its key never reaches a reducer; a common fix is that the driver keeps its old value.
    - **Cost.** Every iteration is a complete MapReduce job that re-reads the whole data set; this is why iterative algorithms such as k-means are the classic motivation for in-memory systems like Spark.
    - **Quality check (sum of squared distances, SSE).** The iteration-1 assignment has SSE 138 measured from the centroids 5 and 21 (22 + 116); the update to 4.5 and 22.5 lowers it to 128 (21 + 107); iteration 2 keeps 128. k-means never increases the SSE and there are only finitely many ways to split the points, so it always stops. For this tiny data set 128 is also the best possible split into two groups (all 7 ways of cutting the sorted list into a lower and an upper group were checked), but in general k-means only finds a *local* optimum.

- **Practice (not on the slides): the start matters (Lecture 5 · Slide 23).** Same data, but initial centroids 2 and 29. Iteration 1: the points 2, 3, 5, 8, 15 go to ID 1 (15 is 13 from 2 but 14 from 29) and 21, 25, 29 to ID 2; the reducers give (2 + 3 + 5 + 8 + 15) / 5 = 33 / 5 = 6.6 and (21 + 25 + 29) / 3 = 75 / 3 = 25. Iteration 2 (centroids 6.6 and 25) gives the same clusters, so it stops with SSE 141.2, which is *worse* than 128 from the start 5 and 21. Different random starts can give different final clusters.

COMMON MISTAKES

- **Stopping after iteration 1** because the slide stops there. The slide ends with "repeat … until convergence"; run iteration 2 with 4.5 and 22.5 and show that nothing moves.
- **Re-using the old centroids (5, 21) in iteration 2** instead of the new ones (4.5, 22.5).
- **Dividing by the wrong number.** Reducer 1 receives only 3 combiner pairs (one per node) but they stand for 4 points. Divide the total sum by the total count (4), not by the number of nodes or pairs.
- **Reading (1, 2) as a 2-D point.** In (centroidID, point) the first number is the cluster key (1 or 2) and the second is the data point; the key is the centroid's ID, not its value (5 or 21).
- **Putting 15 in cluster 1** because it "looks in the middle". Use the numbers: 10 away from 5 but 6 away from 21; in iteration 2, 10.5 away from 4.5 and 7.5 away from 22.5.

Final answer:
(a) Mapper output, iteration 1 (Slide 66): Node 1 emits (1,2), (2,21), (1,5); Node 2 emits (1,8), (2,15); Node 3 emits (1,3), (2,29), (2,25).
(b) Combiners (optional): Node 1 → 1:(7,2), 2:(21,1); Node 2 → 1:(8,1), 2:(15,1); Node 3 → 1:(3,1), 2:(54,2). Reducer 1 receives 2, 3, 5, 8 and outputs (2+3+5+8)/4 = 18/4 = 4.5; Reducer 2 receives 15, 21, 25, 29 and outputs (15+21+25+29)/4 = 90/4 = 22.5 (Slide 67).
(c) Iteration 2 with centroids 4.5 and 22.5 sends every point to the same cluster and outputs 4.5 and 22.5 again, so the centroids stop moving: converged after 2 MapReduce iterations. Final clusters {2, 3, 5, 8} with centroid 4.5 and {15, 21, 25, 29} with centroid 22.5.
(d) Mapper = assignment step, emits (centroidID, point). Combiner = local partial (sum, count) per centroidID. Reducer = update step, new centroid = total sum ÷ total count. Driver = repeats the job until the centroids stop moving (or a fixed number of iterations). The k centroids are broadcast to every mapper because each point must be compared with all of them.

Where the marks come from: (1) The two rules: nearest centroid by distance $|x - c|$ for the mapper, and new centroid = average = sum ÷ count for the reducer. (2) Mapper iteration 1: the correct nearest centroid for all 8 points (especially 15 → ID 2), written as (centroidID, point) pairs per node, with the distances or the midpoint 13 shown. (3) Grouping by key (shuffle) and, if asked, the combiner's (sum, count) pairs, not partial averages. (4) Reducer iteration 1: the points reaching each reducer, the sums 18 and 90, the counts 4 and 4, the results 4.5 and 22.5. (5) Iteration 2 done with the *new* centroids 4.5 and 22.5, with the distances shown. (6) The convergence statement: centroids unchanged, so stop; the final clusters. (7) The roles: mapper = assignment, reducer = update, combiner = partial (sum, count), driver loops, centroids broadcast to all mappers. (8) The flow diagram: driver → mappers → (combiners) → shuffle → reducers → new centroids back to the driver.

SLIDE ERRATUM (OMISSIONS ONLY — NO WRONG NUMBERS)

- Slides 64–67 stop after the **first** iteration and only say "repeat … until convergence or after a fixed number of iterations". The second iteration, which shows that the centroids stay at 4.5 and 22.5 (so the algorithm has converged), is not on the slides; it is solved above.
- Slide 63 says combiners can be used, but the example (Slides 66–67) shows none, and the slides do not say how many reducers are used (one per centroid key here). The combiner tables above are added.
- Slides 61 and 63 write the points as (x, y), i.e. 2-D, but the example uses single numbers (1-D). The method is the same: the distance is $|x - c|$ and the "average" is a plain average of numbers. In 2-D the sum becomes a pair (sum of x, sum of y) and the new centroid is (SUM_x / COUNT, SUM_y / COUNT).
- Minor typo on Slide 62: "Reducer: Reducer: …" (the word is repeated).

Slide check: matches the slides. The mapper output equals Slide 66 and the reducer results equal Slide 67 (Reducer 1: 2, 3, 5, 8 → 4.5; Reducer 2: 15, 21, 25, 29 → 22.5); every number here was also recomputed with Python. Slide erratum: no wrong numbers, only omissions (the later iterations and the combiner), described in the box above.

The homework question (Slide 84)

Lecture 5 · Slide 84

Slide 84, "Quiz: K-Means MapReduce Homework" (the last letters of "Homework" run off the banner), asks you to compute clusters with *k*-means MapReduce for 17 numbers (2, 3, 5, 8, 15, 21, 23, 25, 29, 43, 45, 49, 62, 75, 76, 77, 85) with $k = 3$ on 3 nodes. The slide gives no answer, and it leaves three things open that your answer must state itself: **the initial centroids, how the points are spread over the three nodes, and the stopping rule**. A good answer fixes all three, then shows the mapper output, the combiner pairs and the reducer results of every round until nothing moves.

EXAM TIP

Because *k*-means can stop in different local optima for different starts, write down the starting centroids you chose (for example the farthest-first rule of Slide 23). The final clusters do not depend on how the points are split over the nodes; only the starting centroids matter.

Try it yourself first. Choose three starting centroids, split the 17 points over 3 nodes, and run rounds on paper until the centroids stop moving; then compare with the solution (and note what a different start does).

Quiz L5-Q8 k-means MapReduce on 17 points, k = 3, on 3 nodes — Lecture 5 · Slide 84

QUESTION (AS ON THE SLIDES)

Slide 84, "Quiz: K-Means MapReduce Homework". Compute clusters for the following data points using K-Means MapReduce: `2, 3, 5, 8, 15, 21, 23, 25, 29, 43, 45, 49, 62, 75, 76, 77, 85` (17 one-dimensional points). Assume there are three clusters ($k = 3$), on 3 nodes.

The slide gives **no initial centroids, no distribution of the points over the nodes and no answer** (the method is on Lecture 5 · Slides 60–67; Slides 60–63 only say "start with random initial centroids"). A complete answer therefore **states its own start** and then runs full rounds until convergence. This guide splits the work into the parts below. All slide numbers in the solution are Lecture 5 slides, and every number was checked with Python (exact fractions).

(a) Choose and state the initial centroids and the distribution of the points over the 3 nodes.

(b) Run k-means MapReduce round by round (Mapper output per node, Combiner (sum, count) per node, Reducer results) until the centroids stop moving.

(c) State the final clusters and centroids.

(d) Show that the answer does not depend on how the points are split over the nodes.

(e) Run a second, different initialisation. Does it give the same clustering? What does this teach?

SOLUTION — EVERY STEP IS A POTENTIAL PARTIAL MARK

BEGINNER INTUITION — K-MEANS IN PLAIN ENGLISH

The 17 numbers are house numbers along one long street, and you want to open **3 pizza shops**. (1) Put the 3 shops at three starting spots. (2) Every house orders from its **nearest** shop. (3) Each shop moves to the **average address** of its customers. Repeat (2) and (3) until nobody changes shop and the shops stop moving. That is k-means.

MapReduce only decides **who does the work** when the houses are stored on 3 different computers (*nodes*). Each node lets its own houses pick the nearest shop (**Mapper**), then writes one short note per shop, "total of the addresses and number of houses" (**Combiner**). One clerk per shop (**Reducer**) adds up the notes of all nodes and divides *total by number* to get the shop's new position. A **driver** program checks whether any shop moved and, if so, starts another round. Only the tiny notes and the 3 shop positions travel over the network, never the bulky data.

1. **Formula / idea: one k-means round = one MapReduce job (Lecture 5 · Slides 60–63).** Ordinary k-means (Slide 24) alternates (1) put every point into the cluster of its nearest centroid and (2) move every centroid to the average of its cluster, until points stop moving between clusters. MapReduce spreads these two steps over the machines. (The data-flow picture of one round is Figure L5-Q7-B in Quiz L5-Q7.)

   FORMULAS USED IN THIS QUIZ

   - **Distance** (Slide 23: "assumes Euclidean space/distance"). Every point is a single number, so $d(p, c_j) = |p - c_j|$. (For 2-D points (x, y) as written on Slides 61–63 it would be $\sqrt{((x - c_x)^2 + (y - c_y)^2)}$.)
   - **Mapper** (assignment step, Slide 61): for each of its own points find the nearest centroid j and emit `(j, p)`.
   - **Combiner** (optional, Slide 63): on each node, for each key j add up the points and count them; emit `(j, (sum, count))`. Allowed because "the average can be decomposed into sum and count": a mean cannot be added up, but a sum and a count can, in any order and any grouping.
   - **Reducer j** (update step, Slide 62): $c_j^{new} = \frac{\sum \text{ of all partial sums for key } j}{\sum \text{ of all partial counts for key } j}$ = average of the points of cluster j.
   - **Driver / stop rule** (Slides 24 and 63): repeat until the centroids stop moving (or for a fixed number of rounds).

   Tie convention (a point exactly midway between two centroids): give it to the lower key; no tie occurs in any run below. The data points never move between nodes; only the 3 centroids (to the mappers) and the small (sum, count) pairs (to the reducers) travel.

2. **Part (a) — what the slide leaves open, and our choices.** Slide 84 gives only the data, $k = 3$ and "3 nodes". The result depends on the start (see part (e)), so the start must be **written down** (examiners give marks for stating it).

   - **Initial centroids by farthest-first** (Slide 23: "pick one point at random, then $k-1$ other points, each as far away as possible from the previous points"). First pick: **2** (the first point of the list stands in for "random"). The point farthest from 2 is **85** (distance 83). For the third centroid take, for every remaining point, the distance to the *nearest* centroid already chosen, and pick the point where this is largest:

candidate p	**3**	**5**	**8**	**15**	**21**	**23**	**25**	**29**	**43**	**45**	**49**	**62**	**75**	**76**	**77**
distance to 2	1	3	6	13	19	21	23	27	41	43	47	60	73	74	75
distance to 85	82	80	77	70	64	62	60	56	42	40	36	23	10	9	8
min of the two	1	3	6	13	19	21	23	27	**41**	40	36	23	10	9	8

   So the third centroid is **43** (min(41, 42) = 41; runners-up 45 with 40 and 49 with 36). ("Far from *all* previous points" must mean the largest *minimum* distance: the plain sum of the two distances is 83

for every point between 2 and 85, so it could not choose.) Numbering them in increasing order (the numbering is arbitrary): $\mathbf{c_1 = 2, c_2 = 43, c_3 = 85}$.

- **Distribution over the 3 nodes:** the sorted list cut into three blocks — **Node 1** = [2, 3, 5, 8, 15, 21] (6 points), **Node 2** = [23, 25, 29, 43, 45, 49] (6 points), **Node 3** = [62, 75, 76, 77, 85] (5 points); 6 + 6 + 5 = 17, every point exactly once. (Slide 65 shows that a split may be any mix; part (d) tries another one.)
- **Reducers:** one per centroid, i.e. 3 reducers (keys 1, 2, 3). **Stop rule:** the new centroids equal the old ones.

3. **Part (b) — Round 1 (centroids 2, 43, 85).** The green cell in each row is the smallest distance, i.e. the nearest centroid; the last column is exactly the Mapper output in the style of Slide 66.
Centroids broadcast to every mapper this round: $c_1 = 2$, $c_2 = 43$, $c_3 = 85$. 1-D shortcut for checking: the boundaries are the midpoints (2 + 43)/2 = 22.5 and (43 + 85)/2 = 64; a point below the first boundary goes to key 1, between the two boundaries to key 2, above the second to key 3.

Node	Point p	$\lvert p - 2\rvert$ (c_1)	$\lvert p - 43\rvert$ (c_2)	$\lvert p - 85\rvert$ (c_3)	Nearest	Mapper emits (ID, point)
Node 1	2	**0**	41	83	c_1	`(1, 2)`
	3	**1**	40	82	c_1	`(1, 3)`
	5	**3**	38	80	c_1	`(1, 5)`
	8	**6**	35	77	c_1	`(1, 8)`
	15	**13**	28	70	c_1	`(1, 15)`
	21	**19**	22	64	c_1	`(1, 21)`
Node 2	23	21	**20**	62	c_2	`(2, 23)`
	25	23	**18**	60	c_2	`(2, 25)`
	29	27	**14**	56	c_2	`(2, 29)`
	43	41	**0**	42	c_2	`(2, 43)`
	45	43	**2**	40	c_2	`(2, 45)`
	49	47	**6**	36	c_2	`(2, 49)`
Node 3	62	60	**19**	23	c_2	`(2, 62)`
	75	73	32	**10**	c_3	`(3, 75)`
	76	74	33	**9**	c_3	`(3, 76)`
	77	75	34	**8**	c_3	`(3, 77)`
	85	83	42	**0**	c_3	`(3, 85)`

Combiner (on each node, before the network):

Node	Key	Points of this key on this node	Partial sum	Count	Combiner emits (key, (sum, count))
Node 1 6 points	1	2, 3, 5, 8, 15, 21	2 + 3 + 5 + 8 + 15 + 21 = 54	6	`(1, (54, 6))`
Node 2 6 points	2	23, 25, 29, 43, 45, 49	23 + 25 + 29 + 43 + 45 + 49 = 214	6	`(2, (214, 6))`
Node 3 5 points	2	62	62	1	`(2, (62, 1))`
	3	75, 76, 77, 85	75 + 76 + 77 + 85 = 313	4	`(3, (313, 4))`
All nodes					**4 pairs shuffled** (17 without combiner)

Shuffle and reducers: all pairs with the same key travel to the same reducer, which adds the partial sums, adds the partial counts and divides.

Reducer	Receives (from the combiners)	Total sum	Total count	New centroid = sum / count
Reducer 1 (key 1)	Node 1: (54, 6)	54	6	**54 / 6 = 9**
Reducer 2 (key 2)	Node 2: (214, 6); Node 3: (62, 1)	214 + 62 = 276	6 + 1 = 7	**276 / 7 ≈ 39.43**
Reducer 3 (key 3)	Node 3: (313, 4)	313	4	**313 / 4 = 78.25**

Self-check (sums and counts are conserved): 54 + 276 + 313 = 643 (the sum of the 17 data points) and 6 + 7 + 4 = 17 (the number of data points).

Round 1 result: clusters {2, 3, 5, 8, 15, 21}, {23, 25, 29, 43, 45, 49, 62}, {75, 76, 77, 85}; new centroids c_1 = 9, c_2 = 276/7 ≈ 39.43, c_3 = 78.25. They moved (c_1: 2 → 9; c_2: 43 → 39.43; c_3: 85 → 78.25) because a new centroid is the average of the points it won, not the point where it started (e.g. c_1 started at the extreme point 2, but its six points average 9). The combiners sent only 4 pairs instead of 17 points. Not converged, so the driver starts Round 2 with these new centroids.

4. **Part (b) — Round 2 (centroids 9, 276/7 ≈ 39.43, 313/4 = 78.25).** The Mappers now use the **new** centroids from Round 1. Exact fractions are used in the computation; the table shows distances rounded to 2 decimals.
Centroids broadcast to every mapper this round: c_1 = 9, c_2 = 276/7 ≈ 39.43, c_3 = 313/4 = 78.25. 1-D shortcut for checking: the boundaries are the midpoints (9 + 39.43)/2 = 24.21 and (39.43 + 78.25)/2 = 58.84; a point below the first boundary goes to key 1, between the two boundaries to key 2, above the second to key 3.

Node	Point p	$\lvert p - 9\rvert$ (c_1)	$\lvert p - 39.43\rvert$ (c_2)	$\lvert p - 78.25\rvert$ (c_3)	Nearest	Mapper emits (ID, point)
Node 1	2	**7**	37.43	76.25	c_1	`(1, 2)`
	3	**6**	36.43	75.25	c_1	`(1, 3)`
	5	**4**	34.43	73.25	c_1	`(1, 5)`
	8	**1**	31.43	70.25	c_1	`(1, 8)`
	15	**6**	24.43	63.25	c_1	`(1, 15)`
	21	**12**	18.43	57.25	c_1	`(1, 21)`
Node 2	23	**14**	16.43	55.25	c_1	`(1, 23)`
	25	16	**14.43**	53.25	c_2	`(2, 25)`
	29	20	**10.43**	49.25	c_2	`(2, 29)`
	43	34	**3.57**	35.25	c_2	`(2, 43)`
	45	36	**5.57**	33.25	c_2	`(2, 45)`
	49	40	**9.57**	29.25	c_2	`(2, 49)`
Node 3	62	53	22.57	**16.25**	c_3	`(3, 62)`
	75	66	35.57	**3.25**	c_3	`(3, 75)`
	76	67	36.57	**2.25**	c_3	`(3, 76)`
	77	68	37.57	**1.25**	c_3	`(3, 77)`
	85	76	45.57	**6.75**	c_3	`(3, 85)`

Compared with the previous round the following points **changed key**: 23 (key 2 → key 1); 62 (key 2 → key 3).

Combiner (on each node, before the network):

Node	Key	Points of this key on this node	Partial sum	Count	Combiner emits (key, (sum, count))
Node 1 6 points	1	2, 3, 5, 8, 15, 21	2 + 3 + 5 + 8 + 15 + 21 = 54	6	`(1, (54, 6))`
Node 2 6 points	1	23	23	1	`(1, (23, 1))`
	2	25, 29, 43, 45, 49	25 + 29 + 43 + 45 + 49 = 191	5	`(2, (191, 5))`
Node 3 5 points	3	62, 75, 76, 77, 85	62 + 75 + 76 + 77 + 85 = 375	5	`(3, (375, 5))`
All nodes					**4 pairs shuffled** (17 without combiner)

Shuffle and reducers: all pairs with the same key travel to the same reducer, which adds the partial sums, adds the partial counts and divides.

Reducer	Receives (from the combiners)	Total sum	Total count	New centroid = sum / count
Reducer 1 (key 1)	Node 1: (54, 6); Node 2: (23, 1)	54 + 23 = 77	6 + 1 = 7	**77 / 7 = 11**
Reducer 2 (key 2)	Node 2: (191, 5)	191	5	**191 / 5 = 38.2**
Reducer 3 (key 3)	Node 3: (375, 5)	375	5	**375 / 5 = 75**

Self-check (sums and counts are conserved): 77 + 191 + 375 = 643 (the sum of the 17 data points) and 7 + 5 + 5 = 17 (the number of data points).

Round 2 result: clusters {2, 3, 5, 8, 15, 21, 23}, {25, 29, 43, 45, 49}, {62, 75, 76, 77, 85}; new centroids c_1 = 11, c_2 = 38.2, c_3 = 75. Point 23 now prefers c_1 = 9 (distance 14) to c_2 ≈ 39.43 (distance 16.43), and 62 prefers c_3 = 78.25 (16.25) to c_2 (22.57). The centroids moved (c_1: 9 → 11; c_2: 39.43 → 38.2; c_3: 78.25 → 75), so the algorithm has **not** converged and the driver starts Round 3.

5. **Part (b) — Round 3 (centroids 11, 191/5 = 38.2, 75) and convergence.**
   Centroids broadcast to every mapper this round: c_1 = 11, c_2 = 191/5 = 38.2, c_3 = 75. 1-D shortcut for checking: the boundaries are the midpoints (11 + 38.2)/2 = 24.6 and (38.2 + 75)/2 = 56.6; a point below the first boundary goes to key 1, between the two boundaries to key 2, above the second to key 3.

Node	Point p	$\|p - 11\|$ (c_1)	$\|p - 38.2\|$ (c_2)	$\|p - 75\|$ (c_3)	Nearest	Mapper emits (ID, point)
Node 1	2	**9**	36.2	73	c_1	`(1, 2)`
	3	**8**	35.2	72	c_1	`(1, 3)`
	5	**6**	33.2	70	c_1	`(1, 5)`
	8	**3**	30.2	67	c_1	`(1, 8)`
	15	**4**	23.2	60	c_1	`(1, 15)`
	21	**10**	17.2	54	c_1	`(1, 21)`
Node 2	23	**12**	15.2	52	c_1	`(1, 23)`
	25	14	**13.2**	50	c_2	`(2, 25)`
	29	18	**9.2**	46	c_2	`(2, 29)`
	43	32	**4.8**	32	c_2	`(2, 43)`
	45	34	**6.8**	30	c_2	`(2, 45)`
	49	38	**10.8**	26	c_2	`(2, 49)`
Node 3	62	51	23.8	**13**	c_3	`(3, 62)`
	75	64	36.8	**0**	c_3	`(3, 75)`
	76	65	37.8	**1**	c_3	`(3, 76)`
	77	66	38.8	**2**	c_3	`(3, 77)`
	85	74	46.8	**10**	c_3	`(3, 85)`

Compared with the previous round **no point changed key**.

Combiner (on each node, before the network):

Node	Key	Points of this key on this node	Partial sum	Count	Combiner emits (key, (sum, count))
Node 1 6 points	1	2, 3, 5, 8, 15, 21	2 + 3 + 5 + 8 + 15 + 21 = 54	6	`(1, (54, 6))`
Node 2 6 points	1	23	23	1	`(1, (23, 1))`
	2	25, 29, 43, 45, 49	25 + 29 + 43 + 45 + 49 = 191	5	`(2, (191, 5))`
Node 3 5 points	3	62, 75, 76, 77, 85	62 + 75 + 76 + 77 + 85 = 375	5	`(3, (375, 5))`
All nodes					**4 pairs shuffled** (17 without combiner)

Shuffle and reducers: all pairs with the same key travel to the same reducer, which adds the partial sums, adds the partial counts and divides.

Reducer	Receives (from the combiners)	Total sum	Total count	New centroid = sum / count
Reducer 1 (key 1)	Node 1: (54, 6); Node 2: (23, 1)	54 + 23 = 77	6 + 1 = 7	**77 / 7 = 11**
Reducer 2 (key 2)	Node 2: (191, 5)	191	5	**191 / 5 = 38.2**
Reducer 3 (key 3)	Node 3: (375, 5)	375	5	**375 / 5 = 75**

Self-check (sums and counts are conserved): 77 + 191 + 375 = 643 (the sum of the 17 data points) and 7 + 5 + 5 = 17 (the number of data points).

Round 3 result: new centroids 11, 38.2, 75 are **identical** to the centroids that were used, and no point changed cluster. The centroids stopped moving, so the algorithm has **converged** and the driver stops (Slides 24 and 63).

EXAM TIP

The last round, which changes nothing, is not wasted work: it is the proof of convergence. Write "new centroids = old centroids, no point changed cluster, stop" and you collect that mark. Counting: Rounds 1 and 2 moved the centroids, Round 3 confirmed them, so **3 MapReduce jobs** (some people count only the 2 moving rounds; say which you mean).

6. **Part (c) — Final clusters and centroids (Run 1), with a stability check.**

Cluster	Points	Size	Sum	Centroid = sum / size	Sum of squared distances to the centroid
1	{2, 3, 5, 8, 15, 21, 23}	7	77	**77 / 7 = 11**	450
2	{25, 29, 43, 45, 49}	5	191	**191 / 5 = 38.2**	444.8
3	{62, 75, 76, 77, 85}	5	375	**375 / 5 = 75**	274
Total		**17**	**643**		**1168.8**

Check 1: each centroid is the average of its own cluster (just computed). Check 2: each point is nearer to its own centroid than to any other. The boundaries are (11 + 38.2)/2 = 24.6 and (38.2 + 75)/2 = 56.6; every point of cluster 1 is below 24.6 (the largest is 23), every point of cluster 2 lies between 24.6 and 56.6 (smallest 25, largest 49) and every point of cluster 3 is above 56.6 (smallest 62). The closest call is the point 25: distance 14 to centroid 11 but only 13.2 to centroid 38.2. Both checks hold, so this is a fixed point of k-means. 7 + 5 + 5 = 17: no point lost, none counted twice. (*Beyond the slides:* the last column is the SSE, "sum of squared errors" = ∑ (point − its centroid)²; it is the number k-means tries to make small, and part (e) uses it to compare runs. Total for Run 1: 450 + 444.8 + 274 = 1168.8.)

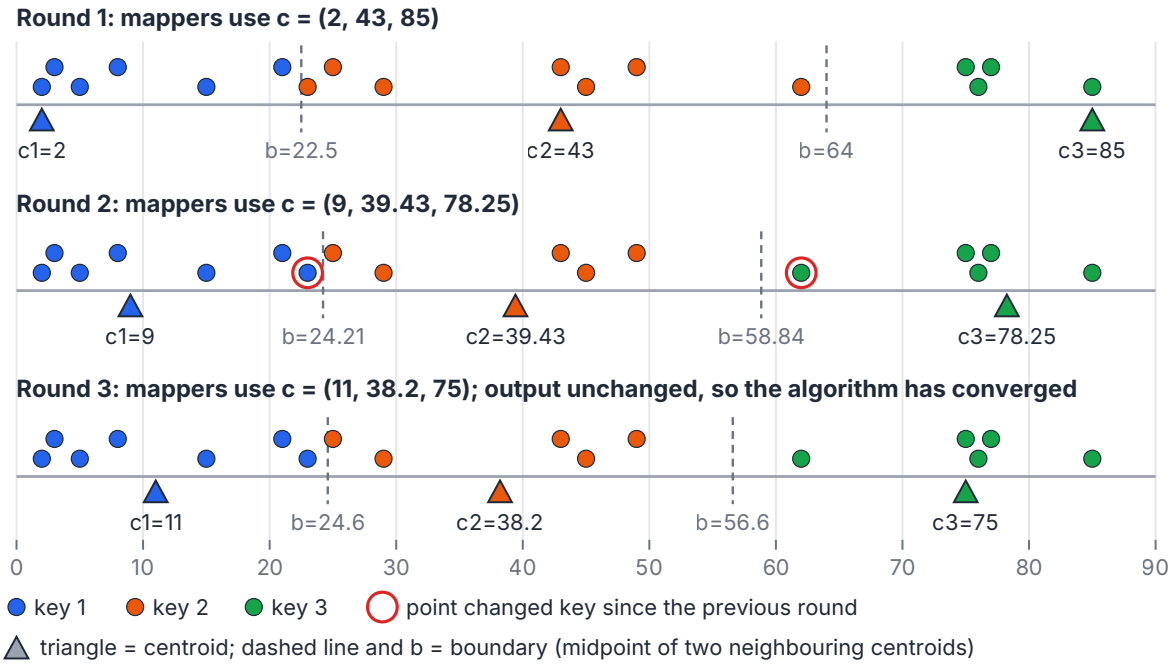


Figure L5-Q8-1. Run 1 on the number line (scale 0 to 90). Each band is one MapReduce round: dots are the 17 points coloured by the key the mappers gave them, triangles are the centroids used in that round, dashed lines are the boundaries (a point goes to the centroid on its side). Round 2 moves only the points 23 and 62 (red rings); Round 3 changes nothing.

7. **Part (d) — the answer does not depend on how the points are split over the nodes.**
Why (the argument). (1) The Mapper's decision for a point p uses only p and the 3 broadcast centroids, never the node that holds p. (2) The combiner and the reducer only **add** sums and counts, and addition is commutative and associative: (sum of the node-wise sums of key j) = (sum of all points of key j), whichever node each point sits on. So the reducers receive the same grand totals, produce the same new centroids, and by repeating the argument every round — and the final answer — is the same. The split only changes how much data crosses the network.

Demonstration. A very different split ("round-robin": the sorted points dealt out like cards, so every node holds small and large numbers): Node 1 = [2, 8, 23, 43, 62, 77], Node 2 = [3, 15, 25, 45, 75, 85], Node 3 = [5, 21, 29, 49, 76]. Same initial centroids (2, 43, 85). Round 1:

Node	Points on the node	Mapper emits	Combiner emits
Node 1	2, 8, 23, 43, 62, 77	(1, 2), (1, 8), (2, 23), (2, 43), (2, 62), (3, 77)	(1, (10, 2)), (2, (128, 3)), (3, (77, 1))
Node 2	3, 15, 25, 45, 75, 85	(1, 3), (1, 15), (2, 25), (2, 45), (3, 75), (3, 85)	(1, (18, 2)), (2, (70, 2)), (3, (160, 2))
Node 3	5, 21, 29, 49, 76	(1, 5), (1, 21), (2, 29), (2, 49), (3, 76)	(1, (26, 2)), (2, (78, 2)), (3, (76, 1))

Reducer	Adds up the partial (sum, count) pairs	New centroid
Reducer 1	(10, 2) + (18, 2) + (26, 2) = (54, 6)	**54 / 6 = 9**
Reducer 2	(128, 3) + (70, 2) + (78, 2) = (276, 7)	**276 / 7 ≈ 39.43**
Reducer 3	(77, 1) + (160, 2) + (76, 1) = (313, 4)	**313 / 4 = 78.25**

Compare the reducer totals of the two splits in every round (identical, hence identical centroids and the same final answer):

Round	Reducer totals, split 1 (sorted blocks)	Reducer totals, split 2 (round-robin)	New centroids (both splits)
1	key 1: (54, 6); key 2: (276, 7); key 3: (313, 4)	key 1: (54, 6); key 2: (276, 7); key 3: (313, 4)	**9, 276/7 ≈ 39.43, 313/4 = 78.25**
2	key 1: (77, 7); key 2: (191, 5); key 3: (375, 5)	key 1: (77, 7); key 2: (191, 5); key 3: (375, 5)	**11, 191/5 = 38.2, 75**
3	key 1: (77, 7); key 2: (191, 5); key 3: (375, 5)	key 1: (77, 7); key 2: (191, 5); key 3: (375, 5)	**11, 191/5 = 38.2, 75**

Only the network traffic differs: split 1 shuffles 4 pairs per round, split 2 shuffles 9 pairs per round, and without any combiner both would shuffle all 17 points. (Python check: 20,000 random distributions of the 17 points over 2–5 nodes, with random sizes and order, for both starts of this quiz, always gave the same reducer totals in every round and the same final centroids.)

> **COMMON MISTAKE — COMBINING AVERAGES INSTEAD OF (SUM, COUNT)**
> For key 2 in the round-robin split the three nodes send (128, 3), (70, 2), (78, 2), i.e. node averages 42.67, 35 and 39. The "average of the averages" is (42.67 + 35 + 39) / 3 = 38.89, which is **wrong**; the correct centroid is (128 + 70 + 78) / (3 + 2 + 2) = 276 / 7 = 39.43. An average cannot be merged without its count, which is exactly why the combiner emits `(sum, count)` (Slide 63). Two more traps: (1) every Mapper must use the **same** 3 global centroids in a round — a node must not cluster its own points with its own centroids; (2) after the centroids move, **all** points must be re-assigned.

8. **Part (e) — a second, different initialisation: the first three points, $c_1 = 2$, $c_2 = 3$, $c_3 = 5$** (Run 2; same nodes and the same stop rule as Run 1). The start is crowded: in Round 1 the boundaries are (2 + 3)/2 = 2.5 and (3 + 5)/2 = 4, so only the point 2 goes to key 1, only the point 3 to key 2, and *every* other point (all are ≥ 5) goes to key 3. Each round has the same Mapper / Combiner / Reducer steps as above, so it is shown compactly: Table 1 is the Mapper phase (what the mappers send to each key; the boundary column lets you check it), Table 2 is the Combiner and Reducer phase. All distances were checked; no ties.

Table 1 — Mapper phase of Run 2

Round	Centroids used by the mappers	Boundaries	Points sent to key 1	to key 2	to key 3
1	2, 3, 5	2.5 ; 4	2	3	5, 8, 15, 21, 23, 25, 29, 43, 45, 49, 62, 75, 76, 77, 85
2	2, 3, 638/15 ≈ 42.53	2.5 ; 22.77	2	3, 5, 8, 15, 21	23, 25, 29, 43, 45, 49, 62, 75, 76, 77, 85
3	2, 10.4, 589/11 ≈ 53.55	6.2 ; 31.97	2, 3, 5	8, 15, 21, 23, 25, 29	43, 45, 49, 62, 75, 76, 77, 85
4	10/3 ≈ 3.33, 121/6 ≈ 20.17, 64	11.75 ; 42.08	2, 3, 5, 8	15, 21, 23, 25, 29	43, 45, 49, 62, 75, 76, 77, 85
5	4.5, 22.6, 64	13.55 ; 43.3	2, 3, 5, 8	15, 21, 23, 25, 29, 43	45, 49, 62, 75, 76, 77, 85
6	4.5, 26, 67	15.25 ; 46.5	2, 3, 5, 8, 15	21, 23, 25, 29, 43, 45	49, 62, 75, 76, 77, 85
7	6.6, 31, 212/3 ≈ 70.67	18.8 ; 50.83	2, 3, 5, 8, 15	21, 23, 25, 29, 43, 45, 49	62, 75, 76, 77, 85
8	6.6, 235/7 ≈ 33.57, 75	20.09 ; 54.29	2, 3, 5, 8, 15	21, 23, 25, 29, 43, 45, 49	62, 75, 76, 77, 85

Table 2 — Combiner and Reducer phase of Run 2 (a combiner pair is (key, (sum, count)); the reducers add the pairs of a key over the three nodes):

Round	Node 1 combiner emits	Node 2 combiner emits	Node 3 combiner emits	Reducers: (total sum, total count) → new centroid
1	(1, (2, 1)) (2, (3, 1)) (3, (49, 4))	(3, (214, 6))	(3, (375, 5))	key 1: (2, 1) → 2 key 2: (3, 1) → 3 key 3: (638, 15) → 638/15 ≈ 42.53
2	(1, (2, 1)) (2, (52, 5))	(3, (214, 6))	(3, (375, 5))	key 1: (2, 1) → 2 key 2: (52, 5) → 10.4 key 3: (589, 11) → 589/11 ≈ 53.55
3	(1, (10, 3)) (2, (44, 3))	(2, (77, 3)) (3, (137, 3))	(3, (375, 5))	key 1: (10, 3) → 10/3 ≈ 3.33 key 2: (121, 6) → 121/6 ≈ 20.17 key 3: (512, 8) → 64
4	(1, (18, 4)) (2, (36, 2))	(2, (77, 3)) (3, (137, 3))	(3, (375, 5))	key 1: (18, 4) → 4.5 key 2: (113, 5) → 22.6 key 3: (512, 8) → 64
5	(1, (18, 4)) (2, (36, 2))	(2, (120, 4)) (3, (94, 2))	(3, (375, 5))	key 1: (18, 4) → 4.5 key 2: (156, 6) → 26 key 3: (469, 7) → 67
6	(1, (33, 5)) (2, (21, 1))	(2, (165, 5)) (3, (49, 1))	(3, (375, 5))	key 1: (33, 5) → 6.6 key 2: (186, 6) → 31 key 3: (424, 6) → 212/3 ≈ 70.67
7	(1, (33, 5)) (2, (21, 1))	(2, (214, 6))	(3, (375, 5))	key 1: (33, 5) → 6.6 key 2: (235, 7) → 235/7 ≈ 33.57 key 3: (375, 5) → 75
8	(1, (33, 5)) (2, (21, 1))	(2, (214, 6))	(3, (375, 5))	key 1: (33, 5) → 6.6 key 2: (235, 7) → 235/7 ≈ 33.57 key 3: (375, 5) → 75 **= old centroids: converged**

Run 2 needs **8 rounds** (7 moving rounds plus 1 confirming round; Run 1 needed 3) and ends with clusters **{2, 3, 5, 8, 15}**, **{21, 23, 25, 29, 43, 45, 49}**, **{62, 75, 76, 77, 85}** and centroids 33/5 = 6.6, 235/7 ≈ 33.57 and 75. This is a **different** answer from Run 1. Why it is slow: in Round 1 two centroids sit on the points 2 and 3 and capture one point each, while the third captures the other 15 points (its average,

638/15 ≈ 42.53, is far from a sensible position), so the centroids have to crawl to the right places. Only the right-hand group {62, 75, 76, 77, 85} is found by both runs.

9. **Part (e) — comparison and what it teaches.** A third start, (2, 45, 85), with the same nodes, ends in 3 rounds in yet another clustering (the best one):

Round	Centroids used by the mappers	Points sent to key 1 / key 2 / key 3	Reducers: (sum, count) → new centroid
1	2, 45, 85	2, 3, 5, 8, 15, 21, 23 25, 29, 43, 45, 49, 62 75, 76, 77, 85	key 1: (77, 7) → 11; key 2: (253, 6) → 253/6 ≈ 42.17; key 3: (313, 4) → 78.25
2	11, 253/6 ≈ 42.17, 78.25	2, 3, 5, 8, 15, 21, 23, 25 29, 43, 45, 49 62, 75, 76, 77, 85	key 1: (102, 8) → 12.75; key 2: (166, 4) → 41.5; key 3: (375, 5) → 75
3	12.75, 41.5, 75	2, 3, 5, 8, 15, 21, 23, 25 29, 43, 45, 49 62, 75, 76, 77, 85	key 1: (102, 8) → 12.75; key 2: (166, 4) → 41.5; key 3: (375, 5) → 75

(*Beyond the slides:* the start 8, 45, 76 — one point from each visible group — reaches the same clustering in only 2 rounds: Round 1 gives the reducer totals (102, 8), (166, 4), (375, 5) → 12.75, 41.5, 75, and Round 2 confirms them.) To say which answer is better we use the SSE:

Run	Initial centroids	Rounds	Final clusters	Final centroids	SSE
1 (farthest-first)	2, 43, 85	3	{2, 3, 5, 8, 15, 21, 23}, {25, 29, 43, 45, 49}, {62, 75, 76, 77, 85}	11, 38.2, 75	1168.8
2 (first three points)	2, 3, 5	8	{2, 3, 5, 8, 15}, {21, 23, 25, 29, 43, 45, 49}, {62, 75, 76, 77, 85}	6.6, 33.57, 75	1204.9
3 (another start)	2, 45, 85	3	**{2, 3, 5, 8, 15, 21, 23, 25}, {29, 43, 45, 49}, {62, 75, 76, 77, 85}**	**12.75, 41.5, 75**	**1122.5**

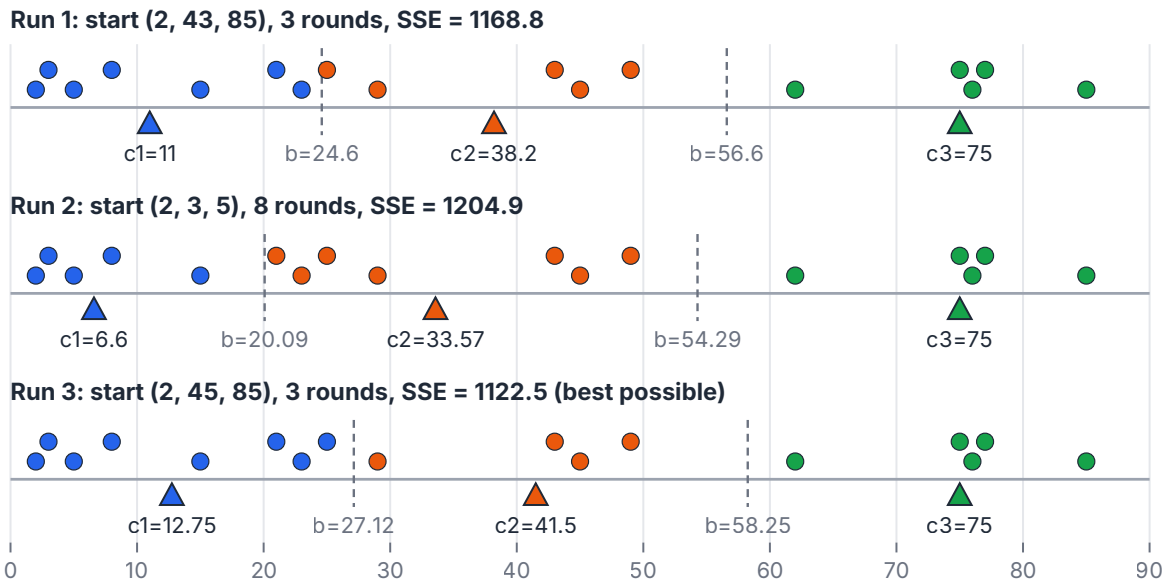


Figure L5-Q8-2. The same 17 points, three different starts, three different stable answers (colour = cluster, triangle = final centroid, dashed line = boundary). Every one of them is a fixed point of k-means, yet their quality (SSE) differs: where k-means ends depends on where it starts.

Beyond the slides: how bad can it get? A Python experiment over all 680 ways to choose 3 of the 17 points as starting centroids (listed in increasing order, ties to the lower key) shows that k-means always stops, but at one of only **six** different clusterings:

Final clusters ("a…b" = all data points from a up to b)	Centroids	SSE	Starts (of 680) ending here
Run 3 (best): {2…25}, {29, 43, 45, 49}, {62…85}	12.75, 41.5, 75	1122.5	43 (6.3%)
{2…29}, {43, 45, 49, 62}, {75, 76, 77, 85}	14.56, 49.75, 78.25	1137.7	171 (25.1%)
{2…29}, {43, 45, 49}, {62…85}	14.56, 45.67, 75	1148.9	79 (11.6%)
Run 1: {2…23}, {25…49}, {62…85}	11, 38.2, 75	1168.8	30 (4.4%)
{2…21}, {23…49}, {62…85}	9, 35.67, 75	1193.3	18 (2.6%)
Run 2: {2…15}, {21…49}, {62…85}	6.6, 33.57, 75	1204.9	339 (49.9%)

(The counts shift by a few starts if ties are broken the other way; the six clusterings and their SSE values do not.) The best clustering is also confirmed by brute force: for points on a line every good clustering consists of 3 consecutive blocks, and checking all 120 ways to cut the sorted list into 3 blocks gives the minimum SSE 1122.5. Even farthest-first, started from each of the 17 points in turn as the "random" first pick (ties in the farthest pick go to the smaller point), ends in all six clusterings, so it is a good heuristic, not a guarantee.

WHAT THIS TEACHES — INITIALISATION MATTERS, SO RUN K-MEANS SEVERAL TIMES

- **k-means stops at a local optimum, not necessarily the best clustering.** "Converged" only means "one more round changes nothing". (*Beyond the slides:* every round can only keep or lower the SSE, so the algorithm must stop — but where it stops depends on the start.) Two sensible starts gave two different, equally "converged" answers (SSE 1168.8 and 1204.9), and even the farthest-first start did not find the best one (1122.5). Farthest-first likes extreme points such as 2 and 85, which are not typical members of a group.
- **Spread-out starting centroids are usually better than clumped ones** (Slide 23 asks for centroids "as far away as possible"): the clumped start 2, 3, 5 needed 8 rounds, the most any of the 680 starts needs, against 3 for Run 1 and 2 for the start 8, 45, 76. In MapReduce every round is a full pass over all the data, so 8 rounds cost about 2.7 times as much as 3.
- **Practical rule:** run k-means several times from different (random but well-spread) initial centroids and keep the run with the smallest SSE (or the smallest average distance to the centroid, the quantity Slide 31 uses for choosing k). This is why Slides 60–63 say "random initial centroids" and Slide 23 offers two ways to pick them. (*Beyond the slides:* the k-means++ rule is a smarter random choice that favours points far from those already chosen.) In MapReduce each extra run is simply another sequence of rounds.
- **What does not matter** is the distribution of the points over the nodes (part (d)): that only changes *who computes*, never the result. The initial centroids do.

EXAM TIP — HOW TO WRITE THIS QUESTION

Because the slide leaves the start open, **write your initial centroids and your node distribution first** (marks for stating them). Then, for every round, show (1) the centroids sent to the mappers, (2) the (centroidID, point) pairs per node, (3) each node's (sum, count) per key, (4) the reducer totals and the division, (5) "moved, so continue" or "unchanged, so stop". Keep fractions exact (276/7) until the end and check that all sums add to 643 and all counts to 17 in every round. Finish with the final clusters and one sentence: "a different start may give a different local optimum, so run k-means several times". If the question *does* give the initial centroids or the node split, use theirs; any correctly executed start should be accepted. **Practice** (self-made): redo Round 1 with the centroids 8, 45, 76 on the same nodes and compare with the reducer totals in the note above.

Final answer (our stated choices: farthest-first start $\{c_1, c_2, c_3\}$ = 2, 43, 85; Node 1 = [2, 3, 5, 8, 15, 21], Node 2 = [23, 25, 29, 43, 45, 49], Node 3 = [62, 75, 76, 77, 85]):
Run 1 converges after 3 MapReduce rounds: centroids 2, 43, 85 → 9, 276/7 ≈ 39.43, 78.25 → 11, 38.2, 75 → 11, 38.2, 75 (unchanged, stop).
Clusters: {2, 3, 5, 8, 15, 21, 23} with centroid 11 (= 77/7); {25, 29, 43, 45, 49} with centroid 38.2 (= 191/5); {62, 75, 76, 77, 85} with centroid 75 (= 375/5); SSE = 1168.8.
The result is the same for any distribution of the points over the nodes (sums and counts add up the same way).
Run 2 (start 2, 3, 5) needs 8 rounds and ends in a different local optimum: {2, 3, 5, 8, 15}, {21, 23, 25, 29, 43, 45, 49}, {62, 75, 76, 77, 85} with centroids 6.6, 235/7 ≈ 33.57, 75 (SSE 1204.9). The best clustering (start 2, 45, 85 or 8, 45, 76): {2, 3, 5, 8, 15, 21, 23, 25}, {29, 43, 45, 49}, {62, 75, 76, 77, 85} with centroids 12.75, 41.5, 75 (SSE 1122.5).
Lesson: initialisation matters, so run k-means several times from different, well-spread starts and keep the best result.

Where the marks come from: (1) stating the initial centroids (k = 3 of them) and how the points are distributed over the 3 nodes, since the slide gives neither; (2) the roles and formulas: Mapper = nearest centroid with $|p - c|$, Combiner = (sum, count) because average = sum / count, Reducer = total sum / total count, driver = repeat; (3) Mapper: distance of every point to every centroid and the emitted `(centroidID, p)` pair, per node, in every round; (4) Combiner: per node and key the `(partial sum, count)`; (5) Reducer: total sum and count per key and the new centroid, in every round (9, 276/7, 313/4, then 11, 38.2, 75); (6) iterating with the *updated* centroids (Round 2 moves 23 and 62) until the centroids stop moving, with the convergence statement; (7) the final clusters and their centroids; (8) the argument that the node split does not change the result; (9) a second start, the observation that it ends in a different local optimum, and the lesson (initialisation matters, run several times). Marks are earned step by step even if a later round has an arithmetic slip.

SLIDE ERRATUM (OMISSION — NO WRONG NUMBER)

- Slide 84 specifies **neither the initial centroids nor how the 17 points are distributed over the 3 nodes**, and the slides print no answer for it. Without those the clusters are **not unique** (part (e) shows six different stable clusterings for this data). The correct version of the question needs both: this guide stated its own and showed which results depend on it (the clusters do, the node split does not).
- Slides 61–63 write the points as (x, y) (2-D) but this quiz has single numbers (1-D). The method is identical: the distance is $|p - c|$ and the average is a plain average; in 2-D the combiner would send (sum of x, sum of y, count). The slides' own worked example (Slides 64–67, k = 2, 8 points, start 5 and 21) was re-checked and is correct: Round 1 gives 4.5 and 22.5 as printed (see Quiz L5-Q7).

Slide check: the slide prints no answer, so there is nothing to contradict; the method and the arithmetic of the slides' example on Slides 62–67 agree with this solution. Slide erratum: the start (centroids and node split) is missing from Slide 84; our result is one valid answer for the stated start, and any other start gives a correct answer if its rounds are correct (compare Runs 2 and 3). Every number above was recomputed with Python (exact fractions).

Two ways to make k-means scale

Lecture 5 · Slides 43–63

	BFR	*k*-means on MapReduce
Problem attacked	data too big for one machine's memory	data too big (or too slow) for one machine's computation; spread over many
How	keep summaries (N, SUM, SUMSQ) instead of points	run the assignment and update steps as parallel map and reduce tasks
Points kept?	mostly discarded after being absorbed	all points stay on disk on their nodes

Passes	one (Beyond the slides)	one job per iteration, until centroids stop moving
Result	approximate; assumes axis-aligned normal clusters	the same clusters as ordinary *k*-means from the same start
Needs	Euclidean space, normal clusters	Euclidean space (centroid = mean)

Chapter summary and what to read next

Lecture 5 · Slides 68, 86–88

The lecture ends with three summary slides and an "Actions" slide. They add no new technical content, but their wording is what the exam is most likely to echo, so each is recorded here.

Summary (Slide 68)

Lecture 5 · Slide 68

Slide 68 is a map of the whole lecture: **Clustering**: given a set of points with a notion of distance between points, group the points into some number of *clusters*. **Algorithms**: agglomerative hierarchical clustering (with its centroid and clustroid representations), *k*-means (initialisation and picking *k*), BFR and CURE, and *k*-means MapReduce.

Topic on Slide 68	One-sentence answer	Where taught
Hierarchical, centroid and clustroid	start with every point alone and repeatedly merge the two nearest clusters; a cluster is represented by its **centroid** (average, Euclidean) or by a **clustroid** (an actual point close to the others) when averaging is impossible	Part 1 of this chapter (Slides 16–18)
k-means: initialisation, picking *k*	pick *k* seeds (random or farthest-first), assign, update, repeat; choose *k* where the average distance to centroids stops improving much	Part 2 (Slides 22–34)
CURE	sample, cluster hierarchically, pick dispersed representatives and move them 20% toward the centroid, rescan to assign each point to the nearest representative	Part 2 (Slides 35–42)
BFR	summarise clusters by N, SUM, SUMSQ; DS / CS / RS; one chunk at a time	this part (Slides 43–58)
k-means MapReduce	mapper assigns, (combiner pre-sums), reducer averages, driver iterates	this part (Slides 59–67)

Summary: clustering and algorithm families (Slide 86)

Lecture 5 · Slide 86

- **Clustering:** "Clusters are often a useful summary of data that is in the form of points in some space."
- **Clustering algorithms** generally have one of two forms:
  - **Hierarchical clustering** algorithms begin with all points in a cluster of their own, and nearby clusters are merged iteratively.
  - **Point-assignment clustering** algorithms consider points in turn and assign them to the cluster in which they best fit.

Sort the lecture into these two forms: hierarchical = Slides 16–18 (and the quiz of Slides 69–75); point-assignment = *k*-means, BFR, and the second pass of CURE (each point goes to the cluster of the nearest representative). (**Beyond the slides:** hierarchical clustering must compare all pairs of clusters, which is expensive for huge data; that is why the big-data methods are point-assignment methods, and CURE uses hierarchical clustering only on a sample.)

Summary: BFR and CURE (Slide 87)

Lecture 5 · Slide 87

- **The BFR algorithm:** "a version of *k*-means designed to handle data that is too large to fit in main memory. It assumes clusters are normally distributed about the axes."
- **Processing points in CURE:** "After creating representative points for each cluster, the entire set of points can be read from disk and assigned to a cluster. We assign a given point to the cluster of the representative point that is closest to the given point."

SLIDE ERRATUM (WORDING)

"Normally distributed about the axes" is loose. The precise assumption (Slides 44 and 79) is that clusters are **normally distributed around a centroid and are axis-aligned ellipses**. Use that wording in the exam.

Actions (Slide 88) and the missing Slide 85

Lecture 5 · Slide 88

Slide 88 (the last slide of Lecture 5) says: **review the slides, the quizzes and the examples**, and **read Chapter 7 of the course book** (*Mining of Massive Datasets* by Leskovec, Rajaraman and Ullman, Chapter 7, "Clustering"; an electronic version is on eClass). Printed slide 85 does not exist in the PDF: the numbering jumps from 84 to 86.

Where every quiz slide of the end of the lecture is covered

Lecture 5 · Slides 69–84

Slides	Slide topic	Quiz in this guide
69–75	Quiz: hierarchical clustering of 12 points down to $k = 6$, dendrogram	Quiz L5-Q2 (p. 289)
76–77	Quiz: *k*-means initialisation (farthest-first), then two rounds	Quiz L5-Q3 (p. 301)
78–79	Quiz: BFR steps and allowed shapes	Quiz L5-Q4 (p. 332)
80–81	Quiz: SUM, SUMSQ, variance (BFR numerical, not examined)	Quiz L5-Q5 (p. 341)
82–83	Quiz: CURE steps and the 20% move	Quiz L5-Q6 (p. 314)
64–67	Worked example: *k*-means MapReduce	Quiz L5-Q7 (p. 348)
84	Quiz: *k*-means MapReduce homework	Quiz L5-Q8 (p. 354)

QUICK RECAP: THIS PART

- BFR keeps **N, SUM, SUMSQ** ($2d + 1$ numbers) per cluster, plus CS groups and raw RS points; it needs normal, axis-aligned clusters.
- Each memory load: close points → DS; leftovers + old RS clustered → CS / RS; update by adding; merge CS; last round: merge CS and RS into the nearest cluster or call them outliers.
- Close enough = Mahalanobis distance < threshold (about 2 σ); merge two CS by the variance of the union or the M.D. of their centroids.
- *k*-means on MapReduce: Mapper = assign, Reducer = average, Combiner = (sum, count), Driver = iterate until the centroids stop moving.
- BFR summarises, CURE samples, MapReduce parallelises; all three keep *k*-means usable on big data.

Theory Q&A bank

Lecture 5 · Slides 1–88

Cover the answer, write your own, then compare. Numbered points are marking points. Most questions are self-made practice; the ones the slides themselves pose are Q9 and Q12 (Slides 78 and 79: BFR steps and allowed shapes), Q8 (Slide 82: CURE steps), Q11 (Slide 80: are the statistics recomputed from scratch?), and Q13 (Slide 53: Q1 and Q2 of BFR). BFR questions here are theory only; no BFR numerical is examined.

Q1. Define clustering. What does the distance measure do, and which measures does the lecture name?

Lecture 5 · Slide 4

1. Given a set of points and a notion of distance, **group the points into clusters** so that members of a cluster are **similar** (small distances) and members of different clusters are **dissimilar** (large distances).
2. The distance measure defines what "similar" means; changing it changes the clusters. Points usually live in a high-dimensional space.
3. Named measures: **Euclidean** (numeric coordinates, straight-line), **cosine** (vectors, direction), **Jaccard** (sets: $1 - |A \cap B| / |A \cup B|$), **edit distance** (strings: minimum number of single-character edits).
4. A distance is small when things are similar (Jaccard similarity 0.4 means Jaccard distance 0.6).

Q2. Why is clustering hard, especially in high dimensions?

Lecture 5 · Slides 8–9

1. Real data is messy: clusters differ in size, density and shape, they overlap, and there is noise, so there is no clean gap between clusters (Slide 8).
2. In 2 dimensions and with little data the eye can check a clustering. Applications often have 10 or 10,000 dimensions, which cannot be plotted (Slide 9).
3. **High-dimensional spaces look different: many pairs of points are at about the same distance.** Then "near" and "far" no longer differ, and distance-based methods have little to grab (the curse of dimensionality, Beyond the slides: the name).
4. Remedy mentioned by the course map (Slide 2): dimensionality reduction.

Q3. Compare hierarchical and point-assignment clustering. What are agglomerative and divisive methods and what is a dendrogram?

Lecture 5 · Slide 14

1. **Hierarchical** methods build a tree of nested clusters. **Agglomerative** = bottom up: every point starts as its own cluster and the two nearest clusters are merged repeatedly. **Divisive** = top down: start with one cluster and split recursively.
2. **Point assignment** keeps a set of clusters (their number fixed in advance) and assigns each point to the nearest one (k-means, BFR, CURE's second pass).
3. A **dendrogram** is the tree of merges; leaves are points, the height of a bar is the distance at which the merge happened; cutting it at a level gives any number of clusters.
4. Hierarchical needs all pairwise cluster distances (expensive); point assignment is the basis of the big-data methods.

Q4. Describe agglomerative clustering in Euclidean and non-Euclidean spaces. What are the centroid and the clustroid?

Lecture 5 · Slides 16–18

1. Key operation: **repeatedly combine the two nearest clusters**. Two decisions: how to represent a cluster of many points, and how to measure nearness of clusters.
2. **Euclidean:** a cluster is represented by its **centroid** = the average of its points (coordinate by coordinate); the distance between clusters is the distance between their centroids. After a merge the new centroid is the weighted average $(n_P \cdot c_P + n_Q \cdot c_Q) / (n_P + n_Q)$, not the midpoint of the two centroids.
3. **Non-Euclidean** (sets, strings): there is no average of two points, so use the **clustroid** = the data point "closest" to the other points of the cluster, treated as if it were the centroid. It is always an actual data point.
4. Beyond the slides: "closest" is commonly the point with the smallest sum of distances (or smallest maximum, or smallest sum of squares) to the others; state the rule you use.

Q5. Why is hierarchical clustering not used for big data?

Lecture 5 · Slide 21

1. Naive implementation: at each step compute the distances between all pairs of clusters, then merge the closest pair: cost **$O(N^3)$**.
2. Reason: about $N - 1$ merges, each looking at up to about $N^2/2$ pairs (for $N = 12$ the exact count is 286 distance computations, trivial; for $N = 10^6$ it is hopeless).
3. Beyond the slides: a priority queue gives $O(N^2 \log N)$, still too slow, and the $O(N^2)$ pairwise distances do not fit in memory.
4. Slide's conclusion: too expensive for really big data sets that do not fit in memory. That motivates CURE (hierarchical clustering only on a sample), BFR and MapReduce.

Q6. Describe the k-means algorithm: what it needs, how it starts, the loop, and when it stops.

Lecture 5 · Slides 23–27

1. **Needs:** Euclidean space and distance (so that the centroid, an average, exists) and a chosen number of clusters k.
2. **Seeds:** pick k starting points, either k random points, or one random point and then $k - 1$ points each as far as possible from the previous ones (farthest-first: choose the candidate whose *smallest* distance to the chosen seeds is largest).
3. **Loop:** (a) assign each point to its nearest centroid; (b) after all points are assigned, move each centroid to the mean of its points; (c) reassign all points to the nearest (new) centroid, which sometimes moves points between clusters; repeat (b) and (c).
4. **Convergence:** stop when points no longer move between clusters and the centroids stabilise.
5. Beyond the slides: the result can depend on the seeds (local optimum), so spread-out seeds are better.

Q7. How do you choose k? What do "too few" and "too many" look like?

Lecture 5 · Slides 31–34

1. Try different k and plot the **average distance to the centroid** against k.
2. It **falls rapidly until the right k, then changes little**; choose k at the bend (Beyond the slides: the "elbow").
3. **Too few** (Slide 32): natural groups are lumped together and many distances to the centroid are long.
4. **Just right** (Slide 33): one centroid per natural group, distances rather short.
5. **Too many** (Slide 34): a natural group is split and the average distance improves little.
6. Do not pick the k with the smallest average distance: that is always the largest k tried.

Q8. Describe in steps how CURE works. Why does it use several representative points instead of one centroid?

Lecture 5 · Slides 35–42, 82–83, 87

1. CURE (Clustering Using REpresentatives): Euclidean distance, clusters of **any shape**, a **collection of representative points** per cluster; a **2-pass** algorithm for large data.
2. **Pass 1:** (0) take a random sample that fits in main memory; (1) cluster it hierarchically; (2) for each cluster pick a sample of points as dispersed as possible (say 4), then move each (say) **20% toward the cluster's centroid**: new = p + 0.2 (c − p). These moved points are the representatives.
3. **Pass 2:** rescan the whole data; assign each point p to the cluster of the **closest representative**.
4. **Why representatives:** one centroid describes only a round blob; several points along a curved or elongated cluster (a ring, a crescent) let a point find the right cluster. Moving them inward damps the effect of outliers (Beyond the slides).

Q9. What is BFR, what does it assume, and which cluster shapes does it allow?

Lecture 5 · Slides 43–45, 78–79, 87

1. **BFR** (Bradley–Fayyad–Reina) is a variant of k-means for **very large, disk-resident data**. Points are read in chunks (memory loads); most are summarised by simple statistics and discarded. Memory is O(clusters), not O(data).
2. It **assumes** each cluster is **normally distributed around a centroid in a Euclidean space** and that the dimensions are independent, so it is an **axis-aligned ellipse**.
3. **Allowed** (Slide 79): horizontal ellipse OK, vertical ellipse OK (circles too), **tilted ellipse not OK**.
4. Reason: standard deviations may differ along different axes, but the axes of the cluster must align with the axes of the space. BFR stores per-dimension totals only, so it cannot capture correlation between dimensions (a tilt). Rings and other non-normal shapes are for CURE.

Q10. Define the discard set, compression set and retained set. What is kept in memory for each, and why have all three?

Lecture 5 · Slides 46–47, 52

1. **DS (discard set):** points close enough to a cluster centroid; assigned to that cluster; only the summary (N, SUM, SUMSQ) is kept, the points are discarded.
2. **CS (compression set):** groups of points that are close together but not close to any centroid; summarised (also N, SUM, SUMSQ) but **not assigned to a cluster**; points discarded.
3. **RS (retained set):** isolated points waiting to be assigned to a compression set; kept in memory **as they are**.
4. Why: DS keeps memory small; CS keeps a possible new cluster without storing its points; RS avoids summarising single points, which give no useful spread (one point has variance 0).

Q11. What does BFR store for a cluster, why is that enough, and must it be recomputed from scratch each round?

Lecture 5 · Slides 48–49, 80–81

1. For each cluster: **N** (number of points), **SUM** (vector of per-dimension sums of coordinates), **SUMSQ** (vector of per-dimension sums of squared coordinates): **2d + 1 numbers** whatever the cluster size.
2. Centroid in dimension i = SUM_i / N. Variance in dimension i = $SUMSQ_i$ / N − (SUM_i / N)^2; standard deviation = its square root. So centre and spread can be read at any time.
3. **No: the statistics are not recomputed from scratch.** A new point x is absorbed by N + 1, SUM + x, SUMSQ + x^2; two summaries merge by adding their N, SUM and SUMSQ. This is at the heart of BFR (Slide 81).
4. Because only per-dimension totals are kept, only per-dimension variances exist, matching the axis-aligned assumption.

Q12. Describe in steps how BFR processes the data.

Lecture 5 · Slides 45, 50–51, 78

1. **Start:** choose k initial centroids from the first memory load by a sensible approach (e.g. k random points).
2. **Step 1:** points sufficiently close to a cluster centroid are added to that cluster (DS) and discarded.
3. **Step 2:** cluster the remaining points together with the old RS using any main-memory clustering algorithm; clusters go to the CS, outlying points to the RS.
4. **Step 3:** update the DS statistics (add the Ns, SUMs and SUMSQs).
5. **Step 4:** consider merging CS groups that are close enough (add their statistics).
6. **Step 5 (last round only):** merge all CS groups and RS points into their nearest cluster, or treat them as outliers.
7. Steps 1–4 repeat for each memory load.

Q13. How does BFR decide whether a point is close enough to a cluster (Q1), and whether two CS sets should be merged (Q2)?

Lecture 5 · Slides 53–58

1. **Q1:** use the **Mahalanobis distance** (normalised Euclidean distance from the centroid): $y_i = (x_i - c_i) / \sigma_i$, then $d = \sqrt{(\Sigma\, y_i^2)}$. Accept the point if d is below a threshold; common practice is **M.D. < 2** (about 2 standard deviations).
2. Why not plain Euclidean distance: it ignores that a cluster is stretched more in some directions; dividing by σ_i measures each direction in units of that cluster's own spread. c_i and σ_i come from N, SUM, SUMSQ, so no raw points are needed.
3. **Q2 (way 1):** add the two summaries and compute the **variance of the combined subcluster**; if it stays small (still tight), merge.
4. **Q2 (way 2):** compute the Mahalanobis distance between the two centroids using a pooled σ per dimension, either $\sqrt{(((\sigma^{(1)})^2 + (\sigma^{(2)})^2) / 2)}$ or $\max(\sigma^{(1)}, \sigma^{(2)})$; merge if below a threshold such as 2.

Q14. Describe k-means on MapReduce. Why must the combiner send (sum, count) and not an average?

Lecture 5 · Slides 60–67, 84

1. **Initialise** with random centroids.
2. **Mapper:** for each point find the nearest centroid and emit (centroidID, point). This is the assignment step.
3. **Combiner (optional):** each node collapses its pairs per centroid into (centroidID, (partial sum, count)); allowed because the average decomposes into sum and count.
4. **Reducer:** for each centroid add the sums, add the counts, divide: the new centroid = mean of its points (the update step).
5. **Iterate**: one MapReduce job per k-means iteration, until the centroids stop moving (or a predefined number of iterations).
6. An average of local averages is wrong when the nodes hold different numbers of points (nodes with {2, 3, 5} and {100}: true mean 27.5, average of averages 51.67); sums and counts add correctly.

Q15. Compare k-means, CURE, BFR and k-means on MapReduce. How does each cope with big data?

Lecture 5 · Slides 23–27, 35–67, 86–87

1. **k-means:** Euclidean, one centroid per cluster, roundish blobs, many passes; the data must fit in memory.
2. **CURE: samples** (memory-sized sample clustered hierarchically), keeps representatives, any shape, two passes, no iteration.
3. **BFR: summarises** (N, SUM, SUMSQ per cluster; DS, CS, RS), reads each chunk once, assumes normal axis-aligned clusters.
4. **k-means on MapReduce: parallelises**: the same clusters as ordinary k-means from the same start, spread over many machines; one job per iteration.
5. All four are Euclidean (a centroid is an average). Hierarchical clustering is the contrast: it works with clustroids but costs $O(N^3)$.

One-page cheat sheet

Lecture 5 · Slides 1–88

Definitions and formulas

Term	Definition / formula	Slide
Clustering	group points so members of a cluster are similar (small distances) and members of different clusters dissimilar	4
Distance measures	Euclidean ($\sqrt{\Sigma(x_i - y_i)^2}$), cosine ($1 - \cos\theta$), Jaccard ($1 - \lvert A \cap B\rvert / \lvert A \cup B\rvert$), edit distance (fewest character edits)	4
Outlier	point far from every cluster, in none of them	6
High dimensions	many pairs of points are at about the same distance	9
Centroid / clustroid	centroid = average of the points (Euclidean, usually not a data point); clustroid = data point "closest" to the others (any space)	16, 18
Merged centroid	$(n_P c_P + n_Q c_Q) / (n_P + n_Q)$ = average of all points	16–17
Hierarchical cost	naive $O(N^3)$; about $N - 1$ merges, each over about $N^2/2$ pairs	21
Farthest-first seeds	next seed = candidate whose smallest distance to the chosen seeds is largest	23, 77
k-means convergence	points do not move between clusters; centroids stabilise	24
Choosing k	average distance to centroid vs k: falls rapidly, then flat; pick the bend	31–34
CURE move	representative = p + 0.2 (c − p) (20% toward the centroid, say)	38, 41, 83
BFR summary	N, SUM, SUMSQ: 2d + 1 numbers per cluster; centroid $c_i = SUM_i/N$; $\sigma_i^2 = SUMSQ_i/N - (SUM_i/N)^2$	48–49
Update by addition	N + 1, SUM + x, SUMSQ + x^2; merging two sets = adding their statistics	51, 81
Mahalanobis distance	$\sqrt{\Sigma((x_i - c_i) / \sigma_i)^2}$; accept if < threshold, common practice 2	55–56
Pooled σ for merging CS	$\sqrt{(((\sigma^{(1)})^2 + (\sigma^{(2)})^2) / 2)}$, or $\max(\sigma^{(1)}, \sigma^{(2)})$	58
DS / CS / RS	DS: close to a centroid, summary only. CS: tight groups far from centroids, summary only, unassigned. RS: isolated points kept raw	46
MapReduce k-means	Map: (centroidID, point). Combine: (centroidID, (sum, count)). Reduce: mean. Repeat until centroids stop moving	60–63

Algorithms in a few steps

Algorithm	Steps
Agglomerative	(1) Every point is a cluster. (2) Find the two clusters with the nearest centroids (clustroids). (3) Merge them; new centroid = weighted average. (4) Repeat until k clusters remain (or the whole dendrogram is built).
k-means	(1) Choose k and k seeds (random or farthest-first). (2) Assign each point to its nearest centroid. (3) Move each centroid to the mean of its points. (4) Reassign; repeat 3 and 4 until nothing moves.
CURE	Pass 1: (1) random sample in memory; (2) cluster it hierarchically; (3) per cluster take (say) 4 dispersed points; (4) move them (say) 20% toward the centroid. Pass 2: rescan all data; each point goes to the cluster of the closest representative.
BFR (per memory load)	(1) Close points to DS (add to N, SUM, SUMSQ). (2) Cluster the rest plus the old RS: clusters to CS, outliers to RS. (3) Update DS statistics. (4) Merge close CS groups. (5) Last load: merge CS and RS into the nearest cluster or call them outliers.
MapReduce k-means	(1) Random centroids. (2) Mappers emit (nearest centroid ID, point). (3) Combiner: (ID, (partial sum, count)). (4) Reducer: sums, counts, divide, giving new centroids. (5) Repeat until they stop moving.

Comparison table

	Hierarchical	k-means	CURE	BFR	k-means MapReduce
Family	hierarchical	point assignment	point assignment (hierarchical on the sample)	point assignment	point assignment
Cluster represented by	centroid or clustroid	centroid	representatives	N, SUM, SUMSQ	centroid
Space	any (clustroid)	Euclidean	Euclidean	Euclidean	Euclidean
Shape	depends on the cluster-distance rule (centroid rule: compact blobs)	round blobs	**any shape**	normal, axis-aligned ellipses	round blobs
Big data	$O(N^3)$	must fit in memory	sample plus 2 passes	summaries, memory loads	parallel on many nodes

Common mistakes & traps

Lecture 5 · Slides 1–88

- **Midpoint instead of weighted average.** A point joining a two-point cluster with centroid (1.5, 1.5): the new centroid is (1·p + 2·(1.5, 1.5)) / 3, not the midpoint of p and (1.5, 1.5) (Slide 17: (1, 1), not (0.75, 0.75)).
- **Centroid for strings or sets.** Under Jaccard or edit distance there is no average; use a clustroid, which must be an actual data point.
- **Farthest-first with k > 2.** Use the smallest distance to all chosen seeds, then take the largest; not the distance to the last seed only, and not a sum.
- **Skipping steps in k-means by hand.** Show the distance of every point to every centroid, underline the smallest, recompute each centroid with its count, and state the round. Do not stop after the update without reassigning.
- **Picking k with the smallest average distance.** It is always the largest k; choose the bend where the drop stops.
- **CURE: centroids as representatives.** Representatives are dispersed points moved 20% toward the centroid (generally not data points); a point goes to the cluster of the nearest *representative*, not the nearest centroid. CURE needs two passes and no iteration.
- **BFR shapes.** Horizontal and vertical ellipses are allowed; tilted ones are not. Rings and crescents are not BFR clusters either (use CURE).
- **BFR sets.** The CS is summarised but not assigned to a cluster; the RS is kept as raw points; "all points are discarded" is false.
- **BFR statistics.** SUMSQ is the sum of squares, not $(SUM)^2$; SUM and SUMSQ are vectors; variance = $SUMSQ/N - (SUM/N)^2$ (divide by N); statistics are updated by adding, never recomputed from old points; do not average two standard deviations to get the union's.
- **Mahalanobis versus Euclidean.** Divide each coordinate difference by σ_i of that cluster before squaring; a point can be far in Euclidean terms yet accepted (and the reverse).
- **MapReduce combiner.** Send (sum, count), not a local average. One job is one k-means iteration; on Slide 84 state your own initial centroids, node split and stopping rule, as the slide gives none.
- **Trusting slide answers blindly.** Known slide quirks: Slide 6 labels only one of the two outliers; Slide 41 draws the move as about one third, not 20%; Slide 81 prints "110/3 − 18/3²", which needs brackets: $110/3 - (18/3)^2 = 0.667$; Slide 87 says "normally distributed about the axes", loosely.

REFERENCE

Master cheat sheet, quiz index & glossary

Quiz index — every quiz and worked example, solved

Cover the solution, try the quiz yourself, then compare. Slide numbers are the printed numbers on the slides.

Quiz	Topic	Where on the slides	Solved on
Quiz L1-Q1	Bonferroni's principle: the hotel example	Lecture 1 · Slides 12–14	(p. 14)
Quiz L2-Q1	How long to read 200 TB from disk?	Lecture 2 · Slides 5–8	(p. 35)
Quiz L2-Q2	Cluster failures and the network bottleneck (the numbers)	Lecture 2 · Slides 9–14	(p. 41)
Quiz L2-Q3	Word count with Map-Reduce (the warm-up task)	Lecture 2 · Slides 26–42	(p. 59)
Quiz L2-Q4	Host size: total bytes per host	Lecture 2 · Slides 45–47	(p. 68)
Quiz L2-Q5	Counting 5-word sequences in a large corpus (language model)	Lecture 2 · Slides 48–49	(p. 76)
Quiz L2-Q7	Combiners: sum vs average vs median	Lecture 2 · Slides 59–65	(p. 93)
Quiz L2-Q6	Natural join R(A,B) ⋈ S(B,C) with Map-Reduce	Lecture 2 · Slides 70–74	(p. 107)
Quiz L4-Q1	Support and frequent itemsets (two examples)	Lecture 4 · Slides 8–9	(p. 157)
Quiz L4-Q2	Confidence and interest of an association rule	Lecture 4 · Slides 10–15	(p. 166)
Quiz L4-Q3	Generating association rules from frequent itemsets	Lecture 4 · Slides 16–18	(p. 174)
Quiz L4-Q4	Memory needed to count pairs: matrix vs triples	Lecture 4 · Slides 20–28	(p. 185)
Quiz L4-Q5	A-Priori step by step: C1, L1, C2, L2, C3, L3	Lecture 4 · Slides 29–36	(p. 197)
Quiz L4-Q8	A-Priori on integer baskets (threshold 3)	Lecture 4 · Slides 78–80	(p. 204)
Quiz L4-Q9	PCY: the bitmap — how it is used and why	Lecture 4 · Slides 81–82 (theory on Slides 39–45)	(p. 214)
Quiz L4-Q10	Multistage and Multihash: key ideas	Lecture 4 · Slides 83–84 (theory: Slides 46–47, 49–51)	(p. 225)
Quiz L4-Q6	SON algorithm: the two-chunk MapReduce example	Lecture 4 · Slides 56–69	(p. 243)
Quiz L4-Q7	Cat-and-dog baskets: pairs, triples, confidence, interest	Lecture 4 · Slides 70–77	(p. 255)

Quiz L5-Q1	Hierarchical clustering example and dendrogram	Lecture 5 · Slides 17–18	(p. 281)
Quiz L5-Q2	Hierarchical clustering of 12 points down to k = 6	Lecture 5 · Slides 69–75	(p. 289)
Quiz L5-Q3	k-means on 12 points (k = 3): farthest-point start, then two rounds	Lecture 5 · Slides 76–77	(p. 301)
Quiz L5-Q6	CURE: steps, and moving the representatives 20% toward the centroid	Lecture 5 · Slides 82–83 (method: Slides 35–42, 87)	(p. 314)
Quiz L5-Q4	BFR: steps of the algorithm and allowed cluster shapes (theory)	Lecture 5 · Slides 78–79	(p. 332)
Quiz L5-Q5	BFR summary statistics: N, SUM, SUMSQ, centroid, variance, standard deviation	Lecture 5 · Slides 80–81	(p. 341)
Quiz L5-Q7	k-means with MapReduce — worked example (2, 21, 5, 8, 15, 3, 29, 25)	Lecture 5 · Slides 59–67	(p. 348)
Quiz L5-Q8	k-means MapReduce on 17 points, k = 3, on 3 nodes	Lecture 5 · Slide 84	(p. 354)

Glossary (A–Z)

Term	Meaning	Where
1-pass job	A job that reads the data, maps, reduces and writes once; what Hadoop MapReduce was designed for.	L3 · Slide 21
1/3 rule	The triples method beats the triangular matrix only if fewer than 1/3 of the possible pairs occur (12·O < 4·T).	L4 · Slide 27
2/3 rule	PCY must eliminate more than about 2/3 of A-Priori's candidate pairs to win, because triples cost 12 bytes versus 4.	L4 · Slide 45
A-Priori	Exact algorithm that counts items in Pass 1 and then only pairs of frequent items, using monotonicity; one pass per itemset size.	L4 · Slides 29–36
Action	Operation that returns a value or exports data and forces computation (count, collect, reduce, save).	L3 · Slides 11, 15
Actionable insight	A finding you can make a decision or take an action on.	L1 · Slide 5
Agglomerative clustering	Bottom-up hierarchical clustering: every point starts as its own cluster and the two nearest clusters are merged repeatedly.	L5 · Slides 14, 16
Analytic processing	The database view of data science: queries that examine large amounts of data.	L1 · Slide 16
Apache Spark	Memory-optimised data-flow engine that extends MapReduce with fast data sharing, DAGs and richer functions.	L3 · Slides 8–10
API	Application programming interface: the set of functions a program may call.	L3 · Slide 11
Artificial intelligence (AI)	A field overlapping this class, named in the text of slide 17.	L1 · Slide 17
Association rule	An if-then rule I → j meaning that baskets containing I are likely to contain j.	L4 · Slides 4, 10
Association rules	If-then rules between items found in frequent itemsets; an application on the course map.	L1 · Slide 20
Automation for large data	The system, not a person, handling the size of the data.	L1 · Slide 17
Availability	Data can still be read while some machines are down.	L2 · Slides 13, 16
Average distance to centroid	The mean, over all points, of the distance to the centroid of the point's own cluster; plotted against k to choose k.	L5 · Slide 31
Axis-aligned ellipse	A cluster shape whose long and short directions are parallel to the axes of the space; the shape BFR allows (a tilted ellipse is not allowed).	L5 · Slides 44, 79
Backbone	The higher-level network (2–10 Gbps) that connects the racks of a data centre.	L2 · Slide 10
Backup task	A duplicate of a still-running task started near the end of a phase on another machine; the first copy to finish wins.	L2 · Slide 68
Basket	A small set of items, for example the products bought in one trip; order and repeats do not matter.	L4 · Slide 3
BFR	Bradley–Fayyad–Reina: a variant of k-means for very large, disk-resident data that keeps only summary statistics of clusters, so memory is O(clusters), not O(data).	L5 · Slides 43–44
Big data	Data so large, fast or varied that ordinary single-machine tools cannot store or analyse it, and which therefore needs special methods and systems.	L1 · Slides 2–4
Big data analytics	Analysing very large data to find knowledge; slide 8 writes it as approximately equal to data science.	L1 · Slide 8
Bit-vector (bitmap)	One bit per bucket, 1 if the bucket count reached s; takes 1/32 of the memory of 4-byte counters.	L4 · Slide 42
Bloom filter	A compact probabilistic set-membership test that may give false positives but no false negatives.	L1 · Slide 19
Bogus finding	A discovered pattern that is meaningless because random data would produce as many.	L1 · Slides 12–14

Bonferroni's principle	If the expected number of matches in random data is much larger than the number of real cases hoped for, then almost any finding is bogus.	L1 · Slides 12–14
Bucket	A hash-table cell holding only a count (a few bytes), the sum of the counts of all pairs hashing to it.	L4 · Slide 39
C_k	The candidate k-itemsets: those that might be frequent given the pass for k–1.	L4 · Slide 34
Cache-aware	Scheduler behaviour that reuses cached data and respects locality.	L3 · Slide 17
Caching	Keeping data in memory so repeated queries or iterations avoid re-reading from disk.	L3 · Slides 10, 13
Candidate itemset (SON)	An itemset found frequent in at least one chunk.	L4 · Slide 56
Candidate pair	A pair that passes the tests for being counted in Pass 2 (both items frequent, in PCY also a frequent bucket).	L4 · Slides 31, 43
Centroid	The coordinate-wise average of the points of a cluster; it usually is not a data point and exists only in Euclidean space.	L5 · Slides 16, 24
Chunk	A fixed-size piece (16–64 MB) of a big file stored on a chunk server; called a block in HDFS.	L2 · Slide 19
Chunk server	A machine that stores chunks on its disk and can also run computation on them.	L2 · Slides 19, 23
Click stream	The continuous record of every click a user makes on a site or app.	L1 · Slide 3
Client library	Code the application uses to access the DFS: it asks the master for locations, then talks directly to chunk servers.	L2 · Slide 24
Cloud computing	The ability to rent computing by the hour, plus services such as persistent storage.	L2 · Slide 77
Cluster	A large group of ordinary computers wired together and used as one big computer.	L2 · Slides 7, 9–10
Clustering	Grouping similar items together without being told the groups in advance; the slide's example of a descriptive method.	L1 · Slides 11, 20
Clustroid	In a non-Euclidean space, the data point of a cluster that is closest to the other points; it represents the cluster in place of a centroid.	L5 · Slide 18
Collision	Two different pairs hashing to the same bucket.	L4 · Slide 39
Combiner	Mapper-side pre-aggregation of the values of each key before the shuffle; needs a commutative and associative reduce.	L2 · Slides 59–62
Commodity hardware	Ordinary, cheap, off-the-shelf computers and networking (Ethernet) used instead of a custom supercomputer.	L2 · Slides 7–8
Communication cost	Total I/O of all processes: how much data all tasks read and write.	L2 · Slide 73
Commutative and associative	Order and grouping of the operation do not matter (as for sum); the condition for using the reduce function as a combiner.	L2 · Slide 62
Compression set (CS)	In BFR, groups of points that are close together but not close to any centroid; summarised but not assigned to a cluster.	L5 · Slide 46
Computation cost	Analogous to communication cost but counting only the running time of the processes.	L2 · Slide 73
Computing architectures	The hardware and software setup for large data, such as clusters, distributed file systems and MapReduce/Spark.	L1 · Slide 17
Confidence	conf(I → j) = support(I ∪ {j}) ÷ support(I): the fraction of baskets containing I that also contain j.	L4 · Slide 10
Confidence threshold (c)	The minimum confidence a rule must have to be output.	L4 · Slide 16
Convergence	The state in which points no longer move between clusters and the centroids stabilise, so k-means stops.	L5 · Slide 24
Cosine distance	One minus the cosine of the angle between two vectors; it compares direction, not length.	L5 · Slide 4

CS theory	The field of algorithms and their provable efficiency.	L1 · Slide 16
CURE	Clustering Using REpresentatives: an extension of k-means for Euclidean data with clusters of any shape and large data, using several representative points per cluster.	L5 · Slides 35–36
CURE Pass 2	Rescanning the whole data and putting each point into the cluster of its closest representative.	L5 · Slide 42
Curse of dimensionality	In very many dimensions (10 or 10,000) many pairs of points are at about the same distance, so near and far stop being informative.	L5 · Slide 9
DAG	Directed acyclic graph: a graph with directed edges and no cycles.	L3 · Slides 7, 10, 17
Data analytics	Discovering knowledge and actionable insights from data.	L1 · Slides 5, 10
Data centre	A building holding many racks of servers joined by switches and a backbone.	L2 · Slides 10–11
Data lake	A central repository holding large amounts of raw data from many sources (beyond the slides).	L3 · Slide 9
Data locality	Running a task on the machine (or at least rack) that already holds its input chunk, so no network transfer is needed.	L2 · Slide 23
Data modalities	The forms data takes: ontologies, structured data, graphs, text, multimedia.	L1 · Slide 15
Data operators	The steps from raw data to insight: collect, prepare, model, reason, visualize.	L1 · Slide 15
Data quality	The challenge that data is dirty, missing or inconsistent.	L1 · Slide 15
Data science	The field combining databases, machine learning and algorithms to extract knowledge from data; treated as roughly equal to big data analytics in this course.	L1 · Slides 8, 16–17
Data-flow system	A system where tasks form a network with any number of ranks and any functions, data flowing one way.	L3 · Slide 7
Database system (DB)	A system for storing and querying large-scale data; one of the three fields data science overlaps.	L1 · Slide 16
DataFrame	Data organised into named columns, like a relational table, on a distributed collection.	L3 · Slides 12, 18
Dataset	Extension of DataFrame with a type-safe, object-oriented interface and compile-time error detection.	L3 · Slides 12, 18
Dendrogram	The tree that records a hierarchical clustering; leaves are points and the height of a bar is the distance at which two clusters were merged.	L5 · Slides 14, 17
Descriptive methods	Methods that find human-interpretable patterns describing the data.	L1 · Slide 11
Deterministic	Same input always gives the same output, so lost pieces can be recomputed.	L3 · Slide 15
Dimensionality reduction	Cutting the number of attributes while keeping the main structure.	L1 · Slide 20
Diminishing returns (multistage)	Each extra stage adds a bitmap that stays in memory, so the bit-vectors eventually consume all of main memory.	L4 · Slide 51
Directed multigraph	Graph with directed edges where several edges may join the same pair of vertices.	L3 · Slide 20
Discard set (DS)	In BFR, points close enough to a cluster centroid; only the cluster summary is kept and the points are discarded.	L5 · Slide 46
Disk I/O overhead	The cost of writing intermediate results to disk and reading them back between MapReduce steps.	L3 · Slides 5–6
Disk read bandwidth	How fast one disk delivers data (30–50 MB/s on the slides, 50 MB/s assumed); the bottleneck of a single node.	L2 · Slides 5–6
Dispersed (remote) points	Points of a cluster chosen far from each other, tracing its outline; the starting point for CURE's representatives.	L5 · Slides 38, 40

Distance measure	A function giving a number for how far apart two points are; small means similar, and it defines what a cluster is.	L5 · Slide 4
Distributed computing	A model of computation where data and work are spread over many machines.	L1 · Slide 18
Distributed file system (DFS)	A file system that stores data across a cluster and stores each piece multiple times for redundancy and availability.	L2 · Slide 17
Divisive clustering	Top-down hierarchical clustering: start with one cluster containing all points and split it recursively.	L5 · Slide 14
Does not compose well	MapReduce jobs can only be chained through files on disk, so multi-step applications are slow and awkward.	L3 · Slide 6
Duplicate document detection	Finding near-identical documents or pages.	L1 · Slides 19–20
Dynamic programming	Solving a problem by reusing answers to smaller sub-problems.	L1 · Slide 19
EC2, S3, EMR	Amazon's Elastic Compute Cloud (rentable machines), S3 storage, and Elastic Map Reduce (hosted Hadoop).	L2 · Slide 77
Edit distance	The minimum number of single-character edits needed to turn one string into another.	L5 · Slide 4
Elbow	The bend in the curve of average distance versus k after which the average changes little; the best k.	L5 · Slides 31–34
Environment (runtime)	The Map-Reduce system that handles scheduling, communication, partitioning, group by key and failures.	L2 · Slide 53
Euclidean distance	The straight-line distance, the square root of the sum of squared coordinate differences; needs numeric coordinates.	L5 · Slides 4, 23
Expected number	The average count of an event, computed as the number of opportunities times the probability of each.	L1 · Slides 13–14
False negative	An itemset frequent in the whole data that is missed because it is not frequent in the sample.	L4 · Slide 55
False positive	An itemset that looks frequent in the sample (or candidate list) but is not frequent in the whole data.	L4 · Slide 55
Farthest-first initialisation	Choosing one random point first and then k − 1 further points, each as far away as possible from those already chosen.	L5 · Slide 23
Fast data sharing	Spark's way of passing results in memory and caching data instead of saving intermediates to disk.	L3 · Slide 10
Fine granularity	Cutting a job into many small tasks, with far more map tasks than machines.	L2 · Slides 58, 69
Flat file	A plain disk file storing the baskets one after another, with no index or database structure.	L4 · Slide 20
FP-growth	Another in-memory frequent-itemset algorithm named on Slide 59 as an alternative to A-Priori and PCY for mining a chunk.	L4 · Slide 59
Frequent bucket	A bucket whose count is at least s; it may contain no frequent pair at all.	L4 · Slide 41
Frequent items	Items whose Pass 1 count is at least s; only they can appear in a frequent pair.	L4 · Slide 31
Frequent itemset	An itemset whose support is at least s.	L4 · Slide 8
Gbps	Gigabits per second (10^9 bits per second); 1 Gbps equals 125 MB/s because 1 byte is 8 bits.	L2 · Slides 9, 14
General execution graphs	Spark's ability to run any DAG of tasks, not just map then reduce.	L3 · Slide 10
GFS	Google File System, Google's distributed file system (chunks, chunk servers, master).	L2 · Slides 17, 24, 76
Grading split	Projects 20%, Midterm I 35%, presentation and Q&A 10%, Midterm II 35%.	L1 · Slide 26

Graph data	Data made of nodes connected by links, such as the web or social networks.	L1 · Slides 18, 20
GraphX	Spark library for graph computation, extending the RDD with a directed multigraph abstraction.	L3 · Slides 8, 20
Group by key	The framework step that collects all pairs with the same key together; done by sorting and shuffling.	L2 · Slides 27, 32–33, 37
Hadoop	Open-source Java implementation of Map-Reduce using HDFS for stable storage.	L2 · Slide 76
Hadoop MapReduce	Disk-based implementation of MapReduce that persists to disk after every map or reduce action.	L3 · Slide 21
Hash function	A function that turns a key into a large number; with mod R it picks a reduce task.	L2 · Slides 66–67
HDFS	Hadoop Distributed File System, the open-source counterpart of GFS.	L2 · Slides 17, 76
Hierarchical clustering	A family of methods that build a tree of nested clusters by merging (or splitting) clusters step by step.	L5 · Slides 14, 16
High-dimensional data	Data where each item has very many attributes.	L1 · Slides 18, 20
Hive and Pig	Systems giving SQL-like abstractions on top of the Hadoop Map-Reduce layer.	L2 · Slide 76
Host (hostname)	The machine-name part of a URL (for example www.youtube.com), used as key in the host-size example.	L2 · Slide 45
Hotel example	The slide example: 10^9 people, 1,000 days, 1% in hotels, 100 per hotel, giving about 250,000 chance suspicious pairs.	L1 · Slides 13–14
Idle / in-progress / completed	The three states of a task tracked by the master.	L2 · Slide 56
Inference of models	The machine-learning view of data science: fitting models to data.	L1 · Slide 16
Infinite data	A never-ending data stream that cannot be stored in full.	L1 · Slides 18, 20
Input split	One chunk of the input file read by a single map task.	L3 · Slide 4
Interdependent data	Records that depend on each other, e.g. pairwise comparisons and machine learning.	L3 · Slide 23
Interest	conf(I → j) minus Pr[j]; high values mark rules where I really raises the chance of j.	L4 · Slide 11
Intermediate pairs	The key-value pairs emitted by Map, not yet the final answer.	L2 · Slides 30–31
Item	One product (or thing) in the catalogue from which baskets are drawn; on Slide 20 items are positive integers.	L4 · Slides 3, 20
Itemset	Any set of items; a basket is one particular itemset.	L4 · Slide 8
Jaccard distance	One minus $\|A \cap B\| / \|A \cup B\|$ for two sets A and B.	L5 · Slide 4
k	The number of clusters wanted, chosen in advance by the user in k-means (and in BFR).	L5 · Slides 23, 31
k-means	A point-assignment algorithm for Euclidean data that repeatedly assigns points to the nearest of k centroids and moves each centroid to the mean of its points.	L5 · Slides 22–24
Key-value pair	A (key, value) item; everything flowing through Map-Reduce is a pair, and the key decides where it is grouped.	L2 · Slides 27, 29
L_k	The set of truly frequent k-itemsets.	L4 · Slide 34
Labeled data	Data whose examples come with known answers, as used in supervised machine learning.	L1 · Slide 18
Language model	Counts of word sequences (here 5-word sequences) in a corpus, computed like word count.	L2 · Slides 48–49
Lazy evaluation	Nothing is computed until an action requires it.	L3 · Slide 15
Lineage	The recorded chain of transformations that produced an RDD, used for recovery and planning.	L3 · Slides 11, 16
Lineage graph	DAG of RDDs and the transformations and actions between them.	L3 · Slide 16

Link analysis	Using the link structure of the web or a graph to rank or analyse nodes.	L1 · Slides 19–20
Local file system (intermediate data)	Where a map worker keeps its output; it is not replicated and is lost if the worker dies.	L2 · Slide 55
Local write	A map worker writing its intermediate pairs to its own local disk.	L3 · Slide 4
Locality Sensitive Hashing (LSH)	A hashing technique that puts similar items in the same bucket to speed up similarity search.	L1 · Slides 19–20
LSH and dimensionality reduction	Techniques for high-dimensional data in the same course column as clustering (locality-sensitive hashing; reducing the number of dimensions).	L5 · Slide 2
M and R	The numbers of map tasks and reduce tasks; M should be much larger than the number of nodes and R usually smaller than M.	L2 · Slide 69
Machine learning (ML)	The field of learning complex models from data.	L1 · Slides 16–17
Mahalanobis distance	BFR's normalised Euclidean distance from a centroid: the square root of the sum over dimensions of $((x_i - c_i)/\sigma_i)^2$.	L5 · Slides 54–56
Mahalanobis threshold	The cut-off below which a point counts as close enough to a cluster; common practice is M.D. < 2.	L5 · Slide 56
Main-memory bottleneck	Counters for the itemsets being counted must fit in RAM at once; memory limits what can be counted per pass.	L4 · Slide 22
Main-memory clustering algorithm	Any ordinary in-memory algorithm (such as k-means or hierarchical) that BFR uses on the leftover points and the old RS.	L5 · Slide 50
Map function	User function that takes one input pair (k, v) and emits zero or more intermediate pairs (k', v').	L2 · Slides 29–31, 43
Map Phase 1 (SON)	Each mapper mines its chunks with an in-memory algorithm and emits the locally frequent itemsets.	L4 · Slide 59
Map Phase 2 (SON)	The candidate list is broadcast to all mappers, each counting every candidate on its own chunks.	L4 · Slide 65
Map task / Reduce task	A unit of work run by a worker; a map task makes many Map calls and a reduce task many Reduce calls.	L2 · Slides 51–52, 69
Map-Reduce	Google's computational model and programming model for large-scale data: Map, group by key, Reduce, executed on a cluster with a DFS.	L2 · Slides 3, 16, 25
Mapper	In k-means MapReduce, the task that finds each point's nearest centroid and emits (centroidID, point).	L5 · Slide 61
MapReduce	A programming model for processing huge data on a cluster using map and reduce phases.	L1 · Slides 19–20
Market-basket model	A large set of items and a large set of baskets, each a small subset of the items; the input of frequent-itemset mining.	L4 · Slides 3–4
Master	The process that assigns map and reduce tasks to workers and tracks them.	L3 · Slide 4
Master (job coordinator)	The Map-Reduce node that tracks task states, receives intermediate file locations, informs reducers and pings workers.	L2 · Slide 56
Master node (Name Node)	The DFS component that stores metadata about where files and chunks are located; may be replicated.	L2 · Slide 24
Memory load	The chunk of points read from disk into main memory at one time in BFR.	L5 · Slide 45
Metadata	Data about data: which chunks form a file and which servers hold each copy.	L2 · Slide 24
MLlib	Spark's user-friendly, scalable machine-learning library.	L3 · Slides 8, 20
MMD	Mining of Massive Datasets by Leskovec, Rajaraman and Ullman, the course textbook.	L1 · Slides 1, 24, 29
Monotonicity (A-Priori)	If an itemset is frequent, so is every subset of it; equivalently an itemset with an infrequent subset cannot be frequent.	L4 · Slide 30
Multihash	PCY refinement that uses several independent hash tables in the same first pass, so only two passes are needed.	L4 · Slides 49–50

Multistage	PCY refinement with three passes: a second, different hash function rehashes the pairs and gives Bitmap 2.	L4 · Slides 46–47
N	The number of points in a cluster summary.	L5 · Slide 48
Naive algorithm	Reads the file once and keeps one counter per possible pair, needing about n(n−1)/2 counters.	L4 · Slide 24
Natural join	R(A,B) ⋈ S(B,C): all tuples (a,b,c) such that (a,b) is in R and (b,c) is in S.	L2 · Slides 71–72
Network bottleneck	The limit imposed by slow network transfer: moving 10 TB over 1 Gbps takes about a day.	L2 · Slide 14
Node	One computer in a cluster: a CPU, memory and a disk.	L2 · Slides 4, 9
Node failure	The routine breakdown of cheap machines: about 1 a day per 1,000 servers (each lasts about 1,000 days), 1,000 a day at 1M machines.	L2 · Slide 12
Non-Euclidean space	A space (sets, strings) where points have no coordinates to average, so only the points themselves can serve as cluster locations.	L5 · Slide 18
Normal (Gaussian) distribution	The bell-shaped distribution BFR assumes for each cluster around its centroid; most points lie within 2–3 standard deviations.	L5 · Slide 44
O(N³) cost	The cost of naive agglomerative clustering: about N merges, each comparing about N^2 pairs of clusters; too expensive for big data.	L5 · Slide 21
Ontology	A formal description of concepts and the relations among them.	L1 · Slide 15
Outlier	A point that is far from every cluster and belongs to none of them.	L5 · Slide 6
PageRank	A graph algorithm that scores page importance from link structure.	L1 · Slide 20
Pairs of people	The number of unordered pairs C(n, 2) = n(n−1)/2 ≈ $n^2/2$; about 5×10^{17} for 10^9 people.	L1 · Slides 13–14
Partition	One piece of an RDD, processed on one machine independently of the others.	L3 · Slides 13–14
Partition function	Rule mapping an intermediate key to one of R reduce tasks, by default hash(key) mod R.	L2 · Slides 66–67
Partitioning-aware	Scheduler behaviour that avoids shuffles when data is already partitioned as needed.	L3 · Slide 17
Pass	One sequential read of the whole basket file; the cost measure of frequent-itemset algorithms.	L4 · Slide 21
PCY (Park-Chen-Yu)	A-Priori improvement that hashes pairs into bucket counters during Pass 1 to prune candidate pairs in Pass 2.	L4 · Slides 37–45
PCY Pass 2 condition	Count {i,j} only if both i and j are frequent items and {i,j} hashes to a bucket whose bit is 1.	L4 · Slide 43
Persistence	Data is not lost when a machine or disk fails.	L2 · Slides 13, 16
Pipelining	Overlapping stages so shuffle reads of finished map outputs run while other maps are still computing.	L2 · Slide 58
Point-assignment clustering	A family of methods that maintain a set of clusters (their number fixed in advance) and assign each point to the nearest one, for example k-means.	L5 · Slides 14, 86
Pooled standard deviation	A single σ per dimension for two CS sets, taken as the root of the average of the two variances or as the larger of the two σ.	L5 · Slide 58
Pr[j]	The fraction of all baskets that contain item j, the baseline used by interest.	L4 · Slides 11, 15
Predictive methods	Methods that use some variables to predict unknown or future values of other variables.	L1 · Slide 11
Prerequisites	EECS 3421 (database management systems) and EECS 3101 (design and analysis of algorithms).	L1 · Slide 28
Programming model	The style of program you must write for a system; for Map-Reduce, you supply Map and Reduce functions.	L2 · Slides 15, 25
Rack	A frame holding 16–64 nodes that all plug into one switch.	L2 · Slide 9

Random (irregular) access	Jumping around the data; MapReduce is inefficient for it.	L3 · Slide 23
Random sampling	Mining a random in-memory sample of the baskets with a threshold scaled to the sample size; approximate.	L4 · Slides 54–55
Rank	The slide's word for a layer of parallel tasks (Map is rank 1, Reduce rank 2).	L3 · Slide 7
RDD (Resilient Distributed Dataset)	Partitioned, read-only collection of records spread across the cluster and cached in memory.	L3 · Slides 11, 13–14
Read-only (immutable)	An RDD is never changed; operations create new RDDs.	L3 · Slide 13
Recommender system	A system that predicts what items a user will like; the slide's example of a predictive method and an application.	L1 · Slides 11, 19–20
Record	A single item (object) in an RDD.	L3 · Slide 13
Reduce function	User function that takes one key k' with the list of all its values and emits the aggregated pair (k', v'').	L2 · Slides 34, 44
Reduce Phase 1 (SON)	Collects the candidates from all mappers into one global list, removing duplicates.	L4 · Slide 60
Reduce Phase 2 (SON)	Adds up the per-chunk counts of each candidate and keeps those whose total meets the support threshold.	L4 · Slide 66
Reducer	In k-means MapReduce, the task that averages all points of one centroid to give the new centroid.	L5 · Slide 62
Redundancy	Storing extra copies of data so that failures do not lose it.	L2 · Slides 16–17
Remote read / shuffle	Reduce workers pulling their pieces from every map worker's local disk.	L3 · Slide 4
Renumbering	Giving the frequent items new ids 1..m (with a table back to the old ones) so the triangular matrix can be used.	L4 · Slide 33
Replication factor	The number of copies of each chunk (usually 2× or 3×), kept on different servers and, if possible, different racks.	L2 · Slide 20
Representative points	The points chosen as dispersed as possible in a cluster and moved (say) 20% toward its centroid; they stand for the cluster in CURE.	L5 · Slides 36, 38
Retained set (RS)	In BFR, isolated points kept in memory as they are, waiting to join a compression set.	L5 · Slide 46
Rule generation (Step 2)	For each frequent itemset I and each non-empty proper subset A, test the rule A → I \ A by its confidence.	L4 · Slide 17
Scalability	The ability of a method to keep working as the data grows by a large factor.	L1 · Slides 15, 17
Scale out	Add many commodity machines working together as a cluster.	L3 · Slide 3
Scale up	Make one machine bigger (more CPU, RAM, disk).	L3 · Slide 3
Scaled threshold	Sample (or chunk) threshold p × s, where p is the fraction of the baskets it contains.	L4 · Slides 54, 57
Seed (initial centroid)	The starting point chosen for each of the k clusters before the k-means loop begins.	L5 · Slide 23
Sequential data access	Scanning data from start to end, the pattern MapReduce is great for.	L3 · Slide 23
Sequential read	Reading a file from start to end, which disks do fast; Map-Reduce needs only such reads.	L2 · Slide 40
Shuffle	Moving intermediate pairs across the network from map workers to the reduce workers that own their keys.	L2 · Slides 51, 54
SimRank	A graph measure of the similarity of nodes.	L1 · Slide 20
Single machine in-memory	A model of computation where all data fits in one computer's memory.	L1 · Slide 18
SON (Savasere-Omiecinski-Navathe)	Exact two-pass algorithm: mine each chunk, take the union of local frequent itemsets as candidates, then count candidates over all data.	L4 · Slides 56–57
SON monotonicity idea	An itemset cannot be frequent in the whole data unless it is frequent in at least one chunk.	L4 · Slide 57

Sort and group	The reduce-side step that orders arrived pairs by key so each key's values are together before Reduce is called.	L2 · Slides 51–52
Spam detection	Recognising spam emails or link/web spam.	L1 · Slides 19–20
Spark	A cluster computing framework that keeps data in memory between steps.	L1 · Slide 19
Spark core	The fast, memory-optimised execution engine underneath all Spark libraries.	L3 · Slide 8
Spark SQL / SparkSQL	Spark library providing a SQL API for scalable processing of relational (structured) data.	L3 · Slides 8, 12, 20
Spark Streaming	Spark library for stream processing of live data streams.	L3 · Slides 8, 20
Stage	A group of pipelined steps; a new stage begins at every shuffle.	L3 · Slide 17
Standard deviation (σ)	The typical spread of a cluster's points around its centroid in one dimension; the square root of the variance.	L5 · Slides 44, 49
Statistical artifact	A pattern that arises from chance or from the way data was searched, not from a real effect.	L1 · Slides 12–14
Statistics	The field of drawing conclusions from data; shown in place of CS theory on slide 17.	L1 · Slide 17
Store, manage, analyze	The three stages of working with data; this class focuses on analysis.	L1 · Slide 8
Straggler	A slow but working machine that delays the end of a map or reduce phase.	L2 · Slide 68
Streaming	The challenge that data arrives continuously and must be processed on the fly.	L1 · Slide 15
Streams and online algorithms	A model where data arrives over time and each item is seen once or few times with limited memory.	L1 · Slide 18
Striping	Spreading a file's chunks across multiple discs so they can be read in parallel.	L2 · Slide 8
Structured data	Data organised in tables with fixed columns.	L1 · Slide 15
SUM	A vector whose i-th component is the sum of the coordinates of the cluster's points in dimension i.	L5 · Slide 48
SUMSQ	A vector whose i-th component is the sum of the squares of the coordinates in dimension i.	L5 · Slide 48
Support	The number of baskets that contain all items of an itemset.	L4 · Slide 8
Support threshold (s)	The minimum number of baskets an itemset must appear in to be called frequent.	L4 · Slide 8
Suspicious pair	In the hotel example, two people who were in the same hotel on the same day at least twice.	L1 · Slides 13–14
Swapping (thrashing)	Moving memory pages to and from disk when counters do not fit in RAM; random increments make it thousands of times slower.	L4 · Slide 22
Switch	A network device that forwards data between the machines connected to it.	L2 · Slides 9–10
Tag (R or S)	A label on each value in the join so that a reducer knows from which relation it came.	L2 · Slide 72
Task scheduler	Spark component that turns the RDD graph into stages and tasks and runs them.	L3 · Slide 17
TID	Transaction ID: the basket number in the input table.	L4 · Slide 3
Too many hash functions (multihash)	With more tables each has fewer buckets and a higher average count, so most or all buckets become frequent and nothing is filtered.	L4 · Slide 51
Total cost vs elapsed cost	Total cost is the sum of work over all machines (the cloud bill); elapsed cost is wall-clock time using parallelism.	L2 · Slide 74
Transformation	Lazy, deterministic operation that builds a new RDD from other RDDs (map, filter, join …).	L3 · Slides 11, 15

Triangular matrix	A flat array holding one 4-byte counter for each pair {i,j} with i < j; requires item ids 1..n.	L4 · Slides 25–26
Triples method	Storing a record [i, j, count] of about 12 bytes for each pair that actually occurs.	L4 · Slides 25–27
Two-pass algorithm	An algorithm that reads the data twice: CURE's first pass clusters a sample, the second assigns every point.	L5 · Slides 38, 42
Type-safe	Type errors are detected by the compiler before the job runs.	L3 · Slide 18
Understandable	A discovered pattern that humans can interpret.	L1 · Slide 10
Unexpected	A discovered pattern that is non-obvious to the system.	L1 · Slide 10
Useful	A discovered pattern that it is possible to act on.	L1 · Slide 10
Valid	A discovered pattern that holds on new data with some certainty.	L1 · Slide 10
Variance	The mean squared distance from the mean; BFR computes it as SUMSQ/N − (SUM/N)² in each dimension.	L5 · Slide 49
Vertex and edge properties	User-defined attributes attached to every vertex and every edge in GraphX.	L3 · Slide 20
Worker	A process that runs map or reduce tasks assigned by the master.	L3 · Slide 4